Principia Softwarica: The Plan 9 Kernel 9

Raspberry Pi (ARM 32 bits) edition

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Ken Thompson, Dave Presotto, and Richard Miller

January 14, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **16**
1.1 Motivations . . . 16
1.2 The Plan 9 kernel: `9` . . . 16
1.3 Other kernels . . . 16
1.4 Getting started . . . 17
1.5 Requirements . . . 17
1.6 About this document . . . 17
1.7 Copyright . . . 17
1.8 Acknowledgments . . . 17

2 Overview **18**
2.1 Operating system principles . . . 18
2.2 `9`'s syscall interface . . . 18
2.3 `helloraw.c` and `helloraw.s` . . . 20
2.4 `minishell.c` . . . 20
2.5 The Raspberry Pi machine (ARM) . . . 20
2.5.1 ARM CPU . . . 20
2.5.2 ARM coprocessors . . . 21
2.5.3 ARM MMU . . . 22
2.5.4 Input/Output devices . . . 22
2.6 C and assembly . . . 23
2.7 Code organization . . . 23
2.7.1 Portable versus arch-specific code . . . 23
2.7.2 Main headers . . . 23
2.7.3 Top directories . . . 24
2.8 Software architecture . . . 24
2.8.1 Booting overview (ARM) . . . 24
2.8.2 Memory overview (ARM) . . . 24
2.8.3 Syscall overview (ARM) . . . 26
2.9 Book structure . . . 26

3 Core Data Structures **27**
3.1 `Conf` and `conf` . . . 27
3.2 `Cpu` and `cpu` . . . 28
3.3 `Proc` and `up` . . . 29
3.4 `Chan` . . . 31
3.5 `Dev` and `devtab` . . . 32
3.6 `Soc` and `soc` (ARM) . . . 33

4 Concurrency **34**
4.1 Critical regions . . . 34
4.2 Interrupts disabling: `arch_splhi()`, `arch_spllo()` (ARM) . . . 34
4.3 Test and set: `arch_tas()` (ARM) . . . 35
4.4 Atomic count: `arch_xinc()` and `arch_xdec()` (ARM) . . . 35
4.5 Spin `Lock` . . . 35
4.6 Uninterruptable `ILock` . . . 37
4.7 Waiting `QLock` . . . 39
4.8 Readers/Writer `RWLock` . . . 41
4.9 Atomic `References` and `Counter` . . . 44
4.10 Safeguards . . . 45
4.10.1 `Lock.p` . . . 45
4.10.2 `Proc.nlocks` . . . 45
4.10.3 `Cpu.ilockdepth` . . . 46
4.11 Synchronization . . . 46
4.11.1 Rendez-vous . . . 46
4.11.2 Producer/Consumer Queue . . . 46
4.11.3 Semaphore . . . 46

5 Memory **47**
5.1 Overview . . . 47
5.2 Long-lived structure allocator: `xalloc()` . . . 47
5.2.1 `Xalloc` . . . 47
5.2.2 `xlists` . . . 48
5.2.3 `Xhdr` . . . 48
5.2.4 `xalloc()` . . . 48
5.2.5 `xhole()` . . . 49
5.2.6 `xmerge()` . . . 50
5.3 Memory pools . . . 51
5.3.1 `Pool` . . . 51
5.3.2 `plock()` . . . 52
5.3.3 `pmainmem()` . . . 53
5.3.4 `poolprint()` . . . 53
5.4 Kernel `malloc()` . . . 54
5.4.1 `malloc()` . . . 54
5.4.2 Sleeping `smalloc()` . . . 55
5.4.3 `mallocxxx()` . . . 55
5.4.4 malloc tags . . . 56
5.5 User `malloc()` . . . 57
5.6 `Page` and `palloc` . . . 57
5.6.1 `Page` . . . 57
5.6.2 `Palloc` . . . 58
5.6.3 `newpage()` . . . 58
5.7 `Segment` and `Pagetable` . . . 61
5.7.1 `Segment` . . . 61
5.7.2 `Pagetable` . . . 62
5.7.3 `newseg()` . . . 63
5.7.4 `smallpagedir` . . . 64
5.7.5 Adding a page to a segment: `segpage()` . . . 65
5.7.6 Process segments . . . 65

5.7.7 /proc/x/segment . . . 69
5.8 sysbrk() . . . 69
5.8.1 Extending a segment . . . 70
5.8.2 Shrinking a segment . . . 71

6 Processes **73**
6.1 Overview . . . 73
6.2 Process allocation . . . 73
6.2.1 Process ID: Proc.pid . . . 73
6.2.2 Procalloc . . . 73
6.2.3 PID hashing . . . 74
6.2.4 newproc() . . . 75
6.2.5 No more processes . . . 75
6.3 The user . . . 77
6.4 sysrfork() . . . 78
6.4.1 Fork: RFPROC . . . 78
6.4.2 Memory sharing: RFMEM . . . 79
6.4.3 arch_forkchild() (ARM) . . . 80
6.4.4 File descriptors: RFFDG, RFCFDG . . . 80
6.4.5 Namespace: RFNAMEG, RFCNAMEG . . . 81
6.4.6 Environment variables: RFENVG, RFCENVG . . . 81
6.4.7 Rendezvous group: RFREND . . . 81
6.4.8 Note group: RFNOTEG . . . 81
6.4.9 Process sharing settings (no RFPROC) . . . 82
6.5 sysexec() . . . 83
6.5.1 Text segment . . . 86
6.5.2 Data segment . . . 86
6.5.3 Bss segment . . . 86
6.5.4 Stack segment . . . 86
6.5.5 arch_execregs() (ARM) . . . 89
6.6 sysawait() . . . 89
6.6.1 Parent and children . . . 89
6.6.2 Waitmsg and Waitq . . . 90
6.6.3 sysawait() . . . 90
6.6.4 Rendez-vous: Proc.waitr . . . 91
6.7 sysexits() . . . 92
6.7.1 pexit() . . . 92
6.7.2 pexit() and schedinit() . . . 93
6.7.3 Wait message . . . 94
6.7.4 Freeing resources . . . 95
6.7.5 No waiting for child: RFNOWAIT . . . 96
6.8 Kernel processes . . . 96
6.8.1 arch_kprocchild() (ARM) . . . 97
6.9 Process control /proc/#/ctl . . . 97

7 Interrupts **99**
7.1 Overview . . . 99
7.2 Exceptions (ARM) . . . 99
7.3 Exception vectors (ARM) . . . 99
7.4 System calls . . . 100

7.4.1 _vsvc() (ARM) . . . 100
7.4.2 Sargs . . . 100
7.4.3 arch__syscall() (ARM) . . . 101
7.5 Exception handlers . . . 103
7.5.1 _vxxx() (ARM) . . . 103
7.5.2 _vswitch() (ARM) . . . 104
7.5.3 arch__trap() (ARM) . . . 105
7.5.4 Undefined instruction fault (ARM) . . . 107
7.5.5 Prefetch fault (ARM) . . . 107
7.5.6 Data fault (ARM) . . . 108
7.6 IRQs (ARM) . . . 110
7.6.1 Vctl (ARM) . . . 110
7.6.2 Intregs (ARM) . . . 111
7.6.3 arch_intrenable() (ARM) . . . 111
7.6.4 Basic IRQs (ARM) . . . 112
7.6.5 irq() (ARM) . . . 112

8 Virtual Memory **113**
8.1 Overview . . . 113
8.2 The ARM MMU (ARM) . . . 113
8.3 Pagination and mmupd/mmupt (ARM) . . . 114
8.4 Page fault . . . 116
8.4.1 faultarm() (ARM) . . . 116
8.4.2 fault() . . . 117
8.4.3 fixfault() . . . 117
8.4.4 arch_putmmu() (ARM) . . . 119
8.5 Copy on write . . . 121
8.6 Page cache . . . 121
8.7 Demand loading . . . 124
8.7.1 Kimage and the image cache . . . 124
8.7.2 Segment and KImage . . . 125
8.7.3 Paging in . . . 127
8.7.4 duppage() . . . 129
8.7.5 Collecting images . . . 131
8.8 Swapping . . . 133
8.8.1 Swapalloc . . . 133
8.8.2 /dev/swap . . . 134
8.8.3 PageOrSwap . . . 135
8.8.4 Swapping plan . . . 136
8.8.5 The pager . . . 138
8.8.6 Paging out . . . 142
8.8.7 Paging in . . . 143
8.8.8 Setting up the swap file . . . 143

9 Scheduling **145**
9.1 Overview . . . 145
9.2 Process priority . . . 145
9.3 The runqueue . . . 146
9.4 The scheduler . . . 148
9.4.1 sched() . . . 148

9.4.2 schedinit() . . . 149
9.4.3 runproc() . . . 150
9.5 Context switch . . . 151
9.5.1 Label . . . 151
9.5.2 Kernel stacks . . . 151
9.5.3 arch_gotolabel() (ARM) . . . 151
9.5.4 sched() and schedinit() dance . . . 152
9.5.5 Delayed sched() . . . 152
9.5.6 arch_mmuswitch() (ARM) . . . 153
9.5.7 arch_procsave() (ARM) . . . 154
9.6 Cooperative scheduling . . . 155
9.6.1 yield() . . . 155
9.6.2 ready() . . . 155
9.6.3 Rendez . . . 156
9.6.4 sleep() . . . 156
9.6.5 wakeup() . . . 158
9.7 Preemptive scheduling . . . 158
9.7.1 hzsched() . . . 158
9.7.2 hzclock() . . . 159
9.8 Customized scheduling . . . 160
9.9 Real-time scheduling . . . 161
9.10 Adjusting priority . . . 161
9.10.1 reprioritize() . . . 161
9.10.2 rebalance() . . . 163
9.11 Halting (ARM) . . . 164

10 Time **165**
10.1 Overview (arm) . . . 165
10.2 Time units . . . 165
10.2.1 Seconds . . . 165
10.2.2 Ticks . . . 165
10.2.3 Fastticks (ARM) . . . 166
10.3 Clock interrupt (ARM) . . . 167
10.4 Timers . . . 167
10.4.1 Timer . . . 167
10.4.2 Timers and CPU timers . . . 168
10.4.3 tadd()/tdel() . . . 168
10.4.4 timeradd()//timerdel() . . . 170
10.4.5 arch_timerset() (arm) . . . 170
10.4.6 timerintr() . . . 171
10.4.7 Clock callbacks: addclock0link() . . . 172
10.4.8 Delay (ARM) . . . 173
10.5 syssleep() . . . 173
10.6 Alarms . . . 175
10.7 sysalarm() . . . 175
10.7.1 alarmkproc() . . . 177
10.7.2 checkalarms() . . . 177
10.8 Time of day . . . 178
10.9 sysnsec() . . . 182
10.10/dev/time . . . 182

10.11 /dev/bintime . . . 183
10.12 Watchdog . . . 185

11 Initialization **186**
11.1 Booting the kernel . . . 186
11.1.1 Loading the kernel in physical memory (ARM) . . . 186
11.1.2 First instructions (ARM) . . . 186
11.1.3 Basic pagination (ARM) . . . 186
11.1.4 Jumping to C main() (ARM) . . . 188
11.2 Kernel initialization (ARM) . . . 189
11.2.1 cpu initialization . . . 190
11.2.2 Memory detection (ARM) . . . 191
11.2.3 conf initialization (ARM) . . . 191
11.2.4 Exception vector initialization (ARM) . . . 193
11.2.5 Clock initialization (ARM) . . . 195
11.2.6 Timer initialization . . . 195
11.2.7 Virtual memory initialisation (ARM) . . . 196
11.2.8 Memory initialisation . . . 196
11.2.9 Device initialisation . . . 199
11.2.10 Displaying hardware information (ARM) . . . 200
11.3 The first process (ARM) . . . 201
11.4 Booting the user . . . 205

12 Files **206**
12.1 Overview . . . 206
12.2 Channels . . . 206
12.3 Devices table devtab . . . 208
12.4 File descriptors, Fgrp . . . 209
12.5 sysopen(), sysclose() . . . 212
12.6 namec() . . . 214
12.6.1 Checking valid filename string . . . 217
12.6.2 Parsing filename . . . 219
12.6.3 Error management . . . 221
12.7 syspread(), syspwrite() . . . 222
12.8 sysseek() . . . 224
12.9 Collecting unclosed files . . . 226
12.10 Special opening modes . . . 228
12.10.1 Close on exec . . . 228
12.10.2 Remove on close . . . 228

13 Directories **230**
13.1 Overview . . . 230
13.2 Path . . . 231
13.3 233
13.4 walk() . . . 233
13.5 syscreate() . . . 238
13.6 sysremove() . . . 240
13.7 syschdir() . . . 241
13.8 sysfd2path() (a.k.a pwd()) . . . 241
13.9 DirEntry . . . 242

13.10 sysstat() . . . 243

14 Namespaces **247**
14.1 Overview . . . 247
14.2 Mount point . . . 247
14.3 Mount table . . . 249
14.4 Device path . . . 250
14.5 sysbind() . . . 251
14.6 /proc/#/ns . . . 254
14.7 findmount(), domount() . . . 256
14.8 Path mount points and 258
14.9 Union mount . . . 259
14.9.1 Union walk . . . 259
14.9.2 ->umh . . . 259
14.9.3 syspread() for directories . . . 260
14.9.4 MCREATE . . . 262
14.9.5 Union of union . . . 262
14.10 Sandboxing . . . 263
14.11 sysmount() . . . 264
14.12 sysumount() . . . 265
14.13 XTODO . . . 267
14.14 XTODO . . . 268

15 Devices **273**
15.1 The keyboard . . . 273
15.1.1 Overview . . . 273
15.1.2 Scan code . . . 273
15.1.3 Esc . . . 277
15.1.4 kbd . . . 278
15.1.5 echo() . . . 279
15.1.6 PC Keyboard (x86) . . . 280
15.1.7 Raspberry Pi USB Keyboard (ARM) . . . 280
15.1.8 Changing keyboard: /dev/kbmap . . . 280
15.1.9 External keyboard input: /dev/kbin . . . 281
15.2 The screen . . . 281
15.2.1 Overview . . . 281
15.2.2 Text output on a PC (x86) . . . 281
15.2.3 Text output on a Raspberry Pi (ARM) . . . 281
15.3 The console device /dev/cons . . . 283
15.3.1 Overview . . . 283
15.3.2 Buffered input . . . 283
15.3.3 Output . . . 285
15.3.4 print(), iprint(), pprint() . . . 286
15.3.5 /dev/cons . . . 288
15.3.6 /dev/consctl and raw input . . . 290
15.4 The mouse . . . 291
15.4.1 Mouse and keyboard . . . 291
15.4.2 PS2 mouse (x86) . . . 292
15.4.3 USB mouse (ARM) . . . 292
15.5 Other devices . . . 292

16 Queue **293**

17 Filesystems **313**
17.1 The root filesystem . . . 313
17.2 The /boot/ filesystem . . . 316
17.3 Helpers . . . 317
17.4 SimpleFS . . . 323
17.5 Block . . . 323
17.6 Fileservers . . . 333
17.7 Fcall . . . 333
17.8 XTODO . . . 333

18 IPC **334**
18.1 Notes (signals) . . . 334
18.1.1 sysnotify() (arm) . . . 336
18.1.2 sysnoted() (arm) . . . 338
18.1.3 postnote() . . . 339
18.2 Pipes . . . 341
18.2.1 syspipe() . . . 341
18.2.2 /dev/pipe . . . 342
18.3 Shared segment and semaphore . . . 349
18.3.1 Segment . . . 349
18.3.2 Semaphore . . . 355
18.4 sysrendezvous() . . . 360
18.5 Network . . . 361

19 Debugging Support **362**
19.1 Core dumps and broken processes . . . 362
19.2 Stopping a process: /proc/x/ctl . . . 364
19.3 Inspecting process memory: /proc/x/mem . . . 366
19.4 /bin/strace . . . 369
19.5 Scheduler events: /proc/trace and /bin/trace . . . 376
19.6 /bin/db . . . 379
19.7 Other . . . 379

20 Profiling Support **380**
20.1 /dev/cputime and /proc/#/status . . . 380
20.2 /bin/time . . . 381
20.3 /bin/tprof . . . 381
20.4 /bin/prof . . . 383

21 Userspace System Programs **385**
21.1 /boot/boot . . . 385
21.2 /root/init . . . 385
21.3 The shell . . . 385
21.3.1 #! . . . 385

22 Advanced Topics **387**
22.1 Floating point . . . 387
22.1.1 `FPSaveStatus` . . . 387
22.1.2 `Arch_FPsave` (ARM) . . . 387
22.1.3 Floating point hooks . . . 388
22.1.4 ARM floating point coprocessors (ARM) . . . 391
22.1.5 `coproc.c` (ARM) . . . 391
22.1.6 `vfp.c` (ARM) . . . 393
22.1.7 `mathxxx()` (ARM) . . . 399
22.1.8 `/proc/x/fpregs` . . . 400
22.2 CPU cache . . . 400
22.2.1 Instruction and data cache (ARM) . . . 400
22.2.2 Cache policy . . . 401
22.2.3 Paging and caching (ARM) . . . 401
22.2.4 Memory barriers (ARM) . . . 401
22.2.5 Cache operations (ARM) . . . 401
22.2.6 `cachexxx()` (ARM) . . . 402
22.3 Memory model . . . 404
22.3.1 `arch_coherence()` . . . 404
22.3.2 Memory barriers (ARM) . . . 405
22.4 The Raspberry Pi 2 machine . . . 405
22.4.1 CPU ID . . . 405
22.4.2 MMU . . . 406
22.4.3 TTB . . . 406
22.4.4 Local registers . . . 406
22.4.5 Timer . . . 407
22.4.6 Performance monitor . . . 407
22.5 SMP (ARM) . . . 407
22.5.1 Initialization adjustments . . . 407
22.5.2 Setting `cpu` . . . 415
22.5.3 Concurrency adjustments . . . 415
22.5.4 Timer adjustments . . . 416
22.5.5 `arch_coherence()` (ARM) . . . 417
22.5.6 Cache adjustments . . . 417
22.5.7 IRQ adjustments . . . 422
22.5.8 Cycle counter adjustments . . . 422
22.6 Advanced virtual memory . . . 423
22.6.1 Page coloring . . . 423
22.6.2 Extra segment types . . . 424
22.6.3 `arch_kmap()` . . . 426
22.6.4 Extra checks . . . 426
22.7 Fast interrupts (FIQ) (ARM) . . . 428
22.7.1 Exception `PsrMfiq` and associated processor mode . . . 428
22.7.2 `IRQfiq` and `vfiq` . . . 429
22.7.3 `fiq()` . . . 430
22.8 Power management (ARM) . . . 430
22.9 Alignements (ARM) . . . 431
22.10 Real-time scheduling . . . 431
22.11 Fileservers . . . 446

22.11.1 Mnt . . . 446
22.11.2 MntRpc . . . 448
22.11.3 Fcall . . . 449
22.11.4 mntversion() . . . 449
22.11.5 mountrpc() . . . 451
22.11.6 devmnt . . . 453
22.12 Network . . . 456
22.13 Security . . . 456
22.14 Graphics . . . 458

23 Special Fileservers **459**
23.1 Cmdtab . . . 459
23.2 /env/ . . . 461
23.2.1 Egrp . . . 461
23.2.2 /env/ . . . 462
23.3 /proc/ . . . 463
23.3.1 /proc/x/ctl . . . 463
23.3.2 /proc/x/mem . . . 463
23.3.3 /proc/x/status . . . 463
23.3.4 /proc/x/args . . . 463
23.3.5 /proc/x/wait . . . 463
23.3.6 Other /proc/x/xxx . . . 464
23.3.7 XXX . . . 464
23.4 /fd/ . . . 483
23.4.1 sysdup() . . . 483
23.4.2 /fd/ . . . 484
23.5 /sys/ . . . 484
23.5.1 /sys/osversion . . . 486
23.5.2 /sys/config . . . 486
23.5.3 /ys/hosthowner, /sys/hostdomain . . . 486
23.5.4 /sys/sysname . . . 486
23.5.5 /sys/drivers . . . 487
23.5.6 /sys/reboot . . . 487
23.6 /dev/ . . . 494
23.6.1 /dev/random . . . 494

24 Conclusion **498**

A Debugging **499**
A.1 sysnop() . . . 499
A.2 Special keys and dumpers . . . 499
A.3 /bin/ktrace (ARM) . . . 505
A.4 /dev/kmesg . . . 507
A.5 /dev/kprint . . . 508
A.6 Remote debugger . . . 509
A.7 Serial line . . . 510
A.7.1 serialoq . . . 510
A.7.2 uartconsinit() . . . 511
A.7.3 uartputs() . . . 511
A.7.4 consuart . . . 511

A.7.5 Uart, PhysUart . . . 511

B Profiling **512**
B.1 /sys/sysstat (ARM) . . . 512
B.2 /dev/kprof . . . 515
B.3 Statistics . . . 515
B.3.1 Lock.lockcycles . . . 515
B.3.2 TaslockStats . . . 516
B.3.3 QlockStats . . . 517
B.3.4 Kimage statistics . . . 517
B.3.5 Scheduling . . . 517
B.3.6 Clock . . . 517

C Error Management **518**
C.1 panic() . . . 518
C.2 error() . . . 519
C.3 waserror(), nexterror(), poperror() . . . 521
C.4 syserrstr() . . . 521

D Utilities **523**
D.1 Memory areas operations . . . 523
D.2 String functions . . . 523
D.3 Conversion functions . . . 525
D.4 Pool allocation . . . 525
D.5 Printf and Fmt . . . 525
D.6 Runes, a.k.a unicode . . . 526

E Extra Code **528**
E.1 arch/ . . . 528
E.2 buses/ . . . 528
E.3 concurrency/ . . . 528
E.3.1 concurrency/portdat_concurrency.h . . . 528
E.3.2 concurrency/portfns_concurrency.h . . . 529
E.3.3 concurrency/taslock.c . . . 529
E.3.4 concurrency/qlock.c . . . 530
E.3.5 concurrency/ref.c . . . 530
E.4 console/ . . . 531
E.4.1 console/portdat_console.h . . . 531
E.4.2 console/portfns_console.h . . . 531
E.4.3 console/devcons.c . . . 531
E.5 core/ . . . 533
E.5.1 core/portdat_forward.h . . . 533
E.5.2 core/portdat_core.h . . . 533
E.5.3 core/portfns_core.h . . . 534
E.5.4 core/portdat.h . . . 534
E.5.5 core/error.h . . . 534
E.5.6 core/error.c . . . 534
E.5.7 core/portdat.c . . . 534
E.5.8 core/portfns.c . . . 535
E.6 devices/ . . . 535

E.6.1 devices/portdat_devices.h . . . 535
E.6.2 devices/portfns_devices.h . . . 536
E.6.3 devices/keyboard/portkbd.c . . . 536
E.6.4 devices/keyboard/latin1.c . . . 537
E.6.5 devices/keyboard/latin1.h . . . 538
E.6.6 TO LP SPLIT devices/keyboard/devkbin.c . . . 538
E.6.7 TO LPSPLIT devices/keyboard/devkbmap.c . . . 540
E.6.8 TO LPSPLIT devices/mouse/portmouse.c . . . 543
E.6.9 TO LPSPLIT devices/mouse/devmouse.c . . . 547
E.6.10 devices/sys/devroot.c . . . 557
E.6.11 devices/sys/devsys.c . . . 559
E.6.12 TO LPSPLIT devices/sys/devenv.c . . . 560
E.6.13 devices/sys/devpipe.c . . . 566
E.6.14 devices/sys/devproc.c . . . 567
E.7 files/ . . . 569
E.7.1 files/portdat_files.h . . . 569
E.7.2 files/portfns_files.h . . . 571
E.7.3 files/chan.c . . . 571
E.7.4 files/dev.c . . . 573
E.7.5 files/allocb.c . . . 574
E.7.6 files/env.c . . . 575
E.7.7 files/file.c . . . 575
E.7.8 files/mnt.c . . . 575
E.7.9 files/qio.c . . . 576
E.7.10 files/sysfile.c . . . 578
E.8 filesystems/ . . . 579
E.9 init/ . . . 579
E.9.1 init/portfns_init.h . . . 579
E.9.2 init/user/preboot/initcode.c . . . 579
E.9.3 init/user/boot/aux.c . . . 579
E.9.4 init/user/boot/boot.c . . . 579
E.9.5 init/user/boot/boot.h . . . 579
E.9.6 init/user/boot/local.c . . . 579
E.10 lib/ . . . 579
E.10.1 lib/lib.h . . . 579
E.11 memory/ . . . 582
E.11.1 memory/portdat_memory.h . . . 582
E.11.2 memory/portfns_memory.h . . . 584
E.11.3 memory/portdat_memory.c . . . 584
E.11.4 memory/xalloc.c . . . 584
E.11.5 memory/pool.c . . . 585
E.11.6 memory/alloc.c . . . 585
E.11.7 memory/fault.c . . . 587
E.11.8 memory/page.c . . . 588
E.11.9 memory/segment.c . . . 589
E.11.10 memory/swap.c . . . 590
E.11.11 memory/sysmemory.c . . . 591
E.12 misc/ . . . 591
E.12.1 misc/portdat_misc.h . . . 591

E.12.2 misc/portfns_misc.h . . . 591
E.12.3 misc/parse.c . . . 591
E.12.4 misc/random.c . . . 592
E.13 interrupts/ . . . 592
E.13.1 interrupts/portfns_interrupts.h . . . 592
E.14 processes/ . . . 592
E.14.1 processes/portdat_processes.h . . . 592
E.14.2 processes/portfns_processes.h . . . 595
E.14.3 processes/portdat_processes.c . . . 595
E.14.4 time/portfns_time.h . . . 595
E.14.5 time/portdat_time.h . . . 596
E.14.6 time/alarm.c . . . 596
E.14.7 time/portclock.c . . . 597
E.14.8 processes/pgrp.c . . . 597
E.14.9 processes/proc.c . . . 598
E.14.10 time/tod.c . . . 602
E.14.11 processes/sysproc.c . . . 603
E.14.12 processes/syssema.c . . . 603
E.15 security/ . . . 605
E.15.1 security/portfns_security.h . . . 605
E.15.2 security/auth.c . . . 605
E.16 syscalls/ . . . 606
E.16.1 syscalls/systab.h . . . 606
E.16.2 syscalls/syscallfmt.c . . . 606
E.16.3 syscalls/systab.c . . . 606
E.17 ../include/arch/arm/ . . . 607
E.17.1 ../include/arch/arm/ureg.h . . . 607
E.18 arch/arm/ . . . 607
E.18.1 arch/arm/dat_arch.h . . . 607
E.18.2 arch/arm/arm.h . . . 607
E.18.3 arch/arm/arminstr.ha . . . 608
E.18.4 arch/arm/io.h . . . 609
E.18.5 arch/arm/coproc.c . . . 609
E.18.6 arch/arm/vcore.c . . . 611
E.19 concurrency/arm/ . . . 616
E.19.1 concurrency/arm/spl.s . . . 616
E.19.2 concurrency/arm/tas_raspi1.s . . . 617
E.19.3 concurrency/arm/tas_raspi2.s . . . 617
E.19.4 concurrency/arm/concurrency_raspi1.c . . . 617
E.19.5 concurrency/arm/concurrency_raspi2.c . . . 618
E.20 core/arm/ . . . 618
E.20.1 core/arm/dat.h . . . 618
E.20.2 core/arm/dat_core.h . . . 619
E.20.3 core/arm/dat_forward.h . . . 619
E.20.4 core/arm/fns.h . . . 619
E.21 buses/arm/ . . . 622
E.22 devices/ . . . 622
E.22.1 devices/keyboard/arm/kbd.c . . . 622
E.22.2 devices/mouse/arm/mouse.c . . . 622

E.22.3 devices/sys/arm/devarch.c . . . 623
E.23 init/arm/ . . . 627
E.23.1 init/arm/start.s . . . 627
E.23.2 init/arm/startv6.s . . . 627
E.23.3 init/arm/startv7.s . . . 627
E.23.4 init/arm/main.c . . . 628
E.23.5 init/arm/raspi1.c . . . 630
E.23.6 init/arm/raspi2.c . . . 631
E.23.7 init/user/preboot/arm/init9.s . . . 632
E.23.8 init/arm/rebootcode.s . . . 632
E.24 memory/arm/ . . . 633
E.24.1 memory/arm/dat_memory.h . . . 633
E.24.2 memory/arm/mem.h . . . 633
E.24.3 memory/arm/mmu.c . . . 634
E.24.4 memory/arm/mmuinvalidate.s . . . 635
E.24.5 memory/arm/cache_raspi1.s . . . 635
E.24.6 memory/arm/cache_raspi2.s . . . 636
E.25 interrupts/arm/ . . . 636
E.25.1 interrupts/arm/trap.c . . . 636
E.25.2 interrupts/arm/fault_helpers.s . . . 637
E.25.3 interrupts/arm/lexception.s . . . 638
E.26 time/arm/ . . . 638
E.26.1 time/arm/dat_time.h . . . 638
E.26.2 time/arm/clock.c . . . 638
E.26.3 time/arm/time_raspi1.s . . . 640
E.26.4 time/arm/time_raspi2.s . . . 640
E.27 processes/arm/ . . . 640
E.27.1 processes/arm/dat_processes.h . . . 640
E.27.2 processes/arm/arch.c . . . 641
E.27.3 processes/arm/vfp3.c . . . 641
E.27.4 processes/arm/lproc.s . . . 643
E.27.5 processes/arm/switch.s . . . 643
E.28 syscalls/arm/ . . . 643
E.28.1 syscalls/arm/syscall.c . . . 643
E.29 Extra files . . . 643
E.29.1 init/arm/bootconf.c . . . 643
E.29.2 devices/storage/arm/emmc.c . . . 648
E.29.3 init/arm/watchdog.c . . . 656
E.29.4 buses/arm/dwcotg.h . . . 657
E.29.5 buses/arm/devusb.c . . . 666
E.29.6 buses/arm/usbdwc.c . . . 694
E.29.7 buses/arm/dma.c . . . 713
E.29.8 buses/arm/uartmini.c . . . 718

Glossary **728**

Indexes **730**

References **730**

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a kernel.

1.1 Motivations

Why a kernel? Because I think you are a better programmer if you fully understand how things work under the hood, and the kernel is the program that everything else rely on.

Here are a few questions I hope this book will answer:

- What happens when the user turn on the computer? Which code gets executed? How does the kernel initialise itself?
- What happens when the user types `ls` in a terminal? What are the set of programs involved in such a command? What is the trace of such a command through the different layers of the software stack, from the keyboard interrupt to the display of text glyphs on the screen?
- Where does the kernel store meta data about the user processes?
- Why do the kernel code and some kernel data structures reside in the virtual memory space of every processes?
- What is the trace of a system call?
- What is the memory image of the kernel and the user processes?

1.2 The Plan 9 kernel: `9`

1.3 Other kernels

Here are a few kernels that I considered for this book, but which I ultimately discarded:

- Unix V6 (or its modern incarnation xv6) is too simple; there is no graphics, no network.
- XINU has a network stack but the kernel itself is too simple with no virtual memory for instance
- Minix is fairly small but it does not provide a simple windowing system as it relies on X11 (which is far more complicated than plan9 Rio windowing system)
- Linux is really big. Of course the Linux kernel contains thousands of specific device drivers and those things could be discarded when presenting the core of the Linux kernel, but this core is still far bigger than Plan 9 equivalent core.

1.4 Getting started

To get started you first need to get the source of the Plan 9 version described in this book. See http://www.principia-softwarica.org/.

Then you need to compile using a regular C compiler (e.g. gcc) the Plan 9 C cross compilers written by Ken Thompson: https://github.com/aryx/fork-kencc.

Then you need to cross compile, e.g. from Linux or macOS, the Plan 9 kernel using the Plan 9 C cross compilers installed in the previous step. You can also cross compile the whole Plan 9 operating system with all its libraries and utilities.

Finally you can test this compiled kernel and utilities by installing them on a SD card and by booting this SD card on a Raspberry Pi.

As a bonus step you can also now compile Plan 9 under Plan 9 itself!

1.5 Requirements

I assume you actually know a lot:

- C: most of this book is made of C source code, so you'll need to have a good knowledge of the C programming language [KR88].
- Operating system: (yep)

This book is not an introduction to operating systems. The goal of this book is to explain with full details a kernel but we assume you already have a vague idea of how an operating system works and so that you are familiar with concepts such as virtual memory, critical regions, interrupts, system calls, etc. I assume you already know most of the theory; this book is here to cover the practice.

- The ARM architecture:
- Assembly:

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course Plan9's authors who wrote most of this book: Ken Thompson, Rob Pike, Dave Presotto, Phil Winterbottom, Russ Cox, and many other people from Bell Labs. Thanks also to Federic Balesteros who wrote a lengthy note about the Plan 9 kernel. Finally, thanks to Richard Miller for porting the Plan 9 kernel to the Raspberry Pi machine.

Chapter 2

Overview

2.1 Operating system principles

An operating system is a program that manages other programs. It provides convenient abstractions of the hardware and multiplex the hardware resources for the benefit of user programs. The operating system usually provides:

- a virtual CPU, so that each process thinks it has all the CPU for itself (via preemptive scheduling and the timer interrupt)
- a virtual memory, so that each process thinks it has all memory for itself (via hardware virtual memory features such as paging)
- a virtual screen, so that each process thinks it has all the screen for itself (via the windowing system)
- a file abstraction, so a process can read and store data whether it's on a disk, network, floppy, etc
- a directory abstraction, so a process can organize data whether it's on a disk, network, floppy, etc

Entire books have been written on those subjects. I assume you already know what is an operating system and we will now focus on the concrete services (API) a kernel provides to the user programs.

2.2 9's syscall interface

⟨lib_core/libc/9syscall/sys.h 18a⟩≡

```
#define NOP     0
⟨sys.h process syscalls 18b⟩
⟨sys.h memory syscalls 19a⟩
⟨sys.h file syscalls 19b⟩
⟨sys.h directory syscalls 19c⟩
⟨sys.h namespace syscalls 19d⟩
⟨sys.h time syscalls 19e⟩
⟨sys.h ipc syscalls 19f⟩
⟨sys.h concurrency syscalls 19g⟩
⟨sys.h special file syscalls 19h⟩
⟨sys.h security syscalls 19i⟩
#define ERRSTR      39
```

⟨sys.h process syscalls 18b⟩≡ (18a)

```
#define RFORK       1
#define EXEC        2
#define EXITS       3
#define AWAIT       4
```

⟨sys.h *memory syscalls* 19a⟩≡ (18a)

```
#define BRK     5
```

⟨sys.h *file syscalls* 19b⟩≡ (18a)

```
#define OPEN        6
#define CLOSE       7
#define PREAD       8
#define PWRITE      9
#define SEEK        10
```

⟨sys.h *directory syscalls* 19c⟩≡ (18a)

```
#define CREATE      11
#define REMOVE      12
#define CHDIR       13
#define FD2PATH     14
#define STAT        15
#define FSTAT       16
#define WSTAT       17
#define FWSTAT      18
```

⟨sys.h *namespace syscalls* 19d⟩≡ (18a)

```
#define BIND        19
#define MOUNT       20
#define UNMOUNT     21
```

⟨sys.h *time syscalls* 19e⟩≡ (18a)

```
#define SLEEP       22
#define ALARM       23
```

⟨sys.h *ipc syscalls* 19f⟩≡ (18a)

```
#define PIPE        26

#define NOTIFY      24
#define NOTED       25

#define SEGATTACH   27
#define SEGDETACH   28
#define SEGFREE     29
#define SEGFLUSH    30
#define SEGBRK      31
```

⟨sys.h *concurrency syscalls* 19g⟩≡ (18a)

```
#define RENDEZVOUS  32
#define SEMACQUIRE  33
#define SEMRELEASE  34
#define TSEMACQUIRE 35
```

⟨sys.h *special file syscalls* 19h⟩≡ (18a)

```
#define DUP         36
```

⟨sys.h *security syscalls* 19i⟩≡ (18a)

```
#define FVERSION    37
#define FAUTH       38
```

2.3 helloraw.c and helloraw.s

2.4 minishell.c

2.5 The Raspberry Pi machine (ARM)

2.5.1 ARM CPU

⟨*constant* BY2WD*(arm)* 20a⟩≡ (633b)

```
#define BY2WD       4
```

⟨*constant* BY2V*(arm)* 20b⟩≡ (633b)

```
#define BY2V        8           /* only used in xalloc.c */
```

Main registers, Ureg

⟨*struct* Ureg*(arm)* 20c⟩≡ (607a)

```
//coupling: do not change the order! some assembly code assumes this order.
struct Ureg {
    ulong   r0;
    ulong   r1;
    ulong   r2;
    ulong   r3;
    ulong   r4;
    ulong   r5;
    ulong   r6;
    ulong   r7;
    ulong   r8;
    ulong   r9;
    ulong   r10;
    ulong   r11;
    ulong   r12;    /* sb */

    union {
        ulong   r13;
        ulong   sp;
    };
    union {
        ulong   r14;
        ulong   link;
    };

    ulong   type;   /* of exception */
    ulong   psr;

    ulong   pc; /* interrupted addr */ // r15
};
```

⟨*function* getcallerpc*(arm)* 20d⟩≡ (643b)

```
TEXT getcallerpc(SB), 1, $-4
    MOVW    0(R13), R0
    RET
```

Program Status Register (PSR)

Processor modes

⟨*type* PsrMode*(arm)* 21a⟩≡ (607c)

```
// Last 4 bits of PSR
#define PsrMusr     0x00000010  /* user mode */
#define PsrMsvc     0x00000013  /* 'protected mode for OS' */
⟨PsrMode other cases(arm) 99a⟩

#define PsrMask     0x0000001F
```

⟨*type* PsrDisable*(arm)* 21b⟩≡ (607c)

```
#define PsrDirq     0x00000080      /* disable IRQ interrupts */
⟨PsrDisable other cases(arm) 428d⟩
```

Exceptions

⟨*constant* HVECTORS*(arm)* 21c⟩≡ (607c)

```
#define HVECTORS    0xffff0000
```

2.5.2 ARM coprocessors

⟨*type* Coprocessors*(arm)* 21d⟩≡ (607c)

```
#define CpSC        15          /* System Control */
⟨Coprocessors other cases(arm) 391a⟩
```

System control coprocessor (CpSC)

⟨*type CpSC primary registers(arm)* 21e⟩≡ (607c)

```
#define CpCONTROL   1           /* miscellaneous control */
#define CpTTB       2           /* Translation Table Base(s) */
#define CpDAC       3           /* Domain Access Control */
#define CpFSR       5           /* Fault Status */
#define CpFAR       6           /* Fault Address */
#define CpTLB       8           /* TLB control */
#define CpSPM       15          /* system performance monitor (arm1176) */
⟨CpSC primary registers other cases(arm) 400d⟩
```

⟨*type CpCONTROL opcode2(arm)* 21f⟩≡ (607c)

```
/*
 * CpCONTROL op2 codes, op1==0, Crm==0.
 */
#define CpMainctl   0
⟨CpCONTROL.opcode2 other cases(arm) 394d⟩
```

⟨*type CpCONTROL CpMainctl(arm)* 21g⟩≡ (607c)

```
/*
 * CpCONTROL: op1==0, CRm==0, op2==CpMainctl.
 * main control register.
 * cortex/armv7 has more ops and CRm values.
 */
#define CpCmmu      0x00000001  /* M: MMU enable */
#define CpChv       0x00002000  /* V: high vectors */
⟨CpCONTROL.CpMainctl other cases(arm) 400e⟩
```

Floating point coprocessors

2.5.3 ARM MMU

⟨*constant* BY2PG*(arm)* 22a⟩≡ (633b)
```
#define BY2PG       (4*KiB)         /* bytes per page */
```

⟨*constant* PGSHIFT*(arm)* 22b⟩≡ (633b)
```
#define PGSHIFT     12          /* log(BY2PG) */
```

2.5.4 Input/Output devices

Memory-mapped IO

⟨*constant* VIRTIO*(arm)* 22c⟩≡ (633b)
```
#define VIRTIO      0x7E000000      /* i/o registers */
```

⟨*constant* IOSIZE*(arm)* 22d⟩≡ (633b)
```
#define IOSIZE      (16*MiB)
```

⟨*constant* SYSTIMERS*(arm)* 22e⟩≡ (639b)
```
#define SYSTIMERS (VIRTIO+0x3000)
```

⟨*constant* ARMTIMER*(arm)* 22f⟩≡ (639b)
```
#define ARMTIMER (VIRTIO+0xB400)
```

⟨*constant* INTREGS*(arm)* 22g⟩≡ (636c)
```
#define INTREGS     (VIRTIO+0xB200)
```

⟨*constant* DMAREGS*(arm)* 22h⟩≡ (717b)
```
#define DMAREGS (VIRTIO+0x7000)
```

⟨*constant* POWERREGS*(arm)* 22i⟩≡ (657c)
```
#define POWERREGS   (VIRTIO+0x100000)
```

⟨*constant* AUXREGS*(arm)* 22j⟩≡ (726)
```
#define AUXREGS     (VIRTIO+0x215000)
```

Output device: VideoCore GPU

⟨*constant* MAILBOX*(arm)* 22k⟩≡ (615d)
```
#define MAILBOX     (VIRTIO+0xB880)
```

⟨*struct* Fbinfo*(arm)* 22l⟩≡ (615d)
```
// Framebuffer
// The order matters! the fields match the memory-mapped external registers.
struct Fbinfo {
    u32int  xres;
    u32int  yres;
    u32int  xresvirtual;
    u32int  yresvirtual;
    u32int  pitch;          /* returned by gpu */
    u32int  bpp;
    u32int  xoffset;
    u32int  yoffset;
    u32int  base;           /* returned by gpu */
    u32int  screensize;     /* returned by gpu */
};
```

⟨*constant* FRAMEBUFFER*(arm)* 23a⟩≡ (633b)

```
#define FRAMEBUFFER 0xC0000000      /* video framebuffer */
```

Input devices: USB keyboard and mouse

⟨*constant* USBREGS*(arm)* 23b⟩≡ (711b)

```
#define  USBREGS (VIRTIO+0x980000)
```

Storage device: SD card

⟨*constant* EMMCREGS*(arm)* 23c⟩≡ (655b)

```
#define EMMCREGS    (VIRTIO+0x300000)
```

Other devices

⟨*constant* GPIOREGS*(arm)* 23d⟩≡ (726)

```
#define GPIOREGS    (VIRTIO+0x200000)
```

2.6 C and assembly

⟨*portdat.h macros* 23e⟩≡ (? 0—1) 23f▷

```
#define ROUND(s, sz)  (((s)+(sz-1)) & ~(sz-1))
```

⟨*portdat.h macros* 23f⟩+≡ (? 0—1) ◁23e 23g▷

```
#define MASK(v) ((1UL << (v)) - 1)  /* mask ‘v’ bits wide */
```

⟨*portdat.h macros* 23g⟩+≡ (? 0—1) ◁23f 23h▷

```
#define MIN(a, b) ((a) < (b)? (a): (b))
```

⟨*portdat.h macros* 23h⟩+≡ (? 0—1) ◁23g 23i▷

```
// used in bcm/mmu.c
#define FEXT(d, o, w) (((d)>>(o)) & ((1<<(w))-1))
```

⟨*portdat.h macros* 23i⟩+≡ (? 0—1) ◁23h 67c▷

```
#define HOWMANY(x, y) (((x)+((y)-1))/(y))
#define ROUNDUP(x, y) (HOWMANY((x), (y))*(y)) /* ceiling */
```

⟨*constant* KiB*(arm)* 23j⟩≡ (633b)

```
#define KiB     1024u           /* Kibi 0x0000000000000400 */
```

⟨*constant* MiB*(arm)* 23k⟩≡ (633b)

```
#define MiB     1048576u        /* Mebi 0x0000000000100000 */
```

⟨*constant* GiB*(arm)* 23l⟩≡ (633b)

```
#define GiB     1073741824u     /* Gibi 000000000040000000 */
```

2.7 Code organization

2.7.1 Portable versus arch-specific code

2.7.2 Main headers

⟨*kernel basic includes* 23m⟩≡ (726 717b 711b 691b 657c 655b 647 643c 641 639b 636c 635a 631 628c 626c 622 618 615d ? ? ? ? ? ? ? ? ? ? ?

```
#include <u.h>
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
```

2.7.3 Top directories

⟨*dat.h includes(arm)* 24a⟩≡ (619a)

```
#include "dat_core.h"                    // Arch_Cpu
#include "../port/portdat_core.h"        // Cpu!!, Conf, Label
#include "../port/portdat_concurrency.h" // Lock
#include "dat_memory.h"                  // Arch_Proc
#include "../port/portdat_memory.h"      // Page, Pagetable, Segment, KImage
#include "../port/portdat_files.h"       // Chan!! Dev!!
#include "dat_time.h"                    // Arch_HZ
#include "../port/portdat_time.h"        // Timer
#include "dat_processes.h"               // Arch_FPsave
#include "../port/portdat_processes.h"   // Proc!!!
#include "../port/portdat_console.h"     // keyboard queue, consdevtab
#include "../port/portdat_devices.h"     // DevConf, DevPort (not that used)
#include "../port/portdat_misc.h"        // Cmd
#include "../port/portdat_buses.h"       // Uart
#include "dat_arch.h"                    // Soc
```

⟨*fns.h includes(arm)* 24b⟩≡ (619d)

```
#include "../port/portfns_core.h"
#include "../port/portfns_concurrency.h"
#include "../port/portfns_memory.h"
#include "../port/portfns_processes.h"
#include "../port/portfns_files.h"
#include "../port/portfns_time.h"
#include "../port/portfns_interrupts.h"
#include "../port/portfns_console.h"
#include "../port/portfns_devices.h"
#include "../port/portfns_buses.h"
#include "../port/portfns_security.h"
#include "../port/portfns_network.h"
#include "../port/portfns_syscalls.h"
#include "../port/portfns_misc.h"
#include "../port/portfns_init.h"
```

2.8 Software architecture

2.8.1 Booting overview (ARM)

2.8.2 Memory overview (ARM)

⟨*pad memory pointer types(arm)* 24c⟩≡ (619a) 24d ▷

```
// physical address
typedef uintptr phys_addr;
// virtual address (a user_addr or kern_addr)
typedef uintptr virt_addr;

// user address (under KZERO)
typedef uintptr user_addr;
// kernel address (mostly physical + KZERO)
typedef uintptr kern_addr;
```

⟨*pad memory pointer types(arm)* 24d⟩+≡ (619a) ◁ 24c

```
// wp = word pointer, vp = void pointer
typedef ulong* virt_wp;
typedef void* virt_vp;
typedef ulong* kern_wp;
```

```
typedef void* kern_vp;
typedef ulong* user_wp;
typedef void* user_vp;
```

⟨*constant* KSEG0*(arm)* 25a⟩≡ (633b)

```
#define KSEG0       0x80000000      /* kernel segment */
```

⟨*constant* KZERO*(arm)* 25b⟩≡ (633b)

```
#define KZERO       KSEG0           /* kernel address space */
```

⟨*constant* KTZERO*(arm)* 25c⟩≡ (633b)

```
#define KTZERO      (KZERO+0x8000)      /* kernel text start */
```

⟨*constant* CPUADDR*(arm)* 25d⟩≡ (633b)

```
#define CPUADDR     (KZERO+0x2000)      /* Cpu structure */
```

⟨*constant* UZERO*(arm)* 25e⟩≡ (633b)

```
#define UZERO       0           /* user segment */
```

⟨*constant* UTZERO*(arm)* 25f⟩≡ (633b)

```
#define UTZERO      (UZERO+BY2PG)       /* user text start */
```

⟨*constant* USTKTOP*(arm)* 25g⟩≡ (633b)

```
#define USTKTOP     0x20000000      /* user segment end +1 */
```

⟨*constant* USTKSIZE*(arm)* 25h⟩≡ (633b)

```
#define USTKSIZE    (8*1024*1024)       /* user stack size */
```

⟨*macro* PADDR*(arm)* 25i⟩≡ (619d)

```
#define PADDR(va)   PTR2UINT(PHYSDRAM | ((uintptr)(va) & ~KSEGM))
```

⟨*macro PADDR bis(arm)* 25j⟩≡ (608)

```
#define PADDR(va)   (((va) & ~KSEGM))
```

⟨*constant* PHYSDRAM*(arm)* 25k⟩≡ (633b)

```
#define PHYSDRAM    0
```

⟨*constant* KSEGM*(arm)* 25l⟩≡ (633b)

```
/* mask to check segment; good for 1GB dram */
#define KSEGM       0xC0000000
```

⟨*macro* KADDR*(arm)* 25m⟩≡ (619d)

```
#define KADDR(pa)   UINT2PTR(KZERO    | ((uintptr)(pa) & ~KSEGM))
```

⟨*macro* PTR2UINT*(arm)* 25n⟩≡ (619d)

```
#define PTR2UINT(p) ((uintptr)(p))
```

⟨*macro* UINT2PTR*(arm)* 25o⟩≡ (619d)

```
#define UINT2PTR(i) ((void*)(i))
```

⟨*function* arch_cankaddr*(arm)* 25p⟩≡ (635a)

```
/*
 * Return the number of bytes that can be accessed via KADDR(pa).
 * If pa is not a valid argument to KADDR, return 0.
 */
uintptr
arch_cankaddr(phys_addr pa)
{
    if(pa < memsize)        /* assumes PHYSDRAM is 0 */
        return memsize - pa;
    return 0;
}
```

2.8.3 Syscall overview (ARM)

⟨*typedef Syscall* 26a⟩≡ (26b)

```
typedef long Syscall(ulong*); // user_wp? or copied in Sargs?
```

⟨systab.h 26b⟩≡

```
// used by systab.c but also by trap.c for certain codes
#include "../../../../lib_core/libc/9syscall/sys.h"

⟨typedef Syscall 26a⟩

extern Syscall *systab[];
extern int nsyscall;
extern char *sysctab[];
```

⟨*global* systab 26c⟩≡ (606b)

```
Syscall *systab[] = {
    [NOP]     sysnop,
    ⟨systab process syscalls 73a⟩
    ⟨systab memory syscalls 47a⟩
    ⟨systab file syscalls 206a⟩
    ⟨systab directory syscalls 230a⟩
    ⟨systab namespace syscalls 247a⟩
    ⟨systab time syscalls 165a⟩
    ⟨systab ipc syscalls 334a⟩
    ⟨systab concurrency syscalls 334b⟩
    ⟨systab special file syscalls 483a⟩
    ⟨systab security syscalls 456c⟩
    [ERRSTR]    syserrstr,
};
```

⟨*global* nsyscall 26d⟩≡ (606b)

```
int nsyscall = nelem(systab);
```

⟨*syscall nop* 26e⟩≡ (603a)

```
// void nop(void);
long
sysnop(ulong*)
{
    print("Hello World\n");
    return 0;
}
```

2.9 Book structure

Chapter 3

Core Data Structures

3.1 Conf and conf

⟨*struct* Conf 27a⟩≡ (533)

```
struct Conf
{
    ulong ncpu;    /* processors */
    Confmem mem[4];   /* physical memory */

    ulong nproc;    /* processes */
    ⟨Conf other fields 121c⟩
};
```

⟨*global* conf 27b⟩≡ (534b)

```
Conf conf;
```

⟨*struct* Confmem 27c⟩≡ (533)

```
// memory "bank"
struct Confmem
{
    phys_addr base;
    uintptr limit; // used by bcm only. phys_addr too?
    ulong npage;
    ⟨Confmem other fields 367b⟩
};
```

⟨*globals confname and confvar* 27d⟩≡ (534b)

```
// conf (boot) parameters *e.g. { "*kernelpercent*" => "60" }
// hash<string, string>
char *confname[MAXCONF];
char *confval[MAXCONF];
```

⟨*constant* MAXCONF 27e⟩≡ (533)

```
#define MAXCONF         64
```

⟨*global* nconf 27f⟩≡ (534b)

```
// Hashtbl.length(confname)
int nconf;
```

⟨*function* getconf 27g⟩≡

```
char* getconf(char *name)
{
        int i;
        for(i = 0; i < nconf; i++)
                if(cistrcmp(confname[i], name) == 0)
                        return confval[i];
        return nil;
}
```

3.2 Cpu and cpu

⟨*struct* Cpu 28a⟩≡ (533)

```
//coupling: do not change the order of the first 4 fields!
// Some assembly code assumes this order.
struct Cpu
{
    int cpuno;     /* physical id of processor (KNOWN TO ASSEMBLY) */
    ⟨Cpu second field 34f⟩
      // ref<Proc>, or None if halting?
    Proc* proc;     /* current process on this processor */

    struct Arch_Cpu;

    ⟨Cpu time fields 28b⟩
    ⟨Cpu stat fields 512a⟩
    ⟨Cpu other fields 46d⟩

    // must be at the end of the structure!
    int stack[1];
};
```

⟨Cpu *time fields* 28b⟩≡ (28a) 28c ▷

```
ulong ticks;      /* of the clock since boot time */
```

⟨Cpu *time fields* 28c⟩+≡ (28a) ◁ 28b

```
uvlong  cpuhz;
int cpumhz; // cpuhz / 1_000_000
```

⟨*constant* CPUSIZE*(arm)* 28d⟩≡ (633b)

```
#define CPUSIZE BY2PG
```

⟨*struct* Arch_Cpu*(arm)* 28e⟩≡ (619b)

```
//coupling: do not change the first field! Some assembly code assumes this order
struct Arch_Cpu {
    ⟨Cpu Arch mmu fields(arm) 114f⟩
    ⟨Cpu Arch float fields(arm) 391c⟩
    ⟨Cpu Arch save of first five registers fields(arm) 194d⟩
};
```

⟨*global* cpu*(arm)* 28f⟩≡ (619a)

```
extern register Cpu* cpu;           /* R10 */
```

⟨*global* cpus 28g⟩≡ (534b)

```
/*
 * Each processor sees its own Cpu structure at address CPUADDR.
 * However, the Cpu structures must also be available via the per-processor
 * MMU information array cpus, mainly for disambiguation and access to
 * the clock which is only maintained by the bootstrap processor (0).
 */
// array<ref<Cpu>>
Cpu* cpus[MAXCPUS];
```

⟨*constant* MAXCPUS*(arm)* 28h⟩≡ (633b)

```
#define MAXCPUS     4           /* max # cpus system can run */
```

⟨*macro* CPUS 28i⟩≡ (533)

```
#define CPUS(n)  (cpus[n])
```

⟨*struct* Active 29a⟩≡ (592c)

```
struct Active
{
    // array<bool> (coupling: sizeof(int) must be >= MAXCPUS)
    int cpus;      /* bitmap of active CPUs */
    bool exiting;    /* shutdown */
    ⟨Active other fields 489b⟩
    // extra
    Lock;
};
```

⟨*global* active 29b⟩≡ (595a)

```
struct Active active;
```

3.3 Proc and up

⟨*struct* Proc 29c⟩≡ (592c)

```
struct Proc
{
//-----------------------------------------------------------------
//coupling: have to be first! Some assembly code assumes this order.
//-----------------------------------------------------------------
    ⟨Proc assembly fields 151e⟩
//-----------------------------------------------------------------
// State
//-----------------------------------------------------------------
    ⟨Proc state fields 30b⟩
//-----------------------------------------------------------------
// Memory
//-----------------------------------------------------------------
    ⟨Proc memory fields 65e⟩

    struct Arch_Proc;
//-----------------------------------------------------------------
// Scheduling
//-----------------------------------------------------------------
    ⟨Proc scheduling fields 145c⟩
//-----------------------------------------------------------------
// Files
//-----------------------------------------------------------------
    ⟨Proc files fields 31b⟩
//-----------------------------------------------------------------
// Notes
//-----------------------------------------------------------------
    ⟨Proc notes fields 335f⟩
//-----------------------------------------------------------------
// Process hierarchy
//-----------------------------------------------------------------
    ⟨Proc hierarchy fields 89b⟩
//-----------------------------------------------------------------
// Synchronization
//-----------------------------------------------------------------
    ⟨Proc synchronization fields 45h⟩
//-----------------------------------------------------------------
// Error management
//-----------------------------------------------------------------
    ⟨Proc error managment fields 520b⟩
//-----------------------------------------------------------------
```

```
// Stats, profiling
//-----------------------------------------------------------------
    ⟨Proc stats and profiling fields 380b⟩
//-----------------------------------------------------------------
// Debugging (the kernel itself)
//-----------------------------------------------------------------
    ⟨Proc debugging fields 36b⟩
//-----------------------------------------------------------------
// For debugger, strace
//-----------------------------------------------------------------
    ⟨Proc debugger fields 102e⟩
//-----------------------------------------------------------------
// Other
//-----------------------------------------------------------------
    ⟨Proc other fields 96b⟩
//-----------------------------------------------------------------
// Extra
//-----------------------------------------------------------------
    ⟨Proc extra fields 39f⟩
};
```

⟨*struct* Arch_Proc*(arm)* 30a⟩≡ (633a)

```
struct Arch_Proc
{
    ⟨Proc Arch memory fields(arm) 114j⟩
};
```

⟨Proc *state fields* 30b⟩≡ (29c) 73b ▷

```
// enum<Procstate>
int state; // Dead, Queuing, etc, (used by /proc/#/status if psstate==nil)
```

⟨*enum* procstate 30c⟩≡ (592c)

```
/* Process states, Proc.state */
enum Procstate
{
    Dead = 0,
    Running,
    ⟨Procstate cases 39g⟩
};
```

⟨*global* statename 30d⟩≡ (599c)

```
// hash<enum<Procstate>, string>, coupling: with enum Procstate
char *statename[] =
{
    [Dead]        = "Dead",
    [Running]     = "Running",
    [Queueing]    = "Queueing",
    [QueueingR]   = "QueueingR",
    [QueueingW]   = "QueueingW",
    [Moribund]    = "Moribund",
    [Ready]       = "Ready",
    [Scheding]    = "Scheding",
    [Wakeme]      = "Wakeme",
    [Broken]      = "Broken",
    [Stopped]     = "Stopped",
    [Rendezvous]  = "Rendezvous",
    [Waitrelease] = "Waitrelease",
};
```

⟨*global* up*(arm)* 30e⟩≡ (619a)

```
extern register Proc* up;           /* R9 */
```

3.4 Chan

⟨*struct* Chan 31a⟩≡ (569c)

```
struct Chan
{
    ushort type; // idx in devtab
    ulong dev;
    Qid qid;

    Path* path;

    vlong offset;     /* in fd */

    //enum<open>, actually restricted to the OREAD|OWRITE|ORDWR of open type
    ushort mode;     /* read/write */

    bool ismtpt; // is a mount point

    union {
       void* aux; // generic pointer, for specific usages
       ⟨Chan union other fields 343c⟩
    };
    ⟨Chan other fields 206c⟩

    // extra
    Ref; /* the Lock in this Ref is also Chan's lock */
    ⟨Chan extra fields 207a⟩
};
```

⟨Proc *files fields* 31b⟩≡ (29c) 209c ▷

```
// ref<Chan>
Chan  *slash; // The root! used by namec()
// ref_counted<Chan>
Chan  *dot; // The current directory
```

⟨*struct* Qid 31c⟩≡ (580e)

```
struct Qid
{
  // note that this is not a string, but an int! it's kind of an inode
  uvlong  path;
  // for cache invalidation
  ulong vers;
  // enum<Qidtype>
  byte type;
};
```

⟨*enum* qidtype 31d⟩≡ (580e)

```
/* bits in Qid.type */
enum Qidtype {
  QTFILE = 0x00,    /* plain file */
  QTDIR = 0x80,    /* type bit for directories */
  ⟨Qidtype cases 317b⟩
};
```

⟨*function* eqqid 31e⟩≡ (571)

```
bool
eqqid(Qid a, Qid b)
{
    return a.path==b.path && a.vers==b.vers;
}
```

⟨*function* mkqid 32a⟩≡ (573)

```
void
mkqid(Qid *q, vlong path, ulong vers, int type)
{
    q->type = type;
    q->vers = vers;
    q->path = path;
}
```

⟨*function* eqchan 32b⟩≡ (571)

```
bool
eqchan(Chan *a, Chan *b, bool skipvers)
{
    if(a->qid.path != b->qid.path)
        return false;
    if(!skipvers && a->qid.vers!=b->qid.vers)
        return false;

    if(a->type != b->type)
        return false;
    if(a->dev != b->dev)
        return false;
    return true;
}
```

⟨*function* eqchantdqid 32c⟩≡ (571)

```
bool
eqchantdqid(Chan *a, int type, int dev, Qid qid, bool skipvers)
{
    if(a->qid.path != qid.path)
        return false;
    if(!skipvers && a->qid.vers!=qid.vers)
        return false;
    if(a->type != type)
        return false;
    if(a->dev != dev)
        return false;
    return true;
}
```

3.5 Dev and devtab

⟨*struct* Dev 32d⟩≡ (569c)

```
struct Dev
{
    Rune dc; // dev character code, e.g. '/' (devroot), 'e' (devenv), ...
    char* name;

    ⟨Dev methods 200a⟩
};
```

⟨*global* devtab 32e⟩≡ (573)

```
// array<ref_own<Dev>>
Dev** devtab = nil;
```

3.6 Soc and soc (ARM)

⟨*struct* Soc*(arm)* 33a⟩≡ (607b)

```
struct Soc {            /* SoC dependent configuration */
    ulong   dramsize;
    phys_addr physio;
    ⟨Soc bus fields 714e⟩
    ⟨Soc mmu fields 401d⟩
    ⟨Soc other fields 406i⟩
};
```

⟨*global* soc*(raspberry pi1)(arm)* 33b⟩≡ (631a)

```
Soc soc = {
    .dramsize   = 512*MiB,    // 0 to 0x20000000
    .physio     = 0x20000000, // after RAM
    ⟨soc(raspberry pi1) other fields(arm) 401e⟩
};
```

Chapter 4

Concurrency

4.1 Critical regions

4.2 Interrupts disabling: `arch_splhi()`, `arch_spllo()` (ARM)

⟨*function* `arch_splhi` *(arm)* 34a⟩≡ (616)

```
// bool   arch_splhi(void);
TEXT arch_splhi(SB), 1, $-4
    ⟨arch_splxxx() save caller information(arm) 34e⟩
    MOVW    CPSR, R0            /* turn off irqs (but not fiqs) */
    ORR $(PsrDirq), R0, R1
    MOVW    R1, CPSR
    RET
```

⟨*function* `arch_spllo` *(arm)* 34b⟩≡ (616)

```
// bool   arch_spllo(void);
TEXT arch_spllo(SB), 1, $-4
    MOVW    CPSR, R0            /* turn on irqs and fiqs */
    BIC $(PsrDirq|PsrDfiq), R0, R1
    MOVW    R1, CPSR
    RET
```

⟨*function* `arch_spllx` *(arm)* 34c⟩≡ (616)

```
// void    arch_splx(bool);
TEXT arch_splx(SB), 1, $-4
    ⟨arch_splxxx() save caller information(arm) 34e⟩
    MOVW    R0, R1              /* reset interrupt level */
    MOVW    CPSR, R0 //dead
    MOVW    R1, CPSR
    RET
```

⟨*function* `arch_islo` *(arm)* 34d⟩≡ (616)

```
// bool arch_islo(void);
TEXT arch_islo(SB), 1, $-4
    MOVW    CPSR, R0
    AND $(PsrDirq), R0
    EOR $(PsrDirq), R0
    RET
```

⟨`arch_splxxx()` *save caller information(arm)* 34e⟩≡ (34)

```
MOVW    $(CPUADDR+4), R2        /* save caller pc in Cpu */
MOVW    R14, 0(R2)
```

⟨`Cpu` *second field* 34f⟩≡ (28a)

```
// must be second field at 0x04, used by arch_splhi()
ulong splpc;      /* pc of last caller to arch_splhi */
```

4.3 Test and set: arch_tas() (ARM)

⟨*function* arch_tas*(raspberry pi1)(arm)* 35a⟩≡ (617a)

```
TEXT    arch_tas(SB), $-4
    MOVW    R0,R1
    MOVW    $1,R0
    SWPW    R0,(R1)         /* fix: deprecated in armv6 */
    RET
```

4.4 Atomic count: arch_xinc() and arch_xdec() (ARM)

⟨*function* arch_xinc*(raspberry pi1)(arm)* 35b⟩≡ (618a)

```
void
arch_xinc(long *p)
{
    int s;

    s = arch_splhi();
    ++*p;
    arch_splx(s);
}
```

⟨*function* arch_xdec*(raspberry pi1)(arm)* 35c⟩≡ (618a)

```
long
arch_xdec(long *p)
{
    int s, v;

    s = arch_splhi();
    v = --*p;
    arch_splx(s);
    return v;
}
```

4.5 Spin Lock

⟨*struct* Lock 35d⟩≡ (528)

```
// a.k.a Spinlock
struct Lock
{
    ulong key; // 0 when unset, 1 (or 0xDEADDEAD) when acquired, could be a bool

    ⟨Lock ilock fields 37e⟩
    ⟨Lock debugging fields 36a⟩
    ⟨Lock other fields 36c⟩
};
```

⟨*function* lock 35e⟩≡ (529b)

```
int
lock(Lock *l)
{
    int i;
    ⟨lock() other locals 36d⟩

    ⟨lock() start of lock, stat 516f⟩
    ⟨lock() increment nlocks 45i⟩
```

```
    if(arch_tas(&l->key) == 0){ // lock old value was 0, the lock was not held
        ⟨lock() got the lock, update other fields 36e⟩
        ⟨lock ifdef LOCKCYCLES 515d⟩
        return 0;
    }
    //else

    ⟨lock() decrement nlocks 45j⟩
    ⟨lock() when lock already held, stat 516g⟩

    // The lock was already held, need to spin
    for(;;){
    ⟨lock() in loop when lock already held, stat 517a⟩
        i = 0;
        while(l->key){
           ⟨lock() optional priority-inversion for real-time process 433f⟩
            if(i++ > 100000000){
                i = 0;
                lockloop(l, pc);
            }
        }
        // try again
        ⟨lock() increment nlocks 45i⟩
        if(arch_tas(&l->key) == 0){
            ⟨lock() got the lock, update other fields 36e⟩
            ⟨lock ifdef LOCKCYCLES 515d⟩
            return 1;
        }
        ⟨lock() decrement nlocks 45j⟩
    }
    return -1; // unreachable
}
```

⟨Lock *debugging fields* 36a⟩≡ (35d) 45c ▷

```
// for debugging, the caller who did the lock()
kern_addr pc;
```

⟨Proc *debugging fields* 36b⟩≡ (29c) 37f ▷

```
Lock* lastlock;
```

⟨Lock *other fields* 36c⟩≡ (35d)

```
// option<ref<Cpu>>, None when key = 0?
Cpu  *cpu; // not that used, only in iprintcanlock apparently
```

⟨lock() *other locals* 36d⟩≡ (35e)

```
ulong pc = getcallerpc(&l);
```

⟨lock() *got the lock, update other fields* 36e⟩≡ (35) 38d ▷

```
if(up)
    up->lastlock = l;
l->pc = pc;
l->cpu = CPUS(cpu->cpuno); // pad's third bugfix
```

⟨*function* unlock 36f⟩≡ (529b)

```
void
unlock(Lock *l)
{
    ⟨unlock ifdef LOCKCYCLES 516b⟩

    ⟨unlock() sanity checks 37a⟩
```

```
    l->cpu = nil;
    l->key = 0;
    ⟨unlock() after modified key, call coherence 404a⟩

    ⟨unlock() if delaysched 153c⟩
}
```

⟨unlock() *sanity checks* 37a⟩≡ (36f) 39b ▷

```
if(l->key == 0)
    print("unlock: not locked: pc %#p\n", getcallerpc(&l));
```

⟨*function* canlock 37b⟩≡ (529b)

```
bool
canlock(Lock *l)
{
    ⟨lock() increment nlocks 45i⟩
    if(arch_tas(&l->key) != 0){ // lock old value != 0, lock was already held
       ⟨lock() decrement nlocks 45j⟩
        return false;
    }
    // else
    ⟨canlock() lock was not already held, update other fields 37c⟩
    ⟨lock ifdef LOCKCYCLES 515d⟩
    return true;
}
```

⟨canlock() *lock was not already held, update other fields* 37c⟩≡ (37b) 45e ▷

```
if(up)
    up->lastlock = l;
l->pc = getcallerpc(&l);
l->cpu = CPUS(cpu->cpuno);
```

4.6 Uninterruptable ILock

⟨*struct* ILock 37d⟩≡ (528)

```
typedef Lock ILock;
```

⟨Lock *ilock fields* 37e⟩≡ (35d)

```
bool_ushort isilock; // false when from lock(), true when from ilock()
ulong sr; // saved priority level when using ilock() to restore in iunlock()
```

⟨Proc *debugging fields* 37f⟩+≡ (29c) ◁36b 39e ▷

```
Lock  *lastilock;
```

⟨*function* ilock 37g⟩≡ (529b)

```
// To provide mutual exclusion with interrupt code and avoiding deadlock.
// By using arch_splhi() we disable interrupts while running the critical region
// code.
void
ilock(Lock *l)
{
    ulong x;
    ulong pc;

    pc = getcallerpc(&l);
    lockstats.locks++;

    x = arch_splhi();
```

```
    // no need to take care of up->nlock++ here, we have disabled interrupt
    // so no risk of getting scheduled
    if(arch_tas(&l->key) != 0){
        lockstats.glare++;
        /*
         * Cannot also check l->pc, l->cpu, or l->isilock here
         * because they might just not be set yet, or
         * (for pc and m) the lock might have just been unlocked.
         */
        for(;;){
            lockstats.inglare++;
            arch_splx(x);
            while(l->key)
                ;
            // let's try again
            x = arch_splhi();
            if(arch_tas(&l->key) == 0)
                goto acquire;
        }
    }
acquire:
    ⟨ilock() got the lock, update other fields 38a⟩
    ⟨lock ifdef LOCKCYCLES 515d⟩
}
```

⟨ilock() *got the lock, update other fields* 38a⟩≡ (37g) 45g ▷

```
if(up)
    up->lastilock = l;
l->isilock = true;
l->sr = x;
l->pc = pc;
l->cpu = CPUS(cpu->cpuno);
```

⟨*function* iunlock 38b⟩≡ (529b)

```
void
iunlock(Lock *l)
{
    ulong sr;

    ⟨iunlock ifdef LOCKCYCLES 516c⟩
    ⟨iunlock() sanity checks 38c⟩
    sr = l->sr;
    l->cpu = nil;
    l->key = 0;
    ⟨iunlock() after modified key, call coherence 404b⟩
    ⟨iunlock() decrement ilockdepth 46f⟩
    if(up)
        up->lastilock = nil;
    arch_splx(sr);
}
```

⟨iunlock() *sanity checks* 38c⟩≡ (38b)

```
if(l->key == 0)
    print("iunlock: not locked: pc %#p\n", getcallerpc(&l));
if(!l->isilock)
    print("iunlock of lock: pc %#p, held by %#lux\n", getcallerpc(&l), l->pc);
if(arch_islo())
    print("iunlock while lo: pc %#p, held by %#lux\n", getcallerpc(&l), l->pc);
```

⟨lock() *got the lock, update other fields* 38d⟩+≡ (35) ◁ 36e 45d ▷

```
l->isilock = false;
```

⟨canlock() *lock not held, update other fields* 39a⟩≡
```
l->isilock = false;
```

⟨unlock() *sanity checks* 39b⟩+≡ (36f) ◁37a 45f▷
```
if(l->isilock)
    print("unlock of ilock: pc %lux, held by %lux\n", getcallerpc(&l), l->pc);
```

4.7 Waiting QLock

⟨*struct* QLock 39c⟩≡ (528)
```
// Kernel basic lock with Queue (renamed to avoid ambiguity with libc.h QLock)
// a.k.a Mutex
struct KQLock
{
    bool  locked;   /* flag */

    // list<ref<Proc>> (next = Proc.qnext)
    Proc  *head;    /* next process waiting for object */
    // option<ref<Proc>> (direct access to tail, queue)
    Proc  *tail;    /* last process waiting for object */

    ⟨KQLock debugging fields 39d⟩

    Lock  use;    /* to access Qlock structure */
};
```

⟨KQLock *debugging fields* 39d⟩≡ (39c)
```
kern_addr qpc;    /* pc of the holder */ // for debugging?
```

⟨Proc *debugging fields* 39e⟩+≡ (29c) ◁37f
```
kern_addr qpc;    /* pc calling last blocking qlock */
```

⟨Proc *extra fields* 39f⟩≡ (29c) 74b▷
```
// list<ref<Proc>> KQlock.head or RWLock.head (or Procalloc.free)
Proc  *qnext;   /* next process on queue for a QLock */
```

⟨Procstate *cases* 39g⟩≡ (30c) 41d▷
```
Queueing, // see qlock()
```

⟨*function* qlock 39h⟩≡ (530a)
```
void
qlock(QLock *q)
{
    Proc *p;

    ⟨qlock() sanity checks 40b⟩
    lock(&q->use);
    ⟨qlock() start of qlock, stat 517e⟩
    if(!q->locked) {
        q->locked = true;
        q->qpc = getcallerpc(&q);
        unlock(&q->use);
        return;
    }
    // else
    ⟨qlock() sanity check up 40a⟩
    ⟨qlock() when qlock already held, stat 517f⟩
```

```
    // add_queue(q, up)
    p = q->tail;
    if(p == nil)
        q->head = up;
    else
        p->qnext = up;
    q->tail = up;
    // up->qnext could be non nil before? no otherwise that means
    // the process is already waiting for a lock and so had no occasion
    // to run another qlock() instruction.
    up->qnext = nil;

    up->state = Queueing;
    up->qpc = getcallerpc(&q);
    unlock(&q->use);

    // switch to another process!
    sched();
    // will resume here when another process unlock() the lock and ready() us
    q->qpc = getcallerpc(&q);
}
```

⟨qlock() *sanity check up* 40a⟩≡ (39h)

```
if(up == nil)
    panic("qlock");
```

⟨qlock() *sanity checks* 40b⟩≡ (39h) 46c ▷

```
if(q->use.key == 0x55555555) // dead code??
    panic("qlock: q %#p, key 5*\n", q);
```

⟨*function* qunlock 40c⟩≡ (530a)

```
void
qunlock(QLock *q)
{
    Proc *p;

    lock(&q->use);
    ⟨qunlock() sanity checks 40d⟩

    p = q->head;
    if(p){
        // dequeue(q)
        q->head = p->qnext;
        if(q->head == nil)
            q->tail = nil;

        unlock(&q->use);
        ready(p);
    }else{
        q->locked = false;
        q->qpc = nilptr;
        unlock(&q->use);
    }
}
```

⟨qunlock() *sanity checks* 40d⟩≡ (40c)

```
if(!q->locked)
    print("qunlock called with qlock not held, from %#p\n",
        getcallerpc(&q));
```

⟨*function* canqlock 41a⟩≡ (530a)

```
bool
canqlock(QLock *q)
{
    if(!canlock(&q->use))
        return false;
    if(q->locked){
        unlock(&q->use);
        return false;
    }
    // else

    q->locked = true;
    q->qpc = getcallerpc(&q);
    unlock(&q->use);
    return true;
}
```

4.8 Readers/Writer RWLock

⟨*struct* RWlock 41b⟩≡ (528)

```
struct RWlock
{
    int readers;  /* number of readers */
    bool writer;   /* have a writer? */

    // list<ref<Proc>> (next = Proc.qnext)
    Proc  *head;    /* list of waiting processes */
    // option<ref<Proc>> (direct access to tail, queue)
    Proc  *tail;
    // option<ref<Proc>>
    Proc  *wproc;   /* writing proc */

    ⟨RWlock debugging fields 41c⟩
    // Extra
    Lock  use;
};
```

⟨RWlock *debugging fields* 41c⟩≡ (41b)

```
kern_addr wpc;    /* pc of writer */
```

⟨Procstate *cases* 41d⟩+≡ (30c) ◁39g 42c▷

```
QueueingR, // see rlock()
```

⟨*function* rlock 41e⟩≡ (530a)

```
void
rlock(RWlock *q)
{
    Proc *p;

    lock(&q->use);
    rwstats.rlock++;
    if(!q->writer && q->head == nil){
        /* no writer, go for it */
        q->readers++;
        unlock(&q->use);
        return;
    }
```

```
    // else, a writer or a waiting queue??

    ⟨rlock() sanity check up 42a⟩
    rwstats.rlockq++;

    // add_queue(q, up)
    p = q->tail;
    if(p == nil)
        q->head = up;
    else
        p->qnext = up;
    q->tail = up;
    up->qnext = nil;

    up->state = QueueingR;
    unlock(&q->use);
    sched();
    // will resume here when another process unlock() the lock and ready() us
}
```

⟨rlock() *sanity check up* 42a⟩≡ (41e)

```
if(up == nil)
    panic("rlock");
```

⟨*function* runlock 42b⟩≡ (530a)

```
void
runlock(RWlock *q)
{
    Proc *p;

    lock(&q->use);
    p = q->head;
    if(--(q->readers) > 0 || p == nil){
        unlock(&q->use);
        return;
    }
    // else, p != nil && q->readers == 0

    /* start waiting writer */
    if(p->state != QueueingW)
        panic("runlock");

    // dequeue(q)
    q->head = p->qnext;
    if(q->head == nil)
        q->tail = nil;

    q->writer = true;
    unlock(&q->use);
    ready(p);
}
```

⟨Procstate *cases* 42c⟩+≡ (30c) ◁41d 75a▷

```
QueueingW, // see wlock()
```

⟨*function* wlock 42d⟩≡ (530a)

```
void
wlock(RWlock *q)
{
    Proc *p;
```

```
    lock(&q->use);
    rwstats.wlock++;
    if(q->readers == 0 && !q->writer){
        /* noone waiting, go for it */
        q->wpc = getcallerpc(&q);
        q->wproc = up;
        q->writer = true;
        unlock(&q->use);
        return;
    }
    // else, already a writer or readers

    /* wait */
    ⟨wlock() sanity check up 43a⟩
    rwstats.wlockq++;

    // add_queue(q, up)
    p = q->tail;
    if(p == nil)
        q->head = up;
    else
        p->qnext = up;
    q->tail = up;
    up->qnext = nil;

    up->state = QueueingW;
    unlock(&q->use);
    sched();
}
```

⟨wlock() *sanity check up* 43a⟩≡ (42d)

```
if(up == nil)
    panic("wlock");
```

⟨*function* wunlock 43b⟩≡ (530a)

```
void
wunlock(RWlock *q)
{
    Proc *p;

    lock(&q->use);
    p = q->head;
    if(p == nil){
        q->writer = false;
        unlock(&q->use);
        return;
    }
    if(p->state == QueueingW){
        /* start waiting writer */
        // dequeue(q)
        q->head = p->qnext;
        if(q->head == nil)
            q->tail = nil;

        unlock(&q->use);
        ready(p);
        return;
    }
```

```
    if(p->state != QueueingR)
        panic("wunlock");

    /* waken waiting readers */
    while(q->head != nil && q->head->state == QueueingR){
        p = q->head;
        q->head = p->qnext;
        q->readers++;
        ready(p);
    }

    if(q->head == nil)
        q->tail = nil;
    q->writer = false;
    unlock(&q->use);
}
```

⟨*function* canrlock 44a⟩≡ (530a)

```
/* same as rlock but punts if there are any writers waiting */
bool
canrlock(RWlock *q)
{
    lock(&q->use);
    rwstats.rlock++;
    if(q->writer == false && q->head == nil){
        /* no writer, go for it */
        q->readers++;
        unlock(&q->use);
        return true;
    }
    unlock(&q->use);
    return false;
}
```

4.9 Atomic References and Counter

⟨*struct* Ref 44b⟩≡ (528)

```
// For reference counting shared things (e.g. a Page)
struct Ref
{
    long ref;

    Lock;
};
```

⟨*function* incref 44c⟩≡ (530b)

```
long
incref(Ref *r)
{
    long x;

    lock(r);
    x = ++r->ref;
    unlock(r);
    return x;
}
```

⟨*function* decref 45a⟩≡ (530b)

```
long
decref(Ref *r)
{
    long x;

    lock(r);
    x = --r->ref;
    unlock(r);
    if(x < 0)
        panic("decref pc=%#p", getcallerpc(&r));
    return x;
}
```

⟨*struct* Counter 45b⟩≡ (528)

```
typedef struct Ref Counter;
```

4.10 Safeguards

4.10.1 Lock.p

⟨Lock *debugging fields* 45c⟩+≡ (35d) ◁36a 516a▷

```
// option<ref<Proc>>, None when key == 0
Proc  *p; // the process who did the locking should be the same unlocking
```

⟨lock() *got the lock, update other fields* 45d⟩+≡ (35) ◁38d

```
l->p = up;
```

⟨canlock() *lock was not already held, update other fields* 45e⟩+≡ (37b) ◁37c

```
l->p = up;
```

⟨unlock() *sanity checks* 45f⟩+≡ (36f) ◁39b

```
if(l->p != up)
    print("unlock: up changed: pc %#p, acquired at pc %lux, lock p %#p, unlock up %#p\n", getcallerpc(&l), l->p
```

⟨ilock() *got the lock, update other fields* 45g⟩+≡ (37g) ◁38a 46e▷

```
l->p = up;
```

4.10.2 Proc.nlocks

⟨Proc *synchronization fields* 45h⟩≡ (29c) 156f▷

```
// As long as the current process hold spinlocks (to kernel data structures),
// we will not schedule another process in sched() or unlock();
// only the last unlock will eventually cause a rescheduling.
Ref nlocks;   /* number of locks held by proc */
```

⟨lock() *increment nlocks* 45i⟩≡ (37b 35)

```
if(up)
    inccnt(&up->nlocks);    /* prevent being scheded */
```

⟨lock() *decrement nlocks* 45j⟩≡ (37b 35)

```
if(up)
    deccnt(&up->nlocks);
```

⟨*function* inccnt 46a⟩≡ (529b)

```
// See also ref.c incref() and decref(), but we can't use them here as they
// themselves rely on lock() and unlock().
static void
inccnt(Ref *r)
{
    arch_xinc(&r->ref);
}
```

⟨*function* deccnt 46b⟩≡ (529b)

```
static int
deccnt(Ref *r)
{
    int x;

    x = arch_xdec(&r->ref);
    if(x < 0)
        panic("deccnt pc=%#p", getcallerpc(&r));
    return x;
}
```

⟨qlock() *sanity checks* 46c⟩+≡ (39h) ◁40b 46g▷

```
if(up != nil && up->nlocks.ref)
    print("qlock: %#p: nlocks %lud\n", getcallerpc(&q), up->nlocks.ref);
```

4.10.3 Cpu.ilockdepth

⟨Cpu *other fields* 46d⟩≡ (28a) 149f▷

```
int ilockdepth;
```

⟨ilock() *got the lock, update other fields* 46e⟩+≡ (37g) ◁45g

```
cpu->ilockdepth++;
```

⟨iunlock() *decrement ilockdepth* 46f⟩≡ (38b)

```
cpu->ilockdepth--;
```

⟨qlock() *sanity checks* 46g⟩+≡ (39h) ◁46c

```
if(cpu->ilockdepth != 0)
    print("qlock: %#p: ilockdepth %d\n", getcallerpc(&q), cpu->ilockdepth);
```

4.11 Synchronization

4.11.1 Rendez-vous

4.11.2 Producer/Consumer Queue

4.11.3 Semaphore

Chapter 5

Memory

⟨systab *memory syscalls* 47a⟩≡ (26c)

```
[BRK]       sysbrk,
```

5.1 Overview

5.2 Long-lived structure allocator: xalloc()

5.2.1 Xalloc

⟨*struct* Hole 47b⟩≡ (582)

```
struct Hole
{
    // between addr and top the memory is free, this is the "hole"
    phys_addr addr;
    phys_addr top;
    ulong size; // top - addr (in bytes)

    // Extra
    ⟨Hole extra fields 47d⟩
};
```

⟨*struct* Xalloc 47c⟩≡ (582)

```
// Long lived data structure allocator (singleton)
struct Xalloc
{
    // array<Hole> where each Hole is linked to another hole
    Hole  hole[Nhole];

    // list<ref<Hole>> (next = Hole.next) list of free hole entries (addr=top=size=0)
    Hole* unused_slots;

    // list<ref<Hole>> (next = Hole.next) memory holes, sorted by their top addr
    Hole* sorted_holes;

    // extra
    Lock;
};
```

⟨Hole *extra fields* 47d⟩≡ (47b)

```
Hole* next; // list<ref<Hole>> of Xalloc.sorted_holes or Xalloc.unused_slots
```

⟨*constants holes* 48a⟩≡ (582)

```
enum
{
    Nhole   = 128,
    Magichole = 0x484F4C45,     /* HOLE */
};
```

5.2.2 xlists

⟨*global* xlists 48b⟩≡ (584b)

```
static Xalloc   xlists;
```

5.2.3 Xhdr

⟨*struct* Xhdr 48c⟩≡ (582)

```
// What is the connection with Hole? A Hole that get used will gets
// its top and size decremented, and this newly allocated part will describe
// a portion of used memory, and at this memory there will be a header
// and then just after the actual memory xalloc'ed by someone
struct Xhdr
{
    // bookkeeping area
    ulong size;
    ulong magix;

    char  data[]; // memory pointer returned by xalloc
};
```

5.2.4 xalloc()

⟨*function* xalloc 48d⟩≡ (584b)

```
kern_vp
xalloc(ulong size)
{
    return xallocz(size, true);
}
```

⟨*function* xallocz 48e⟩≡ (584b)

```
kern_vp
xallocz(ulong size, bool zero)
{
    Xhdr *p;
    Hole *h, **l;

    /* add room for magix & size overhead, round up to nearest vlong */
    size += BY2V + offsetof(Xhdr, data[0]);
    size &= ~(BY2V-1);

    ilock(&xlists);
    l = &xlists.sorted_holes;
    for(h = *l; h; h = h->next) {
        // found an appropriate hole
        if(h->size >= size) {
            p = (Xhdr*)KADDR(h->addr);
            h->addr += size; // shrink towards top
            h->size -= size;
```

```
            // This hole is now fully used (which is rare because one
            // rarely does an xalloc with the remaining size of a hole).
            // We can put it back in the list of free hole entries.
            if(h->size == 0) {
                *l = h->next;
                h->next = xlists.unused_slots;
                xlists.unused_slots = h;
            }

            iunlock(&xlists);
            if(zero)
                memset(p, 0, size);
            p->magix = Magichole;
            p->size = size;
            return p->data;
        }
        l = &h->next;
    }
    iunlock(&xlists);
    return nil; // panic?
}
```

⟨*function* xfree 49a⟩≡ (584b)

```
void
xfree(kern_vp p)
{
    Xhdr *x;

    x = (Xhdr*)((ulong)p - offsetof(Xhdr, data[0]));
    ⟨xfree() sanity check magix mark 49b⟩
    xhole(PADDR((kern_addr)x), x->size);
}
```

⟨xfree() *sanity check magix mark* 49b⟩≡ (49a)

```
if(x->magix != Magichole) {
    xsummary();
    panic("xfree(%#p) %#ux != %#lux", p, Magichole, x->magix);
}
```

5.2.5 xhole()

⟨*function* xhole 49c⟩≡ (584b)

```
void
xhole(phys_addr addr, ulong size)
{
    phys_addr top;
    Hole *h, *c, **l;

    if(size == 0)
        return;

    top = addr + size;
    ilock(&xlists);
    l = &xlists.sorted_holes;
    for(h = *l; h; h = h->next) {
        if(h->top == addr) {
            h->size += size;
            h->top = h->addr+h->size;
            c = h->next;
```

```
            if(c && h->top == c->addr) {
                h->top += c->size;
                h->size += c->size;
                h->next = c->next;
                c->next = xlists.unused_slots;
                xlists.unused_slots = c;
            }
            iunlock(&xlists);
            return;
        }
        if(h->addr > addr)
            break;
        l = &h->next;
    }
    if(h && top == h->addr) {
        h->addr -= size;
        h->size += size;
        iunlock(&xlists);
        return;
    }

    if(xlists.unused_slots == nil) {
        iunlock(&xlists);
        print("xfree: no free holes, leaked %lud bytes\n", size);
        return;
    }

    h = xlists.unused_slots;
    xlists.unused_slots = h->next;
    h->addr = addr;
    h->top = top;
    h->size = size;
    h->next = *l;
    *l = h;
    iunlock(&xlists);
}
```

5.2.6 xmerge()

⟨*function* xmerge 50a⟩≡ (584b)

```
bool
xmerge(kern_vp vp, kern_vp vq)
{
    Xhdr *p, *q;

    p = (Xhdr*)(((ulong)vp - offsetof(Xhdr, data[0])));
    q = (Xhdr*)(((ulong)vq - offsetof(Xhdr, data[0])));

    ⟨xmerge() sanity check magix 50b⟩
    if((byte*)p+p->size == (byte*)q) {
        p->size += q->size;
        return true;
    }
    return false;
}
```

⟨xmerge() *sanity check magix* 50b⟩≡ (50a)

```
if(p->magix != Magichole || q->magix != Magichole) {
    ⟨xmerge() debug info when not magichole ??⟩
```

```
    panic("xmerge(%#p, %#p) bad magic %#lux, %#lux",
        vp, vq, p->magix, q->magix);
}
```

⟨*function* xspanalloc 51a⟩≡ (584b)

```
kern_vp
xspanalloc(ulong size, int align, ulong span)
{
    ulong a, v, t;
    a = (kern_addr)xalloc(size+align+span);
    ⟨xspanalloc() sanity check a 51b⟩

    if(span > 2) {
        v = (a + span) & ~(span-1);
        t = v - a;
        if(t > 0)
            xhole(PADDR(a), t);
        t = a + span - v;
        if(t > 0)
            xhole(PADDR(v+size+align), t);
    }
    else
        v = a;

    if(align > 1)
        v = (v + align) & ~(align-1);

    return (kern_vp)v;
}
```

⟨xspanalloc() *sanity check a* 51b⟩≡ (51a)

```
if(a == nilptr)
    panic("xspanalloc: %lud %d %lux", size, align, span);
```

5.3 Memory pools

5.3.1 Pool

```
// from pool.h
//struct Pool {
//  char* name;
//  ulong maxsize;
//
//  ulong cursize;
//  ulong curfree;
//  ulong curalloc;
//
//  ulong minarena; /* smallest size of new arena */
//  ulong quantum;  /* allocated blocks should be multiple of */
//  ulong minblock; /* smallest newly allocated block */
//
//  void* freeroot; /* actually Free* */
//  void* arenalist;  /* actually Arena* */
//
//  void* (*alloc)(ulong);
```

```
//  int (*merge)(void*, void*);
//  void  (*move)(void* from, void* to);
//
//  int flags;
//  int nfree;
//  int lastcompact;
//
//  void  (*lock)(Pool*);
//  void  (*unlock)(Pool*);
//  void  (*print)(Pool*, char*, ...);
//  void  (*panic)(Pool*, char*, ...);
//  void  (*logstack)(Pool*);
//
//  void* private;
//};
```

⟨*pool.c struct Private* 52a⟩≡ (585b)

```
struct Private {
    Lock        lk;
    char        msg[256]; /* a rock for messages to be printed at unlock */
};
```

5.3.2 plock()

⟨*function* plock 52b⟩≡ (585b)

```
static void
plock(Pool *p)
{
    Private *pv;

    pv = p->private;
    ilock(&pv->lk);
    pv->lk.pc = getcallerpc(&p);
    pv->msg[0] = 0;
}
```

⟨*function* punlock 52c⟩≡ (585b)

```
static void
punlock(Pool *p)
{
    Private *pv;
    char msg[sizeof pv->msg];

    pv = p->private;
    if(pv->msg[0] == 0){
        iunlock(&pv->lk);
        return;
    }

    memmove(msg, pv->msg, sizeof msg);
    iunlock(&pv->lk);
    iprint("%.*s", sizeof pv->msg, msg);
}
```

5.3.3 pmainmem()

⟨*global* pmainmem 53a⟩≡ (585b)

```
static Private pmainpriv;
static Pool pmainmem = {
    .name=  "Main",
    .maxsize=   4*MB,
    .minarena=  128*KB,
    .quantum=   32,

    .alloc= xalloc,
    .merge= xmerge,

    .flags= POOL_TOLERANCE,

    .lock=plock,
    .unlock= punlock,

    .print= poolprint,
    .panic= ppanic,

    .private=   &pmainpriv,
};
```

⟨*global* pimagmem 53b⟩≡ (585b)

```
static Private pimagpriv;
static Pool pimagmem = {
    .name=  "Image",
    .maxsize=   16*MB,
    .minarena=  2*MB,
    .quantum=   32,

    .alloc= xalloc,
    .merge= xmerge,

    .flags= 0,

    .lock= plock,
    .unlock= punlock,

    .print= poolprint,
    .panic= ppanic,

    .private=   &pimagpriv,
};
```

⟨*global mainmem and imagmem* 53c⟩≡ (585b)

```
// exported in include/pool.h, defined here!
Pool*   mainmem = &pmainmem;
Pool*   imagmem = &pimagmem;
```

5.3.4 poolprint()

⟨*function* poolprint 53d⟩≡ (585b)

```
/*
 * because we can't print while we're holding the locks,
 * we have the save the message and print it once we let go.
 */
static void
```

```
poolprint(Pool *p, char *fmt, ...)
{
    va_list v;
    Private *pv;

    pv = p->private;
    va_start(v, fmt);
    vseprint(pv->msg+strlen(pv->msg), pv->msg+sizeof pv->msg, fmt, v);
    va_end(v);
}
```

⟨*function* ppanic 54a⟩≡ (585b)

```
static void
ppanic(Pool *p, char *fmt, ...)
{
    va_list v;
    Private *pv;
    char msg[sizeof pv->msg];

    pv = p->private;
    va_start(v, fmt);
    vseprint(pv->msg+strlen(pv->msg), pv->msg+sizeof pv->msg, fmt, v);
    va_end(v);
    memmove(msg, pv->msg, sizeof msg);
    iunlock(&pv->lk);
    panic("%s", msg);
}
```

5.4 Kernel malloc()

5.4.1 malloc()

⟨*function* malloc 54b⟩≡ (585c)

```
kern_vp
malloc(ulong size)
{
    kern_vp v;

    v = poolalloc(mainmem, size + Npadlong*sizeof(ulong));
    if(v == nil)
        return nil;
    ⟨malloc() if Npadlong 56b⟩
    memset(v, 0, size);
    return v;
}
```

⟨*function* free 54c⟩≡ (585c)

```
void
free(kern_vp v)
{
    if(v != nil)
        poolfree(mainmem, (ulong*)v-Npadlong);
}
```

5.4.2 Sleeping smalloc()

⟨*function* smalloc 55a⟩≡ (585c)

```
// non failing malloc! will repeat until it can
kern_vp
smalloc(ulong size)
{
    kern_vp v;

    for(;;) {
        v = poolalloc(mainmem, size + Npadlong*sizeof(ulong));
        if(v != nil)
            break;
        // else
        tsleep(&up->sleepr, returnfalse, 0, 100);
    }
    ⟨smalloc() if Npadlong 56e⟩
    memset(v, 0, size); // clear
    return v;
}
```

⟨*function* returnfalse 55b⟩≡ (535a)

```
// usually used as default callback for sleep/tsleep
bool
returnfalse(void*)
{
    return false;
}
```

5.4.3 mallocxxx()

⟨*function* mallocz 55c⟩≡ (585c)

```
kern_vp
mallocz(ulong size, bool clr)
{
    kern_vp v;

    v = poolalloc(mainmem, size+Npadlong*sizeof(ulong));
    ⟨mallocz() if Npadlong 56f⟩
    if(clr && v != nil)
        memset(v, 0, size);
    return v;
}
```

⟨*function* mallocalign 55d⟩≡ (585c)

```
kern_vp
mallocalign(ulong size, ulong align, long offset, ulong span)
{
    kern_vp v;

    v = poolallocalign(mainmem, size  +Npadlong*sizeof(ulong), align,
                               offset-Npadlong*sizeof(ulong), span);
    ⟨mallocalign() if Npadlong 57a⟩
    if(v)
        memset(v, 0, size);
    return v;
}
```

⟨*function* realloc 56a⟩≡ (585c)

```
kern_vp
realloc(kern_vp v, ulong size)
{
    kern_vp nv;

    if(v != nil)
        v = (ulong*)v - Npadlong;
    ⟨realloc() if Npadlong 57b⟩

    if(nv = poolrealloc(mainmem, v, size)){
       ⟨realloc() readjust nv if Npadlong 57c⟩
    }
    return nv;
}
```

5.4.4 malloc tags

⟨malloc() *if Npadlong* 56b⟩≡ (54b)

```
if(Npadlong){
    v = (ulong*)v+Npadlong;
    setmalloctag(v, getcallerpc(&size));
    setrealloctag(v, 0);
}
```

⟨*function* setmalloctag 56c⟩≡ (585c)

```
void
setmalloctag(kern_vp v, kern_addr pc)
{
    kern_wp u;
    USED(v, pc); //??
    if(Npadlong <= MallocOffset || v == nil)
        return;
    u = v;
    u[-Npadlong+MallocOffset] = pc;
}
```

⟨*function* setrealloctag 56d⟩≡ (585c)

```
void
setrealloctag(kern_vp v, kern_addr pc)
{
    kern_wp u;
    USED(v, pc);
    if(Npadlong <= ReallocOffset || v == nil)
        return;
    u = v;
    u[-Npadlong+ReallocOffset] = pc;
}
```

⟨smalloc() *if Npadlong* 56e⟩≡ (55a)

```
if(Npadlong){
    v = (ulong*)v+Npadlong;
    setmalloctag(v, getcallerpc(&size));
}
```

⟨mallocz() *if Npadlong* 56f⟩≡ (55c)

```
if(Npadlong && v != nil){
    v = (ulong*)v+Npadlong;
    setmalloctag(v, getcallerpc(&size));
    setrealloctag(v, 0);
}
```

⟨mallocalign() *if Npadlong* 57a⟩≡ (55d)

```
if(Npadlong && v != nil){
    v = (ulong*)v+Npadlong;
    setmalloctag(v, getcallerpc(&size));
    setrealloctag(v, 0);
}
```

⟨realloc() *if Npadlong* 57b⟩≡ (56a)

```
if(Npadlong !=0 && size != 0)
    size += Npadlong*sizeof(ulong);
```

⟨realloc() *readjust nv if Npadlong* 57c⟩≡ (56a)

```
 nv = (ulong*)nv+Npadlong;
 setrealloctag(nv, getcallerpc(&v));
 if(v == nil)
     setmalloctag(nv, getcallerpc(&v));
```

5.5 User malloc()

5.6 Page and palloc

5.6.1 Page

⟨*struct* Page 57d⟩≡ (582)

```
// Page metadata. We will allocate as many Page as to cover all physical memory
// available for the user. xalloc'ed in Palloc.pages
struct Page
{
    phys_addr pa;     /* Physical address in memory */
    user_addr va;     /* Virtual address for user */

    // Why not Ref? to save space (same reason for using 'char' below)
    // but that means needs to use Lock below to access this non-atomic ref.
    ushort  ref;      /* Reference count */ // Pages are shared!

    // bitset<enum<Modref>>
    char  modref;     /* Simulated modify/reference bits */

    ⟨Page other fields 121g⟩

    // extra
    Lock;
    ⟨Page extra fields 58e⟩
};
```

⟨*enum* modref 57e⟩≡ (582)

```
enum Modref
{
    PG_NOTHING = 0x00, // nothing

    PG_MOD    = 0x01,   /* software modified bit */
    PG_REF    = 0x02,   /* software referenced bit */
};
```

5.6.2 Palloc

⟨*struct* Palloc 58a⟩≡ (582)

```
// Page Allocator (singleton)
struct Palloc
{
    // array<Page>, xalloc'ed in pageinit(), huge, cover physical user space
    Page  *pages; /* array of all pages */

    // list<ref<Page>> (next = Page.next), list of free pages
    Page  *head;      /* most recently used */
    // list<ref<Page>> (prev = Page.prev), list of free pages (backward)
    Page  *tail;      /* least recently used */

    ulong freecount;    /* how many pages on free list now */

    ⟨Palloc capacity fields 58c⟩
    ⟨Palloc other fields 122b⟩

    // extra
    Lock; // LOCK ORDERING: always do lock(&palloc); lock(&page)!!
    ⟨Palloc extra fields 139a⟩
};
```

⟨*global* palloc 58b⟩≡ (584a)

```
// Page allocator
struct Palloc palloc;
```

⟨Palloc *capacity fields* 58c⟩≡ (58a)

```
Pallocmem mem[4]; // = Conf.mem minus memory allocated for the kernel
// sum of mem.npage (which should be conf.upages)
ulong user;     /* how many user pages */
```

⟨*struct* Pallocmem 58d⟩≡ (582)

```
// memory banks for user memory, similar to Confmem (and RMap)
struct Pallocmem
{
    phys_addr base;
    ulong npage;
};
```

⟨Page *extra fields* 58e⟩≡ (57d) 122c ▷

```
// list<ref<Page>> Palloc.head, or Proc.mmuused or Proc.mmufree, or in mfreeseg
Page  *next; /* Lru free list */
// list<ref<Page>> Palloc.tail
Page  *prev;
```

5.6.3 newpage()

⟨*constructor* newpage 58f⟩≡ (588)

```
Page*
newpage(bool clear, Segment **s, user_addr va)
{
    Page *p;
    ⟨newpage() other locals 59a⟩

    lock(&palloc);

    ⟨newpage() loop waiting freecount ¿ highwater 138b⟩
```

```
    //when no color: p = palloc.head;
    ⟨newpage() find free page p, set color and ct 423i⟩

    pageunchain(p);

    lock(p);
    if(p->ref != 0)
        panic("newpage: p->ref %d != 0", p->ref);

    ⟨newpage() uncachepage 123b⟩
    p->ref = 1;
    p->va = va;
    p->modref = PG_NOTHING;
    ⟨newpage() set cachectl using ct 423k⟩

    unlock(p);
    unlock(&palloc);
    ⟨newpage() if clear 59b⟩
    return p;
}
```

⟨newpage() other locals 59a⟩≡ (58f) 138a ▷

```
Arch_KMap *k;
```

⟨newpage() if clear 59b⟩≡ (58f)

```
if(clear) {
    k = arch_kmap(p);
    memset((void*)VA(k), 0, BY2PG);
    arch_kunmap(k);
}
```

⟨function pageunchain 59c⟩≡ (588)

```
// assumes palloc is held
static void
pageunchain(Page *p)
{
    if(canlock(&palloc))
        panic("pageunchain (palloc %p)", &palloc);

    // remove_queue(p, palloc);
    if(p->prev)
        p->prev->next = p->next;
    else
        palloc.head = p->next;
    if(p->next)
        p->next->prev = p->prev;
    else
        palloc.tail = p->prev;
    p->prev = p->next = nil;

    palloc.freecount--;
}
```

⟨destructor putpage 59d⟩≡ (588)

```
void
putpage(PageOrSwap *p)
{
    ⟨putpage if p is a swap address 135e⟩
```

```
    lock(&palloc);
    lock(p);

    if(p->ref == 0)
        panic("putpage");

    if(--p->ref > 0) {
        unlock(p);
        unlock(&palloc);
        return;
    }
    // else
    ⟨putpage if p has an image 123a⟩
    else
        pagechainhead(p);

    ⟨putpage wakeup process waiting for freepage if any 139b⟩
    unlock(p);
    unlock(&palloc);
}
```

⟨*function* pagechainhead 60a⟩≡ (588)

```
// assumes palloc is held
void
pagechainhead(Page *p)
{
    if(canlock(&palloc))
        panic("pagechainhead");

    // add_head(p, palloc)
    if(palloc.head) {
        p->next = palloc.head;
        palloc.head->prev = p;
    }
    else {
        palloc.tail = p;
        p->next = nil;
    }
    palloc.head = p;
    p->prev = nil;

    palloc.freecount++;
}
```

⟨*function* pagechaintail 60b⟩≡ (588)

```
// assumes palloc is held
void
pagechaintail(Page *p)
{
    if(canlock(&palloc))
        panic("pagechaintail");

    // add_tail(p, palloc)
    if(palloc.tail) {
        p->prev = palloc.tail;
        palloc.tail->next = p;
    }
    else {
        palloc.head = p;
        p->prev = nil;
```

```
    }
    palloc.tail = p;
    p->next = nil;

    palloc.freecount++;
}
```

5.7 Segment and Pagetable

⟨*enum* segtype 61a⟩≡ (582)

```
/* Segment types */
enum Segtype
{
    SG_TEXT   = 00,
    SG_DATA   = 01,
    SG_BSS    = 02,
    SG_STACK  = 03,
    ⟨Segtype other cases 424l⟩
    SG_TYPE   = 07,   /* Mask type of segment */

    ⟨Segtype other flags 66c⟩
};
```

⟨*global* sname 61b⟩≡ (567)

```
// hash<enum<Segtype, string>>
static char *sname[]={
    [SG_TEXT]     = "Text",
    [SG_DATA]     = "Data",
    [SG_BSS]      = "Bss",
    [SG_STACK]    = "Stack",

    [SG_SHARED]   = "Shared",
    [SG_PHYSICAL] = "Phys"
};
```

5.7.1 Segment

⟨*struct* Segment 61c⟩≡ (582)

```
// smalloc'ed by newseg()
struct Segment
{
    // enum<Segtype>
    ushort  type;   /* segment type */

    user_addr base;   /* virtual base */
    user_addr top;    /* virtual top */
    ulong size;   /* size in pages */ // top - base / BY2PG

    // array<option<ref_own<Pagetable>>>, smalloc'ed (or smallpagedir alias)
    Pagetable **pagedir; // array of PAGEDIRSIZE max
    int pagedirsize; // nelem(pagedir)

    ⟨Segment other fields 64c⟩

    // extra
    Ref; // LOCK ORDERING: always do lock(img); lock(s) ??
    QLock lk; // FOR WHAT???
};
```

⟨*constant* PAGEDIRSIZE*(arm)* 62a⟩≡ (633b)

```
#define PAGEDIRSIZE 1984
```

5.7.2 Pagetable

⟨*struct* Pagetable 62b⟩≡ (582)

```
// ptalloc'ed (malloc'ed)
struct Pagetable
{
    //array<option<ref<PageOrSwap>> will map 1M of memory
    PageOrSwap  *pagetab[PAGETABSIZE];

    //to avoid iterate over all entries in pagetab
    // ref<ref<Page>> in Pagetable.pages
    Page  **first;    /* First used entry */
    // ref<ref<Page>> in Pagetable.pages
    Page  **last;     /* Last used entry */
};
```

⟨*constant* PAGETABMAPMEM*(arm)* 62c⟩≡ (633b)

```
#define PAGETABMAPMEM   (1024*1024) // 1MB
```

⟨*constant* PAGETABSIZE*(arm)* 62d⟩≡ (633b)

```
#define PAGETABSIZE 256 // (PAGETABMAPMEM/BY2PG)
```

⟨*constructor* ptalloc 62e⟩≡ (588)

```
Pagetable*
ptalloc(void)
{
    Pagetable *new;

    new = smalloc(sizeof(Pagetable));
    new->first = &new->pagetab[PAGETABSIZE];
    new->last = new->pagetab;
    return new;
}
```

⟨*destructor* freept 62f⟩≡ (588)

```
void
freept(Segment *s, Pagetable *p)
{
    PageOrSwap **pte;
    ⟨freept() other locals 425e⟩

    switch(s->type&SG_TYPE) {
    ⟨freept() SG_PHYSICAL case 425f⟩
    default:
        for(pte = p->first; pte <= p->last; pte++)
            if(*pte) {
                putpage(*pte);
                *pte = nil;
            }
    }
    free(p);
}
```

⟨*function* ptcpy 63a⟩≡ (588)

```
Pagetable*
ptcpy(Pagetable *old)
{
    Pagetable *new;
    PageOrSwap **src, **dst;

    new = ptalloc();
    dst = &new->pagetab[old->first - old->pagetab];
    new->first = dst;
    for(src = old->first; src <= old->last; src++, dst++)
        if(*src) {
            ⟨ptcpy() if src is a swap page 136b⟩
            else {
                lock(*src);
                (*src)->ref++;
                unlock(*src);
            }
            new->last = dst;
            *dst = *src;
        }
    return new;
}
```

5.7.3 newseg()

⟨*constructor* newseg 63b⟩≡ (589)

```
Segment *
newseg(int type, user_addr base, ulong size)
{
    Segment *s;
    int pagedirsize;

    ⟨newseg() sanity check size 63c⟩
    s = smalloc(sizeof(Segment));

    s->ref = 1; // Segment.Ref.ref
    s->type = type;
    s->base = base;
    s->top = base+(size*BY2PG);
    s->size = size;

    ⟨newseg() sema initialization 355c⟩

    pagedirsize = ROUND(size, PAGETABSIZE)/PAGETABSIZE;

    ⟨newseg() if pagedirsize small 64e⟩
    else{
        pagedirsize *= 2; // room for growing
        if(pagedirsize > PAGEDIRSIZE)
            pagedirsize = PAGEDIRSIZE; // pad's first bugfix :)
        s->pagedir = smalloc(pagedirsize * sizeof(Pagetable*));
        s->pagedirsize = pagedirsize;
    }
    return s;
}
```

⟨newseg() *sanity check size* 63c⟩≡ (63b)

```
if(size > (PAGEDIRSIZE*PAGETABSIZE))
```

```
    error(Enovmem);
```

⟨*destructor* putseg 64a⟩≡ (589)

```
void
putseg(Segment *s)
{
    Pagetable **pde, **emap;
    ⟨putseg() other locals 131a⟩

    ⟨putseg() sanity check s 64b⟩
    ⟨putseg() if s has an image 131b⟩
    else
        lock(s);

    s->ref--;
    if(s->ref != 0) {
        unlock(s);
        return;
    }
    // else
    unlock(s);

    qlock(&s->lk);
    ⟨putseg() if s had an image 131c⟩

    emap = &s->pagedir[s->pagedirsize];
    for(pde = s->pagedir; pde < emap; pde++)
        if(*pde)
            freept(s, *pde);

    qunlock(&s->lk);

    ⟨putseg() if smallpagedir 65a⟩
    else
        free(s->pagedir);
    ⟨putseg() free profile 381i⟩
    free(s);
}
```

⟨putseg() *sanity check s* 64b⟩≡ (64a)

```
if(s == nil)
    return; // TODO: panic("putseg") instead?
```

5.7.4 smallpagedir

⟨Segment *other fields* 64c⟩≡ (61c) 125a ▷

```
// array<option<ref_own<Pagetable>>
Pagetable *smallpagedir[SMALLPAGEDIRSIZE];
```

⟨*constant* SMALLPAGEDIRSIZE*(arm)* 64d⟩≡ (633b)

```
#define SMALLPAGEDIRSIZE    16
```

⟨newseg() *if pagedirsize small* 64e⟩≡ (63b)

```
if(pagedirsize <= nelem(s->smallpagedir)){
    s->pagedir = s->smallpagedir;
    s->pagedirsize = nelem(s->smallpagedir);
}
```

⟨putseg() *if smallpagedir* 65a⟩≡ (64a)

```
if(s->pagedir == s->smallpagedir) {
}
```

5.7.5 Adding a page to a segment: segpage()

⟨*function* segpage 65b⟩≡ (589)

```
void
segpage(Segment *s, Page *p)
{
    Pagetable **pt;
    ulong off;
    Page **pg;

    ⟨segpage() sanity check page in range of segment 65c⟩

    off = p->va - s->base;
    pt = &s->pagedir[off/PAGETABMAPMEM]; // PDX
    if(*pt == nil)
        *pt = ptalloc();

    pg = &(*pt)->pagetab[(off&(PAGETABMAPMEM-1))/BY2PG]; // PTX
    *pg = p;

    if(pg < (*pt)->first)
        (*pt)->first = pg;
    if(pg > (*pt)->last)
        (*pt)->last = pg;
}
```

⟨segpage() *sanity check page in range of segment* 65c⟩≡ (65b)

```
if(p->va < s->base || p->va >= s->top)
    panic("segpage");
```

5.7.6 Process segments

Procseg

⟨*enum* procseg 65d⟩≡ (592c)

```
/*
 *  process memory segments - NSEG always last !
 */
//coupling: must start with SSEG, see sysexec loop
enum Procseg
{
    SSEG, TSEG, DSEG, BSEG, // Stack, Text, Data, Bss

    ESEG, // E = Extra (used for temporary stack segment),
    _SEG0, _SEG1, _SEG2, _SEG3, _SEG4, // free slots for for segattach

    NSEG // to count, see Proc.seg array
};
```

⟨Proc *memory fields* 65e⟩≡ (29c) 116b ▷

```
// hash<enum<procseg>, option<ref_own<Segment>>>, elt smalloc'ed? ref_counted?
Segment *seg[NSEG];
QLock seglock;  /* locked whenever seg[] changes */
```

seg()

⟨*function* seg 66a⟩≡ (587b)

```
Segment*
seg(Proc *p, user_addr addr, bool dolock)
{
    Segment **s, **et, *sg;

    et = &p->seg[NSEG];
    for(s = p->seg; s < et; s++) {
        sg = *s;
        if(sg == nil)
            continue;
        if(addr >= sg->base && addr < sg->top) {
            if(!dolock)
                return sg;
            // else

            qlock(&sg->lk);
            // can have a race, need to check again
            if(addr >= sg->base && addr < sg->top)
                return sg;
            qunlock(&sg->lk);
        }
    }
    return nil;
}
```

Addresses covering multiple segments, validaddr()

⟨*function* okaddr 66b⟩≡ (587b)

```
/*
 * Called only in a system call
 */
bool
okaddr(user_addr addr, ulong len, bool write)
{
    Segment *s;

    if((long)len >= 0) {
        for(;;) {
            s = seg(up, addr, false);
            if(s == nil || (write && (s->type&SG_RONLY)))
                break;

            if(addr+len > s->top) {
                len -= s->top - addr;
                addr = s->top;
            }else{
                return true;
            }
        }
    }
    pprint("suicide: invalid address %#lux/%lud in sys call pc=%#lux\n", addr, len, arch_userpc());
    return false;
}
```

⟨Segtype *other flags* 66c⟩≡ (61a) 349b ▷

```
SG_RONLY  = 0040,   /* Segment is read only */
```

⟨*function* validaddr 67a⟩≡ (587b)

```
void
validaddr(user_addr addr, ulong len, bool write)
{
    if(!okaddr(addr, len, write)){
        postnote(up, 1, "sys: bad address in syscall", NDebug);
        error(Ebadarg);
    }
}
```

⟨*function* vmemchr 67b⟩≡ (587b)

```
/*
 * &s[0] is known to be a valid address.
 */
void*
vmemchr(user_vp s, int c, int n)
{
    int m;
    user_addr a;
    user_vp t;

    a = (user_addr)s;
    while(PGROUND(a) != PGROUND(a+n-1)){
        /* spans pages; handle this page */
        m = BY2PG - (a & (BY2PG-1));
        t = memchr((void*)a, c, m);
        if(t)
            return t;
        a += m;
        n -= m;
        if(a < KZERO)
            validaddr(a, 1, false);
    }

    /* fits in one page */
    return memchr((void*)a, c, n);
}
```

⟨*portdat.h macros* 67c⟩+≡ (? 0—1) ◁23i

```
// BY2PG is defined in mem.h, which should always be included before "dat.h"!
#define PGROUND(s)  ROUNDUP(s, BY2PG)
```

dupseg()

⟨*function* dupseg 67d⟩≡ (589)

```
Segment*
dupseg(Segment **seg, int segno, bool share)
{
    Segment *s;
    Segment *n; // when allocate a new segment
    ⟨dupseg() other locals 68d⟩

    SET(n);

    s = seg[segno];

    qlock(&s->lk);
    if(waserror()){
        qunlock(&s->lk);
```

```
        nexterror();
    }
    switch(s->type&SG_TYPE) {
    ⟨dupseg() switch segment type cases 68a⟩
    }
    // not sameseg, we have allocated a new seg in n above
    ⟨dupseg() when not goto sameseg, when allocated a new segment 68e⟩

    poperror();
    qunlock(&s->lk);
    return n;

sameseg:
    ⟨dupseg() in sameseg, when share a segment 68b⟩

    poperror();
    qunlock(&s->lk);
    return s;
}
```

⟨dupseg() *switch segment type cases* 68a⟩≡ (67d) 68c ▷

```
case SG_TEXT:       /* New segment shares pt set */
    goto sameseg;
```

⟨dupseg() *in sameseg, when share a segment* 68b⟩≡ (67d)

```
incref(s);
```

⟨dupseg() *switch segment type cases* 68c⟩+≡ (67d) ◁ 68a 68f ▷

```
case SG_STACK:
    n = newseg(s->type, s->base, s->size);
    break;
```

⟨dupseg() *other locals* 68d⟩≡ (67d)

```
int i;
int size; // pagedir
Pagetable *pt;
```

⟨dupseg() *when not goto sameseg, when allocated a new segment* 68e⟩≡ (67d)

```
size = s->pagedirsize;
for(i = 0; i < size; i++)
    if(pt = s->pagedir[i])
        n->pagedir[i] = ptcpy(pt); // will actually share the pages

⟨dupseg() copy other fields 424h⟩
⟨dupseg() if original segment was shared, flush it 353b⟩
```

⟨dupseg() *switch segment type cases* 68f⟩+≡ (67d) ◁ 68c 68g ▷

```
case SG_DATA:       /* Copy on write plus demand load info */
    ⟨dupseg() when SG_DATA, if Text segment 127a⟩
    if(share)
        goto sameseg; // threads! clone()
    // else
    n = newseg(s->type, s->base, s->size);
    ⟨dupseg() SG_DATA case, attach image to new segment n 126f⟩
    break;
```

⟨dupseg() *switch segment type cases* 68g⟩+≡ (67d) ◁ 68f 424n ▷

```
case SG_BSS:        /* Just copy on write */
    if(share)
        goto sameseg; // threads! clone()
    // else
    n = newseg(s->type, s->base, s->size);
    break;
```

5.7.7 /proc/x/segment

⟨procread() *locals* 69a⟩≡ (468c) 254b ▷
```
Segment *sg;
```

⟨procread() *cases* 69b⟩≡ (468c) 254e ▷
```
case Qsegment:
    j = 0;
    for(i = 0; i < NSEG; i++) {
        sg = p->seg[i];
        if(sg == nil)
            continue;
        j += snprint(statbuf+j, sizeof statbuf - j,
            "%-6s %c%c %.8lux %.8lux %4ld\n",
            sname[sg->type&SG_TYPE],
            sg->type&SG_RONLY ? 'R' : ' ',
            sg->profile ? 'P' : ' ',
            sg->base, sg->top, sg->ref);
    }
    if(offset >= j)
        return 0;
    if(offset+n > j)
        n = j-offset;
    if(n == 0 && offset == 0)
        exhausted("segments");
    memmove(a, &statbuf[offset], n);
    return n;
```

5.8 sysbrk()

⟨*syscall brk* 69c⟩≡ (603a)
```
// int brk(void*);
long
sysbrk(ulong* arg)
{
    return ibrk(arg[0], BSEG); // BSS, the heap size is changed
}
```

⟨*function* ibrk 69d⟩≡ (589)
```
long
ibrk(user_addr addr, int seg)
{
    Segment *s;
    user_addr newtop;
    ulong newsize;
    ⟨ibrk() other locals 70d⟩

    s = up->seg[seg];
    ⟨ibrk() sanity check s 70a⟩
    ⟨ibrk() if addr nil 70b⟩

    qlock(&s->lk);

    ⟨ibrk() if addr below base 70c⟩

    newtop = PGROUND(addr);
    newsize = (newtop - s->base)/BY2PG; // in nb pages

    ⟨ibrk() if newtop lower than oldtop, shrink the segment 71c⟩
```

```
    // else
    ⟨ibrk() if newtop more than oldtop, extend the segment 70e⟩

}
```

⟨ibrk() *sanity check s* 70a⟩≡ (69d)

```
if(s == nil)
    error(Ebadarg);
```

⟨ibrk() *if addr nil* 70b⟩≡ (69d)

```
if(addr == nilptr)
    return s->base;
```

⟨ibrk() *if addr below base* 70c⟩≡ (69d)

```
/* We may start with the bss overlapping the data */
if(addr < s->base) {
    if(seg != BSEG || up->seg[DSEG] == nil || addr < up->seg[DSEG]->base) {
        qunlock(&s->lk);
        error(Enovmem);
    }
    addr = s->base;
}
```

5.8.1 Extending a segment

⟨ibrk() *other locals* 70d⟩≡ (69d) 70f▷

```
int mapsize;
Pagetable **map;
```

⟨ibrk() *if newtop more than oldtop, extend the segment* 70e⟩≡ (69d)

```
⟨ibrk() sanity check newtop does not overlap other segments 71a⟩
⟨ibrk() sanity check newsize 71b⟩

// similar to code in newseg()
mapsize = ROUND(newsize, PAGETABSIZE)/PAGETABSIZE;
// realloc
if(mapsize > s->pagedirsize){
    map = smalloc(mapsize*sizeof(Pagetable*));
    memmove(map, s->pagedir, s->pagedirsize*sizeof(Pagetable*));
    if(s->pagedir != s->smallpagedir)
        free(s->pagedir);
    s->pagedir = map;
    s->pagedirsize = mapsize;
}

s->top = newtop;
s->size = newsize;
qunlock(&s->lk);
return OK_0;
```

⟨ibrk() *other locals* 70f⟩+≡ (69d) ◁70d

```
Segment *ns;
int i;
```

⟨ibrk() *sanity check newtop does not overlap other segments* 71a⟩≡ (70e)

```
for(i = 0; i < NSEG; i++) {
    ns = up->seg[i];
    if(ns == nil || ns == s)
        continue;
    if(newtop >= ns->base && newtop < ns->top) {
        qunlock(&s->lk);
        error(Esoverlap);
    }
}
```

⟨ibrk() *sanity check newsize* 71b⟩≡ (70e)

```
if(newsize > (PAGEDIRSIZE*PAGETABSIZE)) {
    qunlock(&s->lk);
    error(Enovmem);
}
```

5.8.2 Shrinking a segment

⟨ibrk() *if newtop lower than oldtop, shrink the segment* 71c⟩≡ (69d)

```
if(newtop < s->top) {
    /*
     * do not shrink a segment shared with other procs, as the
     * to-be-freed address space may have been passed to the kernel
     * already by another proc and is past the validaddr stage.
     */
    if(s->ref > 1){
        qunlock(&s->lk);
        error(Einuse);
    }
    mfreeseg(s, newtop, (s->top-newtop)/BY2PG);
    s->top = newtop;
    s->size = newsize;
    qunlock(&s->lk);
    arch_flushmmu();
    return OK_0;
}
```

⟨*function* mfreeseg 71d⟩≡ (589)

```
/*
 *  called with s->lk locked
 */
void
mfreeseg(Segment *s, ulong start, int pages)
{
    int i, j, size;
    ulong soff;
    PageOrSwap *pg;
    Page *list;

    soff = start-s->base;
    j = (soff&(PAGETABMAPMEM-1))/BY2PG; // PTX

    size = s->pagedirsize;
    list = nil;
    for(i = soff/PAGETABMAPMEM; i < size; i++) { // PDX
        if(pages <= 0)
            break;
        if(s->pagedir[i] == nil) { // space was never accessed, good, easier
```

```
            pages -= PAGETABSIZE-j;
            j = 0;
            continue;
        }
        while(j < PAGETABSIZE) {
            pg = s->pagedir[i]->pagetab[j];
            /*
             * We want to zero s->pagedir[i]->page[j] and putpage(pg),
             * but we have to make sure other processors flush the
             * entry from their TLBs before the page is freed.
             * We construct a list of the pages to be freed, zero
             * the entries, then (below) call procflushseg, and call
             * putpage on the whole list.
             *
             * Swapped-out pages don't appear in TLBs, so it's okay
             * to putswap those pages before procflushseg.
             */
            if(pg){
                ⟨mfreeseg if pg is a swap address 135f⟩
                else{
                    pg->next = list;
                    list = pg;
                }
                s->pagedir[i]->pagetab[j] = nil;
            }
            if(--pages == 0)
                goto out;
            j++;
        }
        j = 0;
    }
out:
    ⟨mfreeseg() if segment was shared, flush it 353c⟩
    /* free the pages */
    for(pg = list; pg != nil; pg = list){
        list = list->next;
        putpage(pg);
    }
}
```

Chapter 6

Processes

⟨systab *process syscalls* 73a⟩≡ (26c)

```
[RFORK]     sysrfork,
[EXEC]      sysexec,
[AWAIT]     sysawait,
[EXITS]     sysexits,
```

6.1 Overview

6.2 Process allocation

6.2.1 Process ID: Proc.pid

⟨Proc *state fields* 73b⟩+≡ (29c) ◁30b 75e▷

```
ulong pid;
```

⟨*global* pidalloc 73c⟩≡ (599c)

```
static Counter  pidalloc;
```

6.2.2 Procalloc

⟨*struct* Procalloc 73d⟩≡ (592c)

```
// Process allocator
struct Procalloc
{
    // array<Proc>, xalloc'ed in procinit() (conf.nproc)
    Proc* arena;
    // list<ref<Proc>> (next = Proc.qnext)
    Proc* free;

    ⟨Procalloc other fields 74a⟩

    // extra
    Lock;
};
```

⟨*global* procalloc 73e⟩≡ (599c)

```
static struct Procalloc procalloc;
```

6.2.3 PID hashing

⟨Procalloc *other fields* 74a⟩≡ (73d)

```
// hash<Proc.pid, ref<Proc>> (next = Proc.pidhash)>
Proc* ht[128];
```

⟨Proc *extra fields* 74b⟩+≡ (29c) ◁39f 147a▷

```
// hash<Proc.pid, ref<Proc>> Procalloc.ht
Proc  *pidhash; /* next proc in pid hash */
```

⟨*function* pidhash 74c⟩≡ (599c)

```
static void
pidhash(Proc *p)
{
    int h;

    h = p->pid % nelem(procalloc.ht);
    lock(&procalloc);
    // add_hash(procalloc.ht, p->pid, p)
    p->pidhash = procalloc.ht[h];
    procalloc.ht[h] = p;
    unlock(&procalloc);
}
```

⟨*function* pidunhash 74d⟩≡ (599c)

```
static void
pidunhash(Proc *p)
{
    int h;
    Proc **l;

    h = p->pid % nelem(procalloc.ht);
    lock(&procalloc);
    // remove_hash(procalloc.ht, p->pid, p)
    for(l = &procalloc.ht[h]; *l != nil; l = &(*l)->pidhash)
        if(*l == p){
            *l = p->pidhash;
            break;
        }
    unlock(&procalloc);
}
```

⟨*function* procindex 74e⟩≡ (599c)

```
int
procindex(ulong pid)
{
    Proc *p;
    int h;
    int s;

    s = -1;
    // hash_lookup(pid, procalloc.ht)
    h = pid % nelem(procalloc.ht);
    lock(&procalloc);
    for(p = procalloc.ht[h]; p != nil; p = p->pidhash)
        if(p->pid == pid){
            s = p - procalloc.arena;
            break;
        }
    unlock(&procalloc);
    return s;
}
```

6.2.4 newproc()

⟨Procstate *cases* 75a⟩+≡ (30c) ◁42c 93a▷
```
  Scheding,
```

⟨*constructor* newproc 75b⟩≡ (599c)
```
  Proc*
  newproc(void)
  {
      Proc *p;
      ⟨newproc() other locals 75h⟩

      lock(&procalloc);
      // =~ p = procalloc.free
      ⟨newproc() set p to procalloc.free and wait if nil 76a⟩
      procalloc.free = p->qnext;
      unlock(&procalloc);
      p->qnext = nil;

      p->state = Scheding;
      p->pid = incref(&pidalloc);
      ⟨newproc() sanity check pid 75c⟩
      pidhash(p);

      ⟨newproc() set fields of p 75d⟩

      return p;
  }
```

⟨newproc() *sanity check pid* 75c⟩≡ (75b)
```
  if(p->pid==0)
      panic("pidalloc");
```

⟨newproc() *set fields of* p 75d⟩≡ (75b) 75f▷
```
  memset(p->seg, nilptr, sizeof p->seg);
```

⟨Proc *state fields* 75e⟩+≡ (29c) ◁73b 77b▷
```
  // some debugging information, e.g. "New", "PageOut", or name of syscall
  char  *psstate; /* used by /proc/#/status */
  // e.g. "*init*", or name of executable
  char  *text;
```

⟨newproc() *set fields of* p 75f⟩+≡ (75b) ◁75d 75g▷
```
  p->psstate = "New";
  kstrdup(&p->text, "*notext");
```

⟨newproc() *set fields of* p 75g⟩+≡ (75b) ◁75f 77c▷
```
  p->fgrp = nil;
  p->pgrp = nil;
  p->egrp = nil;
  p->rgrp = nil;
```

6.2.5 No more processes

⟨newproc() *other locals* 75h⟩≡ (75b)
```
  char msg[64];
```

⟨newproc() *set* p *to* procalloc.free *and wait if nil* 76a⟩≡ (75b)

```
while((p = procalloc.free) == nil) {
    unlock(&procalloc);

    snprint(msg, sizeof msg, "no procs; %s forking", up?up->text:"kernel");
    /*
     * the situation is unlikely to heal itself.
     * dump the proc table and restart by default.
     * *noprocspersist in plan9.ini will yield the old
     * behaviour of trying forever.
     */
    if(getconf("*noprocspersist") == nil)
        noprocpanic(msg);
    resrcwait(msg);
    lock(&procalloc);
}
```

⟨*function* noprocpanic 76b⟩≡ (599c)

```
void
noprocpanic(char *msg)
{
    /*
     * setting exiting will make hzclock() on each processor call exit(0).
     * clearing our bit in cpus avoids calling exit(0) from hzclock()
     * on this processor.
     */
    lock(&active);
    active.cpus &= ~(1<<cpu->cpuno);
    active.exiting = true;
    unlock(&active);

    procdump();
    arch_delay(1000);
    panic(msg);
}
```

⟨*function* resrcwait 76c⟩≡ (597b)

```
void
resrcwait(char *reason)
{
    ulong now;
    char *p;
    static ulong lastwhine;

    if(up == nil)
        panic("resrcwait");

    p = up->psstate;
    if(reason) {
        up->psstate = reason;
        now = seconds();
        /* don't tie up the console with complaints */
        if(now - lastwhine > Whinesecs) {
            lastwhine = now;
            print("%s\n", reason);
        }
    }
    tsleep(&up->sleepr, returnfalse, 0, 300);
    up->psstate = p;
}
```

⟨*constant* Whinesecs 77a⟩≡ (597b)

```
Whinesecs = 10,     /* frequency of out-of-resources printing */
```

6.3 The user

⟨Proc *state fields* 77b⟩+≡ (29c) ◁75e 80b▷

```
// e.g.. "eve" (no uid/gid in plan9, because of its distributed nature?)
char  *user;
```

⟨newproc() *set fields of* p 77c⟩+≡ (75b) ◁75g 86e▷

```
kstrdup(&p->user, "*nouser");
```

⟨*function* hostownerwrite 77d⟩≡ (605)

```
/*
 *  called by devcons() for host owner/domain
 *
 *  writing hostowner also sets user
 */
long
hostownerwrite(char *a, int n)
{
    char buf[128];

    if(!iseve())
        error(Eperm);
    if(n <= 0 || n >= sizeof buf)
        error(Ebadarg);
    memmove(buf, a, n);
    buf[n] = 0;

    renameuser(eve, buf);
    kstrdup(&eve, buf);
    kstrdup(&up->user, buf);
    up->basepri = PriNormal;
    return n;
}
```

⟨*function* renameuser 77e⟩≡ (599c)

```
/*
 *  change ownership to 'new' of all processes owned by 'old'.  Used when
 *  eve changes.
 */
void
renameuser(char *old, char *new)
{
    Proc *p, *ep;

    ep = procalloc.arena + conf.nproc;
    for(p = procalloc.arena; p < ep; p++)
        if(p->user!=nil && strcmp(old, p->user)==0)
            kstrdup(&p->user, new);
}
```

6.4 sysrfork()

⟨*enum* rfork 78a⟩≡ (603a)

```
//coupling: with libc.h
enum Rfork
{
    RFPROC      = (1<<4), // fork a new process!! (if unset then set props for up)
    RFMEM       = (1<<5), // share data and bss (kinda thread, a la Linux clone)

    RFNOWAIT    = (1<<6), // child will not leave a waitmsg

    RFNAMEG     = (1<<0), // copy namespace (if unset then share)
    RFENVG      = (1<<1), // copy environment variables (if unset then share)
    RFFDG       = (1<<2), // copy file descriptor table (if unset then share)
    RFCNAMEG    = (1<<10), // clean new namespace
    RFCENVG     = (1<<11), // clean new empty environment variables
    RFCFDG      = (1<<12), // clean new file descriptor table

    RFNOTEG     = (1<<3), // start new group for notes
    RFREND      = (1<<13), // start a new group for rendezvous

    RFNOMNT     = (1<<14), // # paths forbidden, sandboxing
};
```

⟨*syscall rfork* 78b⟩≡ (603a)

```
// int rfork(int flags);
long
sysrfork(ulong* arg)
{
    ulong flag;
    ⟨sysrfork() other locals 78c⟩

    flag = arg[0];
    /* Check flags before we commit */
    ⟨sysrfork() sanity check flags, clean or copy 80d⟩

    ⟨sysrfork() if no RFPROC, no new proc but property settings 82b⟩
    // else
    ⟨sysrfork() when RFPROC, fork a new process 78d⟩
}
```

6.4.1 Fork: RFPROC

⟨sysrfork() *other locals* 78c⟩≡ (78b) 79c ▷

```
Proc *p;
ulong pid;
```

⟨sysrfork() *when* RFPROC, *fork a new process* 78d⟩≡ (78b)

```
p = newproc();
pid = p->pid;

⟨sysrfork() when RFPROC propagate fields from up to p 79a⟩

/* Make a new set of memory segments */
⟨sysrfork() copy or share memory segments 79d⟩

/* File descriptors */
⟨sysrfork() copy, clean, or share file descriptors 80e⟩
/* Process groups */ // Namespace
```

```
⟨sysrfork() copy, clean, or share namespace 81b⟩
// Rendez vous group
⟨sysrfork() new or share rendezvous group 81e⟩
/* Environment group */
⟨sysrfork() copy, clean, or share environment 81d⟩
// Note group
⟨sysrfork() share or not note group 81f⟩

/* Craft a return frame which will cause the child to pop out of
 * the scheduler in user mode with the return register zero
 */
arch_forkchild(p, up->dbgreg);

p->parent = up;
p->parentpid = up->pid;
⟨sysrfork() wait or not for child 96a⟩

⟨sysrfork() setting time field 380c⟩

/*
 *  since the bss/data segments are now shareable,
 *  any mmu info about this process is now stale
 *  (i.e. has bad properties) and has to be discarded.
 */
arch_flushmmu();

⟨sysrfork() if parent is a wired proc 161a⟩

ready(p);
sched(); // !!!
return pid;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 79a⟩≡ (78d) 79b ▷

```
kstrdup(&p->text, up->text);
kstrdup(&p->user, up->user);
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 79b⟩+≡ (78d) ◁ 79a 101c ▷

```
p->slash = up->slash;
p->dot = up->dot;
incref(p->dot);
```

6.4.2 Memory sharing: RFMEM

⟨sysrfork() *other locals* 79c⟩+≡ (78b) ◁ 78c 82a ▷

```
bool share;
int i;
```

⟨sysrfork() *copy or share memory segments* 79d⟩≡ (78d)

```
share = flag & RFMEM;

qlock(&p->seglock);
if(waserror()){
    qunlock(&p->seglock);
    nexterror();
}
for(i = 0; i < NSEG; i++)
    if(up->seg[i])
        p->seg[i] = dupseg(up->seg, i, share);
qunlock(&p->seglock);
poperror();
```

6.4.3 arch_forkchild() (ARM)

⟨*function* arch_forkchild*(arm)* 80a⟩≡ (643c)

```
/*
 *  Craft a return frame which will cause the child to pop out of
 *  the scheduler in user mode with the return register zero.  Set
 *  pc to point to a l.s return function.
 */
void
arch_forkchild(Proc *p, Ureg *ureg)
{
    Ureg *cureg;

    p->sched.sp = (ulong)p->kstack + KSTACK - sizeof(Ureg);
    p->sched.pc = (ulong)arch__forkret;

    cureg = (Ureg*)(p->sched.sp);
    memmove(cureg, ureg, sizeof(Ureg));
    /* syscall returns 0 for child */ // adjust
    cureg->r0 = 0;

    /* Things from bottom of syscall which were never executed */
    p->psstate = nil;
    p->insyscall = false;

    ⟨arch_forkchild() floating point(arm) 388e⟩
}
```

⟨Proc *state fields* 80b⟩+≡ (29c) ◁77b 86d▷

```
bool insyscall; // true when process inside a syscall
```

⟨*function* arch__forkret*(arm)* 80c⟩≡ (643a)

```
/*
 *  here to jump to a newly forked process
 */
TEXT arch__forkret(SB), 1, $-4
    ADD $(4*15), R13        /* make r13 point to ureg->type */
    MOVW    8(R13), R14     /* restore link */
    MOVW    4(R13), R0      /* restore SPSR */
    MOVW    R0, SPSR        /* ... */
    MOVM.DB.S (R13), [R0-R14]   /* restore registers */
    ADD $8, R13         /* pop past ureg->{type+psr} */
    RFE             /* MOVM.IA.S.W (R13), [R15] */
```

6.4.4 File descriptors: RFFDG, RFCFDG

⟨sysrfork() *sanity check flags, clean or copy* 80d⟩≡ (78b) 81a▷

```
if((flag & (RFFDG|RFCFDG)) == (RFFDG|RFCFDG))
    error(Ebadarg);
```

⟨sysrfork() *copy, clean, or share file descriptors* 80e⟩≡ (78d)

```
if(flag & (RFFDG|RFCFDG)) {
    if(flag & RFFDG)
        p->fgrp = dupfgrp(up->fgrp);
    else
        p->fgrp = dupfgrp(nil);
}
else {
    p->fgrp = up->fgrp;
    incref(p->fgrp);
}
```

6.4.5 Namespace: RFNAMEG, RFCNAMEG

⟨sysrfork() *sanity check flags, clean or copy* 81a⟩+≡ (78b) ◁80d 81c▷

```
if((flag & (RFNAMEG|RFCNAMEG)) == (RFNAMEG|RFCNAMEG))
    error(Ebadarg);
```

⟨sysrfork() *copy, clean, or share namespace* 81b⟩≡ (78d)

```
if(flag & (RFNAMEG|RFCNAMEG)) {
    p->pgrp = newpgrp();
    if(flag & RFNAMEG)
        pgrpcpy(p->pgrp, up->pgrp);
    ⟨sysrfork() inherit noattach, RFPROC==1 case 264b⟩
}
else {
    p->pgrp = up->pgrp;
    incref(p->pgrp);
}
⟨sysrfork() set noattach to true when RFNOMNT, RFPROC==1 case 264d⟩
```

6.4.6 Environment variables: RFENVG, RFCENVG

⟨sysrfork() *sanity check flags, clean or copy* 81c⟩+≡ (78b) ◁81a

```
if((flag & (RFENVG|RFCENVG)) == (RFENVG|RFCENVG))
    error(Ebadarg);
```

⟨sysrfork() *copy, clean, or share environment* 81d⟩≡ (78d)

```
if(flag & (RFENVG|RFCENVG)) {
    p->egrp = smalloc(sizeof(Egrp)); // newegrp
    p->egrp->ref = 1;
    if(flag & RFENVG)
        envcpy(p->egrp, up->egrp);
}
else {
    p->egrp = up->egrp;
    incref(p->egrp);
}
```

6.4.7 Rendezvous group: RFREND

⟨sysrfork() *new or share rendezvous group* 81e⟩≡ (78d)

```
if(flag & RFREND)
    p->rgrp = newrgrp();
else {
    incref(up->rgrp);
    p->rgrp = up->rgrp;
}
```

6.4.8 Note group: RFNOTEG

⟨sysrfork() *share or not note group* 81f⟩≡ (78d)

```
if((flag&RFNOTEG) == 0)
    p->noteid = up->noteid; // rectify what was done in newproc
```

6.4.9 Process sharing settings (no RFPROC)

⟨sysrfork() *other locals* 82a⟩+≡ (78b) ◁79c 160f▷

```
Fgrp *ofg;
Pgrp *opg;
Egrp *oeg;
Rgrp *org;
```

⟨sysrfork() *if no* RFPROC, *no new proc but property settings* 82b⟩≡ (78b)

```
if((flag&RFPROC) == 0) { // not a fork, just setting properties for up
    ⟨sysrfork() when no RFPROC, sanity check flag 82c⟩

    ⟨sysrfork() when no RFPROC, clean or copy file descriptors 82d⟩

    ⟨sysrfork() when no RFPROC, clean or copy namespace 82e⟩

    ⟨sysrfork() when no RFPROC, clean or copy environment 82f⟩

    ⟨sysrfork() when no RFPROC, new rendez vous group 83a⟩

    ⟨sysrfork() when no RFPROC, new note group 83b⟩

    return 0;
}
```

⟨sysrfork() *when no* RFPROC, *sanity check flag* 82c⟩≡ (82b)

```
if(flag & (RFMEM|RFNOWAIT))
    error(Ebadarg);
```

⟨sysrfork() *when no* RFPROC, *clean or copy file descriptors* 82d⟩≡ (82b)

```
if(flag & (RFFDG|RFCFDG)) {
    ofg = up->fgrp;
    if(flag & RFFDG)
        up->fgrp = dupfgrp(ofg);
    else
        up->fgrp = dupfgrp(nil);
    closefgrp(ofg);
}
```

⟨sysrfork() *when no* RFPROC, *clean or copy namespace* 82e⟩≡ (82b)

```
if(flag & (RFNAMEG|RFCNAMEG)) {
    opg = up->pgrp;
    up->pgrp = newpgrp();
    if(flag & RFNAMEG)
        pgrpcpy(up->pgrp, opg);
    ⟨sysrfork() inherit noattach, RFPROC==0 case 264a⟩
    closepgrp(opg);
}
⟨sysrfork() set noattach to true when RFNOMNT, RFPROC==0 case 264c⟩
```

⟨sysrfork() *when no* RFPROC, *clean or copy environment* 82f⟩≡ (82b)

```
if(flag & (RFENVG|RFCENVG)) {
    oeg = up->egrp;
    up->egrp = smalloc(sizeof(Egrp)); // newegrp()
    up->egrp->ref = 1;
    if(flag & RFENVG)
        envcpy(up->egrp, oeg);
    closeegrp(oeg);
}
```

⟨sysrfork() *when no* RFPROC, *new rendez vous group* 83a⟩≡ (82b)

```
if(flag & RFREND) {
    org = up->rgrp;
    up->rgrp = newrgrp();
    closergrp(org);
}
```

⟨sysrfork() *when no* RFPROC, *new note group* 83b⟩≡ (82b)

```
if(flag & RFNOTEG)
    up->noteid = incref(&noteidalloc);
```

6.5 sysexec()

⟨*constant* AOUT_MAGIC*(arm)* 83c⟩≡ (640d)

```
/*
 *  parameters for sysproc.c
 */
// E_MAGIC is defined in include/a.out.h
#define AOUT_MAGIC  (E_MAGIC)
```

⟨sysexec() *locals* 83d⟩≡ (83i) 83e ▷

```
char *file;
char *elem; // last element of binary, for up->text
char *file0;
```

⟨sysexec() *locals* 83e⟩+≡ (83i) ◁ 83d 83f ▷

```
Chan *tc;
```

⟨sysexec() *locals* 83f⟩+≡ (83i) ◁ 83e 83g ▷

```
Exec exec;
ulong magic;
user_addr text, entry, data, bss;
```

⟨sysexec() *locals* 83g⟩+≡ (83i) ◁ 83f 83h ▷

```
user_addr t, d, b; // end of text/data/bss in address rounded to page
Segment *s, *ts;
```

⟨sysexec() *locals* 83h⟩+≡ (83i) ◁ 83g **??** ▷

```
ulong ssize;
ulong nargs;
```

⟨*syscall exec* 83i⟩≡ (603a)

```
// void* exec(char *name, char* argv[]);
long
sysexec(ulong* arg)
{
    ⟨sysexec() locals 83d⟩

    elem = nil;
    validaddr(arg[0], 1, false);
    file0 = validnamedup((char*)arg[0], true);
    if(waserror()){
        free(file0);
        free(elem);
        nexterror();
    }
    file = file0;
    for(;;){
```

```
    // this will also adjust up->genbuf to contain the last elt of file path
    ⟨sysexec() call namec() to get a channel in tc from file 214e⟩
    if(waserror()){
        cclose(tc);
        nexterror();
    }
    if(!indir)
        kstrdup(&elem, up->genbuf);

    n = devtab[tc->type]->read(tc, &exec, sizeof(Exec), 0);
    if(n < 2) // need at least 2 bytes to decide if a #! or real binary
        error(Ebadexec);
    magic = l2be(exec.magic);
    text = l2be(exec.text);
    entry = l2be(exec.entry);

    if(n==sizeof(Exec) && (magic == AOUT_MAGIC)){
        if(text >= USTKTOP-UTZERO
        || entry < UTZERO+sizeof(Exec)
        || entry >= UTZERO+sizeof(Exec)+text)
            error(Ebadexec);

        // else
        break; /* for binary */
    }
    ⟨sysexec() process sharpbang 385b⟩
}
// from break above
data = l2be(exec.data);
bss = l2be(exec.bss);

⟨sysexec() compute end of text, data, bss segments 85a⟩


⟨sysexec() build args count 86g⟩

⟨sysexec() build temporary stack segment and qlock seglock 87d⟩

⟨sysexec() build args 87f⟩

⟨sysexec() free old memory segments 85d⟩

⟨sysexec() close files marked as opened with close on exec 228f⟩


⟨sysexec() set Text segment 86a⟩

⟨sysexec() set Data segment 86b⟩

⟨sysexec() set Bss segment 86c⟩

⟨sysexec() relocate temporary stack segment, unlock seglock 88a⟩

⟨sysexec() set prioriry of root processes 146e⟩

poperror();
cclose(tc); // tc has still a reference in the img

/*
 *  At this point, the mmu contains info about the old address
```

```
     *  space and needs to be flushed
     */
    arch_flushmmu();

    qlock(&up->debug);
    ⟨sysexec() when hold debug lock, reset some fields 335i⟩
    arch_procsetup(up);
    qunlock(&up->debug);

    ⟨sysexec() if hang 366g⟩

    return arch_execregs(entry, ssize, nargs);
}
```

⟨sysexec() *compute end of text, data, bss segments* 85a⟩≡ (83i)

```
t = UTROUND(UTZERO+sizeof(Exec)+text);
// data is put at page boundary after text
d = ROUND(t + data, BY2PG);
// note that this is not d + bss here, but t + data + bss!
b = ROUND(t + data + bss, BY2PG);

if(t >= KZERO || d >= KZERO || b >= KZERO)
    error(Ebadexec);
```

⟨*function* l2be 85b⟩≡ (603a)

```
ulong
l2be(long l)
{
    byte *cp;

    cp = (byte*)&l;
    return (cp[0]<<24) | (cp[1]<<16) | (cp[2]<<8) | cp[3];
}
```

⟨*macro* UTROUND*(arm)* 85c⟩≡ (633b)

```
#define UTROUND(t)  ROUNDUP((t), BY2PG)
```

⟨sysexec() *free old memory segments* 85d⟩≡ (83i)

```
/*
 * Committed.
 * Free old memory.
 * Special segments are maintained across exec
 */
for(i = SSEG; i <= BSEG; i++) {
    putseg(up->seg[i]);
    /* prevent a second free if we have an error */
    up->seg[i] = nil;
}
⟨sysexec() free special segments if close on exec 349c⟩
```

⟨*function* arch_procsetup*(arm)* 85e⟩≡ (641b)

```
/*
 *  set mach dependent process state for a new process
 */
void
arch_procsetup(Proc* p)
{
    ⟨arch_procsetup() floating point(arm) 388g⟩
}
```

6.5.1 Text segment

⟨sysexec() *set Text segment* 86a⟩≡ (83i)

```
/* Text.  Shared. Attaches to cache image if possible */
⟨sysexec() get text segment ts via demand loading on tc 126d⟩
up->seg[TSEG] = ts;
```

6.5.2 Data segment

⟨sysexec() *set Data segment* 86b⟩≡ (83i)

```
/* Data. Shared. */
s = newseg(SG_DATA, t, (d-t)>>PGSHIFT);
up->seg[DSEG] = s;
⟨sysexec() adjust data segment s for demand loading on tc 126e⟩
```

6.5.3 Bss segment

⟨sysexec() *set Bss segment* 86c⟩≡ (83i)

```
/* BSS. Zero fill on demand */
up->seg[BSEG] = newseg(SG_BSS, d, (b-d)>>PGSHIFT);
```

6.5.4 Stack segment

Arguments count

⟨Proc *state fields* 86d⟩+≡ (29c) ◁80b 89d▷

```
char  *args;
int nargs;    /* number of bytes of args */
```

⟨newproc() *set fields of* p 86e⟩+≡ (75b) ◁77c 89c▷

```
kstrdup(&p->args, "");
p->nargs = 0;
```

⟨sysexec() *locals* 86f⟩+≡ (83i) ◁?? 87c▷

```
char **argp;
char *a, *args;
ulong nbytes;
```

⟨sysexec() *build args count* 86g⟩≡ (83i)

```
/*
 * Args: pass 1: count
 */
nargs = 0;
nbytes = 0;

⟨sysexec() nbytes tos adjustments 384j⟩
⟨sysexec() if indir arg adjustments 386b⟩

arch_validalign(arg[1], sizeof(char**));
argp = (char**)arg[1];
validaddr((ulong)argp, BY2WD, false);
while(*argp){
    a = *argp++;
    if(((ulong)argp&(BY2PG-1)) < BY2WD)
        validaddr((ulong)argp, BY2WD, false);
    validaddr((ulong)a, 1, false);
    nbytes += ((char*)vmemchr(a, 0, 0x7FFFFFFF) - a) + 1;
    nargs++;
}
ssize = BY2WD*(nargs+1) + ROUND(nbytes, BY2WD);
```

Temporary stack segment

⟨*constant* TSTKTOP *(arm)* 87a⟩≡ (633b)

```
// end of new stack in sysexec
#define TSTKTOP     (USTKTOP-USTKSIZE)  /* sysexec temporary stack */
```

⟨*constant* TSTKSIZ *(arm)* 87b⟩≡ (633b)

```
#define TSTKSIZ     256 // pages in new stack; limits exec args
```

⟨sysexec() *locals* 87c⟩+≡ (83i) ◁86f 87e▷

```
ulong spage;
```

⟨sysexec() *build temporary stack segment and qlock seglock* 87d⟩≡ (83i)

```
/*
 * 8-byte align SP for those (e.g. sparc) that need it.
 * arch_execregs() will subtract another 4 bytes for argc.
 */
if((ssize+4) & 7)
    ssize += 4;
spage = (ssize+(BY2PG-1)) >> PGSHIFT;

/*
 * Build the stack segment, putting it in kernel virtual for the moment
 */
if(spage > TSTKSIZ)
    error(Enovmem);

qlock(&up->seglock);
if(waserror()){
    qunlock(&up->seglock);
    nexterror();
}
// why ESEG? and why not TSTKTOP?? ESEG because will free later
// the current SSEG. If we have an error in sysexec we don't want
// to have messed up with the current stack. TSTKTOP-USTKSIZE for
// the same reason, because we don't want overwrite old stack
// all of that will be relocated later.
up->seg[ESEG] = newseg(SG_STACK, TSTKTOP-USTKSIZE, USTKSIZE/BY2PG);
```

Arguments

⟨sysexec() *locals* 87e⟩+≡ (83i) ◁87c 126c▷

```
char **argv;
char *charp;
```

⟨sysexec() *build args* 87f⟩≡ (83i)

```
/*
 * Args: pass 2: assemble; the pages will be faulted in
 */
⟨sysexec() tos settings 384c⟩

argv = (char**)(TSTKTOP - ssize);
charp = (char*)(TSTKTOP - nbytes);
args = charp;
⟨sysexec() if indir argp adjustments 386c⟩
else
    argp = (char**)arg[1];

for(i=0; i<nargs; i++){
    ⟨sysexec() if indir argp adjustments again 386d⟩
```

```
        *argv++ = charp + (USTKTOP-TSTKTOP);
        n = strlen(*argp) + 1;
        memmove(charp, *argp++, n);
        charp += n;
    }
    free(file0);
    // file0 = nil? to avoid double free?

    free(up->text);
    up->text = elem;
    elem = nil; /* so waserror() won't free elem */
    USED(elem);

    /* copy args; easiest from new process's stack */
    n = charp - args;
    if(n > 128) /* don't waste too much space on huge arg lists */
        n = 128;
    a = up->args;
    up->args = nil;
    free(a);
    up->args = smalloc(n);
    memmove(up->args, args, n);

    if(n>0 && up->args[n-1]!='\0'){
        /* make sure last arg is NUL-terminated */
        /* put NUL at UTF-8 character boundary */
        for(i=n-1; i>0; --i)
            if(fullrune(up->args+i, n-i))
                break;
        up->args[i] = 0;
        n = i+1;
    }
    up->nargs = n;
```

Relocate temporary stack segment

⟨sysexec() *relocate temporary stack segment, unlock seglock* 88a⟩≡ (83i)

```
    /*
     * Move the stack
     */
    s = up->seg[ESEG];
    up->seg[ESEG] = nil;
    up->seg[SSEG] = s;
    qunlock(&up->seglock);

    poperror(); /* seglock */
    poperror(); /* elem */ // really? I think this matches more the cclose(tc)
    s->base = USTKTOP-USTKSIZE;
    s->top = USTKTOP;
    relocateseg(s, USTKTOP-TSTKTOP);
```

⟨*function* relocateseg 88b⟩≡ (589)

```
    void
    relocateseg(Segment *s, ulong offset)
    {
        Page **pg, *x;
        Pagetable *pt, **p, **endpt;

        endpt = &s->pagedir[s->pagedirsize];
```

```
    for(p = s->pagedir; p < endpt; p++) {
        if(*p == nil)
            continue;
        pt = *p;
        for(pg = pt->first; pg <= pt->last; pg++) {
            if(x = *pg)
                x->va += offset;
        }
    }
}
```

6.5.5 arch_execregs() (ARM)

⟨*function* arch_execregs*(arm)* 89a⟩≡ (643c)

```
long
arch_execregs(user_addr entry, ulong ssize, ulong nargs)
{
    user_wp sp;
    Ureg *ureg;

    sp = (user_wp)(USTKTOP - ssize);
    *--sp = nargs;

    ureg = up->dbgreg;
    ureg->r13 = (ulong)sp;
    ureg->pc = entry;

    /*
     * return the address of kernel/user shared data
     * (e.g. clock stuff)
     */
    return USTKTOP-sizeof(Tos);
}
```

6.6 sysawait()

6.6.1 Parent and children

⟨Proc *hierarchy fields* 89b⟩≡ (29c) 90a ▷

```
// option<ref<Proc>> (nil for the boot process)
Proc  *parent;
int nchild;   /* Number of living children */
```

⟨newproc() *set fields of* p 89c⟩+≡ (75b) ◁ 86e 90b ▷

```
p->parent = nil;
p->nchild = 0;
```

⟨Proc *state fields* 89d⟩+≡ (29c) ◁ 86d

```
// option<pid> (None = 0)
ulong parentpid;
```

6.6.2 Waitmsg and Waitq

⟨Proc *hierarchy fields* 90a⟩+≡ (29c) ◁89b 91c▷

```
// list<ref_own<Waitq>>> =~ list<ref_own<Waitmsg>>
Waitq *waitq;   /* Exited processes wait children */
int nwait;    /* Number of uncollected wait records */ // len(waitq)

Lock  exl;    /* Lock count and waitq */ // and nchild
```

⟨newproc() *set fields of* p 90b⟩+≡ (75b) ◁89c 96c▷

```
p->waitq = nil;
p->nwait = 0;
```

⟨*struct* Waitmsg 90c⟩≡ (580e)

```
struct Waitmsg
{
  int pid;    /* of loved one */ // pid of the child
  ⟨Waitmsg time field 381b⟩
  char  msg[ERRMAX];  /* actually variable-size in user mode */
};
```

⟨*struct* Waitq 90d⟩≡ (592c)

```
// essentially a stack<ref_own<Waitmsg>>
struct Waitq
{
    Waitmsg w;

    // extra
    // list<ref_own<Waitq>> Proc.waitq
    Waitq *next;
};
```

6.6.3 sysawait()

⟨*syscall await* 90e⟩≡ (603a)

```
// int await(char *s, int n);
long
sysawait(ulong* arg)
{
    int i;
    int pid;
    Waitmsg w; // allocated in stack!
    ulong n;

    n = arg[1];
    validaddr(arg[0], n, true);
    pid = pwait(&w);
    ⟨sysawait() sanity check pid 90f⟩
    i = snprint((char*)arg[0], n, "%d %lud %lud %lud %q",
        w.pid,
        ⟨sysawait() snprint time field arguments 381f⟩
        w.msg);

    return i;
}
```

⟨sysawait() *sanity check pid* 90f⟩≡ (90e)

```
if(pid < 0)
    return -1;
```

⟨*function* pwait 91a⟩≡ (599c)

```
ulong
pwait(Waitmsg *w)
{
    ulong cpid; // child pid
    Waitq *wq;

    ⟨pwait() lock qwaitr 464b⟩

    ⟨pwait() sanity check process has children 91b⟩

    // sleep!
    ⟨pwait() wait until has a waitq 91d⟩
    // ok, got a waitmsg

    lock(&up->exl);
    // wq = pop(up->waitq), // can't be null, see haswaitq()
    wq = up->waitq;
    up->waitq = wq->next;
    up->nwait--;
    unlock(&up->exl);

    ⟨pwait() unlock qwaitr 464c⟩

    if(w)
        memmove(w, &wq->w, sizeof(Waitmsg));
    cpid = wq->w.pid;
    free(wq);
    return cpid;
}
```

⟨pwait() *sanity check process has children* 91b⟩≡ (91a)

```
lock(&up->exl);
if(up->nchild == 0 && up->waitq == nil) {
    unlock(&up->exl);
    error(Enochild);
}
unlock(&up->exl);
```

6.6.4 Rendez-vous: Proc.waitr

⟨Proc *hierarchy fields* 91c⟩+≡ (29c) ◁90a 463b▷

```
Rendez  waitr;    /* Place to hang out in wait */
```

⟨pwait() *wait until has a waitq* 91d⟩≡ (91a)

```
sleep(&up->waitr, haswaitq, up); // qwaitr is still locked
```

⟨*function* haswaitq 91e⟩≡ (599c)

```
int
haswaitq(void *x)
{
    Proc *p;
    p = (Proc *)x;
    return p->waitq != nil;
}
```

6.7 sysexits()

⟨*syscall exits* 92a⟩≡ (603a)

```
// void exits(char *msg);
long
sysexits(ulong* arg)
{
    char *status;
    ⟨sysexits() other locals 92b⟩

    status = (char*)arg[0];
    ⟨sysexits() sanity check status, copy string in buf 92c⟩
    pexit(status, /*freemem*/true);

    panic("pexit: should never reach this point");
    return -1; // unreachable
}
```

⟨sysexits() *other locals* 92b⟩≡ (92a)

```
char *inval = "invalid exit string";
char buf[ERRMAX];
```

⟨sysexits() *sanity check status, copy string in buf* 92c⟩≡ (92a)

```
if(status){
    if(waserror())
        status = inval;
    else{
        // validaddr() and vmemchr() can generate error()
        validaddr((ulong)status, 1, false);
        if(vmemchr(status, '\0', ERRMAX) == 0){
            memmove(buf, status, ERRMAX);
            buf[ERRMAX-1] = '\0';
            status = buf;
        }
        poperror();
    }

}
```

6.7.1 pexit()

⟨*function* pexit 92d⟩≡ (599c)

```
void
proc_pexit(char *exitstr, bool freemem)
{
    Proc *p; // parent
    ⟨pexit() other locals 94a⟩

    // free 1
    ⟨pexit() at the start, free resources 170c⟩
    ⟨pexit() hook proctrace 377g⟩
    // free 2
    ⟨pexit() nil out resources under lock and free them 95g⟩

    ⟨pexit() if a kernel process, do nothing about parent and child 97a⟩
    else {
        /*
         * if not a kernel process and have a parent,
         * do some housekeeping.
```

```
         */
        p = up->parent;
        ⟨pexit() sanity check parent pointer p 93b⟩
        while(waserror())
            ; // ????

        // exit/await and your parent
        ⟨pexit() build waitq message wq 94b⟩
        ⟨pexit() add wq message in parent process p waitq 94c⟩
    }
    ⟨pexit() if not freemem, add process to broken processes list 363b⟩

    // free 3, must be after built waitq because exitstr in one of the segment
    ⟨pexit() free memory segments 95e⟩

    // exit/await and your (orphan) children
    ⟨pexit() wakeup process waiting on waitr and unhash pid 95a⟩

    // free 4
    ⟨pexit() free waitq 95c⟩
    // free 5
    ⟨pexit() release debuggers 365d⟩

    /* Sched must not loop for these locks */
    lock(&procalloc);
    lock(&palloc);

    ⟨pexit() optional edfstop() for real-time scheduling 433b⟩
    up->state = Moribund;
    // will arch_gotolabel() to schedinit() which has special code around Moribund
    sched();
    panic("pexit: should never reach this point");
}
```

⟨Procstate *cases* 93a⟩+≡ (30c) ◁75a 155b▷

```
Moribund,
```

⟨pexit() *sanity check parent pointer* p 93b⟩≡ (92d)

```
// no parent pointer, must be the very first process
if(p == nil) {
    if(exitstr == nil)
        exitstr = "unknown";
    panic("boot process died: %s", exitstr);
}
```

6.7.2 pexit() and schedinit()

⟨schedinit() *switch process state cases* 93c⟩≡ (152b) 152c▷

```
case Moribund:
    up->state = Dead;
    ⟨schedinit() optional real-time edfstop() 432g⟩
    /*
     * Holding locks from pexit:
     *  procalloc
     *  palloc
     */
    arch_mmurelease(up);

    up->qnext = procalloc.free;
```

```
    procalloc.free = up;

    unlock(&palloc);
    unlock(&procalloc);
    break;
```

6.7.3 Wait message

Wait message and your parent

⟨pexit() *other locals* 94a⟩≡ (92d) 95b ▷

```
Waitq *wq;
```

⟨pexit() *build waitq message* wq 94b⟩≡ (92d)

```
wq = smalloc(sizeof(Waitq));
poperror(); // of ????

wq->w.pid = up->pid;
⟨pexit() set wait msg time field 381d⟩
if(exitstr && exitstr[0])
    snprint(wq->w.msg, sizeof(wq->w.msg), "%s %lud: %s", up->text, up->pid, exitstr);
else
    wq->w.msg[0] = '\0';
```

⟨pexit() *add* wq *message in parent process* p *waitq* 94c⟩≡ (92d)

```
lock(&p->exl);
/*
 * Check that parent is still alive.
 */
// the parent pointer may not be your parent anymore, the parent
// could have died and its slot reallocated to another process
if(p->pid == up->parentpid && p->state != Broken) {
    p->nchild--;
    ⟨pexit() update TC time of parent 381e⟩
    /*
     * If there would be more than 128 wait records
     * processes for my parent, then don't leave a wait
     * record behind.  This helps prevent badly written
     * daemon processes from accumulating lots of wait
     * records.
     */
    if(p->nwait < 128) {
        // push(wq, p->waitq)
        wq->next = p->waitq;
        p->waitq = wq;
        p->nwait++;

        wq = nil; // the parent will do the free
        wakeup(&p->waitr); // haswaitq() is true now
    }
}
unlock(&p->exl);
if(wq)
    free(wq);
```

Wait and my (orphan) children

⟨pexit() *wakeup process waiting on waitr and unhash pid* 95a⟩≡ (92d)

```
lock(&up->exl);     /* Prevent my children from leaving waits */
pidunhash(up);
// so my children will not generate a waitq, I will not be here anymore
up->pid = 0;
wakeup(&up->waitr);
unlock(&up->exl);
```

⟨pexit() *other locals* 95b⟩+≡ (92d) ◁94a 95d▷

```
Waitq *f, *next;
```

⟨pexit() *free waitq* 95c⟩≡ (92d)

```
for(f = up->waitq; f; f = next) {
    next = f->next;
    free(f);
}
```

6.7.4 Freeing resources

⟨pexit() *other locals* 95d⟩+≡ (92d) ◁95b 95f▷

```
Segment **s, **es;
```

⟨pexit() *free memory segments* 95e⟩≡ (92d)

```
qlock(&up->seglock);
es = &up->seg[NSEG];
for(s = up->seg; s < es; s++) {
    if(*s) {
        putseg(*s);
        *s = nil;
    }
}
qunlock(&up->seglock);
```

⟨pexit() *other locals* 95f⟩+≡ (92d) ◁95d 377f▷

```
Fgrp *fgrp;
Egrp *egrp;
Rgrp *rgrp;
Pgrp *pgrp;
Chan *dot;
```

⟨pexit() *nil out resources under lock and free them* 95g⟩≡ (92d)

```
/* nil out all the resources under lock (free later) */
qlock(&up->debug);
fgrp = up->fgrp;
up->fgrp = nil;
egrp = up->egrp;
up->egrp = nil;
rgrp = up->rgrp;
up->rgrp = nil;
pgrp = up->pgrp;
up->pgrp = nil;
dot = up->dot;
up->dot = nil;
qunlock(&up->debug);

if(fgrp)
    closefgrp(fgrp);
```

```
if(egrp)
    closeegrp(egrp);
if(rgrp)
    closergrp(rgrp);
if(dot)
    cclose(dot);
if(pgrp)
    closepgrp(pgrp);
```

6.7.5 No waiting for child: RFNOWAIT

⟨sysrfork() *wait or not for child* 96a⟩≡ (78d)

```
if(flag&RFNOWAIT)
    p->parentpid = 0;
else {
    lock(&up->exl);
    up->nchild++;
    unlock(&up->exl);
}
```

6.8 Kernel processes

⟨Proc *other fields* 96b⟩≡ (29c) 101b ▷

```
bool kp;   /* true if a kernel process */
void  (*kpfun)(void*);
void  *kparg;
```

⟨newproc() *set fields of* p 96c⟩+≡ (75b) ◁ 90b 98c ▷

```
p->kp = false;
```

⟨*function* kproc 96d⟩≡ (599c)

```
// kernel process (aka kernel_thread in Linux?)
void
kproc(char *name, void (*func)(void *), void *arg)
{
    Proc *p;
    static Pgrp *kpgrp;

    p = newproc();
    p->kp = true; // Kernel Process!

    p->psstate = nil;
    p->procmode = 0640;
    p->ureg = nil;
    p->noswap = true;

    p->slash = up->slash;
    p->dot = up->dot;
    if(p->dot)
        incref(p->dot);

    p->fpsave = up->fpsave;
    p->sargs = up->sargs;

    memmove(p->note, up->note, sizeof(p->note));
    p->nnote = up->nnote;
    p->lastnote = up->lastnote;
```

```
    p->notify = up->notify;

    procpriority(p, PriKproc, false);

    arch_kprocchild(p, func, arg);

    kstrdup(&p->user, eve);
    kstrdup(&p->text, name);

    if(kpgrp == nil)
        kpgrp = newpgrp();
    p->pgrp = kpgrp;
    incref(kpgrp);

    ⟨kproc() setting time field 380d⟩
    ready(p);
}
```

⟨pexit() *if a kernel process, do nothing about parent and child* 97a⟩≡ (92d)

```
if(up->kp) {
}
```

6.8.1 arch_kprocchild() (ARM)

⟨*function* arch_kprocchild*(arm)* 97b⟩≡ (641a)

```
/*
 *  setup stack and initial PC for a new kernel proc.  This is architecture
 *  dependent because of the starting stack location
 */
void
arch_kprocchild(Proc *p, void (*func)(void*), void *arg)
{
    p->sched.pc = PTR2UINT(linkproc);
    p->sched.sp = PTR2UINT(p->kstack+KSTACK);

    p->kpfun = func;
    p->kparg = arg;
}
```

⟨*function* linkproc*(arm)* 97c⟩≡ (641a)

```
/*
 *  this is the body for all kproc's
 */
static void
linkproc(void)
{
    arch_spllo();
    up->kpfun(up->kparg);
    // should never reach this place?? kernel processes are supposed
    // to run forever??
    pexit("kproc exiting", false);
}
```

6.9 Process control /proc/#/ctl

⟨Procctl *cases* 97d⟩≡ (364c) 369a ▷

```
Proc_exitme,
```

⟨procctl() Proc_exitme *case* 98a⟩≡ (364e)

```
case Proc_exitme:
    arch_spllo();        /* pexit has locks in it */
    pexit("Killed", true);
```

⟨procctlreq() *CMkill case* 98b⟩≡ (482)

```
case CMkill:
    switch(p->state) {
    ⟨procctlreq() CMkill case, Broken case 364a⟩
    case Stopped:
        p->procctl = Proc_exitme;
        postnote(p, 0, "sys: killed", NExit);
        ready(p);
        break;
    default:
        p->procctl = Proc_exitme;
        postnote(p, 0, "sys: killed", NExit);
    }
    break;
```

⟨newproc() *set fields of* p 98c⟩+≡ (75b) ◁96c 141c▷

```
p->pdbg = nil;
p->procctl = Proc_nothing;
p->syscalltrace = nil;
p->trace = false;
p->dbgreg = nil;
```

Chapter 7

Interrupts

7.1 Overview

7.2 Exceptions (ARM)

⟨PsrMode *other cases(arm)* 99a⟩≡ (21a) 428b ▷

```
#define PsrMirq     0x00000012
#define PsrMabt     0x00000017
#define PsrMund     0x0000001B
```

⟨*global* trapnames*(arm)* 99b⟩≡ (636c)

```
static char *trapnames[PsrMask+1] = {
    [ PsrMusr ] "user mode",
    [ PsrMsvc ] "svc/swi exception",
    [ PsrMirq ] "irq interrupt",
    [ PsrMabt ] "prefetch abort/data abort",
    [ PsrMabt+1 ] "data abort",
    [ PsrMund ] "undefined instruction",
    ⟨trapnames() other entries(arm) 428c⟩
};
```

7.3 Exception vectors (ARM)

⟨*global* vectors*(arm)* 99c⟩≡ (638a)

```
/*
 *  exception vectors, copied by trapinit() to somewhere useful
 */
TEXT vectors(SB), 1, $-4
    MOVW    0x18(R15), R15      /* reset */
    MOVW    0x18(R15), R15      /* undefined instr. */
    MOVW    0x18(R15), R15      /* SWI & SMC */
    MOVW    0x18(R15), R15      /* prefetch abort */
    MOVW    0x18(R15), R15      /* data abort */
    MOVW    0x18(R15), R15      /* reserved */
    MOVW    0x18(R15), R15      /* IRQ */
    MOVW    0x18(R15), R15      /* FIQ */
```

⟨*global* vtable*(arm)* 99d⟩≡ (638a)

```
//coupling: must be just after vectors in memory (at +0x18 = 8 instructions)
TEXT vtable(SB), 1, $-4
    WORD    $_vsvc(SB)      /* reset, in svc mode already */
    WORD    $_vund(SB)      /* undefined, switch to svc mode */
```

```
    WORD    $_vsvc(SB)      /* swi, in svc mode already */
    WORD    $_vpabt(SB)     /* prefetch abort, switch to svc mode */
    WORD    $_vdabt(SB)     /* data abort, switch to svc mode */
    WORD    $_vsvc(SB)      /* reserved */
    WORD    $_virq(SB)      /* IRQ, switch to svc mode */
    WORD    $_vfiq(SB)      /* FIQ, switch to svc mode */
```

7.4 System calls

7.4.1 _vsvc() (ARM)

⟨*constant* CPU*(arm)* 100a⟩≡ (633b)

```
#define CPU     10      /* R10 is cpu-> */
```

⟨*constant* UP*(arm)* 100b⟩≡ (633b)

```
#define UP      9       /* R9 is up-> */
```

⟨*function* _vsvc*(arm)* 100c⟩≡ (638a)

```
TEXT _vsvc(SB), 1, $-4          /* SWI */
    CLREX
    MOVW.W  R14, -4(R13)        /* ureg->pc = interrupted PC */
    MOVW    SPSR, R14       /* ureg->psr = SPSR */
    MOVW.W  R14, -4(R13)        /* ... */
    MOVW    $PsrMsvc, R14       /* ureg->type = PsrMsvc */
    MOVW.W  R14, -4(R13)        /* ... */

    /* avoid the ambiguity described in notes/movm.w. */
    MOVM.DB.S [R0-R14], (R13)   /* save user level registers */
    SUB $(15*4), R13        /* r13 now points to ureg */

    MOVW    $setR12(SB), R12    /* Make sure we've got the kernel's SB loaded */

    /* get R(CPU) for this cpu */
    ⟨_vsvc() set cpu register(arm) 415b⟩

    MOVW    8(R(CPU)), R(UP)        /* up */ // Cpu->proc

    MOVW    R13, R0         /* first arg is pointer to ureg */
    SUB $8, R13         /* space for argument+link */

    BL  arch__syscall(SB)

    ADD $(8+4*15), R13      /* make r13 point to ureg->type */
    MOVW    8(R13), R14     /* restore link */
    MOVW    4(R13), R0      /* restore SPSR */
    MOVW    R0, SPSR        /* ... */
    MOVM.DB.S (R13), [R0-R14]   /* restore registers */
    ADD $8, R13         /* pop past ureg->{type+psr} */
    RFE             /* MOVM.IA.S.W (R13), [R15] */
```

7.4.2 Sargs

⟨*constant* MAXSYSARG 100d⟩≡ (592c)

```
#define MAXSYSARG  5 /* for mount(fd, afd, mpt, flag, arg) */
```

⟨*struct* Sargs 101a⟩≡ (592c)

```
// syscall arguments copied from user stack
struct Sargs
{
    ulong args[MAXSYSARG];
};
```

⟨Proc *other fields* 101b⟩+≡ (29c) ◁96b 174f▷

```
Sargs sargs;    /* address of this is known by db */
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 101c⟩+≡ (78d) ◁79b 141d▷

```
p->sargs = up->sargs;
```

7.4.3 arch__syscall() (ARM)

⟨*function* arch__syscall*(arm)* 101d⟩≡ (643c)

```
void
arch__syscall(Ureg* ureg)
{
    int scallnr;
    long ret;
    user_addr sp;
    ⟨syscall() other locals(arm) 102a⟩

    ⟨syscall() sanity check call from userspace(arm) 103b⟩
    ⟨syscall() adjust kentry 384f⟩

    cpu->syscall++;
    up->insyscall = true;

    up->pc = ureg->pc;
    up->dbgreg = ureg;

    // syscall number!
    scallnr = ureg->r0;
    //up->scallnr = scallnr;

    ⟨syscall() if RFORK, handle floating point(arm) 389e⟩

    arch_spllo();
    sp = ureg->sp;

    ⟨syscall() Proc_tracesyscall if, syscall entry(arm) 369c⟩

    up->nerrlab = 0;
    ret = -1;

    startns = todget(nil);
    if(!waserror()){
        ⟨syscall() sanity check scallnr(arm) 102b⟩
        ⟨syscall() sanity check sp(arm) 102c⟩

        // copy syscall arguments from user stack to up->sargs
        up->sargs = *((Sargs*)(sp+BY2WD)); // extra BY2WD for?
        up->psstate = sysctab[scallnr];

        //!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
        //IMPORTANT: The actual system call
        ret = systab[scallnr](up->sargs.args);
```

```
        //!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

        poperror();
    }else{
        /* failure: save the error buffer for errstr */
        e = up->syserrstr;
        up->syserrstr = up->errstr;
        up->errstr = e;
    }
    ⟨syscall() if nerrlab(arm) 102d⟩

    /*
     *  Put return value in frame.  On the x86 the syscall is
     *  just another trap and the return value from syscall is
     *  ignored.  On other machines the return value is put into
     *  the results register by caller of syscall.
     */
    ureg->r0 = ret;

    ⟨syscall() Proc_tracesyscall if, syscall exit(arm) 370a⟩

    up->insyscall = false;
    up->psstate = nil;

    ⟨syscall() call noted() 338b⟩
    ⟨syscall() call notify(arm) 336c⟩

    ⟨syscall() if delaysched(arm) 153e⟩
    arch__kexit(ureg);
}
```

⟨syscall() *other locals(arm)* 102a⟩≡ (101d) 369d ▷

```
char *e;
u32int s;
int i;
```

⟨syscall() *sanity check scallnr(arm)* 102b⟩≡ (101d)

```
if(scallnr >= nsyscall){
    pprint("bad sys call number %d pc %#lux\n",
        scallnr, ureg->pc);
    postnote(up, 1, "sys: bad sys call", NDebug);
    error(Ebadarg);
}
```

⟨syscall() *sanity check sp(arm)* 102c⟩≡ (101d)

```
if(sp < (USTKTOP-BY2PG) || sp > (USTKTOP-sizeof(Sargs)-BY2WD))
    validaddr(sp, sizeof(Sargs)+BY2WD, 0);
```

⟨syscall() *if nerrlab(arm)* 102d⟩≡ (101d)

```
if(up->nerrlab){
    print("bad errstack [%d]: %d extra\n", scallnr, up->nerrlab);
    for(i = 0; i < NERR; i++)
        print("sp=%#p pc=%#p\n",
            up->errlab[i].sp, up->errlab[i].pc);
    panic("error stack");
}
```

⟨Proc *debugger fields* 102e⟩≡ (29c) 364d ▷

```
void  *dbgreg;  /* User registers for devproc */
```

⟨*function* arch_userpc*(arm)* 103a⟩≡ (641a)

```
/*
 *  return the userpc the last exception happened at
 */
uintptr
arch_userpc(void)
{
    Ureg *ureg = up->dbgreg;
    return ureg->pc;
}
```

⟨syscall() *sanity check call from userspace(arm)* 103b⟩≡ (101d)

```
if(!arch_userureg(ureg))
    panic("syscall: from kernel: pc %#lux r14 %#lux psr %#lux",
        ureg->pc, ureg->r14, ureg->psr);
```

⟨*function* arch_userureg*(arm)* 103c⟩≡ (641a)

```
bool
arch_userureg(Ureg* ureg)
{
    return (ureg->psr & PsrMask) == PsrMusr;
}
```

7.5 Exception handlers

7.5.1 _vxxx() (ARM)

⟨*function* _vund*(arm)* 103d⟩≡ (638a)

```
TEXT _vund(SB), 1, $-4          /* undefined */
    MOVM.IA [R0-R4], (R13)      /* free some working space */
    MOVW    $PsrMund, R0
    B   _vswitch
```

⟨*function* _vpabt*(arm)* 103e⟩≡ (638a)

```
TEXT _vpabt(SB), 1, $-4         /* prefetch abort */
    MOVM.IA [R0-R4], (R13)      /* free some working space */
    MOVW    $PsrMabt, R0        /* r0 = type */
    B   _vswitch
```

⟨*function* _vdabt*(arm)* 103f⟩≡ (638a)

```
TEXT _vdabt(SB), 1, $-4         /* data abort */
    MOVM.IA [R0-R4], (R13)      /* free some working space */
    MOVW    $(PsrMabt+1), R0    /* r0 = type */
    B   _vswitch
```

⟨*function* _virq*(arm)* 103g⟩≡ (638a)

```
TEXT _virq(SB), 1, $-4          /* IRQ */
    MOVM.IA [R0-R4], (R13)      /* free some working space */
    MOVW    $PsrMirq, R0        /* r0 = type */
    B   _vswitch
```

7.5.2 _vswitch() (ARM)

⟨*label* _vswitch*(arm)* 104a⟩≡ (638a)

```
    /*
     *  come here with type in R0 and R13 pointing above saved [r0-r4].
     *  we'll switch to SVC mode and then call trap.
     */
 _vswitch:
    CLREX
    MOVW    SPSR, R1        /* save SPSR for ureg */
    MOVW    R14, R2         /* save interrupted pc for ureg */
    MOVW    R13, R3         /* save pointer to where the original [R0-R4] are */

    /*
     * switch processor to svc mode.  this switches the banked registers
     * (r13 [sp] and r14 [link]) to those of svc mode.
     */
    MOVW    CPSR, R14
    BIC $PsrMask, R14
    ORR $(PsrDirq|PsrMsvc), R14
    MOVW    R14, CPSR       /* switch! */

    AND.S   $0xf, R1, R4        /* interrupted code kernel or user? */
    BEQ _userexcep
    // else
    /* here for trap from SVC mode */
    ⟨_vswitch trap from SVC mode 104c⟩
    /* here for trap from USER mode */
    ⟨_vswitch trap from USER mode 104b⟩
```

⟨_vswitch *trap from USER mode* 104b⟩≡ (104a)

```
 _userexcep:
    MOVM.DB.W [R0-R2], (R13)    /* set ureg->{type, psr, pc}; r13 points to ureg->type  */
    MOVM.IA   (R3), [R0-R4]     /* restore [R0-R4] from previous mode's stack */

    /* avoid the ambiguity described in notes/movm.w. */
    MOVM.DB.S [R0-R14], (R13)   /* save kernel level registers */
    SUB $(15*4), R13        /* r13 now points to ureg */

    MOVW    $setR12(SB), R12    /* Make sure we've got the kernel's SB loaded */

    /* get R(CPU) for this cpu */
    ⟨_vswitch() set cpu register(arm) 415c⟩

    MOVW    8(R(CPU)), R(UP)        /* up */

    MOVW    R13, R0         /* first arg is pointer to ureg */
    SUB $(4*2), R13     /* space for argument+link (for debugger) */

    BL  arch__trap(SB)

    ADD $(4*2+4*15), R13    /* make r13 point to ureg->type */
    MOVW    8(R13), R14     /* restore link */
    MOVW    4(R13), R0      /* restore SPSR */
    MOVW    R0, SPSR        /* ... */
    MOVM.DB.S (R13), [R0-R14]   /* restore registers */
    ADD $(4*2), R13     /* pop past ureg->{type+psr} */
    RFE             /* MOVM.IA.S.W (R13), [R15] */
```

⟨_vswitch *trap from SVC mode* 104c⟩≡ (104a)

```
MOVM.DB.W [R0-R2], (R13)    /* set ureg->{type, psr, pc}; r13 points to ureg->type  */
MOVM.IA   (R3), [R0-R4]     /* restore [R0-R4] from previous mode's stack */

/*
 * avoid the ambiguity described in notes/movm.w.
 * In order to get a predictable value in R13 after the stores,
 * separate the store-multiple from the stack-pointer adjustment.
 * We'll assume that the old value of R13 should be stored on the stack.
 */
/* save kernel level registers, at end r13 points to ureg */
MOVM.DB [R0-R14], (R13)
SUB $(15*4), R13        /* SP now points to saved R0 */

MOVW    $setR12(SB), R12    /* Make sure we've got the kernel's SB loaded */

MOVW    R13, R0         /* first arg is pointer to ureg */
SUB $(4*2), R13     /* space for argument+link (for debugger) */
MOVW    $0xdeaddead, R11    /* marker */

BL  arch__trap(SB)

MOVW    $setR12(SB), R12    /* reload kernel's SB (ORLY?) */
ADD $(4*2+4*15), R13    /* make r13 point to ureg->type */
/*
 * if we interrupted a previous trap's handler and are now
 * returning to it, we need to propagate the current R(CPU) (R10)
 * by overriding the saved one on the stack, since we may have
 * been rescheduled and be on a different processor now than
 * at entry.
 */
MOVW    R(CPU), (-(15-CPU)*4)(R13) /* restore current cpu's CPU */
MOVW    8(R13), R14     /* restore link */
MOVW    4(R13), R0      /* restore SPSR */
MOVW    R0, SPSR        /* ... */

MOVM.DB (R13), [R0-R14]     /* restore registers */

ADD $(4*2), R13     /* pop past ureg->{type+psr} to pc */
RFE             /* MOVM.IA.S.W (R13), [R15] */
```

7.5.3 arch__trap() (ARM)

⟨*function* arch__trap*(arm)* 105⟩≡ (636c)

```
/*
 *  here on all exceptions other than syscall (SWI) and fiq
 */
void
arch__trap(Ureg *ureg)
{
    bool user;
    bool clockintr;
    ⟨trap() other locals(arm) 106c⟩

    ⟨trap() sanity check interrupts are disabled(arm) 106b⟩
    ⟨trap() sanity check enough space in kernel stack(arm) 106d⟩

    user = (ureg->psr & PsrMask) == PsrMusr; // TODO: arch_userureg
    if(user){
        up->dbgreg = ureg;
```

```
        ⟨trap() adjust kentry when interrupt user 384g⟩
    }

    /*
     * All interrupts/exceptions should be resumed at ureg->pc-4,
     * except for Data Abort which resumes at ureg->pc-8.
     */
    if(ureg->type == (PsrMabt+1))
        ureg->pc -= 8;
    else
       ureg->pc -= 4;

    clockintr = false;      /* if set, may call sched() before return */

    switch(ureg->type){
    ⟨trap() switch exception type cases(arm) 107b⟩
    default:
        panic("unknown trap; type %#lux, psr mode %#lux pc %lux", ureg->type,
            ureg->psr & PsrMask, ureg->pc);
        break;
    }
    arch_splhi();

    ⟨trap() if delaysched and clockintr 153d⟩

    if(user){
        ⟨trap() call possibly notify(arm) 336a⟩
        arch__kexit(ureg);
    }
}
```

⟨*function* arch__kexit*(arm)* 106a⟩≡ (641a)

```
/* go to user space */
void
arch__kexit(Ureg*)
{
    ⟨arch__kexit() tos adjustments 384d⟩
    tos->cyclefreq = cpu->cpuhz;
    ⟨arch__kexit() write back cache(arm) 402o⟩
}
```

⟨trap() *sanity check interrupts are disabled(arm)* 106b⟩≡ (105)

```
assert(!arch_islo());
```

⟨trap() *other locals(arm)* 106c⟩≡ (105) 107a ▷

```
int rem;
```

⟨trap() *sanity check enough space in kernel stack(arm)* 106d⟩≡ (105)

```
if(up != nil)
    rem = ((char*)ureg)-up->kstack;
else
    rem = ((char*)ureg)-((char*)cpu + sizeof(Cpu));
if(rem < 256) {
    iprint("trap: %d stack bytes left, up %#p ureg %#p at pc %#lux\n",
        rem, up, ureg, ureg->pc);
    arch_delay(1000);
    arch_dumpstack();
    panic("trap: %d stack bytes left, up %#p ureg %#p at pc %#lux",
        rem, up, ureg, ureg->pc);
}
```

7.5.4 Undefined instruction fault (ARM)

⟨trap() *other locals(arm)* 107a⟩+≡ (105) ◁106c 107d▷

```
char buf[ERRMAX];
```

⟨trap() *switch exception type cases(arm)* 107b⟩≡ (105) 107e▷

```
case PsrMund:           /* undefined instruction */
    if(user){
        ⟨trap() when undefined instruction, if breakpoint(arm) 362a⟩
        else{
            ⟨trap() when undefined instruction, if float instr(arm) 398e⟩
            if(rv == 0){
                snprint(buf, sizeof buf,
                    "undefined instruction: pc %#lux\n",
                    ureg->pc);
                postnote(up, 1, buf, NDebug);
            }
        }
    }else{
        ⟨trap() when undefined instruction, round pc(arm) 107c⟩
        iprint("undefined instruction: pc %#lux inst %#ux\n",
            ureg->pc, *(u32int*)ureg->pc);
        panic("undefined instruction");
    }
    break;
```

⟨trap() *when undefined instruction, round pc(arm)* 107c⟩≡ (107b)

```
if (ureg->pc & 3) {
    iprint("rounding fault pc %#lux down to word\n",
        ureg->pc);
    ureg->pc &= ~3;
}
```

7.5.5 Prefetch fault (ARM)

⟨trap() *other locals(arm)* 107d⟩+≡ (105) ◁107a 108b▷

```
int x;
ulong fsr;
```

⟨trap() *switch exception type cases(arm)* 107e⟩+≡ (105) ◁107b 108c▷

```
case PsrMabt:           /* prefetch fault */
    x = ifsrget();
    fsr = (x>>7) & 0x8 | x & 0x7;
    switch(fsr){
    ⟨trap() when prefetch fault, switch fst cases(arm) 362b⟩
    default:
        // Page fault on code!!
        faultarm(ureg, ureg->pc, user, true);
        break;
    }
    break;
```

⟨*function* ifsrget *(arm)* 107f⟩≡ (637)

```
TEXT ifsrget(SB), 1, $-4            /* instruction fault status */
    MRC CpSC, 0, R0, C(CpFSR), C(0), CpFSRinst
    RET
```

⟨*type* CpFSR*(arm)* 108a⟩≡ (607c)

```
/*
 * CpFSR opcode2 values.
 */
#define CpFSRdata   0           /* armv6, armv7 */
#define CpFSRinst   1           /* armv6, armv7 */
```

7.5.6 Data fault (ARM)

⟨trap() *other locals(arm)* 108b⟩+≡ (105) ◁107d 398d▷

```
virt_addr va; // can be a kern_addr?
ulong inst;
```

⟨trap() *switch exception type cases(arm)* 108c⟩+≡ (105) ◁107e 112d▷

```
case PsrMabt+1:         /* data fault */
    va = farget();
    inst = *(ulong*)(ureg->pc);
    /* bits 12 and 10 have to be concatenated with status */
    x = fsrget();
    fsr = (x>>7) & 0x20 | (x>>6) & 0x10 | x & 0xf;
    switch(fsr){
    ⟨trap() when data fault, switch fst cases(arm) 108f⟩
    default:
    case 0xa:       /* ? was under external abort */
        panic("unknown data fault, 6b fsr %#lux", fsr);
        break;
    }
    break;
```

⟨*function* farget*(arm)* 108d⟩≡ (637)

```
TEXT farget(SB), 1, $-4             /* fault address */
    MRC CpSC, 0, R0, C(CpFAR), C(0x0)
    RET
```

⟨*function* fstget*(arm)* 108e⟩≡ (637)

```
TEXT fsrget(SB), 1, $-4             /* data fault status */
    MRC CpSC, 0, R0, C(CpFSR), C(0), CpFSRdata
    RET
```

⟨trap() *when data fault, switch fst cases(arm)* 108f⟩≡ (108c) 109a▷

```
case 0xd:
case 0xf:
    /* permission error, copy on write or real permission error */
    faultarm(ureg, va, user, !writetomem(inst));
    break;
```

⟨*function* writetomem*(arm)* 108g⟩≡ (636c)

```
/*
 *  returns true if the instruction writes memory, false otherwise
 */
bool
writetomem(ulong inst)
{
    /* swap always write memory */
    if((inst & 0x0FC00000) == 0x01000000)
        return true;

    /* loads and stores are distinguished by bit 20 */
    if(inst & (1<<20))
```

```
        return false;

    return true;
}
```

⟨trap() *when data fault, switch fst cases(arm)* 109a⟩+≡ (108c) ◁108f 109b▷

```
case 0x5:       /* translation fault, no section entry */
case 0x7:       /* translation fault, no page entry */
    // Page fault on data!!
    faultarm(ureg, va, user, !writetomem(inst));
    break;
```

⟨trap() *when data fault, switch fst cases(arm)* 109b⟩+≡ (108c) ◁109a 109c▷

```
case 0x9:
case 0xb:
    /* domain fault, accessing something we shouldn't */
    if(user){
        snprint(buf, sizeof buf,
            "sys: access violation: pc %#lux va %#p\n",
            ureg->pc, va);
        postnote(up, 1, buf, NDebug);
    } else
        panic("kernel access violation: pc %#lux va %#p",
            ureg->pc, va);
    break;
```

⟨trap() *when data fault, switch fst cases(arm)* 109c⟩+≡ (108c) ◁109b 109d▷

```
case 0x0:
    panic("vector exception at %#lux", ureg->pc);
    break;
```

⟨trap() *when data fault, switch fst cases(arm)* 109d⟩+≡ (108c) ◁109c 109e▷

```
case 0x1:       /* alignment fault */
case 0x3:       /* access flag fault (section) */
    if(user){
        snprint(buf, sizeof buf,
            "sys: alignment: pc %#lux va %#p\n",
            ureg->pc, va);
        postnote(up, 1, buf, NDebug);
    } else
        panic("kernel alignment: pc %#lux va %#p", ureg->pc, va);
    break;
```

⟨trap() *when data fault, switch fst cases(arm)* 109e⟩+≡ (108c) ◁109d 109f▷

```
case 0x2:
    panic("terminal exception at %#lux", ureg->pc);
    break;
```

⟨trap() *when data fault, switch fst cases(arm)* 109f⟩+≡ (108c) ◁109e 110a▷

```
case 0x4:       /* icache maint fault */
case 0x6:       /* access flag fault (page) */
case 0x8:       /* precise external abort, non-xlat'n */
case 0x28:
case 0xc:       /* l1 translation, precise ext. abort */
case 0x2c:
case 0xe:       /* l2 translation, precise ext. abort */
case 0x2e:
case 0x16:      /* imprecise ext. abort, non-xlt'n */
case 0x36:
    panic("external abort %#lux pc %#lux addr %#p",
        fsr, ureg->pc, va);
    break;
```

⟨trap() *when data fault, switch fst cases(arm)* 110a⟩+≡ (108c) ◁109f

```
case 0x1c:      /* l1 translation, precise parity err */
case 0x1e:      /* l2 translation, precise parity err */
case 0x18:      /* imprecise parity or ecc err */
    panic("translation parity error %#lux pc %#lux addr %#p",
        fsr, ureg->pc, va);
    break;
```

7.6 IRQs (ARM)

⟨*enum* IRQ*(arm)* 110b⟩≡ (609a)

```
enum IRQ {
    IRQtimer0   = 0, // reserved by GPU?
    IRQtimer1   = 1, // reserved by GPU?
    IRQtimer2   = 2, // reserved by GPU?
    IRQtimer3   = 3,

    IRQclock    = IRQtimer3,

    IRQusb      = 9,  // also IRQfiq
    IRQdma0     = 16, // IRQdma1, IRQdma2, ... via IRQDMA() macro

    IRQaux      = 29,
    IRQi2c      = 53,
    IRQspi      = 54,
    IRQmmc      = 62,

    ⟨IRQ basic cases(arm) 112b⟩
    ⟨IRQ local cases(arm) 422a⟩
    ⟨IRQ other cases(arm) 429b⟩
};
```

7.6.1 Vctl (ARM)

⟨*struct* Vctl*(arm)* 110c⟩≡ (636c)

```
struct Vctl {
    // enum<IRQ>
    int irq;

    u32int  *reg; // VIRTIO address
    u32int  mask;

    void    (*f)(Ureg*, void*); /* handler to call */
    void    *a; /* argument to call it with */

    ⟨Vctl other fields(arm) 422b⟩

    // Extra
    ⟨Vctl extra fields(arm) 110e⟩
};
```

⟨*global* vctl*(arm)* 110d⟩≡ (636c)

```
// list<ref_own<Vctl>>> (next = Vctl.next)
static Vctl *vctl;
```

⟨Vctl *extra fields(arm)* 110e⟩≡ (110c)

```
Vctl    *next;
```

⟨*global* `vctllock`*(arm)* 111a⟩≡ (636c)

```
static Lock vctllock;
```

7.6.2 Intregs (ARM)

⟨*struct* `Intregs`*(arm)* 111b⟩≡ (636c)

```
/*
 * interrupt control registers
 */
// The order matters! the fields match the memory-mapped external registers.
struct Intregs {
    u32int  ARMpending;
    u32int  GPUpending[2];

    u32int  FIQctl;

    u32int  GPUenable[2];
    u32int  ARMenable;

    u32int  GPUdisable[2];
    u32int  ARMdisable;
};
```

7.6.3 arch_intrenable() (ARM)

⟨*macro* `arch_intrenable`*(arm)* 111c⟩≡ (619d)

```
#define arch_intrenable(i, f, a, b, n) irqenable((i), (f), (a))
```

⟨*function* `irqenable`*(arm)* 111d⟩≡ (636c)

```
void
irqenable(int irq, void (*f)(Ureg*, void*), void* a)
{
    Vctl *v;
    Intregs *ip;
    u32int *enable;

    ip = (Intregs*)INTREGS;
    v = (Vctl*)malloc(sizeof(Vctl));
    ⟨irqenable() sanity check v(arm) 112a⟩
    v->irq = irq;
    v->cpu = 0;
    ⟨irqenable() if IRQlocal(arm) 422c⟩
    ⟨irqenable() if IRQbasic(arm) 112c⟩
    else{
        enable = &ip->GPUenable[irq/32];
        v->reg = &ip->GPUpending[irq/32];
        v->mask = 1 << (irq % 32);
    }
    v->f = f;
    v->a = a;
    lock(&vctllock);
    ⟨irqenable() if IRQfiq(arm) 429c⟩
    else{
        v->next = vctl;
        vctl = v;
        ⟨irqenable() if IRQlocal adjust enable(arm) 422d⟩
        else
            *enable = v->mask;
```

```
    }
    unlock(&vctllock);
}
```

⟨irqenable() *sanity check v(arm)* 112a⟩≡ (111d)

```
if(v == nil)
    panic("irqenable: no mem");
```

7.6.4 Basic IRQs (ARM)

⟨IRQ *basic cases(arm)* 112b⟩≡ (110b)

```
IRQbasic    = 64,
IRQtimerArm = IRQbasic + 0,
```

⟨irqenable() *if IRQbasic(arm)* 112c⟩≡ (111d)

```
else if(irq >= IRQbasic){
    enable = &ip->ARMenable;
    v->reg = &ip->ARMpending;
    v->mask = 1 << (irq - IRQbasic);
}
```

7.6.5 irq() (ARM)

⟨trap() *switch exception type cases(arm)* 112d⟩+≡ (105) ◁108c

```
case PsrMirq:
    clockintr = irq(ureg);
    cpu->intr++;
    break;
```

⟨*function* irq*(arm)* 112e⟩≡ (636c)

```
/*
 *  called by trap to handle irq interrupts.
 *  returns true iff a clock interrupt, thus maybe reschedule.
 */
static bool
irq(Ureg* ureg)
{
    Vctl *v;
    bool clockintr;
    bool found;

    cpu->perf.intrts = arch_perfticks();
    clockintr = false;
    found = false;
    for(v = vctl; v; v = v->next)
        if(v->cpu == cpu->cpuno && (*v->reg & v->mask) != 0){
            found = true;
            ⟨irq() before dispatch(arm) 405a⟩
            // Dispatch
            v->f(ureg, v->a);
            ⟨irq() after dispatch(arm) 405b⟩
            if(v->irq == IRQclock || v->irq == IRQcntps || v->irq == IRQcntpns)
                clockintr = true;
        }
    if(!found)
        cpu->spuriousintr++;
    intrtime();
    return clockintr;
}
```

Chapter 8

Virtual Memory

8.1 Overview

8.2 The ARM MMU (ARM)

⟨*macro* L1X*(arm)* 113a⟩≡ (635a)
```
#define L1X(va)     FEXT((va), 20, 12)
```

⟨*macro* L2X*(arm)* 113b⟩≡ (635a)
```
#define L2X(va)     FEXT((va), 12, 8)
```

⟨*macro L1X bis(arm)* 113c⟩≡ (608)
```
#define L1X(va)     (((((va))>>20) & 0x0fff)<<2)
```

⟨*type* PageDirGranularity*(arm)* 113d⟩≡ (607c)
```
#define Coarse      (Mbz|1)         /* L1 */
#define Section     (Mbz|2)         /* L1 1MB */
```

⟨*constant* Mbz*(arm)* 113e⟩≡ (607c)
```
/* Mbz (0x10) bit is implementation-defined and must be 0 on the cortex. */
#define Mbz     (0<<4)
```

⟨*type* PageTableGranularity*(arm)* 113f⟩≡ (607c)
```
#define Large       0x00000001      /* L2 64KB */
#define Small       0x00000002      /* L2 4KB */
```

⟨*type* DAC*(arm)* 113g⟩≡ (607c)
```
#define Client      1           /* DAC */
#define Manager     3           /* DAC */
```

⟨*type* PageTableEntryAttribute3*(arm)* 113h⟩≡ (607c)
```
#define Noaccess    0       /* AP, DAC */
#define Krw     1           /* AP */
/* armv7 deprecates AP[2] == 1 & AP[1:0] == 2 (Uro), prefers 3 (new in v7) */
#define Uro     2           /* AP */
#define Urw     3           /* AP */
```

⟨*constant* Fault*(arm)* 113i⟩≡ (607c)
```
#define Fault       0x00000000      /* L[12] pte: unmapped */
```

⟨*constant* Dom0*(arm)* 113j⟩≡ (607c)
```
#define Dom0        0
```

⟨*macro* L1AP*(arm)* 113k⟩≡ (607c)
```
#define L1AP(ap)    (AP(3, (ap)))
```

⟨*macro* AP*(arm)* 114a⟩≡ (607c)
```
#define AP(n, v)    F((v), ((n)*2)+4, 2)
```

⟨*macro* F*(arm)* 114b⟩≡ (607c)
```
#define F(v, o, w)  (((v) & ((1<<(w))-1))<<(o))
```

⟨*macro* DAC*(arm)* 114c⟩≡ (607c)
```
#define DAC(n, v)   F((v), (n)*2, 2)
```

⟨*macro* L2AP*(arm)* 114d⟩≡ (635a)
```
#define L2AP(ap)    l2ap(ap)
```

⟨*function* l2ap*(raspberry pi1)(arm)* 114e⟩≡ (631a)
```
int
l2ap(int ap)
{
    return (AP(3, (ap))|AP(2, (ap))|AP(1, (ap))|AP(0, (ap)));
}
```

8.3 Pagination and mmupd/mmupt (ARM)

⟨Cpu Arch *mmu fields(arm)* 114f⟩≡ (28e) 120c ▷
```
PTE*    mmul1;      /* l1 for this processor */
```

⟨*type* PTE*(arm)* 114g⟩≡ (619b)
```
typedef u32int      PTE;
```

⟨*constant* L1lo*(arm)* 114h⟩≡ (634)
```
L1lo        = UZERO/MiB,        /* L1X(UZERO)? */
```

⟨*constant* L1hi*(arm)* 114i⟩≡ (634)
```
L1hi        = (USTKTOP+MiB-1)/MiB,  /* L1X(USTKTOP+MiB-1)? */
```

⟨Proc Arch *memory fields(arm)* 114j⟩≡ (30a) 115e ▷
```
// list<ref<Page> > pages used to store page tables
Page*   mmul2;
```

⟨*macro* PPN*(arm)* 114k⟩≡ (633b)
```
#define PPN(x)      ((x)&~(BY2PG-1))
```

⟨*macro* VA*(arm)* 114l⟩≡ (633a)
```
#define VA(k)       ((uintptr)(k))
```

⟨*function* arch_mmurelease*(arm)* 114m⟩≡ (635a)
```
void
arch_mmurelease(Proc* proc)
{
    Page *page, *next;

    ⟨arch_mmurelease() invalidate cache before adjusting page tables(arm) 402k⟩
    mmul2empty(proc, false);
    ⟨arch_mmurelease() free l2 page cache(arm) 115g⟩
    mmul1empty();
    ⟨arch_mmurelease() write back cache after adjusting page tables(arm) 402d⟩
    /* lose any possible stale tlb entries */
    mmuinvalidate();
}
```

⟨*function* mmuinvalidate*(arm)* 115a⟩≡ (635b)

```
/*
 * invalidate tlb
 */
TEXT mmuinvalidate(SB), 1, $-4
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
    BARRIERS
    RET
```

⟨*type CpTLB secondary registers(arm)* 115b⟩≡ (607c)

```
/*
 * CpTLB Secondary (CRm) registers and opcode2 fields.
 */
#define CpTLBinvu   7           /* unified */
```

⟨*type CpTLB opcode2(arm)* 115c⟩≡ (607c)

```
#define CpTLBinv    0           /* invalidate all */
#define CpTLBinvse  1           /* invalidate single entry */
```

⟨*function* mmul2empty*(arm)* 115d⟩≡ (635a)

```
static void
mmul2empty(Proc* proc, bool clear)
{
    PTE *l1;
    Page **l2, *page;

    l1 = cpu->mmul1;
    l2 = &proc->mmul2;
    for(page = *l2; page != nil; page = page->next){
        if(clear)
            memset(UINT2PTR(page->va), 0, BY2PG);
        l1[page->daddr] = Fault;
        l2 = &page->next;
    }
    ⟨mmul2empty() remember free pages(arm) 115f⟩
}
```

⟨Proc Arch *memory fields(arm)* 115e⟩+≡ (30a) ◁114j

```
// list<ref<Page> > (next = Page.next)
Page*   mmul2cache; /* free mmu pages */
```

⟨mmul2empty() *remember free pages(arm)* 115f⟩≡ (115d)

```
*l2 = proc->mmul2cache;
proc->mmul2cache = proc->mmul2;
proc->mmul2 = nil;
```

⟨arch_mmurelease() *free l2 page cache(arm)* 115g⟩≡ (114m)

```
for(page = proc->mmul2cache; page != nil; page = next){
    next = page->next;
    if(--page->ref)
        panic("mmurelease: page->ref %d", page->ref);
    pagechainhead(page);
}
if(proc->mmul2cache && palloc.freememr.p)
    wakeup(&palloc.freememr);
proc->mmul2cache = nil;
```

⟨*function* mmul1empty*(arm)* 116a⟩≡ (635a)

```
static void
mmul1empty(void)
{
    memset(&cpu->mmul1[L1lo], 0, (L1hi - L1lo)*sizeof(PTE));
}
```

⟨Proc *memory fields* 116b⟩+≡ (29c) ◁65e 141b▷

```
bool newtlb;   /* Pager has changed my pte's, I must flush */
```

⟨*function* arch_flushmmu*(arm)* 116c⟩≡ (635a)

```
void
arch_flushmmu(void)
{
    int s;

    s = arch_splhi();
    up->newtlb = true;
    arch_mmuswitch(up);
    arch_splx(s);
}
```

8.4 Page fault

8.4.1 faultarm() (ARM)

⟨*function* faultarm*(arm)* 116d⟩≡ (636c)

```
/*
 *  called by trap to handle access faults
 */
static void
faultarm(Ureg *ureg, virt_addr va, bool user, bool read)
{
    bool insyscall;
    int n;
    char buf[ERRMAX];

    if(up == nil) {
        //dumpregs(ureg);
        panic("fault: nil up in faultarm, pc %#p accessing %#p", ureg->pc, va);
    }
    insyscall = up->insyscall;
    up->insyscall = true; // ???
    ⟨faultarm() if debug 505c⟩
    n = fault(va, read); // portable code

    if(n < 0){
        if(!user){
            dumpregs(ureg);
            panic("fault: kernel accessing %#p", va);
        }
        /* don't dump registers; programs suicide all the time */
        snprint(buf, sizeof buf, "sys: trap: fault %s va=%#p",
            read? "read": "write", va);
        postnote(up, 1, buf, NDebug);
    }
    up->insyscall = insyscall;
}
```

8.4.2 fault()

⟨*function* fault 117a⟩≡ (587b)

```
int
fault(virt_addr addr, bool read)
{
    Segment *s;
    char *sps;

    if(up == nil)
        panic("fault: nil up");
    if(up->nlocks.ref)
        print("fault: addr %#p: nlocks %ld\n", addr, up->nlocks.ref);

    sps = up->psstate;
    up->psstate = "Fault";
    arch_spllo();

    cpu->pfault++;
    for(;;) {
        s = seg(up, addr, /*dolock*/true); /* leaves s->lk qlocked if seg != nil */
        if(s == nil) {
            up->psstate = sps;
            return -1;
        }
        if(!read && (s->type&SG_RONLY)) {
            qunlock(&s->lk);
            up->psstate = sps;
            return -1;
        }

        if(fixfault(s, addr, read, /*putmmu*/true) == 0) /* qunlocks s->lk */
            break;

        // else? try again?
    }

    up->psstate = sps;
    return 0;
}
```

8.4.3 fixfault()

⟨*type* PTExxx 117b⟩≡ (582)

```
#define PTEVALID    (1<<0)
#define PTERONLY    0
#define PTEWRITE    (1<<1)
⟨PTExxx other cases 426b⟩
```

⟨*function* fixfault 117c⟩≡ (587b)

```
errorneg1
fixfault(Segment *s, virt_addr addr, bool read, bool doputmmu)
{
    int type;
    int ref;
    Pagetable **pde, *pt;
    Page **pte, *lkp, *new;
    ulong mmupte = nilptr;
    ulong soff;
```

```
addr &= ~(BY2PG-1);
soff = addr - s->base;

// walk
pde = &s->pagedir[soff/PAGETABMAPMEM];
if(*pde == nil)
    *pde = ptalloc();
pt = *pde;
pte = &pt->pagetab[(soff&(PAGETABMAPMEM-1))/BY2PG];
if(pte < pt->first)
    pt->first = pte;
if(pte > pt->last)
    pt->last = pte;

type = s->type&SG_TYPE;

switch(type) {
case SG_TEXT:           /* Demand load */
    ⟨fixfault() page in for SG_TEXT pte if pagedout 127d⟩

    mmupte = PPN((*pte)->pa) | PTERONLY|PTEVALID;
    (*pte)->modref = PG_REF;
    break;

case SG_BSS:
case SG_SHARED:         /* Zero fill on demand */
case SG_STACK:
    if(*pte == nil) {
        new = newpage(true, &s, addr); // true so clear! zeroed BSS & heap!
        if(s == nil) //?? when can be nil at exit?
            return ERROR_NEG1;
        *pte = new;
    }
    goto common;

case SG_DATA:
common:         /* Demand load/pagein/copy on write */
    ⟨fixfault() page in for SG_DATA or swapin (SG_BSS, etc) pte if pagedout 127e⟩

    ⟨fixfault() if read and copy on write, adjust mmupte and break 121d⟩
    // else

    lkp = *pte;

    lock(lkp);

    if(lkp->image == &swapimage)
        ref = lkp->ref + swapcount(lkp->daddr);
    else
        ref = lkp->ref;
    ⟨fixfault() if one ref and page has an image 129b⟩
    unlock(lkp);

    ⟨fixfault() if write and more than one ref then copy page 121e⟩

    mmupte = PPN((*pte)->pa) | PTEWRITE | PTEVALID;
    (*pte)->modref = PG_MOD|PG_REF;
    break;
```

```
    ⟨fixfault() SG_PHYSICAL case 426a⟩

    default:
        panic("fault");
        break;
    }
    qunlock(&s->lk);

    if(doputmmu)
        arch_putmmu(addr, mmupte, *pte);

    return OK_0;
}
```

8.4.4 arch_putmmu() (ARM)

⟨*function* arch_putmmu*(arm)* 119⟩≡ (635a)

```
void
arch_putmmu(virt_addr va, phys_addr pa, Page* page)
{
    int x;
    Page *pg;
    PTE *l1, *pte;

    x = L1X(va);
    l1 = &cpu->mmul1[x];
    if(*l1 == Fault){
        /* wasteful - l2 pages only have 256 entries - fix */
        ⟨arch_putmmu() if mmul2cache not empty(arm) 120a⟩
        else{
            /* auxpg since we don't need much? memset if so */
            pg = newpage(true, nil, 0);
            pg->va = VA(arch_kmap(pg));
        }
        pg->daddr = x;

        //add_list(pg, up->mmul2)
        pg->next = up->mmul2;
        up->mmul2 = pg;

        ⟨arch_putmmu() write back cache before fixing va fault(arm) 402j⟩

        *l1 = PPN(pg->pa)|Dom0|Coarse;
        ⟨arch_putmmu() write back cache after changing pde(arm) 402h⟩

        ⟨arch_putmmu() maintain mmul1lo and mmul1hi(arm) 120d⟩
    }
    pte = UINT2PTR(KADDR(PPN(*l1)));

    /* protection bits are
     *  PTERONLY|PTEVALID;
     *  PTEWRITE|PTEVALID;
     *  PTEWRITE|PTEUNCACHED|PTEVALID;
     */
    x = Small;
    ⟨arch_putmmu() if PTEUNCACHED, do not change x(arm) 426c⟩
    else
        x |= L2ptedramattrs;
```

```
    if(pa & PTEWRITE)
        x |= L2AP(Urw);
    else
        x |= L2AP(Uro);

    pte[L2X(va)] = PPN(pa)|x;
    ⟨arch_putmmu() write back cache after changing pte(arm) 402i⟩

    /* clear out the current entry */
    mmuinvalidateaddr(PPN(va));

    /*  write back dirty entries - we need this because the pio() in
     *  fault.c is writing via a different virt addr and won't clean
     *  its changes out of the dcache.  Page coloring doesn't work
     *  on this mmu because the virtual cache is set associative
     *  rather than direct mapped.
     */
    ⟨arch_putmmu() if PG_TXTFLUSH, invalidate cache(arm) 424a⟩
    arch_checkmmu(va, PPN(pa));
}
```

⟨arch_putmmu() *if mmul2cache not empty(arm)* 120a⟩≡ (119)

```
if(up->mmul2cache != nil){
    pg = up->mmul2cache;
    up->mmul2cache = pg->next;
    memset(UINT2PTR(pg->va), 0, BY2PG);
}
```

⟨*function* mmuinvalidateaddr*(arm)* 120b⟩≡ (635b)

```
/*
 * mmuinvalidateaddr(va)
 *   invalidate tlb entry for virtual page address va, ASID 0
 */
TEXT mmuinvalidateaddr(SB), 1, $-4
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinvse
    BARRIERS
    RET
```

⟨Cpu Arch *mmu fields(arm)* 120c⟩+≡ (28e) ◁114f 153f▷

```
// index in mmul1
int mmul1lo;
int mmul1hi;
```

⟨arch_putmmu() *maintain mmul1lo and mmul1hi(arm)* 120d⟩≡ (119)

```
if(x >= cpu->mmul1lo && x < cpu->mmul1hi){
    if(x+1 - cpu->mmul1lo < cpu->mmul1hi - x)
        cpu->mmul1lo = x+1;
    else
        cpu->mmul1hi = x;
}
```

⟨*function* arch_checkmmu*(arm)* 120e⟩≡ (635a)

```
void
arch_checkmmu(virt_addr va, phys_addr pa)
{
    USED(va);
    USED(pa);
}
```

⟨*function* arch_kmap*(arm)* 121a⟩≡ (635a)

```
Arch_KMap*
arch_kmap(Page *p) {
  return (Arch_KMap*)((p)->pa|KZERO);
}
```

⟨*function* arch_kunmap*(arm)* 121b⟩≡ (635a)

```
void
arch_kunmap(Arch_KMap *k)
{
    ⟨arch_kunmap() write back cache for page(arm) 402l⟩
}
```

8.5 Copy on write

⟨Conf *other fields* 121c⟩≡ (27a) 134b ▷

```
int copymode; /* 0 is copy on write, 1 is copy on reference */
```

⟨fixfault() *if read and copy on write, adjust mmupte and break* 121d⟩≡ (117c)

```
/*
 *  It's only possible to copy on write if
 *  we're the only user of the segment.
 */
if(read && conf.copymode == false && s->ref == 1) {
    mmupte = PPN((*pte)->pa)|PTERONLY|PTEVALID;
    (*pte)->modref |= PG_REF;
    break;
}
```

⟨fixfault() *if write and more than one ref then copy page* 121e⟩≡ (117c)

```
if(ref > 1){
    new = newpage(false, &s, addr);
    if(s == nil)
        return -1;
    *pte = new;
    copypage(lkp, *pte);
    putpage(lkp); //why??
}
```

⟨*function* copypage 121f⟩≡ (588)

```
void
copypage(Page *f, Page *t)
{
    Arch_KMap *ks, *kd;

    ks = arch_kmap(f);
    kd = arch_kmap(t);
    memmove((void*)VA(kd), (void*)VA(ks), BY2PG);
    arch_kunmap(ks);
    arch_kunmap(kd);
}
```

8.6 Page cache

⟨Page *other fields* 121g⟩≡ (57d) 142d ▷

```
// option<ref<Kimage>>
KImage  *image;     /* Associated binary image or swap image */
ulong daddr;      /* Disc address on image */
```

⟨*function* pghash 122a⟩≡ (582)

```
enum
{
    PGHLOG  = 9, // 2^9 = 512
    PGHSIZE = 1<<PGHLOG,  /* Page hash for image lookup */
};
#define pghash(daddr) palloc.hash[(daddr>>PGSHIFT)&(PGHSIZE-1)]
```

⟨Palloc *other fields* 122b⟩≡ (58a)

```
// hash<Page.daddr, ref<Page>> (next = Page.hash)>
Page  *hash[PGHSIZE];
Lock  hashlock;
```

⟨Page *extra fields* 122c⟩+≡ (57d) ◁58e

```
// hash<daddr, ref<Page>> Palloc.hash
Page  *hash; /* Image hash chains */
```

⟨*function* cachepage 122d⟩≡ (588)

```
void
cachepage(Page *p, KImage *i)
{
    Page **l; // list

    /* If this ever happens it should be fixed by calling
     * uncachepage instead of panic. I think there is a race
     * with pio in which this can happen. Calling uncachepage is
     * correct - I just wanted to see if we got here.
     */
    if(p->image)
        panic("cachepage");

    incref(i);
    lock(&palloc.hashlock);
    p->image = i;
    // add_hash(palloc.hash, p->daddr, p)
    l = &pghash(p->daddr);
    p->hash = *l;
    *l = p;
    unlock(&palloc.hashlock);
}
```

⟨*function* lookpage 122e⟩≡ (588)

```
Page *
lookpage(KImage *i, ulong daddr)
{
    Page *pg;

    lock(&palloc.hashlock);
    for(pg = pghash(daddr); pg; pg = pg->hash) {
        if(pg->image == i && pg->daddr == daddr) {
            unlock(&palloc.hashlock);

            lock(&palloc);
            lock(pg);
            if(pg->image != i || pg->daddr != daddr) { // race, not the one anymore
                unlock(pg);
                unlock(&palloc);
                return nil;
            }
            if(++pg->ref == 1)
```

```
                pageunchain(pg);
            unlock(&palloc);
            unlock(pg);

            return pg;
        }
    }
    unlock(&palloc.hashlock);

    return nil;
}
```

⟨putpage *if p has an image* 123a⟩≡ (59d)

```
if(p->image && p->image != &swapimage)
    pagechaintail(p);
```

⟨newpage() *uncachepage* 123b⟩≡ (58f)

```
uncachepage(p);
```

⟨*function* uncachepage 123c⟩≡ (588)

```
/* Always called with a locked page */
void
uncachepage(Page *p)
{
    Page **l, *f;

    if(p->image == nil)
        return;

    lock(&palloc.hashlock);
    // remove_hash(palloc.hash, p->daddr, p)
    l = &pghash(p->daddr);
    for(f = *l; f; f = f->hash) {
        if(f == p) {
            *l = p->hash;
            break;
        }
        l = &f->hash;
    }
    unlock(&palloc.hashlock);
    putimage(p->image);
    p->image = nil;
    p->daddr = 0;
}
```

⟨*function* cachedel 123d⟩≡ (588)

```
void
cachedel(KImage *i, ulong daddr)
{
    Page *f, **l;

    lock(&palloc.hashlock);
    l = &pghash(daddr);
    for(f = *l; f; f = f->hash) {
        if(f->image == i && f->daddr == daddr) {
            lock(f);
            // can have a race? things could have changed, so rested under lock
            if(f->image == i && f->daddr == daddr){
                *l = f->hash;
                putimage(f->image); // =~ decref
```

```
                f->image = nil;
                f->daddr = 0;
            }
            unlock(f);
            break;
        }
        l = &f->hash;
    }
    unlock(&palloc.hashlock);
}
```

8.7 Demand loading

8.7.1 Kimage and the image cache

⟨*struct* KImage 124a⟩≡ (582)

```
// a KImage is essentially a channel to an executable or swapfile
struct KImage
{
    Chan  *c;     /* channel to text file */
    bool  notext;     /* no file associated */ // for swapfile
    ⟨Kimage other fields 126a⟩

    // extra
    Ref;
    ⟨Kimage extra fields 124c⟩
};
```

⟨*struct* Imagealloc 124b⟩≡ (582)

```
// Image allocator (internal to segment.c, but important so here, singleton)
struct Imagealloc
{
    // hash<qid.path, ref<Kimage>> (next = Kimage.hash)
    KImage  *hash[IHASHSIZE];

    // list<ref<Kimage> (next = Kimage.next)
    KImage  *free; // originally  xalloc'ed in imageinit() (conf.nimage)
    QLock ireclaim; /* mutex on reclaiming free images */

    ⟨Imagealloc other fields 132a⟩

    // extra
    Lock;
};
```

⟨Kimage *extra fields* 124c⟩≡ (124a) 125b ▷

```
// list<ref<Kimage>> of Imagealloc.free
KImage  *next; /* Free list */
// hash<qid.path, ref<Kimage>> Imagealloc.hash
KImage  *hash; /* Qid hash chains */
```

⟨*function* ihash 124d⟩≡ (582)

```
#define IHASHSIZE 64
// actually internal to page.c, but important so here
#define ihash(qidpath)  imagealloc.hash[qidpath%IHASHSIZE]
```

⟨*global* imagealloc 124e⟩≡ (589)

```
static struct Imagealloc imagealloc;
```

8.7.2 Segment and KImage

⟨Segment *other fields* 125a⟩+≡ (61c) ◁64c 126b▷

```
KImage  *image;   /* text in file attached to this segment */
```

⟨Kimage *extra fields* 125b⟩+≡ (124a) ◁124c

```
// option<ref<Segment>>?
Segment *s;     /* TEXT segment for image if running */
```

⟨*constructor* attachimage 125c⟩≡ (589)

```
KImage*
attachimage(int type, Chan *c, virt_addr base, ulong len)
{
    KImage *img, **l;

    /* reclaim any free channels from reclaimed segments */
    if(imagealloc.nfreechan)
        imagechanreclaim();

    lock(&imagealloc);

    /*
     * Search the image cache for remains of the text from a previous
     * or currently running incarnation
     */
    for(img = ihash(c->qid.path); img; img = img->hash) {
        if(c->qid.path == img->qid.path) {
            lock(img);
            if(eqqid(c->qid, img->qid) &&
               eqqid(c->mqid, img->mqid) &&
               c->mchan == img->mchan &&
               c->type == img->type) {
                goto found;
            }
            unlock(img);
        }
    }

    /*
     * imagereclaim dumps pages from the free list which are cached by image
     * structures. This should free some image structures.
     */
    while(!(img = imagealloc.free)) {
        unlock(&imagealloc);
        imagereclaim();
        sched();
        lock(&imagealloc);
    }

    imagealloc.free = img->next;

    lock(img);
    incref(c);
    img->c = c;
    img->type = c->type;
    img->qid = c->qid;
    img->mqid = c->mqid;
    img->mchan = c->mchan;
    //add_hash(imagealloc.hash, c->qid.path, img)
    l = &ihash(c->qid.path);
```

```
    img->hash = *l;
    *l = img;

found:
    unlock(&imagealloc);

    if(img->s == nil) {
        /* Disaster after commit in exec */
        if(waserror()) {
            unlock(img);
            pexit(Enovmem, /*freemem*/true);
        }
        img->s = newseg(type, base, len);
        img->s->image = img;
        img->ref++;
        poperror();
    }
    else
        incref(img->s);

    return img;
}
```

⟨Kimage *other fields* 126a⟩≡ (124a)

```
Qid   qid;      /* Qid for page cache coherence */
Chan  *mchan;
Qid   mqid;
ushort  type;     /* Device type of owning channel */
```

⟨Segment *other fields* 126b⟩+≡ (61c) ◁125a 355b▷

```
ulong fstart;   /* start address in file for demand load */
ulong flen;   /* length of segment in file */
```

⟨sysexec() *locals* 126c⟩+≡ (83i) ◁87e 228e▷

```
KImage *img;
```

⟨sysexec() *get text segment ts via demand loading on tc* 126d⟩≡ (86a)

```
/* attachimage returns a locked cache image */
img = attachimage(SG_TEXT|SG_RONLY, tc, UTZERO, (t-UTZERO)>>PGSHIFT);
ts = img->s;
ts->fstart = 0;
ts->flen = sizeof(Exec)+text;
⟨sysexec() initialize other fields 424d⟩
unlock(img);
```

⟨sysexec() *adjust data segment s for demand loading on tc* 126e⟩≡ (86b)

```
/* Attached by hand */
incref(img);
s->image = img;
s->fstart = ts->fstart+ts->flen;
s->flen = data;
// data is also in binary
```

⟨dupseg() SG_DATA *case, attach image to new segment n* 126f⟩≡ (68f)

```
incref(s->image); // how sure non nil? data always attached to an img
n->image = s->image;
n->fstart = s->fstart;
n->flen = s->flen;
```

⟨dupseg() *when* SG_DATA, *if Text segment* 127a⟩≡ (68f)

```
if(segno == TSEG){ // why not SG_TEXT then?
    poperror();
    qunlock(&s->lk);
    return data2txt(s);// ????
}
```

⟨*function* data2txt 127b⟩≡ (589)

```
Segment*
data2txt(Segment *s)
{
    Segment *ps;

    ps = newseg(SG_TEXT, s->base, s->size);
    ps->image = s->image;
    incref(ps->image);
    ps->fstart = s->fstart;
    ps->flen = s->flen;
    ⟨data2txt() initialize other segment fields 424f⟩

    return ps;
}
```

8.7.3 Paging in

⟨*function* pagedout 127c⟩≡ (582)

```
#define pagedout(s) (((ulong)s)==0 || onswap(s))
```

⟨fixfault() *page in for* SG_TEXT *pte if pagedout* 127d⟩≡ (117c)

```
if(pagedout(*pte))
    pio(s, addr, soff, pte);
```

⟨fixfault() *page in for* SG_DATA *or swapin (*SG_BSS*, etc) pte if pagedout* 127e⟩≡ (117c)

```
if(pagedout(*pte))
    pio(s, addr, soff, pte);
```

⟨*function* pio 127f⟩≡ (587b)

```
void
pio(Segment *s, virt_addr addr, ulong soff, PageOrSwap **p)
{
    Page *new;
    Arch_KMap *k;
    Chan *c;
    int n, ask;
    char *kaddr;
    ulong daddr; // disk address
    Page *loadrec;

retry:
    loadrec = *p;

    if(loadrec == nil) {  /* from a text/data image */
        daddr = s->fstart+soff;
        new = lookpage(s->image, daddr);
        if(new != nil) {
            *p = new;
            return;
        }
```

```
    c = s->image->c;
    ask = s->flen-soff;
    if(ask > BY2PG)
        ask = BY2PG;
}else{          /* from a swap image */
    daddr = swapaddr(loadrec);
    new = lookpage(&swapimage, daddr);
    if(new != nil) {
        putswap(loadrec);
        *p = new;
        return;
    }
    c = swapimage.c;
    ask = BY2PG;
}
qunlock(&s->lk);

new = newpage(false, nil, addr);
k = arch_kmap(new);
kaddr = (char*)VA(k);

while(waserror()) {
    if(strcmp(up->errstr, Eintr) == 0)
        continue;
    arch_kunmap(k);
    putpage(new);
    faulterror(Eioload, c, false);
}

// reading the Page!! slow! which is why it's done without s->lk locked
n = devtab[c->type]->read(c, kaddr, ask, daddr);

if(n != ask)
    faulterror(Eioload, c, false);
if(ask < BY2PG)
    memset(kaddr+ask, 0, BY2PG-ask);

poperror();
arch_kunmap(k);
qlock(&s->lk);

if(loadrec == nil) {  /* This is demand load */
    /*
     *  race, another proc may have gotten here first while
     *  s->lk was unlocked
     */
    if(*p == nil) {
        new->daddr = daddr;
        cachepage(new, s->image);
        *p = new;
    }
    else
        putpage(new);
}else{          /* This is paged out */
    /*
     *  race, another proc may have gotten here first
     *  (and the pager may have run on that page) while
     *  s->lk was unlocked
     */
    if(*p != loadrec){
```

```
            if(!pagedout(*p)){
                /* another process did it for me */
                putpage(new);
                goto done; // =~ return, just need adjust cachectl before
            } else {
                /* another process and the pager got in */
                putpage(new);
                goto retry;
            }
        }

        new->daddr = daddr;
        cachepage(new, &swapimage);
        *p = new;
        putswap(loadrec);
    }
done:
⟨pio() set cachectl if flushme segment 424j⟩
}
```

⟨*function* faulterror 129a⟩≡ (587b)

```
static void
faulterror(char *s, Chan *c, bool freemem)
{
    char buf[ERRMAX];

    if(c && c->path){
        snprint(buf, sizeof buf, "%s accessing %s: %s", s, c->path->s, up->errstr);
        s = buf;
    }
    if(up->nerrlab) {
        postnote(up, 1, s, NDebug);
        error(s);
    }
    pexit(s, freemem);
}
```

8.7.4 duppage()

⟨fixfault() *if one ref and page has an image* 129b⟩≡ (117c)

```
if(ref == 1 && lkp->image){
    /* save a copy of the original for the image cache */
    duppage(lkp);
    ref = lkp->ref;
}
```

⟨*global* dupretries 129c⟩≡ (588)

```
static int dupretries = 15000;
```

⟨*function* duppage 129d⟩≡ (588)

```
/* Always call with p locked */
int
duppage(Page *p)
{
    Page *np;
    int retries;
 int color;

    retries = 0;
```

```
retry:

    if(retries++ > dupretries){
        print("duppage %d, up %p\n", retries, up);
        dupretries += 100;
        if(dupretries > 100000)
            panic("duppage\n");
        uncachepage(p);
        return 1;
    }


    /* don't dup pages with no image */
    if(p->ref == 0 || p->image == nil || p->image->notext)
        return 0;

    /*
     *  normal lock ordering is to call
     *  lock(&palloc) before lock(p).
     *  To avoid deadlock, we have to drop
     *  our locks and try again.
     */
    if(!canlock(&palloc)){
        unlock(p);
        if(up)
            sched();
        lock(p);
        goto retry;
    }

    /* No freelist cache when memory is very low */
    if(palloc.freecount < swapalloc.highwater) {
        unlock(&palloc);
        uncachepage(p);
        return 1;
    }

 color = getpgcolor(p->va);
 for(np = palloc.head; np; np = np->next)
  if(np->color == color)
   break;

 /* No page of the correct color */
 if(np == 0) {
  unlock(&palloc);
  uncachepage(p);
  return 1;
 }
    //when no color: np = palloc.head;

    pageunchain(np);
    pagechaintail(np);
/*
* XXX - here's a bug? - np is on the freelist but it's not really free.
* when we unlock palloc someone else can come in, decide to
* use np, and then try to lock it.  they succeed after we've
* run copypage and cachepage and unlock(np).  then what?
* they call pageunchain before locking(np), so it's removed
* from the freelist, but still in the cache because of
* cachepage below.  if someone else looks in the cache
```

```
 * before they remove it, the page will have a nonzero ref
 * once they finally lock(np).
 */
     lock(np);
     unlock(&palloc);

     /* Cache the new version */
     uncachepage(np);
     np->va = p->va;
     np->daddr = p->daddr;
     copypage(p, np);
     cachepage(np, p->image);
     unlock(np);
     uncachepage(p);

     return 0;
 }
```

8.7.5 Collecting images

⟨putseg() *other locals* 131a⟩≡ (64a)

```
 KImage *img;
```

⟨putseg() *if s has an image* 131b⟩≡ (64a)

```
 img = s->image;
 if(img != nil) {
     lock(img);
     lock(s);
     if(img->s == s && s->ref == 1) // race?
         img->s = nil;
     unlock(img);
 }
```

⟨putseg() *if s had an image* 131c⟩≡ (64a)

```
 if(img)
     putimage(img);
```

⟨*destructor* putimage 131d⟩≡ (589)

```
 void
 putimage(KImage *img)
 {
     Chan *c, **cp;
     KImage *f, **l;

     if(img->notext)
         return;

     lock(img);
     if(--img->ref == 0) {
         l = &ihash(img->qid.path);
         mkqid(&img->qid, ~0, ~0, QTFILE);
         unlock(img);
         c = img->c;

         lock(&imagealloc);
         //remove_hash(imagealloc.hash, img)
         for(f = *l; f; f = f->hash) {
             if(f == img) {
                 *l = img->hash;
```

```
                break;
            }
            l = &f->hash;
        }

        //add_list(imagealloc.free, img)
        img->next = imagealloc.free;
        imagealloc.free = img;

        /* defer freeing channel till we're out of spin lock's */

        if(imagealloc.nfreechan == imagealloc.szfreechan){
            //realloc(imagealloc.freenchan, szfreechan+NFREECHAN)
            imagealloc.szfreechan += NFREECHAN;
            cp = malloc(imagealloc.szfreechan*sizeof(Chan*));
            if(cp == nil)
                panic("putimage");
            memmove(cp, imagealloc.freechan, imagealloc.nfreechan*sizeof(Chan*));
            free(imagealloc.freechan);
            imagealloc.freechan = cp;
        }

        imagealloc.freechan[imagealloc.nfreechan++] = c;
        unlock(&imagealloc);

        return;
    }
    unlock(img);
}
```

⟨Imagealloc *other fields* 132a⟩≡ (124b)

```
Chan  **freechan; /* free image channels */
int nfreechan;  /* number of free channels */
int szfreechan; /* size of freechan array */
QLock fcreclaim;  /* mutex on reclaiming free channels */
```

⟨*function* imagechanreclaim 132b⟩≡ (589)

```
/*
 *  since close can block, this has to be called outside of
 *  spin locks.
 */
static void
imagechanreclaim(void)
{
    Chan *c;

    /* Somebody is already cleaning the image chans */
    if(!canqlock(&imagealloc.fcreclaim))
        return;

    /*
     * We don't have to recheck that nfreechan > 0 after we
     * acquire the lock, because we're the only ones who decrement
     * it (the other lock contender increments it), and there's only
     * one of us thanks to the qlock above.
     */
    while(imagealloc.nfreechan > 0){
        lock(&imagealloc);
        imagealloc.nfreechan--;
        c = imagealloc.freechan[imagealloc.nfreechan];
```

```
        unlock(&imagealloc);
        cclose(c);
    }

    qunlock(&imagealloc.fcreclaim);
}
```

⟨*function* imagereclaim 133a⟩≡ (589)

```
static void
imagereclaim(void)
{
    int n;
    Page *p;
    uvlong ticks; // fastticks

    irstats.calls++;
    /* Somebody is already cleaning the page cache */
    if(!canqlock(&imagealloc.ireclaim))
        return;

    lock(&palloc);
    ticks = arch_fastticks(nil);
    n = 0;
    /*
     * All the pages with images backing them are at the
     * end of the list (see putpage) so start there and work
     * backward.
     */
    for(p = palloc.tail; p && p->image && n<1000; p = p->prev) {
        if(p->ref == 0 && canlock(p)) {
            if(p->ref == 0) {
                n++;
                uncachepage(p); // will call putimage()
            }
            unlock(p);
        }
    }
    ticks = arch_fastticks(nil) - ticks;
    unlock(&palloc);
    irstats.loops++;
    irstats.ticks += ticks;
    if(ticks > irstats.maxt)
        irstats.maxt = ticks;
    //print("T%llud+", ticks);
    qunlock(&imagealloc.ireclaim);
}
```

8.8 Swapping

8.8.1 Swapalloc

⟨*struct* Swapalloc 133b⟩≡ (582)

```
// Swap allocator (singleton)
struct Swapalloc
{
    // array<byte> xalloc'ed in swapinit()
    // each idx represents a chunk of swapimage, each value a ref count
    byte*  swmap;      /* Base of swap map in memory */
```

```
    int free;     /* currently free swap pages */

    // ref<byte> in swmap
    byte*  alloc;     /* Round robin allocator */
    // ref<byte> in swmap
    byte*  top;      /* Top of swap map */

    // ref<byte> in swmap
    byte*  last;     /* Speed swap allocation */

    ulong highwater;    /* Pager start threshold */ // = 5% conf.upages
    ulong headroom;   /* Space pager frees under highwater */ // = 1.25*hw

    //extra
    Lock;       /* Free map lock */
    Rendez r;      /* Pager kproc idle sleep */ // needpages()
};
```

⟨*global* swapalloc 134a⟩≡ (584a)

```
// Swap allocator
struct Swapalloc swapalloc;
```

⟨Conf *other fields* 134b⟩+≡ (27a) ◁121c 137a▷

```
ulong nswap;    /* number of swap pages */
```

⟨*global* swapimage 134c⟩≡ (584a)

```
KImage  swapimage;
```

⟨*function* newswap 134d⟩≡ (590)

```
ulong
newswap(void)
{
    byte *look;

    lock(&swapalloc);

    if(swapalloc.free == 0){
        unlock(&swapalloc);
        return ~0; //???
    }

    look = memchr(swapalloc.last, 0, swapalloc.top-swapalloc.last);
    if(look == nil)
        panic("inconsistent swap"); // swapalloc.free != 0, should find a page

    *look = 1;
    swapalloc.last = look;
    swapalloc.free--;
    unlock(&swapalloc);
    return (look-swapalloc.swmap) * BY2PG; // offset in swapfile
}
```

8.8.2 /dev/swap

⟨consread() *cases* 134e⟩≡ (288d) 182b▷

```
case Qswap:
    snprint(tmp, sizeof tmp,
        "%lud memory\n"
```

```
        "%d pagesize\n"
        "%lud kernel\n"
        "%lud/%lud user\n"
        "%lud/%lud swap\n"
        "%lud/%lud kernel malloc\n"
        "%lud/%lud kernel draw\n",
        conf.npage*BY2PG,
        BY2PG,
        conf.npage-conf.upages,
        palloc.user - palloc.freecount, palloc.user,
        conf.nswap - swapalloc.free, conf.nswap,
        mainmem->cursize, mainmem->maxsize,
        imagmem->cursize, imagmem->maxsize);

    return readstr((ulong)offset, buf, n, tmp);
```

8.8.3 PageOrSwap

⟨*constant* PG_ONSWAP 135a⟩≡ (582)

```
#define PG_ONSWAP 1
```

⟨*type* PageOrSwap 135b⟩≡ (582)

```
typedef Page PageOrSwap;
```

⟨*function* onswap 135c⟩≡ (582)

```
#define onswap(s) (((kern_addr)s)&PG_ONSWAP)
```

⟨*function* putswap 135d⟩≡ (590)

```
void
putswap(Page *p)
{
    byte *idx;

    lock(&swapalloc);
    idx = &swapalloc.swmap[((ulong)p)/BY2PG];
    if(--(*idx) == 0) {
        swapalloc.free++;
        if(idx < swapalloc.last)
            swapalloc.last = idx;
    }
    if(*idx >= 254)
        panic("putswap %#p == %ud", p, *idx);
    unlock(&swapalloc);
}
```

⟨putpage *if p is a swap address* 135e⟩≡ (59d)

```
    if(onswap(p)) {
        putswap(p);
        return;
    }
```

⟨mfreeseg *if pg is a swap address* 135f⟩≡ (71d)

```
if(onswap(pg))
    putswap(pg);
```

⟨*function* dupswap 136a⟩≡ (590)

```
void
dupswap(Page *p)
{
    lock(&swapalloc);
    if(++swapalloc.swmap[((ulong)p)/BY2PG] == 0)
        panic("dupswap");
    unlock(&swapalloc);
}
```

⟨ptcpy() *if src is a swap page* 136b⟩≡ (63a)

```
if(onswap(*src))
    dupswap(*src);
```

⟨*function* swapcount 136c⟩≡ (590)

```
int
swapcount(ulong daddr)
{
    return swapalloc.swmap[daddr/BY2PG];
}
```

8.8.4 Swapping plan

⟨*function* pagepte 136d⟩≡ (590)

```
static void
pagepte(int type, Page **pte)
{
    ulong daddr;
    Page *outp;

    outp = *pte;
    switch(type) {
    case SG_TEXT:               /* Revert to demand load */
        putpage(outp);
        *pte = nil;
        break;

    case SG_DATA:
    case SG_BSS:
    case SG_STACK:
    case SG_SHARED:
        /*
         *  get a new swap address and clear any pages
         *  referring to it from the cache
         */
        daddr = newswap();
        if(daddr == ~0)
            break; // return;
        cachedel(&swapimage, daddr);

        lock(outp);

        /* forget anything that it used to cache */
        uncachepage(outp);

        /*
         *  incr the reference count to make sure it sticks around while
         *  being written
         */
```

```
        outp->ref++;

        /*
         *  enter it into the cache so that a fault happening
         *  during the write will grab the page from the cache
         *  rather than one partially written to the disk
         */
        outp->daddr = daddr;
        cachepage(outp, &swapimage);
        *pte = (Page*)(daddr|PG_ONSWAP); // turn a Page pte into a Swap pte!
        unlock(outp);

        /* Add page to IO transaction list */
        iolist[ioptr++] = outp;
        break;
    }
}
```

⟨Conf *other fields* 137a⟩+≡ (27a) ◁134b 192a▷

```
int nswppo;   /* max # of pageouts per segment pass */
```

⟨*global* iolist 137b⟩≡ (590)

```
// array<option<ref<Page>>>, xalloc'ed in swapinit(), size = Conf.nswppo
static  Page    **iolist;
```

⟨*global* ioptr 137c⟩≡ (590)

```
// index in iolist
static  int ioptr;
```

⟨*function* pageout 137d⟩≡ (590)

```
static void
pageout(Proc *p, Segment *s)
{
    int type, i, size;
    ulong age;
    Pagetable *pt;
    Page **pg, *entry;

    if(!canqlock(&s->lk))   /* We cannot afford to wait, we will surely deadlock */
        return;

    if(s->steal) {      /* Protected by /dev/proc */
        qunlock(&s->lk);
        return;
    }

    if(!canflush(p, s)) {   /* Able to invalidate all tlbs with references */
        qunlock(&s->lk);
        putseg(s);
        return;
    }

    if(waserror()) {
        qunlock(&s->lk);
        putseg(s);
        return;
    }

    /* Pass through the tables looking for memory pages to swap out */
    type = s->type&SG_TYPE;
```

```
    size = s->pagedirsize;
    for(i = 0; i < size; i++) {
        pt = s->pagedir[i];
        if(pt == nil)
            continue;
        for(pg = pt->first; pg < pt->last; pg++) {
            entry = *pg;
            if(pagedout(entry)) // already swapped, or nil
                continue;

            if(entry->modref & PG_REF) {
                entry->modref &= ~PG_REF;
                entry->gen = genclock;
            }


            if(genclock < entry->gen)
                age = ~(entry->gen - genclock);
            else
                age = genclock - entry->gen;
            gensum += age;
            gencount++;
            if(age <= genage)
                continue;


            pagepte(type, pg);

            if(ioptr >= conf.nswppo)
                goto out;
        }
    }
 out:
    poperror();
    qunlock(&s->lk);
    putseg(s);
 }
```

8.8.5 The pager

⟨newpage() *other locals* 138a⟩+≡ (58f) ◁59a 423h▷

```
 bool dontalloc;
```

⟨newpage() *loop waiting freecount ¿ highwater* 138b⟩≡ (58f)

```
 for(;;) {
    if(palloc.freecount > swapalloc.highwater)
        break;
    if(up->kp && palloc.freecount > 0)
        break;

    // in highwater, not so many free pages, need to wait

    unlock(&palloc);
    dontalloc = false;
    if(s && *s) {
        qunlock(&((*s)->lk));
        *s = nil;// !!
        dontalloc = true;
    }
```

```
    qlock(&palloc.pwait);   /* Hold memory requesters here */

    while(waserror())   /* Ignore interrupts */
        ;

    kickpager();
    tsleep(&palloc.freememr, hasfreepages, 0, 1000);

    poperror();
    qunlock(&palloc.pwait);

    /*
     * If called from fault and we lost the segment from
     * underneath don't waste time allocating and freeing
     * a page. Fault will call newpage again when it has
     * reacquired the segment locks
     */
    if(dontalloc)
        return nil;

    lock(&palloc);
}
```

⟨Palloc *extra fields* 139a⟩≡ (58a)

```
Rendez  freememr; /* Sleep for free mem */ // hasfreepages()
QLock pwait; /* Queue of procs waiting for memory */
```

⟨putpage *wakeup process waiting for freepage if any* 139b⟩≡ (59d)

```
if(palloc.freememr.p != nil)
    wakeup(&palloc.freememr);
```

⟨*function* hasfreepages 139c⟩≡ (588)

```
int
hasfreepages(void*)
{
    return palloc.freecount >= swapalloc.highwater;
}
```

⟨*function* kickpager 139d⟩≡ (590)

```
void
kickpager(void)
{
    static bool started;

    if(started)
        wakeup(&swapalloc.r);
    else {
        kproc("kpager", pager, nil);
        started = true;
    }
}
```

⟨*function* pager 139e⟩≡ (590)

```
static void
pager(void *junk)
{
    int i;
    Segment *s;
    Proc *p, *ep;
```

```
    if(waserror())
        panic("pager: os error");

    p = proctab(0);
    ep = &p[conf.nproc];

loop:
    up->psstate = "Idle";
    wakeup(&palloc.freememr);
    sleep(&swapalloc.r, needpages, nil);

    while(needpages(junk)) {
        if(swapimage.c) {
            p++;
            if(p >= ep){
                p = proctab(0);
                gentick();
            }

            if(p->state == Dead || p->noswap)
                continue;

            if(!canqlock(&p->seglock))
                continue;       /* process changing its segments */

            for(i = 0; i < NSEG; i++) {
                if(!needpages(junk)){
                    qunlock(&p->seglock);
                    goto loop;
                }

                if(s = p->seg[i]) {
                    switch(s->type&SG_TYPE) {
                    case SG_TEXT:
                        pageout(p, s);
                        break;
                    case SG_DATA:
                    case SG_BSS:
                    case SG_STACK:
                    case SG_SHARED:
                        up->psstate = "Pageout";
                        pageout(p, s);
                        if(ioptr != 0) {
                            up->psstate = "I/O";
                            executeio();
                        }
                        break;
                    default:
                        break;
                    }
                }
            }
            qunlock(&p->seglock);
        } else {
            print("out of memory\n");
            killbig("out of memory");
            freebroken();       /* can use the memory */

            /* Emulate the old system if no swap channel */
            if(!swapimage.c)
```

```
                tsleep(&up->sleepr, returnfalse, 0, 5000);
            }
        }
        goto loop;
    }
```

⟨*function* needpages 141a⟩≡ (590)

```
    static bool
    needpages(void*)
    {
        return palloc.freecount < swapalloc.headroom;
    }
```

⟨Proc *memory fields* 141b⟩+≡ (29c) ◁116b

```
    bool noswap;   /* process is not swappable */
```

⟨newproc() *set fields of* p 141c⟩+≡ (75b) ◁98c 145d▷

```
    p->noswap = false;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 141d⟩+≡ (78d) ◁101c 145g▷

```
    p->noswap = up->noswap;
```

⟨procctlreq() *CMnoswap case* 141e⟩≡ (482)

```
    case CMnoswap:
        p->noswap = true;
        break;
```

⟨*function* canflush 141f⟩≡ (590)

```
    static bool
    canflush(Proc *p, Segment *s)
    {
        int i;
        Proc *ep;

        lock(s);
        if(s->ref == 1) {       /* Easy if we are the only user */
            s->ref++;
            unlock(s);
            return canpage(p);
        }
        s->ref++;
        unlock(s);

        /* Now we must do hardwork to ensure all processes which have tlb
         * entries for this segment will be flushed if we succeed in paging it out
         */
        p = proctab(0);
        ep = &p[conf.nproc];
        while(p < ep) {
            if(p->state != Dead) {
                for(i = 0; i < NSEG; i++)
                    if(p->seg[i] == s)
                        if(!canpage(p))
                            return false;
            }
            p++;
        }
        return true;
    }
```

⟨*function* canpage 142a⟩≡ (599c)

```
int
canpage(Proc *p)
{
    bool ok = false;

    arch_splhi();
    lock(runq);
    /* Only reliable way to see if we are Running */
    if(p->cpu == nil) {
        p->newtlb = true;
        ok = true;
    }
    unlock(runq);
    arch_spllo();

    return ok;
}
```

⟨*function* gentick 142b⟩≡ (590)

```
static void
gentick(void)
{
    genclock++;
    if(gencount)
        genage = gensum / gencount;
    else
        genage = 0;
    gensum = gencount = 0;
}
```

⟨*global* genxxx 142c⟩≡ (590)

```
static  ulong   genage, genclock, gencount;
static  uvlong  gensum;
```

⟨Page *other fields* 142d⟩+≡ (57d) ◁121g 423a▷

```
ulong gen;      /* Generation counter for swap */
```

8.8.6 Paging out

⟨*function* executeio 142e⟩≡ (590)

```
static void
executeio(void)
{
    Page *out;
    int i, n;
    Chan *c;
    char *kaddr;
    Arch_KMap *k;

    c = swapimage.c;
    qsort(iolist, ioptr, sizeof iolist[0], pageiocomp);
    for(i = 0; i < ioptr; i++) {
        if(ioptr > conf.nswppo)
            panic("executeio: ioptr %d > %d", ioptr, conf.nswppo);
        out = iolist[i];
        k = arch_kmap(out);
        kaddr = (char*)VA(k);
```

```
        if(waserror())
            panic("executeio: page out I/O error");

        n = devtab[c->type]->write(c, kaddr, BY2PG, out->daddr);// swap 1 page
        if(n != BY2PG)
            nexterror();

        arch_kunmap(k);
        poperror();

        /* Free up the page after I/O */
        lock(out);
        out->ref--;
        unlock(out);
        putpage(out);
    }
    ioptr = 0;
}
```

⟨*function* pageiocomp 143a⟩≡ (590)

```
static int
pageiocomp(void *a, void *b)
{
    Page *p1, *p2;

    p1 = *(Page **)a;
    p2 = *(Page **)b;
    if(p1->daddr > p2->daddr)
        return 1;
    else
        return -1;
}
```

8.8.7 Paging in

⟨*function* swapaddr 143b⟩≡ (582)

```
#define swapaddr(s) (((ulong)s)&~PG_ONSWAP)
```

8.8.8 Setting up the swap file

⟨conswrite() *cases* 143c⟩≡ (289) 182d ▷

```
case Qswap:
    if(n >= sizeof buf)
        error(Egreg);
    memmove(buf, va, n);    /* so we can NUL-terminate */
    buf[n] = 0;
    /* start a pager if not already started */
    if(strncmp(buf, "start", 5) == 0){
        kickpager();
        break;
    }
    if(!iseve())
        error(Eperm);
    if(buf[0]<'0' || '9'<buf[0])
        error(Ebadarg);
    fd = strtoul(buf, 0, 0);
    swc = fdtochan(fd, -1, true, true);
    setswapchan(swc);
    break;
```

⟨*function* setswapchan 144⟩≡ (590)

```
void
setswapchan(Chan *c)
{
    byte dirbuf[sizeof(DirEntry)+100];
    DirEntry d;
    int n;

    if(swapimage.c) {
        if(swapalloc.free != conf.nswap){
            cclose(c);
            error(Einuse);
        }
        cclose(swapimage.c);
    }

    /*
     *  if this isn't a file, set the swap space
     *  to be at most the size of the partition
     */
    if(devtab[c->type]->dc != L'M'){
        n = devtab[c->type]->stat(c, dirbuf, sizeof dirbuf);
        if(n <= 0){
            cclose(c);
            error("stat failed in setswapchan");
        }
        convM2D(dirbuf, n, &d, nil);
        if(d.length < conf.nswap*BY2PG){
            conf.nswap = d.length/BY2PG;
            swapalloc.top = &swapalloc.swmap[conf.nswap];
            swapalloc.free = conf.nswap;
        }
    }

    swapimage.c = c;
}
```

Chapter 9

Scheduling

9.1 Overview

9.2 Process priority

⟨*constant* Npriq 145a⟩≡ (592c)
```
Npriq   = 20,   /* number of scheduler priority levels */
```

⟨*enum* priority 145b⟩≡ (592c)
```
enum Priority
{
    PriNormal = 10,   /* base priority for normal processes */
    PriKproc  = 13,   /* base priority for kernel processes */
    ⟨Priority cases 146d⟩
    ⟨Priority cases for real-time priority 431c⟩
};
```

⟨Proc *scheduling fields* 145c⟩≡ (29c) 145f▷
```
// enum<Priority>
ulong priority; /* priority level */
```

⟨newproc() *set fields of* p 145d⟩+≡ (75b) ◁141c 149d▷
```
/* sched params */
procpriority(p, PriNormal, false);
```

⟨*function* procpriority 145e⟩≡ (599c)
```
void
procpriority(Proc *p, int pri, bool fixed)
{
    ⟨procpriority() sanity check pri 146a⟩
    p->priority = pri;

    p->basepri = pri;
    p->fixedpri = fixed;
}
```

⟨Proc *scheduling fields* 145f⟩+≡ (29c) ◁145c 149c▷
```
// enum<Priority>
ulong basepri;  /* base priority level */
bool fixedpri; /* priority level doesn't change */
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 145g⟩+≡ (78d) ◁141d 151c▷
```
p->priority = up->basepri;
p->basepri = up->basepri;
p->fixedpri = up->fixedpri;
```

⟨procpriority() *sanity check pri* 146a⟩≡ (145e)

```
if(pri >= Npriq)
    pri = Npriq - 1;
else if(pri < 0)
    pri = 0;
```

⟨procctlreq() *CMpri case* 146b⟩≡ (482)

```
case CMpri:
    pri = atoi(cb->f[1]);
    if(pri > PriNormal && !iseve())
        error(Eperm);
    procpriority(p, pri, false);
    break;
```

⟨procctlreq() *CMfixedpri case* 146c⟩≡ (482)

```
case CMfixedpri:
    pri = atoi(cb->f[1]);
    if(pri > PriNormal && !iseve())
        error(Eperm);
    procpriority(p, pri, true);
    break;
```

⟨Priority *cases* 146d⟩≡ (145b)

```
PriRoot   = 13,   /* base priority for root processes */
```

⟨sysexec() *set prioriry of root processes* 146e⟩≡ (83i)

```
/*
 *  '/' processes are higher priority (hack to make /ip more responsive).
 */
if(devtab[tc->type]->dc == L'/')
    up->basepri = PriRoot;
up->priority = up->basepri;
```

9.3 The runqueue

⟨*constant* Nrq 146f⟩≡ (592c)

```
Nrq   = Npriq+2,  /* number of priority levels including real time */
```

⟨*global* runq 146g⟩≡ (599c)

```
// The run queue!!
// hash<enum<priority>, queue<ref<Proc>>>
Schedq  runq[Nrq];
```

⟨*struct* Schedq 146h⟩≡ (592c)

```
// essentially a queue<ref<Proc>>
struct Schedq
{
    // list<ref<Proc>> (next = Proc.rnext)
    Proc* head;
    // ref<Proc>, the tail
    Proc* tail;
    // size of list
    int n;

    // extra
    Lock; // protects also the globals nrdy and runvec
};
```

⟨Proc *extra fields* 147a⟩+≡ (29c) ◁74b 175e▷

```
// list<ref<Proc>> of Schedq.head
Proc  *rnext;   /* next process in run queue */
```

⟨*global* runveq 147b⟩≡ (599c)

```
// array<bool>, each bit i represents whether the runq at pri i has processes
ulong   runvec; // coupling: sizeof(ulong) must be >= Nrq
```

⟨*function* anyready 147c⟩≡ (599c)

```
bool
anyready(void)
{
    return runvec;
}
```

⟨*global* nrdy 147d⟩≡ (599c)

```
int nrdy;
```

⟨*function* queueproc 147e⟩≡ (599c)

```
/*
 * add a process to a scheduling queue
 */
void
queueproc(Schedq *rq, Proc *p)
{
    int pri;

    pri = rq - runq;
    lock(runq);
    p->priority = pri; // because a priority can evolve

    // add_queue(p, rq)
    p->rnext = nil;
    if(rq->tail)
        rq->tail->rnext = p;
    else
        rq->head = p;
    rq->tail = p;
    rq->n++;

    nrdy++;
    runvec |= 1<<pri;
    unlock(runq);
}
```

⟨*function* dequeueproc 147f⟩≡ (599c)

```
/*
 *  try to remove a process from a scheduling queue (called arch_splhi)
 */
Proc*
dequeueproc(Schedq *rq, Proc *tp)
{
    Proc *l, *p;

    if(!canlock(runq))
        return nil;

    /*
     *  the queue may have changed before we locked runq,
     *  refind the target process.
```

```
     */
    l = nil;
    for(p = rq->head; p; p = p->rnext){
        if(p == tp)
            break;
        l = p;
    }
    ⟨dequeueproc() return if process not found or already associated 148a⟩

    // remove_queue(p, rq)
    if(p->rnext == nil)
        rq->tail = l;
    if(l)
        l->rnext = p->rnext;
    else
        rq->head = p->rnext;

    if(rq->head == nil)
        runvec &= ~(1<<(rq-runq));
    rq->n--;
    nrdy--;
    ⟨dequeueproc() sanity check p 148b⟩
    unlock(runq);
    return p;
}
```

⟨dequeueproc() *return if process not found or already associated* 148a⟩≡ (147f)

```
/*
 *  p->cpu==nil only when process state is saved
 */
if(p == nil || p->cpu){
    unlock(runq);
    return nil;
}
```

⟨dequeueproc() *sanity check p* 148b⟩≡ (147f)

```
if(p->state != Ready)
    print("dequeueproc %s %lud %s\n", p->text, p->pid, statename[p->state]);
```

9.4 The scheduler

9.4.1 sched()

⟨*function* sched 148c⟩≡ (599c)

```
/*
 *  If changing this routine, look also at sleep().  It
 *  contains a copy of the guts of sched().
 */
void
proc_sched(void)
{
    ⟨sched() locals 149a⟩

    ⟨sched() sanity check ilockdepth 149e⟩
    ⟨sched() if up 152a⟩
    // else
    ⟨sched() if no up 149b⟩
}
```

⟨sched() *locals* 149a⟩≡ (148c)

```
Proc *p;
```

⟨sched() *if no up* 149b⟩≡ (148c)

```
// We should execute this code using the main kernel stack, as
// we should arrive here from schedinit()

p = runproc();
⟨sched() after found process p to schedule, possibly update priority 161b⟩
⟨sched() if readied process, do not adjust schedticks 159d⟩
else {
    ⟨sched() after found a new proc, adjust schedticks 159a⟩
}
⟨sched() reset readied 156d⟩

cpu->proc = p;
up = p; // same instruction than previous line on some archi (e.g. x86)

up->state = Running;
up->cpu = CPUS(cpu->cpuno);

arch_mmuswitch(up);
arch_gotolabel(&up->sched);
```

⟨Proc *scheduling fields* 149c⟩+≡ (29c) ◁145f 151a▷

```
// option<ref<Cpu>>, null when not associated to a processor
Cpu  *cpu;    /* processor running this proc */
```

⟨newproc() *set fields of* p 149d⟩+≡ (75b) ◁145d 151b▷

```
p->cpu = nil;
```

⟨sched() *sanity check ilockdepth* 149e⟩≡ (148c)

```
if(cpu->ilockdepth)
    panic("cpu%d: ilockdepth %d, last lock %#p at %#p, sched called from %#p",
        cpu->cpuno, cpu->ilockdepth, up? up->lastilock: nil,
        (up && up->lastilock)? up->lastilock->pc: 0, getcallerpc(&p+2));
```

9.4.2 schedinit()

⟨Cpu *other fields* 149f⟩+≡ (28a) ◁46d 156b▷

```
Label sched;      /* scheduler wakeup */ // address of schedinit()
```

⟨*function* schedinit 149g⟩≡ (599c)

```
/*
 * Always arch_splhi()'ed.
 */
void
schedinit(void)     /* never returns */
{
    ⟨schedinit() locals 432e⟩

    arch_setlabel(&cpu->sched);
    ⟨schedinit() after setlabel, if up 152b⟩
    // ok at this point up is nil
    sched();
}
```

9.4.3 runproc()

⟨*function* runproc 150⟩≡ (599c)

```
/*
 *  pick a process to run
 */
Proc*
runproc(void)
{
    Proc *p;
    Schedq *rq;
    int i;
    ⟨runproc() other locals 377d⟩

    ⟨runproc() start 512d⟩
    ⟨runproc() cooperative scheduling if process readied 156c⟩
    //else

    preempts++;

loop:
    /*
     *  find a process that last ran on this processor (affinity),
     *  or one that hasn't moved in a while (load balancing). Every
     *  time around the loop affinity goes down.
     */
    arch_spllo(); // from schedinit()

    for(i = 0;; i++){
        /*
         *  find the highest priority target process that this
         *  processor can run given affinity constraints.
         *
         */
        for(rq = &runq[Nrq-1]; rq >= runq; rq--){
            for(p = rq->head; p; p = p->rnext){
                if(p->lastcpu == nil || p->lastcpu == CPUS(cpu->cpuno)
                   || (p->wired == nil && i > 0)) // favor affinity for first round
                    goto found;
            }
        }
        // nothing found
        /* waste time or halt the CPU */
        arch_idlehands();
        ⟨runproc() after arch_idlehands, before restarting loop 512e⟩
    }

found:
    arch_splhi();
    p = dequeueproc(rq, p);
    if(p == nil) // the runq may have changed between spllo and splhi
        goto loop;

    p->state = Scheding;
    p->lastcpu = CPUS(cpu->cpuno);

    ⟨runproc() test if p is a real-time process 432b⟩
    ⟨runproc() hook proctrace 377e⟩
    return p;
}
```

⟨Proc *scheduling fields* 151a⟩+≡ (29c) ◁149c 153a▷
```
  Cpu *lastcpu;    /* processor this process last ran on */
```

⟨newproc() *set fields of* p 151b⟩+≡ (75b) ◁149d 151f▷
```
  p->lastcpu = nil;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 151c⟩+≡ (78d) ◁145g 335d▷
```
  p->lastcpu = up->lastcpu;
```

9.5 Context switch

9.5.1 Label

⟨*struct* Label 151d⟩≡ (533)
```
  // =~ a jumpbuf in C, for coroutines
  struct Label
  {
      // or virt_addr? used also for saving context of user code?
      kern_addr sp;
      kern_addr pc;
  };
```

⟨Proc *assembly fields* 151e⟩≡ (29c)
```
  Label sched;    /* known to l.s */
  char  *kstack;  /* known to l.s */
```

9.5.2 Kernel stacks

⟨newproc() *set fields of* p 151f⟩+≡ (75b) ◁151b 153b▷
```
  if(p->kstack == nil)
      p->kstack = smalloc(KSTACK);
```

⟨*constant* KSTACK*(arm)* 151g⟩≡ (633b)
```
  #define KSTACK      (8*KiB)
```

9.5.3 arch_gotolabel() (ARM)

⟨*function* arch_gotolabel*(arm)* 151h⟩≡ (643b)
```
  TEXT arch_gotolabel(SB), 1, $-4
      MOVW    0(R0), R13      /* sp */
      MOVW    4(R0), R14      /* pc */
      MOVW    $1, R0
      RET
```

⟨*function* arch_setlabel*(arm)* 151i⟩≡ (643b)
```
  TEXT arch_setlabel(SB), 1, $-4
      MOVW    R13, 0(R0)      /* sp */
      MOVW    R14, 4(R0)      /* pc */
      MOVW    $0, R0
      RET
```

9.5.4 sched() and schedinit() dance

⟨sched() *if up* 152a⟩≡ (148c)

```
if(up){
    ⟨sched() when up, possibly increment delaysched and return 152d⟩
    arch_splhi(); // schedinit requires this (also sensitive moment)
    cpu->cs++;

    arch_procsave(up);
    if(arch_setlabel(&up->sched)){
        //
        // here when the process has been scheduled back
        // from a arch_gotolabel(up->sched) by another process, see below
        //
        arch_procrestore(up);
        arch_spllo();
        return;
    }else{
        //
        // here to go to schedinit() (which will call sched() back)
        //
        arch_gotolabel(&cpu->sched); // goto schedinit()
        panic("sched: should never reach this point");
    }
}
```

⟨schedinit() *after setlabel, if up* 152b⟩≡ (149g)

```
if(up) {
    ⟨schedinit() optional real-time edfrecord() 432f⟩
    //old: cpu->proc = nil;
    // but now that on x86 up = cpu->proc and not cpu->externup
    // we can't do that anymore. Is there a place that rely on
    // cpu->proc and cpu->externup to not be in sync?
    switch(up->state) {
    ⟨schedinit() switch process state cases 93c⟩
    }
    up->cpu = nil;
    ⟨schedinit() updatecpu before reseting up 161d⟩
    cpu->proc = nil;
    up = nil; // same instruction than previous line on some archi (e.g. PC)
}
```

⟨schedinit() *switch process state cases* 152c⟩+≡ (152b) ◁93c

```
case Running:
    ready(up);
    break;
```

9.5.5 Delayed sched()

⟨sched() *when up, possibly increment delaysched and return* 152d⟩≡ (152a)

```
/*
 * Delay the sched until the process gives up the locks
 * it is holding.  This avoids dumb lock loops.
 *
 * But don't delay if the process is Moribund.
 * It called sched to die.
 * But do sched eventually. This avoids a missing unlock
 * from hanging the entire kernel.
 * But don't reschedule procs holding palloc or procalloc.
```

```
 * Those are far too important to be holding while asleep.
 *
 * This test is not exact.  There can still be a few instructions
 * in the middle of taslock when a process holds a lock
 * but Lock.p has not yet been initialized.
 */
if(up->nlocks.ref)
  if(up->state != Moribund)
    if(up->delaysched < 20
      || palloc.Lock.p == up
      || procalloc.Lock.p == up){

        up->delaysched++;
        delayedscheds++;
        return;
    }
// else
up->delaysched = 0;
```

⟨Proc *scheduling fields* 153a⟩+≡ (29c) ◁151a 160b▷

```
ulong delaysched;
```

⟨newproc() *set fields of* p 153b⟩+≡ (75b) ◁151f 160c▷

```
p->nlocks.ref = 0;
p->delaysched = 0;
```

⟨unlock() *if delaysched* 153c⟩≡ (36f)

```
if(up && deccnt(&up->nlocks) == 0 && up->delaysched && arch_islo()){
    /*
     * Call sched if the need arose while locks were held
     * But, don't do it from interrupt routines, hence the arch_islo() test
     */
    sched();
}
```

⟨trap() *if delaysched and clockintr* 153d⟩≡ (105)

```
/* delaysched set because we held a lock or because our quantum ended */
if(up && up->delaysched && clockintr){
    sched();
    arch_splhi();
}
```

⟨syscall() *if delaysched(arm)* 153e⟩≡ (101d)

```
/* if we delayed sched because we held a lock, sched now */
if(up->delaysched){
    sched();
    arch_splhi();
}
```

9.5.6 arch_mmuswitch() (ARM)

⟨Cpu Arch *mmu fields(arm)* 153f⟩+≡ (28e) ◁120c

```
int mmupid;
```

⟨*function* arch_mmuswitch*(arm)* 154a⟩≡ (635a)

```
void
arch_mmuswitch(Proc* proc)
{
    int x;
    PTE *l1;
    Page *page;

    /* do kprocs get here and if so, do they need to? */
/*** "This is plausible, but wrong" - Charles Forsyth 1 Mar 2015
    if(cpu->mmupid == proc->pid && !proc->newtlb)
        return;
***/
    cpu->mmupid = proc->pid;

    ⟨arch_mmuswitch() write back and invalidate cache before switch(arm) 402c⟩

    if(proc->newtlb){
        mmul2empty(proc, 1);
        proc->newtlb = false;
    }

    mmul1empty();

    /* move in new map */
    l1 = cpu->mmul1;
    for(page = proc->mmul2; page != nil; page = page->next){
        x = page->daddr;
        l1[x] = PPN(page->pa)|Dom0|Coarse;
        /* know here that L1lo < x < L1hi */
        if(x+1 - cpu->mmul1lo < cpu->mmul1hi - x)
            cpu->mmul1lo = x+1;
        else
            cpu->mmul1hi = x;
    }

    ⟨arch_mmuswitch() write back cache after adjusting page tables(arm) 402e⟩

    /* lose any possible stale tlb entries */
    mmuinvalidate();
}
```

9.5.7 arch_procsave() (ARM)

⟨*function* arch_procsave*(arm)* 154b⟩≡ (641b)

```
/*
 *  Save the mach dependent part of the process state.
 */
void
arch_procsave(Proc* p)
{
    ⟨arch_procsave() cycles adjustments 384h⟩
    ⟨arch_procsave() floating point(arm) 389a⟩
}
```

⟨*function* arch_procrestore*(arm)* 154c⟩≡ (641b)

```
void
arch_procrestore(Proc* p)
{
```

```
    uvlong t;

    if(p->kp)
        return;
    ⟨arch_procrestore cycles adjustments 384i⟩
    ⟨arch_procrestore() floating point(arm) 389c⟩
}
```

9.6 Cooperative scheduling

9.6.1 yield()

⟨*function* yield 155a⟩≡ (599c)

```
/*
 *  yield the processor and drop our priority
 */
void
yield(void)
{
    if(anyready()){
        ⟨yield() adjust lastupdate before scheduling another process 161i⟩
        sched();
    }
}
```

9.6.2 ready()

⟨Procstate *cases* 155b⟩+≡ (30c) ◁93a 156h▷

```
Ready,
```

⟨*function* ready 155c⟩≡ (599c)

```
/*
 *  ready(p) picks a new priority for a process and sticks it in the
 *  runq for that priority.
 */
void
proc_ready(Proc *p)
{
    int pri;
    int s; // spl
    Schedq *rq;
    ⟨ready() other locals 377b⟩

    s = arch_splhi();
    ⟨ready() optional edfready() for real-time scheduling 432h⟩

    ⟨ready() possibly set cpu->readied 156a⟩
    ⟨ready() update priority 161c⟩

    rq = &runq[pri];
    p->state = Ready;
    queueproc(rq, p);
    ⟨ready() hook proctrace 377c⟩
    arch_splx(s);
}
```

⟨ready() *possibly set* cpu->readied 156a⟩≡ (155c)

```
if(up != p && (p->wired == nil || p->wired == CPUS(cpu->cpuno))) // pad fix
    cpu->readied = p; /* group scheduling */
```

⟨Cpu *other fields* 156b⟩+≡ (28a) ◁149f 158f▷

```
Proc* readied;    /* for runproc */
```

⟨runproc() *cooperative scheduling if process readied* 156c⟩≡ (150)

```
/* cooperative scheduling until the clock ticks */
p=cpu->readied;
if(p && p->cpu==nil && p->state==Ready &&
  (p->wired == nil || p->wired == CPUS(cpu->cpuno)) && // pad's bugfix!
  ⟨runproc() test for empty real-time scheduling queue 432a⟩
){
    skipscheds++;
    rq = &runq[p->priority];
    goto found;
}
```

⟨sched() *reset readied* 156d⟩≡ (149b)

```
cpu->readied = nil;
```

9.6.3 Rendez

⟨*struct* Rendez 156e⟩≡ (528)

```
struct Rendez
{
    // option<ref<Proc>>
    Proc  *p; // sleeping process

    Lock; // LOCK ORDERING: always do lock(r); lock(p->rlock);
};
```

⟨Proc *synchronization fields* 156f⟩+≡ (29c) ◁45h 156g▷

```
// option<ref<Rendez>>, can point to waitr, freememr, sleepr, etc
Rendez  *r;   /* rendezvous point slept on */
```

⟨Proc *synchronization fields* 156g⟩+≡ (29c) ◁156f 173f▷

```
Lock  rlock;    /* sync sleep/wakeup with postnote */
```

9.6.4 sleep()

⟨Procstate *cases* 156h⟩+≡ (30c) ◁155b 361a▷

```
Wakeme,
```

⟨*function* sleep 156i⟩≡ (599c)

```
/*
 *  sleep if a condition is not true.  Another process will
 *  awaken us after it sets the condition.  When we awaken
 *  the condition may no longer be true.
 *
 *  we lock both the process and the rendezvous to keep r->p
 *  and p->r synchronized.
 */
void
proc_sleep(Rendez *r, bool (*f)(void*), void *arg)
{
    int s; // spl
```

```
    ⟨sleep() other locals 377h⟩

    s = arch_splhi();
    ⟨sleep() sanity check nlocks 158a⟩
    lock(r);
    lock(&up->rlock);
    ⟨sleep() sanity check double sleep on this rendez vous 158b⟩

    /*
     *  Wakeup only knows there may be something to do by testing
     *  r->p in order to get something to lock on.
     *  Flush that information out to memory in case the sleep is
     *  committed.
     */
    r->p = up;

    if((*f)(arg) || up->notepending){
        /*
         *  if condition happened or a note is pending
         *  never mind
         */
        r->p = nil;
        unlock(&up->rlock);
        unlock(r);
    } else {
        /*
         *  now we are committed to
         *  change state and call scheduler
         */
        ⟨sleep() hook proctrace 377i⟩
        up->state = Wakeme;
        up->r = r;

        // similar code to sched(), why not call sched()? because of the unlock?

        /* statistics */
        cpu->cs++;

        arch_procsave(up);
        if(arch_setlabel(&up->sched)) {
            /*
             *  here when the process is awakened
             */
            arch_procrestore(up);
            arch_spllo();
        } else {
            /*
             *  here to go to sleep (i.e. stop Running)
             */
            unlock(&up->rlock);
            unlock(r);

            arch_gotolabel(&cpu->sched);
            panic("sleep: should never reach this point");
        }
    }
    ⟨sleep() if notepending 335e⟩
    arch_splx(s);
}
```

⟨sleep() *sanity check nlocks* 158a⟩≡ (156i)

```
if(up->nlocks.ref)
    print("process %lud sleeps with %lud locks held, last lock %#p locked at pc %#lux, sleep called from %#p\n"
        up->pid, up->nlocks.ref, up->lastlock, up->lastlock->pc, getcallerpc(&r));
```

⟨sleep() *sanity check double sleep on this rendez vous* 158b⟩≡ (156i)

```
if(r->p){
    print("double sleep called from %#p, %lud %lud\n", getcallerpc(&r), r->p->pid, up->pid);
    arch_dumpstack();
}
```

9.6.5 wakeup()

⟨*function* wakeup 158c⟩≡ (599c)

```
/*
 *  Expects that only one process can call wakeup for any given Rendez.
 *  We hold both locks to ensure that r->p and p->r remain consistent.
 *  Richard Miller has a better solution that doesn't require both to
 *  be held simultaneously, but I'm a paranoid - presotto.
 */
Proc*
proc_wakeup(Rendez *r)
{
    Proc *p;
    int s; // spl

    s = arch_splhi();
    lock(r);
    p = r->p;

    if(p != nil){
        lock(&p->rlock);
        ⟨wakeup() sanity check process state 158d⟩
        r->p = nil;
        p->r = nil;
        ready(p);
        unlock(&p->rlock);
    }
    unlock(r);
    arch_splx(s);
    return p;
}
```

⟨wakeup() *sanity check process state* 158d⟩≡ (158c)

```
if(p->state != Wakeme || p->r != r){
    iprint("%p %p %d\n", p->r, r, p->state);
    panic("wakeup: state");
}
```

9.7 Preemptive scheduling

⟨*constant* Arch_HZ *(arm)* 158e⟩≡ (638b)

```
#define Arch_HZ     100         /* clock frequency */
```

9.7.1 hzsched()

⟨Cpu *other fields* 158f⟩+≡ (28a) ◁156b 354b▷

```
ulong schedticks;   /* next forced context switch */
```

⟨sched() *after found a new proc, adjust schedticks* 159a⟩≡ (149b)

```
cpu->schedticks = cpu->ticks + Arch_HZ/10; // 100ms of allocated time
```

⟨*function* hzsched 159b⟩≡ (599c)

```
/*
 *  here once per clock tick to see if we should resched
 */
void
hzsched(void)
{
    ⟨hzsched() rebalance ready procs 163a⟩
    /* unless preempted, get to run for at least 100ms */
    if(anyhigher()
    || (!up->fixedpri && cpu->ticks > cpu->schedticks && anyready())){

        cpu->readied = nil;   /* avoid cooperative scheduling */
        up->delaysched++;
    }
}
```

⟨*function* anyhigher 159c⟩≡ (599c)

```
int
anyhigher(void)
{
    return runvec & ~((1<<(up->priority+1))-1);
}
```

⟨sched() *if readied process, do not adjust schedticks* 159d⟩≡ (149b)

```
if(p == cpu->readied) {
}
```

9.7.2 hzclock()

⟨*clock callback hzclock* 159e⟩≡ (597a)

```
void
hzclock(Ureg *ur)
{
    cpu->ticks++;
    ⟨hzclock() adjust pc of proc 159f⟩
    ⟨hzclock() if flushmmu 354a⟩
    ⟨hzclock() accounttime 512g⟩
    ⟨hzclock() if kproftimer 515b⟩

    ⟨hzclock() if not active cpu, return 159g⟩
    // else
    ⟨hzclock() if exiting, exit 160a⟩
    // else

    ⟨hzclock() check the alarms 177d⟩
    if(up && up->state == Running)
        hzsched();  /* in proc.c */
}
```

⟨hzclock() *adjust pc of proc* 159f⟩≡ (159e)

```
if(cpu->proc) // why not using up here? why cpu->proc?
    cpu->proc->pc = ur->pc;
```

⟨hzclock() *if not active cpu, return* 159g⟩≡ (159e)

```
if((active.cpus & (1<<cpu->cpuno)) == 0)
    return;
```

⟨hzclock() *if exiting, exit* 160a⟩≡ (159e)

```
if(active.exiting) {
    print("someone's exiting\n");
    arch_exit(0);
}
```

9.8 Customized scheduling

⟨Proc *scheduling fields* 160b⟩+≡ (29c) ◁153a 161e▷

```
Cpu  *wired;
```

⟨newproc() *set fields of* p 160c⟩+≡ (75b) ◁153b 161f▷

```
p->wired = nil;
```

⟨procctlreq() *CMwired case* 160d⟩≡ (482)

```
case CMwired:
    procwired(p, atoi(cb->f[1]));
    break;
```

⟨*function* procwired 160e⟩≡ (599c)

```
/*
 * wire this proc to a processor
 */
void
procwired(Proc *p, int bm)
{
    Proc *pp;
    int i;
    char nwired[MAXCPUS];
    Cpu *wm;

    if(bm < 0){
        /* pick a processor to wire to */
        memset(nwired, 0, sizeof(nwired));
        p->wired = nil;
        pp = proctab(0);
        for(i=0; i<conf.nproc; i++, pp++){
            wm = pp->wired;
            if(wm && pp->pid)
                nwired[wm->cpuno]++;
        }
        bm = 0;
        for(i=0; i<conf.ncpu; i++)
            if(nwired[i] < nwired[bm])
                bm = i;
    } else {
        /* use the virtual processor requested */
        bm = bm % conf.ncpu;
    }

    p->wired = CPUS(bm);
    p->lastcpu = p->wired;
}
```

⟨sysrfork() *other locals* 160f⟩+≡ (78b) ◁82a

```
Cpu *wm;
```

⟨sysrfork() *if parent is a wired proc* 161a⟩≡ (78d)
```
wm = up->wired;
if(wm)
    procwired(p, wm->cpuno);
```

9.9 Real-time scheduling

9.10 Adjusting priority

9.10.1 reprioritize()

⟨sched() *after found process* p *to schedule, possibly update priority* 161b⟩≡ (149b)
```
⟨sched() if real-time process then do nothing, do not reprioritize 433a⟩
else {
    updatecpu(p);
    p->priority = reprioritize(p);
}
```

⟨ready() *update priority* 161c⟩≡ (155c)
```
updatecpu(p);
pri = reprioritize(p);
p->priority = pri;
```

⟨schedinit() *updatecpu before reseting up* 161d⟩≡ (152b)
```
updatecpu(up);
```

⟨Proc *scheduling fields* 161e⟩+≡ (29c) ◁160b 433c▷
```
ulong lastupdate; // dimension?? ticks * Scaling;
ulong cpuavg;    /* cpu average */
```

⟨newproc() *set fields of* p 161f⟩+≡ (75b) ◁160c 335b▷
```
p->cpuavg = 0;
p->lastupdate = CPUS(0)->ticks * Scaling;
```

⟨*constant* Scaling 161g⟩≡ (599c)
```
Scaling=2,
```

⟨*constant* Schedagain 161h⟩≡ (599c)
```
schedgain = 30, /* units in seconds */
```

⟨yield() *adjust lastupdate before scheduling another process* 161i⟩≡ (155a)
```
/* pretend we just used 1/2 tick */
up->lastupdate -= Scaling/2;
```

⟨*function* updatecpu 161j⟩≡ (599c)
```
/*
 * Update the cpu time average for this particular process,
 * which is about to change from up -> not up or vice versa.
 * p->lastupdate is the last time an updatecpu happened.
 *
 * The cpu time average is a decaying average that lasts
 * about D clock ticks.  D is chosen to be approximately
 * the cpu time of a cpu-intensive "quick job".  A job has to run
 * for approximately D clock ticks before we home in on its
 * actual cpu usage.  Thus if you manage to get in and get out
 * quickly, you won't be penalized during your burst.  Once you
 * start using your share of the cpu for more than about D
 * clock ticks though, your p->cpu hits 1000 (1.0) and you end up
```

```
 * below all the other quick jobs.  Interactive tasks, because
 * they basically always use less than their fair share of cpu,
 * will be rewarded.
 *
 * If the process has not been running, then we want to
 * apply the filter
 *
 *  cpu = cpu * (D-1)/D
 *
 * n times, yielding
 *
 *  cpu = cpu * ((D-1)/D)^n
 *
 * but D is big enough that this is approximately
 *
 *  cpu = cpu * (D-n)/D
 *
 * so we use that instead.
 *
 * If the process has been running, we apply the filter to
 * 1 - cpu, yielding a similar equation.  Note that cpu is
 * stored in fixed point (* 1000).
 *
 * Updatecpu must be called before changing up, in order
 * to maintain accurate cpu usage statistics.  It can be called
 * at any time to bring the stats for a given proc up-to-date.
 */
void
updatecpu(Proc *p)
{
    int n, t, ocpu;
    int D = schedgain * Arch_HZ * Scaling;

    ⟨updatecpu() return if real-time process 432d⟩

    t = CPUS(0)->ticks*Scaling + Scaling/2;
    n = t - p->lastupdate;
    p->lastupdate = t;

    if(n == 0)
        return;
    if(n > D)
        n = D;

    ocpu = p->cpuavg;
    if(p != up)
        p->cpuavg = (ocpu*(D-n))/D;
    else{
        t = 1000 - ocpu;
        t = (t*(D-n))/D;
        p->cpuavg = 1000 - t;
    }

}
```

⟨*function* reprioritize 162⟩≡ (599c)

```
/*
 * On average, p has used p->cpuavg of a cpu recently.
 * Its fair share is conf.ncpu/cpu->load of a cpu.  If it has been getting
 * too much, penalize it.  If it has been getting not enough, reward it.
```

```
 * I don't think you can get much more than your fair share that
 * often, so most of the queues are for using less.  Having a priority
 * of 3 means you're just right.  Having a higher priority (up to p->basepri)
 * means you're not using as much as you could.
 */
int
reprioritize(Proc *p)
{
    int fairshare, n, load, ratio;

    load = CPUS(0)->load;
    if(load == 0)
        return p->basepri;

    /*
     *  fairshare = 1.000 * conf.nproc * 1.000/load,
     * except the decimal point is moved three places
     * on both load and fairshare.
     */
    fairshare = (conf.ncpu*1000*1000)/load;
    n = p->cpuavg;
    if(n == 0)
        n = 1;
    ratio = (fairshare+n/2) / n;
    if(ratio > p->basepri)
        ratio = p->basepri;
    if(ratio < 0)
        panic("reprioritize");
//iprint("pid %d cpu %d load %d fair %d pri %d\n", p->pid, p->cpuavg, load, fairshare, ratio);
    return ratio;
}
```

9.10.2 rebalance()

⟨hzsched() *rebalance ready procs* 163a⟩≡ (159b)

```
/* once a second, rebalance will reprioritize ready procs */
if(cpu->cpuno == 0)
    rebalance();
```

⟨*function* rebalance 163b⟩≡ (599c)

```
static void
rebalance(void)
{
    int pri, npri, t, x;
    Schedq *rq;
    Proc *p;

    ⟨rebalance() return if less than one second since last rebalance 164b⟩

    for(pri=0, rq=runq; pri<Npriq; pri++, rq++){
another:
        p = rq->head;
        if(p == nil)
            continue;
        if(p->lastcpu != CPUS(cpu->cpuno)) // this has been the source of comments on 9fans, todo look if bug (
            continue;
        if(pri == p->basepri)
            continue;
        updatecpu(p);
```

```
        npri = reprioritize(p);
        if(npri != pri){
            x = arch_splhi();
            p = dequeueproc(rq, p);
            if(p)
                queueproc(&runq[npri], p);
            arch_splx(x);
            goto another;
        }
    }
}
```

⟨*global* `balancetime` 164a⟩≡ (599c)

```
/*
 *  recalculate priorities once a second.  We need to do this
 *  since priorities will otherwise only be recalculated when
 *  the running process blocks.
 */
ulong balancetime;
```

⟨`rebalance()` *return if less than one second since last rebalance* 164b⟩≡ (163b)

```
t = cpu->ticks;
if(t - balancetime < Arch_HZ)
    return;
balancetime = t;
```

9.11 Halting (ARM)

⟨*function* `arch_idlehands`*(raspberry pi1)(arm)* 164c⟩≡ (627b)

```
TEXT arch_idlehands(SB), $-4
    MOVW    CPSR, R3
    ORR $(PsrDirq|PsrDfiq), R3, R1      /* splfhi */
    MOVW    R1, CPSR

    DSB
    MOVW    nrdy(SB), R0
    CMP $0, R0
    MCR.EQ  CpSC, 0, R0, C(CpCACHE), C(CpCACHEintr), CpCACHEwait
    DSB

    MOVW    R3, CPSR            /* splx */
    RET
```

Chapter 10

Time

⟨systab *time syscalls* 165a⟩≡ (26c)
```
[SLEEP]     syssleep,
[ALARM]     sysalarm,
```

10.1 Overview (arm)

10.2 Time units

10.2.1 Seconds

⟨*type* Txxx 165b⟩≡ (596a) 165d ▷
```
typedef vlong   Tsec; // seconds
typedef int     Tms; // milliseconds
typedef vlong   Tmicro; // microseconds
typedef vlong   Tnano; // nanoseconds
```

⟨*macro* MS2NS 165c⟩≡ (595b)
```
#define MS2NS(n) (((vlong)(n))*1000000LL)
```

10.2.2 Ticks

⟨*type* Txxx 165d⟩+≡ (596a) ◁ 165b 166d ▷
```
typedef vlong   Tval; // ticks
```

⟨*macro* TK2MS 165e⟩≡ (595b)
```
#define TK2MS(x) ((x)*(1000/Arch_HZ))
```

⟨*function* tk2ms 165f⟩≡ (597a)
```
/*
 *  This tk2ms avoids overflows that the macro version is prone to.
 *  It is a LOT slower so shouldn't be used if you're just converting
 *  a delta.
 */
ulong
tk2ms(ulong ticks)
{
    uvlong t, hz;

    t = ticks;
    hz = Arch_HZ;
    t *= 1000L;
```

```
    t = t/hz;
    ticks = t;
    return ticks;
}
```

⟨*function* ms2tk 166a⟩≡ (597a)

```
ulong
ms2tk(ulong ms)
{
    ⟨ms2tk() overflow approximation 166b⟩
    return (ms * Arch_HZ + 500) / 1000;
}
```

⟨ms2tk() *overflow approximation* 166b⟩≡ (166a)

```
/* avoid overflows at the cost of precision */
if(ms >= 1000000000 / Arch_HZ)
    return (ms / 1000) * Arch_HZ;
```

10.2.3 Fastticks (ARM)

⟨*constant* SystimerFreq*(arm)* 166c⟩≡ (638c)

```
SystimerFreq    = 1*Mhz,
```

⟨*type* Txxx 166d⟩+≡ (596a) ◁165d

```
typedef vlong   Tfast; // fastticks
typedef uvlong   Tufast; // fastticks
```

⟨*function* arch_us*(arm)* 166e⟩≡ (639b)

```
ulong // Tmicro
arch_us(void)
{
    ⟨arch_us() if non-standard systimer frequency 181b⟩
    return arch_fastticks(nil);
}
```

⟨*function* clock_arch_fastticks*(arm)* 166f⟩≡ (639b)

```
Tufast
clock_arch_fastticks(uvlong *hz)
{
    Systimers *tn;
    ulong lo, hi;
    Tufast now;
    int s;

    ⟨arch_fastticks() if hz pointer given 167a⟩

    tn = (Systimers*)SYSTIMERS;
    s = arch_splhi();

    do{
        hi = tn->chi;
        lo = tn->clo;
    }while(tn->chi != hi);

    now = (uvlong)hi<<32 | lo;
    arch_splx(s);
    return now;
}
```

⟨arch_fastticks() *if hz pointer given* 167a⟩≡ (166f)

```
if(hz)
    *hz = SystimerFreq;
```

10.3 Clock interrupt (ARM)

⟨*struct* Systimers*(arm)* 167b⟩≡ (639b)

```
// The order matters! the fields match the memory-mapped external registers.
struct Systimers {
    u32int  cs;

    u32int  clo;
    u32int  chi;

    u32int  c0;
    u32int  c1;
    u32int  c2;
    u32int  c3;
};
```

⟨*function* clockintr*(arm)* 167c⟩≡ (639b)

```
static void
clockintr(Ureg *ureg, void*)
{
    Systimers *tn;

    tn = (Systimers*)SYSTIMERS;
    /* dismiss interrupt */
    tn->cs = 1<<3;
    // portable code
    timerintr(ureg, 0);
}
```

10.4 Timers

10.4.1 Timer

⟨*struct* Timer 167d⟩≡ (596a)

```
struct Timer
{
    /* Public interface */
    // enum<Timermode>
    int tmode;    /* See above */
    Tnano tns;    /* meaning defined by mode */ //nanosecond

    void  (*tf)(Ureg*, Timer*);
    void  *ta; // extra argument (generic pointer)

    /* Internal */
    Tfast  twhen;    /* ns represented in fastticks */
    ILock; // LOCK ORDERING: ilock(timer); ilock(timers)

    ⟨Timer extra fields 168c⟩
    };
```

⟨*enum* timermode 168a⟩≡ (596a)

```
/*
 * fasttick timer interrupts
 */
enum Timermode
{
    Trelative,  /* timer programmed in ns from now */
    Tperiodic,  /* periodic timer, period in ns */
};
```

10.4.2 Timers and CPU timers

⟨*struct* Timers 168b⟩≡ (596a)

```
struct Timers
{
    // list<Timer> (next = Timer.tnext)
    Timer *head;
    // extra
    ILock;
};
```

⟨Timer *extra fields* 168c⟩≡ (167d) 168d ▷

```
// list<Timer> of Timers.head
Timer *tnext;
```

⟨Timer *extra fields* 168d⟩+≡ (167d) ◁ 168c

```
// option<ref<list<Timer>> Timers.head (of one of global timers[MAXCPUS])
Timers  *tt;    /* Timers queue this timer runs on */
```

⟨*global* timers 168e⟩≡ (597a)

```
static Timers timers[MAXCPUS];
```

10.4.3 tadd()/tdel()

⟨*function* tadd 168f⟩≡ (597a)

```
static Tfast
tadd(Timers *tt, Timer *nt)
{
    Timer *t, **last;

    /* Called with tt locked */
    ⟨tadd() sanity check nt 169a⟩
    ⟨tadd() set nt->twhen 169b⟩

    // insert_sorted(nt, tt, (a->twhen <=> b->twhen))
    for(last = &tt->head; t = *last; last = &t->tnext){
        if(t->twhen > nt->twhen)
            break;
    }
    nt->tnext = *last;
    *last = nt;

    nt->tt = tt;
    if(last == &tt->head)
        return nt->twhen;
    return 0;
}
```

⟨tadd() *sanity check nt* 169a⟩≡ (168f)

```
assert(nt->tt == nil);
```

⟨tadd() *set* nt->twhen 169b⟩≡ (168f)

```
switch(nt->tmode){
case Trelative:
    if(nt->tns <= 0)
        nt->tns = 1;
    nt->twhen = arch_fastticks(nil) + ns2fastticks(nt->tns);
    break;
case Tperiodic:
    assert(nt->tns >= 100000);  /* At least 100 Âµs period */
    if(nt->twhen == 0){
        ⟨tadd() look if can combine with identical timer 169c⟩
        else
            nt->twhen = arch_fastticks(nil);
    }
    nt->twhen += ns2fastticks(nt->tns);
    break;
default:
    panic("timer: impossible");
    break;
}
```

⟨tadd() *look if can combine with identical timer* 169c⟩≡ (169b)

```
/* look for another timer at same frequency for combining */
for(t = tt->head; t; t = t->tnext){
    if(t->tmode == Tperiodic && t->tns == nt->tns)
        break;
}
if (t)
    nt->twhen = t->twhen;
```

⟨*function* tdel 169d⟩≡ (597a)

```
static Tfast
tdel(Timer *dt)
{
    Timer *t, **last;
    Timers *tt;

    tt = dt->tt;
    ⟨tdel() sanity check tt 169e⟩
    // remove_list(dt, tt)
    for(last = &tt->head; t = *last; last = &t->tnext){
        if(t == dt){
            assert(dt->tt);
            dt->tt = nil;
            *last = t->tnext;
            break;
        }
    }
    if(last == &tt->head && tt->head)
        return tt->head->twhen;
    return 0;
}
```

⟨tdel() *sanity check tt* 169e⟩≡ (169d)

```
if (tt == nil)
    return 0; //todo: panic instead
```

10.4.4 timeradd()//timerdel()

⟨*function* timeradd 170a⟩≡ (597a)

```
/* add or modify a timer */
void
timeradd(Timer *nt)
{
    Timers *tt;
    Tfast when;

    /* Must lock Timer struct before Timers struct */
    ilock(nt);

    ⟨timeradd() if timer was already in a list 170b⟩
    tt = &timers[cpu->cpuno];
    ilock(tt);
    when = tadd(tt, nt);
    if(when)
        arch_timerset(when);
    iunlock(tt);
    iunlock(nt);
}
```

⟨timeradd() *if timer was already in a list* 170b⟩≡ (170a)

```
if(tt = nt->tt){
    ilock(tt);
    tdel(nt);
    iunlock(tt);
}
```

⟨pexit() *at the start, free resources* 170c⟩≡ (92d) 176a ▷

```
if (up->tt)
    timerdel(up);
```

⟨*function* timerdel 170d⟩≡ (597a)

```
void
timerdel(Timer *dt)
{
    Timers *tt;
    Tfast when;

    ilock(dt);
    if(tt = dt->tt){
        ilock(tt);
        when = tdel(dt);
        if(when && tt == &timers[cpu->cpuno])
            arch_timerset(tt->head->twhen);
        iunlock(tt);
    }
    iunlock(dt);
}
```

10.4.5 arch_timerset() (arm)

⟨*function* arch_timerset*(arm)* 170e⟩≡ (639b)

```
void
arch_timerset(Tfast next)
{
    Systimers *tn;
```

```
    Tufast now;
    long period;

    now = arch_fastticks(nil);
    period = next - now;
    ⟨arch_timerset() sanitize period(arm) 171a⟩
    ⟨arch_timerset() if not cpu0(arm) 417b⟩
    else{
        tn = (Systimers*)SYSTIMERS;
        tn->c3 = (ulong)(now + period);
    }
}
```

⟨arch_timerset() *sanitize period(arm)* 171a⟩≡ (170e)

```
if(period < MinPeriod)
    period = MinPeriod;
else if(period > MaxPeriod)
    period = MaxPeriod;
```

⟨*constant* MaxPeriod*(arm)* 171b⟩≡ (638c)

```
MaxPeriod   = SystimerFreq / Arch_HZ,
```

⟨*constant* MinPeriod*(arm)* 171c⟩≡ (638c)

```
MinPeriod   = SystimerFreq / (100*Arch_HZ),
```

10.4.6 timerintr()

⟨*interrupt callback timerintr* 171d⟩≡ (597a)

```
void
timerintr(Ureg *u, Tval)
{
    Timer *t;
    Timers *tt;
    Tufast when, now;
    bool callhzclock = false;
    ⟨timerintr() other locals 172b⟩

    ⟨timerintr() start, stat 517k⟩
    tt = &timers[cpu->cpuno];
    now = arch_fastticks(nil);
    ⟨timerintr() sanity check now 172a⟩
    ilock(tt);
    ⟨timerintr() before loop, initialize count 172c⟩
    while((t = tt->head) != nil){
        /*
         * No need to ilock t here: any manipulation of t
         * requires tdel(t) and this must be done with a
         * lock to tt held.  We have tt, so the tdel will
         * wait until we're done
         */
        when = t->twhen;
        if(when > now){
            arch_timerset(when);
            iunlock(tt);
            if(callhzclock)
                hzclock(u);
            return;
        }
        // else
```

```
        // pop_list(tt)
        tt->head = t->tnext;
        assert(t->tt == tt);
        t->tt = nil;
        ⟨timerintr() in timers loop, stat 517l⟩
        iunlock(tt);

        // call the callback
        if(t->tf)
            (*t->tf)(u, t);
        else
            callhzclock = true;

        ilock(tt);
        if(t->tmode == Tperiodic)
            tadd(tt, t);
        ⟨timerintr() end of timers loop, sanity check count 172e⟩
    }
    iunlock(tt); // unreachable?
}
```

⟨timerintr() *sanity check now* 172a⟩≡ (171d)

```
if(now == 0)
    panic("timerintr: zero arch_fastticks()");
```

⟨timerintr() *other locals* 172b⟩≡ (171d)

```
int count;
```

⟨timerintr() *before loop, initialize count* 172c⟩≡ (171d)

```
count = Maxtimerloops;
```

⟨*constant* Maxtimerloops 172d⟩≡ (597a)

```
Maxtimerloops = 20*1000,
```

⟨timerintr() *end of timers loop, sanity check count* 172e⟩≡ (171d)

```
if (--count <= 0) {
    count = Maxtimerloops;
    iprint("timerintr: probably stuck in while loop; "
        "scrutinise clock.c or use faster cycle "
        "counter\n");
}
```

10.4.7 Clock callbacks: addclock0link()

⟨*function* addclock0link 172f⟩≡ (597a)

```
Timer*
addclock0link(void (*f)(void), Tms ms)
{
    Timer *nt;
    Tfast when;

    ⟨addclock0link() sanity check timersinit 173a⟩

    nt = malloc(sizeof(Timer));
    ⟨addclock0link() sanity check nt 173b⟩
    ⟨addclock0link() sanitize ms 173c⟩

    nt->tns = (Tnano)ms * 1000000LL; // MS2NS(ms)
    nt->tmode = Tperiodic;
```

```
    nt->tt = nil;
    nt->tf = (void (*)(Ureg*, Timer*))f;

    // those clock callbacks are all done on the bootstrap processor
    ilock(&timers[0]);
    when = tadd(&timers[0], nt);
    if(when)
        arch_timerset(when);
    iunlock(&timers[0]);
    return nt;
}
```

⟨addclock0link() *sanity check timersinit* 173a⟩≡ (172f)

```
if(!timersinited)
    panic("addclock0link: timersinit not called yet");
```

⟨addclock0link() *sanity check nt* 173b⟩≡ (172f)

```
if(nt == nil)
    error(Enomem);
```

⟨addclock0link() *sanitize ms* 173c⟩≡ (172f)

```
/* Synchronize to hztimer if ms is 0 */
if(ms == 0)
    ms = 1000 / Arch_HZ; // tk2ms(1)
```

10.4.8 Delay (ARM)

⟨*function* clock_arch_microdelay*(arm)* 173d⟩≡ (639b)

```
void
clock_arch_microdelay(int n) // Tmicro
{
    Systimers *tn;
    u32int now, diff;

    diff = n + 1;
    tn = (Systimers*)SYSTIMERS;
    now = tn->clo;
    while(tn->clo - now < diff)
        ;
}
```

⟨*function* clock_arch_delay*(arm)* 173e⟩≡ (639b)

```
void
clock_arch_delay(Tms n)
{
    while(--n >= 0)
        arch_microdelay(1000);
}
```

10.5 syssleep()

⟨Proc *synchronization fields* 173f⟩+≡ (29c) ◁156g 360c▷

```
Rendez  sleepr;    /* place for syssleep/debug/tsleep */
```

⟨*syscall sleep* 174a⟩≡ (603a)

```
// int sleep(long millisecs);
long
syssleep(ulong* arg)
{
    int n;
    n = arg[0];
    if(n <= 0) {
        ⟨syssleep() optional edfyield() for real-time scheduling 432i⟩
        yield();
        return 0;
    }
    ⟨syssleep() sanitize n 174b⟩
    tsleep(&up->sleepr, returnfalse, 0, n);
    return 0;
}
```

⟨syssleep() *sanitize n* 174b⟩≡ (174a)

```
if(n < TK2MS(1))
    n = TK2MS(1);
```

⟨*function* tsleep 174c⟩≡ (599c)

```
void
proc_tsleep(Rendez *r, int (*fn)(void*), void *arg, ulong ms)
{
    ⟨tsleep() sanity check up timer 174e⟩
    up->tns = MS2NS(ms);
    up->tmode = Trelative;
    up->tf = twakeup;

    up->ta = up; // so can access r from wakeup
    up->trend = r;

    up->tfn = fn; // needed?

    timeradd(up);

    if(waserror()){
        timerdel(up);
        nexterror();
    }
    sleep(r, tfn, arg);
    // back after sleep
    ⟨tsleep() reset timer after sleep 174d⟩
    poperror();
}
```

⟨tsleep() *reset timer after sleep* 174d⟩≡ (174c)

```
if(up->tt)
    timerdel(up);
up->twhen = 0;
```

⟨tsleep() *sanity check up timer* 174e⟩≡ (174c)

```
if (up->tt){
    print("tsleep: timer active: mode %d, tf %#p\n", up->tmode, up->tf);
    timerdel(up);
}
```

⟨Proc *other fields* 174f⟩+≡ (29c) ◁101b 175a▷

```
Timer;      /* For tsleep and real-time */
```

⟨Proc *other fields* 175a⟩+≡ (29c) ◁174f 175h▷

```
Rendez  *trend;
int (*tfn)(void*);
```

⟨*function* tfn 175b⟩≡ (599c)

```
static int
tfn(void *arg)
{
    return up->trend == nil || up->tfn(arg);
}
```

⟨*function* twakeup 175c⟩≡ (599c)

```
void
twakeup(Ureg*, Timer *t)
{
    Proc *p;
    Rendez *trend;

    p = t->ta;
    trend = p->trend;
    p->trend = nil;
    if(trend)
        wakeup(trend);
}
```

10.6 Alarms

⟨*struct* Alarms 175d⟩≡ (592c)

```
struct Alarms
{
    // sorted_list<ref<Proc> (next = Proc.palarm, sort_key = Proc.alarm)
    Proc  *head;

    // Extra
    QLock;
};
```

⟨Proc *extra fields* 175e⟩+≡ (29c) ◁147a

```
// Alarms.head chain?
Proc  *palarm;  /* Next alarm time */
```

⟨*global* alarms 175f⟩≡ (596b)

```
static Alarms   alarms;
```

10.7 sysalarm()

⟨*syscall alarm* 175g⟩≡ (603a)

```
// long alarm(unsigned long millisecs);
long
sysalarm(ulong* arg)
{
    return procalarm(arg[0]);
}
```

⟨Proc *other fields* 175h⟩+≡ (29c) ◁175a 217c▷

```
// option<ticks>
ulong alarm;    /* Time of call */ // Tval ticks
```

⟨pexit() *at the start, free resources* 176a⟩+≡ (92d) ◁170c 370d▷

```
up->alarm = 0;
```

⟨*function* procalarm 176b⟩≡ (596b)

```
ulong
procalarm(ulong time) // Tms
{
    Proc **l, *f;
    ulong when, old; // Tval ticks

    ⟨procalarm() compute remaining time in old alarm 176c⟩
    ⟨procalarm() if time is zero, reset alarm 176d⟩
    when = ms2tk(time) + CPUS(0)->ticks;
    ⟨procalarm() sanitize when 177a⟩

    qlock(&alarms);
    l = &alarms.head;
    // remove_list(up, alarms)
    for(f = *l; f; f = f->palarm) {
        if(up == f){
            *l = up->palarm;
            break;
        }
        l = &f->palarm;
    }
    up->palarm = nil;

    // add_sorted_list(up, alarms)
    if(alarms.head) {
        l = &alarms.head;
        for(f = *l; f; f = f->palarm) {
            if((long)(f->alarm - when) >= 0) {
                up->palarm = f;
                *l = up;
                goto done;
            }
            l = &f->palarm;
        }
        *l = up;
    }
    else
        alarms.head = up;

done:
    up->alarm = when;
    qunlock(&alarms);

    return old;
}
```

⟨procalarm() *compute remaining time in old alarm* 176c⟩≡ (176b)

```
if(up->alarm)
    old = tk2ms(up->alarm - CPUS(0)->ticks);
else
    old = 0;
```

⟨procalarm() *if time is zero, reset alarm* 176d⟩≡ (176b)

```
if(time == 0) {
    up->alarm = 0;
    return old;
}
```

⟨procalarm() *sanitize when* 177a⟩≡ (176b)

```
if(when == 0)       /* ticks have wrapped to 0? */
    when = 1;   /* distinguish a wrapped alarm from no alarm */
```

10.7.1 alarmkproc()

⟨*global* alarmr 177b⟩≡ (596b)

```
static Rendez alarmr;
```

⟨*kernel process alarmkproc* 177c⟩≡ (596b)

```
// Kernel Process for alarm management
void
alarmkproc(void*)
{
    Proc *rp;
    ulong now;

    for(;;){
        now = CPUS(0)->ticks;

        qlock(&alarms);
        /*
         * the odd test of now vs. rp->alarm is to cope with
         * now wrapping around.
         */
        while((rp = alarms.head) && (long)(now - rp->alarm) >= 0){
            if(rp->alarm != 0L){
                if(canqlock(&rp->debug)){
                    if(!waserror()){
                        postnote(rp, 0, "alarm", NUser);
                        poperror();
                    }
                    qunlock(&rp->debug);
                    rp->alarm = 0L;
                }else
                    break;
            }
            alarms.head = rp->palarm;
        }
        qunlock(&alarms);

        sleep(&alarmr, returnfalse, 0);
    }
}
```

10.7.2 checkalarms()

⟨hzclock() *check the alarms* 177d⟩≡ (159e)

```
checkalarms();
```

⟨*function* checkalarms 177e⟩≡ (596b)

```
/*
 *  called every clock tick
 */
void
checkalarms(void)
{
    Proc *p;
```

```
    ulong now;

    p = alarms.head;
    now = CPUS(0)->ticks;

    if(p && (long)(now - p->alarm) >= 0)
        wakeup(&alarmr);
}
```

10.8 Time of day

⟨*struct* TOD 178a⟩≡ (602)

```
struct TOD {
    bool init;       /* true if initialized */

    ulong   cnt;

    uvlong  multiplier; /* ns = off + (multiplier*ticks)>>31 */
    uvlong  divider;    /* ticks = (divider*(ns-off))>>31 */
    uvlong  umultiplier;/* Âµs = (Âµmultiplier*ticks)>>31 */
    uvlong  udivider;   /* ticks = (Âµdivider*Âµs)>>31 */

    vlong   hz;     /* frequency of fast clock */
    vlong   last;       /* last reading of fast clock */

    vlong   off;        /* offset from epoch to last */
    vlong   lasttime;   /* last return value from todget */
    vlong   delta;  /* add 'delta' each slow clock tick from sstart to send */

    ulong   sstart;     /* ... */
    ulong   send;       /* ... */

    // Extra
    Lock;
};
```

⟨*global* tod 178b⟩≡ (602)

```
struct TOD tod;
```

⟨*devcons.c Exxx errors* 178c⟩≡ (531)

```
char *Ebadtimectl = "bad time control";
```

⟨*constant* VLNUMSIZE 178d⟩≡ (531)

```
VLNUMSIZE=  22,
```

⟨*function* todinit 178e⟩≡ (602)

```
void
todinit(void)
{
    if(tod.init)
        return;
    ilock(&tod);
    tod.init = true;           /* prevent reentry via arch_fastticks */
    tod.last = arch_fastticks((uvlong *)&tod.hz);
    iunlock(&tod);
    todsetfreq(tod.hz);
    addclock0link(todfix, 100);
}
```

⟨*function* todsetfreq 179a⟩≡ (602)

```
/*
 *  calculate multiplier
 */
void
todsetfreq(vlong f)
{
    if (f <= 0)
        panic("todsetfreq: freq %lld <= 0", f);
    ilock(&tod);
    tod.hz = f;

    /* calculate multiplier for time conversion */
    tod.multiplier  = mk64fract(TODFREQ, f);
    tod.divider     = mk64fract(f, TODFREQ) + 1;
    tod.umultiplier = mk64fract(MicroFREQ, f);
    tod.udivider    = mk64fract(f, MicroFREQ) + 1;
    iunlock(&tod);
}
```

⟨*function* todset 179b⟩≡ (602)

```
/*
 *  Set the time of day struct
 */
void
todset(vlong t, vlong delta, int n)
{
    if(!tod.init)
        todinit();

    ilock(&tod);
    if(t >= 0){
        tod.off = t;
        tod.last = arch_fastticks(nil);
        tod.lasttime = 0;
        tod.delta = 0;
        tod.sstart = tod.send;
    } else {
        if(n <= 0)
            n = 1;
        n *= Arch_HZ;
        if(delta < 0 && n > -delta)
            n = -delta;
        if(delta > 0 && n > delta)
            n = delta;
        if (n == 0) {
            iprint("todset: n == 0, delta == %lld\n", delta);
            delta = 0;
        } else
            delta /= n;
        tod.sstart = CPUS(0)->ticks;
        tod.send = tod.sstart + n;
        tod.delta = delta;
    }
    iunlock(&tod);
}
```

⟨*function* todget 179c⟩≡ (602)

```
/*
 *  get time of day
```

```
 */
vlong
todget(vlong *ticksp)
{
    uvlong x;
    vlong ticks, diff;
    ulong t;

    if(!tod.init)
        todinit();

    /*
     * we don't want time to pass twixt the measuring of arch_fastticks
     * and grabbing tod.last.  Also none of the vlongs are atomic so
     * we have to look at them inside the lock.
     */
    ilock(&tod);
    tod.cnt++;
    ticks = arch_fastticks(nil);

    /* add in correction */
    if(tod.sstart != tod.send){
        t = CPUS(0)->ticks;
        if(t >= tod.send)
            t = tod.send;
        tod.off = tod.off + tod.delta*(t - tod.sstart);
        tod.sstart = t;
    }

    /* convert to epoch */
    diff = ticks - tod.last;
    if(diff < 0)
        diff = 0;
    arch_mul64fract(&x, diff, tod.multiplier);
    x += tod.off;

    /* time can't go backwards */
    if(x < tod.lasttime)
        x = tod.lasttime;
    else
        tod.lasttime = x;

    iunlock(&tod);

    if(ticksp != nil)
        *ticksp = ticks;

    return x;
}
```

⟨*function* todfix 180⟩≡ (602)

```
/*
 *  called regularly to avoid calculation overflows
 */
static void
todfix(void)
{
    vlong ticks, diff;
    uvlong x;
```

```
    ticks = arch_fastticks(nil);

    diff = ticks - tod.last;
    if(diff > tod.hz){
        ilock(&tod);

        /* convert to epoch */
        arch_mul64fract(&x, diff, tod.multiplier);
if(x > 30000000000ULL) iprint("todfix %llud\n", x);
        x += tod.off;

        /* protect against overflows */
        tod.last = ticks;
        tod.off = x;

        iunlock(&tod);
    }
}
```

⟨*function* `seconds` 181a⟩≡ (602)

```
long
seconds(void)
{
    return (vlong)todget(nil) / TODFREQ;
}
```

⟨`arch_us()` *if non-standard systimer frequency* 181b⟩≡ (166e)

```
if(SystimerFreq != 1*Mhz)
    return fastticks2us(arch_fastticks(nil));
```

⟨*function* `fastticks2us` 181c⟩≡ (602)

```
uvlong
fastticks2us(uvlong ticks)
{
    uvlong res;

    if(!tod.init)
        todinit();
    arch_mul64fract(&res, ticks, tod.umultiplier);
    return res;
}
```

⟨*function* `ns2fastticks` 181d⟩≡ (602)

```
/*
 *  convert nanoseconds to fast ticks
 */
uvlong
ns2fastticks(uvlong ns)
{
    uvlong res;

    if(!tod.init)
        todinit();
    arch_mul64fract(&res, ns, tod.divider);
    return res;
}
```

⟨*function* mk64fract 182a⟩≡ (602)

```
/*
 * Make a 64 bit fixed point number that has a decimal point
 * to the left of the low order 32 bits.  This is used with
 * arch_mul64fract for converting twixt nanoseconds and arch_fastticks.
 *
 *  multiplier = (to<<32)/from
 */
uvlong
mk64fract(uvlong to, uvlong from)
{
    return (to<<32) / from;
}
```

10.9 sysnsec()

10.10 /dev/time

⟨consread() *cases* 182b⟩+≡ (288d) ◁134e 183b▷

```
case Qtime:
    return readtime((ulong)offset, buf, n);
```

⟨*global* fasthz 182c⟩≡ (531)

```
vlong   fasthz;
```

⟨conswrite() *cases* 182d⟩+≡ (289) ◁143c 183c▷

```
case Qtime:
    if(!iseve())
        error(Eperm);
    return writetime(a, n);
```

⟨*function* readtime 182e⟩≡ (531)

```
/*
 *  like the old #c/time but with added info.  Return
 *
 *  secs    nanosecs    fastticks   fasthz
 */
static int
readtime(ulong off, char *buf, int n)
{
    vlong   nsec, ticks;
    long sec;
    char str[7*NUMSIZE];

    nsec = todget(&ticks);
    if(fasthz == 0LL)
        arch_fastticks((uvlong*)&fasthz);
    sec = nsec/1000000000ULL;
    snprint(str, sizeof(str), "%*lud %*llud %*llud %*llud ",
        NUMSIZE-1, sec,
        VLNUMSIZE-1, nsec,
        VLNUMSIZE-1, ticks,
        VLNUMSIZE-1, fasthz);
    return readstr(off, buf, n, str);
}
```

⟨*function* writetime 183a⟩≡ (531)

```
/*
 *  set the time in seconds
 */
static int
writetime(char *buf, int n)
{
    char b[13];
    long i;
    vlong now;

    if(n >= sizeof(b))
        error(Ebadtimectl);
    strncpy(b, buf, n);
    b[n] = 0;
    i = strtol(b, 0, 0);
    if(i <= 0)
        error(Ebadtimectl);
    now = i*1000000000LL;
    todset(now, 0, 0);
    return n;
}
```

10.11 /dev/bintime

⟨consread() *cases* 183b⟩+≡ (288d) ◁182b 380f▷

```
case Qbintime:
    return readbintime(buf, n);
```

⟨conswrite() *cases* 183c⟩+≡ (289) ◁182d 291a▷

```
case Qbintime:
    if(!iseve())
        error(Eperm);
    return writebintime(a, n);
```

⟨*function* readbintime 183d⟩≡ (531)

```
/*
 *  read binary time info.  all numbers are little endian.
 *  ticks and nsec are syncronized.
 */
static int
readbintime(char *buf, int n)
{
    int i;
    vlong nsec, ticks;
    byte *b = (byte*)buf;

    i = 0;
    if(fasthz == 0LL)
        arch_fastticks((uvlong*)&fasthz);
    nsec = todget(&ticks);
    if(n >= 3*sizeof(uvlong)){
        vlong2le(b+2*sizeof(uvlong), fasthz);
        i += sizeof(uvlong);
    }
    if(n >= 2*sizeof(uvlong)){
        vlong2le(b+sizeof(uvlong), ticks);
        i += sizeof(uvlong);
```

```
    }
    if(n >= 8){
        vlong2le(b, nsec);
        i += sizeof(vlong);
    }
    return i;
}
```

⟨*function* writebintime 184a⟩≡ (531)

```
/*
 *  set any of the following
 *  - time in nsec
 *  - nsec trim applied over some seconds
 *  - clock frequency
 */
static int
writebintime(char *buf, int n)
{
    byte *p;
    vlong delta;
    long period;

    n--;
    p = (byte*)buf + 1;
    switch(*buf){
    case 'n':
        if(n < sizeof(vlong))
            error(Ebadtimectl);
        le2vlong(&delta, p);
        todset(delta, 0, 0);
        break;
    case 'd':
        if(n < sizeof(vlong)+sizeof(long))
            error(Ebadtimectl);
        p = le2vlong(&delta, p);
        le2long(&period, p);
        todset(-1, delta, period);
        break;
    case 'f':
        if(n < sizeof(uvlong))
            error(Ebadtimectl);
        le2vlong(&fasthz, p);
        if(fasthz <= 0)
            error(Ebadtimectl);
        todsetfreq(fasthz);
        break;
    }
    return n;
}
```

⟨*global* uvorder 184b⟩≡ (531)

```
static uvlong uvorder = 0x0001020304050607ULL;
```

⟨*function* le2vlong 184c⟩≡ (531)

```
static byte*
le2vlong(vlong *to, byte *f)
{
    byte *t, *o;
    int i;
```

```
    t = (byte*)to;
    o = (byte*)&uvorder;
    for(i = 0; i < sizeof(vlong); i++)
        t[o[i]] = f[i];
    return f+sizeof(vlong);
}
```

⟨*function* vlong2le 185a⟩≡ (531)

```
static byte*
vlong2le(byte *t, vlong from)
{
    byte *f, *o;
    int i;

    f = (byte*)&from;
    o = (byte*)&uvorder;
    for(i = 0; i < sizeof(vlong); i++)
        t[i] = f[o[i]];
    return t+sizeof(vlong);
}
```

⟨*global* order 185b⟩≡ (531)

```
static long order = 0x00010203;
```

⟨*function* le2long 185c⟩≡ (531)

```
static byte*
le2long(long *to, byte *f)
{
    byte *t, *o;
    int i;

    t = (byte*)to;
    o = (byte*)&order;
    for(i = 0; i < sizeof(long); i++)
        t[o[i]] = f[i];
    return f+sizeof(long);
}
```

10.12 Watchdog

Chapter 11

Initialization

11.1 Booting the kernel

11.1.1 Loading the kernel in physical memory (ARM)

11.1.2 First instructions (ARM)

⟨*function* `_start` *(arm)* 186a⟩≡ (627a)

```
/*
 * on bcm2836, only cpu0 starts here
 * other cpus enter at cpureset in armv7.s
 */
TEXT _start(SB), 1, $-4
    /*
     * load physical base for SB addressing while mmu is off
     * keep a handy zero in R0 until first function call
     */
    MOVW    $setR12(SB), R12
    SUB $KZERO, R12
    MOVW    $0, R0

    /*
     * SVC mode, interrupts disabled
     */
    MOVW    $(PsrDirq|PsrDfiq|PsrMsvc), R1
    MOVW    R1, CPSR

    /*
     * start stack at top of 'cpu' (physical addr)
     */
    MOVW    $PADDR(CPUADDR+CPUSIZE-4), R13

    /*
     * do arch-dependent startup (no return)
     */
    BL  armstart(SB)

    // Unreached
    B   0(PC)
    RET
```

11.1.3 Basic pagination (ARM)

⟨*function* `armstart` *(raspberry pi1)(arm)* 186b⟩≡ (627b)

```
TEXT armstart(SB), 1, $-4

    /*
     * disable the mmu and L1 caches
     * invalidate caches and tlb
     */
    MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    BIC $(CpCmmu), R1
    ⟨armstart() disable features(arm) 400f⟩
    MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl

    ⟨armstart() invalidate caches(arm) 400g⟩
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
    ISB

    /*
     * clear mach and page tables
     */
    MOVW    $PADDR(CPUADDR), R1
    MOVW    $PADDR(KTZERO), R2
_ramZ:
    MOVW    R0, (R1)
    ADD $4, R1
    CMP R1, R2
    BNE _ramZ

    /*
     * start stack at top of CPUADDR (physical addr)
     * set up page tables for kernel
     */
    MOVW    $PADDR(CPUADDR+CPUSIZE-4), R13 // done already in start.s
    MOVW    $PADDR(L1), R0

    BL  mmuinit(SB)

    /*
     * set up domain access control and page table base
     */
    MOVW    $Client, R1
    MCR CpSC, 0, R1, C(CpDAC), C(0)

    MOVW    $PADDR(L1), R1
    MCR CpSC, 0, R1, C(CpTTB), C(0)

    /*
     * enable caches, mmu, and high vectors
     */
    MRC CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
    ORR $(CpCmmu|CpChv), R0
    ⟨armstart() reenable features(arm) 400h⟩
    MCR CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
    ISB

    /*
     * switch SB, SP, and PC into KZERO space
     */
    MOVW    $setR12(SB), R12
    MOVW    $(CPUADDR+CPUSIZE-4), R13

    MOVW    $_startpg(SB), R15
```

⟨*constant* L1*(arm)* 188a⟩≡ (633b)

```
#define L1          (KZERO+0x4000)      /* ttb ptes: 16KiB aligned */
```

⟨*constant* L2*(arm)* 188b⟩≡ (633b)

```
#define L2          (KZERO+0x3000)      /* L2 ptes for vectors etc */
```

⟨*function* mmuinit*(arm)* 188c⟩≡ (635a)

```
/*
 * Set up initial PTEs for this cpu (called with mmu off)
 */
void
mmuinit(void *a)
{
    PTE *l1, *l2;
    phys_addr pa;
    kern_addr va;

    l1 = (PTE*)a; // PADDR(L1) for cpu0

    /*
     * map all of ram at KZERO
     */
    va = KZERO;
    for(pa = 0; pa < soc.dramsize; pa += MiB, va += MiB)
    {
        l1[L1X(va)] = pa|Dom0|L1AP(Krw)|Section|L1ptedramattrs;
    }

    /*
     * identity map first MB of ram so mmu can be enabled
     */
    // ???
    l1[L1X(0)] = 0|Dom0|L1AP(Krw)|Section|L1ptedramattrs;

    /*
     * map i/o registers
     */
    va = VIRTIO;
    for(pa = soc.physio; pa < soc.physio + IOSIZE; pa += MiB, va += MiB){
        // No L1ptedramattrs (Cached | Buffered) here, because volatile!
        l1[L1X(va)] = pa|Dom0|L1AP(Krw)|Section;
    }
    ⟨mmuinit() set armlocal virtual memory mapping(arm) 407c⟩

    /*
     * double map exception vectors at top of virtual memory
     */
    // ???
    l2 = (PTE*)PADDR(L2);
    va = HVECTORS;
    l1[L1X(va)] = (uintptr)l2|Dom0|Coarse;
    l2[L2X(va)] = L2AP(Krw)|Small|L2ptedramattrs;
}
```

11.1.4 Jumping to C main() (ARM)

⟨*function* _startpg*(raspberry pi1)(arm)* 188d⟩≡ (627b)

```
TEXT _startpg(SB), 1, $-4
```

```
⟨_startpg() enable cycle counter(raspberry pi1)(arm) 383h⟩
/*
 * call main and loop forever if it returns
 */
BL  main(SB)

// Unreached
B   0(PC)
BL  _div(SB)        /* hack to load _div, etc. */
```

11.2 Kernel initialization (ARM)

⟨*function* main*(arm)* 189⟩≡ (628c)

```
void
main(void)
{
    ⟨main() locals(arm) 200b⟩

    ⟨main() initial assignments for backward deps(arm) ??⟩

    ⟨main() clear bss(arm) 190a⟩

    // Let's go!

    cpu = (Cpu*)CPUADDR;
    arch__cpu0init(); // cpu0 initialization (calls arch__cpuinit())
    mmuinit1((void*)L1); // finish mmu initialization started in mmuinit0
    machon(0); // enable cpu0 for scheduling

    ⟨main() parsing options and boot arguments(arm) 191e⟩
    arch__confinit();     /* figures out amount of memory */
    xinit();

    uartconsinit();

    arch__screeninit(); // screenputs = swconsole_screenputs
    quotefmtinstall(); // libc printf initialization

    print("\nPlan 9 from Bell Labs\n"); // yeah!

    ⟨main() print board and firmware information(arm) 200c⟩
    /* set clock rate to arm_freq from config.txt (default pi1:700Mhz pi2:900MHz) */
    setclkrate(ClkArm, 0);

    arch__trapinit();
    clockinit(); // enable timer interrupts

    //TODO? kbdqinit(); // setup kbdq
    lineqinit(); // setup lineq

    timersinit(); // set initial timer for timer interrupt
    swcursor_init();

    arch_cpuidprint();
    archreset();

    procinit();
    imageinit();
```

```
    links();

    // initialize all devices
    chandevreset();         /* most devices are discovered here */

    pageinit(); // setup palloc.pages and swapalloc.highwater
    swapinit(); // setup swapalloc

    // let's craft our first process (that will then exec("boot/boot"))
    arch__userinit();

    ⟨main() start the other processors(arm) 409g⟩

    // schedule the only ready user process (the one created by userinit)
    schedinit();

    assert(0);          /* shouldn't have returned */
}
```

⟨main() *clear bss(arm)* 190a⟩≡ (189)

```
memset(edata, 0, end - edata);  /* clear bss */
```

⟨*lib.h exxx decl* 190b⟩≡ (580e)

```
extern  char  etext[];
//@Scheck: Assembly, not dead used by 386/l.s and arm/main.c
extern  char  edata[];
extern  char  end[];
```

11.2.1 cpu initialization

⟨*function* arch__cpu0init*(arm)* 190c⟩≡ (628c)

```
void
arch__cpu0init(void)
{
    conf.ncpu = 0; // set in machon() instead (machon() is called after cpuinit)

    cpu->cpuno = 0;
    cpus[cpu->cpuno] = cpu;

    arch__cpuinit();
    active.exiting = false;

    up = nil;
}
```

⟨*function* arch__cpuinit*(arm)* 190d⟩≡ (628c)

```
void
arch__cpuinit(void)
{
    Cpu *cpu0;

    cpu->ticks = 1;
    cpu->perf.period = 1;

    cpu0 = CPUS(0);
    ⟨arch__cpuinit() it not cpu0 414a⟩
}
```

⟨*function* machon*(arm)* 191a⟩≡ (628c)

```
/* enable scheduling of this cpu */
void
machon(uint xcpu)
{
    ulong cpubit;

    cpubit = 1 << xcpu;
    lock(&active);
    if ((active.cpus & cpubit) == 0) {  /* currently off? */
        conf.ncpu++;
        active.cpus |= cpubit;
    }
    unlock(&active);
}
```

⟨*function* arch_cpuidprint*(raspberry pi1)(arm)* 191b⟩≡ (631a)

```
void
arch_cpuidprint(void)
{
    char name[64];

    cputype2name(name, sizeof name);
    arch_delay(50);             /* let uart catch up */
    print("cpu%d: %dMHz ARM %s\n", cpu->cpuno, cpu->cpumhz, name);
}
```

⟨*function* cputype2name*(raspberry pi1)(arm)* 191c⟩≡ (631a)

```
char*
cputype2name(char *buf, int size)
{
    seprint(buf, buf + size, "1176JZF-S");
    return buf;
}
```

11.2.2 Memory detection (ARM)

⟨*function* getramsize*(arm)* 191d⟩≡ (615d)

```
/*
 * Get ARM ram
 */
void
getramsize(Confmem *mem)
{
    u32int buf[2];

    if(vcreq(TagGetram, buf, 0, sizeof buf) != sizeof buf)
        return;
    mem->base = buf[0];
    mem->limit = buf[1];
}
```

11.2.3 conf initialization (ARM)

⟨main() *parsing options and boot arguments(arm)* 191e⟩≡ (189)

```
//optionsinit("/boot/boot boot"); // setup values for getconf()
//ataginit((Atag*)BOOTARGS); // from bootloader config
// example of manual config:
// TODO confname(``console'') = 1?
```

⟨Conf *other fields* 192a⟩+≡ (27a) ◁137a 343a▷

```
ulong npage;    /* total physical pages of memory */

ulong upages;   /* user page pool */
// kpages = npage - upages

ulong nimage;   /* number of page cache image headers */
ulong ialloc;   /* max interrupt time allocation in bytes */
```

⟨*global* memsize*(arm)* 192b⟩≡ (628c)

```
// used also in mmu.c
ulong   memsize = 128*1024*1024;
```

⟨*function* arch__confinit*(arm)* 192c⟩≡ (628c)

```
void
arch__confinit(void)
{
    int i;
    char *p;
    phys_addr pa;
    ulong kpages;
    ulong kmem;

    ⟨arch__confinit() if maxmem boot parameter(arm) 193a⟩
    // simpler than for x86 :)
    getramsize(&conf.mem[0]);
    ⟨arch__confinit() sanity check conf.mem[0](arm) 193b⟩

    conf.npage = 0;
    pa = PADDR(PGROUND(PTR2UINT(end)));

    /*
     *  we assume that the kernel is at the beginning of one of the
     *  contiguous chunks of memory and fits therein.
     */
    for(i=0; i<nelem(conf.mem); i++){
        /* take kernel out of allocatable space */
        if(pa > conf.mem[i].base && pa < conf.mem[i].limit)
            conf.mem[i].base = pa;

        conf.mem[i].npage = (conf.mem[i].limit - conf.mem[i].base)/BY2PG;
        conf.npage += conf.mem[i].npage;
    }

    conf.upages = (conf.npage*80)/100;
    kpages = conf.npage - conf.upages;

    /* set up other configuration parameters */
    conf.ialloc = (kpages/2)*BY2PG; // max bytes for iallocb

    conf.nproc = 100 + ((conf.npage*BY2PG)/MB)*5;
    if(cpuserver)
        conf.nproc *= 3;
    if(conf.nproc > 2000)
        conf.nproc = 2000;

    conf.nswap = conf.npage*3;
    conf.nswppo = 4096;
    conf.nimage = 200;
```

```
    conf.copymode = 1;      /* copy on reference, not copy on write */

    /*
     * Guess how much is taken by the large permanent
     * datastructures. Mntcache and Mntrpc are not accounted for
     * (probably ~300KB).
     */
    kmem = kpages * BY2PG;
    kmem -=
          conf.upages * sizeof(Page)
        + conf.nproc  * sizeof(Proc)
        + conf.nimage * sizeof(KImage)
        + conf.nswap
        + conf.nswppo * sizeof(Page*); // pad's second bugfix :)

    // memory pool
    mainmem->maxsize = kmem;

    if(!cpuserver)
        /*
         * give terminals lots of image memory, too; the dynamic
         * allocation will balance the load properly, hopefully.
         * be careful with 32-bit overflow.
         */
        imagmem->maxsize = kmem;
}
```

⟨arch__confinit() *if maxmem boot parameter(arm)* 193a⟩≡ (192c)

```
if((p = getconf("*maxmem")) != nil){
    memsize = strtoul(p, 0, 0);
    if (memsize < 16*MB)        /* sanity */
        memsize = 16*MB;
}
```

⟨arch__confinit() *sanity check* conf.mem[0]*(arm)* 193b⟩≡ (192c)

```
if(conf.mem[0].limit == 0){
    conf.mem[0].base = 0;
    conf.mem[0].limit = memsize;
}
```

⟨arch__confinit() *if maxmem parameter and limit mem is zero(arm)* 193c⟩

⟨arch__confinit() *if maxmem parameter and limit mem is zero(arm)* 193c⟩≡ (193b)

```
else if(p != nil)
    conf.mem[0].limit = conf.mem[0].base + memsize;
```

11.2.4 Exception vector initialization (ARM)

⟨*struct* Vpage0*(arm)* 193d⟩≡ (636c)

```
/*
 *   Layout at virtual address KZERO (double mapped at HVECTORS).
 */
struct Vpage0 {
    void    (*vectors[Nvec])(void);
    u32int  vtable[Nvec];
};
```

⟨*constant* Nvec*(arm)* 193e⟩≡ (636b)

```
Nvec = 8,       /* # of vectors at start of lexception.s */
```

⟨*function* `arch__trapinit` *(arm)* 194a⟩≡ (636c)

```
/*
 *  set up for exceptions
 */
void
arch__trapinit(void)
{
    Vpage0 *vpage0;

    if (cpu->cpuno == 0) {
        /* disable everything */
        intrsoff();
        /* set up the exception vectors */
        vpage0 = (Vpage0*)HVECTORS;
        memmove(vpage0->vectors, vectors, sizeof(vpage0->vectors));
        memmove(vpage0->vtable, vtable, sizeof(vpage0->vtable));
        ⟨arch__trapinit() invalidate cache after adjust vectors(arm) 402m⟩
    }

    /* set up the stacks for the interrupt modes */
    ⟨arch__trapinit() set stack for other exception/processor-modes 194c⟩
    ⟨arch__trapinit() coherence(arm) 405c⟩
}
```

⟨*function* `intrsoff` *(arm)* 194b⟩≡ (636c)

```
void
intrsoff(void)
{
    Intregs *ip;
    int disable;

    ip = (Intregs*)INTREGS;
    disable = ~0;
    ip->GPUdisable[0] = disable;
    ip->GPUdisable[1] = disable;
    ip->ARMdisable = disable;
    ip->FIQctl = 0;
}
```

⟨`arch__trapinit()` *set stack for other exception/processor-modes* 194c⟩≡ (194a) 428f▷

```
setr13(PsrMirq, cpu->sirq);
setr13(PsrMabt, cpu->sabt);
setr13(PsrMund, cpu->sund);
```

⟨`Cpu Arch` *save of first five registers fields(arm)* 194d⟩≡ (28e)

```
/* save areas for exceptions, hold R0-R4 */
u32int  sirq[5];
u32int  sund[5];
u32int  sabt[5];
```

⟨*function* `setr13` *(arm)* 194e⟩≡ (638a)

```
/*
 *  set the stack value for the mode passed in R0
 */
TEXT setr13(SB), 1, $-4
    MOVW    4(FP), R1

    MOVW    CPSR, R2
    BIC $PsrMask, R2, R3
    ORR $(PsrDirq|PsrDfiq), R3
```

```
    ORR R0, R3
    MOVW    R3, CPSR        /* switch to new mode */

    MOVW    R13, R0         /* return old sp */
    MOVW    R1, R13         /* install new one */

    MOVW    R2, CPSR        /* switch back to old mode */
    RET
```

11.2.5 Clock initialization (ARM)

⟨*function* `clockinit`*(arm)* 195a⟩≡ (639b)

```
void
clockinit(void)
{
    Systimers *tn;
    Armtimer *tm;
    u32int t0, t1, tstart, tend;

    ⟨clockinit() if many processors 416d⟩

    tn = (Systimers*)SYSTIMERS;
    tstart = tn->clo;
    do{
        t0 = arch_lcycles();
    }while(tn->clo == tstart);
    tend = tstart + 10000;
    do{
        t1 = arch_lcycles();
    }while(tn->clo != tend);
    t1 -= t0;

    cpu->cpuhz = 100 * t1;
    cpu->cpumhz = (cpu->cpuhz + Mhz/2 - 1) / Mhz;

    cpu->cyclefreq = cpu->cpuhz;

    if(cpu->cpuno == 0){
        tn->c3 = tn->clo - 1;

        tm = (Armtimer*)ARMTIMER;
        tm->load = 0;
        tm->ctl = TmrPrescale1|CntEnable|CntWidth32;

        arch_intrenable(IRQtimer3, clockintr, nil, 0, "clock");
    }
    ⟨clockinit() if not cpu0 416e⟩
}
```

11.2.6 Timer initialization

⟨*global* `timersinited` 195b⟩≡ (597a)

```
static bool timersinited;
```

⟨*function* `timersinit` 195c⟩≡ (597a)

```
void
timersinit(void)
{
```

```
    Timer *t;

    timersinited = true;
    todinit();

    t = malloc(sizeof(Timer));
    if(t == nil)
        error(Enomem);
    t->tmode = Tperiodic;
    t->tt = nil;
    t->tns = 1000000000 / Arch_HZ;
    /*
     * T->tf == nil means the HZ clock for this processor.
     */
    t->tf = nil;
    timeradd(t);
}
```

11.2.7 Virtual memory initialisation (ARM)

⟨*function* `mmuinit1`*(arm)* 196a⟩≡ (635a)

```
void
mmuinit1(void *a)
{
    PTE *l1;

    l1 = (PTE*)a;
    cpu->mmul1 = l1;

    /*
     * undo identity map of first MB of ram
     */
    l1[L1X(0)] = 0;
    ⟨mmuinit1() write back cache after adjusting page tables(arm) 402f⟩
    mmuinvalidateaddr(0);
}
```

11.2.8 Memory initialisation

`xinit()`

⟨*function* `xinit` 196b⟩≡ (584b)

```
void
xinit(void)
{
    int i;
    int nkpages;
    int kpages;
    ulong maxpages;
    Confmem *m;
    Pallocmem *pm;
    Hole *h, *eh;

    eh = &xlists.hole[Nhole-1];
    for(h = xlists.hole; h < eh; h++)
        h->next = h+1;

    xlists.unused_slots = xlists.hole;
```

```
    kpages = conf.npage - conf.upages;

    pm = palloc.mem;
    for(i=0; i<nelem(conf.mem); i++){
        m = &conf.mem[i];
        nkpages = m->npage;
        if(nkpages > kpages)
            nkpages = kpages; // will be zero once kpages has been filled
        /* don't try to use non-KADDR-able memory for kernel */
        maxpages = arch_cankaddr(m->base)/BY2PG;
        if(nkpages > maxpages)
            nkpages = maxpages;

        /* first give to kernel */
        if(nkpages > 0){
            xhole(m->base, nkpages*BY2PG);
            kpages -= nkpages;
            ⟨xinit() nkpages kernel memory in m 367a⟩
        }

        /* if anything left over, give to user */
        if(m->npage > nkpages){
            if(pm >= palloc.mem+nelem(palloc.mem)){
                print("xinit: losing %lud pages\n", m->npage-nkpages);
                continue;
            }
            pm->base = m->base+nkpages*BY2PG;
            pm->npage = m->npage - nkpages;
            pm++;
        }
    }
}
```

procinit()

⟨*function* procinit 197a⟩≡ (599c)

```
void
procinit(void)
{
    Proc *p;
    int i;

    procalloc.free = xalloc(conf.nproc * sizeof(Proc));
    ⟨procinit() sanity check procalloc.free 197b⟩
    procalloc.arena = procalloc.free;

    p = procalloc.free;
    for(i=0; i<conf.nproc-1; i++,p++)
        p->qnext = p+1;
    p->qnext = nil;
}
```

⟨procinit() *sanity check* procalloc.free 197b⟩≡ (197a)

```
if(procalloc.free == nil){
    xsummary();
    panic("cannot allocate %lud procs (%ludMB)\n",
                  conf.nproc, conf.nproc*sizeof(Proc)/MB);
}
```

pageinit()

⟨*function* pageinit 198a⟩≡ (588)

```
void
pageinit(void)
{
    int i, j;
    Page *p;
    Pallocmem *pm;
    ulong m, np, k, vkb, pkb;
    ⟨pageinit() other locals 423b⟩

    np = 0;
    for(i=0; i<nelem(palloc.mem); i++){
        pm = &palloc.mem[i];
        np += pm->npage;
    }

    palloc.pages = xalloc(np * sizeof(Page));
    ⟨pageinit() sanity check palloc.pages 198b⟩

    palloc.head = palloc.pages;
    p = palloc.head;
    for(i=0; i<nelem(palloc.mem); i++){
        pm = &palloc.mem[i];
        for(j=0; j < pm->npage; j++){
            p->prev = p-1;
            p->next = p+1;
            p->pa = pm->base + j*BY2PG;
            palloc.freecount++;
            ⟨pageinit() set page color 423c⟩
            p++;
        }
    }
    palloc.tail = p - 1;
    palloc.head->prev = nil;
    palloc.tail->next = nil;

    palloc.user = p - palloc.pages;
    assert(palloc.user == np);

    pkb = palloc.user*BY2PG/KB;
    vkb = pkb + (conf.nswap*BY2PG)/KB;

    /* Paging numbers */
    swapalloc.highwater = (palloc.user*5)/100;
    swapalloc.headroom = swapalloc.highwater + (swapalloc.highwater/4);

    // diagnostic
    m = 0;
    for(i=0; i<nelem(conf.mem); i++)
        if(conf.mem[i].npage)
            m += conf.mem[i].npage*BY2PG;
    k = PGROUND(end - (char*)KTZERO);
    print("%ldM memory: ", (m+k+1024*1024-1)/(1024*1024));
    print("%ldM kernel data, ", (m+k-pkb*1024+1024*1024-1)/(1024*1024));
    print("%ldM user, ", pkb/1024);
    print("%ldM swap\n", vkb/1024);
}
```

⟨pageinit() *sanity check* palloc.pages 198b⟩≡ (198a)

```
  if(palloc.pages == nil)
      panic("pageinit");
```

swapinit()

⟨*function* swapinit 199a⟩≡ (590)

```
  void
  swapinit(void)
  {
      swapalloc.swmap = xalloc(conf.nswap);

      swapalloc.top = &swapalloc.swmap[conf.nswap];
      swapalloc.alloc = swapalloc.swmap;
      swapalloc.last = swapalloc.swmap;
      swapalloc.free = conf.nswap;

      iolist = xalloc(conf.nswppo*sizeof(Page*));

      ⟨swapinit() sanity check xalloc return values 199b⟩
      swapimage.notext = true;
  }
```

⟨swapinit() *sanity check xalloc return values* 199b⟩≡ (199a)

```
  if(swapalloc.swmap == nil || iolist == nil)
      panic("swapinit: not enough memory");
```

imageinit()

⟨*constant* NFREECHAN 199c⟩≡ (582)

```
  #define NFREECHAN 64
```

⟨*function* imageinit 199d⟩≡ (589)

```
  void
  imageinit(void)
  {
      KImage *i, *ie;

      imagealloc.free = xalloc(conf.nimage * sizeof(KImage));
      ⟨imageinit() sanity check imagealloc.free 199e⟩
      ie = &imagealloc.free[conf.nimage-1];
      for(i = imagealloc.free; i < ie; i++)
          i->next = i+1;
      i->next = nil;
      imagealloc.freechan = malloc(NFREECHAN * sizeof(Chan*));
      imagealloc.szfreechan = NFREECHAN;
  }
```

⟨imageinit() *sanity check* imagealloc.free 199e⟩≡ (199d)

```
  if (imagealloc.free == nil)
      panic("imageinit: no memory");
```

11.2.9 Device initialisation

⟨*function* chandevreset 199f⟩≡ (571)

```
  void
  chandevreset(void)
  {
      int i;
```

```
    todinit();  /* avoid later reentry causing infinite recursion */

    for(i=0; devtab[i] != nil; i++) {
        print("reset %d, %s\n", i, devtab[i]->name);
        devtab[i]->reset();
    }
}
```

⟨Dev *methods* 200a⟩≡ (32d) 203b ▷

```
void  (*reset)(void); // done once at boot time
```

11.2.10 Displaying hardware information (ARM)

⟨main() *locals(arm)* 200b⟩≡ (189)

```
uint firmware, board;
```

⟨main() *print board and firmware information(arm)* 200c⟩≡ (189)

```
board = getboardrev();
firmware = getfirmware();
print("board rev: %#ux firmware rev: %d\n", board, firmware);
if(firmware < Minfirmrev){
    print("Sorry, firmware (start*.elf) must be at least rev %d"
          " or newer than %s\n", Minfirmrev, Minfirmdate);
    for(;;)
        ;
}
```

⟨*constant* Minfirmrev*(arm)* 200d⟩≡ (628c)

```
/* Firmware compatibility */
#define Minfirmrev  326770
```

⟨*constant* Minfirmdate*(arm)* 200e⟩≡ (628c)

```
#define Minfirmdate "19 Aug 2013"
```

⟨*function* getboardrev*(arm)* 200f⟩≡ (615d)

```
/*
 * Get board revision
 */
uint
getboardrev(void)
{
    u32int buf[1];

    if(vcreq(TagGetrev, buf, 0, sizeof buf) != sizeof buf)
        return 0;
    return buf[0];
}
```

⟨*function* getfirmware*(arm)* 200g⟩≡ (615d)

```
/*
 * Get firmware revision
 */
uint
getfirmware(void)
{
    u32int buf[1];

    if(vcreq(TagGetfwrev, buf, 0, sizeof buf) != sizeof buf)
        return 0;
    return buf[0];
}
```

11.3 The first process (ARM)

⟨*global* sp*(arm)* 201a⟩≡ (628c)

```
static user_addr sp;      /* XXX - must go - user stack of init proc */
```

⟨*macro* STACKALIGN*(arm)* 201b⟩≡ (633b)

```
#define STACKALIGN(sp)  ((sp) & ~3)     /* bug: assure with alloc */
```

⟨*function* arch__userinit*(arm)* 201c⟩≡ (628c)

```
/*
 *  create the first process
 */
void
arch__userinit(void)
{
    Proc *p;
    Segment *s;
    Arch_KMap *k;
    Page *pg;

    /* no processes yet */
    up = nil;

    p = newproc();
    p->pgrp = newpgrp();
    p->egrp = smalloc(sizeof(Egrp));
    p->egrp->ref = 1;
    p->fgrp = dupfgrp(nil);
    p->rgrp = newrgrp();
    p->procmode = 0640;

    kstrdup(&eve, "");
    kstrdup(&p->text, "*init*");
    kstrdup(&p->user, eve);

    /*
     * Kernel Stack
     */
    p->sched.pc = PTR2UINT(init0);
    p->sched.sp = PTR2UINT(p->kstack + KSTACK
                           - sizeof(up->sargs.args)
                           - sizeof(uintptr));
    p->sched.sp = STACKALIGN(p->sched.sp);

    /*
     * User Stack
     *
     * Technically, newpage can't be called here because it
     * should only be called when in a user context as it may
     * try to sleep if there are no pages available, but that
     * shouldn't be the case here.
     */
    s = newseg(SG_STACK, USTKTOP-USTKSIZE, USTKSIZE/BY2PG);
    ⟨arch__userinit() set flushme for stack segment 424e⟩
    p->seg[SSEG] = s;
    pg = newpage(true, nil, USTKTOP-BY2PG);
    segpage(s, pg);

    k = arch_kmap(pg);
    sp = bootargs(VA(k));
```

```
    arch_kunmap(k);

    /*
     * Text
     */
    s = newseg(SG_TEXT, UTZERO, 1); // initcode needs only 1 page
    p->seg[TSEG] = s;
    pg = newpage(true, nil, UTZERO);
    ⟨arch__userinit() set cachectl(arm) 424b⟩
    segpage(s, pg);

    k = arch_kmap(s->pagedir[0]->pagetab[0]);
    memmove(UINT2PTR(VA(k)), initcode, sizeof initcode);
    arch_kunmap(k);

    // ready to go!
    ready(p);
}
```

⟨*function* init0*(arm)* 202⟩≡ (628c)

```
/*
 *  starting place for first process
 */
void
init0(void)
{
    int i;
    char buf[2*KNAMELEN]; // has to be the same than in ksetenv?

    up->nerrlab = 0;

    ⟨init0() coherence(arm) 405d⟩
    arch_spllo(); // Allow interrupts!!!

    /*
     * These are o.k. because rootinit is null.
     * Then early kproc's will have a root and dot.
     */
    up->slash = namec("#/", Atodir, 0, 0);
    pathclose(up->slash->path);
    up->slash->path = newpath("/");
    up->dot = cclone(up->slash);

    chandevinit();

    if(!waserror()){
        snprint(buf, sizeof(buf), "%s %s", "ARM", conffile);
        ksetenv("terminal", buf, 0);
        ksetenv("cputype", "arm", 0); // used by mkfile!
        if(cpuserver)
            ksetenv("service", "cpu", 0);
        else
            ksetenv("service", "terminal", 0);
        snprint(buf, sizeof(buf), "-a %s", getethermac());
        ksetenv("etherargs", buf, 0);

        /* convert plan9.ini variables to #e and #ec */
        for(i = 0; i < nconf; i++) {
            ksetenv(confname[i], confval[i], 0);
            ksetenv(confname[i], confval[i], 1);
```

```
        }
        poperror();
    }
    kproc("alarm", alarmkproc, nil); // ??

    // !! Go to user space !!
    arch_touser(sp);

    assert(0);          /* shouldn't have returned */
}
```

⟨*function* chandevinit 203a⟩≡ (571)

```
void
chandevinit(void)
{
    int i;
    for(i=0; devtab[i] != nil; i++) {
        devtab[i]->init();
    }
}
```

⟨Dev *methods* 203b⟩+≡ (32d) ◁200a 214c▷

```
void  (*init)(void);
```

⟨*function* arch_touser*(arm)* 203c⟩≡ (643a)

```
/*
 *  This is the first jump from kernel to user mode.
 *  Fake a return from interrupt.
 *
 *  Enter with R0 containing the user stack pointer.
 *  UTZERO + 0x20 is always the entry point.
 *
 */
TEXT arch_touser(SB), 1, $-4
    /* store the user stack pointer into the USR_r13 */
    MOVM.DB.W [R0], (R13)
    /* avoid the ambiguity described in notes/movm.w. */
//  MOVM.S.IA.W (R13), [R13]
    MOVM.S  (R13), [R13]
    ADD $4, R13

    /* set up a PSR for user level */
    MOVW    $(PsrMusr), R0
    MOVW    R0, SPSR

    /* save the PC on the stack */
    MOVW    $(UTZERO+0x20), R0
    MOVM.DB.W [R0], (R13)

    /*
     * note that 5a's RFE is not the v6 arch. instruction (0xe89d0a00,
     * I think), which loads CPSR from the word after the PC at (R13),
     * but rather the pre-v6 simulation 'MOVM.IA.S.W (R13), [R15]'
     * (0xe8fd8000 since MOVM is LDM in this case), which loads CPSR
     * not from memory but from SPSR due to '.S'.
     */
    RFE
```

⟨init9.c 204a⟩≡

```
//@Scheck: def can actually be found initcode.c, but this file is in skip list
extern void startboot(char*, char**);

//@Scheck: entry point looked for by the linker 8l
void
_main(char *argv0)
{
    startboot(argv0, &argv0);
}
```

⟨init/user/preboot/arm/init9.s 204b⟩≡

```
/*
 * This is the same as the C program:
 *
 *  void
 *  main(char* argv0)
 *  {
 *      startboot(argv0, &argv0);
 *  }
 *
 * It is in assembler because SB needs to be
 * set and doing this in C drags in too many
 * other routines.
 */
TEXT main(SB), 1, $8
    MOVW    $setR12(SB), R12        /* load the SB */
    MOVW    $boot(SB), R0

    ADD $12, R13, R1            /* pointer to 0(FP) */

    MOVW    R0, 4(R13)          /* pass argc, argv */
    MOVW    R1, 8(R13)

    BL  startboot(SB)
_loop:
    B   _loop
```

⟨initcode.c 204c⟩≡

```
/*
 * IMPORTANT!  DO NOT ADD LIBRARY CALLS TO THIS FILE.
 * The entire text image must fit on one page
 * (and there's no data segment, so any read/write data must be on the stack).
 */

#include <u.h>
#include <libc.h>

char cons[] = "#c/cons";
char boot[] = "/boot/boot";
char dev[] = "/dev";
char c[] = "#c";
char e[] = "#e";
char ec[] = "#ec";
char s[] = "#s";
char srv[] = "/srv";
char env[] = "/env";

// called from init9.s
void
```

```
startboot(char *argv0, char **argv)
{
    char buf[200];  /* keep this fairly large to capture error details */

    /* in case boot is a shell script */
    open(cons, OREAD);
    open(cons, OWRITE);
    open(cons, OWRITE);

    bind(c, dev, MAFTER);
    bind(ec, env, MAFTER);
    bind(e, env, MCREATE|MAFTER);
    bind(s, srv, MREPL|MCREATE);

    USED(argv0);
    exec(boot, argv);

    rerrstr(buf, sizeof buf);
    buf[sizeof buf - 1] = '\0';
    _exits(buf);
}
```

11.4 Booting the user

Chapter 12

Files

⟨systab *file syscalls* 206a⟩≡ (26c)

```
[OPEN]      sysopen,
[CLOSE]     sysclose,
[PREAD]     syspread,
[PWRITE]    syspwrite,
[SEEK]      sysseek,
```

12.1 Overview

12.2 Channels

⟨*enum* channelflag 206b⟩≡ (569c)

```
/*
 * channel flags
 */
enum ChannelFlag
{
  COPEN = 0x0001,   /* for i/o */
  CFREE = 0x0010,   /* not in use */
  ⟨ChannelFlag cases 228b⟩
};
```

⟨Chan *other fields* 206c⟩≡ (31a) 260a ▷

```
// enum<channelflag>> (actually a bitset for certain properties)
ushort  flag;
```

⟨*struct* Chanalloc 206d⟩≡ (571)

```
// Channel allocator
struct Chanalloc
{
    //list<ref<Chan>> (next = Chan.next), the free one
    Chan    *free;
    //list<ref<Chan>> (next = Chan.link), the used one
    Chan    *list;

    int fid; // type could be 'Counter', but already have a Lock anyway

    // extra
    Lock;
};
```

⟨Chan *extra fields* 207a⟩≡ (31a)

```
// list<ref<Chan> of Chanalloc.free
Chan* next;     /* allocation */
// list<ref<Chan> of Chanalloc.list
Chan* link;
```

⟨*global* chanalloc 207b⟩≡ (571)

```
struct Chanalloc chanalloc;
```

⟨*constructor* newchan 207c⟩≡ (571)

```
Chan*
newchan(void)
{
    Chan *c;

    lock(&chanalloc);
    c = chanalloc.free;
    if(c != nil)
        chanalloc.free = c->next;
    unlock(&chanalloc);

    if(c == nil){
        c = smalloc(sizeof(Chan));
        lock(&chanalloc);
        c->fid = ++chanalloc.fid;
        c->link = chanalloc.list;
        chanalloc.list = c;
        unlock(&chanalloc);
    }

    /* if you get an error before associating with a dev,
       close calls rootclose, a nop */
    c->type = 0;
    c->dev = 0;

    c->path = nil;

    c->offset = 0;
    c->devoffset = 0;
    // c->mode?

    c->flag = 0;
    c->ref = 1;

    c->ismtpt = false;
    c->umh = nil;
    // c->umc?
    c->uri = 0;

    c->mux = nil;
    c->mchan = nil;
    memset(&c->mqid, 0, sizeof(c->mqid));
    c->mcp = nil;
    c->iounit = 0;
    c->dri = 0;

    c->aux = nil;

    return c;
}
```

⟨*destructor* chanfree 208a⟩≡ (571)

```
void
chanfree(Chan *c)
{
    c->flag = CFREE;

    ⟨chanfree() optional free 260b⟩

    pathclose(c->path);
    c->path = nil;

    lock(&chanalloc);
    c->next = chanalloc.free;
    chanalloc.free = c;
    unlock(&chanalloc);
}
```

⟨*function* uniquechan 208b⟩≡ (571)

```
/*
 * Make sure we have the only copy of c.  (Copy on write.)
 */
Chan*
uniquechan(Chan *c)
{
    Chan *nc;

    if(c->ref != 1){
        nc = cclone(c);
        cclose(c);
        return nc;
    }else{
      return c;
    }
}
```

12.3 Devices table devtab

⟨*function* devattach 208c⟩≡ (573)

```
Chan*
devattach(Rune tc, char *spec)
{
    int n;
    Chan *c;
    char *buf;

    c = newchan();
    mkqid(&c->qid, 0, 0, QTDIR); // root of a device is a QTDIR
    c->type = devno(tc, false);

    if(spec == nil)
        spec = "";
    n = 1+UTFmax+strlen(spec)+1; // '#' + Rune + spec + '\0'
    buf = smalloc(n);
    snprint(buf, n, "#%C%s", tc, spec);
    c->path = newpath(buf);
    free(buf);

    return c;
}
```

⟨*function* devno 209a⟩≡ (573)

```
int
devno(Rune c, bool user)
{
    int i;

    for(i = 0; devtab[i] != nil; i++) {
        if(devtab[i]->dc == c)
            return i;
    }
    if(!user)
        panic("devno %C %#ux", c, c);

    return -1;
}
```

12.4 File descriptors, Fgrp

⟨*struct* Fgrp 209b⟩≡ (592c)

```
struct Fgrp
{
    // array<option<ref_counted<Chan>>>
    Chan  **fd;
    // nelem(fd)
    int nfd;      /* number allocated */
    int maxfd;      /* highest fd in use */ // <= nfd

    ⟨Fgrp other fields 212a⟩
    // extra
    Ref;
};
```

⟨Proc *files fields* 209c⟩+≡ (29c) ◁31b 226a▷

```
// ref_counted<Fgrp>
Fgrp  *fgrp;    /* File descriptor group */
```

⟨*function* dupfgrp 209d⟩≡ (597b)

```
Fgrp*
dupfgrp(Fgrp *f)
{
    Fgrp *new;
    Chan *c;
    int i;

    new = smalloc(sizeof(Fgrp));
    if(f == nil){
        new->fd = smalloc(DELTAFD*sizeof(Chan*));
        new->nfd = DELTAFD;
        new->ref = 1;
        return new;
    }

    lock(f);
    /* Make new fd list shorter if possible, preserving quantization */
    new->nfd = f->maxfd+1;
    i = new->nfd%DELTAFD;
    if(i != 0)
        new->nfd += DELTAFD - i;
```

```
    new->fd = malloc(new->nfd*sizeof(Chan*));
    if(new->fd == nil){
        unlock(f);
        free(new);
        error("no memory for fgrp");
    }
    new->ref = 1;

    new->maxfd = f->maxfd;
    for(i = 0; i <= f->maxfd; i++) {
        if(c = f->fd[i]){
            incref(c);
            new->fd[i] = c;
        }
    }
    unlock(f);

    return new;
}
```

⟨*constant* DELTAFD 210a⟩≡ (592c)

```
DELTAFD = 20    /* incremental increase in Fgrp.fd's */
```

⟨*function* closefgrp 210b⟩≡ (597b)

```
void
closefgrp(Fgrp *f)
{
    int i;
    Chan *c;

    if(f == nil)
        return;

    if(decref(f) != 0)
        return;

    /*
     * If we get into trouble, forceclosefgrp
     * will bail us out.
     */
    up->closingfgrp = f;
    for(i = 0; i <= f->maxfd; i++)
        if(c = f->fd[i]){
            f->fd[i] = nil;
            cclose(c);
        }
    up->closingfgrp = nil;

    free(f->fd);
    free(f);
}
```

⟨*function* newfd 210c⟩≡ (578)

```
int
newfd(Chan *c)
{
    int fd;
    Fgrp *f;

    f = up->fgrp;
```

```
    lock(f);
    fd = findfreefd(f, 0);
    if(fd < 0){
        unlockfgrp(f);
        return -1;
    }
    if(fd > f->maxfd)
        f->maxfd = fd;
    f->fd[fd] = c;
    unlockfgrp(f);
    return fd;
}
```

⟨*function* findfreefd 211a⟩≡ (578)

```
/*
 *  this assumes that the fgrp is locked
 */
int
findfreefd(Fgrp *f, int start)
{
    int fd;

    for(fd=start; fd<f->nfd; fd++)
        if(f->fd[fd] == nil)
            break;
    if(fd >= f->nfd && growfd(f, fd) < 0)
        return -1;
    return fd;
}
```

⟨*function* growfd 211b⟩≡ (578)

```
int
growfd(Fgrp *f, int fd) /* fd is always >= 0 */
{
    Chan **newfd, **oldfd;

    if(fd < f->nfd)
        return 0; // should never happen no?
    if(fd >= f->nfd+DELTAFD)
        return -1;  /* out of range */ // when can this happen? from sysdup?
    /*
     * Unbounded allocation is unwise
     */
    if(f->nfd >= 5000){
    Exhausted:
        print("no free file descriptors\n");
        return -1;
    }
    // realloc
    newfd = malloc((f->nfd+DELTAFD)*sizeof(Chan*));
    if(newfd == nil)
        goto Exhausted;
    oldfd = f->fd;
    memmove(newfd, oldfd, f->nfd*sizeof(Chan*));
    f->fd = newfd;
    free(oldfd);
    f->nfd += DELTAFD;
    if(fd > f->maxfd){
        if(fd/100 > f->maxfd/100)
            f->exceed = (fd/100)*100;
```

```
        f->maxfd = fd;
    }
    return 1;
}
```

⟨Fgrp *other fields* 212a⟩≡ (209b)

```
int exceed;     /* debugging */
```

⟨*function* unlockfgrp 212b⟩≡ (578)

```
static void
unlockfgrp(Fgrp *f)
{
    int ex;

    ex = f->exceed;
    f->exceed = 0;
    unlock(f);
    if(ex)
        pprint("warning: process exceeds %d file descriptors\n", ex);
}
```

12.5 sysopen(), sysclose()

⟨*enum* open 212c⟩≡ (580e)

```
enum OpenFlags {
    OREAD = 0, /* open for read */
    OWRITE = 1, /* write */
    ORDWR = 2, /* read and write */
    ⟨OpenFlags cases 215a⟩
};
```

⟨*syscall open* 212d⟩≡ (578)

```
// int open(char *file, int omode);
long
sysopen(ulong* arg)
{
    int fd;
    Chan *c;

    openmode(arg[1]);   /* error check only */
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aopen, arg[1], 0);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    fd = newfd(c);
    if(fd < 0)
        error(Enofd);
    poperror();
    return fd;
}
```

⟨*function* openmode 212e⟩≡ (575b)

```
int
openmode(ulong o)
{
    o &= ~(OTRUNC|OCEXEC|ORCLOSE);
```

```
    if(o > OEXEC)
        error(Ebadarg);
    if(o == OEXEC)
        return OREAD;
    return o;
}
```

⟨*syscall close* 213a⟩≡ (578)

```
// int close(int fd);
long
sysclose(ulong* arg)
{
    fdtochan(arg[0], -1, false, false); // for its error checking side effect
    fdclose(arg[0], 0);
    return 0;
}
```

⟨*function* fdtochan 213b⟩≡ (575b)

```
Chan*
fdtochan(int fd, int mode, bool chkmnt, bool iref)
{
    Chan *c;
    Fgrp *f;

    c = nil;
    f = up->fgrp;

    lock(f);
    if(fd<0 || f->nfd<=fd || (c = f->fd[fd])==nil) {
        unlock(f);
        error(Ebadfd);
    }
    // c = f->fd[fd] != nil

    if(iref)
        incref(c);
    unlock(f);

    if(chkmnt && (c->flag&CMSG)) {
        ⟨fdtochan() undo iref incref 214a⟩
        error(Ebadusefd);
    }

    if(mode<0 || c->mode==ORDWR) // mode < 0? mean no check?
        return c;

    if((mode&OTRUNC) && c->mode==OREAD) { // BUG impossible, dead code
        ⟨fdtochan() undo iref incref 214a⟩
        error(Ebadusefd);
    }
    // the access mode must match the opening mode
    if((mode&~OTRUNC) != c->mode) {
        ⟨fdtochan() undo iref incref 214a⟩
        error(Ebadusefd);
    }

    return c;
}
```

⟨fdtochan() *undo iref incref* 214a⟩≡ (213)

```
if(iref)
    cclose(c);
```

⟨*function* fdclose 214b⟩≡ (575b)

```
void
fdclose(int fd, int flag)
{
    int i;
    Chan *c;
    Fgrp *f = up->fgrp;

    lock(f);
    c = f->fd[fd]; // bound checking??
    if(c == nil){
        /* can happen for users with shared fd tables */
        unlock(f);
        return;
    }
    if(flag){
        if(c==nil || !(c->flag&flag)){ // dead test c==nil? see above
            unlock(f);
            return;
        }
    }
    f->fd[fd] = nil;

    if(fd == f->maxfd)
        for(i=fd; --i>=0 && f->fd[i]==nil; )
            f->maxfd = i;

    unlock(f);
    cclose(c);
}
```

⟨Dev *methods* 214c⟩+≡ (32d) ◁203b 217a▷

```
void  (*close)(Chan*);
```

⟨*function* cclose 214d⟩≡ (571)

```
void
chan_cclose(Chan *c)
{
    if(c->flag&CFREE)
        panic("cclose %#p", getcallerpc(&c));

    DBG("cclose %p name=%s ref=%ld\n", c, c->path->s, c->ref);
    if(decref(c))
        return;

    if(!waserror()){
        devtab[c->type]->close(c);
        poperror();
    }
    chanfree(c);
}
```

12.6 namec()

⟨sysexec() *call namec() to get a channel in tc from file* 214e⟩≡ (83i)

```
tc = namec(file, Aopen, OEXEC, 0);
```

⟨OpenFlags *cases* 215a⟩≡ (212c) 228a ▷

```
OEXEC = 3, /* execute, == read but check execute permission */
```

⟨*enum* accessnamec 215b⟩≡ (569c)

```
/*
 * Access types in namec
 */
enum AccessFlags
{
  Aopen,        /* for i/o */
  Acreate,      /* is to be created */
  Aremove,      /* will be removed by caller */
  Aaccess,      /* as in stat, wstat */
  Atodir,       /* as in chdir */
  ⟨AccessFlags cases 252a⟩
};
```

⟨namec() *locals* 215c⟩≡ (215f) 215d ▷

```
char *name;
```

⟨namec() *locals* 215d⟩+≡ (215f) ◁ 215c 215e ▷

```
Elemlist e;
```

⟨namec() *locals* 215e⟩+≡ (215f) ◁ 215d ?? ▷

```
Chan *c;
```

⟨*function* namec 215f⟩≡ (571)

```
/*
 * Turn a name into a channel.
 * &name[0] is known to be a valid address.  It may be a kernel address.
 *
 * Opening with amode Aopen, Acreate, Aremove, or Aaccess guarantees
 * that the result will be the only reference to that particular fid.
 * This is necessary since we might pass the result to
 * devtab[]->remove().
 *
 * Opening Atodir or Amount does not guarantee this.
 *
 * Under certain circumstances, opening Aaccess will cause
 * an unnecessary clone in order to get a uniquechan Chan so it
 * can attach the correct name.  Sysstat and sys_stat need the
 * correct name so they can rewrite the stat info.
 */
Chan*
namec(char *aname, int amode, int omode, ulong perm)
{
    ⟨namec() locals 215c⟩

    if(aname[0] == '\0')
        error("empty file name");

    aname = validnamedup(aname, true);

    if(waserror()){
        free(aname);
        nexterror();
    }
    DBG("namec %s %d %d\n", aname, amode, omode);
    name = aname;
```

```
sharppath = (name[0] == '#');

/*
 * Find the starting off point (the current slash, the root of
 * a device tree, or the current dot) as well as the name to
 * evaluate starting there.
 */
switch(name[0]){
⟨namec() if name[0] is a sharp 251a⟩
case '/':
    c = up->slash;
    incref(c);
    break;

default:
    c = up->dot;
    incref(c);
    break;
}

⟨namec() initializes Elemlist e 219c⟩

if(waserror()){
    cclose(c);
    ⟨namec() if waserror free Elemlist e and prepare nice error 221b⟩
}

/*
 * Build a list of elements in the name.
 */
parsename(name, &e);

/*
 * On create, ....
 */
⟨namec() adjust Elemlist e if Acreate 238b⟩

if(walk(&c, e.elems, e.nelems, sharppath, &e.nerror) < 0){
    if(e.nerror < 0 || e.nerror > e.nelems){
        print("namec %s walk error nerror=%d\n", aname, e.nerror);
        e.nerror = 0;
    }
    nexterror();
}

⟨namec() error if not a directory or cannot exec directory 230e⟩

switch(amode){
case Aopen:
case Aremove:
case Aaccess:
Open:
    ⟨namec() case Aopen, Acreate, Aremove, Aaccess, handle mountpoint part1 256b⟩

    switch(amode){
    case Aopen:
    case Acreate:
        ⟨namec() print error if c-¿umh != nil 260c⟩
        ⟨namec() case Aopen, Acreate, handle mountpoint part2 259d⟩
```

```
            ⟨namec() set channel flag before open ??⟩
            c = devtab[c->type]->open(c, omode&~OCEXEC);
            ⟨namec() set channel flag after open 228c⟩
            break;

        case Aaccess:
        case Aremove:
            ⟨namec() case Aremove, Aaccess, handle mountpoint part2 256c⟩
            break;

        }
        break;
    ⟨namec() other cases 238e⟩
    default:
        panic("unknown namec access %d\n", amode);
    }

    ⟨namec() set genbuf from Elemlist e 217b⟩

    ⟨namec() free Elemlist e 219d⟩
    poperror(); /* e c */
    free(aname);
    poperror(); /* aname */

    return c;
}
```

⟨Dev *methods* 217a⟩+≡ (32d) ◁214c 222c▷

```
Chan* (*open)(Chan*, int);
```

⟨namec() *set genbuf from Elemlist e* 217b⟩≡ (215f)

```
/* place final element in genbuf for e.g. exec */
if(e.nelems > 0)
    kstrcpy(up->genbuf, e.elems[e.nelems-1], sizeof up->genbuf);
else
    kstrcpy(up->genbuf, ".", sizeof up->genbuf);
```

⟨Proc *other fields* 217c⟩+≡ (29c) ◁175h 387b▷

```
char  genbuf[128];  /* buffer used e.g. for last name element from namec */
```

12.6.1 Checking valid filename string

⟨*function* validnamedup 217d⟩≡ (571)

```
char*
validnamedup(char *aname, bool slashok)
{
    return validname0(aname, slashok, true, getcallerpc(&aname));
}
```

⟨*function* validname 217e⟩≡ (571)

```
void
validname(char *aname, bool slashok)
{
    validname0(aname, slashok, false, getcallerpc(&aname));
}
```

⟨*function* validname0 218⟩≡ (571)

```
/*
 * Check that the name
 *  a) is in valid memory.
 *  b) is shorter than 2^16 bytes, so it can fit in a 9P string field.
 *  c) contains no frogs.
 * The first byte is known to be addressible by the requester, so the
 * routine works for kernel and user memory both.
 * The parameter slashok flags whether a slash character is an error
 * or a valid character.
 *
 * The parameter dup flags whether the string should be copied
 * out of user space before being scanned the second time.
 * (Otherwise a malicious thread could remove the NUL, causing us
 * to access unchecked addresses.)
 */
static char*
validname0(char *aname, bool slashok, bool dup, kern_addr pc)
{
    char *ename, *name, *s;
    int n;
    int c;
    Rune r;

    name = aname;

    if((virt_addr)name < KZERO){
        if(!dup)
            print("warning: validname called from %#p with user pointer", pc);
        ename = vmemchr(name, 0, (1<<16));
    }else
        ename = memchr(name, 0, (1<<16));

    if(ename==nil || ename-name>=(1<<16))
        error("name too long");

    s = nil;
    if(dup){
        n = ename-name;
        s = smalloc(n+1);
        memmove(s, name, n);
        s[n] = '\0';
        aname = s;
        name = s;
        setmalloctag(s, pc);
    }

    // checking validity of name
    while(*name){
        /* all characters above '~' are ok */
        c = *(byte*)name;
        if(c >= Runeself)
            name += chartorune(&r, name);
        else{
            if(isfrog[c])
                if(!slashok || c!='/'){
                    snprint(up->genbuf, sizeof(up->genbuf), "%s: %q", Ebadchar, aname);
                    free(s);
                    error(up->genbuf);
            }
```

```
            name++;
        }
    }
    return s;
}
```

⟨*global* isfrog 219a⟩≡ (571)

```
char isfrog[256]={
    /*NUL*/ 1, 1, 1, 1, 1, 1, 1, 1,
    /*BKS*/ 1, 1, 1, 1, 1, 1, 1, 1,
    /*DLE*/ 1, 1, 1, 1, 1, 1, 1, 1,
    /*CAN*/ 1, 1, 1, 1, 1, 1, 1, 1,
    ['/']   1,
    [0x7f]  1,
};
```

12.6.2 Parsing filename

⟨*struct* Elemlist 219b⟩≡ (571)

```
struct Elemlist
{
    char    *aname; /* original name */
    char    *name;  /* copy of name, so '/' can be overwritten */

    //array<string> pointing to subparts of name
    char    **elems; // subparts
    // size(elems)
    int nelems;
    //array<int>, offset in name for the subparts of name in elems
    int *off;

    bool mustbedir; // if last part of filename is a '/'

    int nerror;
    int prefix;
};
```

⟨namec() *initializes Elemlist e* 219c⟩≡ (215f)

```
e.aname = aname;
e.prefix = name - aname;
e.name = nil;
e.elems = nil;
e.off = nil;
e.nelems = 0;
e.nerror = 0;
```

⟨namec() *free Elemlist e* 219d⟩≡ (215f)

```
free(e.name);
free(e.elems);
free(e.off);
```

⟨*function* parsename 219e⟩≡ (571)

```
/*
 * The name is known to be valid.
 * Copy the name so slashes can be overwritten.
 * An empty string will set nelem=0.
 * A path ending in / or /. or /.//./ etc. will have
 * e.mustbedir = true, so that we correctly
 * reject, e.g., "/adm/users/." when /adm/users is a file
```

```
 * rather than a directory.
 */
static void
parsename(char *aname, Elemlist *e)
{
    char *name, *slash;

    kstrdup(&e->name, aname);
    name = e->name;
    e->nelems = 0;
    e->elems = nil;
    e->off = smalloc(sizeof(int));
    e->off[0] = skipslash(name) - name;
    for(;;){
        name = skipslash(name);
        if(*name == '\0'){
            e->off[e->nelems] = name+strlen(name) - e->name;
            e->mustbedir = true;
            break;
        }
        growparse(e);
        e->elems[e->nelems++] = name;
        slash = utfrune(name, '/');
        if(slash == nil){
            e->off[e->nelems] = name+strlen(name) - e->name;
            e->mustbedir = false;
            break;
        }
        e->off[e->nelems] = slash - e->name;
        *slash++ = '\0';
        name = slash;
    }
}
```

⟨*function* growparse 220a⟩≡ (571)

```
static void
growparse(Elemlist *e)
{
    char **new;
    int *inew;
    enum { Delta = 8 };

    if(e->nelems % Delta == 0){
        new = smalloc((e->nelems+Delta) * sizeof(char*));
        memmove(new, e->elems, e->nelems*sizeof(char*));
        free(e->elems);
        e->elems = new;
        inew = smalloc((e->nelems+Delta+1) * sizeof(int));
        memmove(inew, e->off, (e->nelems+1)*sizeof(int));
        free(e->off);
        e->off = inew;
    }
}
```

⟨*function* skipslash 220b⟩≡ (571)

```
/*
 * name is valid. skip leading / and ./ as much as possible
 */
char*
skipslash(char *name)
```

```
{
    while(name[0]=='/' || (name[0]=='.' && (name[1]=='\0' || name[1]=='/')))
        name++;
    return name;
}
```

12.6.3 Error management

⟨namec() *locals* 221a⟩+≡ (215f) ◁?? 234g▷

```
char tmperrbuf[ERRMAX];
```

⟨namec() *if waserror free Elemlist e and prepare nice error* 221b⟩≡ (215f)

```
free(e.name);
free(e.elems);
/*
 * Prepare nice error, showing first e.nerror elements of name.
 */
if(e.nerror == 0)
    nexterror();
strcpy(tmperrbuf, up->errstr);
if(e.off[e.nerror]==0)
    print("nerror=%d but off=%d\n",e.nerror, e.off[e.nerror]);
len = e.prefix+e.off[e.nerror];
free(e.off);
namelenerror(aname, len, tmperrbuf);
```

⟨*function* namelenerror 221c⟩≡ (571)

```
void
namelenerror(char *aname, int len, char *err)
{
    char *ename, *name, *next;
    int i, errlen;

    /*
     * If the name is short enough, just use the whole thing.
     */
    errlen = strlen(err);
    if(len < ERRMAX/3 || len+errlen < 2*ERRMAX/3)
        snprint(up->genbuf, sizeof up->genbuf, "%.*s",
            utfnlen(aname, len), aname);
    else{
        /*
         * Print a suffix of the name, but try to get a little info.
         */
        ename = aname+len;
        next = ename;
        do{
            name = next;
            next = memrchr(aname, '/', name-aname);
            if(next == nil)
                next = aname;
            len = ename-next;
        }while(len < ERRMAX/3 || len + errlen < 2*ERRMAX/3);

        /*
         * If the name is ridiculously long, chop it.
         */
        if(name == ename){
            name = ename-ERRMAX/4;
```

```
            if(name <= aname)
                panic("bad math in namelenerror");
            /* walk out of current UTF sequence */
            for(i=0; (*name&0xC0)==0x80 && i<UTFmax; i++)
                name++;
        }
        snprint(up->genbuf, sizeof up->genbuf, "...%.*s",
            utfnlen(name, ename-name), name);
    }
    snprint(up->errstr, ERRMAX, "%#q %s", up->genbuf, err);
    nexterror();
}
```

⟨*function* nameerror 222a⟩≡ (571)

```
void
nameerror(char *name, char *err)
{
    namelenerror(name, strlen(name), err);
}
```

12.7 syspread(), syspwrite()

⟨*syscall pread* 222b⟩≡ (578)

```
// long pread(int fd, void *buf, long nbytes, vlong offset);
long
syspread(ulong* arg)
{
    vlong v;
    va_list list;

    /* use varargs to guarantee alignment of vlong */
    va_start(list, arg[2]);
    v = va_arg(list, vlong);
    va_end(list);

    if(v == ~0ULL)
        return read(arg, nil);

    return read(arg, &v);
}
```

⟨Dev *methods* 222c⟩+≡ (32d) ◁217a 223b▷

```
long  (*read)(Chan*, void*, long, vlong);
```

⟨*function* read 222d⟩≡ (578)

```
// long pread(int fd, void *buf, long nbytes, vlong offset);
static long
read(ulong *arg, vlong *offp)
{
    long n, nn, nnn;
    byte *p;
    Chan *c;
    vlong off;

    n = arg[2];
    validaddr(arg[1], n, true);
    p = (void*)arg[1];
    c = fdtochan(arg[0], OREAD, true, true);
```

```
    if(waserror()){
        cclose(c);
        nexterror();
    }

    if(offp == nil) /* use and maintain channel's offset */
        off = c->offset;
    else
        off = *offp;

    if(off < 0)
        error(Enegoff);

    if(off == 0){   /* rewind to the beginning of the directory */
        if(offp == nil){
            c->offset = 0;
            c->devoffset = 0;
        }
        ⟨read() rewind when off == 0 261b⟩
    }

    ⟨read() if c is a QTDIR 260g⟩
    else
        nnn = nn = devtab[c->type]->read(c, p, n, off);

    lock(c);
    c->offset += nnn;
    c->devoffset += nn;
    unlock(c);

    poperror();
    cclose(c);

    return nnn;
}
```

⟨*syscall pwrite* 223a⟩≡ (578)

```
// long pwrite(int fd, void *buf, long nbytes, vlong offset);
long
syspwrite(ulong* arg)
{
    vlong v;
    va_list list;

    /* use varargs to guarantee alignment of vlong */
    va_start(list, arg[2]);
    v = va_arg(list, vlong);
    va_end(list);

    if(v == ~0ULL)
        return write(arg, nil);

    return write(arg, &v);
}
```

⟨Dev *methods* 223b⟩+≡ (32d) ◁222c 233c▷

```
long  (*write)(Chan*, void*, long, vlong);
```

⟨*function* write 224a⟩≡ (578)

```
// long pwrite(int fd, void *buf, long nbytes, vlong offset);
static long
write(ulong *arg, vlong *offp)
{
    Chan *c;
    long m, n;
    vlong off;

    validaddr(arg[1], arg[2], false);
    n = 0;
    c = fdtochan(arg[0], OWRITE, true, true);
    if(waserror()) {
        if(offp == nil){
            lock(c);
            c->offset -= n;
            unlock(c);
        }
        cclose(c);
        nexterror();
    }

    if(c->qid.type & QTDIR)
        error(Eisdir);

    n = arg[2];

    if(offp == nil){    /* use and maintain channel's offset */
        lock(c);
        off = c->offset;
        c->offset += n;
        unlock(c);
    }else
        off = *offp;

    if(off < 0)
        error(Enegoff);

    m = devtab[c->type]->write(c, (void*)arg[1], n, off);

    if(offp == nil && m < n){
        lock(c);
        c->offset -= n - m;
        unlock(c);
    }

    poperror();
    cclose(c);

    return m;
}
```

12.8 sysseek()

⟨*syscall seek* 224b⟩≡ (578)

```
// vlong seek(int fd, vlong n, int type);
long
sysseek(ulong* arg)
```

```
{
    validaddr(arg[0], BY2V, true);
    arch_validalign(arg[0], sizeof(vlong));
    sseek(arg);
    return 0;
}
```

⟨*function* sseek 225⟩≡ (578)

```
union v_or_u2 {
  vlong v;
  ulong u[2];
};

// vlong seek(int fd, vlong n, int type);
static void
sseek(ulong *arg)
{
    Chan *c;
    byte buf[sizeof(DirEntry)+100];
    DirEntry dir;
    int n;
    vlong off;
    union v_or_u2 o;

    c = fdtochan(arg[1], -1, true, true);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    ⟨sseek() and pipes 342a⟩

    off = 0;
    o.u[0] = arg[2];
    o.u[1] = arg[3];
    switch(arg[4]){
    case 0: // from the start
        off = o.v;
        ⟨sseek() ensures off is 0 for directories 230c⟩
        if(off < 0)
            error(Enegoff);
        c->offset = off; // just write, no need for lock
        break;

    case 1: // from the current location
        ⟨sseek() disallows seek type 1 or 2 for directories 230d⟩
        lock(c);    /* lock for read/write update */
        off = o.v + c->offset;
        if(off < 0){
            unlock(c);
            error(Enegoff);
        }
        c->offset = off;
        unlock(c);
        break;

    case 2: // from the end of the file
        ⟨sseek() disallows seek type 1 or 2 for directories 230d⟩
        n = devtab[c->type]->stat(c, buf, sizeof buf);
        if(convM2D(buf, n, &dir, nil) == 0)
            error("internal error: stat error in seek");
```

```
        off = dir.length + o.v;
        if(off < 0)
            error(Enegoff);
        c->offset = off;
        break;

    default:
        error(Ebadarg);
    }
    *(vlong*)arg[0] = off;
    c->uri = 0;
    c->dri = 0;
    cclose(c);
    poperror();
}
```

12.9 Collecting unclosed files

⟨Proc *files fields* 226a⟩+≡ (29c) ◁209c 249d▷

```
Fgrp  *closingfgrp; /* used during teardown */
```

⟨sleep() *forceclosefgrp* 226b⟩≡ (335e)

```
if(up->procctl == Proc_exitme && up->closingfgrp)
    forceclosefgrp();
```

⟨*function* forceclosefgrp 226c⟩≡ (597b)

```
/*
 * Called from sleep because up is in the middle
 * of closefgrp and just got a kill ctl message.
 * This usually means that up has wedged because
 * of some kind of deadly embrace with mntclose
 * trying to talk to itself.  To break free, hand the
 * unclosed channels to the close queue.  Once they
 * are finished, the blocked cclose that we've
 * interrupted will finish by itself.
 */
void
forceclosefgrp(void)
{
    int i;
    Chan *c;
    Fgrp *f;

    if(up->procctl != Proc_exitme || up->closingfgrp == nil){
        print("bad forceclosefgrp call");
        return;
    }

    f = up->closingfgrp;
    for(i = 0; i <= f->maxfd; i++)
        if(c = f->fd[i]){
            f->fd[i] = nil;
            ccloseq(c);
        }
}
```

⟨*struct* Clunkq 227a⟩≡ (571)

```
/*
 * Queue a chan to be closed by one of the clunk procs.
 */
struct Clunkq {
    Chan *head;
    Chan *tail;
    int nqueued;
    int nclosed;
    Lock l;
    QLock q;
    Rendez r;
};
```

⟨*global* clunkq 227b⟩≡ (571)

```
struct Clunkq clunkq;
```

⟨*function* ccloseq 227c⟩≡ (571)

```
void
ccloseq(Chan *c)
{
    if(c->flag&CFREE)
        panic("cclose %#p", getcallerpc(&c));

    DBG("ccloseq %p name=%s ref=%ld\n", c, c->path->s, c->ref);

    if(decref(c))
        return;

    lock(&clunkq.l);
    clunkq.nqueued++;
    c->next = nil;
    if(clunkq.head)
        clunkq.tail->next = c;
    else
        clunkq.head = c;
    clunkq.tail = c;
    unlock(&clunkq.l);

    if(!wakeup(&clunkq.r))
        kproc("kcloseproc", closeproc, nil);
}
```

⟨*function* clunkwork 227d⟩≡ (571)

```
static int
clunkwork(void*)
{
    return clunkq.head != nil;
}
```

⟨*function* closeproc 227e⟩≡ (571)

```
void
closeproc(void*)
{
    Chan *c;

    for(;;){
        qlock(&clunkq.q);
        if(clunkq.head == nil){
            if(!waserror()){
```

```
                tsleep(&clunkq.r, clunkwork, nil, 5000);
                poperror();
            }
            if(clunkq.head == nil){
                qunlock(&clunkq.q);
                pexit("no work", /*freemem*/true);
            }
        }
        lock(&clunkq.l);
        c = clunkq.head;
        clunkq.head = c->next;
        clunkq.nclosed++;
        unlock(&clunkq.l);
        qunlock(&clunkq.q);
        if(!waserror()){
            devtab[c->type]->close(c);
            poperror();
        }
        chanfree(c);
    }
}
```

12.10 Special opening modes

12.10.1 Close on exec

⟨OpenFlags *cases* 228a⟩+≡ (212c) ◁215a 228g▷

```
OCEXEC = 32,  /* or'ed in, close on exec */
```

⟨ChannelFlag *cases* 228b⟩≡ (206b) 228h▷

```
CCEXEC  = 0x0008,   /* close on exec */
```

⟨namec() *set channel flag after open* 228c⟩≡ (215f) 229a▷

```
if(omode & OCEXEC)
    c->flag |= CCEXEC;
```

⟨namec() *set channel flag after create* 228d⟩≡ (238e) 229b▷

```
if(omode & OCEXEC)
    cnew->flag |= CCEXEC;
```

⟨sysexec() *locals* 228e⟩+≡ (83i) ◁126c 384b▷

```
Fgrp *f;
```

⟨sysexec() *close files marked as opened with close on exec* 228f⟩≡ (83i)

```
/*
 * Close on exec
 */
f = up->fgrp;
for(i=0; i<=f->maxfd; i++)
    fdclose(i, CCEXEC);
```

12.10.2 Remove on close

⟨OpenFlags *cases* 228g⟩+≡ (212c) ◁228a 240a▷

```
ORCLOSE = 64,  /* or'ed in, remove on close */
```

⟨ChannelFlag *cases* 228h⟩+≡ (206b) ◁228b 449a▷

```
CRCLOSE = 0x0020,   /* remove on close */
```

⟨`namec()` *set channel flag after open* 229a⟩+≡ (215f) ◁228c

```
if(omode & ORCLOSE)
    c->flag |= CRCLOSE;
```

⟨`namec()` *set channel flag after create* 229b⟩+≡ (238e) ◁228d

```
if(omode & ORCLOSE)
    cnew->flag |= CRCLOSE;
```

Chapter 13

Directories

⟨systab *directory syscalls* 230a⟩≡ (26c)

```
[CREATE]    syscreate,
[REMOVE]    sysremove,

[CHDIR]     syschdir,
[FD2PATH]   sysfd2path, // =~ pwd

[STAT]      sysstat,
[FSTAT]     sysfstat,
[WSTAT]     syswstat,
[FWSTAT]    sysfwstat,
```

13.1 Overview

⟨*function* error_if_not_dir 230b⟩≡ (571)

```
void
error_if_not_dir(Chan *c)
{
    if(c->qid.type & QTDIR)
        return;
    error(Enotdir);
}
```

⟨sseek() *ensures off is 0 for directories* 230c⟩≡ (225)

```
if((c->qid.type & QTDIR) && off != 0)
    error(Eisdir);
```

⟨sseek() *disallows seek type 1 or 2 for directories* 230d⟩≡ (225)

```
if(c->qid.type & QTDIR)
    error(Eisdir);
```

⟨namec() *error if not a directory or cannot exec directory* 230e⟩≡ (215f)

```
if(e.mustbedir && !(c->qid.type & QTDIR))
    error(Enotdir);

if((amode == Aopen) && (omode & OEXEC) && (c->qid.type & QTDIR))
    error("cannot exec directory");
```

13.2 Path

⟨*struct* Path 231a⟩≡ (569c)

```
struct Path
{
    char  *s;
    int len;      /* strlen(s) */
    int alen;     /* allocated length of s */

    ⟨Path mtpt fields 258a⟩

    // extra
    Ref;
};
```

⟨*constructor* newpath 231b⟩≡ (571)

```
Path*
newpath(char *s)
{
    int i;
    Path *p;

    p = smalloc(sizeof(Path));
    i = strlen(s);
    p->len = i;
    p->alen = i+PATHSLOP;
    p->s = smalloc(p->alen);
    memmove(p->s, s, i+1);
    p->ref = 1;
    incref(&npath);

    ⟨newpath() mtpt handling 258c⟩
    return p;
}
```

⟨*constant* PATHSLOP 231c⟩≡ (571)

```
PATHSLOP    = 20,
```

⟨*function* copypath 231d⟩≡ (571)

```
static Path*
copypath(Path *p)
{
    int i;
    Path *pp;

    pp = smalloc(sizeof(Path));
    pp->ref = 1;
    incref(&npath);
    DBG("copypath %s %p => %p\n", p->s, p, pp);

    pp->len = p->len;
    pp->alen = p->alen;
    pp->s = smalloc(p->alen);
    memmove(pp->s, p->s, p->len+1);

    ⟨copypath() mtpt handling 258d⟩
    return pp;
}
```

⟨*destructor* pathclose 232a⟩≡ (571)

```
void
pathclose(Path *p)
{
    int i;

    if(p == nil)
        return;
    ⟨pathclose() debugging ??⟩
    if(decref(p))
        return;

    decref(&npath);
    free(p->s);

    ⟨pathclose() mtpt handling 258e⟩
    free(p);
}
```

⟨*function* uniquepath 232b⟩≡ (571)

```
static Path*
uniquepath(Path *p)
{
    Path *new;

    if(p->ref > 1){
        /* copy on write */
        new = copypath(p);
        pathclose(p);
        p = new;
    }
    return p;
}
```

⟨*function* addelem 232c⟩≡ (571)

```
static Path*
addelem(Path *p, char *s, Chan *from)
{
    char *t;
    int a, i;
    Chan *c, **tt;

    if(s[0]=='.' && s[1]=='\0')
        return p;

    p = uniquepath(p);

    i = strlen(s);
    // +1 for the '\0'
    if(p->len+1+i+1 > p->alen){
        a = p->len+1+i+1 + PATHSLOP;
        t = smalloc(a);
        memmove(t, p->s, p->len+1);
        free(p->s);
        p->s = t;
        p->alen = a;
    }
    /* don't insert extra slash if one is present */
    if(p->len>0 && p->s[p->len-1]!='/' && s[0]!='/')
        p->s[p->len++] = '/';
```

```
    memmove(p->s+p->len, s, i+1);
    p->len += i;

    if(isdotdot(s)){
        fixdotdotname(p);
    }
    ⟨addelem() mtpt handling 258f⟩

    return p;
}
```

13.3 . .

⟨*function* isdotdot 233a⟩≡ (571)

```
int
isdotdot(char *p)
{
    return p[0]=='.' && p[1]=='.' && p[2]=='\0';
}
```

⟨*function* fixdotdotname 233b⟩≡ (571)

```
/*
 * In place, rewrite name to compress multiple /, eliminate ., and process ..
 * (Really only called to remove a trailing .. that has been added.
 * Otherwise would need to update n->mtpt as well.)
 */
static void
fixdotdotname(Path *p)
{
    char *r;

    if(p->s[0] == '#'){
        r = strchr(p->s, '/');
        if(r == nil)
            return;
        cleanname(r);

        /*
         * The correct name is #i rather than #i/,
         * but the correct name of #/ is #/.
         */
        if(strcmp(r, "/")==0 && p->s[1] != '/')
            *r = '\0';
    }else
        cleanname(p->s);
    p->len = strlen(p->s);
}
```

13.4 walk()

⟨Dev *methods* 233c⟩+≡ (32d) ◁223b 238d▷

```
Walkqid*(*walk)(Chan*, Chan*, char**, int);
```

⟨*struct* Walkqid 234a⟩≡ (569c)

```
struct Walkqid
{
  Chan  *clone;
  int nqid;
  // variable array length, size = nqid
  Qid qid[1];
};
```

⟨walk() *locals* 234b⟩≡ (234h) 234c ▷

```
Chan *c;
Path *path;
Walkqid *wq;
Chan *nc;
```

⟨walk() *locals* 234c⟩+≡ (234h) ◁ 234b 234d ▷

```
Mhead *mh;
Mhead *nmh;
int type, dev;
```

⟨walk() *locals* 234d⟩+≡ (234h) ◁ 234c 234e ▷

```
Chan *mtpt;
bool didmount;
```

⟨walk() *locals* 234e⟩+≡ (234h) ◁ 234d 234f ▷

```
bool dotdot;
```

⟨walk() *locals* 234f⟩+≡ (234h) ◁ 234e **??** ▷

```
int nhave, ntry;
```

⟨namec() *locals* 234g⟩+≡ (215f) ◁ 221a 238c ▷

```
bool sharppath;
```

⟨*function* walk 234h⟩≡ (571)

```
/*
 * Either walks all the way or not at all.  No partial results in *cp.
 * *nerror is the number of names to display in an error message.
 */
int
walk(Chan **cp, char **names, int nnames, bool sharppath, int *nerror)
{
    ⟨walk() locals 234b⟩

    c = *cp;
    incref(c);
    path = c->path;
    incref(path);

    mh = nil;

    /*
     * While we haven't gotten all the way down the path:
     *    1. step through a mount point, if any
     *    2. send a walk request for initial dotdot or initial prefix without dotdot
     *    3. move to the first mountpoint along the way.
     *    4. repeat.
     *
     * Each time through the loop:
     *
     *  If didmount==false, c is on the undomount side of the mount point.
```

```
 *  If didmount==true, c is on the domount side of the mount point.
 *  Either way, c's full path is path.
 */
didmount = false;
for(nhave=0; nhave<nnames; nhave+=n){

    ⟨walk() return error if channel c is not a dir 237a⟩

    ⟨walk() set dotdot and ntry to index of next '..' or end 236b⟩

    if(!sharppath && !dotdot && !didmount)
        domount(&c, &mh, &path);

    type = c->type;
    dev = c->dev;

    if((wq = ewalk(c, names+nhave, ntry)) == nil){
        // didn't find the element, look in union mount
        ⟨walk() if c is a union mount, find and adjust type and dev 259c⟩
        ⟨walk() if wq is still nil, close and return 237b⟩
    }

    didmount = false;
    if(dotdot){
        assert(wq->nqid == 1);
        assert(wq->clone != nil);
        path = addelem(path, "..", nil);
        nc = undomount(wq->clone, path);
        nmh = nil;
        n = 1;
    }else{
        nc = nil;
        nmh = nil;
        if(!sharppath){
            // mount points along path?
            for(i=0; i<wq->nqid && i<ntry-1; i++){
                if(findmount(&nc, &nmh, type, dev, wq->qid[i])){
                    didmount = true;
                    break;
                }
            }
        }

        if(!didmount){  /* no mount points along path */
            ⟨walk() if nc == nil and wq-¿clone == nil return error 237c⟩
            n = wq->nqid;
            nc = wq->clone;
        }else{      /* stopped early, at a mount point */
            if(wq->clone != nil){
                cclose(wq->clone);
                wq->clone = nil;
            }
            n = i+1;
        }

        for(i=0; i<n; i++){
            mtpt = nil;
            if(i==n-1 && nmh)
                mtpt = nmh->from;
            path = addelem(path, names[nhave+i], mtpt);
```

```
            }
        }
        cclose(c);
        c = nc;
        putmhead(mh);
        mh = nmh;
        free(wq);
    }

    putmhead(mh);

    c = uniquechan(c); // late? why now? we've modified in place c already

    ⟨walk() print error if c-¿umh != nil 260d⟩

    pathclose(c->path);
    c->path = path;

    cclose(*cp);
    *cp = c;
    if(nerror)
        *nerror = nhave;
    return 0;
}
```

⟨*function* ewalk 236a⟩≡ (571)

```
/*
 * Call dev walk but catch errors.
 */
static Walkqid*
ewalk(Chan *c, char **names, int nnames)
{
    Walkqid *wq;

    if(waserror())
        return nil;
    wq = devtab[c->type]->walk(c, nil, names, nnames);
    poperror();
    return wq;
}
```

⟨walk() *set dotdot and ntry to index of next '..' or end* 236b⟩≡ (234h)

```
dotdot = false;
// remaining names
ntry = nnames - nhave;
if(ntry > MAXWELEM)
    ntry = MAXWELEM;

for(i=0; i<ntry; i++){
    if(isdotdot(names[nhave+i])){
        if(i==0){
            dotdot = true;
            ntry = 1;
        }else
            ntry = i;
        break;
    }
}
// ntry is now the index of the next '..' or the remaining names to walk
```

⟨walk() *return error if channel c is not a dir* 237a⟩≡ (234h)

```
if(!(c->qid.type & QTDIR)){
    pathclose(path);
    cclose(c);
    if(nerror)
        *nerror = nhave;
    if(mh != nil)
        putmhead(mh);
    strcpy(up->errstr, Enotdir);
    return -1;
}
```

⟨walk() *if wq is still nil, close and return* 237b⟩≡ (234h)

```
if(wq == nil){
    pathclose(path);
    cclose(c);
    if(nerror)
        *nerror = nhave+1;
    if(mh != nil)
        putmhead(mh);
    return -1;
}
```

⟨walk() *if nc == nil and wq-¿clone == nil return error* 237c⟩≡ (234h)

```
if(wq->clone == nil){
    pathclose(path);
    cclose(c);
    if(wq->nqid==0 || (wq->qid[wq->nqid-1].type&QTDIR)){
        if(nerror)
            *nerror = nhave+wq->nqid+1;
        strcpy(up->errstr, Edoesnotexist);
    }else{
        if(nerror)
            *nerror = nhave+wq->nqid;
        strcpy(up->errstr, Enotdir);
    }
    free(wq);
    if(mh != nil)
        putmhead(mh);
    return -1;
}
```

⟨*function* cclone 237d⟩≡ (571)

```
Chan*
cclone(Chan *c)
{
    Chan *nc;
    Walkqid *wq;

    wq = devtab[c->type]->walk(c, nil, nil, 0);

    if(wq == nil)
        error("clone failed");
    nc = wq->clone;
    free(wq);
    nc->path = c->path;
    if(c->path)
        incref(c->path);
    return nc;
}
```

13.5 syscreate()

⟨*syscall create* 238a⟩≡ (578)

```
// int create(char *file, int omode, ulong perm);
long
syscreate(ulong* arg)
{
    int fd;
    Chan *c;

    openmode(arg[1]&~OEXCL);    /* error check only; OEXCL okay here */
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Acreate, arg[1], arg[2]);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    fd = newfd(c);
    if(fd < 0)
        error(Enofd);
    poperror();
    return fd;
}
```

⟨namec() *adjust Elemlist e if Acreate* 238b⟩≡ (215f)

```
if(amode == Acreate){
    /* perm must have DMDIR if last element is / or /. */
    if(e.mustbedir && !(perm&DMDIR)){
        e.nerror = e.nelems;
        error("create without DMDIR");
    }

    /* don't try to walk the last path element just yet. */
    if(e.nelems == 0)
        error(Eexist);
    e.nelems--;
}
```

⟨namec() *locals* 238c⟩+≡ (215f) ◁234g 250f▷

```
Chan *cnew;
char *createerr;
```

⟨Dev *methods* 238d⟩+≡ (32d) ◁233c 240c▷

```
void  (*create)(Chan*, char*, int, ulong);
```

⟨namec() *other cases* 238e⟩≡ (215f) 241b▷

```
case Acreate:
    /*
     * We've already walked all but the last element.
     * If the last exists, try to open it OTRUNC.
     * If omode&OEXCL is set, just give up.
     */
    e.nelems++;
    e.nerror++;
    if(walk(&c, e.elems+e.nelems-1, 1, sharppath, nil) == 0){
        if(omode&OEXCL)
            error(Eexist);
        omode |= OTRUNC;
        goto Open;
    }
```

```
/*
 * The semantics of the create(2) system call are that if the
 * file exists and can be written, it is to be opened with truncation.
 * On the other hand, the create(5) message fails if the file exists.
 * If we get two create(2) calls happening simultaneously,
 * they might both get here and send create(5) messages, but only
 * one of the messages will succeed.  To provide the expected create(2)
 * semantics, the call with the failed message needs to try the above
 * walk again, opening for truncation.  This correctly solves the
 * create/create race, in the sense that any observable outcome can
 * be explained as one happening before the other.
 * The create/create race is quite common.  For example, it happens
 * when two rc subshells simultaneously update the same
 * environment variable.
 *
 * The implementation still admits a create/create/remove race:
 * (A) walk to file, fails
 * (B) walk to file, fails
 * (A) create file, succeeds, returns
 * (B) create file, fails
 * (A) remove file, succeeds, returns
 * (B) walk to file, return failure.
 *
 * This is hardly as common as the create/create race, and is really
 * not too much worse than what might happen if (B) got a hold of a
 * file descriptor and then the file was removed -- either way (B) can't do
 * anything with the result of the create call.  So we don't care about this race.
 *
 * Applications that care about more fine-grained decision of the races
 * can use the OEXCL flag to get at the underlying create(5) semantics;
 * by default we provide the common case.
 *
 * We need to stay behind the mount point in case we
 * need to do the first walk again (should the create fail).
 *
 * We also need to cross the mount point and find the directory
 * in the union in which we should be creating.
 *
 * The channel staying behind is c, the one moving forward is cnew.
 */
m = nil;
cnew = nil; /* is this assignment necessary? */
if(!waserror()){    /* try create */
    ⟨namec() Acreate case, if cnew is a mountpoint 262b⟩
    else{
        cnew = c;
        incref(cnew);
    }

    /*
     * We need our own copy of the Chan because we're
     * about to send a create, which will move it.  Once we have
     * our own copy, we can fix the name, which might be wrong
     * if findmount gave us a new Chan.
     */
    cnew = uniquechan(cnew);
    pathclose(cnew->path);
    cnew->path = c->path;
    incref(cnew->path);
```

```
        devtab[cnew->type]->create(cnew, e.elems[e.nelems-1], omode&~(OEXCL|OCEXEC), perm);
        poperror();
        ⟨namec() set channel flag after create 228d⟩
        if(m)
            putmhead(m);
        cclose(c);
        c = cnew;
        c->path = addelem(c->path, e.elems[e.nelems-1], nil);
        break; // switch
    }

    /* create failed */
    cclose(cnew);
    if(m)
        putmhead(m);
    if(omode & OEXCL)
        nexterror();

    /* save error */
    createerr = up->errstr;
    up->errstr = tmperrbuf;
    /* note: we depend that walk does not error */
    if(walk(&c, e.elems+e.nelems-1, 1, sharppath, nil) < 0){
        up->errstr = createerr;
        error(createerr);   /* report true error */
    }
    up->errstr = createerr;
    omode |= OTRUNC;
    goto Open;
```

⟨OpenFlags *cases* 240a⟩+≡ (212c) ◁228g 240b▷

```
  OTRUNC = 16,  /* or'ed in (except for exec), truncate file first */
```

⟨OpenFlags *cases* 240b⟩+≡ (212c) ◁240a

```
  OEXCL = 0x1000,  /* or'ed in, exclusive create */
```

13.6 sysremove()

⟨Dev *methods* 240c⟩+≡ (32d) ◁238d 243b▷

```
  void  (*remove)(Chan*);
```

⟨*syscall remove* 240d⟩≡ (578)

```
  // int remove(char *file);
  long
  sysremove(ulong* arg)
  {
      Chan *c;

      validaddr(arg[0], 1, false);
      c = namec((char*)arg[0], Aremove, 0, 0);
      /*
       * Removing mount points is disallowed to avoid surprises
       * (which should be removed: the mount point or the mounted Chan?).
       */
      if(c->ismtpt){
          cclose(c);
          error(Eismtpt);
```

```
    }
    if(waserror()){
        c->type = 0;    /* see below */
        cclose(c);
        nexterror();
    }
    devtab[c->type]->remove(c);
    /*
     * Remove clunks the fid, but we need to recover the Chan
     * so fake it up.  rootclose() is known to be a nop.
     */
    c->type = 0;
    poperror();
    cclose(c);
    return 0;
}
```

13.7 syschdir()

⟨*syscall chdir* 241a⟩≡ (578)

```
// int chdir(char *dirname);
long
syschdir(ulong* arg)
{
    Chan *c;

    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Atodir, 0, 0);
    cclose(up->dot);
    up->dot = c;
    return 0;
}
```

⟨namec() *other cases* 241b⟩+≡ (215f) ◁238e 252b▷

```
case Atodir:
    /*
     * Directories (e.g. for cd) are left before the mount point,
     * so one may mount on / or . and see the effect.
     */
    if(!(c->qid.type & QTDIR))
        error(Enotdir);
    break;
```

13.8 sysfd2path() (a.k.a pwd())

⟨*syscall fd2path* 241c⟩≡ (578)

```
// int fd2path(int fd, char *buf, int nbuf);
long
sysfd2path(ulong* arg)
{
    Chan *c;

    validaddr(arg[1], arg[2], true);
    c = fdtochan(arg[0], -1, false, true);
    snprint((char*)arg[1], arg[2], "%s",
              chanpath(c));
```

```
    cclose(c);
    return 0;
}
```

⟨*function* chanpath 242a⟩≡ (571)

```
char*
chanpath(Chan *c)
{
    if(c == nil)
        return "<nil chan>";
    if(c->path == nil)
        return "<nil path>";
    if(c->path->s == nil)
        return "<nil path.s>";
    return c->path->s;
}
```

13.9 DirEntry

⟨*struct* DirEntry 242b⟩≡ (580e)

```
struct DirEntry {
  /* system-modified data */
  ushort  type; /* server type */
  uint  dev;  /* server subtype */

  /* file data */
  Qid qid;  /* unique id from server */
  // bitset<enum<dirmode>>
  ulong mode; /* permissions */

  ulong atime;  /* last read time */
  ulong mtime;  /* last write time */

  vlong length; /* file length: see <u.h> */
  char  *name;  /* last element of path */

  char  *uid; /* owner name */
  char  *gid; /* group name */
  char  *muid;  /* last modifier name */
};
```

⟨*enum* dirmode 242c⟩≡ (580e)

```
/* bits in DirEntry.mode */
enum Dirmode {
    DMDIR = 0x80000000,  /* mode bit for directories */

    DMREAD = 0x4,   /* mode bit for read permission */
    DMWRITE = 0x2,   /* mode bit for write permission */
    DMEXEC = 0x1,   /* mode bit for execute permission */
    ⟨Dirmode cases 321a⟩
};
```

⟨*function* direntry_setname 242d⟩≡ (578)

```
static long
direntry_setname(char *name, int len, byte *p, long n, long maxn)
{
    char *oname;
    int olen;
```

```
    long nn;

    if(n == BIT16SZ)
        return BIT16SZ;

    oname = offsetof_name_direntry(p, &olen);

    nn = n+len-olen;
    PBIT16(p, nn-BIT16SZ);
    if(nn > maxn)
        return BIT16SZ;

    if(len != olen)
        memmove(oname+len, oname+olen, p+n-(byte*)(oname+olen));
    PBIT16((byte*)(oname-2), len);
    memmove(oname, name, len);
    return nn;
}
```

⟨*function* offsetof_name_direntry 243a⟩≡ (578)

```
static char*
offsetof_name_direntry(byte *p, int *n)
{
    p += BIT16SZ+BIT16SZ+BIT32SZ +
          BIT8SZ+BIT32SZ+BIT64SZ +
          BIT32SZ+BIT32SZ+BIT32SZ +
          BIT64SZ;
    *n = GBIT16(p);
    return (char*)p+BIT16SZ;
}
```

13.10 sysstat()

⟨Dev *methods* 243b⟩+≡ (32d) ◁240c 244c▷

```
int (*stat)(Chan*, byte*, int);
```

⟨*syscall stat* 243c⟩≡ (578)

```
// int stat(char *name, byte *edir, int nedir);
long
sysstat(ulong* arg)
{
    char *name;
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, true);
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aaccess, 0, 0);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    l = devtab[c->type]->stat(c, (byte*)arg[1], l);

    name = pathlast(c->path);
    if(name)
        l = direntry_setname(name, strlen(name), (byte*)arg[1], l, arg[2]);
```

```
    poperror();
    cclose(c);
    return l;
}
```

⟨*syscall fstat* 244a⟩≡ (578)

```
// int fstat(int fd, byte *edir, int nedir);
long
sysfstat(ulong* arg)
{
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, true);
    c = fdtochan(arg[0], -1, false, true);
    if(waserror()) {
        cclose(c);
        nexterror();
    }
    l = devtab[c->type]->stat(c, (byte*)arg[1], l);
    poperror();
    cclose(c);
    return l;
}
```

⟨*function* pathlast 244b⟩≡ (578)

```
static char*
pathlast(Path *p)
{
    char *s;

    if(p == nil)
        return nil;
    if(p->len == 0)
        return nil;
    s = strrchr(p->s, '/');
    if(s)
        return s+1;
    return p->s;
}
```

⟨Dev *methods* 244c⟩+≡ (32d) ◁243b 251b▷

```
int (*wstat)(Chan*, byte*, int);
```

⟨*syscall wstat* 244d⟩≡ (578)

```
// int wstat(char *name, byte *edir, int nedir);
long
syswstat(ulong* arg)
{
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, false);
    validstat((byte*)arg[1], l);
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aaccess, 0, 0);
    return wstat(c, (byte*)arg[1], l);
}
```

⟨*syscall fwstat* 245a⟩≡ (578)

```
// int fwstat(int fd, byte *edir, int nedir);
long
sysfwstat(ulong* arg)
{
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, false);
    validstat((byte*)arg[1], l);
    c = fdtochan(arg[0], -1, true, true);
    return wstat(c, (byte*)arg[1], l);
}
```

⟨*function* validstat 245b⟩≡ (575b)

```
void
validstat(byte *s, int n)
{
    int m;
    char buf[64];

    if(statcheck(s, n) < 0)
        error(Ebadstat);
    /* verify that name entry is acceptable */
    s += STATFIXLEN - 4*BIT16SZ;    /* location of first string */
    /*
     * s now points at count for first string.
     * if it's too long, let the server decide; this is
     * only for his protection anyway. otherwise
     * we'd have to allocate and waserror.
     */
    m = GBIT16(s);
    s += BIT16SZ;
    if(m+1 > sizeof buf)
        return;
    memmove(buf, s, m);
    buf[m] = '\0';
    /* name could be '/' */
    if(strcmp(buf, "/") != 0)
        validname(buf, false);
}
```

⟨*function* wstat 245c⟩≡ (578)

```
static long
wstat(Chan *c, byte *d, int nd)
{
    long l;
    int namelen;

    if(waserror()){
        cclose(c);
        nexterror();
    }
    if(c->ismtpt){
        /*
         * Renaming mount points is disallowed to avoid surprises
         * (which should be renamed? the mount point or the mounted Chan?).
         */
        offsetof_name_direntry(d, &namelen);
```

```
        if(namelen)
            nameerror(chanpath(c), Eismtpt);
    }
    l = devtab[c->type]->wstat(c, d, nd);
    poperror();
    cclose(c);
    return l;
}
```

Chapter 14

Namespaces

⟨systab *namespace syscalls* 247a⟩≡ (26c)

```
[BIND]      sysbind,
[MOUNT]     sysmount,
[UNMOUNT]   sysunmount,
```

14.1 Overview

⟨*enum* mount 247b⟩≡ (580e)

```
enum MountFlags {
    MREPL = 0x0000,  /* mount replaces object */

    MBEFORE = 0x0001,  /* mount goes before others in union directory */
    MAFTER = 0x0002,  /* mount goes after others in union directory */

    MCREATE = 0x0004,  /* permit creation in mounted directory */
    ⟨MountFlags cases 265b⟩

    MORDERMASK =  0x0003,  /* mask for bits defining order of mounting */
    MMASK = 0x0017,  /* all bits on */
};
```

14.2 Mount point

⟨*struct* Mhead 247c⟩≡ (569c)

```
struct Mhead
{
    // ref<Chan>
    Chan* from;     /* channel mounted upon */
    // list<ref<Mount>> list because of union mount (ordered via MBEFORE|MAFTER)
    Mount*  mount;      /* what's mounted upon it */

    Ref;
    RWlock  lock;
    ⟨Mhead extra fields 250a⟩
};
```

⟨*constructor* newmhead 247d⟩≡ (571)

```
Mhead*
newmhead(Chan *from)
{
    Mhead *mh;
```

```
    mh = smalloc(sizeof(Mhead));
    mh->ref = 1;
    mh->from = from;
    incref(from);
    return mh;
}
```

⟨*destructor* putmhead 248a⟩≡ (571)

```
/*
 * This is necessary because there are many
 * pointers to the top of a given mount list:
 *
 *  - the mhead in the namespace hash table
 *  - the mhead in chans returned from findmount:
 *    used in namec and then by unionread.
 *  - the mhead in chans returned from createdir:
 *    used in the open/create race protect, which is gone.
 *
 * The RWlock in the Mhead protects the mount list it contains.
 * The mount list is deleted when we cunmount.
 * The RWlock ensures that nothing is using the mount list at that time.
 *
 * It is okay to replace c->mh with whatever you want as
 * long as you are sure you have a unique reference to it.
 *
 * This comment might belong somewhere else.
 */
void
putmhead(Mhead *m)
{
    if(m && decref(m) == 0){
        m->mount = (Mount*)0xCafeBeef;
        free(m);
    }
}
```

⟨*struct* Mount 248b⟩≡ (569c)

```
struct Mount
{
    // ref<Chan>
    Chan* to;     /* channel replacing channel */

    // enum<mount>, but mainly for MCREATE, the MBEFORE|MAFTER is encoded via list
    int mflag;

    // option<string>
    char  *spec; // for mount(), nil for bind()
    ⟨Mount other fields 267a⟩

    ⟨Mount extra fields 249a⟩
};
```

⟨*constructor* newmount 248c⟩≡ (597b)

```
Mount*
newmount(Chan *to, int flag, char *spec)
{
    Mount *m;

    m = smalloc(sizeof(Mount));
```

```
    m->to = to;
    incref(to);
    m->mountid = incref(&mountid);
    m->mflag = flag;
    if(spec != nil)
        kstrdup(&m->spec, spec);

    return m;
}
```

⟨Mount *extra fields* 249a⟩≡ (248b) 267d ▷

```
Mount*  next;
```

⟨*destructor* mountfree 249b⟩≡ (597b)

```
void
mountfree(Mount *m)
{
    Mount *f;

    while(m) {
        f = m->next;
        cclose(m->to);
        m->mountid = 0;
        free(m->spec);
        free(m);
        m = f;
    }
}
```

14.3 Mount table

⟨*struct* Pgrp 249c⟩≡ (592c)

```
// Mount table, aka Namespace, aka process group
struct Pgrp
{
    // hash<qid.path, list<ref<Mhead> (next = Mhead.next)>
    Mhead *mnthash[MNTHASH];

    ⟨Pgrp other fields 250b⟩

    // extra
    Ref;        /* also used as a lock when mounting */

    // ORDER OF LOCK: first debug and then ns
    QLock debug;      /* single access via devproc.c */
    RWlock  ns;     /* Namespace n read/one write lock */
};
```

⟨Proc *files fields* 249d⟩+≡ (29c) ◁ 226a

```
// ref_counted<pgrp>
Pgrp  *pgrp;    /* Process group for namespace */
```

⟨*function* MOUNTH 249e⟩≡ (592c)

```
enum
{
    MNTLOG  = 5,
    MNTHASH = 1<<MNTLOG,  /* Hash to walk mount table */
};
#define MOUNTH(p,qid) ((p)->mnthash[(qid).path&((1<<MNTLOG)-1)])
```

⟨Mhead *extra fields* 250a⟩≡ (247c)

```
// hash<qid.path, list<ref<Mhead>> of Pgrp.mnthash
Mhead*  hash;     /* Hash chain */
```

⟨Pgrp *other fields* 250b⟩≡ (249c) 263c ▷

```
ulong pgrpid;
```

⟨*global* pgrpid 250c⟩≡ (597b)

```
static Counter pgrpid;
```

⟨*constructor* newpgrp 250d⟩≡ (597b)

```
Pgrp*
newpgrp(void)
{
    Pgrp *p;

    p = smalloc(sizeof(Pgrp));
    p->ref = 1;
    p->pgrpid = incref(&pgrpid);
    return p;
}
```

⟨*destructor* closepgrp 250e⟩≡ (597b)

```
void
closepgrp(Pgrp *p)
{
    Mhead **h, **e, *f, *next;

    if(decref(p) != 0)
        return;

    qlock(&p->debug);
    wlock(&p->ns);
    p->pgrpid = -1;

    e = &p->mnthash[MNTHASH];
    for(h = p->mnthash; h < e; h++) {
        for(f = *h; f; f = next) {
            wlock(&f->lock);
            cclose(f->from);
            mountfree(f->mount);
            f->mount = nil;
            next = f->hash;
            wunlock(&f->lock);
            putmhead(f);
        }
    }
    wunlock(&p->ns);
    qunlock(&p->debug);
    free(p);
}
```

14.4 Device path

⟨namec() *locals* 250f⟩+≡ (215f) ◁238c 256a ▷

```
Rune r;
```

⟨namec() *if name[0] is a sharp* 251a⟩≡ (215f)

```
case '#':
    up->genbuf[0] = '\0';
    n = 0;
    while(*name != '\0' && (*name != '/' || n < 2)){ // "#/" is ok hence n < 2
        if(n >= sizeof(up->genbuf)-1)
            error(Efilename);
        up->genbuf[n++] = *name++;
    }
    up->genbuf[n] = '\0';

    n = chartorune(&r, up->genbuf+1)+1; // +1 for the leading '#'

    ⟨namec() noattach checking 263e⟩

    t = devno(r, true);
    if(t == -1)
        error(Ebadsharp);

    c = devtab[t]->attach(up->genbuf+n);
    break;
```

⟨Dev *methods* 251b⟩+≡ (32d) ◁244c 323a▷

```
Chan* (*attach)(char*);
```

14.5 sysbind()

⟨*syscall bind* 251c⟩≡ (578)

```
// int bind(char *to, char *from, int flag);
long
sysbind(ulong* arg)
{
    return bindmount(false, -1, -1, (char*)arg[0], (char*)arg[1], arg[2], nil);
}
```

⟨bindmount() *locals* 251d⟩≡ (251e) ??▷

```
Chan *c0;
Chan *c1;
```

⟨*function* bindmount 251e⟩≡ (578)

```
long
bindmount(bool ismount, int fd, int afd, char* arg0, char* arg1, ulong flag, char* spec)
{
    ⟨bindmount() locals 251d⟩

    if((flag&~MMASK) || (flag&MORDERMASK)==(MBEFORE|MAFTER))
        error(Ebadarg);

    ⟨bindmount() if ismount 264g⟩
    else{
        spec = nil;
        validaddr((ulong)arg0, 1, false);
        c0 = namec(arg0, Abind, 0, 0);
    }

    if(waserror()){
        cclose(c0);
```

```
        nexterror();
    }

    validaddr((ulong)arg1, 1, false);
    c1 = namec(arg1, Amount, 0, 0);

    if(waserror()){
        cclose(c1);
        nexterror();
    }

    ret = cmount(c0, c1, flag, spec);

    poperror();
    cclose(c1);
    poperror();
    cclose(c0);
    ⟨bindmount() if ismount free 265a⟩
    return ret;
}
```

⟨AccessFlags *cases* 252a⟩≡ (215b)

```
Abind,        /* for left-hand-side of bind */
Amount,       /* to be mounted or mounted upon */
```

⟨namec() *other cases* 252b⟩+≡ (215f) ◁241b 252c▷

```
case Abind:
    m = nil;
    if(!sharppath)
        /* no need to maintain path - cannot dotdot an Abind */ // so pass nil
        domount(&c, &m, nil);
    ⟨namec() set c-¿umh in Abind if mounted point 262d⟩
    break;
```

⟨namec() *other cases* 252c⟩+≡ (215f) ◁252b

```
case Amount:
    /*
     * When mounting on an already mounted upon directory,
     * one wants subsequent mounts to be attached to the
     * original directory, not the replacement.  Don't domount.
     */
    break;
```

⟨cmount() *locals* 252d⟩≡ (252e) 262e▷

```
Pgrp *pg;
Mhead *mh;
Mount *m;
int order;
```

⟨*function* cmount 252e⟩≡ (571)

```
int
cmount(Chan *new, Chan *old, int flag, char *spec)
{
    ⟨cmount() locals 252d⟩

    if((old->qid.type ^ new->qid.type) & QTDIR)
        error(Emount);
    ⟨cmount() print error if old-¿umh != nil 260e⟩

    order = flag&MORDERMASK;
```

```
if(!(old->qid.type & QTDIR) && order != MREPL)
    error(Emount);
```

⟨cmount() *if new is itself a mount point, error if cant create there* 263a⟩

```
pg = up->pgrp;
wlock(&pg->ns);

// mh = lookup(old->gdid, pg.mnthash)
l = &MOUNTH(pg, old->qid);
for(mh = *l; mh; mh = mh->hash){
    if(eqchan(mh->from, old, true))
        break;
    l = &mh->hash;
}

if(mh == nil){
    /*
     *  nothing mounted here yet.  create a mount
     *  head and add to the hash table.
     */
    mh = newmhead(old);
    *l = mh;

    /*
     *  if this is a union mount, add the old
     *  node to the mount chain.
     */
    if(order != MREPL)
        mh->mount = newmount(old, 0, nil);
}

wlock(&mh->lock);
if(waserror()){
    wunlock(&mh->lock);
    nexterror();
}
wunlock(&pg->ns);

m = newmount(new, flag, spec);
```

⟨cmount() *if new is itself a mount point, copy mounts* 263b⟩

```
if(mh->mount && order == MREPL){
    mountfree(mh->mount);
    mh->mount = nil;
}
if(flag & MCREATE)
    m->mflag |= MCREATE;

if(mh->mount && order == MAFTER){
    for(f = mh->mount; f->next; f = f->next)
        ;
    f->next = m;
}else{ // MBEFORE or MREPL
    // when new was itself a mount point, can have union of mount again
    for(f = m; f->next; f = f->next)
        ;
    f->next = mh->mount; // =~ m->next = mh->mount for the most common case
    mh->mount = m;
```

```
    }

    wunlock(&mh->lock);
    poperror();
    return m->mountid;
}
```

14.6 /proc/#/ns

⟨*struct* Mntwalk 254a⟩≡ (567)

```
struct Mntwalk              /* state for /proc/#/ns */
{
    bool cddone;
    Mhead*  mh;
    Mount*  cm;
};
```

⟨procread() *locals* 254b⟩+≡ (468c) ◁69a 468b▷

```
Mntwalk *mw;
char flag[10];
```

⟨procopen() *cases* 254c⟩≡ (473b)

```
case Qns:
    if(omode != OREAD)
        error(Eperm);
    c->aux = malloc(sizeof(Mntwalk));
    break;
```

⟨procclose() *hooks* 254d⟩≡ (479a)

```
if(QID(c->qid) == Qns && c->aux != nil)
    free(c->aux);
```

⟨procread() *cases* 254e⟩+≡ (468c) ◁69b 366i▷

```
case Qns:
    qlock(&p->debug);
    if(waserror()){
        qunlock(&p->debug);
        nexterror();
    }
    if(p->pgrp == nil || p->pid != PID(c->qid))
        error(Eprocdied);

    mw = c->aux;
    if(mw == nil)
        error(Enomem);
    if(mw->cddone){
        qunlock(&p->debug);
        poperror();
        return 0;
    }

    mntscan(mw, p);

    if(mw->mh == nil){
        mw->cddone = true;
        i = snprint(a, n, "cd %s\n", p->dot->path->s);
        qunlock(&p->debug);
        poperror();
```

```
        return i;
    }

    int2flag(mw->cm->mflag, flag);
    if(strcmp(mw->cm->to->path->s, "#M") == 0){
        srv = srvname(mw->cm->to->mchan);
        i = snprint(a, n, "mount %s %s %s %s\n", flag,
            srv==nil? mw->cm->to->mchan->path->s : srv,
            mw->mh->from->path->s, mw->cm->spec? mw->cm->spec : "");
        free(srv);
    }else
        i = snprint(a, n, "bind %s %s %s\n", flag,
            mw->cm->to->path->s, mw->mh->from->path->s);
    qunlock(&p->debug);
    poperror();
    return i;
```

⟨*function* int2flag 255a⟩≡ (567)

```
static void
int2flag(int flag, char *s)
{
    if(flag == 0){
        *s = '\0';
        return;
    }
    *s++ = '-';
    if(flag & MAFTER)
        *s++ = 'a';
    if(flag & MBEFORE)
        *s++ = 'b';
    if(flag & MCREATE)
        *s++ = 'c';
    if(flag & MCACHE)
        *s++ = 'C';
    *s = '\0';
}
```

⟨*function* mntscan 255b⟩≡ (567)

```
void
mntscan(Mntwalk *mw, Proc *p)
{
    Pgrp *pg;
    Mount *t;
    Mhead *f;
    int i;
    bool nxt;
    ulong last, bestmid;

    pg = p->pgrp;
    rlock(&pg->ns);

    nxt = false;
    bestmid = ~0;

    last = 0;
    if(mw->mh)
        last = mw->cm->mountid;

    for(i = 0; i < MNTHASH; i++) {
        for(f = pg->mnthash[i]; f; f = f->hash) {
```

```
            for(t = f->mount; t; t = t->next) {
                if(mw->mh == nil ||
                  (t->mountid > last && t->mountid < bestmid)) {
                    mw->cm = t;
                    mw->mh = f;
                    bestmid = mw->cm->mountid;
                    nxt = true;
                }
            }
        }
    }
    if(nxt == false)
        mw->mh = nil;

    runlock(&pg->ns);
}
```

14.7 findmount(), domount()

⟨namec() *locals* 256a⟩+≡ (215f) ◁250f

```
Mhead *m;
Path *path;
```

⟨namec() *case Aopen, Acreate, Aremove, Aaccess, handle mountpoint part1* 256b⟩≡ (215f)

```
/* save&update the name; domount might change c */
path = c->path;
incref(path);

m = nil;
if(!sharppath)
    domount(&c, &m, &path);

/* our own copy to open or remove */
c = uniquechan(c);

/* now it's our copy anyway, we can put the name back */
pathclose(c->path);
c->path = path;

/* record whether c is on a mount point */
c->ismtpt = (m!=nil);
```

⟨namec() *case Aremove, Aaccess, handle mountpoint part2* 256c⟩≡ (215f)

```
putmhead(m);
```

⟨*function* findmount 256d⟩≡ (571)

```
/* also used by sysfile.c:/^mountfix */
bool
findmount(Chan **cp, Mhead **mp, int type, int dev, Qid qid)
{
    Pgrp *pg;
    Mhead *m;

    pg = up->pgrp;
    rlock(&pg->ns);

    // *mp = lookup(gqid, pg.mnthash); *cp = mp->mount->to
    for(m = MOUNTH(pg, qid); m; m = m->hash){
```

```
        rlock(&m->lock);
        if(m->from == nil){
            print("m %p m->from 0\n", m);
            runlock(&m->lock);
            continue;
        }
        if(eqchantdqid(m->from, type, dev, qid, true)){
            runlock(&pg->ns);
            if(mp != nil){
                if(*mp != nil)
                    putmhead(*mp);
                *mp = m;
                incref(m);
            }
            if(*cp != nil)
                cclose(*cp);
            *cp = m->mount->to;
            incref(m->mount->to);
            runlock(&m->lock);
            return true;
        }
        runlock(&m->lock);
    }

    runlock(&pg->ns);
    return false;
}
```

⟨*function* domount 257⟩≡ (571)

```
/*
 * Calls findmount but also updates path.
 */
static void
domount(Chan **cp, Mhead **mp, Path **path)
{
    Chan **lc;
    Path *p;

    if(!findmount(cp, mp, (*cp)->type, (*cp)->dev, (*cp)->qid))
        return;

    if(path){
        p = *path;
        p = uniquepath(p);
        if(p->mlen <= 0)
            print("domount: path %s has mlen==%d\n", p->s, p->mlen);
        else{
            lc = &p->mtpt[p->mlen-1];
            DBG("domount %p %s => add %p (was %p)\n", p, p->s, (*mp)->from, p->mtpt[p->mlen-1]);
            if(*lc)
                cclose(*lc);
            *lc = (*mp)->from;
            incref((*mp)->from);
        }
        *path = p;
    }
    return;
}
```

14.8 Path mount points and ..

⟨Path *mtpt fields* 258a⟩≡ (231a)

```
// array<option<ref_counted<Chan>>, nil for elements which are not mount point
Chan  **mtpt;     /* mtpt history */
int mlen;     /* number of path elements */
int malen;      /* allocated length of mtpt */
```

⟨*constant* PATHMSLOP 258b⟩≡ (571)

```
PATHMSLOP   = 20,
```

⟨newpath() *mtpt handling* 258c⟩≡ (231b)

```
/*
 * Cannot use newpath for arbitrary names because the mtpt
 * array will not be populated correctly.  The names #/ and / are
 * allowed, but other names with / in them draw warnings.
 */
if(strchr(s, '/') && strcmp(s, "#/") != 0 && strcmp(s, "/") != 0)
    print("newpath: %s from %#p\n", s, getcallerpc(&s));

p->mlen = 1;
p->malen = PATHMSLOP;
p->mtpt = smalloc(p->malen*sizeof p->mtpt[0]); // smalloc set p->mtpt[0] = nil
```

⟨copypath() *mtpt handling* 258d⟩≡ (231d)

```
pp->mlen = p->mlen;
pp->malen = p->malen;
pp->mtpt = smalloc(p->malen*sizeof pp->mtpt[0]);
for(i=0; i<pp->mlen; i++){
    pp->mtpt[i] = p->mtpt[i];
    if(pp->mtpt[i])
        incref(pp->mtpt[i]);
}
```

⟨pathclose() *mtpt handling* 258e⟩≡ (232a)

```
for(i=0; i<p->mlen; i++)
    if(p->mtpt[i])
        cclose(p->mtpt[i]);
free(p->mtpt);
```

⟨addelem() *mtpt handling* 258f⟩≡ (232c)

```
if(isdotdot(s)){
    DBG("addelem %s .. => rm %p\n", p->s, p->mtpt[p->mlen-1]);
    if(p->mlen>1 && (c = p->mtpt[--p->mlen])){
        p->mtpt[p->mlen] = nil;
        cclose(c);
    }
}else{
    if(p->mlen >= p->malen){
        // realloc
        p->malen = p->mlen+1+PATHMSLOP;
        tt = smalloc(p->malen*sizeof tt[0]);
        memmove(tt, p->mtpt, p->mlen*sizeof tt[0]);
        free(p->mtpt);
        p->mtpt = tt;
    }
    DBG("addelem %s %s => add %p\n", p->s, s, from);
    p->mtpt[p->mlen++] = from;
    if(from)
        incref(from);
}
```

⟨*function* undomount 259a⟩≡ (571)

```
/*
 * If c is the right-hand-side of a mount point, returns the left hand side.
 * Changes name to reflect the fact that we've uncrossed the mountpoint,
 * so name had better be ours to change!
 */
static Chan*
undomount(Chan *c, Path *path)
{
    Chan *nc;

    if(path->ref != 1 || path->mlen == 0)
        print("undomount: path %s ref %ld mlen %d caller %#p\n",
            path->s, path->ref, path->mlen, getcallerpc(&c));

    if(path->mlen>0 && (nc=path->mtpt[path->mlen-1]) != nil){
        DBG("undomount %p %s => remove %p\n", path, path->s, nc);
        cclose(c);
        path->mtpt[path->mlen-1] = nil;
        // c = nc = mh->from
        c = nc;
    }
    return c;
}
```

14.9 Union mount

14.9.1 Union walk

⟨walk() *locals* 259b⟩+≡ (234h) ◁??

```
Mount *f;
```

⟨walk() *if c is a union mount, find and adjust type and dev* 259c⟩≡ (234h)

```
/* try a union mount, if any */
if(mh && !sharppath){
    /*
     * mh->mount->to == c, so start at mh->mount->next
     */
    rlock(&mh->lock);
    f = mh->mount;
    for(f = (f? f->next: f); f; f = f->next)
        if((wq = ewalk(f->to, names+nhave, ntry)) != nil)
            break;
    runlock(&mh->lock);
    if(f != nil){
        type = f->to->type;
        dev = f->to->dev;
    }
}
```

14.9.2 ->umh

⟨namec() *case Aopen, Acreate, handle mountpoint part2* 259d⟩≡ (215f)

```
/* only save the mount head if it's a multiple element union */
if(m && m->mount && m->mount->next)
    c->umh = m;
else
    putmhead(m);
```

⟨Chan *other fields* 260a⟩+≡ (31a) ◁206c 260f▷

```
Mhead*  umh;      /* mount point that derived Chan; used in unionread */
```

⟨chanfree() *optional free* 260b⟩≡ (208a) 260i▷

```
if(c->umh != nil){
    putmhead(c->umh);
    c->umh = nil;
}
```

⟨namec() *print error if c-¿umh != nil* 260c⟩≡ (215f)

```
if(c->umh != nil){
    print("uniquechan umh Open\n");
    putmhead(c->umh);
    c->umh = nil;
}
```

⟨walk() *print error if c-¿umh != nil* 260d⟩≡ (234h)

```
if(c->umh != nil){  //BUG
    print("walk umh\n");
    putmhead(c->umh);
    c->umh = nil;
}
```

⟨cmount() *print error if old-¿umh != nil* 260e⟩≡ (252e)

```
if(old->umh)
    print("cmount: unexpected umh, caller %#p\n", getcallerpc(&new));
```

14.9.3 syspread() for directories

⟨Chan *other fields* 260f⟩+≡ (31a) ◁260a 260h▷

```
vlong devoffset;    /* in underlying device; see read */
```

⟨read() *if c is a QTDIR* 260g⟩≡ (222d)

```
    if(c->qid.type & QTDIR){
        if(mountrockread(c, p, n, &nn)){
            /* do nothing: mountrockread filled buffer */
        }else if(c->umh){
            nn = unionread(c, p, n);
        }else{
            if(off != c->offset)
                error(Edirseek);
            nn = devtab[c->type]->read(c, p, n, c->devoffset);
        }
        nnn = mountfix(c, p, nn, n);
    }
```

⟨Chan *other fields* 260h⟩+≡ (31a) ◁260f 266a▷

```
Chan* umc;      /* channel in union; held for union read */
QLock umqlock;    /* serialize unionreads */
int uri;      /* union read index */
```

⟨chanfree() *optional free* 260i⟩+≡ (208a) ◁260b 268b▷

```
if(c->umc != nil){
    cclose(c->umc);
    c->umc = nil;
}
```

⟨*function* unionread 261a⟩≡ (578)

```
long
unionread(Chan *c, virt_vp va, long n)
{
    int i;
    long nr;
    Mhead *m;
    Mount *mount;

    qlock(&c->umqlock);
    m = c->umh;
    rlock(&m->lock);
    mount = m->mount;

    /* bring mount in sync with c->uri and c->umc */
    for(i = 0; mount != nil && i < c->uri; i++)
        mount = mount->next;

    nr = 0;
    while(mount != nil){
        /* Error causes component of union to be skipped */
        if(mount->to && !waserror()){
            if(c->umc == nil){
                c->umc = cclone(mount->to);
                c->umc = devtab[c->umc->type]->open(c->umc, OREAD);
            }

            nr = devtab[c->umc->type]->read(c->umc, va, n, c->umc->offset);
            c->umc->offset += nr;
            poperror();
        }
        if(nr > 0)
            break;

        /* Advance to next element */
        c->uri++;
        if(c->umc){
            cclose(c->umc);
            c->umc = nil;
        }
        mount = mount->next;
    }
    runlock(&m->lock);
    qunlock(&c->umqlock);
    return nr;
}
```

⟨read() *rewind when off == 0* 261b⟩≡ (222d)

```
/*
 * The offset is passed through on directories, normally.
 * Sysseek complains, but pread is used by servers like exportfs,
 * that shouldn't need to worry about this issue.
 *
 * Notice that c->devoffset is the offset that c's dev is seeing.
 * The number of bytes read on this fd (c->offset) may be different
 * due to rewritings in rockfix.
 */
    mountrewind(c);
    unionrewind(c);
```

⟨*function* unionrewind 262a⟩≡ (578)

```
static void
unionrewind(Chan *c)
{
    qlock(&c->umqlock);
    c->uri = 0;
    if(c->umc){
        cclose(c->umc);
        c->umc = nil;
    }
    qunlock(&c->umqlock);
}
```

14.9.4 MCREATE

⟨namec() *Acreate case, if cnew is a mountpoint* 262b⟩≡ (238e)

```
if(!sharppath && findmount(&cnew, &m, c->type, c->dev, c->qid))
    cnew = createdir(cnew, m);
```

⟨*function* createdir 262c⟩≡ (571)

```
/*
 * c is a mounted non-creatable directory.  find a creatable one.
 */
Chan*
createdir(Chan *c, Mhead *m)
{
    Chan *nc;
    Mount *f;

    rlock(&m->lock);
    if(waserror()){
        runlock(&m->lock);
        nexterror();
    }
    for(f = m->mount; f; f = f->next){
        if(f->mflag&MCREATE){
            nc = cclone(f->to);
            runlock(&m->lock);
            poperror();
            cclose(c);
            return nc;
        }
    }
    error(Enocreate);
    panic("createdir: should not reach this point");
    return nil; // unreachable
}
```

14.9.5 Union of union

⟨namec() *set c-¿umh in Abind if mounted point* 262d⟩≡ (252b)

```
    if(c->umh != nil)
        putmhead(c->umh);
    c->umh = m;
```

⟨cmount() *locals* 262e⟩+≡ (252e) ◁252d

```
int flg;
Mhead *umh, **l;
Mount *f, *um, **h;
```

⟨cmount() *if new is itself a mount point, error if cant create there* 263a⟩≡ (252e)

```
umh = new->umh;
/*
 * Not allowed to bind when the old directory is itself a union.
 * (Maybe it should be allowed, but I don't see what the semantics
 * would be.)
 *
 * We need to check umh->mount->next to tell unions apart from
 * simple mount points, so that things like
 *  mount -c fd /root
 *  bind -c /root /
 * work.
 *
 * The check of mount->mflag allows things like
 *  mount fd /root
 *  bind -c /root /
 *
 * This is far more complicated than it should be, but I don't
 * see an easier way at the moment.
 */
if((flag&MCREATE) && umh && umh->mount
   && (umh->mount->next || !(umh->mount->mflag&MCREATE)))
    error(Emount);
```

⟨cmount() *if new is itself a mount point, copy mounts* 263b⟩≡ (252e)

```
if(umh != nil && umh->mount != nil){
    /*
     *  copy a union when binding it onto a directory
     */
    flg = order;
    if(order == MREPL)
        flg = MAFTER;
    h = &m->next;
    um = umh->mount;
    for(um = um->next; um; um = um->next){
        f = newmount(um->to, flg, um->spec);
        *h = f;
        h = &f->next;
    }
}
```

14.10 Sandboxing

⟨Pgrp *other fields* 263c⟩+≡ (249c) ◁250b

```
bool noattach;
```

⟨bindmount() *error if noattach* 263d⟩≡ (264g)

```
if(up->pgrp->noattach)
    error(Enoattach);
```

⟨namec() *noattach checking* 263e⟩≡ (251a)

```
/*
 *  noattach is sandboxing.
 *
 *  the OK exceptions are:
 *  |  it only gives access to pipes you create
 *  d  this process's file descriptors
 *  e  this process's environment
```

```
 *  the iffy exceptions are:
 *  c  time and pid, but also cons and consctl
 *  p  control of your own processes (and unfortunately
 *     any others left unprotected)
 */
/* actually / is caught by parsing earlier */
if(utfrune("M", r))
    error(Enoattach);
if(up->pgrp->noattach && utfrune("|decp", r)==nil)
    error(Enoattach);
```

⟨sysrfork() *inherit noattach, RFPROC==0 case* 264a⟩≡ (82e)

```
/* inherit noattach */
up->pgrp->noattach = opg->noattach;
```

⟨sysrfork() *inherit noattach, RFPROC==1 case* 264b⟩≡ (81b)

```
/* inherit noattach */
p->pgrp->noattach = up->pgrp->noattach;
```

⟨sysrfork() *set noattach to true when RFNOMNT, RFPROC==0 case* 264c⟩≡ (82e)

```
if(flag & RFNOMNT)
    up->pgrp->noattach = true;
```

⟨sysrfork() *set noattach to true when RFNOMNT, RFPROC==1 case* 264d⟩≡ (81b)

```
if(flag & RFNOMNT)
    p->pgrp->noattach = true;
```

14.11 sysmount()

⟨*syscall mount* 264e⟩≡ (578)

```
// int mount(int fd, int afd, char *old, int flag, char *aname);
long
sysmount(ulong* arg)
{
    return bindmount(true, arg[0], arg[1], nil, (char*)arg[2], arg[3], (char*)arg[4]);
}
```

⟨bindmount() *locals* 264f⟩+≡ (251e) ◁ ??

```
Chan *ac, *bc;

struct Bogus bogus;
```

⟨bindmount() *if ismount* 264g⟩≡ (251e)

```
if(ismount){
    validaddr((ulong)spec, 1, false);
    spec = validnamedup(spec, true);
    if(waserror()){
        free(spec);
        nexterror();
    }
    ⟨bindmount() error if noattach 263d⟩

    ac = nil;
    bc = fdtochan(fd, ORDWR, false, true);
    if(waserror()) {
        if(ac)
            cclose(ac);
        cclose(bc);
```

```
        nexterror();
    }

    if(afd >= 0)
        ac = fdtochan(afd, ORDWR, false, true);

    bogus.flags = flag & MCACHE;
    bogus.chan = bc;
    bogus.authchan = ac;
    bogus.spec = spec;
    ret = devno('M', false);
    c0 = devtab[ret]->attach((char*)&bogus);
    poperror(); /* ac bc */
    if(ac)
        cclose(ac);
    cclose(bc);
}
```

⟨bindmount() *if ismount free* 265a⟩≡ (251e)

```
if(ismount){
    fdclose(fd, 0);
    poperror();
    free(spec);
}
```

⟨MountFlags *cases* 265b⟩≡ (247b)

```
MCACHE = 0x0010,  /* cache some data */
```

14.12 sysumount()

⟨*syscall unmount* 265c⟩≡ (578)

```
// int unmount(char *name, char *old);
long
sysunmount(ulong* arg)
{
    Chan *cmount, *cmounted;

    cmounted = nil;

    validaddr(arg[1], 1, false);
    cmount = namec((char *)arg[1], Amount, 0, 0);
    if(waserror()) {
        cclose(cmount);
        if(cmounted)
            cclose(cmounted);
        nexterror();
    }

    if(arg[0]) {
        /*
         * This has to be namec(..., Aopen, ...) because
         * if arg[0] is something like /srv/cs or /fd/0,
         * opening it is the only way to get at the real
         * Chan underneath.
         */
        validaddr(arg[0], 1, false);
        cmounted = namec((char*)arg[0], Aopen, OREAD, 0);
    }
```

```
    cunmount(cmount, cmounted);
    poperror();
    cclose(cmount);
    if(cmounted)
        cclose(cmounted);
    return 0;
}
```

⟨Chan *other fields* 266a⟩+≡ (31a) ◁260h 269b▷

```
Chan* mchan;      /* channel to mounted server */
```

⟨cunmount() *print error if mnt-¿umh* 266b⟩≡ (266c)

```
if(mnt->umh)    /* should not happen */
    print("cunmount newp extra umh %p has %p\n", mnt, mnt->umh);
```

⟨*function* cunmount 266c⟩≡ (571)

```
void
cunmount(Chan *mnt, Chan *mounted)
{
    Pgrp *pg;
    Mhead *m, **l;
    Mount *f, **p;

    ⟨cunmount() print error if mnt-¿umh 266b⟩

    /*
     * It _can_ happen that mounted->umh is non-nil,
     * because mounted is the result of namec(Aopen)
     * (see sysfile.c:/^sysunmount).
     * If we open a union directory, it will have a umh.
     * Although surprising, this is okay, since the
     * cclose will take care of freeing the umh.
     */

    pg = up->pgrp;
    wlock(&pg->ns);

    l = &MOUNTH(pg, mnt->qid);
    for(m = *l; m; m = m->hash){
        if(eqchan(m->from, mnt, true))
            break;
        l = &m->hash;
    }

    if(m == nil){
        wunlock(&pg->ns);
        error(Eunmount);
    }

    wlock(&m->lock);
    if(mounted == nil){
        *l = m->hash;
        wunlock(&pg->ns);
        mountfree(m->mount);
        m->mount = nil;
        cclose(m->from);
        wunlock(&m->lock);
        putmhead(m);
        return;
    }
```

```
    p = &m->mount;
    for(f = *p; f; f = f->next){
        /* BUG: Needs to be 2 pass */
        if(eqchan(f->to, mounted, true) ||
          (f->to->mchan && eqchan(f->to->mchan, mounted, true))){
            *p = f->next;
            f->next = nil;
            mountfree(f);
            if(m->mount == nil){
                *l = m->hash;
                cclose(m->from);
                wunlock(&m->lock);
                wunlock(&pg->ns);
                putmhead(m);
                return;
            }
            wunlock(&m->lock);
            wunlock(&pg->ns);
            return;
        }
        p = &f->next;
    }
    wunlock(&m->lock);
    wunlock(&pg->ns);
    error(Eunion);
}
```

14.13 XTODO

⟨Mount *other fields* 267a⟩≡ (248b) 267c ▷

```
ulong mountid;
```

⟨*global* mountid 267b⟩≡ (597b)

```
static Counter mountid;
```

⟨Mount *other fields* 267c⟩+≡ (248b) ◁ 267a

```
// list<ref<Mount>> sorted list by mount->mountid
Mount*  order;
```

⟨Mount *extra fields* 267d⟩+≡ (248b) ◁ 249a

```
Mount*  copy;
```

⟨*function* pgrpinsert 267e⟩≡ (597b)

```
void
pgrpinsert(Mount **order, Mount *m)
{
    Mount *f;

    m->order = nil;
    if(*order == nil) {
        *order = m;
        return;
    }
    for(f = *order; f; f = f->order) {
        if(m->mountid < f->mountid) {
            m->order = f;
            *order = m;
```

```
            return;
        }
        order = &f->order;
    }
    *order = m;
}
```

⟨*function* pgrpcpy 268a⟩≡ (597b)

```
/*
 * pgrpcpy MUST preserve the mountid allocation order of the parent group
 */
void
pgrpcpy(Pgrp *to, Pgrp *from)
{
    int i;
    Mount *n, *m, **link, *order;
    Mhead *f, **tom, **l, *mh;

    wlock(&from->ns);
    order = nil;
    tom = to->mnthash;
    for(i = 0; i < MNTHASH; i++) {
        l = tom++;
        for(f = from->mnthash[i]; f; f = f->hash) {
            rlock(&f->lock);
            mh = newmhead(f->from);
            *l = mh;
            l = &mh->hash;
            link = &mh->mount;
            for(m = f->mount; m; m = m->next) {
                n = newmount(m->to, m->mflag, m->spec);

                m->copy = n;
                pgrpinsert(&order, m);

                *link = n;
                link = &n->next;
            }
            runlock(&f->lock);
        }
    }
    /*
     * Allocate mount ids in the same sequence as the parent group
     */
    lock(&mountid);
    for(m = order; m; m = m->order)
        m->copy->mountid = mountid.ref++;
    unlock(&mountid);
    wunlock(&from->ns);
}
```

14.14 XTODO

⟨chanfree() *optional free* 268b⟩+≡ (208a) ◁260i 269a▷

```
if(c->mux != nil){
    muxclose(c->mux);
    c->mux = nil;
}
```

```
if(c->mchan != nil){
    cclose(c->mchan);
    c->mchan = nil;
}
```

⟨chanfree() *optional free* 269a⟩+≡ (208a) ◁268b

```
if(c->dirrock != nil){
    free(c->dirrock);
    c->dirrock = nil;
    c->nrock = 0;
    c->mrock = 0;
}
```

⟨Chan *other fields* 269b⟩+≡ (31a) ◁266a 321b▷

```
byte*  dirrock;    /* directory entry rock for translations */
int nrock;
int mrock;
QLock rockqlock;
```

⟨*function* mountrock 269c⟩≡ (578)

```
/*
 * Mountfix might have caused the fixed results of the directory read
 * to overflow the buffer.  Catch the overflow in c->dirrock.
 */
static void
mountrock(Chan *c, byte *p, byte **pe)
{
    byte *e, *r;
    int len, n;

    e = *pe;

    /* find last directory entry */
    for(;;){
        len = BIT16SZ+GBIT16(p);
        if(p+len >= e)
            break;
        p += len;
    }

    /* save it away */
    qlock(&c->rockqlock);
    if(c->nrock+len > c->mrock){
        n = ROUND(c->nrock+len, 1024);
        r = smalloc(n);
        memmove(r, c->dirrock, c->nrock);
        free(c->dirrock);
        c->dirrock = r;
        c->mrock = n;
    }
    memmove(c->dirrock+c->nrock, p, len);
    c->nrock += len;
    qunlock(&c->rockqlock);

    /* drop it */
    *pe = p;
}
```

⟨*function* mountrockread 270a⟩≡ (578)

```
/*
 * Satisfy a directory read with the results saved in c->dirrock.
 */
static bool
mountrockread(Chan *c, byte *op, long n, long *nn)
{
    long dirlen;
    byte *rp, *erp, *ep, *p;

    /* common case */
    if(c->nrock == 0)
        return false;

    /* copy out what we can */
    qlock(&c->rockqlock);
    rp = c->dirrock;
    erp = rp+c->nrock;
    p = op;
    ep = p+n;
    while(rp+BIT16SZ <= erp){
        dirlen = BIT16SZ+GBIT16(rp);
        if(p+dirlen > ep)
            break;
        memmove(p, rp, dirlen);
        p += dirlen;
        rp += dirlen;
    }

    if(p == op){
        qunlock(&c->rockqlock);
        return false;
    }

    /* shift the rest */
    if(rp != erp)
        memmove(c->dirrock, rp, erp-rp);
    c->nrock = erp - rp;

    *nn = p - op;
    qunlock(&c->rockqlock);
    return true;
}
```

⟨*function* mountrewind 270b⟩≡ (578)

```
static void
mountrewind(Chan *c)
{
    c->nrock = 0;
}
```

⟨*function* mountfix 270c⟩≡ (578)

```
/*
 * Rewrite the results of a directory read to reflect current
 * name space bindings and mounts.  Specifically, replace
 * directory entries for bind and mount points with the results
 * of statting what is mounted there.  Except leave the old names.
 */
static long
mountfix(Chan *c, byte *op, long n, long maxn)
```

```
{
    char *name;
    int nbuf, nname;
    Chan *nc;
    Mhead *mh;
    Mount *m;
    byte *p;
    int dirlen, rest;
    long l;
    byte *buf, *e;
    DirEntry d;

    p = op;
    buf = nil;
    nbuf = 0;
    for(e=&p[n]; p+BIT16SZ<e; p+=dirlen){
        dirlen = dirfixed(p, e, &d);
        if(dirlen < 0)
            break;
        nc = nil;
        mh = nil;
        if(findmount(&nc, &mh, d.type, d.dev, d.qid)){
            /*
             * If it's a union directory and the original is
             * in the union, don't rewrite anything.
             */
            for(m=mh->mount; m; m=m->next)
                if(eqchantdqid(m->to, d.type, d.dev, d.qid, true))
                    goto Norewrite;

            name = offsetof_name_direntry(p, &nname);
            /*
             * Do the stat but fix the name.  If it fails, leave old entry.
             * BUG: If it fails because there isn't room for the entry,
             * what can we do?  Nothing, really.  Might as well skip it.
             */
            if(buf == nil){
                buf = smalloc(4096);
                nbuf = 4096;
            }
            if(waserror())
                goto Norewrite;
            l = devtab[nc->type]->stat(nc, buf, nbuf);
            l = direntry_setname(name, nname, buf, l, nbuf);
            if(l == BIT16SZ)
                error("direntry_setname");
            poperror();

            /*
             * Shift data in buffer to accomodate new entry,
             * possibly overflowing into rock.
             */
            rest = e - (p+dirlen);
            if(l > dirlen){
                while(p+l+rest > op+maxn){
                    mountrock(c, p, &e);
                    if(e == p){
                        dirlen = 0;
                        goto Norewrite;
                    }
```

```
                rest = e - (p+dirlen);
            }
        }
        if(l != dirlen){
            memmove(p+l, p+dirlen, rest);
            dirlen = l;
            e = p+dirlen+rest;
        }

        /*
         * Rewrite directory entry.
         */
        memmove(p, buf, l);

        Norewrite:
        cclose(nc);
        putmhead(mh);
    }
}
if(buf)
    free(buf);

if(p != e)
    error("oops in rockfix");

return e-op;
}
```

⟨*function* dirfixed 272⟩≡ (578)

```
static int
dirfixed(byte *p, byte *e, DirEntry *d)
{
    int len;

    len = GBIT16(p)+BIT16SZ;
    if(p + len > e)
        return -1;

    p += BIT16SZ;   /* ignore size */
    d->type = devno(GBIT16(p), true);
    p += BIT16SZ;
    d->dev = GBIT32(p);
    p += BIT32SZ;
    d->qid.type = GBIT8(p);
    p += BIT8SZ;
    d->qid.vers = GBIT32(p);
    p += BIT32SZ;
    d->qid.path = GBIT64(p);
    p += BIT64SZ;
    d->mode = GBIT32(p);
    p += BIT32SZ;
    d->atime = GBIT32(p);
    p += BIT32SZ;
    d->mtime = GBIT32(p);
    p += BIT32SZ;
    d->length = GBIT64(p);

    return len;
}
```

Chapter 15

Devices

15.1 The keyboard

15.1.1 Overview

15.1.2 Scan code

⟨*enum* specialkey 273⟩≡ (536)

```
enum {
    KF=         0xF000,     /* function key (begin Unicode private space) */
    Spec=       0xF800,     /* Unicode private space */


    PF=         Spec|0x20,  /* num pad function key */
    View=       Spec|0x00,  /* view (shift window up) */

    Shift=      Spec|0x60,
    Break=      Spec|0x61,
    Ctrl=       Spec|0x62,
    Alt=        Spec|0x63,
    Caps=       Spec|0x64,
    Num=        Spec|0x65,
    Middle=     Spec|0x66,
    Altgr=      Spec|0x67,

    Kmouse=     Spec|0x100,

    No=         0x00,       /* peter */

    /* KF|1, KF|2, ..., KF|0xC is F1, F2, ..., F12 */

    Home=       KF|13,
    Up=         KF|14,
    Pgup=       KF|15,
    Print=      KF|16,
    Left=       KF|17,
    Right=      KF|18,
    End=        KF|24,
    Down=       View,
    Pgdown=     KF|19,
    Ins=        KF|20,
    Del=        0x7F,
    Scroll=     KF|21,
```

```
    Nscan=  128,
};
```

⟨*global* kbtab 274a⟩≡ (536)

```
Rune kbtab[Nscan] =
{
[0x00]  No, 0x1b,   '1',    '2',    '3',    '4',    '5',    '6',
[0x08]  '7',    '8',    '9',    '0',    '-',    '=',    '\b',   '\t',
[0x10]  'q',    'w',    'e',    'r',    't',    'y',    'u',    'i',
[0x18]  'o',    'p',    '[',    ']',    '\n',   Ctrl,   'a',    's',
[0x20]  'd',    'f',    'g',    'h',    'j',    'k',    'l',    ';',
[0x28]  '\'',   '`',    Shift,  '\\',   'z',    'x',    'c',    'v',
[0x30]  'b',    'n',    'm',    ',',    '.',    '/',    Shift,  '*',
[0x38]  Alt,  ' ',    Ctrl,   KF|1,   KF|2,   KF|3,   KF|4,   KF|5,
[0x40]  KF|6,   KF|7,   KF|8,   KF|9,   KF|10,  Num,    Scroll, '7',
[0x48]  '8',    '9',    '-',    '4',    '5',    '6',    '+',    '1',
[0x50]  '2',    '3',    '0',    '.',    No, No, No, KF|11,
[0x58]  KF|12,  No, No, No, No, No, No, No,
[0x60]  No, No, No, No, No, No, No, No,
[0x68]  No, No, No, No, No, No, No, No,
[0x70]  No, No, No, No, No, No, No, No,
[0x78]  No, No, No, No, No, No, No, No,
};
```

⟨*global* kbtabshift 274b⟩≡ (536)

```
Rune kbtabshift[Nscan] =
{
[0x00]  No, 0x1b,   '!',    '@',    '#',    '$',    '%',    '^',
[0x08]  '&',    '*',    '(',    ')',    '_',    '+',    '\b',   '\t',
[0x10]  'Q',    'W',    'E',    'R',    'T',    'Y',    'U',    'I',
[0x18]  'O',    'P',    '{',    '}',    '\n',   Ctrl,   'A',    'S',
[0x20]  'D',    'F',    'G',    'H',    'J',    'K',    'L',    ':',
[0x28]  '"',    '~',    Shift,  '|',    'Z',    'X',    'C',    'V',
[0x30]  'B',    'N',    'M',    '<',    '>',    '?',    Shift,  '*',
[0x38]  Alt,  ' ',    Ctrl,   KF|1,   KF|2,   KF|3,   KF|4,   KF|5,
[0x40]  KF|6,   KF|7,   KF|8,   KF|9,   KF|10,  Num,    Scroll, '7',
[0x48]  '8',    '9',    '-',    '4',    '5',    '6',    '+',    '1',
[0x50]  '2',    '3',    '0',    '.',    No, No, No, KF|11,
[0x58]  KF|12,  No, No, No, No, No, No, No,
[0x60]  No, No, No, No, No, No, No, No,
[0x68]  No, No, No, No, No, No, No, No,
[0x70]  No, No, No, No, No, No, No, No,
[0x78]  No, No, No, No, No, No, No, No,
};
```

⟨*global kbtabctrl decl* 274c⟩≡ (536)

```
extern Rune kbtabctrl[];
```

⟨*global* kbtabaltgr 274d⟩≡ (536)

```
Rune kbtabaltgr[Nscan] =
{
[0x00]  No, No, No, No, No, No, No, No,
[0x08]  No, No, No, No, No, No, No, No,
[0x10]  No, No, No, No, No, No, No, No,
[0x18]  No, No, No, No, '\n',   Ctrl,   No, No,
[0x20]  No, No, No, No, No, No, No, No,
[0x28]  No, No, Shift,  No, No, No, No, No,
[0x30]  No, No, No, No, No, '/',    No, Print,
[0x38]  Altgr,  No, No, No, No, No, No, No,
[0x40]  No, No, No, No, No, No, Break,  Home,
```

```
[0x48]  Up, Pgup,   No, Left,   No, Right,  No, End,
[0x50]  Down,   Pgdown, Ins,    Del,    No, No, No, No,
[0x58]  No, No, No, No, No, No, No, No,
[0x60]  No, No, No, No, No, No, No, No,
[0x68]  No, No, No, No, No, No, No, No,
[0x70]  No, No, No, No, No, No, No, No,
[0x78]  No, No, No, No, No, No, No, No,
};
```

⟨*struct* Kbscan 275a⟩≡ (535d)

```
struct Kbscan {
    bool ctl;
    bool shift;
    bool caps;
    bool alt;
    bool altgr;
    bool num;
    ⟨Kbscan other fields 277c⟩
};
```

⟨*enum* kbscan 275b⟩≡ (535d)

```
/* kbscans indices */
enum KbscanKind {
    KbInt=    0,
    KbExt,

    KbNscans,
};
```

⟨*global* kbscans 275c⟩≡ (536)

```
// hash<enum<KbscanKind>, Kbscan>
Kbscan kbscans[KbNscans]; /* kernel and external scan code state */
```

⟨*function* kbdputsc 275d⟩≡ (536)

```
/*
 * Scan code processing
 */
void
kbdputsc(byte k, int external)
{
    bool keyup;
    Kbscan *kbscan;
    // Rune is (uint) but actually 'c' can get the result of functions
    // like latin1() which can return negative numbers.
    long c = k;

    if(external)
        kbscan = &kbscans[KbExt];
    else
        kbscan = &kbscans[KbInt];

    ⟨kbdputsc() debugging 499c⟩

    ⟨kbdputsc() esc key handling part1 and possible return 277d⟩

    keyup = c & 0x80; // key released (1 bit)
    c &= 0x7f; // 128

    ⟨kbdputsc() ensures c is in boundary ??⟩
```

```
⟨kbdputsc() esc key handling part2 and possible return 278a⟩
else if(kbscan->shift)
    c = kbtabshift[c];
else if(kbscan->altgr)
    c = kbtabaltgr[c];
else if(kbscan->ctl)
    c = kbtabctrl[c];
else
    c = kbtab[c];

if(kbscan->caps && c<='z' && c>='a')
    c += 'A' - 'a';

/*
 *  keyup only important for shifts
 */
if(keyup){
    switch(c){
    case Ctrl:
        kbscan->ctl = false;
        break;
    case Alt:
        kbscan->alt = false;
        break;
    case Altgr:
        kbscan->altgr = false;
        break;
    case Shift:
        kbscan->shift = false;
        ⟨kbdputsc() reset mouseshift 291c⟩
        ⟨kbdputsc() debugging up shift 499d⟩
        break;
    ⟨kbdputsc() mouse keyup cases 291e⟩
    }
    return;
}

/*
 *  normal character
 */
if(!(c & (Spec|KF))){
    ⟨kbdputsc() reboot if ctl-alt-del 488d⟩
    ⟨kbdputsc() if collecting 527c⟩
    else {
        kbdputc(c); //!! adding the character in kbd staging area
    }
}else{
    switch(c){
    case Ctrl:
        kbscan->ctl = true;
        break;
    case Alt:
        kbscan->alt = true;
        ⟨kbdputsc() start collecting 527d⟩
        break;
    case Altgr:
        kbscan->altgr = true;
        break;
    case Shift:
        kbscan->shift = true;
```

```
            ⟨kbdputsc() set mouseshift 291d⟩
            ⟨kbdputsc() debugging down shift 499e⟩
            break;
        case Caps:
            kbscan->caps ^= true;
            break;
        case Num:
            kbscan->num ^= true;
            if(!external)
                arch_setleds(kbscan);
            break;
        ⟨kbdputsc() mouse keydown cases 291f⟩
        ⟨kbdputsc() special keyboard debug keys cases 499b⟩
        // e.g. Left, Right or other special key
        default:
            kbdputc(c);
            break;
        }
    }
}
```

⟨*function* `arch_setleds`*(arm)* 277a⟩≡ (622a)

```
// empty callback for portkbd.c
void arch_setleds(Kbscan *kbscan) { USED(kbscan); }
```

15.1.3 Esc

⟨*global* `kbtabesc1` 277b⟩≡ (536)

```
Rune kbtabesc1[Nscan] =
{
[0x00]  No, No, No, No, No, No, No, No,
[0x08]  No, No, No, No, No, No, No, No,
[0x10]  No, No, No, No, No, No, No, No,
[0x18]  No, No, No, No, '\n',   Ctrl,   No, No,
[0x20]  No, No, No, No, No, No, No, No,
[0x28]  No, No, Shift,  No, No, No, No, No,
[0x30]  No, No, No, No, No, '/',    No, Print,
[0x38]  Altgr,  No, No, No, No, No, No, No,
[0x40]  No, No, No, No, No, No, Break,  Home,
[0x48]  Up, Pgup,   No, Left,   No, Right,  No, End,
[0x50]  Down,   Pgdown, Ins,    Del,    No, No, No, No,
[0x58]  No, No, No, No, No, No, No, No,
[0x60]  No, No, No, No, No, No, No, No,
[0x68]  No, No, No, No, No, No, No, No,
[0x70]  No, No, No, No, No, No, No, No,
[0x78]  No, No, No, No, No, No, No, No,
};
```

⟨`Kbscan` *other fields* 277c⟩≡ (275a) 291b ▷

```
bool esc1;
int esc2;
```

⟨`kbdputsc()` *esc key handling part1 and possible return* 277d⟩≡ (275d)

```
/*
 *  e0's is the first of a 2 character sequence, e1 the first
 *  of a 3 character sequence (on the safari)
 */
if(c == 0xe0){
    kbscan->esc1 = true;
```

```
        return;
    } else if(c == 0xe1){
        kbscan->esc2 = 2;
        return;
    }
```

⟨kbdputsc() *esc key handling part2 and possible return* 278a⟩≡ (275d)

```
    if(kbscan->esc1){
        c = kbtabesc1[c];
        kbscan->esc1 = false;
    } else if(kbscan->esc2){
        kbscan->esc2--;
        return;
    }
```

15.1.4 kbd

⟨*struct* ConsKbd 278b⟩≡ (531)

```
    struct ConsKbd
    {
        /* a place to save up characters at interrupt time before dumping them in the queue */
        char    istage[1024];
        // pointers into istage to implement a circular buffer
        char    *iw; // write
        char    *ir; // read
        char    *ie; // end
        ILock    lockputc;

        ⟨ConsKbd other fields 284d⟩
        // extra
        QLock;
    };
```

⟨*global* kbd 278c⟩≡ (531)

```
    static struct ConsKbd kbd =
    {
        .iw = kbd.istage,
        .ir = kbd.istage,
        .ie = kbd.istage + sizeof(kbd.istage),
    };
```

⟨*function* kbdputc 278d⟩≡ (531)

```
    /*
     *  Put character, possibly a rune, into kbd at interrupt time.
     *  Called at interrupt time to process a character.
     */
    void
    kbdputc(Rune ch)
    {
        int i, n;
        char buf[UTFmax]; // pad's fourth bugfix :)
        Rune r;
        char *next;

        ⟨kbdputc() debugging 499f⟩

        if(kbd.ir == nil)
            return;       /* in case we're not inited yet */
```

```
    ilock(&kbd.lockputc);       /* just a mutex */
    r = ch;
    n = runetochar(buf, &r);
    for(i = 0; i < n; i++){
        next = kbd.iw+1;
        // circular buffer
        if(next >= kbd.ie)
            next = kbd.istage;
        if(next == kbd.ir) // full
            break;
        *kbd.iw = buf[i];
        kbd.iw = next;
    }
    iunlock(&kbd.lockputc);
    return;
}
```

⟨*clock callback kbdputcclock* 279a⟩≡ (531)

```
/*
 *  we save up input characters till clock time to reduce
 *  per character interrupt overhead.
 */
static void
kbdputcclock(void)
{
    char *iw;

    /* this amortizes cost of qproduce */
    if(kbd.iw != kbd.ir){
        iw = kbd.iw;
        if(iw < kbd.ir){
            echo(kbd.ir, kbd.ie-kbd.ir);
            kbd.ir = kbd.istage;
        }
        if(kbd.ir != iw){
            echo(kbd.ir, iw-kbd.ir);
            kbd.ir = iw;
        }
    }
}
```

⟨`consinit()` *initializing things* 279b⟩≡ (288a) 494c ▷

```
/*
 * at 115200 baud, the 1024 char buffer takes 56 ms to process,
 * processing it every 22 ms should be fine
 */
addclock0link(kbdputcclock, 22);
```

15.1.5 echo()

⟨*function* `echo` 279c⟩≡ (531)

```
static void
echo(char *buf, int n)
{
    ⟨echo() locals 499g⟩

    if(n == 0)
        return;
```

```
    ⟨echo() special keys handler 500⟩

    qproduce(kbdq, buf, n); //!! add in kbd queue to be read from /dev/cons

    ⟨echo() return before any echoscreen if raw mode 290e⟩

    ⟨echo() hooks 280a⟩
}
```

⟨echo() *hooks* 280a⟩≡ (279c) 507d ▷

```
if(screenputs != nil)
   echoscreen(buf, n);
```

⟨*hook screenputs* 280b⟩≡ (531)

```
void    (*screenputs)(char*, int) = nil;
```

⟨*function* echoscreen 280c⟩≡ (531)

```
static void
echoscreen(char *buf, int n)
{
    char *e, *p;
    char ebuf[128];
    int x;

    p = ebuf;
    e = ebuf + sizeof(ebuf) - 4;
    while(n-- > 0){
        if(p >= e){
            screenputs(ebuf, p - ebuf);
            p = ebuf;
        }
        x = *buf++;
        if(x == 0x15){ // ??
            *p++ = '^';
            *p++ = 'U';
            *p++ = '\n';
        } else
            *p++ = x;
    }
    if(p != ebuf)
        screenputs(ebuf, p - ebuf);
}
```

15.1.6 PC Keyboard (x86)

15.1.7 Raspberry Pi USB Keyboard (ARM)

15.1.8 Changing keyboard: /dev/kbmap

⟨*function* kbdgetmap 280d⟩≡ (536)

```
int
kbdgetmap(uint offset, int *t, int *sc, Rune *r)
{
    if ((int)offset < 0)
        error(Ebadarg);
    *t = offset/Nscan;
    *sc = offset%Nscan;
    switch(*t) {
    default:
```

```
        return 0;
    case 0:
        *r = kbtab[*sc];
        return 1;
    case 1:
        *r = kbtabshift[*sc];
        return 1;
    case 2:
        *r = kbtabesc1[*sc];
        return 1;
    case 3:
        *r = kbtabaltgr[*sc];
        return 1;
    case 4:
        *r = kbtabctrl[*sc];
        return 1;
    }
}
```

⟨*function* kbdputmap 281a⟩≡ (536)

```
void
kbdputmap(ushort m, ushort scanc, Rune r)
{
    if(scanc >= Nscan)
        error(Ebadarg);
    switch(m) {
    default:
        error(Ebadarg);
    case 0:
        kbtab[scanc] = r;
        break;
    case 1:
        kbtabshift[scanc] = r;
        break;
    case 2:
        kbtabesc1[scanc] = r;
        break;
    case 3:
        kbtabaltgr[scanc] = r;
        break;
    case 4:
        kbtabctrl[scanc] = r;
        break;
    }
}
```

15.1.9 External keyboard input: /dev/kbin

15.2 The screen

15.2.1 Overview

15.2.2 Text output on a PC (x86)

15.2.3 Text output on a Raspberry Pi (ARM)

⟨*constant* VCBUFFER*(arm)* 281b⟩≡ (633b)

```
#define VCBUFFER    (KZERO+0x3400)      /* videocore mailbox buffer */
```

⟨*function* fbinit*(arm)* 282a⟩≡ (615d)

```
void*
fbinit(bool set, int *width, int *height, int *depth)
{
    Fbinfo *fi = (Fbinfo*)(VCBUFFER);
    uintptr va;

    if(!set)
        fbdefault(width, height, depth);
    /* Screen width must be a multiple of 16 */
    *width &= ~0xF;

    memset(fi, 0, sizeof(Fbinfo));
    fi->xres = fi->xresvirtual = *width;
    fi->yres = fi->yresvirtual = *height;
    fi->bpp = *depth;
    ⟨fbinit() write back cache after modified fb info(arm) 402n⟩
    vcwrite(ChanFb, dmaaddr(fi));

    if(vcread(ChanFb) != 0)
        return nil;
    //TODO: fi->base is in virtual space?? how vcore knows about that? PADDR?
    va = mmukmap(FRAMEBUFFER, PADDR(fi->base), fi->screensize);
    // make it a blue screen
    if(va)
        memset((char*)va, 0x7F, fi->screensize);
    return (void*)va;
}
```

⟨*function* mmukmap*(arm)* 282b⟩≡ (635a)

```
uintptr
mmukmap(uintptr va, uintptr pa, usize size)
{
    int o;
    usize n;
    PTE *pte, *pte0;

    assert((va & (MiB-1)) == 0);
    o = pa & (MiB-1);
    pa -= o;
    size += o;
    pte = pte0 = &cpu->mmul1[L1X(va)];
    for(n = 0; n < size; n += MiB)
        if(*pte++ != Fault)
            return 0;
    pte = pte0;
    for(n = 0; n < size; n += MiB){
        *pte++ = (pa+n)|Dom0|L1AP(Krw)|Section;
        mmuinvalidateaddr(va+n);
    }
    ⟨mmukmap() write back cache after changing page tables(arm) 402g⟩
    return va + o;
}
```

⟨*function* fbdefault*(arm)* 282c⟩≡ (615d)

```
/*
 * Framebuffer
 */
static errorneg1
fbdefault(int *width, int *height, int *depth)
```

```
{
    u32int buf[3];

    if(vcreq(TagGetres, &buf[0], 0, 2*4) != 2*4 ||
       vcreq(TagGetdepth, &buf[2], 0, 4) != 4)
        return ERROR_NEG1;
    *width = buf[0];
    *height = buf[1];
    *depth = buf[2];
    return OK_0;
}
```

⟨*function* fbblank*(arm)* 283a⟩≡ (615d)

```
int
fbblank(bool blank)
{
    u32int buf[1];

    buf[0] = blank;
    if(vcreq(TagFbblank, buf, sizeof buf, sizeof buf) != sizeof buf)
        return ERROR_NEG1;
    return buf[0] & 1;
}
```

15.3 The console device /dev/cons

15.3.1 Overview

⟨*devcons.c enum Qxxx* 283b⟩≡ (531)

```
enum{
    Qdir,

    Qcons,
    Qconsctl,

    ⟨devcons.c enum Qxxx cases 494a⟩
};
```

⟨*global* consdir 283c⟩≡ (531)

```
static Dirtab consdir[]={
    ".",    {Qdir, 0, QTDIR},   0,      DMDIR|0555,

    "cons",     {Qcons},    0,      0660,
    "consctl",  {Qconsctl}, 0,      0220,

    ⟨consdir fields 494b⟩
};
```

15.3.2 Buffered input

⟨*global* kbdq 283d⟩≡ (531)

```
Queue*  kbdq;           /* unprocessed console input */
```

⟨*function* kbdqinit 284a⟩≡ (531)

```
void
kbdqinit(void)
{
    kbdq = qopen(4*1024, 0, 0, 0);
    if(kbdq == nil)
        panic("kbdinit");
    qnoblock(kbdq, true);
}
```

⟨*global* lineq 284b⟩≡ (531)

```
Queue*  lineq;          /* processed console input */
```

⟨*function* lineqinit 284c⟩≡ (531)

```
void
lineqinit(void)
{

    lineq = qopen(2*1024, 0, nil, nil);
    if(lineq == nil)
        panic("lineqinit");
    qnoblock(lineq, true);
}
```

⟨ConsKbd *other fields* 284d⟩≡ (278b) 290c ▷

```
char    line[1024]; /* current input line */
int x;      /* index into line */
```

⟨consread() *Qcons case* 284e⟩≡ (288d)

```
case Qcons:
    qlock(&kbd);
    if(waserror()) {
        qunlock(&kbd);
        nexterror();
    }
    while(!qcanread(lineq)){
        if(qread(kbdq, &ch, 1) == 0)
            continue;
        send = false;
        if(ch == 0){
            /* flush output on rawoff -> rawon */
            if(kbd.x > 0)
                send = !qcanread(kbdq);

        ⟨consread() else if raw mode 290d⟩
        }else{
            switch(ch){
            case '\b':
                if(kbd.x > 0)
                    kbd.x--;
                break;
            case 0x15:  /* ^U */
                kbd.x = 0;
                break;
            case '\n':
                send = true;
                kbd.line[kbd.x++] = ch;
                break;
            case 0x04:  /* ^D */
                send = true;
```

```
                break;
            default:
                kbd.line[kbd.x++] = ch;
                break;
            }
        }
        if(send || kbd.x == sizeof kbd.line){
            qwrite(lineq, kbd.line, kbd.x);
            kbd.x = 0;
        }
    }
    n = qread(lineq, buf, n);
    qunlock(&kbd);
    poperror();
    return n;
```

15.3.3 Output

⟨conswrite() *Qcons case* 285a⟩≡ (289)

```
case Qcons:
    /*
     * Can't page fault in putstrn, so copy the data locally.
     */
    l = n;
    while(l > 0){
        bp = l;
        if(bp > sizeof buf)
            bp = sizeof buf;
        memmove(buf, a, bp);
        putstrn0(buf, bp, 1);
        a += bp;
        l -= bp;
    }
    break;
```

⟨*function* putstrn0 285b⟩≡ (531)

```
/*
 *   Print a string on the console.
 */
static void
putstrn0(char *str, int n, bool usewrite)
{
    int m;
    char *t;

    if(!arch_islo())
        usewrite = false;

    ⟨putstrn0() kmesg handling 507e⟩
    ⟨putstrn0() if kprint 509c⟩
    else if(screenputs != nil)
        screenputs(str, n);

    ⟨putstrn0() serialoq handling 510e⟩
}
```

15.3.4 print(), iprint(), pprint()

print()

⟨*constant* PRINTSIZE 286a⟩≡ (591b)

```
PRINTSIZE = 256,
```

⟨*function* print 286b⟩≡ (531)

```
bool noprint; // to debug?

int
devcons_print(char *fmt, ...)
{
    int n;
    va_list arg;
    char buf[PRINTSIZE];

    if(noprint)
        return -1;

    va_start(arg, fmt);
    n = vseprint(buf, buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);
    putstrn(buf, n);

    return n;
}
```

⟨*function* putstrn 286c⟩≡ (531)

```
void
putstrn(char *str, int n)
{
    putstrn0(str, n, false);
}
```

iprint()

⟨*global* iprintscreenputs 286d⟩≡ (531)

```
bool iprintscreenputs = true;
```

⟨*function* iprint 286e⟩≡ (531)

```
int
devcons_iprint(char *fmt, ...)
{
    int n, s, locked;
    va_list arg;
    char buf[PRINTSIZE];

    s = arch_splhi();

    va_start(arg, fmt);
    n = vseprint(buf, buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);

    locked = iprintcanlock(&iprintlock);

    kmesgputs(buf, n); // addon
    if(screenputs != nil && iprintscreenputs)
        screenputs(buf, n);
    uartputs(buf, n);
```

```
    if(locked)
        unlock(&iprintlock);

    arch_splx(s);

    return n;
}
```

⟨*global* iprintlock 287a⟩≡ (531)

```
/*
 * Want to interlock iprints to avoid interlaced output on
 * multiprocessor, but don't want to deadlock if one processor
 * dies during print and another has something important to say.
 * Make a good faith effort.
 */
static Lock iprintlock;
```

⟨*function* iprintcanlock 287b⟩≡ (531)

```
static bool
iprintcanlock(Lock *l)
{
    int i;

    for(i=0; i<1000; i++){
        if(canlock(l))
            return true;
        if(l->cpu == CPUS(cpu->cpuno))
            return false;
        arch_microdelay(100);
    }
    return false;
}
```

pprint()

⟨*function* pprint 287c⟩≡ (531)

```
int
devcons_pprint(char *fmt, ...)
{
    int n;
    Chan *c;
    va_list arg;
    char buf[2*PRINTSIZE];

    if(up == nil || up->fgrp == nil)
        return 0;

    c = up->fgrp->fd[2]; // stderr
    if(c==nil || (c->mode!=OWRITE && c->mode!=ORDWR))
        return 0;

    n = snprint(buf, sizeof buf, "%s %lud: ", up->text, up->pid);
    va_start(arg, fmt);
    n = vseprint(buf+n, buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);

    if(waserror())
        return 0;
```

```
    devtab[c->type]->write(c, buf, n, c->offset);
    poperror();

    lock(c);
    c->offset += n;
    unlock(c);

    return n;
}
```

15.3.5 /dev/cons

⟨*method consinit* 288a⟩≡ (531)

```
static void
consinit(void)
{
    ⟨consinit() initializing things 279b⟩
}
```

⟨*method consopen* 288b⟩≡ (531)

```
static Chan*
consopen(Chan *c, int omode)
{
    c->aux = nil;
    c = devopen(c, omode, consdir, nelem(consdir), devgen);

    switch((ulong)c->qid.path){
    ⟨consopen() cases 290f⟩
    }
    return c;
}
```

⟨*method consclose* 288c⟩≡ (531)

```
static void
consclose(Chan *c)
{
    switch((ulong)c->qid.path){
    ⟨consclose() cases 290g⟩
    }
}
```

⟨*method consread* 288d⟩≡ (531)

```
static long
consread(Chan *c, void *buf, long n, vlong off)
{
    ulong l;
    char ch;
    char tmp[256];      /* must be >= 18*NUMSIZE (Qswap) */
    int i, k, send;
    vlong offset = off;

    if(n <= 0)
        return n;

    switch((ulong)c->qid.path){
    case Qdir:
        return devdirread(c, buf, n, consdir, nelem(consdir), devgen);

    ⟨consread() Qcons case 284e⟩
```

```
    ⟨consread() cases 134e⟩


    case Quser:
        return readstr((ulong)offset, buf, n, up->user);

    case Qpid:
        return readnum((ulong)offset, buf, n, up->pid, NUMSIZE);

    case Qppid:
        return readnum((ulong)offset, buf, n, up->parentpid, NUMSIZE);

    case Qpgrpid:
        return readnum((ulong)offset, buf, n, up->pgrp->pgrpid, NUMSIZE);

    case Qnull:
        return 0;

    case Qzero:
        memset(buf, 0, n);
        return n;

    default:
        print("consread %#llux\n", c->qid.path);
        error(Egreg);
    }
    panic("consread: should not reach this point");
    return -1;
}
```

⟨*method conswrite* 289⟩≡ (531)

```
static long
conswrite(Chan *c, void *va, long n, vlong off)
{
    char *a;

    ⟨conswrite locals 290a⟩

    a = va;
    USED(off);

    switch((ulong)c->qid.path){

    ⟨conswrite() Qcons case 285a⟩

    ⟨conswrite() cases 143c⟩

    case Quser:
        return userwrite(a, n);


    // > /dev/null :)
    case Qnull:
        break;

    default:
        print("conswrite: %#llux\n", c->qid.path);
        error(Egreg);
    }
```

```
    return n;
}
```

⟨conswrite *locals* 290a⟩≡ (289)

```
    char buf[256], ch;
    long l, bp;
    int fd;
    Chan *swc;
```

⟨*global* consdevtab 290b⟩≡ (531)

```
Dev consdevtab = {
    .dc       =    'c',
    .name     =    "cons",

    .reset    =    devreset,
    .init     =    consinit,
    .shutdown =    devshutdown,
    .attach   =    consattach,
    .walk     =    conswalk,
    .stat     =    consstat,
    .open     =    consopen,
    .create   =    devcreate,
    .close    =    consclose,
    .read     =    consread,
    .bread    =    devbread,
    .write    =    conswrite,
    .bwrite   =    devbwrite,
    .remove   =    devremove,
    .wstat    =    devwstat,
};
```

15.3.6 /dev/consctl and raw input

⟨ConsKbd *other fields* 290c⟩+≡ (278b) ◁284d 488e▷

```
bool raw;        /* true if we shouldn't process input */
Ref ctl;        /* number of opens to the control file */
```

⟨consread() *else if raw mode* 290d⟩≡ (284e)

```
}else if(kbd.raw){
    kbd.line[kbd.x++] = ch;
    send = !qcanread(kbdq);
```

⟨echo() *return before any echoscreen if raw mode* 290e⟩≡ (279c)

```
if(kbd.raw)
    return;
```

⟨consopen() *cases* 290f⟩≡ (288b) 508i▷

```
case Qconsctl:
    incref(&kbd.ctl);
    break;
```

⟨consclose() *cases* 290g⟩≡ (288c) 509a▷

```
/* last close of control file turns off raw */
case Qconsctl:
    if(c->flag&COPEN){
        if(decref(&kbd.ctl) == 0)
            kbd.raw = false;
    }
    break;
```

⟨conswrite() *cases* 291a⟩+≡ (289) ◁183c

```
  case Qconsctl:
      if(n >= sizeof(buf))
          n = sizeof(buf)-1;
      strncpy(buf, a, n);
      buf[n] = 0;
      for(a = buf; a;){
          if(strncmp(a, "rawon", 5) == 0){
              kbd.raw = true;
              /* clumsy hack - wake up reader */
              ch = 0;
              qwrite(kbdq, &ch, 1);
          } else if(strncmp(a, "rawoff", 6) == 0){
              kbd.raw = false;
          }
          ⟨conswrite() Qconsctl other ifs 488f⟩
          if(a = strchr(a, ' '))
              a++;
      }
      break;
```

15.4 The mouse

15.4.1 Mouse and keyboard

⟨Kbscan *other fields* 291b⟩+≡ (275a) ◁277c 527b▷

```
  int buttons;
```

⟨kbdputsc() *reset mouseshift* 291c⟩≡ (275d)

```
  mouseshifted = false;
```

⟨kbdputsc() *set mouseshift* 291d⟩≡ (275d)

```
  mouseshifted = true;
```

⟨kbdputsc() *mouse keyup cases* 291e⟩≡ (275d)

```
  case Kmouse|1:
  case Kmouse|2:
  case Kmouse|3:
  case Kmouse|4:
  case Kmouse|5:
      kbscan->buttons &= ~(1<<(c-Kmouse-1));
      if(kbdmouse)
          kbdmouse(kbscan->buttons);
      break;
```

⟨kbdputsc() *mouse keydown cases* 291f⟩≡ (275d)

```
  case Kmouse|1:
  case Kmouse|2:
  case Kmouse|3:
  case Kmouse|4:
  case Kmouse|5:
      kbscan->buttons |= 1<<(c-Kmouse-1);
      if(kbdmouse)
          kbdmouse(kbscan->buttons);
      break;
```

15.4.2 PS2 mouse (x86)

15.4.3 USB mouse (ARM)

15.5 Other devices

Chapter 16

Queue

⟨*enum* queuestate 293a⟩≡ (569c)

```
/* queue state bits,  Qmsg, Qcoalesce, and Qkick can be set in qopen */
enum QueueState
{
  /* Queue.state */
  Qstarve   = (1<<0), /* consumer starved */
  Qmsg    = (1<<1), /* message stream */
  Qclosed   = (1<<2), /* queue has been closed/hungup */
  Qflow   = (1<<3), /* producer flow controlled */
  Qcoalesce = (1<<4), /* coalesce packets on read */
  Qkick   = (1<<5), /* always call the kick routine after qwrite */
};
```

⟨*struct* Queue 293b⟩≡ (569c)

```
struct Queue
{
  Lock;

  Block*  bfirst;   /* buffer */
  Block*  blast;

  int len;    /* bytes allocated to queue */
  int dlen;   /* data bytes in queue */
  int limit;    /* max bytes in queue */
  int inilim;   /* initial limit */
  int state;
  int noblock;  /* true if writes return immediately when q full */
  int eof;    /* number of eofs read by user */

  void  (*kick)(void*); /* restart output */
  void  (*bypass)(void*, Block*); /* bypass queue altogether */
  void* arg;    /* argument to kick */

  QLock rlock;    /* mutex for reading processes */
  Rendez  rr;   /* process waiting to read */
  QLock wlock;    /* mutex for writing processes */
  Rendez  wr;   /* process waiting to write */

  char  err[ERRMAX];
};
```

⟨*function* pullupqueue 293c⟩≡ (576b)

```
/*
 *  make sure the first block has at least n bytes
 */
```

```
Block*
pullupqueue(Queue *q, int n)
{
    Block *b;

    if(BLEN(q->bfirst) >= n)
        return q->bfirst;
    q->bfirst = pullupblock(q->bfirst, n);
    for(b = q->bfirst; b != nil && b->next != nil; b = b->next)
        ;
    q->blast = b;
    return q->bfirst;
}
```

⟨*function* qwrite 294a⟩≡ (576b)

```
/*
 *  write to a queue.  only Maxatomic bytes at a time is atomic.
 */
int
qwrite(Queue *q, void *vp, int len)
{
    int n, sofar;
    Block *b;
    byte *p = vp;

//    QDEBUG if(!arch_islo())  print("qwrite hi %#p\n", getcallerpc(&q));

    sofar = 0;
    do {
        n = len-sofar;
        if(n > Maxatomic)
            n = Maxatomic;

        b = allocb(n);
        setmalloctag(b, (up->text[0]<<24)|(up->text[1]<<16)|(up->text[2]<<8)|up->text[3]);
        if(waserror(){
            freeb(b);
            nexterror();
        }
        memmove(b->wp, p+sofar, n);
        poperror();
        b->wp += n;

        qbwrite(q, b);

        sofar += n;
    } while(sofar < len && (q->state & Qmsg) == 0);

    return len;
}
```

⟨*function* qget 294b⟩≡ (576b)

```
/*
 *  get next block from a queue, return null if nothing there
 */
Block*
qget(Queue *q)
{
    int dowakeup;
    Block *b;
```

```
    /* sync with qwrite */
    ilock(q);

    b = q->bfirst;
    if(b == nil){
        q->state |= Qstarve;
        iunlock(q);
        return nil;
    }
    q->bfirst = b->next;
    b->next = 0;
    q->len -= BALLOC(b);
    q->dlen -= BLEN(b);
//    QDEBUG checkb(b, "qget");

    /* if writer flow controlled, restart */
    if((q->state & Qflow) && q->len < q->limit/2){
        q->state &= ~Qflow;
        dowakeup = 1;
    } else
        dowakeup = 0;

    iunlock(q);

    if(dowakeup)
        wakeup(&q->wr);

    return b;
}
```

⟨*function* qdiscard 295⟩≡ (576b)

```
/*
 *  throw away the next 'len' bytes in the queue
 */
int
qdiscard(Queue *q, int len)
{
    Block *b;
    int dowakeup, n, sofar;

    ilock(q);
    for(sofar = 0; sofar < len; sofar += n){
        b = q->bfirst;
        if(b == nil)
            break;
//        QDEBUG checkb(b, "qdiscard");
        n = BLEN(b);
        if(n <= len - sofar){
            q->bfirst = b->next;
            b->next = 0;
            q->len -= BALLOC(b);
            q->dlen -= BLEN(b);
            freeb(b);
        } else {
            n = len - sofar;
            b->rp += n;
            q->dlen -= n;
        }
    }
```

```
    /*
     *  if writer flow controlled, restart
     *
     *  This used to be
     *  q->len < q->limit/2
     *  but it slows down tcp too much for certain write sizes.
     *  I really don't understand it completely.  It may be
     *  due to the queue draining so fast that the transmission
     *  stalls waiting for the app to produce more data.  - presotto
     */
    if((q->state & Qflow) && q->len < q->limit){
        q->state &= ~Qflow;
        dowakeup = 1;
    } else
        dowakeup = 0;

    iunlock(q);

    if(dowakeup)
        wakeup(&q->wr);

    return sofar;
}
```

⟨*function* qconsume 296⟩≡ (576b)

```
/*
 *  Interrupt level copy out of a queue, return # bytes copied.
 */
int
qconsume(Queue *q, void *vp, int len)
{
    Block *b;
    int n, dowakeup;
    byte *p = vp;
    Block *tofree = nil;

    /* sync with qwrite */
    ilock(q);

    for(;;) {
        b = q->bfirst;
        if(b == 0){
            q->state |= Qstarve;
            iunlock(q);
            return -1;
        }
//        QDEBUG checkb(b, "qconsume 1");

        n = BLEN(b);
        if(n > 0)
            break;
        q->bfirst = b->next;
        q->len -= BALLOC(b);

        /* remember to free this */
        b->next = tofree;
        tofree = b;
    };
```

```
    if(n < len)
        len = n;
    memmove(p, b->rp, len);
    consumecnt += n;
    b->rp += len;
    q->dlen -= len;

    /* discard the block if we're done with it */
    if((q->state & Qmsg) || len == n){
        q->bfirst = b->next;
        b->next = 0;
        q->len -= BALLOC(b);
        q->dlen -= BLEN(b);

        /* remember to free this */
        b->next = tofree;
        tofree = b;
    }

    /* if writer flow controlled, restart */
    if((q->state & Qflow) && q->len < q->limit/2){
        q->state &= ~Qflow;
        dowakeup = 1;
    } else
        dowakeup = 0;

    iunlock(q);

    if(dowakeup)
        wakeup(&q->wr);

    if(tofree != nil)
        freeblist(tofree);

    return len;
}
```

⟨*function* qpass 297⟩≡ (576b)

```
int
qpass(Queue *q, Block *b)
{
    int dlen, len, dowakeup;

    /* sync with qread */
    dowakeup = 0;
    ilock(q);
    if(q->len >= q->limit){
        freeblist(b);
        iunlock(q);
        return -1;
    }
    if(q->state & Qclosed){
        len = BALLOC(b);
        freeblist(b);
        iunlock(q);
        return len;
    }

    /* add buffer to queue */
    if(q->bfirst)
```

```
        q->blast->next = b;
    else
        q->bfirst = b;
    len = BALLOC(b);
    dlen = BLEN(b);
//    QDEBUG checkb(b, "qpass");
    while(b->next){
        b = b->next;
//        QDEBUG checkb(b, "qpass");
        len += BALLOC(b);
        dlen += BLEN(b);
    }
    q->blast = b;
    q->len += len;
    q->dlen += dlen;

    if(q->len >= q->limit/2)
        q->state |= Qflow;

    if(q->state & Qstarve){
        q->state &= ~Qstarve;
        dowakeup = 1;
    }
    iunlock(q);

    if(dowakeup)
        wakeup(&q->rr);

    return len;
}
```

⟨*function* qpassnolim 298⟩≡ (576b)

```
int
qpassnolim(Queue *q, Block *b)
{
    int dlen, len, dowakeup;

    /* sync with qread */
    dowakeup = 0;
    ilock(q);

    if(q->state & Qclosed){
        freeblist(b);
        iunlock(q);
        return BALLOC(b);
    }

    /* add buffer to queue */
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    len = BALLOC(b);
    dlen = BLEN(b);
//    QDEBUG checkb(b, "qpass");
    while(b->next){
        b = b->next;
//        QDEBUG checkb(b, "qpass");
        len += BALLOC(b);
        dlen += BLEN(b);
```

```
    }
    q->blast = b;
    q->len += len;
    q->dlen += dlen;

    if(q->len >= q->limit/2)
        q->state |= Qflow;

    if(q->state & Qstarve){
        q->state &= ~Qstarve;
        dowakeup = 1;
    }
    iunlock(q);

    if(dowakeup)
        wakeup(&q->rr);

    return len;
}
```

⟨*function* qproduce 299⟩≡ (576b)

```
int
qproduce(Queue *q, void *vp, int len)
{
    Block *b;
    int dowakeup;
    byte *p = vp;

    /* sync with qread */
    dowakeup = 0;
    ilock(q);

    /* no waiting receivers, room in buffer? */
    if(q->len >= q->limit){
        q->state |= Qflow;
        iunlock(q);
        return -1;
    }

    /* save in buffer */
    b = iallocb(len);
    if(b == 0){
        iunlock(q);
        return 0;
    }
    memmove(b->wp, p, len);
    producecnt += len;
    b->wp += len;
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    q->blast = b;
    /* b->next = 0; done by iallocb() */
    q->len += BALLOC(b);
    q->dlen += BLEN(b);
//    QDEBUG checkb(b, "qproduce");

    if(q->state & Qstarve){
        q->state &= ~Qstarve;
```

```
        dowakeup = 1;
    }

    if(q->len >= q->limit)
        q->state |= Qflow;
    iunlock(q);

    if(dowakeup)
        wakeup(&q->rr);

    return len;
}
```

⟨*function* qcopy 300⟩≡ (576b)

```
/*
 *  copy from offset in the queue
 */
Block*
qcopy(Queue *q, int len, ulong offset)
{
    int sofar;
    int n;
    Block *b, *nb;
    byte *p;

    nb = allocb(len);

    ilock(q);

    /* go to offset */
    b = q->bfirst;
    for(sofar = 0; ; sofar += n){
        if(b == nil){
            iunlock(q);
            return nb;
        }
        n = BLEN(b);
        if(sofar + n > offset){
            p = b->rp + offset - sofar;
            n -= offset - sofar;
            break;
        }
//        QDEBUG checkb(b, "qcopy");
        b = b->next;
    }

    /* copy bytes from there */
    for(sofar = 0; sofar < len;){
        if(n > len - sofar)
            n = len - sofar;
        memmove(nb->wp, p, n);
        qcopycnt += n;
        sofar += n;
        nb->wp += n;
        b = b->next;
        if(b == nil)
            break;
        n = BLEN(b);
        p = b->rp;
    }
```

```
    iunlock(q);

    return nb;
}
```

⟨*function* qopen 301a⟩≡ (576b)

```
/*
 *  called by non-interrupt code
 */
Queue*
qopen(int limit, int msg, void (*kick)(void*), void *arg)
{
    Queue *q;

    q = malloc(sizeof(Queue));
    if(q == nil)
        return nil;

    q->limit = q->inilim = limit;
    q->kick = kick;
    q->arg = arg;
    q->state = msg;

    q->state |= Qstarve;
    q->eof = 0;
    q->noblock = 0;

    return q;
}
```

⟨*function* qbypass 301b⟩≡ (576b)

```
/* open a queue to be bypassed */
Queue*
qbypass(void (*bypass)(void*, Block*), void *arg)
{
    Queue *q;

    q = malloc(sizeof(Queue));
    if(q == nil)
        return nil;

    q->limit = 0;
    q->arg = arg;
    q->bypass = bypass;
    q->state = 0;

    return q;
}
```

⟨*function* notempty 301c⟩≡ (576b)

```
static int
notempty(void *a)
{
    Queue *q = a;

    return (q->state & Qclosed) || q->bfirst != 0;
}
```

⟨*function* qwait 302a⟩≡ (576b)

```
/*
 *  wait for the queue to be non-empty or closed.
 *  called with q ilocked.
 */
static int
qwait(Queue *q)
{
    /* wait for data */
    for(;;){
        if(q->bfirst != nil)
            break;

        if(q->state & Qclosed){
            if(++q->eof > 3)
                return -1;
            if(*q->err && strcmp(q->err, Ehungup) != 0)
                return -1;
            return 0;
        }

        q->state |= Qstarve;    /* flag requesting producer to wake me */
        iunlock(q);
        sleep(&q->rr, notempty, q);
        ilock(q);
    }
    return 1;
}
```

⟨*function* qaddlist 302b⟩≡ (576b)

```
/*
 * add a block list to a queue
 */
void
qaddlist(Queue *q, Block *b)
{
    /* queue the block */
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    q->len += blockalloclen(b);
    q->dlen += blocklen(b);
    while(b->next)
        b = b->next;
    q->blast = b;
}
```

⟨*function* qremove 302c⟩≡ (576b)

```
/*
 *  called with q ilocked
 */
Block*
qremove(Queue *q)
{
    Block *b;

    b = q->bfirst;
    if(b == nil)
        return nil;
```

```
    q->bfirst = b->next;
    b->next = nil;
    q->dlen -= BLEN(b);
    q->len -= BALLOC(b);
//    QDEBUG checkb(b, "qremove");
    return b;
}
```

⟨*function* qputback 303a⟩≡ (576b)

```
/*
 *  put a block back to the front of the queue
 *  called with q ilocked
 */
void
qputback(Queue *q, Block *b)
{
    b->next = q->bfirst;
    if(q->bfirst == nil)
        q->blast = b;
    q->bfirst = b;
    q->len += BALLOC(b);
    q->dlen += BLEN(b);
}
```

⟨*function* qwakeup_iunlock 303b⟩≡ (576b)

```
/*
 *  flow control, get producer going again
 *  called with q ilocked
 */
static void
qwakeup_iunlock(Queue *q)
{
    int dowakeup = 0;

    /* if writer flow controlled, restart */
    if((q->state & Qflow) && q->len < q->limit/2){
        q->state &= ~Qflow;
        dowakeup = 1;
    }

    iunlock(q);

    /* wakeup flow controlled writers */
    if(dowakeup){
        if(q->kick)
            q->kick(q->arg);
        wakeup(&q->wr);
    }
}
```

⟨*function* qbread 303c⟩≡ (576b)

```
/*
 *  get next block from a queue (up to a limit)
 */
Block*
qbread(Queue *q, int len)
{
    Block *b, *nb;
    int n;
```

```
    qlock(&q->rlock);
    if(waserror()){
        qunlock(&q->rlock);
        nexterror();
    }

    ilock(q);
    switch(qwait(q)){
    case 0:
        /* queue closed */
        iunlock(q);
        qunlock(&q->rlock);
        poperror();
        return nil;
    case -1:
        /* multiple reads on a closed queue */
        iunlock(q);
        error(q->err);
    }

    /* if we get here, there's at least one block in the queue */
    b = qremove(q);
    n = BLEN(b);

    /* split block if it's too big and this is not a message queue */
    nb = b;
    if(n > len){
        if((q->state&Qmsg) == 0){
            n -= len;
            b = allocb(n);
            memmove(b->wp, nb->rp+len, n);
            b->wp += n;
            qputback(q, b);
        }
        nb->wp = nb->rp + len;
    }

    /* restart producer */
    qwakeup_iunlock(q);

    poperror();
    qunlock(&q->rlock);
    return nb;
}
```

⟨*function* qread 304⟩≡ (576b)

```
/*
 *  read a queue.  if no data is queued, post a Block
 *  and wait on its Rendez.
 */
long
qread(Queue *q, void *vp, int len)
{
    Block *b, *first, **l;
    int m, n;

    qlock(&q->rlock);
    if(waserror()){
        qunlock(&q->rlock);
        nexterror();
```

```
    }

    ilock(q);
again:
    switch(qwait(q)){
    case 0:
        /* queue closed */
        iunlock(q);
        qunlock(&q->rlock);
        poperror();
        return 0;
    case -1:
        /* multiple reads on a closed queue */
        iunlock(q);
        error(q->err);
    }

    /* if we get here, there's at least one block in the queue */
    if(q->state & Qcoalesce){
        /* when coalescing, 0 length blocks just go away */
        b = q->bfirst;
        if(BLEN(b) <= 0){
            freeb(qremove(q));
            goto again;
        }

        /*  grab the first block plus as many
         *  following blocks as will completely
         *  fit in the read.
         */
        n = 0;
        l = &first;
        m = BLEN(b);
        for(;;) {
            *l = qremove(q);
            l = &b->next;
            n += m;

            b = q->bfirst;
            if(b == nil)
                break;
            m = BLEN(b);
            if(n+m > len)
                break;
        }
    } else {
        first = qremove(q);
        n = BLEN(first);
    }

    /* copy to user space outside of the ilock */
    iunlock(q);
    b = bl2mem(vp, first, len);
    ilock(q);

    /* take care of any left over partial block */
    if(b != nil){
        n -= BLEN(b);
        if(q->state & Qmsg)
            freeb(b);
```

```
        else
            qputback(q, b);
    }

    /* restart producer */
    qwakeup_iunlock(q);

    poperror();
    qunlock(&q->rlock);
    return n;
}
```

⟨*function* qnotfull 306a⟩≡ (576b)

```
static int
qnotfull(void *a)
{
    Queue *q = a;

    return q->len < q->limit || (q->state & Qclosed);
}
```

⟨*function* qbwrite 306b⟩≡ (576b)

```
/*
 *  add a block to a queue obeying flow control
 */
long
qbwrite(Queue *q, Block *b)
{
    int n, dowakeup;
    Proc *p;

    n = BLEN(b);

    if(q->bypass){
        (*q->bypass)(q->arg, b);
        return n;
    }

    dowakeup = 0;
    qlock(&q->wlock);
    if(waserror()){
        if(b != nil)
            freeb(b);
        qunlock(&q->wlock);
        nexterror();
    }

    ilock(q);

    /* give up if the queue is closed */
    if(q->state & Qclosed){
        iunlock(q);
        error(q->err);
    }

    /* if nonblocking, don't queue over the limit */
    if(q->len >= q->limit){
        if(q->noblock){
            iunlock(q);
            freeb(b);
```

```
            noblockcnt += n;
            qunlock(&q->wlock);
            poperror();
            return n;
        }
    }

    /* queue the block */
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    q->blast = b;
    b->next = 0;
    q->len += BALLOC(b);
    q->dlen += n;
//    QDEBUG checkb(b, "qbwrite");
    b = nil;

    /* make sure other end gets awakened */
    if(q->state & Qstarve){
        q->state &= ~Qstarve;
        dowakeup = 1;
    }
    iunlock(q);

    /*  get output going again */
    if(q->kick && (dowakeup || (q->state&Qkick)))
        q->kick(q->arg);

    /* wakeup anyone consuming at the other end */
    if(dowakeup){
        p = wakeup(&q->rr);

        /* if we just wokeup a higher priority process, let it run */
        if(p != nil && p->priority > up->priority)
            sched();
    }

    /*
     *  flow control, wait for queue to get below the limit
     *  before allowing the process to continue and queue
     *  more.  We do this here so that postnote can only
     *  interrupt us after the data has been queued.  This
     *  means that things like 9p flushes and ssl messages
     *  will not be disrupted by software interrupts.
     *
     *  Note - this is moderately dangerous since a process
     *  that keeps getting interrupted and rewriting will
     *  queue infinite crud.
     */
    for(;;){
        if(q->noblock || qnotfull(q))
            break;

        ilock(q);
        q->state |= Qflow;
        iunlock(q);
        sleep(&q->wr, qnotfull, q);
    }
```

```
    USED(b);

    qunlock(&q->wlock);
    poperror();
    return n;
}
```

⟨*function* qiwrite 308⟩≡ (576b)

```
/*
 *  used by print() to write to a queue.  Since we may be arch_splhi or not in
 *  a process, don't qlock.
 *
 *  this routine merges adjacent blocks if block n+1 will fit into
 *  the free space of block n.
 */
int
qiwrite(Queue *q, void *vp, int len)
{
    int n, sofar, dowakeup;
    Block *b;
    byte *p = vp;

    dowakeup = 0;

    sofar = 0;
    do {
        n = len-sofar;
        if(n > Maxatomic)
            n = Maxatomic;

        b = iallocb(n);
        if(b == nil)
            break;
        memmove(b->wp, p+sofar, n);
        b->wp += n;

        ilock(q);

        /* we use an artificially high limit for kernel prints since anything
         * over the limit gets dropped
         */
        if(q->dlen >= 16*1024){
            iunlock(q);
            freeb(b);
            break;
        }

//        QDEBUG checkb(b, "qiwrite");
        if(q->bfirst)
            q->blast->next = b;
        else
            q->bfirst = b;
        q->blast = b;
        q->len += BALLOC(b);
        q->dlen += n;

        if(q->state & Qstarve){
            q->state &= ~Qstarve;
            dowakeup = 1;
        }
```

```
        iunlock(q);

        if(dowakeup){
            if(q->kick)
                q->kick(q->arg);
            wakeup(&q->rr);
        }

        sofar += n;
    } while(sofar < len && (q->state & Qmsg) == 0);

    return sofar;
}
```

⟨*function* qfree 309a⟩≡ (576b)

```
/*
 *  be extremely careful when calling this,
 *  as there is no reference accounting
 */
void
qfree(Queue *q)
{
    qclose(q);
    free(q);
}
```

⟨*function* qclose 309b⟩≡ (576b)

```
/*
 *  Mark a queue as closed.  No further IO is permitted.
 *  All blocks are released.
 */
void
qclose(Queue *q)
{
    Block *bfirst;

    if(q == nil)
        return;

    /* mark it */
    ilock(q);
    q->state |= Qclosed;
    q->state &= ~(Qflow|Qstarve);
    strcpy(q->err, Ehungup);
    bfirst = q->bfirst;
    q->bfirst = 0;
    q->len = 0;
    q->dlen = 0;
    q->noblock = 0;
    iunlock(q);

    /* free queued blocks */
    freeblist(bfirst);

    /* wake up readers/writers */
    wakeup(&q->rr);
    wakeup(&q->wr);
}
```

⟨*function* qhangup 310a⟩≡ (576b)

```
/*
 *  Mark a queue as closed.  Wakeup any readers.  Don't remove queued
 *  blocks.
 */
void
qhangup(Queue *q, char *msg)
{
    /* mark it */
    ilock(q);
    q->state |= Qclosed;
    if(msg == 0 || *msg == 0)
        strcpy(q->err, Ehungup);
    else
        strncpy(q->err, msg, ERRMAX-1);
    iunlock(q);

    /* wake up readers/writers */
    wakeup(&q->rr);
    wakeup(&q->wr);
}
```

⟨*function* qiclosed 310b⟩≡ (576b)

```
/*
 *  return non-zero if the q is hungup
 */
int
qisclosed(Queue *q)
{
    return q->state & Qclosed;
}
```

⟨*function* qreopen 310c⟩≡ (576b)

```
/*
 *  mark a queue as no longer hung up
 */
void
qreopen(Queue *q)
{
    ilock(q);
    q->state &= ~Qclosed;
    q->state |= Qstarve;
    q->eof = 0;
    q->limit = q->inilim;
    iunlock(q);
}
```

⟨*function* qlen 310d⟩≡ (576b)

```
/*
 *  return bytes queued
 */
int
qlen(Queue *q)
{
    return q->dlen;
}
```

⟨*function* qwindow 310e⟩≡ (576b)

```
/*
 * return space remaining before flow control
```

```
 */
int
qwindow(Queue *q)
{
    int l;

    l = q->limit - q->len;
    if(l < 0)
        l = 0;
    return l;
}
```

⟨*function* qcanread 311a⟩≡ (576b)

```
/*
 *  return true if we can read without blocking
 */
int
qcanread(Queue *q)
{
    return q->bfirst!=0;
}
```

⟨*function* qsetlimit 311b⟩≡ (576b)

```
/*
 *  change queue limit
 */
void
qsetlimit(Queue *q, int limit)
{
    q->limit = limit;
}
```

⟨*function* qnoblock 311c⟩≡ (576b)

```
/*
 *  set blocking/nonblocking
 */
void
qnoblock(Queue *q, bool onoff)
{
    q->noblock = onoff;
}
```

⟨*function* qflush 311d⟩≡ (576b)

```
/*
 *  flush the output queue
 */
void
qflush(Queue *q)
{
    Block *bfirst;

    /* mark it */
    ilock(q);
    bfirst = q->bfirst;
    q->bfirst = 0;
    q->len = 0;
    q->dlen = 0;
    iunlock(q);

    /* free queued blocks */
```

```
    freeblist(bfirst);

    /* wake up readers/writers */
    wakeup(&q->wr);
}
```

⟨*function* qfull 312⟩≡ (576b)

```
int
qfull(Queue *q)
{
    return q->state & Qflow;
}
```

Chapter 17

Filesystems

⟨*struct* Dirtab 313a⟩≡ (569c)

```
struct Dirtab
{
  char  name[KNAMELEN];
  Qid qid;
  vlong length;
  long  perm;
};
```

17.1 The root filesystem

⟨*global* rootdevtab 313b⟩≡ (558)

```
Dev rootdevtab = {
    .dc       = '/',
    .name     = "root",

    .reset    = rootreset,
    .init     = devinit,
    .shutdown = devshutdown,
    .attach   = rootattach,
    .walk     = rootwalk,
    .stat     = rootstat,
    .open     = rootopen,
    .create   = devcreate,
    .close    = rootclose,
    .read     = rootread,
    .bread    = devbread,
    .write    = rootwrite,
    .bwrite   = devbwrite,
    .remove   = devremove,
    .wstat    = devwstat,
};
```

⟨*devroot enum Qxxx* 313c⟩≡ (558)

```
enum
{
    Qdir = 0,
    Qboot = 0x1000,

    Nrootfiles = 32,
    Nbootfiles = 32,
};
```

⟨*struct* Dirlist 314a⟩≡ (558)

```
struct Dirlist
{
    uint base; // for unique qids
    Dirtab *dir;
    byte **data;
    int ndir; // number of dir used
    int mdir; // max dir entries
};
```

⟨*globals rootdir, rootdata, rootlist* 314b⟩≡ (558)

```
static Dirtab rootdir[Nrootfiles] = {
  {
    .name = "#/",
    .qid = {Qdir, 0, QTDIR},
    .length = 0,
    .perm = DMDIR|0555,
  },
  {
    .name = "boot",
    .qid = {Qboot, 0, QTDIR},
    .length = 0,
    .perm = DMDIR|0555,
  }
};
static byte *rootdata[Nrootfiles];
static Dirlist rootlist =
{
  .base = 0,
  .dir = rootdir,
  .data = rootdata,
  .ndir = 2,
  .mdir = Nrootfiles
};
```

⟨*function* addlist 314c⟩≡ (558)

```
/*
 *  add a file to the list
 */
static void
addlist(Dirlist *l, char *name, byte *contents, ulong len, int perm)
{
    Dirtab *d;

    if(l->ndir >= l->mdir)
        panic("too many root files");
    l->data[l->ndir] = contents;
    d = &l->dir[l->ndir];
    strcpy(d->name, name);
    d->length = len;
    d->perm = perm;
    d->qid.type = 0;
    d->qid.vers = 0;
    d->qid.path = ++l->ndir + l->base;
    if(perm & DMDIR)
        d->qid.type |= QTDIR;
}
```

⟨*function* addrootdir 314d⟩≡ (558)

```
/*
```

```
 *  add a root directory
 */
static void
addrootdir(char *name)
{
    addlist(&rootlist, name, nil, 0, DMDIR|0555);
}
```

⟨*method rootreset* 315a⟩≡ (558)

```
static void
rootreset(void)
{
    addrootdir("bin");
    addrootdir("dev");
    addrootdir("env");
    addrootdir("fd");
    addrootdir("mnt");
    addrootdir("net");
    addrootdir("proc");
    addrootdir("root");
    addrootdir("srv");
    addrootdir("sys");
}
```

⟨*function* rootgen 315b⟩≡ (558)

```
static int
rootgen(Chan *c, char *name, Dirtab*, int, int s, DirEntry *dp)
{
    int t;
    Dirtab *d;
    Dirlist *l;

    switch((int)c->qid.path){
    case Qdir:
        if(s == DEVDOTDOT){
            devdir(c, (Qid){Qdir, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            return 1;
        }
        return devgen(c, name, rootlist.dir, rootlist.ndir, s, dp);
    case Qboot:
        if(s == DEVDOTDOT){
            devdir(c, (Qid){Qdir, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            return 1;
        }
        return devgen(c, name, bootlist.dir, bootlist.ndir, s, dp);
    default:
        if(s == DEVDOTDOT){
            if((int)c->qid.path < Qboot)
                devdir(c, (Qid){Qdir, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            else
                devdir(c, (Qid){Qboot, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            return 1;
        }
        if(s != 0)
            return -1;
        if((int)c->qid.path < Qboot){
            t = c->qid.path-1;
            l = &rootlist;
        }else{
            t = c->qid.path - Qboot - 1;
```

```
            l = &bootlist;
        }
        if(t >= l->ndir)
            return -1;
        d = &l->dir[t];
        devdir(c, d->qid, d->name, d->length, eve, d->perm, dp);
        return 1;
    }
}
```

⟨*method rootread* 316a⟩≡ (558)

```
static long
rootread(Chan *c, void *buf, long n, vlong off)
{
    ulong t;
    Dirtab *d;
    Dirlist *l;
    byte *data;
    ulong offset = off;

    t = c->qid.path;
    switch(t){
    case Qdir:
    case Qboot:
        return devdirread(c, buf, n, nil, 0, rootgen);
    }

    if(t<Qboot)
        l = &rootlist;
    else{
        t -= Qboot;
        l = &bootlist;
    }

    t--;
    if(t >= l->ndir)
        error(Egreg);

    d = &l->dir[t];
    data = l->data[t];
    if(offset >= d->length)
        return 0;
    if(offset+n > d->length)
        n = d->length - offset;
    memmove(buf, data+offset, n);
    return n;
}
```

17.2 The /boot/ filesystem

⟨*globals bootdir, bootdata, bootlist* 316b⟩≡ (558)

```
static Dirtab bootdir[Nbootfiles] = {
  {
    .name = "boot",
    .qid = {Qboot, 0, QTDIR},
    .length = 0,
    .perm = DMDIR|0555,
  }
```

```
};

static byte *bootdata[Nbootfiles];
static Dirlist bootlist =
{
    .base = Qboot,
    .dir = bootdir,
    .data = bootdata,
    .ndir = 1,
    .mdir = Nbootfiles
};
```

⟨*function* addbootfile 317a⟩≡ (558)

```
/*
 *  add a boot file
 */
void
addbootfile(char *name, byte *contents, ulong len)
{
    addlist(&bootlist, name, contents, len, 0555);
}
```

17.3 Helpers

⟨Qidtype *cases* 317b⟩≡ (31d) 333b ▷

```
QTMOUNT = 0x10,    /* type bit for mounted channel */
```

⟨*function* devdir 317c⟩≡ (573)

```
void
devdir(Chan *c, Qid qid, char *n, vlong length, char *user, long perm, DirEntry *db)
{
    db->name = n;
    if(c->flag&CMSG)
        qid.type |= QTMOUNT;
    db->qid = qid;
    db->type = devtab[c->type]->dc;
    db->dev = c->dev;
    db->mode = perm;
    db->mode |= qid.type << 24;
    db->atime = seconds();
    db->mtime = kerndate;
    db->length = length;
    db->uid = user;
    db->gid = eve;
    db->muid = user;
}
```

⟨*function* devgen 317d⟩≡ (573)

```
/*
 * (here, Devgen is the prototype; devgen is the function in dev.c.)
 *
 * a Devgen is expected to return the directory entry for ".."
 * if you pass it s==DEVDOTDOT (-1).  otherwise...
 *
 * there are two contradictory rules.
 *
 * (i) if c is a directory, a Devgen is expected to list its children
 * as you iterate s.
```

```
 *
 * (ii) whether or not c is a directory, a Devgen is expected to list
 * its siblings as you iterate s.
 *
 * devgen always returns the list of children in the root
 * directory.  thus it follows (i) when c is the root and (ii) otherwise.
 * many other Devgens follow (i) when c is a directory and (ii) otherwise.
 *
 * devwalk assumes (i).  it knows that devgen breaks (i)
 * for children that are themselves directories, and explicitly catches them.
 *
 * devstat assumes (ii).  if the Devgen in question follows (i)
 * for this particular c, devstat will not find the necessary info.
 * with our particular Devgen functions, this happens only for
 * directories, so devstat makes something up, assuming
 * c->name, c->qid, eve, DMDIR|0555.
 *
 * devdirread assumes (i).  the callers have to make sure
 * that the Devgen satisfies (i) for the chan being read.
 */
/*
 * the zeroth element of the table MUST be the directory itself for ..
*/
int
devgen(Chan *c, char *name, Dirtab *tab, int ntab, int i, DirEntry *dp)
{
    if(tab == 0)
        return -1;
    if(i == DEVDOTDOT){
        /* nothing */
    }else if(name){
        for(i=1; i<ntab; i++)
            if(strcmp(tab[i].name, name) == 0)
                break;
        if(i==ntab)
            return -1;
        tab += i;
    }else{
        /* skip over the first element, that for . itself */
        i++;
        if(i >= ntab)
            return -1;
        tab += i;
    }
    devdir(c, tab->qid, tab->name, tab->length, eve, tab->perm, dp);
    return 1;
}
```

⟨*function* devclone 318⟩≡ (573)

```
Chan*
devclone(Chan *c)
{
    Chan *nc;

    if(c->flag & COPEN)
        panic("clone of open file type %C\n", devtab[c->type]->dc);

    nc = newchan();

    nc->type = c->type;
```

```
    nc->dev = c->dev;
    nc->mode = c->mode;
    nc->qid = c->qid;
    nc->offset = c->offset;
    nc->umh = nil;
    nc->aux = c->aux;
    nc->mqid = c->mqid;
    nc->mcp = c->mcp;
    return nc;
}
```

⟨*function* devwalk 319⟩≡ (573)

```
Walkqid*
devwalk(Chan *c, Chan *nc, char **name, int nname, Dirtab *tab, int ntab, Devgen *gen)
{
    int i, j;
    bool alloc;
    Walkqid *wq;
    char *n;
    DirEntry dir;

    if(nname > 0)
        error_if_not_dir(c);

    alloc = false;
    wq = smalloc(sizeof(Walkqid)+(nname-1)*sizeof(Qid));
    if(waserror()){
        if(alloc && wq->clone!=nil)
            cclose(wq->clone);
        free(wq);
        return nil;
    }
    if(nc == nil){
        nc = devclone(c);
        nc->type = 0;   /* device doesn't know about this channel yet */
        alloc = true;
    }
    wq->clone = nc;

    for(j=0; j<nname; j++){
        if(!(nc->qid.type&QTDIR)){
            if(j==0)
                error(Enotdir);
            goto Done;
        }
        n = name[j];
        if(strcmp(n, ".") == 0){
    Accept:
            wq->qid[wq->nqid++] = nc->qid;
            continue;
        }
        if(strcmp(n, "..") == 0){
            if((*gen)(nc, nil, tab, ntab, DEVDOTDOT, &dir) != 1){
                print("devgen walk .. in dev%s %llux broken\n",
                    devtab[nc->type]->name, nc->qid.path);
                error("broken devgen");
            }
            nc->qid = dir.qid;
            goto Accept;
        }
```

```
        /*
         * Ugly problem: If we're using devgen, make sure we're
         * walking the directory itself, represented by the first
         * entry in the table, and not trying to step into a sub-
         * directory of the table, e.g. /net/net. Devgen itself
         * should take care of the problem, but it doesn't have
         * the necessary information (that we're doing a walk).
         */
        if(gen==devgen && nc->qid.path!=tab[0].qid.path)
            goto Notfound;
        for(i=0;; i++) {
            switch((*gen)(nc, n, tab, ntab, i, &dir)){
            case -1:
            Notfound:
                if(j == 0)
                    error(Enonexist);
                kstrcpy(up->errstr, Enonexist, ERRMAX);
                goto Done;
            case 0:
                continue;
            case 1:
                if(strcmp(n, dir.name) == 0){
                    nc->qid = dir.qid;
                    goto Accept;
                }
                continue;
            }
        }
    }
    /*
     * We processed at least one name, so will return some data.
     * If we didn't process all nname entries succesfully, we drop
     * the cloned channel and return just the Qids of the walks.
     */
Done:
    poperror();
    if(wq->nqid < nname){
        if(alloc)
            cclose(wq->clone);
        wq->clone = nil;
    }else if(wq->clone){
        /* attach cloned channel to same device */
        wq->clone->type = c->type;
    }
    return wq;
}
```

⟨*function* devstat 320⟩≡ (573)

```
int
devstat(Chan *c, byte *db, int n, Dirtab *tab, int ntab, Devgen *gen)
{
    int i;
    DirEntry dir;
    char *p, *elem;

    for(i=0;; i++){
        switch((*gen)(c, nil, tab, ntab, i, &dir)){
        case -1:
            if(c->qid.type & QTDIR){
                if(c->path == nil)
```

```
                    elem = "???";
                else if(strcmp(c->path->s, "/") == 0)
                    elem = "/";
                else
                    for(elem=p=c->path->s; *p; p++)
                        if(*p == '/')
                            elem = p+1;
                devdir(c, c->qid, elem, 0, eve, DMDIR|0555, &dir);
                n = convD2M(&dir, db, n);
                if(n == 0)
                    error(Ebadarg);
                return n;
            }

            error(Enonexist);
        case 0:
            break;
        case 1:
            if(c->qid.path == dir.qid.path) {
                if(c->flag&CMSG)
                    dir.mode |= DMMOUNT;
                n = convD2M(&dir, db, n);
                if(n == 0)
                    error(Ebadarg);
                return n;
            }
            break;
        }
    }
    return -1; // unreachable
}
```

⟨Dirmode *cases* 321a⟩≡ (242c) 508f▷

```
DMMOUNT = 0x10000000,  /* mode bit for mounted channel */
```

⟨Chan *other fields* 321b⟩+≡ (31a) ◁269b **??**▷

```
int dri;      /* devdirread index */
```

⟨*function* devdirread 321c⟩≡ (573)

```
long
devdirread(Chan *c, char *d, long n, Dirtab *tab, int ntab, Devgen *gen)
{
    long m, dsz;
    DirEntry dir;

    for(m=0; m<n; c->dri++) {
        switch((*gen)(c, nil, tab, ntab, c->dri, &dir)){
        case -1:
            return m;

        case 0:
            break;

        case 1:
            dsz = convD2M(&dir, (byte*)d, n-m);
            if(dsz <= BIT16SZ){ /* <= not < because this isn't stat; read is stuck */
                if(m == 0)
                    error(Eshort);
                return m;
            }
```

```
            m += dsz;
            d += dsz;
            break;
        }
    }

    return m;
}
```

⟨*function* devpermcheck 322a⟩≡ (573)

```
/*
 * error(Eperm) if open permission not granted for up->user.
 */
void
devpermcheck(char *fileuid, ulong perm, int omode)
{
    ulong t;
    static int access[] = { 0400, 0200, 0600, 0100 };

    if(strcmp(up->user, fileuid) == 0)
        perm <<= 0;
    else
    if(strcmp(up->user, eve) == 0)
        perm <<= 3;
    else
        perm <<= 6;

    t = access[omode&3];
    if((t&perm) != t)
        error(Eperm);
}
```

⟨*function* devopen 322b⟩≡ (573)

```
Chan*
devopen(Chan *c, int omode, Dirtab *tab, int ntab, Devgen *gen)
{
    int i;
    DirEntry dir;

    for(i=0;; i++) {
        switch((*gen)(c, nil, tab, ntab, i, &dir)){
        case -1:
            goto Return;
        case 0:
            break;
        case 1:
            if(c->qid.path == dir.qid.path) {
                devpermcheck(dir.uid, dir.mode, omode);
                goto Return;
            }
            break;
        }
    }
Return:
    c->offset = 0;
    if((c->qid.type&QTDIR) && omode!=OREAD)
        error(Eperm);
    c->mode = openmode(omode);
    c->flag |= COPEN;
    return c;
}
```

⟨Dev *methods* 323a⟩+≡ (32d) ◁251b 489d▷

```
Block* (*bread)(Chan*, long, ulong);
long  (*bwrite)(Chan*, Block*, ulong);
```

⟨*function* devbread 323b⟩≡ (573)

```
Block*
devbread(Chan *c, long n, ulong offset)
{
    Block *bp;

    bp = allocb(n);
    if(bp == 0)
        error(Enomem);
    if(waserror()) {
        freeb(bp);
        nexterror();
    }
    bp->wp += devtab[c->type]->read(c, bp->wp, n, offset);
    poperror();
    return bp;
}
```

⟨*function* devbrwrite 323c⟩≡ (573)

```
long
devbwrite(Chan *c, Block *bp, ulong offset)
{
    long n;

    if(waserror()) {
        freeb(bp);
        nexterror();
    }
    n = devtab[c->type]->write(c, bp->rp, BLEN(bp), offset);
    poperror();
    freeb(bp);

    return n;
}
```

17.4 SimpleFS

17.5 Block

⟨*enum* blockflag 323d⟩≡ (569c)

```
/* flag values */
enum BlockFlags
{
  BINTR = (1<<0),
  BFREE = (1<<1),
  Bipck = (1<<2),   /* ip checksum */
  Budpck  = (1<<3),   /* udp checksum */
  Btcpck  = (1<<4),   /* tcp checksum */
  Bpktck  = (1<<5),   /* packet checksum */
};
```

⟨*struct* Block 324a⟩≡ (569c)

```
struct Block
{
  long  ref;
  Block*  next;
  Block*  list;
  byte*  rp;     /* first unconsumed byte */
  byte*  wp;     /* first empty byte */
  byte*  lim;      /* 1 past the end of the buffer */
  byte*  base;     /* start of the buffer */
  void  (*free)(Block*);
  ushort  flag;
  ushort  checksum;   /* IP checksum of complete packet (minus media header) */
};
```

⟨*function* BLEN 324b⟩≡ (569c)

```
#define BLEN(s) ((s)->wp - (s)->rp)
```

⟨*function* BALLOC 324c⟩≡ (569c)

```
#define BALLOC(s) ((s)->lim - (s)->base)
```

⟨*constant* BLOCKALIGN*(arm)* 324d⟩≡ (633b)

```
// possible alternative fix: git show 2342e706e42d1e6653c51cc1433f44816cf53fe6
#define BLOCKALIGN  32          /* only used in allocb.c */
```

⟨*function* _allocb 324e⟩≡ (574)

```
static Block*
_allocb(int size)
{
    Block *b;
    ulong addr;

    if((b = mallocz(sizeof(Block)+size+Hdrspc, false)) == nil)
        return nil;

    b->next = nil;
    b->list = nil;
    b->free = 0;
    b->flag = 0;
    b->ref = 0;
    arch_xinc(&b->ref); // why not incref?? or at least inccnt?

    /* align start of data portion by rounding up */
    addr = (ulong)b;
    addr = ROUND(addr + sizeof(Block), BLOCKALIGN);
    b->base = (byte*)addr;

    /* align end of data portion by rounding down */
    b->lim = ((byte*)b) + msize(b);
    addr = (ulong)(b->lim);
    addr = addr & ~(BLOCKALIGN-1);
    b->lim = (byte*)addr;

    /* leave sluff at beginning for added headers */
    b->rp = b->lim - ROUND(size, BLOCKALIGN);
    if(b->rp < b->base)
        panic("_allocb");
    b->wp = b->rp;

    return b;
}
```

⟨*function* msize 325a⟩≡ (585c)

```
ulong
msize(void *v)
{
    return poolmsize(mainmem, (ulong*)v-Npadlong)-Npadlong*sizeof(ulong);
}
```

⟨*function* allocb 325b⟩≡ (574)

```
Block*
allocb(int size)
{
    Block *b;

    /*
     * Check in a process and wait until successful.
     * Can still error out of here, though.
     */
    if(up == nil)
        panic("allocb without up: %#p", getcallerpc(&size));
    if((b = _allocb(size)) == nil){
        arch_splhi();
        xsummary();
        mallocsummary();
        arch_delay(500);
        panic("allocb: no memory for %d bytes; caller %#p", size,
            getcallerpc(&size));
    }
    setmalloctag(b, getcallerpc(&size));

    return b;
}
```

⟨*struct* Ialloc 325c⟩≡ (574)

```
struct Ialloc
{
    Lock;
    ulong   bytes;
};
```

⟨*global* ialloc 325d⟩≡ (574)

```
struct Ialloc ialloc;
```

⟨*function* iallocb 325e⟩≡ (574)

```
Block*
iallocb(int size)
{
    Block *b;
    static int m1, m2, mp;

    if(ialloc.bytes > conf.ialloc){
        if((m1++%10000)==0){
            if(mp++ > 1000){
                active.exiting = true;
                arch_exit(0);
            }
            iprint("iallocb: limited %lud/%lud\n",
                ialloc.bytes, conf.ialloc);
        }
        return nil;
    }
```

```
    if((b = _allocb(size)) == nil){
        if((m2++%10000)==0){
            if(mp++ > 1000){
                active.exiting = true;
                arch_exit(0);
            }
            iprint("iallocb: no memory %lud/%lud\n",
                ialloc.bytes, conf.ialloc);
        }
        return nil;
    }
    setmalloctag(b, getcallerpc(&size));
    b->flag = BINTR;

    ilock(&ialloc);
    ialloc.bytes += b->lim - b->base;
    iunlock(&ialloc);

    return b;
}
```

⟨*function* freeb 326⟩≡ (574)

```
void
freeb(Block *b)
{
    void *dead = (void*)Bdead;
    long ref;

    if(b == nil || (ref = arch_xdec(&b->ref)) > 0)
        return;

    if(ref < 0){
        arch_dumpstack();
        panic("freeb: ref %ld; caller pc %#p", ref, getcallerpc(&b));
    }

    /*
     * drivers which perform non cache coherent DMA manage their own buffer
     * pool of uncached buffers and provide their own free routine.
     */
    if(b->free) {
        b->free(b);
        return;
    }
    if(b->flag & BINTR) {
        ilock(&ialloc);
        ialloc.bytes -= b->lim - b->base;
        iunlock(&ialloc);
    }

    /* poison the block in case someone is still holding onto it */
    b->next = dead;
    b->rp = dead;
    b->wp = dead;
    b->lim = dead;
    b->base = dead;

    free(b);
}
```

⟨*function* checkb 327a⟩≡ (574)

```
void
checkb(Block *b, char *msg)
{
    void *dead = (void*)Bdead;

    if(b == dead)
        panic("checkb b %s %#p", msg, b);
    if(b->base == dead || b->lim == dead || b->next == dead
      || b->rp == dead || b->wp == dead){
        print("checkb: base %#p lim %#p next %#p\n",
            b->base, b->lim, b->next);
        print("checkb: rp %#p wp %#p\n", b->rp, b->wp);
        panic("checkb dead: %s", msg);
    }

    if(b->base > b->lim)
        panic("checkb 0 %s %#p %#p", msg, b->base, b->lim);
    if(b->rp < b->base)
        panic("checkb 1 %s %#p %#p", msg, b->base, b->rp);
    if(b->wp < b->base)
        panic("checkb 2 %s %#p %#p", msg, b->base, b->wp);
    if(b->rp > b->lim)
        panic("checkb 3 %s %#p %#p", msg, b->rp, b->lim);
    if(b->wp > b->lim)
        panic("checkb 4 %s %#p %#p", msg, b->wp, b->lim);
}
```

⟨*function* freeblist 327b⟩≡ (576b)

```
/*
 *  free a list of blocks
 */
void
freeblist(Block *b)
{
    Block *next;

    for(; b != 0; b = next){
        next = b->next;
        if(b->ref == 1)
            b->next = nil;
        freeb(b);
    }
}
```

⟨*function* padblock 327c⟩≡ (576b)

```
/*
 *  pad a block to the front (or the back if size is negative)
 */
Block*
padblock(Block *bp, int size)
{
    int n;
    Block *nbp;

//    QDEBUG checkb(bp, "padblock 1");
    if(size >= 0){
        if(bp->rp - bp->base >= size){
            bp->rp -= size;
            return bp;
```

```
        }

        if(bp->next)
            panic("padblock %#p", getcallerpc(&bp));
        n = BLEN(bp);
        padblockcnt++;
        nbp = allocb(size+n);
        nbp->rp += size;
        nbp->wp = nbp->rp;
        memmove(nbp->wp, bp->rp, n);
        nbp->wp += n;
        freeb(bp);
        nbp->rp -= size;
    } else {
        size = -size;

        if(bp->next)
            panic("padblock %#p", getcallerpc(&bp));

        if(bp->lim - bp->wp >= size)
            return bp;

        n = BLEN(bp);
        padblockcnt++;
        nbp = allocb(size+n);
        memmove(nbp->wp, bp->rp, n);
        nbp->wp += n;
        freeb(bp);
    }
//    QDEBUG checkb(nbp, "padblock 1");
    return nbp;
}
```

⟨*function* blocklen 328a⟩≡ (576b)

```
/*
 *  return count of bytes in a string of blocks
 */
int
blocklen(Block *bp)
{
    int len;

    len = 0;
    while(bp) {
        len += BLEN(bp);
        bp = bp->next;
    }
    return len;
}
```

⟨*function* blockalloclen 328b⟩≡ (576b)

```
/*
 * return count of space in blocks
 */
int
blockalloclen(Block *bp)
{
    int len;

    len = 0;
```

```
    while(bp) {
        len += BALLOC(bp);
        bp = bp->next;
    }
    return len;
}
```

⟨*function* concatblock 329a⟩≡ (576b)

```
/*
 *  copy the  string of blocks into
 *  a single block and free the string
 */
Block*
concatblock(Block *bp)
{
    int len;
    Block *nb, *f;

    if(bp->next == 0)
        return bp;

    nb = allocb(blocklen(bp));
    for(f = bp; f; f = f->next) {
        len = BLEN(f);
        memmove(nb->wp, f->rp, len);
        nb->wp += len;
    }
    concatblockcnt += BLEN(nb);
    freeblist(bp);
//    QDEBUG checkb(nb, "concatblock 1");
    return nb;
}
```

⟨*function* pullupblock 329b⟩≡ (576b)

```
/*
 *  make sure the first block has at least n bytes
 */
Block*
pullupblock(Block *bp, int n)
{
    int i;
    Block *nbp;

    /*
     *  this should almost always be true, it's
     *  just to avoid every caller checking.
     */
    if(BLEN(bp) >= n)
        return bp;

    /*
     *  if not enough room in the first block,
     *  add another to the front of the list.
     */
    if(bp->lim - bp->rp < n){
        nbp = allocb(n);
        nbp->next = bp;
        bp = nbp;
    }
```

```
    /*
     *  copy bytes from the trailing blocks into the first
     */
    n -= BLEN(bp);
    while(nbp = bp->next){
        i = BLEN(nbp);
        if(i > n) {
            memmove(bp->wp, nbp->rp, n);
            pullupblockcnt++;
            bp->wp += n;
            nbp->rp += n;
//            QDEBUG checkb(bp, "pullupblock 1");
            return bp;
        } else {
            /* shouldn't happen but why crash if it does */
            if(i < 0){
                print("pullup negative length packet, called from %#p\n",
                    getcallerpc(&bp));
                i = 0;
            }
            memmove(bp->wp, nbp->rp, i);
            pullupblockcnt++;
            bp->wp += i;
            bp->next = nbp->next;
            nbp->next = 0;
            freeb(nbp);
            n -= i;
            if(n == 0){
//                QDEBUG checkb(bp, "pullupblock 2");
                return bp;
            }
        }
    }
    freeb(bp);
    return nil;
}
```

⟨*function* packblock 330⟩≡ (576b)

```
/*
 *  if the allocated space is way out of line with the used
 *  space, reallocate to a smaller block
 */
Block*
packblock(Block *bp)
{
    Block **l, *nbp;
    int n;

    for(l = &bp; *l; l = &(*l)->next){
        nbp = *l;
        n = BLEN(nbp);
        if((n<<2) < BALLOC(nbp)){
            *l = allocb(n);
            memmove((*l)->wp, nbp->rp, n);
            (*l)->wp += n;
            (*l)->next = nbp->next;
            freeb(nbp);
        }
    }
```

```
    return bp;
}
```

⟨*function* trimblock 331a⟩≡ (576b)

```
/*
 *  trim to len bytes starting at offset
 */
Block *
trimblock(Block *bp, int offset, int len)
{
    ulong l;
    Block *nb, *startb;

//    QDEBUG checkb(bp, "trimblock 1");
    if(blocklen(bp) < offset+len) {
        freeblist(bp);
        return nil;
    }

    while((l = BLEN(bp)) < offset) {
        offset -= l;
        nb = bp->next;
        bp->next = nil;
        freeb(bp);
        bp = nb;
    }

    startb = bp;
    bp->rp += offset;

    while((l = BLEN(bp)) < len) {
        len -= l;
        bp = bp->next;
    }

    bp->wp -= (BLEN(bp) - len);

    if(bp->next) {
        freeblist(bp->next);
        bp->next = nil;
    }

    return startb;
}
```

⟨*function* copyblock 331b⟩≡ (576b)

```
/*
 *  copy 'count' bytes into a new block
 */
Block*
copyblock(Block *bp, int count)
{
    int l;
    Block *nbp;

//    QDEBUG checkb(bp, "copyblock 0");
    nbp = allocb(count);
    for(; count > 0 && bp != 0; bp = bp->next){
        l = BLEN(bp);
        if(l > count)
```

```
            l = count;
        memmove(nbp->wp, bp->rp, l);
        nbp->wp += l;
        count -= l;
    }
    if(count > 0){
        memset(nbp->wp, 0, count);
        nbp->wp += count;
    }
    copyblockcnt++;
//    QDEBUG checkb(nbp, "copyblock 1");

    return nbp;
}
```

⟨*function* adjustblock 332a⟩≡ (576b)

```
Block*
adjustblock(Block* bp, int len)
{
    int n;
    Block *nbp;

    if(len < 0){
        freeb(bp);
        return nil;
    }

    if(bp->rp+len > bp->lim){
        nbp = copyblock(bp, len);
        freeblist(bp);
//        QDEBUG checkb(nbp, "adjustblock 1");

        return nbp;
    }

    n = BLEN(bp);
    if(len > n)
        memset(bp->wp, 0, len-n);
    bp->wp = bp->rp+len;
//    QDEBUG checkb(bp, "adjustblock 2");

    return bp;
}
```

⟨*function* pullblock 332b⟩≡ (576b)

```
/*
 *  throw away up to count bytes from a
 *  list of blocks.  Return count of bytes
 *  thrown away.
 */
int
pullblock(Block **bph, int count)
{
    Block *bp;
    int n, bytes;

    bytes = 0;
    if(bph == nil)
        return 0;
```

```
    while(*bph != nil && count != 0) {
        bp = *bph;
        n = BLEN(bp);
        if(count < n)
            n = count;
        bytes += n;
        count -= n;
        bp->rp += n;
//        QDEBUG checkb(bp, "pullblock ");
        if(BLEN(bp) == 0) {
            *bph = bp->next;
            bp->next = nil;
            freeb(bp);
        }
    }
    return bytes;
}
```

⟨*function* bl2mem 333a⟩≡ (576b)

```
/*
 *  copy the contents of a string of blocks into
 *  memory.  emptied blocks are freed.  return
 *  pointer to first unconsumed block.
 */
Block*
bl2mem(byte *p, Block *b, int n)
{
    int i;
    Block *next;

    for(; b != nil; b = next){
        i = BLEN(b);
        if(i > n){
            memmove(p, b->rp, n);
            b->rp += n;
            return b;
        }
        memmove(p, b->rp, i);
        n -= i;
        p += i;
        b->rp += i;
        next = b->next;
        freeb(b);
    }
    return nil;
}
```

17.6 Fileservers

17.7 Fcall

17.8 XTODO

⟨Qidtype *cases* 333b⟩+≡ (31d) ◁317b

```
QTAUTH = 0x08,    /* type bit for authentication file */
QTAPPEND = 0x40,    /* type bit for append only files */
QTEXCL = 0x20,    /* type bit for exclusive use files */
```

Chapter 18

IPC

⟨systab *ipc syscalls* 334a⟩≡ (26c)

```
[NOTIFY]    sysnotify,
[NOTED]     sysnoted,

[PIPE]      syspipe,

[SEGATTACH] syssegattach,
[SEGDETACH] syssegdetach,
[SEGFREE]   syssegfree,
[SEGFLUSH]  syssegflush,
[SEGBRK]    syssegbrk,
```

⟨systab *concurrency syscalls* 334b⟩≡ (26c)

```
[RENDEZVOUS]    sysrendezvous,

[SEMACQUIRE]    syssemacquire,
[SEMRELEASE]    syssemrelease,
[TSEMACQUIRE]   systsemacquire,
```

18.1 Notes (signals)

⟨*enum* notekind 334c⟩≡ (592c)

```
enum NoteKind
{
    NUser,        /* note provided externally */
    NExit,        /* deliver note quietly */
    NDebug,       /* print debug message */
};
```

⟨*enum* note 334d⟩≡ (580e)

```
enum NoteHook {
  NCONT = 0, /* continue after note */
  NDFLT = 1, /* terminate after note */
  NSAVE = 2, /* clear note but hold state */
  NRSTR = 3, /* restore saved state */
};
```

⟨*global* noteidalloc 334e⟩≡ (599c)

```
// also used by sysrfork()
Counter noteidalloc;
```

⟨*struct* Note 335a⟩≡ (592c)

```
// a kind of Unix signal
struct Note
{
    char  msg[ERRMAX];
    // enum<NoteKind>
    int flag;     /* whether system posted it */
};
```

⟨newproc() *set fields of* p 335b⟩+≡ (75b) ◁161f 335c▷

```
p->noteid = incref(&noteidalloc);
if(p->noteid==0)
    panic("pidalloc");
```

⟨newproc() *set fields of* p 335c⟩+≡ (75b) ◁335b 335g▷

```
p->notepending = false;
p->notified = false;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 335d⟩+≡ (78d) ◁151c 335h▷

```
memmove(p->note, up->note, sizeof(p->note));
p->nnote = up->nnote;
p->lastnote = up->lastnote;
p->notify = up->notify;
```

⟨sleep() *if notepending* 335e⟩≡ (156i)

```
if(up->notepending) {
    up->notepending = false;
    arch_splx(s);
    ⟨sleep() forceclosefgrp 226b⟩
    error(Eintr);
}
```

⟨Proc *notes fields* 335f⟩≡ (29c)

```
Note  note[NNOTE];
short nnote;

int (*notify)(void*, char*);
bool_ushort notified; /* sysnoted is due */

ulong noteid;   /* Equivalent of note group */

bool notepending;  /* note issued but not acted on */

Note  lastnote;

void  *ureg;    /* User registers for notes */
```

⟨newproc() *set fields of* p 335g⟩+≡ (75b) ◁335c 387c▷

```
p->ureg = nil;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 335h⟩+≡ (78d) ◁335d 366f▷

```
p->ureg = up->ureg; // will be adjusted for R0??
```

⟨sysexec() *when hold debug lock, reset some fields* 335i⟩≡ (83i) 465c▷

```
up->nnote = 0;
up->notify = nil;
up->notified = false;
```

⟨*constant* NNOTE 335j⟩≡ (592c)

```
NNOTE = 5,
```

⟨trap() *call possibly notify(arm)* 336a⟩≡ (105)

```
if(up->procctl || up->nnote)
    arch__notify(ureg);
```

18.1.1 sysnotify() (arm)

⟨*syscall notify* 336b⟩≡ (603a)

```
// int notify(void (*f)(void*, char*));
long
sysnotify(ulong* arg)
{
    if(arg[0] != 0)
        validaddr(arg[0], sizeof(ulong), false);
    up->notify = (int(*)(void*, char*))(arg[0]);
    return 0;
}
```

⟨syscall() *call notify(arm)* 336c⟩≡ (101d)

```
arch_splhi();
if(scallnr != RFORK && (up->procctl || up->nnote))
    arch__notify(ureg);
```

⟨*struct* NFrame*(arm)* 336d⟩≡ (643c)

```
struct NFrame {
    uintptr ip;
    Ureg*   arg0;
    char*   arg1;
    char    msg[ERRMAX];
    Ureg*   old;
    Ureg    ureg;
};
```

⟨*function* arch__notify*(arm)* 336e⟩≡ (643c)

```
/*
 *  Call user, if necessary, with note.
 *  Pass user the Ureg struct and the note on his stack.
 */
int
arch__notify(Ureg* ureg)
{
    int l;
    Note *n;
    u32int s;
    uintptr sp;
    NFrame *nf;

    if(up->procctl)
        procctl(up);
    if(up->nnote == 0)
        return 0;

    fpunotify(ureg);

    s = arch_spllo();
    qlock(&up->debug);

    up->notepending = 0;
    n = &up->note[0];
    if(strncmp(n->msg, "sys:", 4) == 0){
```

```
        l = strlen(n->msg);
        if(l > ERRMAX-23)   /* " pc=0x0123456789abcdef\0" */
            l = ERRMAX-23;
        snprint(n->msg + l, sizeof n->msg - l, " pc=%#lux", ureg->pc);
    }

    if(n->flag != NUser && (up->notified || up->notify == 0)){
        if(n->flag == NDebug)
            pprint("suicide: %s\n", n->msg);
        qunlock(&up->debug);
        pexit(n->msg, n->flag != NDebug);
    }

    if(up->notified){
        qunlock(&up->debug);
        arch_splhi();
        return 0;
    }

    if(up->notify == nil){
        qunlock(&up->debug);
        pexit(n->msg, n->flag != NDebug);
    }
    if(!okaddr(PTR2UINT(up->notify), 1, 0)){
        pprint("suicide: notify function address %#p\n", up->notify);
        qunlock(&up->debug);
        pexit("Suicide", 0);
    }

    sp = ureg->sp - sizeof(NFrame);
    if(!okaddr(sp, sizeof(NFrame), 1)){
        qunlock(&up->debug);
        pprint("suicide: notify stack address %#p\n", sp);
        pexit("Suicide", 0);
    }

    nf = UINT2PTR(sp);
    memmove(&nf->ureg, ureg, sizeof(Ureg));
    nf->old = up->ureg;
    up->ureg = nf;
    memmove(nf->msg, up->note[0].msg, ERRMAX);
    nf->arg1 = nf->msg;
    nf->arg0 = &nf->ureg;
    ureg->r0 = PTR2UINT(nf->arg0);
    nf->ip = 0;

    ureg->sp = sp;
    ureg->pc = PTR2UINT(up->notify);
    up->notified = 1;
    up->nnote--;
    memmove(&up->lastnote, &up->note[0], sizeof(Note));
    memmove(&up->note[0], &up->note[1], up->nnote*sizeof(Note));

    qunlock(&up->debug);
    arch_splx(s);

    return 1;
}
```

18.1.2 sysnoted() (arm)

⟨*syscall noted* 338a⟩≡ (603a)

```
// int noted(int v);
long
sysnoted(ulong* arg)
{
    if(arg[0]!=NRSTR && !up->notified)
        error(Egreg);
    return 0;
}
```

⟨syscall() *call noted()* 338b⟩≡ (101d)

```
if(scallnr == NOTED)
    arch__noted(ureg, *(ulong*)(sp+BY2WD));
```

⟨*function* arch__noted*(arm)* 338c⟩≡ (643c)

```
/*
 *   Return user to state before notify()
 */
static void
arch__noted(Ureg* cur, uintptr arg0)
{
    NFrame *nf;
    Ureg *nur;

    qlock(&up->debug);
    if(arg0 != NRSTR && !up->notified){
        qunlock(&up->debug);
        pprint("call to noted() when not notified\n");
        pexit("Suicide", 0);
    }
    up->notified = 0;
    fpunoted();

    nf = up->ureg;

    /* sanity clause */
    if(!okaddr(PTR2UINT(nf), sizeof(NFrame), 0)){
        qunlock(&up->debug);
        pprint("bad ureg in noted %#p\n", nf);
        pexit("Suicide", 0);
    }

    /* don't let user change system flags */
    nur = &nf->ureg;
    nur->psr &= PsrMask|PsrDfiq|PsrDirq;
    nur->psr |= (cur->psr & ~(PsrMask|PsrDfiq|PsrDirq));

    memmove(cur, nur, sizeof(Ureg));

    switch((int)arg0){
    case NCONT:
    case NRSTR:
        if(!okaddr(nur->pc, BY2WD, 0) || !okaddr(nur->sp, BY2WD, 0)){
            qunlock(&up->debug);
            pprint("suicide: trap in noted\n");
            pexit("Suicide", 0);
        }
        up->ureg = nf->old;
```

```
        qunlock(&up->debug);
        break;
    case NSAVE:
        if(!okaddr(nur->pc, BY2WD, 0) || !okaddr(nur->sp, BY2WD, 0)){
            qunlock(&up->debug);
            pprint("suicide: trap in noted\n");
            pexit("Suicide", 0);
        }
        qunlock(&up->debug);

        arch_splhi();
        nf->arg1 = nf->msg;
        nf->arg0 = &nf->ureg;
        nf->ip = 0;
        cur->sp = PTR2UINT(nf);
        cur->r0 = PTR2UINT(nf->arg0);
        break;
    default:
        pprint("unknown noted arg %#p\n", arg0);
        up->lastnote.flag = NDebug;
        /*FALLTHROUGH*/
    case NDFLT:
        if(up->lastnote.flag == NDebug){
            qunlock(&up->debug);
            pprint("suicide: %s\n", up->lastnote.msg);
        }
        else
            qunlock(&up->debug);
        pexit(up->lastnote.msg, up->lastnote.flag != NDebug);
    }
}
```

18.1.3 postnote()

⟨*function* postnote 339⟩≡ (599c)

```
/*
 *  if waking a sleeping process, this routine must hold both
 *  p->rlock and r->lock.  However, it can't know them in
 *  the same order as wakeup causing a possible lock ordering
 *  deadlock.  We break the deadlock by giving up the p->rlock
 *  lock if we can't get the r->lock and retrying.
 */
int
proc_postnote(Proc *p, int dolock, char *n, int flag)
{
    int s, ret;
    Rendez *r;
    Proc *d, **l;

    if(dolock)
        qlock(&p->debug);

    if(flag != NUser && (p->notify == 0 || p->notified))
        p->nnote = 0;

    ret = 0;
    if(p->nnote < NNOTE) {
        strcpy(p->note[p->nnote].msg, n);
        p->note[p->nnote++].flag = flag;
```

```
        ret = 1;
    }
    p->notepending = true;
    if(dolock)
        qunlock(&p->debug);

    /* this loop is to avoid lock ordering problems. */
    for(;;){
        s = arch_splhi();
        lock(&p->rlock);
        r = p->r;

        /* waiting for a wakeup? */
        if(r == nil)
            break;  /* no */

        /* try for the second lock */
        if(canlock(r)){
            if(p->state != Wakeme || r->p != p)
                panic("postnote: state %d %d %d", r->p != p, p->r != r, p->state);
            p->r = nil;
            r->p = nil;
            ready(p);
            unlock(r);
            break;
        }

        /* give other process time to get out of critical section and try again */
        unlock(&p->rlock);
        arch_splx(s);
        sched();
    }
    unlock(&p->rlock);
    arch_splx(s);

    if(p->state != Rendezvous)
        return ret;

    /* Try and pull out of a rendezvous */
    lock(p->rgrp);
    if(p->state == Rendezvous) {
        p->rendval = ~0;
        l = &REND(p->rgrp, p->rendtag);
        for(d = *l; d; d = d->rendhash) {
            if(d == p) {
                *l = p->rendhash;
                break;
            }
            l = &d->rendhash;
        }
        ready(p);
    }
    unlock(p->rgrp);
    return ret;
}
```

18.2 Pipes

18.2.1 syspipe()

⟨*syscall pipe* 341a⟩≡ (578)

```
// int pipe(int fd[2]);
long
syspipe(ulong* arg)
{
    int fd[2];
    Chan *c[2];
    Dev *d;
    static char *datastr[] = {"data", "data1"};

    validaddr(arg[0], 2*BY2WD, true);
    arch_validalign(arg[0], sizeof(int));
    d = devtab[devno('|', false)];
    c[0] = namec("#|", Atodir, 0, 0);
    c[1] = nil;
    fd[0] = -1;
    fd[1] = -1;

    if(waserror()){
        cclose(c[0]);
        if(c[1])
            cclose(c[1]);
        nexterror();
    }
    c[1] = cclone(c[0]);
    if(walk(&c[0], datastr+0, 1, 1, nil) < 0)
        error(Egreg);
    if(walk(&c[1], datastr+1, 1, 1, nil) < 0)
        error(Egreg);
    c[0] = d->open(c[0], ORDWR);
    c[1] = d->open(c[1], ORDWR);
    if(newfd2(fd, c) < 0)
        error(Enofd);
    poperror();

    ((long*)arg[0])[0] = fd[0];
    ((long*)arg[0])[1] = fd[1];
    return 0;
}
```

⟨*function* newfd2 341b⟩≡ (578)

```
int
newfd2(int fd[2], Chan *c[2])
{
    Fgrp *f;

    f = up->fgrp;
    lock(f);
    fd[0] = findfreefd(f, 0);
    if(fd[0] < 0){
        unlockfgrp(f);
        return -1;
    }
    fd[1] = findfreefd(f, fd[0]+1);
    if(fd[1] < 0){
        unlockfgrp(f);
```

```
        return -1;
    }
    if(fd[1] > f->maxfd)
        f->maxfd = fd[1];
    f->fd[fd[0]] = c[0];
    f->fd[fd[1]] = c[1];
    unlockfgrp(f);

    return 0;
}
```

⟨sseek() *and pipes* 342a⟩≡ (225)

```
if(devtab[c->type]->dc == '|')
    error(Eisstream);
```

18.2.2 /dev/pipe

⟨*struct* Pipe 342b⟩≡ (566)

```
struct Pipe
{
    int ref;
    ulong   path;
    long    perm;
    Queue   *q[2];
    int qref[2];

    // extra
    QLock;
    Pipe    *next;
};
```

⟨*struct* Pipealloc 342c⟩≡ (566)

```
struct Pipealloc
{
    ulong   path;

    // extra
    Lock;
};
```

⟨*global* pipealloc 342d⟩≡ (566)

```
struct Pipealloc pipealloc;
```

⟨*devpipe.c enum Qxxx* 342e⟩≡ (566)

```
enum
{
    Qdir,
    Qdata0,
    Qdata1,
};
```

⟨*global* pipedir 342f⟩≡ (566)

```
Dirtab pipedir[] =
{
    ".",        {Qdir,0,QTDIR}, 0,      DMDIR|0500,
    "data",     {Qdata0},   0,      0600,
    "data1",    {Qdata1},   0,      0600,
};
```

⟨Conf *other fields* 343a⟩+≡ (27a) ◁192a

```
ulong pipeqsize;  /* size in bytes of pipe queues */
```

⟨*method pipeinit* 343b⟩≡ (566)

```
static void
pipeinit(void)
{
    if(conf.pipeqsize == 0){
        if(conf.ncpu > 1)
            conf.pipeqsize = 256*1024;
        else
            conf.pipeqsize = 32*1024;
    }
}
```

⟨Chan *union other fields* 343c⟩≡ (31a) 467c▷

```
/*Pipe*/void* chanpipe; // for pipes
```

⟨*method pipeattach* 343d⟩≡ (566)

```
/*
 *  create a pipe, no streams are created until an open
 */
static Chan*
pipeattach(char *spec)
{
    Pipe *p;
    Chan *c;

    c = devattach('|', spec);
    p = malloc(sizeof(Pipe));
    if(p == 0)
        exhausted("memory");
    p->ref = 1;

    p->q[0] = qopen(conf.pipeqsize, 0, 0, 0);
    if(p->q[0] == 0){
        free(p);
        exhausted("memory");
    }
    p->q[1] = qopen(conf.pipeqsize, 0, 0, 0);
    if(p->q[1] == 0){
        free(p->q[0]);
        free(p);
        exhausted("memory");
    }

    lock(&pipealloc);
    p->path = ++pipealloc.path;
    unlock(&pipealloc);
    p->perm = pipedir[Qdata0].perm;

    mkqid(&c->qid, NETQID(2*p->path, Qdir), 0, QTDIR);
    c->aux = p;
    c->dev = 0;
    return c;
}
```

⟨*function* pipegen 343e⟩≡ (566)

```
static int
pipegen(Chan *c, char*, Dirtab *tab, int ntab, int i, DirEntry *dp)
```

```
{
    Qid q;
    int len;
    Pipe *p;

    if(i == DEVDOTDOT){
        devdir(c, c->qid, "#|", 0, eve, DMDIR|0555, dp);
        return 1;
    }
    i++;    /* skip . */
    if(tab==0 || i>=ntab)
        return -1;

    tab += i;
    p = c->aux;
    switch((ulong)tab->qid.path){
    case Qdata0:
        len = qlen(p->q[0]);
        break;
    case Qdata1:
        len = qlen(p->q[1]);
        break;
    default:
        len = tab->length;
        break;
    }
    mkqid(&q, NETQID(NETID(c->qid.path), tab->qid.path), 0, QTFILE);
    devdir(c, q, tab->name, len, eve, p->perm, dp);
    return 1;
}
```

⟨*method pipewalk* 344a⟩≡ (566)

```
static Walkqid*
pipewalk(Chan *c, Chan *nc, char **name, int nname)
{
    Walkqid *wq;
    Pipe *p;

    wq = devwalk(c, nc, name, nname, pipedir, NPIPEDIR, pipegen);
    if(wq != nil && wq->clone != nil && wq->clone != c){
        p = c->aux;
        qlock(p);
        p->ref++;
        if(c->flag & COPEN){
            print("channel open in pipewalk\n");
            switch(NETTYPE(c->qid.path)){
            case Qdata0:
                p->qref[0]++;
                break;
            case Qdata1:
                p->qref[1]++;
                break;
            }
        }
        qunlock(p);
    }
    return wq;
}
```

⟨*method pipestat* 344b⟩≡ (566)

```
static int
pipestat(Chan *c, byte *db, int n)
{
    Pipe *p;
    DirEntry dir;

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdir:
        devdir(c, c->qid, ".", 0, eve, DMDIR|0555, &dir);
        break;
    case Qdata0:
        devdir(c, c->qid, "data", qlen(p->q[0]), eve, p->perm, &dir);
        break;
    case Qdata1:
        devdir(c, c->qid, "data1", qlen(p->q[1]), eve, p->perm, &dir);
        break;
    default:
        panic("pipestat");
    }
    n = convD2M(&dir, db, n);
    if(n < BIT16SZ)
        error(Eshortstat);
    return n;
}
```

⟨*method pipewstat* 345a⟩≡ (566)

```
static int
pipewstat(Chan* c, byte* db, int n)
{
    int m;
    DirEntry *dir;
    Pipe *p;

    p = c->aux;
    if(strcmp(up->user, eve) != 0)
        error(Eperm);
    if(NETTYPE(c->qid.path) == Qdir)
        error(Eisdir);

    dir = smalloc(sizeof(DirEntry)+n);
    if(waserror()){
        free(dir);
        nexterror();
    }
    m = convM2D(db, n, &dir[0], (char*)&dir[1]);
    if(m == 0)
        error(Eshortstat);
    if(!emptystr(dir[0].uid))
        error("can't change owner");
    if(dir[0].mode != ~0UL)
        p->perm = dir[0].mode;
    poperror();
    free(dir);
    return m;
}
```

⟨*method pipeopen* 345b⟩≡ (566)

```
/*
```

```
 *  if the stream doesn't exist, create it
 */
static Chan*
pipeopen(Chan *c, int omode)
{
    Pipe *p;

    if(c->qid.type & QTDIR){
        if(omode != OREAD)
            error(Ebadarg);
        c->mode = omode;
        c->flag |= COPEN;
        c->offset = 0;
        return c;
    }

    p = c->aux;
    qlock(p);
    switch(NETTYPE(c->qid.path)){
    case Qdata0:
        p->qref[0]++;
        break;
    case Qdata1:
        p->qref[1]++;
        break;
    }
    qunlock(p);

    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    c->iounit = qiomaxatomic;
    return c;
}
```

⟨*method pipeclose* 346⟩≡ (566)

```
static void
pipeclose(Chan *c)
{
    Pipe *p;

    p = c->aux;
    qlock(p);

    if(c->flag & COPEN){
        /*
         *  closing either side hangs up the stream
         */
        switch(NETTYPE(c->qid.path)){
        case Qdata0:
            p->qref[0]--;
            if(p->qref[0] == 0){
                qhangup(p->q[1], 0);
                qclose(p->q[0]);
            }
            break;
        case Qdata1:
            p->qref[1]--;
            if(p->qref[1] == 0){
                qhangup(p->q[0], 0);
```

```
                qclose(p->q[1]);
            }
            break;
        }
    }


    /*
     *  if both sides are closed, they are reusable
     */
    if(p->qref[0] == 0 && p->qref[1] == 0){
        qreopen(p->q[0]);
        qreopen(p->q[1]);
    }

    /*
     *  free the structure on last close
     */
    p->ref--;
    if(p->ref == 0){
        qunlock(p);
        free(p->q[0]);
        free(p->q[1]);
        free(p);
    } else
        qunlock(p);
}
```

⟨*method piperead* 347a⟩≡ (566)

```
static long
piperead(Chan *c, void *va, long n, vlong)
{
    Pipe *p;

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdir:
        return devdirread(c, va, n, pipedir, NPIPEDIR, pipegen);
    case Qdata0:
        return qread(p->q[0], va, n);
    case Qdata1:
        return qread(p->q[1], va, n);
    default:
        panic("piperead");
    }
    panic("piperead: should not reach this point");
    return -1; // unreachable
}
```

⟨*method pipebread* 347b⟩≡ (566)

```
static Block*
pipebread(Chan *c, long n, ulong offset)
{
    Pipe *p;

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdata0:
```

```
        return qbread(p->q[0], n);
    case Qdata1:
        return qbread(p->q[1], n);
    }

    return devbread(c, n, offset);
}
```

⟨*method pipewrite* 348a⟩≡ (566)

```
/*
 *  a write to a closed pipe causes a note to be sent to
 *  the process.
 */
static long
pipewrite(Chan *c, void *va, long n, vlong)
{
    Pipe *p;

    if(!arch_islo())
        print("pipewrite hi %#p\n", getcallerpc(&c));
    if(waserror()) {
        /* avoid notes when pipe is a mounted queue */
        if((c->flag & CMSG) == 0)
            postnote(up, 1, "sys: write on closed pipe", NUser);
        nexterror();
    }

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdata0:
        n = qwrite(p->q[1], va, n);
        break;

    case Qdata1:
        n = qwrite(p->q[0], va, n);
        break;

    default:
        panic("pipewrite");
    }

    poperror();
    return n;
}
```

⟨*method pipebwrite* 348b⟩≡ (566)

```
static long
pipebwrite(Chan *c, Block *bp, ulong)
{
    long n;
    Pipe *p;

    if(waserror()) {
        /* avoid notes when pipe is a mounted queue */
        if((c->flag & CMSG) == 0)
            postnote(up, 1, "sys: write on closed pipe", NUser);
        nexterror();
    }
```

```
    p = c->aux;
    switch(NETTYPE(c->qid.path)){
    case Qdata0:
        n = qbwrite(p->q[1], bp);
        break;

    case Qdata1:
        n = qbwrite(p->q[0], bp);
        break;

    default:
        n = 0;
        panic("pipebwrite");
    }

    poperror();
    return n;
}
```

⟨*global* pipedevtab 349a⟩≡ (566)

```
Dev pipedevtab = {
    .dc       =    '|',
    .name     =    "pipe",

    .reset    =    devreset,
    .init     =    pipeinit,
    .shutdown =    devshutdown,
    .attach   =    pipeattach,
    .walk     =    pipewalk,
    .stat     =    pipestat,
    .open     =    pipeopen,
    .create   =    devcreate,
    .close    =    pipeclose,
    .read     =    piperead,
    .bread    =    pipebread,
    .write    =    pipewrite,
    .bwrite   =    pipebwrite,
    .remove   =    devremove,
    .wstat    =    pipewstat,
};
```

18.3 Shared segment and semaphore

18.3.1 Segment

⟨Segtype *other flags* 349b⟩+≡ (61a) ◁66c

```
SG_CEXEC  = 0100,   /* Detach at exec */
```

⟨sysexec() *free special segments if close on exec* 349c⟩≡ (85d)

```
for(i = BSEG+1; i < NSEG; i++) {
    s = up->seg[i];
    if(s != nil && (s->type&SG_CEXEC)) { // close on exec
        putseg(s);
        up->seg[i] = nil;
    }
}
```

⟨*syscall segbrk* 350a⟩≡ (603a)

```
// void* segbrk(void *saddr, void *addr);
long
syssegbrk(ulong* arg)
{
    int i;
    ulong addr;
    Segment *s;

    addr = arg[0];
    for(i = 0; i < NSEG; i++) {
        s = up->seg[i];
        if(s == nil || addr < s->base || addr >= s->top)
            continue;
        switch(s->type&SG_TYPE) {
        case SG_TEXT:
        case SG_DATA:
        case SG_STACK:
            error(Ebadarg);
        default:
            return ibrk(arg[1], i);
        }
    }

    error(Ebadarg);
    panic("syssegbrk: should not reach this point");
    return -1; // unreachable
}
```

⟨*syscall segattach* 350b⟩≡ (603a)

```
// void* segattach(int attr, char *class, void *va, ulong len);
long
syssegattach(ulong* arg)
{
    return segattach(up, arg[0], (char*)arg[1], arg[2], arg[3]);
}
```

⟨*hook* _globalsegattach 350c⟩≡ (589)

```
Segment* (*_globalsegattach)(Proc*, char*);
```

⟨*function* segattach 350d⟩≡ (589)

```
ulong
segattach(Proc *p, ulong attr, char *name, ulong va, ulong len)
{
    int sno;
    Segment *s, *os;
    Physseg *ps;

    if(va != 0 && va >= USTKTOP)
        error(Ebadarg);

    validaddr((ulong)name, 1, false);
    vmemchr(name, 0, ~0);

    for(sno = 0; sno < NSEG; sno++)
        if(p->seg[sno] == nil && sno != ESEG)
            break;

    if(sno == NSEG)
        error(Enovmem);
```

```
    /*
     *  first look for a global segment with the
     *  same name
     */
    if(_globalsegattach != nil){
        s = (*_globalsegattach)(p, name);
        if(s != nil){
            p->seg[sno] = s;
            return s->base;
        }
    }

    len = PGROUND(len);
    if(len == 0)
        error(Ebadarg);

    /*
     * Find a hole in the address space.
     * Starting at the lowest possible stack address - len,
     * check for an overlapping segment, and repeat at the
     * base of that segment - len until either a hole is found
     * or the address space is exhausted.  Ensure that we don't
     * map the zero page.
     */
    if(va == 0) {
        for (os = p->seg[SSEG]; os != nil; os = isoverlap(p, va, len)) {
            va = os->base;
            if(len >= va)
                error(Enovmem);
            va -= len;
        }
        va &= ~(BY2PG-1);
    } else {
        va &= ~(BY2PG-1);
        if(va == 0 || va >= USTKTOP)
            error(Ebadarg);
    }

    if(isoverlap(p, va, len) != nil)
        error(Esoverlap);


    for(ps = physseg; ps->name; ps++)
        if(strcmp(name, ps->name) == 0)
            goto found;

    error(Ebadarg);
found:
    if(len > ps->size)
        error(Enovmem);

    attr &= ~SG_TYPE;       /* Turn off what is not allowed */
    attr |= ps->attr;       /* Copy in defaults */

    s = newseg(attr, va, len/BY2PG);
    s->pseg = ps;
    p->seg[sno] = s;

    return va;
```

```
}
```

⟨*function* isoverlap 352a⟩≡ (589)

```
Segment*
isoverlap(Proc *p, ulong va, int len)
{
    int i;
    Segment *ns;
    ulong newtop;

    newtop = va+len;
    for(i = 0; i < NSEG; i++) {
        ns = p->seg[i];
        if(ns == 0)
            continue;
        if((newtop > ns->base && newtop <= ns->top) ||
           (va >= ns->base && va < ns->top))
            return ns;
    }
    return nil;
}
```

⟨*syscall segdetach* 352b⟩≡ (603a)

```
// int segdetach(void *addr);
long
syssegdetach(ulong* arg)
{
    int i;
    ulong addr;
    Segment *s;

    qlock(&up->seglock);
    if(waserror()){
        qunlock(&up->seglock);
        nexterror();
    }

    s = 0;
    addr = arg[0];
    for(i = 0; i < NSEG; i++)
        if(s = up->seg[i]) {
            qlock(&s->lk);
            if((addr >= s->base && addr < s->top) ||
               (s->top == s->base && addr == s->base))
                goto found;
            qunlock(&s->lk);
        }

    error(Ebadarg);

found:
    /*
     * Check we are not detaching the initial stack segment.
     */
    if(s == up->seg[SSEG]){
        qunlock(&s->lk);
        error(Ebadarg);
    }
    up->seg[i] = 0;
    qunlock(&s->lk);
```

```
    putseg(s);
    qunlock(&up->seglock);
    poperror();

    /* Ensure we flush any entries from the lost segment */
    arch_flushmmu();
    return 0;
}
```

⟨*syscall segfree* 353a⟩≡ (603a)

```
// int segfree(void *va, ulong len);
long
syssegfree(ulong* arg)
{
    Segment *s;
    ulong from, to;

    from = arg[0];
    s = seg(up, from, 1);
    if(s == nil)
        error(Ebadarg);
    to = (from + arg[1]) & ~(BY2PG-1);
    from = PGROUND(from);

    if(to > s->top) {
        qunlock(&s->lk);
        error(Ebadarg);
    }

    mfreeseg(s, from, (to - from) / BY2PG);
    qunlock(&s->lk);
    arch_flushmmu();

    return 0;
}
```

⟨`dupseg()` *if original segment was shared, flush it* 353b⟩≡ (68e)

```
if(s->ref > 1)
    procflushseg(s); // ??
```

⟨`mfreeseg()` *if segment was shared, flush it* 353c⟩≡ (71d)

```
/* flush this seg in all other processes */
if(s->ref > 1)
    procflushseg(s);
```

⟨*function* `procflushseg` 353d⟩≡ (599c)

```
/*
 *  wait till all processes have flushed their mmu
 *  state about segment s
 */
void
procflushseg(Segment *s)
{
    int i, ns, nm;
    bool wait;
    Proc *p;

    /*
     *  tell all processes with this
     *  segment to flush their mmu's
```

```
     */
    wait = false;
    for(i=0; i<conf.nproc; i++) {
        p = &procalloc.arena[i];
        if(p->state == Dead)
            continue;
        for(ns = 0; ns < NSEG; ns++)
            if(p->seg[ns] == s){
                p->newtlb = true;
                for(nm = 0; nm < conf.ncpu; nm++){
                    if(CPUS(nm)->proc == p){
                        CPUS(nm)->flushmmu = true;
                        wait = true;
                    }
                }
                break;
            }
    }
    if(!wait)
        return;

    /*
     *  wait for all processors to take a clock interrupt
     *  and flush their mmu's
     */
    for(nm = 0; nm < conf.ncpu; nm++)
        if(CPUS(nm) != cpu)
            while(CPUS(nm)->flushmmu)
                sched();
}
```

⟨hzclock() *if flushmmu* 354a⟩≡ (159e)

```
if(cpu->flushmmu){
    if(up)
        arch_flushmmu();
    cpu->flushmmu = false;
}
```

⟨Cpu *other fields* 354b⟩+≡ (28a) ◁158f 383c▷

```
bool flushmmu;   /* make current proc flush it's mmu state */
```

⟨*syscall segflush* 354c⟩≡ (591a)

```
long
syssegflush(ulong* arg)
{
    Segment *s;
    ulong addr, l;
    Pagetable *pt;
    int chunk, ps, pe, len;

    addr = arg[0];
    len = arg[1];

    while(len > 0) {
        s = seg(up, addr, 1);
        if(s == 0)
            error(Ebadarg);

    ⟨syssegflush() set flushme 424i⟩
    more:
```

```
        l = len;
        if(addr+l > s->top)
            l = s->top - addr;

        ps = addr-s->base;
        pt = s->pagedir[ps/PAGETABMAPMEM];
        ps &= PAGETABMAPMEM-1;
        pe = PAGETABMAPMEM;
        if(pe-ps > l){
            pe = ps + l;
            pe = ROUND(pe, BY2PG);
        }
        if(pe == ps) {
            qunlock(&s->lk);
            error(Ebadarg);
        }

        // when have cachectl
        if(pt)
            ptflush(pt, ps/BY2PG, pe/BY2PG);

        chunk = pe-ps;
        len -= chunk;
        addr += chunk;

        if(len > 0 && addr < s->top)
            goto more;

        qunlock(&s->lk);
    }
    arch_flushmmu();
    return 0;
}
```

18.3.2 Semaphore

⟨*struct* Sema 355a⟩≡ (528)

```
// user level semaphores, used to implement user-level lock,
// see libc/port/lock.c
struct Sema
{
    long  *addr; // value stored in user space!
    bool waiting;

    //list<Sema> of Segment.sema
    Sema  *next;
    Sema  *prev;

    Rendez;
};
```

⟨Segment *other fields* 355b⟩+≡ (61c) ◁126b 367d▷

```
Sema sema;
```

⟨newseg() *sema initialization* 355c⟩≡ (63b)

```
    // no list, just one sema
    s->sema.prev = &s->sema;
    s->sema.next = &s->sema;
```

cmpswap() (arm)

⟨*function* arch_cmpswap*(raspberry pi1)(arm)* 356a⟩≡ (618a)

```
bool
arch_cmpswap(long *addr, long old, long new)
{
    return cas32(addr, old, new);
}
```

⟨*function* cas32*(arm)* 356b⟩≡ (618a)

```
bool
cas32(void* addr, u32int old, u32int new)
{
    bool r;
    int s;

    s = arch_splhi();
    r = (*(u32int*)addr == old);
    if(r)
        *(u32int*)addr = new;
    arch_splx(s);
    ⟨cas32() if address contained old value, call coherence(arm) 405e⟩
    return r;
}
```

semxxx()

⟨*function* semqueue 356c⟩≡ (603b)

```
/* Add semaphore p with addr a to list in seg. */
static void
semqueue(Segment *s, long *a, Sema *p)
{
    memset(p, 0, sizeof *p);
    p->addr = a;
    lock(&s->sema); /* uses s->sema.Rendez.Lock, but no one else is */
    p->next = &s->sema;
    p->prev = s->sema.prev;
    p->next->prev = p;
    p->prev->next = p;
    unlock(&s->sema);
}
```

⟨*function* semdequeue 356d⟩≡ (603b)

```
/* Remove semaphore p from list in seg. */
static void
semdequeue(Segment *s, Sema *p)
{
    lock(&s->sema);
    p->next->prev = p->prev;
    p->prev->next = p->next;
    unlock(&s->sema);
}
```

⟨*function* semwakeup 356e⟩≡ (603b)

```
/* Wake up n waiters with addr a on list in seg. */
static void
semwakeup(Segment *s, long *a, long n)
{
    Sema *p;
```

```
    lock(&s->sema);
    for(p=s->sema.next; p!=&s->sema && n>0; p=p->next){
        if(p->addr == a && p->waiting){
            p->waiting = false;
            arch_coherence();
            wakeup(p);
            n--;
        }
    }
    unlock(&s->sema);
}
```

⟨*function* semrelease 357a⟩≡ (603b)

```
/* Add delta to semaphore and wake up waiters as appropriate. */
static long
semrelease(Segment *s, long *addr, long delta)
{
    long value;

    do
        value = *addr;
    while(!arch_cmpswap(addr, value, value+delta));
    semwakeup(s, addr, delta);
    return value+delta;
}
```

⟨*function* canacquire 357b⟩≡ (603b)

```
/* Try to acquire semaphore using compare-and-swap */
static bool
canacquire(long *addr)
{
    long value;

    while((value=*addr) > 0)
        if(arch_cmpswap(addr, value, value-1))
            return true;
    return false;
}
```

⟨*function* semawoke 357c⟩≡ (603b)

```
/* Should we wake up? */
static bool
semawoke(void *p)
{
    arch_coherence();
    return !((Sema*)p)->waiting;
}
```

⟨*function* semacquire 357d⟩≡ (603b)

```
/* Acquire semaphore (subtract 1). */
static bool
semacquire(Segment *s, long *addr, bool block)
{
    int acquired;
    Sema phore;

    if(canacquire(addr))
        return true;
    if(!block)
        return false;
```

```
    acquired = false;
    semqueue(s, addr, &phore);
    for(;;){
        phore.waiting = true;
        arch_coherence();
        if(canacquire(addr)){
            acquired = true;
            break;
        }
        if(waserror())
            break;
        sleep(&phore, semawoke, &phore);
        poperror();
    }
    semdequeue(s, &phore);
    arch_coherence();    /* not strictly necessary due to lock in semdequeue */
    if(!phore.waiting)
        semwakeup(s, addr, 1);
    if(!acquired)
        nexterror();
    return false;
}
```

⟨*function* tsemacquire 358⟩≡ (603b)

```
/* Acquire semaphore or time-out */
static bool
tsemacquire(Segment *s, long *addr, ulong ms)
{
    int acquired, timedout;
    ulong t, elms;
    Sema phore;

    if(canacquire(addr))
        return true;
    if(ms == 0)
        return false;
    acquired = timedout = false;
    semqueue(s, addr, &phore);
    for(;;){
        phore.waiting = true;
        arch_coherence();
        if(canacquire(addr)){
            acquired = true;
            break;
        }
        if(waserror())
            break;
        t = cpu->ticks;
        tsleep(&phore, semawoke, &phore, ms);
        elms = TK2MS(cpu->ticks - t);
        poperror();
        if(elms >= ms){
            timedout = true;
            break;
        }
        ms -= elms;
    }
    semdequeue(s, &phore);
    arch_coherence();    /* not strictly necessary due to lock in semdequeue */
```

```
    if(!phore.waiting)
        semwakeup(s, addr, 1);
    if(timedout)
        return false;
    if(!acquired)
        nexterror();
    return false;
}
```

⟨*syscall semacquire* 359a⟩≡ (603b)

```
// int semacquire(long *addr, int block);
long
syssemacquire(ulong* arg)
{
    int block;
    long *addr;
    Segment *s;

    validaddr(arg[0], sizeof(long), true);
    arch_validalign(arg[0], sizeof(long));
    addr = (long*)arg[0];
    block = arg[1];

    if((s = seg(up, (ulong)addr, 0)) == nil)
        error(Ebadarg);
    if(*addr < 0)
        error(Ebadarg);
    return semacquire(s, addr, block);
}
```

⟨*syscall tsemacquire* 359b⟩≡ (603b)

```
// int tsemacquire(long *addr, ulong ms);
long
systsemacquire(ulong* arg)
{
    long *addr;
    ulong ms;
    Segment *s;

    validaddr(arg[0], sizeof(long), true);
    arch_validalign(arg[0], sizeof(long));
    addr = (long*)arg[0];
    ms = arg[1];

    if((s = seg(up, (ulong)addr, 0)) == nil)
        error(Ebadarg);
    if(*addr < 0)
        error(Ebadarg);
    return tsemacquire(s, addr, ms);
}
```

⟨*syscall semrelease* 359c⟩≡ (603b)

```
// long semrelease(long *addr, long count);
long
syssemrelease(ulong* arg)
{
    long *addr, delta;
    Segment *s;

    validaddr(arg[0], sizeof(long), true);
```

```
    arch_validalign(arg[0], sizeof(long));
    addr = (long*)arg[0];
    delta = arg[1];

    if((s = seg(up, (ulong)addr, 0)) == nil)
        error(Ebadarg);
    /* delta == 0 is a no-op, not a release */
    if(delta < 0 || *addr < 0)
        error(Ebadarg);
    return semrelease(s, addr, delta);
}
```

18.4 sysrendezvous()

⟨*function* REND 360a⟩≡ (592c)

```
enum
{
    RENDLOG = 5,
    RENDHASH =  1<<RENDLOG, /* Hash to lookup rendezvous tags */
};
#define REND(p,s) ((p)->rendhash[(s)&((1<<RENDLOG)-1)])
```

⟨*struct* Rgrp 360b⟩≡ (592c)

```
struct Rgrp
{
    // hash<??, list<ref<Proc>>>
    Proc  *rendhash[RENDHASH];  /* Rendezvous tag hash */

    // extra
    Ref;        /* the Ref's lock is also the Rgrp's lock */
};
```

⟨Proc *synchronization fields* 360c⟩+≡ (29c) ◁173f

```
Rgrp  *rgrp;    /* Rendez group */

uintptr rendtag;  /* Tag for rendezvous */
uintptr rendval;  /* Value for rendezvous */
//??
Proc  *rendhash;  /* Hash list for tag values */
```

⟨*constructor* newrgrp 360d⟩≡ (597b)

```
Rgrp*
newrgrp(void)
{
    Rgrp *r;

    r = smalloc(sizeof(Rgrp));
    r->ref = 1;
    return r;
}
```

⟨*destructor* closergrp 360e⟩≡ (597b)

```
void
closergrp(Rgrp *r)
{
    if(decref(r) == 0)
        free(r);
}
```

⟨Procstate *cases* 361a⟩+≡ (30c) ◁156h 362d▷

```
Rendezvous,
```

⟨*syscall rendezvous* 361b⟩≡ (603a)

```
// void* rendezvous(void* tag, void* value);
long
sysrendezvous(ulong* arg)
{
    uintptr tag, val;
    Proc *p, **l;

    tag = arg[0];
    l = &REND(up->rgrp, tag);
    up->rendval = ~(uintptr)0;

    lock(up->rgrp);
    for(p = *l; p; p = p->rendhash) {
        if(p->rendtag == tag) {
            *l = p->rendhash;
            val = p->rendval;
            p->rendval = arg[1];

            while(p->cpu != 0) // ????
                ;

            ready(p);
            unlock(up->rgrp);
            return val;
        }
        l = &p->rendhash;
    }

    /* Going to sleep here */
    up->rendtag = tag;
    up->rendval = arg[1];
    up->rendhash = *l;
    *l = up;
    up->state = Rendezvous;
    unlock(up->rgrp);

    sched();

    return up->rendval;
}
```

18.5 Network

Chapter 19

Debugging Support

⟨trap() *when undefined instruction, if breakpoint(arm)* 362a⟩≡ (107b)

```
if(seg(up, ureg->pc, false) != nil &&
   *(u32int*)ureg->pc == 0xD1200070)
    postnote(up, 1, "sys: breakpoint", NDebug);
```

⟨trap() *when prefetch fault, switch fst cases(arm)* 362b⟩≡ (107e)

```
case 0x02:      /* instruction debug event (BKPT) */
    if(user){
        snprint(buf, sizeof buf, "sys: breakpoint");
        postnote(up, 1, buf, NDebug);
    }else{
        iprint("kernel bkpt: pc %#lux inst %#ux\n",
            ureg->pc, *(u32int*)ureg->pc);
        panic("kernel bkpt");
    }
    break;
```

⟨*function* arch_setregisters*(arm)* 362c⟩≡ (641a)

```
/* This routine must save the values of registers the user is not permitted
 * to write from devproc and then restore the saved values before returning.
 */
void
arch_setregisters(Ureg* ureg, char* pureg, char* uva, int n)
{
    USED(ureg, pureg, uva, n);
}
```

19.1 Core dumps and broken processes

⟨Procstate *cases* 362d⟩+≡ (30c) ◁361a 364b▷

```
Broken,
```

⟨*struct* Broken 362e⟩≡ (599c)

```
/*
 * weird thing: keep at most NBROKEN around
 */
#define NBROKEN 4
struct Broken
{
    // array<ref<Proc>>
    Proc    *p[NBROKEN];
    // number of entries used in p
    int n;
```

```
    // extra
    QLock;
};
```

⟨*global* broken 363a⟩≡ (599c)

```
struct Broken broken;
```

⟨pexit() *if not freemem, add process to broken processes list* 363b⟩≡ (92d)

```
// instead of a core dump we just keep the process around
if(!freemem)
    addbroken(up); // will call sched()
```

⟨*function* addbroken 363c⟩≡ (599c)

```
void
addbroken(Proc *p)
{
    qlock(&broken);
    if(broken.n == NBROKEN) {
        ready(broken.p[0]);
        memmove(&broken.p[0], &broken.p[1], sizeof(Proc*)*(NBROKEN-1));
        --broken.n;
    }
    broken.p[broken.n++] = p;
    qunlock(&broken);

    edfstop(up);
    p->state = Broken;
    p->psstate = nil;
    sched();
}
```

⟨*function* unbreak 363d⟩≡ (599c)

```
void
unbreak(Proc *p)
{
    int b;

    qlock(&broken);
    for(b=0; b < broken.n; b++)
        if(broken.p[b] == p) {
            broken.n--;
            memmove(&broken.p[b], &broken.p[b+1],
                    sizeof(Proc*)*(NBROKEN-(b+1)));
            ready(p);
            break;
        }
    qunlock(&broken);
}
```

⟨*function* freebroken 363e⟩≡ (599c)

```
int
freebroken(void)
{
    int i, n;

    qlock(&broken);
    n = broken.n;
    for(i=0; i<n; i++) {
        ready(broken.p[i]);
```

```
        broken.p[i] = nil;
    }
    broken.n = 0;
    qunlock(&broken);
    return n;
}
```

⟨procctlreq() *CMkill case, Broken case* 364a⟩≡ (98b)

```
        case Broken:
            // will resume the broken process to finally die and free its mem
            unbreak(p);
            break;
```

19.2 Stopping a process: /proc/x/ctl

⟨Procstate *cases* 364b⟩+≡ (30c) ◁362d 434d▷

```
Stopped,
```

⟨*enum* procctl 364c⟩≡ (592c)

```
enum Procctl
{
    Proc_nothing = 0,
    Proc_stopme,
    ⟨Procctl cases 97d⟩
};
```

⟨Proc *debugger fields* 364d⟩+≡ (29c) ◁102e 365c▷

```
// enum<procctl>
int procctl;  /* Control for /proc debugging */
```

⟨*function* procctl 364e⟩≡ (599c)

```
/*
 *  called arch_splhi() by notify().  See comment in notify for the
 *  reasoning.
 */
void
procctl(Proc *p)
{
    char *oldstate;
    ulong s;

    switch(p->procctl) {
    ⟨procctl() Proc_traceme case (and fallthrough Proc_stopme) 379d⟩
    case Proc_stopme:
        p->procctl = Proc_nothing;
        oldstate = p->psstate;
        p->psstate = "Stopped";
        s = arch_spllo();
        ⟨procctl() wakeup waiting debugger 366b⟩
        arch_splhi();
        p->state = Stopped;
        sched();
        // here when something (debugger, strace, user) ready(p) back
        p->psstate = oldstate;
        arch_splx(s);
        return;
    ⟨procctl() Proc_exitbig case 598b⟩
    ⟨procctl() Proc_exitme case 98a⟩
    }
}
```

⟨procctlreq() *CMstop case* 365a⟩≡ (482)

```
case CMstop:
    procstopwait(p, Proc_stopme);
    break;
```

⟨procctlreq() *CMstart case* 365b⟩≡ (482)

```
case CMstart:
    if(p->state != Stopped)
        error(Ebadctl);
    ready(p);
    break;
```

⟨Proc *debugger fields* 365c⟩+≡ (29c) ◁364d 365e▷

```
Proc  *pdbg;    /* the debugging process */
```

⟨pexit() *release debuggers* 365d⟩≡ (92d)

```
/* release debuggers */
qlock(&up->debug);
if(up->pdbg) {
    wakeup(&up->pdbg->sleepr);
    up->pdbg = nil;
}
qunlock(&up->debug);
```

⟨Proc *debugger fields* 365e⟩+≡ (29c) ◁365c 366c▷

```
QLock debug;    /* to access debugging elements */ // used for many things
```

⟨*function* procstopwait 365f⟩≡ (567)

```
// assumes p->debug is held
void
procstopwait(Proc *p, int ctl)
{
    int pid;

    if(p->pdbg)
        error(Einuse);
    if(procstopped(p) || p->state == Broken)
        return;

    if(ctl != Proc_nothing)
        p->procctl = ctl;

    p->pdbg = up;
    pid = p->pid;

    qunlock(&p->debug);
    up->psstate = "Stopwait";
    if(waserror()) {
        p->pdbg = nil;
        qlock(&p->debug);
        nexterror();
    }

    sleep(&up->sleepr, procstopped, p);

    poperror();
    qlock(&p->debug);
    if(p->pid != pid) // p was reallocated to a new process
        error(Eprocdied);
}
```

⟨*function* procstopped 366a⟩≡ (567)

```
bool
procstopped(void *a)
{
    Proc *p = a;
    return p->state == Stopped;
}
```

⟨procctl() *wakeup waiting debugger* 366b⟩≡ (364e)

```
/* free a waiting debugger */
qlock(&p->debug);
if(p->pdbg) {
    wakeup(&p->pdbg->sleepr);
    p->pdbg = nil;
}
qunlock(&p->debug);
```

⟨Proc *debugger fields* 366c⟩+≡ (29c) ◁365e 370c▷

```
bool hang;   /* hang at next exec for debug */
```

⟨procctlreq() *CMhang case* 366d⟩≡ (482)

```
case CMhang:
    p->hang = true;
    break;
```

⟨procctlreq() *CMnohang case* 366e⟩≡ (482)

```
 case CMnohang:
     p->hang = false;
     break;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 366f⟩+≡ (78d) ◁335h 387e▷

```
p->hang = up->hang;
```

⟨sysexec() *if hang* 366g⟩≡ (83i)

```
if(up->hang)
    up->procctl = Proc_stopme;
```

⟨procctlreq() *CMwaitstop case* 366h⟩≡ (482)

```
case CMwaitstop:
    procstopwait(p, Proc_nothing);
    break;
```

19.3 Inspecting process memory: /proc/x/mem

⟨procread() *cases* 366i⟩+≡ (468c) ◁254e 370e▷

```
case Qmem:
    if(offset < KZERO)
        return procctlmemio(p, offset, n, va, true);

    if(!iseve())
        error(Eperm);

    // why allowing read access to kernel code or kernel memory?
    // who is using that?

    /* validate kernel addresses */
    if(offset < (kern_addr)end) {
        if(offset+n > (kern_addr)end)
```

```
            n = (kern_addr)end - offset;
        memmove(a, (char*)offset, n);
        return n;
    }
    for(i=0; i<nelem(conf.mem); i++){
        cm = &conf.mem[i];
        /* klimit-1 because klimit might be zero! */
        if(cm->kbase <= offset && offset <= cm->klimit-1){
            if(offset+n >= cm->klimit-1)
                n = cm->klimit - offset;
            memmove(a, (char*)offset, n);
            return n;
        }
    }
    error(Ebadarg);
```

⟨xinit() *nkpages kernel memory in m* 367a⟩≡ (196b)

```
m->kbase = (ulong)KADDR(m->base);
m->klimit = (ulong)KADDR(m->base+nkpages*BY2PG);
```

⟨Confmem *other fields* 367b⟩≡ (27c)

```
kern_addr kbase; // KADDR(Confmem.base)
kern_addr klimit; // KADDR(base+ x*BY2PG) where x <= Confmem.npage
```

⟨procwrite() *cases* 367c⟩≡ (471)

```
case Qmem:
    if(p->state != Stopped)
        error(Ebadctl);

    n = procctlmemio(p, offset, n, va, false);
    break;
```

⟨Segment *other fields* 367d⟩+≡ (61c) ◁355b 381h▷

```
ushort  steal;    /* Page stealer lock */
```

⟨*function* procctlmemio 367e⟩≡ (567)

```
int
procctlmemio(Proc *p, ulong offset, int n, virt_vp va, bool read)
{
    Arch_KMap *k;
    Pagetable *pt;
    Page *pg;
    Segment *s;
    ulong soff, l;
    char *a = va, *b;

    for(;;) {
        s = seg(p, offset, 1);
        if(s == nil)
            error(Ebadarg);

        if(offset+n >= s->top)
            n = s->top-offset;

        if(!read && (s->type&SG_TYPE) == SG_TEXT)
            s = text2data(p, s);

        s->steal++;
        soff = offset-s->base;
        if(waserror()) {
```

```
            s->steal--;
            nexterror();
        }
        if(fixfault(s, offset, read, /*putmmu*/false) == 0)
            break;
        poperror();
        s->steal--;
    }
    poperror();
    pt = s->pagedir[soff/PAGETABMAPMEM];
    if(pt == nil)
        panic("procctlmemio");
    pg = pt->pagetab[(soff&(PAGETABMAPMEM-1))/BY2PG];
    if(pagedout(pg))
        panic("procctlmemio1");

    l = BY2PG - (offset&(BY2PG-1));
    if(n > l)
        n = l;

    k = arch_kmap(pg);
    if(waserror()) {
        s->steal--;
        arch_kunmap(k);
        nexterror();
    }
    b = (char*)VA(k);
    b += offset&(BY2PG-1);
    if(read == 1)
        memmove(a, b, n);   /* This can fault */
    else
        memmove(b, a, n);
    arch_kunmap(k);
    poperror();

    ⟨procctlmemio() set cachectl if flushme segment 424k⟩

    s->steal--;

    if(read == 0)
        p->newtlb = false;

    return n;
}
```

⟨*function* text2data 368⟩≡ (567)

```
Segment*
text2data(Proc *p, Segment *s)
{
    int i;
    Segment *ps;

    ps = newseg(SG_DATA, s->base, s->size);
    ps->image = s->image;
    incref(ps->image);
    ps->fstart = s->fstart;
    ps->flen = s->flen;
    ⟨text2data() initialize other fields 424g⟩

    qlock(&p->seglock);
```

```
    for(i = 0; i < NSEG; i++)
        if(p->seg[i] == s)
            break;
    if(i == NSEG)
        panic("segment gone");

    qunlock(&s->lk);
    putseg(s);
    qlock(&ps->lk);
    p->seg[i] = ps;
    qunlock(&p->seglock);

    return ps;
}
```

19.4 /bin/strace

⟨Procctl *cases* 369a⟩+≡ (364c) ◁97d 379c▷

```
Proc_tracesyscall,
```

⟨procctlreq() *CMstartsyscall case* 369b⟩≡ (482)

```
case CMstartsyscall:
    if(p->state != Stopped)
        error(Ebadctl);
    p->procctl = Proc_tracesyscall;
    ready(p);
    procstopwait(p, Proc_tracesyscall); // will sleep
    break;
```

⟨syscall() Proc_tracesyscall *if, syscall entry(arm)* 369c⟩≡ (101d)

```
if(up->procctl == Proc_tracesyscall){
    /*
     * Redundant validaddr.  Do we care?
     * Tracing syscalls is not exactly a fast path...
     * Beware, validaddr currently does a pexit rather
     * than an error if there's a problem; that might
     * change in the future.
     */
    if(sp < (USTKTOP-BY2PG) || sp > (USTKTOP-sizeof(Sargs)-BY2WD))
        validaddr(sp, sizeof(Sargs)+BY2WD, false);

    syscallfmt(scallnr, ureg->pc, (va_list)(sp+BY2WD));
    up->procctl = Proc_stopme;
    // this will call sched() and wakeup the tracer process
    procctl(up);
    // back here when the tracer process readied us back and
    // should have set procctl back to Proc_tracesyscall
    if(up->syscalltrace)
        free(up->syscalltrace);
    up->syscalltrace = nil;
}
```

⟨syscall() *other locals(arm)* 369d⟩+≡ (101d) ◁102a

```
vlong startns, stopns;
```

⟨syscall() Proc_tracesyscall *if, syscall exit(arm)* 370a⟩≡ (101d)

```
if(up->procctl == Proc_tracesyscall){
    stopns = todget(nil);
    up->procctl = Proc_stopme;
    sysretfmt(scallnr, (va_list)(sp+BY2WD), ret, startns, stopns);
    s = arch_splhi();
    procctl(up); // again, will call sched() and wakeup tracer process
    arch_splx(s);
    if(up->syscalltrace)
        free(up->syscalltrace);
    up->syscalltrace = nil;
}
```

⟨sysrfork() *inherit* Proc_tracesyscall 370b⟩≡

```
    if(up && up->procctl == Proc_tracesyscall)
        p->procctl = Proc_tracesyscall;
```

⟨Proc *debugger fields* 370c⟩+≡ (29c) ◁366c 376b▷

```
char  *syscalltrace;  /* syscall trace */
```

⟨pexit() *at the start, free resources* 370d⟩+≡ (92d) ◁176a

```
if(up->syscalltrace)
    free(up->syscalltrace);
```

⟨procread() *cases* 370e⟩+≡ (468c) ◁366i 382d▷

```
case Qsyscall:
    if(!p->syscalltrace)
        return 0;
    n = readstr(offset, a, n, p->syscalltrace);
    return n;
```

⟨*function* syscallfmt 370f⟩≡ (606a)

```
void
syscallfmt(int syscallno, ulong pc, va_list list)
{
    long l;
    Fmt fmt;
    void *v;
    vlong vl;
    uintptr p;
    int i[2], len;
    char *a, **argv;

    fmtstrinit(&fmt);
    fmtprint(&fmt, "%uld %s ", up->pid, up->text);

    if(syscallno > nsyscall)
        fmtprint(&fmt, " %d ", syscallno);
    else
        fmtprint(&fmt, "%s ", sysctab[syscallno]?
            sysctab[syscallno]: "huh?");

    fmtprint(&fmt, "%ulx ", pc);
    if(up->syscalltrace != nil)
        free(up->syscalltrace);

    switch(syscallno){
    case NOP:
        p = va_arg(list, uintptr);
        fmtprint(&fmt, "%#p", p);
```

```
    break;
case CHDIR:
case EXITS:
case REMOVE:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    break;
case BIND:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    i[0] = va_arg(list, int);
    fmtprint(&fmt, "%#ux",  i[0]);
    break;
case CLOSE:
case NOTED:
    i[0] = va_arg(list, int);
    fmtprint(&fmt, "%d", i[0]);
    break;
case DUP:
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fmtprint(&fmt, "%d %d", i[0], i[1]);
    break;
case ALARM:
    l = va_arg(list, unsigned long);
    fmtprint(&fmt, "%#lud ", l);
    break;
case EXEC:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    argv = va_arg(list, char**);
    //validalign(PTR2UINT(argv), sizeof(char*)); ??
    for(;;){
        validaddr((ulong)argv, sizeof(char**), false);
        a = *(char **)argv;
        if(a == nil)
            break;
        fmtprint(&fmt, " ");
        fmtuserstring(&fmt, a, "");
        argv++;
    }
    break;
case FAUTH:
    i[0] = va_arg(list, int);
    a = va_arg(list, char*);
    fmtprint(&fmt, "%d", i[0]);
    fmtuserstring(&fmt, a, "");
    break;
case SEGBRK:
case RENDEZVOUS:
    v = va_arg(list, void*);
    fmtprint(&fmt, "%#p ", v);
    v = va_arg(list, void*);
    fmtprint(&fmt, "%#p", v);
    break;
case OPEN:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
```

```
        i[0] = va_arg(list, int);
        fmtprint(&fmt, "%#ux", i[0]);
        break;
    case SLEEP:
        l = va_arg(list, long);
        fmtprint(&fmt, "%ld", l);
        break;
    case RFORK:
        i[0] = va_arg(list, int);
        fmtprint(&fmt, "%#ux", i[0]);
        break;
    case PIPE:
    case BRK:
        v = va_arg(list, int*);
        fmtprint(&fmt, "%#p", v);
        break;
    case CREATE:
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, " ");
        i[0] = va_arg(list, int);
        i[1] = va_arg(list, int);
        fmtprint(&fmt, "%#ux %#ux", i[0], i[1]);
        break;
    case FD2PATH:
    case FSTAT:
    case FWSTAT:
        i[0] = va_arg(list, int);
        a = va_arg(list, char*);
        l = va_arg(list, unsigned long);
        fmtprint(&fmt, "%d %#p %lud", i[0], a, l);
        break;
    case NOTIFY:
    case SEGDETACH:
        v = va_arg(list, void*);
        fmtprint(&fmt, "%#p", v);
        break;
    case SEGATTACH:
        i[0] = va_arg(list, int);
        fmtprint(&fmt, "%d ", i[0]);
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, " ");
        /*FALLTHROUGH*/
    case SEGFREE:
    case SEGFLUSH:
        v = va_arg(list, void*);
        l = va_arg(list, unsigned long);
        fmtprint(&fmt, "%#p %lud", v, l);
        break;
    case UNMOUNT:
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, " ");
        a = va_arg(list, char*);
        fmtuserstring(&fmt, a, "");
        break;
    case SEMACQUIRE:
    case SEMRELEASE:
        v = va_arg(list, int*);
        i[0] = va_arg(list, int);
        fmtprint(&fmt, "%#p %d", v, i[0]);
        break;
```

```
case TSEMACQUIRE:
    v = va_arg(list, long*);
    l = va_arg(list, ulong);
    fmtprint(&fmt, "%#p %ld", v, l);
    break;
case SEEK:
    v = va_arg(list, vlong*);
    i[0] = va_arg(list, int);
    vl = va_arg(list, vlong);
    i[1] = va_arg(list, int);
    fmtprint(&fmt, "%#p %d %#llux %d", v, i[0], vl, i[1]);
    break;
case FVERSION:
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fmtprint(&fmt, "%d %d ", i[0], i[1]);
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    l = va_arg(list, unsigned long);
    fmtprint(&fmt, "%lud", l);
    break;
case WSTAT:
case STAT:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    /*FALLTHROUGH*/
case ERRSTR:
case AWAIT:
    a = va_arg(list, char*);
    l = va_arg(list, unsigned long);
    fmtprint(&fmt, "%#p %lud", a, l);
    break;
case MOUNT:
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fmtprint(&fmt, "%d %d ", i[0], i[1]);
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    i[0] = va_arg(list, int);
    fmtprint(&fmt, "%#ux ", i[0]);
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    break;
case PREAD:
    i[0] = va_arg(list, int);
    v = va_arg(list, void*);
    l = va_arg(list, long);
    fmtprint(&fmt, "%d %#p %ld", i[0], v, l);
    if(syscallno == PREAD){
        vl = va_arg(list, vlong);
        fmtprint(&fmt, " %lld", vl);
    }
    break;
case PWRITE:
    i[0] = va_arg(list, int);
    v = va_arg(list, void*);
    l = va_arg(list, long);
    fmtprint(&fmt, "%d ", i[0]);
    len = MIN(l, 64);
    fmtrwdata(&fmt, v, len, " ");
```

```
        fmtprint(&fmt, "%ld", l);
        if(syscallno == PWRITE){
            vl = va_arg(list, vlong);
            fmtprint(&fmt, " %lld", vl);
        }
        break;
    }

    up->syscalltrace = fmtstrflush(&fmt);
}
```

⟨*function* sysretfmt 374⟩≡ (606a)

```
void
sysretfmt(int syscallno, va_list list, long ret, uvlong start, uvlong stop)
{
    long l;
    void* v;
    Fmt fmt;
    vlong vl;
    int i, len;
    char *a, *errstr;

    fmtstrinit(&fmt);

    if(up->syscalltrace)
        free(up->syscalltrace);

    errstr = "\"\"";
    switch(syscallno){
    default:
    case ALARM:
    case PWRITE:
        if(ret == -1)
            errstr = up->syserrstr;
        fmtprint(&fmt, " = %ld", ret);
        break;
    case EXEC:
    case SEGBRK:
    case SEGATTACH:
    case RENDEZVOUS:
        if((void *)ret == (void*)-1)
            errstr = up->syserrstr;
        fmtprint(&fmt, " = %#p", (void *)ret);
        break;
    case AWAIT:
        a = va_arg(list, char*);
        l = va_arg(list, unsigned long);
        if(ret > 0){
            fmtuserstring(&fmt, a, " ");
            fmtprint(&fmt, "%lud = %ld", l, ret);
        }
        else{
            fmtprint(&fmt, "%#p/\"\" %lud = %ld", a, l, ret);
            errstr = up->syserrstr;
        }
        break;
    case ERRSTR:
        a = va_arg(list, char*);
        l = va_arg(list, unsigned long);
        if(ret > 0){
```

```
            fmtuserstring(&fmt, a, " ");
            fmtprint(&fmt, "%lud = %ld", l, ret);
        }
        else{
            fmtprint(&fmt, "\"\" %lud = %ld", l, ret);
            errstr = up->syserrstr;
        }
        break;
    case FD2PATH:
        i = va_arg(list, int);
        USED(i);
        a = va_arg(list, char*);
        l = va_arg(list, unsigned long);
        if(ret > 0){
            fmtuserstring(&fmt, a, " ");
            fmtprint(&fmt, "%lud = %ld", l, ret);
        }
        else{
            fmtprint(&fmt, "\"\" %lud = %ld", l, ret);
            errstr = up->syserrstr;
        }
        break;
    case PREAD:
        i = va_arg(list, int);
        USED(i);
        v = va_arg(list, void*);
        l = va_arg(list, long);
        if(ret > 0){
            len = MIN(ret, 64);
            fmtrwdata(&fmt, v, len, "");
        }
        else{
            fmtprint(&fmt, "/\"\"");
            errstr = up->syserrstr;
        }
        fmtprint(&fmt, " %ld", l);
        if(syscallno == PREAD){
            vl = va_arg(list, vlong);
            fmtprint(&fmt, " %lld", vl);
        }
        fmtprint(&fmt, " = %ld", ret);
        break;
    }
    fmtprint(&fmt, " %s %#llud %#llud\n", errstr, start, stop);
    up->syscalltrace = fmtstrflush(&fmt);
}
```

⟨*function* fmtrwdata 375⟩≡ (606a)

```
// WE ARE OVERRUNNING SOMEHOW
static void
fmtrwdata(Fmt* f, char* a, int n, char* suffix)
{
    int i;
    char *t;

    if(a == nil){
        fmtprint(f, "0x0%s", suffix);
        return;
    }
    validaddr((ulong)a, n, false);
```

```
    t = smalloc(n+1);
    for(i = 0; i < n; i++)
        if(a[i] > 0x20 && a[i] < 0x7f)  /* printable ascii? */
            t[i] = a[i];
        else
            t[i] = '.';

    fmtprint(f, " %#p/\"%s\"%s", a, t, suffix);
    free(t);
}
```

⟨*function* fmtuserstring 376a⟩≡ (606a)

```
static void
fmtuserstring(Fmt* f, char* a, char* suffix)
{
    int n;
    char *t;

    if(a == nil){
        fmtprint(f, "0/\"\"%s", suffix);
        return;
    }
    validaddr((ulong)a, 1, false);
    n = ((char*)vmemchr(a, 0, 0x7fffffff) - a) + 1;
    t = smalloc(n+1);
    memmove(t, a, n);
    t[n] = 0;
    fmtprint(f, "%#p/\"%s\"%s", a, t, suffix);
    free(t);
}
```

19.5 Scheduler events: /proc/trace and /bin/trace

⟨Proc *debugger fields* 376b⟩+≡ (29c) ◁370c 479b▷

```
bool trace;    /* process being traced? */
```

⟨procctlreq() *CMtrace case* 376c⟩≡ (482)

```
case CMtrace:
    switch(cb->nf){
    case 1:
        p->trace ^= true;
        break;
    case 2:
        p->trace = (atoi(cb->f[1]) != 0);
        break;
    default:
        error("args");
    }
    break;
```

⟨*hook proctrace* 376d⟩≡ (595a)

```
void (*proctrace)(Proc*, /*enum<tevent>*/int, vlong) = 0; // was in devproc.c
```

⟨*enum* Tevent 376e⟩≡ (? 0—1)

```
enum Tevent {
    SReady = 0,  /* runnable but not running  */
    SRun,  /* running best effort */
    SDead,  /* proc dies */
```

```
    SSleep,  /* blocked */
    SUser,  /* user event */

    /* real-time extensions */
    ⟨enum Tevent real-time scheduling events 446a⟩

    Nevent, // must be last
};
```

⟨*struct* Traceevent 377a⟩≡ (? 0—1)

```
struct Traceevent {
    ulong pid;
    // enum<tevent>
    ulong etype; /* Event type */
    vlong time; /* time stamp  */ // dimension?
};
```

⟨ready() *other locals* 377b⟩≡ (155c)

```
void (*pt)(Proc*, int, vlong);
```

⟨ready() *hook proctrace* 377c⟩≡ (155c)

```
pt = proctrace;
if(pt)
    pt(p, SReady, 0);
```

⟨runproc() *other locals* 377d⟩≡ (150) 512c ▷

```
void (*pt)(Proc*, int, vlong);
```

⟨runproc() *hook proctrace* 377e⟩≡ (150)

```
pt = proctrace;
if(pt)
    pt(p, SRun, 0);
```

⟨pexit() *other locals* 377f⟩+≡ (92d) ◁ 95f 381c ▷

```
void (*pt)(Proc*, int, vlong);
```

⟨pexit() *hook proctrace* 377g⟩≡ (92d)

```
pt = proctrace;
if(pt)
    pt(up, SDead, 0);
```

⟨sleep() *other locals* 377h⟩≡ (156i)

```
void (*pt)(Proc*, int, vlong);
```

⟨sleep() *hook proctrace* 377i⟩≡ (156i)

```
pt = proctrace;
if(pt)
    pt(up, SSleep, 0);
```

⟨procctlreq() *CMevent case* 377j⟩≡ (482)

```
case CMevent:
    pt = proctrace;
    if(up->trace && pt)
        pt(up, SUser, 0);
    break;
```

⟨*global trace txxx* 377k⟩≡ (567)

```
// array<Traceevent>
static Traceevent *tevents;
static int topens;
static int tproduced, tconsumed;
static Lock tlock;
```

⟨*function* eventsavailable 378a⟩≡ (567)

```
static int
eventsavailable(void *)
{
    return tproduced > tconsumed;
}
```

⟨*function* _proctrace 378b⟩≡ (567)

```
static void
_proctrace(Proc* p, Tevent etype, vlong ts)
{
    Traceevent *te;

    if (p->trace == false || topens == 0 ||
        tproduced - tconsumed >= Nevents)
        return;

    te = &tevents[tproduced&Emask];
    te->pid = p->pid;
    te->etype = etype;
    if (ts == 0)
        te->time = todget(nil);
    else
        te->time = ts;
    tproduced++;
}
```

⟨*constant* Nevents 378c⟩≡ (567)

```
Nevents = 0x4000,
```

⟨*constant* Emask 378d⟩≡ (567)

```
Emask = Nevents - 1,
```

⟨procopen() *Qtrace if* 378e⟩≡ (473b)

```
if(QID(c->qid) == Qtrace){
    if (omode != OREAD)
        error(Eperm);
    lock(&tlock);
    if (waserror()){
        unlock(&tlock);
        nexterror();
    }
    if (topens > 0)
        error("already open");
    topens++;
    if (tevents == nil){
        tevents = (Traceevent*)malloc(Nevents * sizeof(Traceevent));
        if(tevents == nil)
            error(Enomem);
        tproduced = tconsumed = 0;
    }
    proctrace = _proctrace;
    unlock(&tlock);
    poperror();

    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}
```

⟨procclose() *Qtrace if* 379a⟩≡ (479a)

```
if(QID(c->qid) == Qtrace){
    lock(&tlock);
    if(topens > 0)
        topens--;
    if(topens == 0)
        proctrace = nil;
    unlock(&tlock);
}
```

⟨procread() *Qtrace if* 379b⟩≡ (468c)

```
if(QID(c->qid) == Qtrace){
    if(!eventsavailable(nil))
        return 0;

    rptr = (byte*)va;
    navail = tproduced - tconsumed;
    if(navail > n / sizeof(Traceevent))
        navail = n / sizeof(Traceevent);
    while(navail > 0) {
        ne = ((tconsumed & Emask) + navail > Nevents)?
                Nevents - (tconsumed & Emask): navail;
        memmove(rptr, &tevents[tconsumed & Emask],
                ne * sizeof(Traceevent));

        tconsumed += ne;
        rptr += ne * sizeof(Traceevent);
        navail -= ne;
    }
    return rptr - (byte*)va;
}
```

19.6 /bin/db

19.7 Other

⟨Procctl *cases* 379c⟩+≡ (364c) ◁369a 598a▷

```
Proc_traceme,
```

⟨procctl() Proc_traceme *case (and fallthrough* Proc_stopme*)* 379d⟩≡ (364e)

```
case Proc_traceme:
    if(p->nnote == 0)
        return;
    /* No break */
```

⟨procctlreq() *CMstartstop case* 379e⟩≡ (482)

```
case CMstartstop:
    if(p->state != Stopped)
        error(Ebadctl);
    p->procctl = Proc_traceme;
    ready(p);
    procstopwait(p, Proc_traceme);
    break;
```

Chapter 20

Profiling Support

20.1 /dev/cputime and /proc/#/status

⟨*enum* proctimer 380a⟩≡ (592c)
```
enum Proctime
{
    TUser = 0,    /* Proc.time */
    TSys,
    TReal,

    // to accumulate also the time of the children of the process
    TCUser,
    TCSys,
};
```

⟨Proc *stats and profiling fields* 380b⟩≡ (29c) 383d ▷
```
// hash<enum<proctime>, ulong>
ulong time[5];  /* User, Sys, Real; child U, S */
```

⟨sysrfork() *setting time field* 380c⟩≡ (78d)
```
memset(p->time, 0, sizeof(p->time));
p->time[TReal] = CPUS(0)->ticks;
```

⟨kproc() *setting time field* 380d⟩≡ (96d)
```
memset(p->time, 0, sizeof(p->time));
p->time[TReal] = CPUS(0)->ticks;
```

⟨accountime() *update time of current process* 380e⟩≡ (513a)
```
p->time[p->insyscall ? TSys : TUser]++;
```

⟨consread() *cases* 380f⟩+≡ (288d) ◁ 183b 497 ▷
```
case Qcputime:
    k = offset;
    if(k >= 5*NUMSIZE)
        return 0;
    if(k+n > 5*NUMSIZE)
        n = 5*NUMSIZE - k;
    /* easiest to format in a separate buffer and copy out */

    for(i=0; i<5 && NUMSIZE*i<k+n; i++){
        l = up->time[i];
        if(i == TReal)
            l = CPUS(0)->ticks - l;
        l = TK2MS(l);
        readnum(0, tmp+NUMSIZE*i, NUMSIZE, l, NUMSIZE);
```

```
    }
    memmove(buf, tmp+k, n);
    return n;
```

⟨procread() *Qstatus case, time part* 381a⟩≡ (468c)

```
for(i = 0; i < 5; i++) {
    l = p->time[i];
    if(i == TReal)
        l = CPUS(0)->ticks - l;
    l = TK2MS(l);
    readnum(0, statbuf+j+NUMSIZE*i, NUMSIZE, l, NUMSIZE);
}
```

20.2 /bin/time

⟨Waitmsg *time field* 381b⟩≡ (90c)

```
ulong time[3];  /* of loved one and descendants */
```

⟨pexit() *other locals* 381c⟩+≡ (92d) ◁377f

```
long utime, stime;
```

⟨pexit() *set wait msg time field* 381d⟩≡ (94b)

```
utime = up->time[TUser] + up->time[TCUser];
stime = up->time[TSys] + up->time[TCSys];

wq->w.time[TUser] = tk2ms(utime);
wq->w.time[TSys] = tk2ms(stime);
wq->w.time[TReal] = tk2ms(CPUS(0)->ticks - up->time[TReal]);
```

⟨pexit() *update TC time of parent* 381e⟩≡ (94c)

```
p->time[TCUser] += utime;
p->time[TCSys] += stime;
```

⟨sysawait() *snprint time field arguments* 381f⟩≡ (90e)

```
w.time[TUser], w.time[TSys], w.time[TReal],
```

⟨procread() *Qwait case, snprint time field arguments* 381g⟩≡ (463c)

```
wq->w.time[TUser], wq->w.time[TSys], wq->w.time[TReal],
```

20.3 /bin/tprof

⟨Segment *other fields* 381h⟩+≡ (61c) ◁367d 424c▷

```
kern_wp  profile;  /* Tick profile area */ // for TSEG only
```

⟨putseg() *free profile* 381i⟩≡ (64a)

```
if(s->profile != nil)
    free(s->profile);
```

⟨*constant* LRESPROF 381j⟩≡ (582)

```
LRESPROF  = 3,
```

⟨proc_init() *other init* 381k⟩≡ (467d)

```
addclock0link((void (*)(void))profclock, 113);  /* Relative prime to HZ */
```

⟨*clock callback profclock* 382a⟩≡ (567)

```
static void
profclock(Ureg *ur, Timer *)
{
    if(up == nil || up->state != Running)
        return;

    /* user profiling clock */
    if(arch_userureg(ur)){
        ⟨profclock tos adjustments when user 384e⟩
        segclock(ur->pc);
    }
}
```

⟨*clock callback segclock* 382b⟩≡ (589)

```
// called via profclock
void
segclock(ulong pc)
{
    Segment *s;

    s = up->seg[TSEG];
    if(s == nil || s->profile == nil)
        return;

    s->profile[0] += TK2MS(1);
    if(pc >= s->base && pc < s->top) {
        pc -= s->base;
        s->profile[pc>>LRESPROF] += TK2MS(1);
    }
}
```

⟨procctlreq() *CMprofile case* 382c⟩≡ (482)

```
case CMprofile:
    s = p->seg[TSEG];
    if(s == nil || (s->type&SG_TYPE) != SG_TEXT)
        error(Ebadctl);
    if(s->profile != nil)
        free(s->profile);
    npc = (s->top-s->base)>>LRESPROF;
    s->profile = malloc(npc*sizeof(*s->profile));
    if(s->profile == nil)
        error(Enomem);
    break;
```

⟨procread() *cases* 382d⟩+≡ (468c) ◁370e 463c▷

```
case Qprofile:
    s = p->seg[TSEG];
    if(s == nil || s->profile == nil)
        error("profile is off");
    i = (s->top-s->base)>>LRESPROF;
    i *= sizeof(*s->profile);
    if(offset >= i)
        return 0;
    if(offset+n > i)
        n = i - offset;
    memmove(a, ((char*)s->profile)+offset, n);
    return n;
```

⟨procgen() *Qprofile case* 383a⟩≡ (476d)

```
case Qprofile:
    q = p->seg[TSEG];
    if(q && q->profile) {
        len = (q->top-q->base)>>LRESPROF;
        len *= sizeof(*q->profile);
    }
    break;
```

20.4 /bin/prof

⟨*struct* Tos 383b⟩≡ (? 0—1)

```
struct Tos {
    ⟨Tos prof field ??⟩
    uvlong cyclefreq; /* cycle clock frequency if there is one, 0 otherwise */

    vlong kcycles; /* cycles spent in kernel */
    vlong pcycles; /* cycles spent in process (kernel + user) */
    ulong clock; // in ms

    ulong pid;  /* might as well put the pid here */
    ⟨Tos other fields ??⟩
    /* top of stack is here */
};
```

⟨Cpu *other fields* 383c⟩+≡ (28a) ◁354b

```
// cyclefreq == cpuhz if havetsc, 0 otherwise
uvlong  cyclefreq;    /* Frequency of user readable cycle counter */
```

⟨Proc *stats and profiling fields* 383d⟩+≡ (29c) ◁380b

```
uvlong  kentry;   /* Kernel entry time stamp (for profiling) */
/*
 * pcycles: cycles spent in this process (updated on procsave/restore)
 * when this is the current proc and we're in the kernel
 * (procrestores outnumber procsaves by one)
 * the number of cycles spent in the proc is pcycles + cycles()
 * when this is not the current process or we're in user mode
 * (procrestores and procsaves balance), it is pcycles.
 */
vlong pcycles;
```

⟨*macro* arch_cycles*(arm)* 383e⟩≡ (619d)

```
#define arch_cycles(ip) *(ip) = arch_lcycles()
```

⟨*type CpSPM secondary registers(arm)* 383f⟩≡ (607c)

```
/*
 * CpSPM Secondary (CRm) registers and opcode2 fields (armv6)
 */
#define CpSPMperf   12          /* various counters */
```

⟨*type CpSPM opcode2(arm)* 383g⟩≡ (607c)

```
#define CpSPMctl    0           /* performance monitor control */
#define CpSPMcyc    1           /* cycle counter register */
```

⟨_startpg() *enable cycle counter(raspberry pi1)(arm)* 383h⟩≡ (188d)

```
/*
 * enable cycle counter
 */
MOVW    $1, R1
MCR CpSC, 0, R1, C(CpSPM), C(CpSPMperf), CpSPMctl
```

⟨*function* arch_lcycles*(raspberry pi1)(arm)* 384a⟩≡ (640a)

```
TEXT arch_lcycles(SB), 1, $-4
    MRC CpSC, 0, R0, C(CpSPM), C(CpSPMperf), CpSPMcyc
    RET
```

⟨sysexec() *locals* 384b⟩+≡ (83i) ◁228e 385a▷

```
Tos *tos;
```

⟨sysexec() *tos settings* 384c⟩≡ (87f)

```
tos = (Tos*)(TSTKTOP - sizeof(Tos));

tos->cyclefreq = cpu->cyclefreq;
arch_cycles((uvlong*)&tos->pcycles);
tos->pcycles = -tos->pcycles; // see comment above on Proc->pcycle
tos->kcycles = tos->pcycles;
tos->clock = 0;
// what about other fields? like pid? will be set in kexit! but could be
// done here? what about sysrfork? call kexit?
```

⟨arch__kexit() *tos adjustments* 384d⟩≡ (106a)

```
uvlong t;
Tos *tos;

/* precise time accounting, kernel exit */
tos = (Tos*)(USTKTOP-sizeof(Tos));
arch_cycles(&t);
tos->kcycles += t - up->kentry;
tos->pcycles = up->pcycles;
tos->pid = up->pid;
```

⟨profclock *tos adjustments when user* 384e⟩≡ (382a)

```
Tos *tos;
tos = (Tos*)(USTKTOP-sizeof(Tos));
tos->clock += TK2MS(1);
```

⟨syscall() *adjust kentry* 384f⟩≡ (101d)

```
arch_cycles(&up->kentry);
```

⟨trap() *adjust kentry when interrupt user* 384g⟩≡ (105)

```
arch_cycles(&up->kentry);
```

⟨arch_procsave() *cycles adjustments* 384h⟩≡ (154b)

```
uvlong t;

arch_cycles(&t);
p->pcycles += t;
```

⟨arch_procrestore *cycles adjustments* 384i⟩≡ (154c)

```
arch_cycles(&t);
p->pcycles -= t;
```

⟨sysexec() *nbytes tos adjustments* 384j⟩≡ (86g)

```
nbytes += sizeof(Tos); /* hole for profiling clock at top of stack (and more) */
```

Chapter 21

Userspace System Programs

21.1 /boot/boot

21.2 /root/init

21.3 The shell

21.3.1 #!

⟨sysexec() *locals* 385a⟩+≡ (83i) ◁384b

```
char line[sizeof(Exec)];
char *progarg[sizeof(Exec)/2+1];
char progelem[64];

bool indir = false;
```

⟨sysexec() *process sharpbang* 385b⟩≡ (83i)

```
/*
 * Process #! /bin/sh args ...
 */
memmove(line, &exec, sizeof(Exec));
if(indir || line[0]!='#' || line[1]!='!')
    error(Ebadexec);
n = shargs(line, n, progarg);
if(n == 0)
    error(Ebadexec);
indir = true;
/*
 * First arg becomes complete file name
 */
progarg[n++] = file;
progarg[n] = 0;
validaddr(arg[1], BY2WD, true);
arg[1] += BY2WD;
file = progarg[0];
if(strlen(elem) >= sizeof progelem)
    error(Ebadexec);
strcpy(progelem, elem);
progarg[0] = progelem;
poperror();
cclose(tc);
```

⟨*function* shargs 386a⟩≡ (603a)

```
int
shargs(char *s, int n, char **ap)
{
    int i;

    s += 2;
    n -= 2;     /* skip #! */
    for(i=0; s[i]!='\n'; i++)
        if(i == n-1)
            return 0;
    s[i] = 0;
    *ap = 0;
    i = 0;
    for(;;) {
        while(*s==' ' || *s=='\t')
            s++;
        if(*s == 0)
            break;
        i++;
        *ap++ = s;
        *ap = 0;
        while(*s && *s!=' ' && *s!='\t')
            s++;
        if(*s == 0)
            break;
        else
            *s++ = 0;
    }
    return i;
}
```

⟨sysexec() *if indir arg adjustments* 386b⟩≡ (86g)

```
if(indir){
    argp = progarg;
    while(*argp){
        a = *argp++;
        nbytes += strlen(a) + 1;
        nargs++;
    }
}
```

⟨sysexec() *if indir argp adjustments* 386c⟩≡ (87f)

```
if(indir)
    argp = progarg;
```

⟨sysexec() *if indir argp adjustments again* 386d⟩≡ (87f)

```
if(indir && *argp==nil) {
    indir = false;
    argp = (char**)arg[1];
}
```

Chapter 22

Advanced Topics

22.1 Floating point

22.1.1 FPSaveStatus

⟨*enum* fpsavestatus 387a⟩≡ (592c)

```
/*
 * FPsave.status
 */
enum FPSaveStatus
{
    /* this is a state */
    FPinit=   0,

    FPactive= 1,
    FPinactive= 2,

    /* the following is a bit that can be or'd into the state */
    FPillegal=  0x100,
};
```

⟨Proc *other fields* 387b⟩+≡ (29c) ◁217c 387g▷

```
// enum<FPSaveStatus>
int fpstate;
```

⟨newproc() *set fields of* p 387c⟩+≡ (75b) ◁335g 432c▷

```
⟨newproc() fp init 387d⟩
```

⟨newproc() *fp init* 387d⟩≡ (387c)

```
p->fpstate = FPinit;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 387e⟩+≡ (78d) ◁366f 387h▷

```
⟨sysrfork() propagate fpstate 387f⟩
```

⟨sysrfork() *propagate fpstate* 387f⟩≡ (387e)

```
/* don't penalize the child, it hasn't done FP in a note handler. */
p->fpstate = up->fpstate & ~FPillegal;
```

22.1.2 Arch_FPsave (ARM)

⟨Proc *other fields* 387g⟩+≡ (29c) ◁387b 462a▷

```
Arch_FPsave  fpsave;   /* address of this is known by db */
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 387h⟩+≡ (78d) ◁387e 464e▷

```
⟨sysrfork() propagate fpsave 388a⟩
```

⟨sysrfork() *propagate fpsave* 388a⟩≡ (387h)

```
p->fpsave = up->fpsave;
```

⟨*struct* Arch_FPsave*(arm)* 388b⟩≡ (640d)

```
/*
 * emulated or vfp3 floating point
 */
struct Arch_FPsave
{
    /*
     * vfp3 with ieee fp regs; uvlong is sufficient for hardware but
     * each must be able to hold an Internal from fpi.h for sw emulation.
     */
    ulong   regs[Maxfpregs][3];
    ⟨Proc Arch FPsave other fields 388c⟩
};
```

⟨Proc Arch FPsave *other fields* 388c⟩≡ (388b)

```
int fpstate; // vs Proc.fpstate??

ulong   status;
ulong   control;

uintptr pc;     /* of failed fp instr. */
```

⟨*enum* _anon_*(*(arch/arm/dat_processes.h)(arm)*)* 388d⟩≡ (640d)

```
enum {
    Maxfpregs   = 32,   /* could be 16 or 32, see Mach.fpnregs */
    Nfpctlregs  = 16,
};
```

22.1.3 Floating point hooks

⟨arch_forkchild() *floating point(arm)* 388e⟩≡ (80a)

```
fpusysrforkchild(p, cureg, up);
```

⟨*function* fpusysrforkchild*(arm)* 388f⟩≡ (641b)

```
/*
 * Called later in sysrfork() via the machine-dependent
 * sysrforkchild() routine.
 * Copy the parent FPU state to the child.
 */
void
fpusysrforkchild(Proc *p, Ureg *, Proc *up)
{
    /* don't penalize the child, it hasn't done FP in a note handler. */
    p->fpstate = up->fpstate & ~FPillegal;
}
```

⟨arch_procsetup() *floating point(arm)* 388g⟩≡ (85e)

```
fpusysprocsetup(p);
```

⟨*function* fpusysprocsetup*(arm)* 388h⟩≡ (641b)

```
/*
 * Disable the FPU.
 * Called from sysexec() via sysprocsetup() to
 * set the FPU for the new process.
 */
void
```

```
fpusysprocsetup(Proc *p)
{
    p->fpstate = FPinit;
    fpoff();
}
```

⟨arch_procsave() *floating point(arm)* 389a⟩≡ (154b)

```
/* TODO: save and restore VFPv3 FP state once 5[cal] know the new registers.*/
fpuprocsave(p);
```

⟨*function* fpuprocsave*(arm)* 389b⟩≡ (641b)

```
/*
 * Called from sched() and sleep() via the machine-dependent
 * procsave() routine.
 * About to go in to the scheduler.
 * If the process wasn't using the FPU
 * there's nothing to do.
 */
void
fpuprocsave(Proc *p)
{
    if(p->fpstate == FPactive){
        if(p->state == Moribund)
            fpclear();
        else{
            /*
             * Fpsave() stores without handling pending
             * unmasked exeptions. Postnote() can't be called
             * here as sleep() already has up->rlock, so
             * the handling of pending exceptions is delayed
             * until the process runs again and generates an
             * emulation fault to activate the FPU.
             */
            fpsave(&p->fpsave);
        }
        p->fpstate = FPinactive;
    }
}
```

⟨arch_procrestore() *floating point(arm)* 389c⟩≡ (154c)

```
fpuprocrestore(p);
```

⟨*function* fpuprocrestore*(arm)* 389d⟩≡ (641b)

```
/*
 * The process has been rescheduled and is about to run.
 * Nothing to do here right now. If the process tries to use
 * the FPU again it will cause a Device Not Available
 * exception and the state will then be restored.
 */
void
fpuprocrestore(Proc *)
{
}
```

⟨syscall() *if RFORK, handle floating point(arm)* 389e⟩≡ (101d)

```
if(scallnr == RFORK)
    fpusysrfork(ureg);
```

⟨*function* fpusysrfork*(arm)* 390a⟩≡ (641b)

```
/*
 * Called early in the non-interruptible path of
 * sysrfork() via the machine-dependent syscall() routine.
 * Save the state so that it can be easily copied
 * to the child process later.
 */
void
fpusysrfork(Ureg*)
{
    if(up->fpstate == FPactive){
        fpsave(&up->fpsave);
        up->fpstate = FPinactive;
    }
}
```

⟨*function* fpsave*(arm)* 390b⟩≡ (641b)

```
/* should only be called if p->fpstate == FPactive */
void
fpsave(Arch_FPsave *fps)
{
    int n;

    fpon();
    fps->control = fps->status = fprd(Fpscr);
    assert(cpu->fpnregs);
    for (n = 0; n < cpu->fpnregs; n++)
        fpsavereg(n, (uvlong *)fps->regs[n]);
    fpoff();
}
```

⟨*function* fpunotify*(arm)* 390c⟩≡ (641b)

```
/*
 * Called when a note is about to be delivered to a
 * user process, usually at the end of a system call.
 * Note handlers are not allowed to use the FPU so
 * the state is marked (after saving if necessary) and
 * checked in the Device Not Available handler.
 */
void
fpunotify(Ureg*)
{
    if(up->fpstate == FPactive){
        fpsave(&up->fpsave);
        up->fpstate = FPinactive;
    }
    up->fpstate |= FPillegal;
}
```

⟨*function* fpunoted*(arm)* 390d⟩≡ (641b)

```
/*
 * Called from sysnoted() via the machine-dependent
 * noted() routine.
 * Clear the flag set above in fpunotify().
 */
void
fpunoted(void)
{
    up->fpstate &= ~FPillegal;
}
```

22.1.4 ARM floating point coprocessors (ARM)

⟨Coprocessors *other cases(arm)* 391a⟩≡ (21d)

```
#define CpFP        10          /* float FP, VFP cfg. */
#define CpDFP       11          /* double FP */
```

⟨*enum* _anon_*(*(processes/arm/vfp3.c)(arm)*)* 391b⟩≡ (641b)

```
/* subarchitecture code in cpu->havefp */
enum VFPKind {
    VFPv2   = 2,
    VFPv3   = 3,
};
```

⟨Cpu Arch *float fields(arm)* 391c⟩≡ (28e) 391d ▷

```
// option<VFPKind>, None = 0 /* vfp2 or vfp3 fpu */
int havefp;
bool havefpvalid;
// 16 for VFPv2, 32 for VFPv3
int fpnregs;
```

⟨Cpu Arch *float fields(arm)* 391d⟩+≡ (28e) ◁ 391c 391e ▷

```
bool fpon;
```

⟨Cpu Arch *float fields(arm)* 391e⟩+≡ (28e) ◁ 391d 396a ▷

```
ulong   fpscr;          /* sw copy */
```

22.1.5 coproc.c (ARM)

⟨*function* setupfpctlop*(arm)* 391f⟩≡ (611b)

```
/* fp coproc control */
static void
setupfpctlop(ulong instr[2], int opcode, int fpctlreg)
{
    ulong instrsz[2];

    fpctlreg &= Nfpctlregs - 1;
    instr[0] = opcode | fpctlreg << 16 | 0 << 12 | CpFP << 8;
    instr[1] = RETinst;

    ⟨setupfpctlop() write back cache for modified instr(arm) 403a⟩
    ⟨setupfpctlop() invalidate instruction cache(arm) 403c⟩
}
```

⟨*function* fprd*(arm)* 391g⟩≡ (611b)

```
ulong
fprd(int fpreg)
{
    int s, r;
    volatile ulong instr[2];
    Pufv fp;

    if (!cpu->fpon) {
        arch_dumpstack();
        panic("fprd: cpu%d fpu off", cpu->cpuno);
    }
    s = arch_splhi();
    /*
     * VMRS.  return value will be in R0, which is convenient.
     * Rt will be R0.
```

```
     */
    setupfpctlop(instr, 0xeef00010, fpreg);
    fp = (Pufv)instr;
    r = fp();
    arch_splx(s);
    return r;
}
```

⟨*function* fpwr*(arm)* 392a⟩≡ (611b)

```
void
fpwr(int fpreg, ulong val)
{
    int s;
    volatile ulong instr[2];
    Pvfu fp;

    /* fpu might be off and this VMSR might enable it */
    s = arch_splhi();
    setupfpctlop(instr, 0xeee00010, fpreg);     /* VMSR, Rt is R0 */
    fp = (Pvfu)instr;
    fp(val);
    arch_coherence();
    arch_splx(s);
}
```

⟨*function* setupfpop*(arm)* 392b⟩≡ (611b)

```
/* fp register access; don't bother with single precision */
static void
setupfpop(ulong instr[2], int opcode, int fpreg)
{
    ulong instrsz[2];

    instr[0] = opcode | 0 << 16 | (fpreg & (16 - 1)) << 12;
    if (fpreg >= 16)
        instr[0] |= 1 << 22;        /* high bit of dfp reg # */
    instr[1] = RETinst;

    ⟨setupfpop() write back cache for modified instr(arm) 403b⟩
    ⟨setupfpop() invalidate instruction cache(arm) 403d⟩
}
```

⟨*function* fprestreg*(arm)* 392c⟩≡ (611b)

```
void
fprestreg(int fpreg, uvlong val)
{
    int s;
    volatile ulong instr[2];
    void (*fp)(uvlong *);

    if (!cpu->fpon)
        panic("fprestreg: cpu%d fpu off", cpu->cpuno);
    s = arch_splhi();
    setupfpop(instr, 0xed100000 | CpDFP << 8, fpreg); /* VLDR, Rt is R0 */
    fp = (void (*)(uvlong *))instr;
    fp(&val);
    arch_coherence();
    arch_splx(s);
}
```

⟨*function* fpsavereg*(arm)* 393a⟩≡ (611b)

```
ulong
fpsavereg(int fpreg, uvlong *fpp)
{
    int s, r;
    volatile ulong instr[2];
    ulong (*fp)(uvlong *);

    if (!cpu->fpon)
        panic("fpsavereg: cpu%d fpu off", cpu->cpuno);
    s = arch_splhi();
    /*
     * VSTR.  pointer will be in R0, which is convenient.
     * Rt will be R0.
     */
    setupfpop(instr, 0xed000000 | CpDFP << 8, fpreg);
    fp = (ulong (*)(uvlong *))instr;
    r = fp(fpp);
    arch_splx(s);
    arch_coherence();
    return r;           /* not too meaningful */
}
```

22.1.6 vfp.c (ARM)

⟨*enum* _anon_*(*(processes/arm/vfp3.c)2(arm)*)* 393b⟩≡ (641b)

```
/* fp control regs.  most are read-only */
enum {
    Fpsid = 0,
    Fpscr = 1,          /* rw */
    Mvfr1 = 6,
    Mvfr0 = 7,
    Fpexc = 8,          /* rw */
    Fpinst= 9,          /* optional, for exceptions */
    Fpinst2=10,
};
```

⟨*enum* _anon_*(*(processes/arm/vfp3.c)3(arm)*)* 393c⟩≡ (641b)

```
enum {
    /* Fpexc bits */
    Fpex =      1u << 31,
    Fpenabled = 1 << 30,
    Fpdex =     1 << 29,    /* defined synch exception */
    /* FSR bits appear here */
    Fpmbc =     Fpdex,      /* bits exception handler must clear */

    /* Fpscr bits; see u.h for more */
    Stride =    MASK(2) << 20,
    Len =       MASK(3) << 16,
    Dn=     1 << 25,
    Fz=     1 << 24,
    /* trap exception enables (not allowed in vfp3) */
    FPIDNRM =   1 << 15,    /* input denormal */
    Alltraps = FPIDNRM | FPINEX | FPUNFL | FPOVFL | FPZDIV | FPINVAL,
    /* pending exceptions */
    FPAIDNRM =  1 << 7,     /* input denormal */
    Allexc = FPAIDNRM | FPAINEX | FPAUNFL | FPAOVFL | FPAZDIV | FPAINVAL,
    /* condition codes */
    Allcc =     MASK(4) << 28,
```

```
};
```

⟨*enum* _anon_*(*(processes/arm/vfp3.c)4(arm)*)* 394a⟩≡ (641b)

```
enum {
    /* CpCPaccess bits */
    Cpaccnosimd =   1u << 31,
    Cpaccd16 =  1 << 30,
};
```

⟨*function* subarch*(arm)* 394b⟩≡ (641b)

```
static char *
subarch(int impl, uint sa)
{
    static char *armarchs[] = {
        "VFPv1 (unsupported)",
        "VFPv2",
        "VFPv3+ with common VFP subarch v2",
        "VFPv3+ with null subarch",
        "VFPv3+ with common VFP subarch v3",
    };

    if (impl != 'A' || sa >= nelem(armarchs))
        return "GOK";
    else
        return armarchs[sa];
}
```

⟨*function* implement*(arm)* 394c⟩≡ (641b)

```
static char *
implement(uchar impl)
{
    if (impl == 'A')
        return "arm";
    else
        return "unknown";
}
```

⟨CpCONTROL.opcode2 *other cases(arm)* 394d⟩≡ (21f) 409d ▷

```
#define CpCPaccess  2
```

⟨*function* havefp*(arm)* 394e⟩≡ (641b)

```
static int
havefp(void)
{
    int gotfp;
    ulong acc, sid;

    if (cpu->havefpvalid)
        return cpu->havefp;

    cpu->havefp = 0;
    gotfp = 1 << CpFP | 1 << CpDFP;
    cpwrsc(0, CpCONTROL, 0, CpCPaccess, MASK(28));
    acc = cprdsc(0, CpCONTROL, 0, CpCPaccess);
    if ((acc & (MASK(2) << (2*CpFP))) == 0) {
        gotfp &= ~(1 << CpFP);
        print("fpon: no single FP coprocessor\n");
    }
    if ((acc & (MASK(2) << (2*CpDFP))) == 0) {
        gotfp &= ~(1 << CpDFP);
```

```
        print("fpon: no double FP coprocessor\n");
    }
    if (!gotfp) {
        print("fpon: no FP coprocessors\n");
        cpu->havefpvalid = true;
        return 0;
    }
    cpu->fpon = true;          /* don't panic */
    sid = fprd(Fpsid);
    cpu->fpon = false;

    switch((sid >> 16) & MASK(7)){
    case 0:             /* VFPv1 */
        break;
    case 1:             /* VFPv2 */
        cpu->havefp = VFPv2;
        cpu->fpnregs = 16;
        break;
    default:            /* VFPv3 or later */
        cpu->havefp = VFPv3;
        cpu->fpnregs = (acc & Cpaccd16) ? 16 : 32;
        break;
    }
    if (cpu->cpuno == 0)
        print("fp: %d registers, %s simd\n", cpu->fpnregs,
            (acc & Cpaccnosimd? " no": ""));
    cpu->havefpvalid = true;
    return 1;
}
```

⟨*function* fpoff *(arm)* 395a⟩≡ (641b)

```
/*
 * these can be called to turn the fpu on or off for user procs,
 * not just at system start up or shutdown.
 */
void
fpoff(void)
{
    if (cpu->fpon) {
        fpwr(Fpexc, 0);
        cpu->fpon = false;
    }
}
```

⟨*function* fpononly *(arm)* 395b⟩≡ (641b)

```
void
fpononly(void)
{
    if (!cpu->fpon && havefp()) {
        /* enable fp.  must be first operation on the FPUs. */
        fpwr(Fpexc, Fpenabled);
        cpu->fpon = true;
    }
}
```

⟨*function* fpcfg *(arm)* 395c⟩≡ (641b)

```
static void
fpcfg(void)
{
    int impl;
```

```
    ulong sid;
    static int printed;

    /* clear pending exceptions; no traps in vfp3; all v7 ops are scalar */
    cpu->fpscr = Dn | Fz | FPRNR | (FPINVAL | FPZDIV | FPOVFL) & ~Alltraps;
    fpwr(Fpscr, cpu->fpscr);
    cpu->fpconfiged = true;

    if (printed)
        return;
    sid = fprd(Fpsid);
    impl = sid >> 24;
    print("fp: %s arch %s; rev %ld\n", implement(impl),
        subarch(impl, (sid >> 16) & MASK(7)), sid & MASK(4));
    printed = 1;
}
```

⟨Cpu Arch *float fields(arm)* 396a⟩+≡ (28e) ◁391e 397b▷

```
bool fpconfiged;
```

⟨*function* fpinit*(arm)* 396b⟩≡ (641b)

```
void
fpinit(void)
{
    if (havefp()) {
        fpononly();
        fpcfg();
    }
}
```

⟨*function* fpon*(arm)* 396c⟩≡ (641b)

```
void
fpon(void)
{
    if (havefp()) {
      fpononly();
        if (cpu->fpconfiged)
            fpwr(Fpscr, (fprd(Fpscr) & Allcc) | cpu->fpscr);
        else
            fpcfg();    /* 1st time on this fpu; configure it */
    }
}
```

⟨*function* fpclear*(arm)* 396d⟩≡ (641b)

```
void
fpclear(void)
{
//  ulong scr;

    fpon();
//  scr = fprd(Fpscr);
//  cpu->fpscr = scr & ~Allexc;
//  fpwr(Fpscr, cpu->fpscr);

    fpwr(Fpexc, fprd(Fpexc) & ~Fpmbc);
}
```

⟨*function* fpstuck*(arm)* 397a⟩≡ (641b)

```
void
fpstuck(uintptr pc)
{
    if (cpu->fppc == pc && cpu->fppid == up->pid) {
        cpu->fpcnt++;
        if (cpu->fpcnt > 4)
            panic("fpuemu: cpu%d stuck at pid %ld %s pc %#p "
                "instr %#8.8lux", cpu->cpuno, up->pid, up->text,
                pc, *(ulong *)pc);
    } else {
        cpu->fppid = up->pid;
        cpu->fppc = pc;
        cpu->fpcnt = 0;
    }
}
```

⟨Cpu Arch *float fields(arm)* 397b⟩+≡ (28e) ◁396a

```
int fppid;          /* pid of last fault */
uintptr fppc;       /* addr of last fault */
int fpcnt;          /* how many consecutive at that addr */
```

⟨*enum* _anon_*(*(processes/arm/vfp3.c)5(arm)*)* 397c⟩≡ (641b)

```
enum {
    N = 1<<31,
    Z = 1<<30,
    C = 1<<29,
    V = 1<<28,
    REGPC = 15,
};
```

⟨*function* condok*(arm)* 397d⟩≡ (641b)

```
static int
condok(int cc, int c)
{
    switch(c){
    case 0: /* Z set */
        return cc&Z;
    case 1: /* Z clear */
        return (cc&Z) == 0;
    case 2: /* C set */
        return cc&C;
    case 3: /* C clear */
        return (cc&C) == 0;
    case 4: /* N set */
        return cc&N;
    case 5: /* N clear */
        return (cc&N) == 0;
    case 6: /* V set */
        return cc&V;
    case 7: /* V clear */
        return (cc&V) == 0;
    case 8: /* C set and Z clear */
        return cc&C && (cc&Z) == 0;
    case 9: /* C clear or Z set */
        return (cc&C) == 0 || cc&Z;
    case 10:    /* N set and V set, or N clear and V clear */
        return (~cc&(N|V))==0 || (cc&(N|V)) == 0;
    case 11:    /* N set and V clear, or N clear and V set */
        return (cc&(N|V))==N || (cc&(N|V))==V;
```

```
    case 12:    /* Z clear, and either N set and V set or N clear and V clear */
        return (cc&Z) == 0 && ((~cc&(N|V))==0 || (cc&(N|V))==0);
    case 13:    /* Z set, or N set and V clear or N clear and V set */
        return (cc&Z) || (cc&(N|V))==N || (cc&(N|V))==V;
    case 14:    /* always */
        return 1;
    case 15:    /* never (reserved) */
        return 0;
    }
    return 0;   /* not reached */
}
```

⟨*macro* ISCPOP*(arm)* 398a⟩≡ (641b)

```
#define ISCPOP(op)  ((op) == 0xE || ((op) & ~1) == 0xC)
```

⟨*macro* ISVFPOP*(arm)* 398b⟩≡ (641b)

```
#define ISVFPOP(cp, op) (((cp) == CpDFP || (cp) == CpFP) && ISCPOP(op))
```

⟨*function* fpuemu*(arm)* 398c⟩≡ (641b)

```
/* only called to deal with user-mode instruction faults */
int
fpuemu(Ureg* ureg)
{
    int s, nfp, cop, op;
    uintptr pc;

    if(waserror()){
        postnote(up, 1, up->errstr, NDebug);
        return 1;
    }

    if(up->fpstate & FPillegal)
        error("floating point in note handler");

    nfp = 0;
    pc = ureg->pc;
    validaddr(pc, 4, 0);
    if(!condok(ureg->psr, *(ulong*)pc >> 28))
        iprint("fpuemu: conditional instr shouldn't have got here\n");
    op  = (*(ulong *)pc >> 24) & MASK(4);
    cop = (*(ulong *)pc >>  8) & MASK(4);
    if(cpu->fpon)
        fpstuck(pc);        /* debugging; could move down 1 line */
    if (ISVFPOP(cop, op)) {  /* if vfp, fpu must be off */
        mathemu(ureg);      /* enable fpu & retry */
        nfp = 1;
    }

    poperror();
    return nfp;
}
```

⟨trap() *other locals(arm)* 398d⟩+≡ (105) ◁108b

```
int rv;
```

⟨trap() *when undefined instruction, if float instr(arm)* 398e⟩≡ (107b)

```
/* look for floating point instructions to interpret */
rv = fpuemu(ureg);
```

22.1.7 mathxxx() (ARM)

⟨*function* mathnote*(arm)* 399a⟩≡ (641b)

```
static void
mathnote(void)
{
    ulong status;
    char *msg, note[ERRMAX];

    status = up->fpsave.status;

    /*
     * Some attention should probably be paid here to the
     * exception masks and error summary.
     */
    if (status & FPAINEX)
        msg = "inexact";
    else if (status & FPAOVFL)
        msg = "overflow";
    else if (status & FPAUNFL)
        msg = "underflow";
    else if (status & FPAZDIV)
        msg = "divide by zero";
    else if (status & FPAINVAL)
        msg = "bad operation";
    else
        msg = "spurious";
    snprint(note, sizeof note, "sys: fp: %s fppc=%#p status=%#lux",
        msg, up->fpsave.pc, status);
    postnote(up, 1, note, NDebug);
}
```

⟨*function* mathemu*(arm)* 399b⟩≡ (641b)

```
static void
mathemu(Ureg *)
{
    switch(up->fpstate){
    case FPinit:
        fpinit();
        up->fpstate = FPactive;
        break;
    case FPinactive:
        /*
         * Before restoring the state, check for any pending
         * exceptions.  There's no way to restore the state without
         * generating an unmasked exception.
         * More attention should probably be paid here to the
         * exception masks and error summary.
         */
        if(up->fpsave.status & (FPAINEX|FPAUNFL|FPAOVFL|FPAZDIV|FPAINVAL)){
            mathnote();
            break;
        }
        fprestore(up);
        up->fpstate = FPactive;
        break;
    case FPactive:
        error("illegal instruction: bad vfp fpu opcode");
        break;
    }
```

```
    fpclear();
}
```

⟨*function* fprestore*(arm)* 400a⟩≡ (641b)

```
static void
fprestore(Proc *p)
{
    int n;

    fpon();
    fpwr(Fpscr, p->fpsave.control);
    cpu->fpscr = fprd(Fpscr) & ~Allcc;
    assert(cpu->fpnregs);
    for (n = 0; n < cpu->fpnregs; n++)
        fprestreg(n, *(uvlong *)p->fpsave.regs[n]);
}
```

22.1.8 /proc/x/fpregs

⟨procread() *Qfpregs case* 400b⟩≡ (468c)

```
case Qfpregs:
    rptr = (uchar*)&p->fpsave;
    rsize = sizeof(Arch_FPsave);
    goto regread;
```

⟨procwrite *Qfpregs case* 400c⟩≡ (471)

```
case Qfpregs:
    if(offset >= sizeof(Arch_FPsave))
        n = 0;
    else if(offset+n > sizeof(Arch_FPsave))
        n = sizeof(Arch_FPsave) - offset;
    memmove((uchar*)&p->fpsave+offset, va, n);
    break;
```

22.2 CPU cache

22.2.1 Instruction and data cache (ARM)

⟨CpSC primary registers *other cases(arm)* 400d⟩≡ (21e) 405h ▷

```
#define CpCACHE     7           /* cache/write buffer control */
```

⟨CpCONTROL.CpMainctl *other cases(arm)* 400e⟩≡ (21g) 409a ▷

```
#define CpCdcache   0x00000004  /* C: data cache on */
#define CpCicache   0x00001000  /* I: instruction cache on */
```

⟨armstart() *disable features(arm)* 400f⟩≡ (186b)

```
BIC $(CpCdcache|CpCicache), R1
```

⟨armstart() *invalidate caches(arm)* 400g⟩≡ (186b)

```
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvu), CpCACHEall
```

⟨armstart() *reenable features(arm)* 400h⟩≡ (186b)

```
ORR $(CpCdcache|CpCicache), R0
```

22.2.2 Cache policy

⟨*type CpCACHERANGE opcode2(arm)* 401a⟩≡ (607c)

```
/*
 * CpCACHERANGE opcode2 fields for MCRR instruction (armv6)
 */
#define CpCACHERANGEinvd    6       /* invalidate data */
#define CpCACHERANGEdwb     12      /* writeback */
#define CpCACHERANGEdwbi    14      /* writeback+invalidate */
```

⟨*instruction MCRR(arm)* 401b⟩≡ (608)

```
#define MCRR(coproc, op, rd, rn, crm) \
    WORD $(0xec400000|(rn)<<16|(rd)<<12|(coproc)<<8|(op)<<4|(crm))
```

22.2.3 Paging and caching (ARM)

⟨*type* PageTableEntryAttribute1*(arm)* 401c⟩≡ (607c)

```
#define Buffered    0x00000004      /* L[12]: write-back not -thru */
#define Cached      0x00000008      /* L[12] */
```

⟨Soc *mmu fields* 401d⟩≡ (33a)

```
u32int  l1ptedramattrs;
u32int  l2ptedramattrs;
```

⟨soc(raspberry pi1) *other fields(arm)* 401e⟩≡ (33b) 407a ▷

```
.l1ptedramattrs = Cached | Buffered,
.l2ptedramattrs = Cached | Buffered,
```

⟨*constant* L1ptedramattrs*(arm)* 401f⟩≡ (635a)

```
#define L1ptedramattrs  soc.l1ptedramattrs
```

⟨*constant* L2ptedramattrs*(arm)* 401g⟩≡ (635a)

```
#define L2ptedramattrs  soc.l2ptedramattrs
```

22.2.4 Memory barriers (ARM)

⟨*instruction ISB(arm)* 401h⟩≡ (608)

```
#define ISB \
    MOVW    $0, R0; \
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEwait
```

⟨*instruction DSB(arm)* 401i⟩≡ (608)

```
#define DSB \
    MOVW    $0, R0; \
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEwait
```

22.2.5 Cache operations (ARM)

⟨*type CpCACHE secondary registers(arm)* 401j⟩≡ (607c)

```
/*
 * CpCACHE Secondary (CRm) registers and opcode2 fields.  op1==0.
 * In ARM-speak, 'flush' means invalidate and 'clean' means writeback.
 */
#define CpCACHEintr 0           /* interrupt (op2==4) */
#define CpCACHEinvi 5           /* instruction, branch table */
#define CpCACHEinvd 6           /* data or unified */
#define CpCACHEinvu 7           /* unified (not on cortex) */
#define CpCACHEwb   10          /* writeback to PoC */
#define CpCACHEwbi  14          /* writeback+invalidate (to PoC) */
```

⟨*type CpCACHE opcode2(arm)* 402a⟩≡ (607c)

```
#define CpCACHEall  0          /* entire (not for invd nor wb(i) on cortex) */
#define CpCACHEse   1          /* single entry */
#define CpCACHEsi   2          /* set/index (set/way) */
#define CpCACHEwait 4          /* wait (prefetch flush on cortex) */
```

⟨*constant CACHELINESZ(raspberry pi1)(ram)* 402b⟩≡ (635c)

```
#define CACHELINESZ 32
```

22.2.6 cachexxx() (ARM)

⟨arch_mmuswitch() *write back and invalidate cache before switch(arm)* 402c⟩≡ (154a)

```
/* write back dirty and invalidate l1 caches */
cacheuwbinv();
```

⟨arch_mmurelease() *write back cache after adjusting page tables(arm)* 402d⟩≡ (114m)

```
/* make sure map is in memory */
/* could be smarter about how much? */
cachedwbse(&cpu->mmul1[L1X(UZERO)], (L1hi - L1lo)*sizeof(PTE));
```

⟨arch_mmuswitch() *write back cache after adjusting page tables(arm)* 402e⟩≡ (154a)

```
/* make sure map is in memory */
/* could be smarter about how much? */
cachedwbse(&l1[L1X(UZERO)], (L1hi - L1lo)*sizeof(PTE));
```

⟨mmuinit1() *write back cache after adjusting page tables(arm)* 402f⟩≡ (196a)

```
cachedwbse(&l1[L1X(0)], sizeof(PTE));
```

⟨mmukmap() *write back cache after changing page tables(arm)* 402g⟩≡ (282b)

```
cachedwbse(pte0, (uintptr)pte - (uintptr)pte0);
```

⟨arch_putmmu() *write back cache after changing pde(arm)* 402h⟩≡ (119)

```
cachedwbse(l1, sizeof *l1);
```

⟨arch_putmmu() *write back cache after changing pte(arm)* 402i⟩≡ (119)

```
cachedwbse(&pte[L2X(va)], sizeof pte[0]);
```

⟨arch_putmmu() *write back cache before fixing va fault(arm)* 402j⟩≡ (119)

```
/* force l2 page to memory */
cachedwbse((void *)pg->va, BY2PG);
```

⟨arch_mmurelease() *invalidate cache before adjusting page tables(arm)* 402k⟩≡ (114m)

```
/* write back dirty and invalidate l1 caches */
cacheuwbinv();
```

⟨arch_kunmap() *write back cache for page(arm)* 402l⟩≡ (121b)

```
cachedwbinvse(k, BY2PG);
```

⟨arch__trapinit() *invalidate cache after adjust vectors(arm)* 402m⟩≡ (194a)

```
cacheuwbinv();
l2cacheuwbinv();
```

⟨fbinit() *write back cache after modified fb info(arm)* 402n⟩≡ (282a)

```
cachedwbinvse(fi, sizeof(Fbinfo));
```

⟨arch__kexit() *write back cache(arm)* 402o⟩≡ (106a)

```
/* make visible immediately to user proc */
cachedwbinvse(tos, sizeof *tos);
```

⟨setupfpctlop() *write back cache for modified instr(arm)* 403a⟩≡ (391f)

```
cachedwbse(instr, sizeof instrsz);
```

⟨setupfpop() *write back cache for modified instr(arm)* 403b⟩≡ (392b)

```
cachedwbse(instr, sizeof instrsz);
```

⟨setupfpctlop() *invalidate instruction cache(arm)* 403c⟩≡ (391f)

```
cacheiinv();
```

⟨setupfpop() *invalidate instruction cache(arm)* 403d⟩≡ (392b)

```
cacheiinv();
```

⟨*function cachexxx(raspberry pi1)(ram)* 403e⟩≡ (635c)

```
/*
 * drain write buffer
 * writeback data cache
 */
TEXT cachedwb(SB), 1, $-4
    DSB
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEall
    RET

/*
 * drain write buffer
 * writeback and invalidate data cache
 */
TEXT cachedwbinv(SB), 1, $-4
    DSB
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEall
    RET

/*
 * cachedwbinvse(va, n)
 *   drain write buffer
 *   writeback and invalidate data cache range [va, va+n)
 */
TEXT cachedwbinvse(SB), 1, $-4
    MOVW    R0, R1      /* DSB clears R0 */
    DSB
    MOVW    n+4(FP), R2
    ADD R1, R2
    SUB $1, R2
    BIC $(CACHELINESZ-1), R1
    BIC $(CACHELINESZ-1), R2
    MCRR(CpSC, 0, 2, 1, CpCACHERANGEdwbi)
    RET

/*
 * cachedwbse(va, n)
 *   drain write buffer
 *   writeback data cache range [va, va+n)
 */
TEXT cachedwbse(SB), 1, $-4
    MOVW    R0, R1      /* DSB clears R0 */
    DSB
    MOVW    n+4(FP), R2
    ADD R1, R2
    BIC $(CACHELINESZ-1), R1
```

```
    BIC $(CACHELINESZ-1), R2
    MCRR(CpSC, 0, 2, 1, CpCACHERANGEdwb)
    RET

/*
 * cachedinvse(va, n)
 *   drain write buffer
 *   invalidate data cache range [va, va+n)
 */
TEXT cachedinvse(SB), 1, $-4
    MOVW    R0, R1      /* DSB clears R0 */
    DSB
    MOVW    n+4(FP), R2
    ADD R1, R2
    SUB $1, R2
    BIC $(CACHELINESZ-1), R1
    BIC $(CACHELINESZ-1), R2
    MCRR(CpSC, 0, 2, 1, CpCACHERANGEinvd)
    RET

/*
 * drain write buffer and prefetch buffer
 * writeback and invalidate data cache
 * invalidate instruction cache
 */
TEXT cacheuwbinv(SB), 1, $-4
    BARRIERS
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEall
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall
    RET

/*
 * L2 cache is not enabled
 */
TEXT l2cacheuwbinv(SB), 1, $-4
    RET

/*
 * invalidate instruction cache
 */
TEXT cacheiinv(SB), 1, $-4
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall
    RET
```

22.3 Memory model

22.3.1 arch_coherence()

⟨unlock() *after modified key, call coherence* 404a⟩≡ (36f)

```
// for processor caches, to ensure the lock value is seen by other
// processors so that if they were doing while(l->key) { ... } they
// can finally exit the while loop.
arch_coherence();
```

⟨iunlock() *after modified key, call coherence* 404b⟩≡ (38b)

```
arch_coherence();
```

⟨irq() *before dispatch(arm)* 405a⟩≡ (112e)
```
arch_coherence();
```

⟨irq() *after dispatch(arm)* 405b⟩≡ (112e)
```
arch_coherence();
```

⟨arch__trapinit() *coherence(arm)* 405c⟩≡ (194a)
```
arch_coherence();
```

⟨init0() *coherence(arm)* 405d⟩≡ (202)
```
arch_coherence();
```

⟨cas32() *if address contained old value, call coherence(arm)* 405e⟩≡ (356b)
```
if (r)
    arch_coherence();
```

22.3.2 Memory barriers (ARM)

⟨*instruction BARRIERS(arm)* 405f⟩≡ (608)
```
#define BARRIERS    ISB; DSB
```

22.4 The Raspberry Pi 2 machine

⟨*global* soc*(raspberry pi2)(arm)* 405g⟩≡ (631b)
```
Soc soc = {
    .dramsize   = 1024*MiB,
    .physio     = 0x3F000000,
     ⟨soc(raspberry pi2) other fields(arm) 406b⟩
};
```

22.4.1 CPU ID

⟨CpSC primary registers *other cases(arm)* 405h⟩+≡ (21e) ◁400d 407d▷
```
#define CpID        0           /* ID and cache type */
```

⟨*type CpID secondary registers(arm)* 405i⟩≡ (607c)
```
/*
 * CpID Secondary (CRm) registers.
 */
#define CpIDidct    0
#define CpIDfeat    1
```

⟨*type CpID opcode2(arm)* 405j⟩≡ (607c)
```
/*
 * CpID op1==0 opcode2 fields.
 * the cortex has more op1 codes for cache size, etc.
 */
#define CpIDid      0           /* main ID */
#define CpIDmpid    5           /* multiprocessor id (cortex) */
```

⟨*instruction CPUID(arm)* 405k⟩≡ (608)
```
/*
 * get cpu id, or zero if armv6
 */
#define CPUID(r) \
    MRC CpSC, 0, r, C(CpID), C(CpIDfeat), 7; \
    CMP $0, r; \
    B.EQ    2(PC); \
    MRC CpSC, 0, r, C(CpID), C(CpIDidct), CpIDmpid; \
    AND.S   $(MAXCPUS-1), r
```

22.4.2 MMU

⟨*type* PageTableEntryAttribute2*(arm)* 406a⟩≡ (607c)

```
#define L1wralloc   (1<<12)         /* L1 TEX */
#define L1sharable  (1<<16)
#define L2wralloc   (1<<6)          /* L2 TEX (small pages) */
#define L2sharable  (1<<10)
```

⟨soc(raspberry pi2) *other fields(arm)* 406b⟩≡ (405g) 407b ▷

```
.l1ptedramattrs = Cached | Buffered | L1wralloc | L1sharable,
.l2ptedramattrs = Cached | Buffered | L2wralloc | L2sharable,
```

⟨*function* l2ap*(raspberry pi2)(arm)* 406c⟩≡ (631b)

```
int
l2ap(int ap)
{
    return (AP(0, (ap)));
}
```

22.4.3 TTB

⟨*type* CpTTB*(arm)* 406d⟩≡ (607c)

```
/*
 * CpTTB op1==0, Crm==0 opcode2 values.
 */
#define CpTTB1      1           /* cortex */
```

⟨*type CpTTB cache control bits(arm)* 406e⟩≡ (607c)

```
/*
 * CpTTB cache control bits
 */
#define CpTTBs  (1<<1)  /* page table in shareable memory */
```

22.4.4 Local registers

⟨*struct* Mbox*(arm)* 406f⟩≡ (631b)

```
/*
 * Arm local regs for SMP
 */
struct Mbox {
    u32int  doorbell;
    u32int  mbox1;
    u32int  mbox2;
    u32int  startcpu;
};
```

⟨*struct* Mboxes*(arm)* 406g⟩≡ (631b)

```
struct Mboxes {
    Mbox    set[4];
    Mbox    clr[4];
};
```

⟨*constant* ARMLOCAL*(arm)* 406h⟩≡ (639b 631b)

```
#define ARMLOCAL    (VIRTIO+IOSIZE)
```

⟨Soc *other fields* 406i⟩≡ (33a)

```
uintptr armlocal;
```

⟨soc(raspberry pi1) *other fields(arm)* 407a⟩+≡ (33b) ◁401e 714f▷

```
.armlocal   = 0,
```

⟨soc(raspberry pi2) *other fields(arm)* 407b⟩+≡ (405g) ◁406b 714g▷

```
.armlocal   = 0x40000000,
```

⟨mmuinit() *set armlocal virtual memory mapping(arm)* 407c⟩≡ (188c)

```
// for raspi2
pa = soc.armlocal;
if(pa)
    l1[L1X(va)] = pa|Dom0|L1AP(Krw)|Section;
```

22.4.5 Timer

⟨CpSC primary registers *other cases(arm)* 407d⟩+≡ (21e) ◁405h 407f▷

```
#define CpTIMER     14          /* Generic timer (cortex-a7) */
```

⟨*type* CpTIMER*(arm)* 407e⟩≡ (607c)

```
/*
 * CpTIMER op1==0 Crm and opcode2 registers (cortex-a7)
 */
#define CpTIMERphys     2

#define CpTIMERphysval  0
#define CpTIMERphysctl  1
```

22.4.6 Performance monitor

⟨CpSC primary registers *other cases(arm)* 407f⟩+≡ (21e) ◁407d

```
#define CpCLD       9        // Performance monitor
```

⟨*type CpCLD secondary registers(arm)* 407g⟩≡ (607c)

```
/*
 * CpCLD Secondary (CRm) registers and opcode2 fields for op1==0. (cortex)
 */
#define CpCLDena    12          /* enables */
#define CpCLDcyc    13          /* cycle counter */
```

⟨*type CpCLD opcode2(arm)* 407h⟩≡ (607c)

```
#define CpCLDenapmnc    0
#define CpCLDenacyc 1
```

22.5 SMP (ARM)

22.5.1 Initialization adjustments

Basic pagination cpu0

⟨*function* armstart*(raspberry pi2)(arm)* 407i⟩≡ (627c)

```
TEXT armstart(SB), 1, $-4

    /*
     * if not cpu0, go to secondary startup
     */
    CPUID(R1)
    BNE reset
```

```
    /*
     * disable the mmu and caches
     * invalidate tlb
     */
    MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    BIC $(CpCdcache|CpCicache|CpCmmu), R1
    ORR $(CpCsbo|CpCsw), R1
    BIC $CpCsbz, R1
    MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
    ISB

    /*
     * clear mach and page tables
     */
    MOVW    $PADDR(CPUADDR), R1
    MOVW    $PADDR(KTZERO), R2
_ramZ:
    MOVW    R0, (R1)
    ADD $4, R1
    CMP R1, R2
    BNE _ramZ

    /*
     * start stack at top of mach (physical addr)
     * set up page tables for kernel
     */
    MOVW    $PADDR(CPUADDR+CPUSIZE-4), R13
    MOVW    $PADDR(L1), R0
    BL  mmuinit(SB)

    /*
     * set up domain access control and page table base
     */
    MOVW    $Client, R1
    MCR CpSC, 0, R1, C(CpDAC), C(0)
    MOVW    $PADDR(L1), R1
    ORR     $(CpTTBs/*|CpTTBowba|CpTTBiwba*/), R1
    MCR CpSC, 0, R1, C(CpTTB), C(0)
    MCR CpSC, 0, R1, C(CpTTB), C(0), CpTTB1 /* cortex has two */

    /*
     * invalidate my caches before enabling
     */
    BL  cachedinv(SB)
    BL  cacheiinv(SB)
    BL  l2cacheuinv(SB)
    BARRIERS

    /*
     * enable caches, mmu, and high vectors
     */
    MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
    ORR $CpACsmp, R1        /* turn SMP on */
    MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
    BARRIERS

    MRC CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
    ORR $(CpChv|CpCdcache|CpCicache|CpCmmu), R0
```

```
MCR CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
BARRIERS

/*
 * switch SB, SP, and PC into KZERO space
 */
MOVW    $setR12(SB), R12
MOVW    $(CPUADDR+CPUSIZE-4), R13
MOVW    $_startpg(SB), R15
```

⟨CpCONTROL.CpMainctl *other cases(arm)* 409a⟩+≡ (21g) ◁400e 409b▷

```
#define CpCsw       (1<<10)     /* SW: SWP(B) enable (deprecated in v7) */
#define CpCpredict  0x00000800  /* Z: branch prediction (armv7) */
```

⟨CpCONTROL.CpMainctl *other cases(arm)* 409b⟩+≡ (21g) ◁409a 409c▷

```
#define CpCve       (1<<24)     /* VE: intr vectors enable */
#define CpCtre      (1<<28)     /* TRE: TEX remap enable */
```

⟨CpCONTROL.CpMainctl *other cases(arm)* 409c⟩+≡ (21g) ◁409b

```
#define CpCsbo (3<<22|1<<18|1<<16|017<<3)       /* must be 1 (armv7) */
#define CpCsbz (CpCtre|1<<26|CpCve|1<<15|7<<7)  /* must be 0 (armv7) */
```

⟨CpCONTROL.opcode2 *other cases(arm)* 409d⟩+≡ (21f) ◁394d

```
#define CpAuxctl    1
```

⟨*type CpCONTROL CpAuxctl(arm)* 409e⟩≡ (607c)

```
/*
 * CpCONTROL: op1==0, CRm==0, op2==CpAuxctl.
 * Auxiliary control register on cortex at least.
 */
/* cortex-a7 and cortex-a9 */
#define CpACsmp         (1<<6)  /* SMP l1 caches coherence; needed for ldrex/strex */
```

Jumping to C main() cpu0

⟨*function* _startpg*(raspberry pi2)(arm)* 409f⟩≡ (627c)

```
TEXT _startpg(SB), 1, $-4

    ⟨_startpg() enable cycle counter(raspberry pi2)(arm) 422i⟩
    /*
     * call main and loop forever if it returns
     */
    BL  main(SB)
    B   0(PC)

    BL  _div(SB)        /* hack to load _div, etc. */
```

Starting the other processors

⟨main() *start the other processors(arm)* 409g⟩≡ (189)

```
launchinit(getncpus());
```

⟨*function* launchinit*(arm)* 410a⟩≡ (628c)

```
void
launchinit(int ncpus)
{
    int xcpu;
    Cpu *mm;
    PTE *l1;

    ⟨launchinit() sanity check ncpus(arm) 410c⟩
    for(xcpu = 1; xcpu < ncpus; xcpu++){
        cpus[xcpu] = mm = mallocalign(CPUSIZE, CPUSIZE, 0, 0);
        l1 = mallocalign(L1SIZE, L1SIZE, 0, 0);
        if(mm == nil || l1 == nil)
            panic("launchinit");
        memset(mm, 0, CPUSIZE);
        mm->cpuno = xcpu;

        memmove(l1, cpu->mmul1, L1SIZE);  /* clone cpu0's l1 table */
        cachedwbse(l1, L1SIZE);
        mm->mmul1 = l1;
        cachedwbse(mm, CPUSIZE);

    }
    cachedwbse(cpus, sizeof cpus);
    xcpu = startcpus(ncpus);
    if(xcpu < ncpus)
            panic("only %d cpu%s started", xcpu, xcpu == 1? "" : "s");
}
```

⟨*constant* L1SIZE*(arm)* 410b⟩≡ (633b)

```
#define L1SIZE      (4 * BY2PG)
```

⟨launchinit() *sanity check ncpus(arm)* 410c⟩≡ (410a)

```
if(ncpus > MAXCPUS)
    ncpus = MAXCPUS;
```

⟨*function* startcpus*(raspberry pi1)(arm)* 410d⟩≡ (631a)

```
int
startcpus(uint)
{
    return 1;
}
```

⟨*function* getncpus*(raspberry pi1)(arm)* 410e⟩≡ (631a)

```
int
getncpus(void)
{
    return 1;
}
```

⟨*function* getncpus*(raspberry pi2)(arm)* 410f⟩≡ (631b)

```
int
getncpus(void)
{
    int n, max;
    char *p;

    n = 4;
    if(n > MAXCPUS)
        n = MAXCPUS;
```

```
    ⟨getncpus() adjust n if ncpu config parameter(arm) 411a⟩
    return n;
}
```

⟨getncpus() *adjust n if ncpu config parameter(arm)* 411a⟩≡ (410f)

```
p = getconf("*ncpu");
if(p && (max = atoi(p)) > 0 && n > max)
    n = max;
```

⟨*function* startcpus*(raspberry pi2)(arm)* 411b⟩≡ (631b)

```
int
startcpus(uint ncpu)
{
    int i;

    for(i = 0; i < ncpu; i++)
        lock(&startlock[i]);
    cachedwbse(startlock, sizeof startlock);
    for(i = 1; i < ncpu; i++){
        if(startcpu(i) < 0)
            return i;
        lock(&startlock[i]);
        unlock(&startlock[i]);
    }
    return ncpu;
}
```

⟨*function* startcpu*(raspberry pi2)(arm)* 411c⟩≡ (631b)

```
static int
startcpu(uint xcpu)
{
    Mboxes *mb;
    int i;
    void cpureset();

    mb = (Mboxes*)(ARMLOCAL + Mboxregs);
    if(mb->clr[xcpu].startcpu)
        return -1;
    mb->set[xcpu].startcpu = PADDR(cpureset);
    for(i = 0; i < 1000; i++)
        if(mb->clr[xcpu].startcpu == 0)
            return 0;
    mb->clr[xcpu].startcpu = PADDR(cpureset);
    mb->set[xcpu].doorbell = 1;
    return 0;
}
```

Basic pagination other cpus

⟨*function* cpureset*(raspberry pi2)(arm)* 411d⟩≡ (627c)

```
/*
 * startup entry for cpu(s) other than 0
 */
TEXT cpureset(SB), 1, $-4
reset:
    /*
     * load physical base for SB addressing while mmu is off
     * keep a handy zero in R0 until first function call
     */
```

```
MOVW    $setR12(SB), R12
SUB $KZERO, R12
MOVW    $0, R0

/*
 * SVC mode, interrupts disabled
 */
MOVW    $(PsrDirq|PsrDfiq|PsrMsvc), R1
MOVW    R1, CPSR

/*
 * disable the mmu and caches
 * invalidate tlb
 */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
BIC $(CpCdcache|CpCicache|CpCmmu), R1
ORR $(CpCsbo|CpCsw), R1
BIC $CpCsbz, R1
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
ISB

/*
 * find Mach for this cpu
 */
MRC CpSC, 0, R2, C(CpID), C(CpIDidct), CpIDmpid
AND $(MAXCPUS-1), R2    /* mask out non-cpu-id bits */
SLL $2, R2          /* convert to word index */
MOVW    $cpus(SB), R0
ADD R2, R0          /* R0 = &cpus[cpuid] */
MOVW    (R0), R0        /* R0 = cpus[cpuid] */
CMP $0, R0
MOVW.EQ $CPUADDR, R0        /* paranoia: use CPUADDR if 0 */
SUB $KZERO, R0      /* phys addr */
MOVW    R0, R(CPU)      /* m = PADDR(cpus[cpuid]) */

/*
 * start stack at top of local Mach
 */
MOVW    R(CPU), R13
ADD     $(CPUSIZE-4), R13

/*
 * set up page tables for kernel
 */
MOVW    12(R(CPU)), R0  /* m->mmul1 */
SUB $KZERO, R0      /* phys addr */
BL  mmuinit(SB)

/*
 * set up domain access control and page table base
 */
MOVW    $Client, R1
MCR CpSC, 0, R1, C(CpDAC), C(0)
MOVW    12(R(CPU)), R1  /* m->mmul1 */
SUB $KZERO, R1      /* phys addr */
ORR     $(CpTTBs/*|CpTTBowba|CpTTBiwba*/), R1
MCR CpSC, 0, R1, C(CpTTB), C(0)
MCR CpSC, 0, R1, C(CpTTB), C(0), CpTTB1 /* cortex has two */
```

```
/*
 * invalidate my caches before enabling
 */
BL  cachedinv(SB)
BL  cacheiinv(SB)
BARRIERS

/*
 * enable caches, mmu, and high vectors
 */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
ORR $CpACsmp, R1        /* turn SMP on */
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
BARRIERS

MRC CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
ORR $(CpChv|CpCdcache|CpCicache|CpCmmu), R0
MCR CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
BARRIERS

/*
 * switch CPU, SB, SP, and PC into KZERO space
 */
ADD $KZERO, R(CPU)
MOVW    $setR12(SB), R12
ADD $KZERO, R13
MOVW    $_startpg2(SB), R15
```

Jumping to C `cpustart()`

⟨*function* `_startpg2`*(raspberry pi2)(arm)* 413a⟩≡ (627c)

```
TEXT _startpg2(SB), 1, $-4

    ⟨_startpg2() enable cycle counter(raspberry pi2)(arm) 422j⟩
    /*
     * call cpustart and loop forever if it returns
     */
    MRC CpSC, 0, R0, C(CpID), C(CpIDidct), CpIDmpid
    AND $(MAXCPUS-1), R0            /* mask out non-cpu-id bits */
    BL  ,cpustart(SB)
    B   ,0(PC)
```

`main()` other processors (`cpustart()`)

⟨*function* `cpustart`*(raspberry pi2)(arm)* 413b⟩≡ (631b)

```
void
cpustart(int xcpu)
{
    Mboxes *mb;

    up = nil;
    arch__cpuinit();
    mmuinit1((void*)cpu->mmul1);

    mb = (Mboxes*)(ARMLOCAL + Mboxregs);
    mb->clr[xcpu].doorbell = 1;

    arch__trapinit();
```

```
    clockinit();
    timersinit();

    arch_cpuidprint();
    archreset();

    machon(cpu->cpuno);

    unlock(&startlock[xcpu]);

    // schedule a ready process (not necessarily the first one now)
    schedinit();

    panic("schedinit returned");
}
```

⟨arch__cpuinit() *it not cpu0* 414a⟩≡ (190d)

```
if (cpu->cpuno != 0) {
    /* synchronise with cpu 0 */
    cpu->ticks = cpu0->ticks;
}
```

Displaying processor information

⟨*function* arch_cpuidprint*(raspberry pi2)(arm)* 414b⟩≡ (631b)

```
void
arch_cpuidprint(void)
{
    char name[64];

    cputype2name(name, sizeof name);
    arch_delay(50);             /* let uart catch up */
    print("cpu%d: %dMHz ARM %s\n", cpu->cpuno, cpu->cpumhz, name);
}
```

⟨*function* cputype2name*(raspberry pi2)(arm)* 414c⟩≡ (631b)

```
char*
cputype2name(char *buf, int size)
{
    ulong r;

    r = cpidget();          /* main id register */
    assert((r >> 24) == 'A');
    seprint(buf, buf + size, "Cortex-A7 r%ldp%ld",
        (r >> 20) & MASK(4), r & MASK(4));
    return buf;
}
```

⟨*function* cpidget*(raspberry pi2)(arm)* 414d⟩≡ (627c)

```
TEXT cpidget(SB), 1, $-4            /* main ID */
    MRC CpSC, 0, R0, C(CpID), C(0), CpIDid
    RET
```

Halting

⟨*function* arch_idlehands*(raspberry pi2)(arm)* 414e⟩≡ (627c)

```
TEXT arch_idlehands(SB), $-4
    MOVW    CPSR, R3
    ORR $(PsrDirq|PsrDfiq), R3, R1      /* splfhi */
```

```
    MOVW    R1, CPSR

    DSB
    MOVW    nrdy(SB), R0
    CMP $0, R0
/*** with WFI, local timer interrupts can be lost and dispatching stops
    WFI_EQ
***/
    DSB

    MOVW    R3, CPSR            /* splx */
    RET
```

⟨*instruction* WFI_EQ*(arm)* 415a⟩≡ (627c)

```
#define WFI_EQ  WORD    $0x0320f003 /* wait for interrupt if eq */
```

22.5.2 Setting cpu

⟨_vsvc() *set cpu register(arm)* 415b⟩≡ (100c)

```
CPUID(R1)
SLL $2, R1          /* convert to word index */
MOVW    $cpus(SB), R2
ADD R1, R2
MOVW    (R2), R(CPU)        /* m = cpus[cpuid] */
CMP $0, R(CPU)
MOVW.EQ $CPUADDR, R0        /* paranoia: use CPUADDR if 0 */
```

⟨_vswitch() *set cpu register(arm)* 415c⟩≡ (104b)

```
CPUID(R1)
SLL $2, R1          /* convert to word index */
MOVW    $cpus(SB), R2
ADD R1, R2
MOVW    (R2), R(CPU)        /* m = cpus[cpuid] */
CMP $0, R(CPU)
MOVW.EQ $CPUADDR, R0        /* paranoia: use CPUADDR if 0 */
```

22.5.3 Concurrency adjustments

⟨*instruction LDREX(arm)* 415d⟩≡ (608)

```
#define LDREX(fp,t)   WORD $(0xe<<28|0x01900f9f | (fp)<<16 | (t)<<12)
```

⟨*instruction STREX(arm)* 415e⟩≡ (608)

```
/* 'The order of operands is from left to right in dataflow order' - asm man */
#define STREX(f,tp,r) WORD $(0xe<<28|0x01800f90 | (tp)<<16 | (r)<<12 | (f)<<0)
```

⟨*instruction CLREX(arm)* 415f⟩≡ (608)

```
#define CLREX   WORD    $0xf57ff01f
```

⟨*instruction DMB(arm)* 415g⟩≡ (617b)

```
#define DMB WORD    $0xf57ff05f /* data mem. barrier; last f = SY */
```

⟨*function* arch_tas*(raspberry pi2)(arm)* 416a⟩≡ (617b)

```
TEXT    arch_tas(SB), $-4
    /* returns old (R0) after modifying (R0) */
    MOVW    R0,R5
    DMB

    MOVW    $1,R2       /* new value of (R0) */
    MOVW    $MAXSC, R8
tas1:
    LDREX(5,7)      /* LDREX 0(R5),R7 */
    CMP.S   $0, R7      /* old value non-zero (lock taken)? */
    BNE lockbusy    /* we lose */
    SUB.S   $1, R8
    BEQ lockloop2
    STREX(2,5,4)        /* STREX R2,(R5),R4 */
    CMP.S   $0, R4
    BNE tas1        /* strex failed? try again */
    DMB
    B   tas0
lockloop2:
    BL  abort(SB)
lockbusy:
    CLREX
tas0:
    MOVW    R7, R0      /* return old value */
    RET
```

22.5.4 Timer adjustments

⟨*constant* Localctl*(arm)* 416b⟩≡ (638c)

```
Localctl    = 0x00,
```

⟨*constant* Prescaler*(arm)* 416c⟩≡ (638c)

```
Prescaler   = 0x08,
```

⟨clockinit() *if many processors* 416d⟩≡ (195a)

```
if(((cprdsc(0, CpID, CpIDfeat, 1) >> 16) & 0xF) != 0) {
    /* generic timer supported */
    if(cpu->cpuno == 0){
        *(ulong*)(ARMLOCAL + Localctl) = 0;             /* magic */
        *(ulong*)(ARMLOCAL + Prescaler) = 0x06aaaaab;   /* magic for 1 Mhz */
    }
    cpwrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysctl, Imask);
}
```

⟨clockinit() *if not cpu0* 416e⟩≡ (195a)

```
else
    arch_intrenable(IRQcntpns, localclockintr, nil, 0, "clock");
```

⟨*function* localclockintr*(arm)* 416f⟩≡ (639b)

```
static void
localclockintr(Ureg *ureg, void *)
{
    if(cpu->cpuno == 0)
        panic("cpu0: Unexpected local generic timer interrupt");
    cpwrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysctl, Imask|Enable);
    timerintr(ureg, 0);
}
```

⟨*function* arch_lcycles*(raspberry pi2)(arm)* 417a⟩≡ (640c)

```
TEXT arch_lcycles(SB), 1, $-4
    MRC CpSC, 0, R0, C(CpCLD), C(CpCLDcyc), 0
    RET
```

⟨arch_timerset() *if not cpu0(arm)* 417b⟩≡ (170e)

```
if(cpu->cpuno > 0){
    cpwrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysval, period);
    cpwrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysctl, Enable);
}
```

22.5.5 arch_coherence() (ARM)

⟨*function* arm_arch_coherence*(arm)* 417c⟩≡ (616)

```
TEXT arm_arch_coherence(SB), $-4
    BARRIERS
    RET
```

22.5.6 Cache adjustments

⟨*constant CACHELINESZ(raspberry pi2)(ram)* 417d⟩≡ (636a)

```
#define CACHELINESZ 64
```

⟨*instruction CPSID(arm)* 417e⟩≡ (608)

```
#define CPSID   WORD    $0xf10c0080 /* intr disable: sets I bit */
```

⟨*type CpID CpIDid op1(arm)* 417f⟩≡ (607c)

```
/* CpIDid op1 values */
#define CpIDcsize   1           /* cache size (cortex) */
#define CpIDcssel   2           /* cache size select (cortex) */
```

⟨*function cachexxx(raspberry pi2)(ram)* 417g⟩≡ (636a)

```
/*
 * 'single-element' cache operations.
 * in arm arch v7, they operate on all cache levels, so separate
 * l2 functions are unnecessary.
 */

TEXT cachedwbse(SB), $-4            /* D writeback SE */
    MOVW    R0, R2

    MOVW    CPSR, R3
    CPSID                   /* splhi */

    BARRIERS            /* force outstanding stores to cache */
    MOVW    R2, R0
    MOVW    4(FP), R1
    ADD R0, R1              /* R1 is end address */
    BIC $(CACHELINESZ-1), R0        /* cache line start */
_dwbse:
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEse
    /* can't have a BARRIER here since it zeroes R0 */
    ADD $CACHELINESZ, R0
    CMP.S   R0, R1
    BGT _dwbse
    B   _wait

TEXT cachedwbinvse(SB), $-4         /* D writeback+invalidate SE */
```

```
    MOVW    R0, R2

    MOVW    CPSR, R3
    CPSID                   /* splhi */

    BARRIERS            /* force outstanding stores to cache */
    MOVW    R2, R0
    MOVW    4(FP), R1
    ADD R0, R1              /* R1 is end address */
    BIC $(CACHELINESZ-1), R0        /* cache line start */
_dwbinvse:
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEse
    /* can't have a BARRIER here since it zeroes R0 */
    ADD $CACHELINESZ, R0
    CMP.S   R0, R1
    BGT _dwbinvse
_wait:                      /* drain write buffer */
    BARRIERS

    MOVW    R3, CPSR            /* splx */
    RET

TEXT cachedinvse(SB), $-4           /* D invalidate SE */
    MOVW    R0, R2

    MOVW    CPSR, R3
    CPSID                   /* splhi */

    BARRIERS            /* force outstanding stores to cache */
    MOVW    R2, R0
    MOVW    4(FP), R1
    ADD R0, R1              /* R1 is end address */
    BIC $(CACHELINESZ-1), R0        /* cache line start */
_dinvse:
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvd), CpCACHEse
    /* can't have a BARRIER here since it zeroes R0 */
    ADD $CACHELINESZ, R0
    CMP.S   R0, R1
    BGT _dinvse
    B   _wait

/*
 * cortex arm arch v7 cache flushing and invalidation
 * shared by l.s and rebootcode.s
 */

TEXT cacheiinv(SB), $-4             /* I invalidate */
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall /* ok on cortex */
    ISB
    RET

/*
 * set/way operators, passed a suitable set/way value in R0.
 */
TEXT cachedwb_sw(SB), $-4
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEsi
    RET

TEXT cachedwbinv_sw(SB), $-4
```

```
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEsi
    RET

TEXT cachedinv_sw(SB), $-4
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvd), CpCACHEsi
    RET

    /* set cache size select */
TEXT setcachelvl(SB), $-4
    MCR CpSC, CpIDcssel, R0, C(CpID), C(CpIDidct), 0
    ISB
    RET

    /* return cache sizes */
TEXT getwayssets(SB), $-4
    MRC CpSC, CpIDcsize, R0, C(CpID), C(CpIDidct), 0
    RET

/*
 * l1 cache operations.
 * l1 and l2 ops are intended to be called from C, thus need save no
 * caller's regs, only those we need to preserve across calls.
 */

TEXT cachedwb(SB), $-4
    MOVW.W  R14, -8(R13)
    MOVW    $cachedwb_sw(SB), R0
    MOVW    $1, R8
    BL  wholecache(SB)
    MOVW.P  8(R13), R15

TEXT cachedwbinv(SB), $-4
    MOVW.W  R14, -8(R13)
    MOVW    $cachedwbinv_sw(SB), R0
    MOVW    $1, R8
    BL  wholecache(SB)
    MOVW.P  8(R13), R15

TEXT cachedinv(SB), $-4
    MOVW.W  R14, -8(R13)
    MOVW    $cachedinv_sw(SB), R0
    MOVW    $1, R8
    BL  wholecache(SB)
    MOVW.P  8(R13), R15

TEXT cacheuwbinv(SB), $-4
    MOVM.DB.W [R14], (R13)  /* save lr on stack */
    MOVW    CPSR, R1
    CPSID           /* splhi */

    MOVM.DB.W [R1], (R13)   /* save R1 on stack */

    BL  cachedwbinv(SB)
    BL  cacheiinv(SB)

    MOVM.IA.W (R13), [R1]   /* restore R1 (saved CPSR) */
    MOVW    R1, CPSR
    MOVM.IA.W (R13), [R14]  /* restore lr */
    RET
```

```
/*
 * l2 cache operations
 */

TEXT l2cacheuwb(SB), $-4
    MOVW.W  R14, -8(R13)
    MOVW    $cachedwb_sw(SB), R0
    MOVW    $2, R8
    BL  wholecache(SB)
    MOVW.P  8(R13), R15

TEXT l2cacheuwbinv(SB), $-4
    MOVW.W  R14, -8(R13)
    MOVW    CPSR, R1
    CPSID           /* splhi */

    MOVM.DB.W [R1], (R13)   /* save R1 on stack */

    MOVW    $cachedwbinv_sw(SB), R0
    MOVW    $2, R8
    BL  wholecache(SB)
    BL  l2cacheuinv(SB)

    MOVM.IA.W (R13), [R1]   /* restore R1 (saved CPSR) */
    MOVW    R1, CPSR
    MOVW.P  8(R13), R15

TEXT l2cacheuinv(SB), $-4
    MOVW.W  R14, -8(R13)
    MOVW    $cachedinv_sw(SB), R0
    MOVW    $2, R8
    BL  wholecache(SB)
    MOVW.P  8(R13), R15

/*
 * these shift values are for the Cortex-A8 L1 cache (A=2, L=6) and
 * the Cortex-A8 L2 cache (A=3, L=6).
 * A = log2(# of ways), L = log2(bytes per cache line).
 * see armv7 arch ref p. 1403.
 */
#define L1WAYSH 30
#define L1SETSH 6
#define L2WAYSH 29
#define L2SETSH 6

/*
 * callers are assumed to be the above l1 and l2 ops.
 * R0 is the function to call in the innermost loop.
 * R8 is the cache level (one-origin: 1 or 2).
 *
 * initial translation by 5c, then massaged by hand.
 */
TEXT wholecache+0(SB), $-4
    MOVW    R0, R1      /* save argument for inner loop in R1 */
    SUB $1, R8      /* convert cache level to zero origin */

    /* we may not have the MMU on yet, so map R1 to PC's space */
    BIC $KSEGM, R1  /* strip segment from address */
    MOVW    PC, R2      /* get PC's segment ... */
    AND $KSEGM, R2
```

```
    ORR R2, R1      /* combine them */

    /* drain write buffers */
    BARRIERS
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEwait
    ISB

    MOVW    CPSR, R2
    MOVM.DB.W [R2,R14], (SP) /* save regs on stack */
    CPSID           /* splhi to make entire op atomic */

    /* get cache sizes */
    SLL $1, R8, R0  /* R0 = (cache - 1) << 1 */
    MCR CpSC, CpIDcssel, R0, C(CpID), C(CpIDidct), 0 /* set cache size select */
    ISB
    MRC CpSC, CpIDcsize, R0, C(CpID), C(CpIDidct), 0 /* get cache sizes */

    /* compute # of ways and sets for this cache level */
    SRA $3, R0, R5  /* R5 (ways) = R0 >> 3 */
    AND $1023, R5   /* R5 = (R0 >> 3) & MASK(10) */
    ADD $1, R5      /* R5 (ways) = ((R0 >> 3) & MASK(10)) + 1 */

    SRA $13, R0, R2 /* R2 = R0 >> 13 */
    AND $32767, R2  /* R2 = (R0 >> 13) & MASK(15) */
    ADD $1, R2      /* R2 (sets) = ((R0 >> 13) & MASK(15)) + 1 */

    /* precompute set/way shifts for inner loop */
    CMP $0, R8      /* cache == 1? */
    MOVW.EQ $L1WAYSH, R3    /* yes */
    MOVW.EQ $L1SETSH, R4
    MOVW.NE $L2WAYSH, R3    /* no */
    MOVW.NE $L2SETSH, R4

    /* iterate over ways */
    MOVW    $0, R7      /* R7: way */
outer:
    /* iterate over sets */
    MOVW    $0, R6      /* R6: set */
inner:
    /* compute set/way register contents */
    SLL R3, R7, R0  /* R0 = way << R3 (L?WAYSH) */
    ORR R8<<1, R0   /* R0 = way << L?WAYSH | (cache - 1) << 1 */
    ORR R6<<R4, R0  /* R0 = way<<L?WAYSH | (cache-1)<<1 |set<<R4 */

    BL  (R1)        /* call set/way operation with R0 */

    ADD $1, R6      /* set++ */
    CMP R2, R6      /* set >= sets? */
    BLT inner       /* no, do next set */

    ADD $1, R7      /* way++ */
    CMP R5, R7      /* way >= ways? */
    BLT outer       /* no, do next way */

    MOVM.IA.W (SP), [R2,R14] /* restore regs */
    MOVW    R2, CPSR    /* splx */

    /* drain write buffers */
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEwait
    ISB
```

```
    RET
```

22.5.7 IRQ adjustments

⟨IRQ *local cases(arm)* 422a⟩≡ (110b)

```
IRQlocal    = 96,
IRQcntps    = IRQlocal + 0,
IRQcntpns   = IRQlocal + 1,
```

⟨Vctl *other fields(arm)* 422b⟩≡ (110c)

```
int cpu;
```

⟨irqenable() *if IRQlocal(arm)* 422c⟩≡ (111d)

```
if(irq >= IRQlocal){
    enable = (u32int*)(LOCALREGS + Localtimerint) + cpu->cpuno;
    v->reg = (u32int*)(LOCALREGS + Localintpending) + cpu->cpuno;
    v->mask = 1 << (irq - IRQlocal);
    v->cpu = cpu->cpuno;
}
```

⟨irqenable() *if IRQlocal adjust enable(arm)* 422d⟩≡ (111d)

```
if(irq >= IRQlocal)
    *enable |= 1 << (irq - IRQlocal);
```

⟨*constant* Localtimerint*(arm)* 422e⟩≡ (636b)

```
Localtimerint   = 0x40,
```

⟨*constant* Localintpending*(arm)* 422f⟩≡ (636b)

```
Localintpending = 0x60,
```

⟨*constant* Localmboxint*(arm)* 422g⟩≡ (636b)

```
Localmboxint    = 0x50,
```

⟨*constant* LOCALREGS*(arm)* 422h⟩≡ (636c)

```
#define LOCALREGS   (VIRTIO+IOSIZE)
```

22.5.8 Cycle counter adjustments

⟨_startpg() *enable cycle counter(raspberry pi2)(arm)* 422i⟩≡ (409f)

```
/*
 * enable cycle counter
 */
MOVW    $(1<<31), R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenacyc
MOVW    $1, R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenapmnc
```

⟨_startpg2() *enable cycle counter(raspberry pi2)(arm)* 422j⟩≡ (413a)

```
/*
 * enable cycle counter
 */
MOVW    $(1<<31), R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenacyc
MOVW    $1, R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenapmnc
```

22.6 Advanced virtual memory

22.6.1 Page coloring

⟨Page *other fields* 423a⟩+≡ (57d) ◁142d 423g▷
```
// int [0..NCOLOR[
char color;   /* Cache coloring */
```

⟨pageinit() *other locals* 423b⟩≡ (198a)
```
int color = 0;
```

⟨pageinit() *set page color* 423c⟩≡ (198a)
```
p->color = color;
color = (color+1)%NCOLOR;
```

⟨*macro* getpgcolor*(arm)* 423d⟩≡ (619d)
```
#define getpgcolor(a)   0
```

⟨*constant* NCOLOR*(arm)* 423e⟩≡ (633a)
```
#define NCOLOR  1       /* 1 level cache, don't worry about VCE's */
```

⟨*enum* Cachectl 423f⟩≡ (582)
```
enum Cachectl {
    PG_NOFLUSH = 0,
    PG_TXTFLUSH = 1,  /* flush dcache and invalidate icache */

    PG_NEWCOL = 3,  /* page has been recolored */
};
```

⟨Page *other fields* 423g⟩+≡ (57d) ◁423a
```
// array<enum<Cachectl>>
char cachectl[MAXCPUS]; /* Cache flushing control for putmmu */
```

⟨newpage() *other locals* 423h⟩+≡ (58f) ◁138a 423j▷
```
int color;
// enum<Cachectl>
uchar ct;
```

⟨newpage() *find free page p, set color and ct* 423i⟩≡ (58f)
```
color = getpgcolor(va);

/* First try for our colour */
for(p = palloc.head; p; p = p->next)
     if(p->color == color)
         break;

ct = PG_NOFLUSH;
if(p == nil) {
    p = palloc.head;
    p->color = color;
    ct = PG_NEWCOL;
}
```

⟨newpage() *other locals* 423j⟩+≡ (58f) ◁423h
```
int i;
```

⟨newpage() *set cachectl using ct* 423k⟩≡ (58f)
```
for(i = 0; i < MAXCPUS; i++)
    p->cachectl[i] = ct;
```

⟨arch_putmmu() *if* PG_TXTFLUSH, *invalidate cache(arm)* 424a⟩≡ (119)

```
if(page->cachectl[cpu->cpuno] == PG_TXTFLUSH){
    /* pio() sets PG_TXTFLUSH whenever a text pg has been written */
    cacheiinv();
    page->cachectl[cpu->cpuno] = PG_NOFLUSH;
}
```

⟨arch__userinit() *set cachectl(arm)* 424b⟩≡ (201c)

```
memset(pg->cachectl, PG_TXTFLUSH, sizeof(pg->cachectl));
```

⟨Segment *other fields* 424c⟩+≡ (61c) ◁381h 425a▷

```
bool flushme; /* maintain icache for this segment */
```

⟨sysexec() *initialize other fields* 424d⟩≡ (126d)

```
ts->flushme = true;
```

⟨arch__userinit() *set flushme for stack segment* 424e⟩≡ (201c)

```
s->flushme = true;
```

⟨data2txt() *initialize other segment fields* 424f⟩≡ (127b)

```
ps->flushme = true;
```

⟨text2data() *initialize other fields* 424g⟩≡ (368)

```
ps->flushme = true;
```

⟨dupseg() *copy other fields* 424h⟩≡ (68e)

```
n->flushme = s->flushme;
```

⟨syssegflush() *set flushme* 424i⟩≡ (354c)

```
s->flushme = true;
```

⟨pio() *set cachectl if flushme segment* 424j⟩≡ (127f)

```
if(s->flushme)
    memset((*p)->cachectl, PG_TXTFLUSH, sizeof((*p)->cachectl));
```

⟨procctlmemio() *set cachectl if flushme segment* 424k⟩≡ (367e)

```
/* Ensure the process sees text page changes */
if(s->flushme)
    memset(pg->cachectl, PG_TXTFLUSH, sizeof(pg->cachectl));
```

22.6.2 Extra segment types

⟨Segtype *other cases* 424l⟩≡ (61a) 424m▷

```
SG_SHARED = 04,
```

⟨Segtype *other cases* 424m⟩+≡ (61a) ◁424l

```
SG_PHYSICAL = 05,
```

⟨dupseg() *switch segment type cases* 424n⟩+≡ (67d) ◁68g

```
case SG_SHARED:
case SG_PHYSICAL:
    goto sameseg;
```

⟨*struct* Physseg 424o⟩≡ (582)

```
struct Physseg
{
    ulong attr;     /* Segment attributes */
    char  *name;      /* Attach name */
    phys_addr pa;     /* Physical address */
    ulong size;     /* Maximum segment size in pages */
};
```

⟨Segment *other fields* 425a⟩+≡ (61c) ◁424c 427b▷

```
Physseg *pseg;
```

⟨*global* physseg 425b⟩≡ (589)

```
/*
 * Attachable segment types
 */
static Physseg physseg[10] = {
    { .attr= SG_SHARED,
      .name= "shared",
      .pa= 0,
      .size= SEGMAXSIZE,
    },
    { .attr= SG_BSS,
      .name= "memory",
      .pa = 0,
      .size = SEGMAXSIZE,
    },
    { 0, 0, 0, 0},
};
```

⟨*global* physseglock 425c⟩≡ (589)

```
static Lock physseglock;
```

⟨*function* addphysseg 425d⟩≡ (589)

```
int
addphysseg(Physseg* new)
{
    Physseg *ps;

    /*
     * Check not already entered and there is room
     * for a new entry and the terminating null entry.
     */
    lock(&physseglock);
    for(ps = physseg; ps->name; ps++){
        if(strcmp(ps->name, new->name) == 0){
            unlock(&physseglock);
            return -1;
        }
    }
    if(ps-physseg >= nelem(physseg)-2){
        unlock(&physseglock);
        return -1;
    }

    *ps = *new;
    unlock(&physseglock);

    return 0;
}
```

⟨freept() *other locals* 425e⟩≡ (62f)

```
int ref;
Page *pt, **ptop;
```

⟨freept() SG_PHYSICAL *case* 425f⟩≡ (62f)

```
case SG_PHYSICAL:
    ptop = &p->pagetab[PAGETABSIZE];
    for(pte = p->pagetab; pte < ptop; pte++) {
```

```
        pt = *pte;
        if(pt == nil)
            continue;
        lock(pt);
        ref = --pt->ref;
        unlock(pt);
        if(ref == 0)
            free(pt); // because was smalloc'ed in fixfault
    }
    break;
```

⟨fixfault() SG_PHYSICAL *case* 426a⟩≡ (117c)

```
  case SG_PHYSICAL:
    if(*pte == nil) {
        new = smalloc(sizeof(Page));
        new->va = addr;
        new->pa = s->pseg->pa+(addr-s->base);
        new->ref = 1;
        *pte = new;
    }

    if (checkaddr && addr == addr2check)
        (*checkaddr)(addr, s, *pte);
    mmupte = PPN((*pte)->pa) |PTEWRITE|PTEUNCACHED|PTEVALID;
    (*pte)->modref = PG_MOD|PG_REF;
    break;
```

⟨PTExxx *other cases* 426b⟩≡ (117b)

```
  #define PTEUNCACHED (1<<4)
```

⟨arch_putmmu() *if PTEUNCACHED, do not change x(arm)* 426c⟩≡ (119)

```
  if(pa & PTEUNCACHED) {
  }
```

22.6.3 arch_kmap()

22.6.4 Extra checks

⟨*function* checkpagerefs 426d⟩≡ (588)

```
  void
  checkpagerefs(void)
  {
      int s;
      ulong i, np, nwrong;
      ulong *ref;

      np = palloc.user;
      ref = malloc(np*sizeof ref[0]);
      if(ref == nil){
          print("checkpagerefs: out of memory\n");
          return;
      }

      /*
       * This may not be exact if there are other processes
       * holding refs to pages on their stacks.  The hope is
       * that if you run it on a quiescent system it will still
       * be useful.
       */
```

```
    s = arch_splhi();
    lock(&palloc);
    arch_countpagerefs(ref, 0);
    portcountpagerefs(ref, 0);
    nwrong = 0;
    for(i=0; i<np; i++){
        if(palloc.pages[i].ref != ref[i]){
            iprint("page %#.8lux ref %d actual %lud\n",
                palloc.pages[i].pa, palloc.pages[i].ref, ref[i]);
            ref[i] = 1;
            nwrong++;
        }else
            ref[i] = 0;
    }
    arch_countpagerefs(ref, 1);
    portcountpagerefs(ref, 1);
    iprint("%lud mistakes found\n", nwrong);
    unlock(&palloc);
    arch_splx(s);
}
```

⟨*macro* arch_countpagerefs*(arm)* 427a⟩≡ (619d)

```
// same signatures but optimized away
#define arch_countpagerefs(a, b)
```

⟨Segment *other fields* 427b⟩+≡ (61c) ◁425a

```
ulong mark;   /* portcountrefs */
```

⟨*function* portcountpagerefs 427c⟩≡ (588)

```
void
portcountpagerefs(ulong *ref, int print)
{
    ulong i, j, k, ns, n;
    Page **pg, *entry;
    Proc *p;
    Pagetable *pt;
    Segment *s;

    /*
     * Pages in segments.  s->mark avoids double-counting.
     */
    n = 0;
    ns = 0;
    for(i=0; i<conf.nproc; i++){
        p = proctab(i);
        for(j=0; j<NSEG; j++){
            s = p->seg[j];
            if(s)
                s->mark = 0;
        }
    }
    for(i=0; i<conf.nproc; i++){
        p = proctab(i);
        for(j=0; j<NSEG; j++){
            s = p->seg[j];
            if(s == nil || s->mark++)
                continue;
            ns++;
            for(k=0; k<s->pagedirsize; k++){
                pt = s->pagedir[k];
```

```
                if(pt == nil)
                    continue;
                for(pg = pt->first; pg <= pt->last; pg++){
                    entry = *pg;
                    if(pagedout(entry))
                        continue;
                    if(print){
                        if(ref[pagenumber(entry)])
                            iprint("page %#.8lux in segment %#p\n", entry->pa, s);
                        continue;
                    }
                    if(ref[pagenumber(entry)]++ == 0)
                        n++;
                }
            }
        }
    }
    if(!print){
        iprint("%lud pages in %lud segments\n", n, ns);
        for(i=0; i<conf.nproc; i++){
            p = proctab(i);
            for(j=0; j<NSEG; j++){
                s = p->seg[j];
                if(s == nil)
                    continue;
                if(s->ref != s->mark){
                    iprint("segment %#p (used by proc %lud pid %lud) has bad ref count %lud actual %lud\n",
                        s, i, p->pid, s->ref, s->mark);
                }
            }
        }
    }
}
```

⟨*function* pagenumber 428a⟩≡ (588)

```
ulong
pagenumber(Page *p)
{
    return p - palloc.pages;
}
```

22.7 Fast interrupts (FIQ) (ARM)

22.7.1 Exception PsrMfiq and associated processor mode

⟨PsrMode *other cases(arm)* 428b⟩+≡ (21a) ◁99a

```
#define PsrMfiq     0x00000011
```

⟨trapnames() *other entries(arm)* 428c⟩≡ (99b)

```
[ PsrMfiq ] "fiq interrupt",
```

⟨PsrDisable *other cases(arm)* 428d⟩≡ (21b)

```
#define PsrDfiq     0x00000040      /* disable FIQ interrupts */
```

⟨*constant* FIQSTKTOP*(arm)* 428e⟩≡ (633b)

```
#define FIQSTKTOP   (KZERO+0x4000)      /* FIQ stack */
```

⟨arch__trapinit() *set stack for other exception/processor-modes* 428f⟩+≡ (194a) ◁194c

```
setr13(PsrMfiq, (u32int*)(FIQSTKTOP));
```

⟨*function* _vfiq*(arm)* 429a⟩≡ (638a)

```
TEXT _vfiq(SB), 1, $-4          /* FIQ */
    CLREX
    MOVW    $PsrMfiq, R8        /* trap type */
    MOVW    SPSR, R9        /* interrupted psr */
    MOVW    R14, R10        /* interrupted pc */
    MOVM.DB.W [R8-R10], (R13)   /* save in ureg */
    MOVM.DB.S [R0-R14], (R13)   /* save interrupted regs */
    SUB $(15*4), R13
    MOVW    $setR12(SB), R12    /* Make sure we've got the kernel's SB loaded */

    /* get R(CPU) for this cpu */
    CPUID(R1)
    SLL $2, R1          /* convert to word index */
    MOVW    $cpus(SB), R2
    ADD R1, R2
    MOVW    (R2), R(CPU)        /* m = cpus[cpuid] */
    CMP $0, R(CPU)
    MOVW.EQ $CPUADDR, R(CPU)        /* paranoia: use CPUADDR if 0 */

    MOVW    8(R(CPU)), R(UP)        /* up */
    MOVW    R13, R0         /* first arg is pointer to ureg */
    SUB $(4*2), R13     /* space for argument+link (for debugger) */

    BL  fiq(SB)

    ADD $(8+4*15), R13      /* make r13 point to ureg->type */
    MOVW    8(R13), R14     /* restore link */
    MOVW    4(R13), R0      /* restore SPSR */
    MOVW    R0, SPSR        /* ... */
    MOVM.DB.S (R13), [R0-R14]   /* restore registers */
    ADD $8, R13         /* pop past ureg->{type+psr} */
    RFE             /* MOVM.IA.S.W (R13), [R15] */
```

22.7.2 IRQfiq and vfiq

⟨IRQ *other cases(arm)* 429b⟩≡ (110b)

```
IRQfiq      = IRQusb,   /* only one source can be FIQ */
```

⟨irqenable() *if IRQfiq(arm)* 429c⟩≡ (111d)

```
if(irq == IRQfiq){
    ⟨irqenable() sanity check no previous FIQ(arm) 429f⟩
    vfiq = v;
    ip->FIQctl = Fiqenable | irq;
}
```

⟨*global* vfiq*(arm)* 429d⟩≡ (636c)

```
static Vctl *vfiq;
```

⟨*constant* Fiqenable*(arm)* 429e⟩≡ (636b)

```
Fiqenable = 1<<7,
```

⟨irqenable() *sanity check no previous FIQ(arm)* 429f⟩≡ (429c)

```
assert((ip->FIQctl & Fiqenable) == 0);
assert((*enable & v->mask) == 0);
```

22.7.3 fiq()

⟨*function* fiq*(arm)* 430a⟩≡ (636c)

```
/*
 * called direct from lexception.s to handle fiq interrupt.
 */
void
fiq(Ureg *ureg)
{
    Vctl *v;

    cpu->perf.intrts = arch_perfticks();
    v = vfiq;
    if(v == nil)
        panic("cpu%d: unexpected item in bagging area", cpu->cpuno);
    cpu->intr++;
    ureg->pc -= 4;
    arch_coherence();

    // Dispatch!
    v->f(ureg, v->a);

    arch_coherence();
    intrtime();
}
```

⟨*function* splfhi*(arm)* 430b⟩≡ (616)

```
TEXT splfhi(SB), 1, $-4
    MOVW    $(CPUADDR+4), R2        /* save caller pc in Mach */
    MOVW    R14, 0(R2)

    MOVW    CPSR, R0            /* turn off irqs and fiqs */
    ORR $(PsrDirq|PsrDfiq), R0, R1
    MOVW    R1, CPSR
    RET
```

22.8 Power management (ARM)

⟨*enum* Power*(arm)* 430c⟩≡ (609a)

```
enum {
    PowerSd     = 0,
    PowerUart0,
    PowerUart1,
    PowerUsb,
    PowerI2c0,
    PowerI2c1,
    PowerI2c2,
    PowerSpi,
    PowerCcp2tx,
};
```

⟨*function* setpower*(arm)* 430d⟩≡ (615d)

```
/*
 * Power management
 */
void
setpower(int dev, bool on)
{
    u32int buf[2];
```

```
    buf[0] = dev;
    buf[1] = Powerwait | on;
    vcreq(TagSetpower, buf, sizeof buf, sizeof buf);
}
```

⟨*function* getpower*(arm)* 431a⟩≡ (615d)

```
int
getpower(int dev)
{
    u32int buf[2];

    buf[0] = dev;
    buf[1] = 0;
    if(vcreq(TagGetpower, buf, sizeof buf[0], sizeof buf) != sizeof buf)
        return ERROR_NEG1;
    return buf[0] & 1;
}
```

22.9 Alignements (ARM)

⟨*function* arch_validalign*(arm)* 431b⟩≡ (641a)

```
/*
 * called in syscallfmt.c, sysfile.c, sysproc.c
 */
void
arch_validalign(uintptr addr, unsigned align)
{
    /*
     * Plan 9 is a 32-bit O/S, and the hardware it runs on
     * does not usually have instructions which move 64-bit
     * quantities directly, synthesizing the operations
     * with 32-bit move instructions. Therefore, the compiler
     * (and hardware) usually only enforce 32-bit alignment,
     * if at all.
     *
     * Take this out if the architecture warrants it.
     */
    if(align == sizeof(vlong))
        align = sizeof(long);

    /*
     * Check align is a power of 2, then addr alignment.
     */
    if((align != 0 && !(align & (align-1))) && !(addr & (align-1)))
        return;
    postnote(up, 1, "sys: odd address", NDebug);
    error(Ebadarg);
    /*NOTREACHED*/
}
```

22.10 Real-time scheduling

⟨Priority *cases for real-time priority* 431c⟩≡ (145b)

```
PriRelease  = Npriq,  /* released edf processes */
PriEdf    = Npriq+1,  /* active edf processes */
PriExtra  = Npriq-1,  /* edf processes at high best-effort pri */
```

⟨runproc() *test for empty real-time scheduling queue* 432a⟩≡ (156c)

```
runq[Nrq-1].head == nil && runq[Nrq-2].head == nil
```

⟨runproc() *test if p is a real-time process* 432b⟩≡ (150)

```
if(edflock(p)){
    edfrun(p, rq == &runq[PriEdf]); /* start deadline timer and do admin */
    edfunlock();
}
```

⟨newproc() *set fields of* p 432c⟩+≡ (75b) ◁387c 465a▷

```
p->edf = nil;
```

⟨updatecpu() *return if real-time process* 432d⟩≡ (161j)

```
if(p->edf)
    return;
```

⟨schedinit() *locals* 432e⟩≡ (149g)

```
Edf *e;
```

⟨schedinit() *optional real-time* edfrecord() 432f⟩≡ (152b)

```
if((e = up->edf) && (e->flags & Admitted))
    edfrecord(up);
```

⟨schedinit() *optional real-time* edfstop() 432g⟩≡ (93c)

```
edfstop(up);
if (up->edf)
    free(up->edf);
up->edf = nil;
```

⟨ready() *optional* edfready() *for real-time scheduling* 432h⟩≡ (155c)

```
if(edfready(p)){
    arch_splx(s);
    return;
}
```

⟨syssleep() *optional* edfyield() *for real-time scheduling* 432i⟩≡ (174a)

```
if (up->edf && (up->edf->flags & Admitted))
    edfyield();
else
```

⟨procctlreq() *optional real-time commands* 432j⟩≡ (482)

```
/* real time */
case CMperiod:
    if(p->edf == nil)
        edfinit(p);
    if(e=parsetime(&time, cb->f[1]))    /* time in ns */
        error(e);
    edfstop(p);
    p->edf->T = time/1000;  /* Edf times are in microseconds */
    break;
case CMdeadline:
    if(p->edf == nil)
        edfinit(p);
    if(e=parsetime(&time, cb->f[1]))
        error(e);
    edfstop(p);
    p->edf->D = time/1000;
    break;
case CMcost:
    if(p->edf == nil)
```

```
        edfinit(p);
    if(e=parsetime(&time, cb->f[1]))
        error(e);
    edfstop(p);
    p->edf->C = time/1000;
    break;
case CMsporadic:
    if(p->edf == nil)
        edfinit(p);
    p->edf->flags |= Sporadic;
    break;
case CMdeadlinenotes:
    if(p->edf == nil)
        edfinit(p);
    p->edf->flags |= Sendnotes;
    break;
case CMadmit:
    if(p->edf == 0)
        error("edf params");
    if(e = edfadmit(p))
        error(e);
    break;
case CMextra:
    if(p->edf == nil)
        edfinit(p);
    p->edf->flags |= Extratime;
    break;
case CMexpel:
    if(p->edf)
        edfstop(p);
    break;
```

⟨sched() *if real-time process then do nothing, do not reprioritize* 433a⟩≡ (161b)

```
if(p->edf) {
}
```

⟨pexit() *optional* edfstop() *for real-time scheduling* 433b⟩≡ (92d)

```
edfstop(up);
```

⟨Proc *scheduling fields* 433c⟩+≡ (29c) ◁161e 433d▷

⟨Proc *optional* edf *field for real-time scheduling* 433e⟩

⟨Proc *scheduling fields* 433d⟩+≡ (29c) ◁433c 599a▷

```
ulong readytime;  /* time process came ready */
```

⟨Proc *optional* edf *field for real-time scheduling* 433e⟩≡ (433c)

```
// option<ref_own?<edf>>
Edf *edf; /* if non-null, real-time proc, edf contains scheduling params */
```

⟨lock() *optional priority-inversion for real-time process* 433f⟩≡ (35e)

```
if(conf.ncpu < 2 && up && up->edf && (up->edf->flags & Admitted)){
    /*
     * Priority inversion, yield on a uniprocessor; on a
     * multiprocessor, the other processor will unlock
     */
    print("inversion %#p pc %#lux proc %lud held by pc %#lux proc %lud\n",
        l, pc, up ? up->pid : 0, l->pc, l->p ? l->p->pid : 0);
    up->edf->d = todget(nil);   /* yield to process with lock */
}
```

⟨*enum* edfflags 434a⟩≡ (592c)

```
enum EdfFlags
{
    /* Edf.flags field */
    Admitted    = 0x01,
    Sporadic    = 0x02,
    Yieldonblock    = 0x04,
    Sendnotes   = 0x08,
    Deadline    = 0x10,
    Yield     = 0x20,
    Extratime   = 0x40,
};
```

⟨*struct* Edf 434b⟩≡ (592c)

```
struct Edf {
    /* All times in µs */
    /* time intervals */
    long    D;    /* Deadline */
    long    Delta;    /* Inherited deadline */
    long    T;    /* period */
    long    C;    /* Cost */
    long    S;    /* Slice: time remaining in this period */
    /* times (only low-order bits of absolute time) */
    long    r;    /* (this) release time */
    long    d;    /* (this) deadline */
    long    t;    /* Start of next period, t += T at release */
    long    s;    /* Time at which this proc was last scheduled */

    /* for schedulability testing */
    long    testDelta;
    int   testtype; /* Release or Deadline */
    long    testtime;
    Proc    *testnext;

    /* other */
    // set<enum<edfflags>>
    ushort    flags;

    Timer;

    /* Stats */
    long    edfused;
    long    extraused;
    long    aged;
    ulong   periods;
    ulong   missed;
};
```

processes/edf.c

⟨*edf.c statistics* 434c⟩≡ (435)

```
/* Statistics stuff */
ulong       edfnrun;
//ulong     nilcount;
//ulong     scheds;
//int       misseddeadlines;
```

⟨Procstate *cases* 434d⟩+≡ (30c) ◁364b

```
Waitrelease, // for real-time scheduling
```

⟨edf.c 435⟩≡

```
/* EDF scheduling */
⟨kernel basic includes 23m⟩

#include    <trace.h>

// in proc.c
extern Schedq   runq[Nrq];
extern ulong    delayedscheds;
extern int  nrdy;
extern ulong    runvec;
// in devcons.c
extern int panicking;
// used to be in edf.h
//unused: extern Lock edftestlock;  /* for atomic admitting/expelling */

⟨edf.c debugging macro ??⟩

enum
{
    Maxsteps = 200 * 100 * 2, /* 100 periods of 200 procs */
};


static long now;    /* Low order 32 bits of time in µs */


⟨edf.c statistics 434c⟩

int     edfinited;

/* Edfschedlock protects modification of admission params */
QLock       edfschedlock;
static Lock thelock;

enum{
    Dl, /* Invariant for schedulability test: Dl < Rl */
    Rl,
};

static char *testschedulability(Proc*);
static Proc *qschedulability;

enum {
    Onemicrosecond =    1,
    Onemillisecond =    1000,
    Onesecond =     1000000,
    OneRound =      Onemillisecond/2,
};

static int
timeconv(Fmt *f)
{
    char buf[128], *sign;
    vlong t;

    buf[0] = 0;
    switch(f->r) {
    case 'U':
        t = va_arg(f->args, uvlong);
```

```
        break;
    case 't':           /* vlong in nanoseconds */
        t = va_arg(f->args, long);
        break;
    default:
        return fmtstrcpy(f, "(timeconv)");
    }
    if (t < 0) {
        sign = "-";
        t = -t;
    }
    else
        sign = "";
    if (t > Onesecond){
        t += OneRound;
        snprint(buf, sizeof buf, "%s%d.%.3ds", sign,
            (int)(t / Onesecond),
            (int)(t % Onesecond)/Onemillisecond);
    }else if (t > Onemillisecond)
        snprint(buf, sizeof buf, "%s%d.%.3dms", sign,
            (int)(t / Onemillisecond), (int)(t % Onemillisecond));
    else
        snprint(buf, sizeof buf, "%s%dµs", sign, (int)t);
    return fmtstrcpy(f, buf);
}

#ifdef EDFCYCLES
long edfcycles;
#endif

Edf*
edflock(Proc *p)
{
    Edf *e;

    if (p->edf == nil)
        return nil;
    ilock(&thelock);
    if((e = p->edf) && (e->flags & Admitted)){
        thelock.pc = getcallerpc(&p);
#ifdef EDFCYCLES
        edfcycles -= lcycles();
#endif
        now = arch_us();
        return e;
    }
    iunlock(&thelock);
    return nil;
}

void
edfunlock(void)
{

#ifdef EDFCYCLES
    edfcycles += lcycles();
#endif
    edfnrun++;
    iunlock(&thelock);
}
```

```
void
edfinit(Proc* p)
{
    if(!edfinited){
        fmtinstall('t', timeconv);
        edfinited++;
    }
    now = arch_us();
    DPRINT("%lud edfinit %lud[%s]\n", now, p->pid, statename[p->state]);
    p->edf = malloc(sizeof(Edf));
    if(p->edf == nil)
        error(Enomem);
    return;
}

static void
deadlineintr(Ureg*, Timer *t)
{
    /* Proc reached deadline */
    Proc *p;
    void (*pt)(Proc*, int, vlong);

    if(panicking || active.exiting)
        return;

    p = t->ta;
    now = arch_us();
    DPRINT("%lud deadlineintr %lud[%s]\n", now, p->pid, statename[p->state]);
    /* If we're interrupting something other than the proc pointed to by t->a,
     * we've already achieved recheduling, so we need not do anything
     * Otherwise, we must cause a reschedule, but if we call sched()
     * here directly, the timer interrupt routine will not finish its business
     * Instead, we cause the resched to happen when the interrupted proc
     * returns to user space
     */
    if(p == up){
        if(up->trace && (pt = proctrace))
            pt(up, SInts, 0);
        up->delaysched++;
        delayedscheds++;
    }
}

static void
release(Proc *p)
{
    /* Called with edflock held */
    Edf *e;
    void (*pt)(Proc*, int, vlong);
    long n;
    vlong nowns;

    e = p->edf;
    e->flags &= ~Yield;
    if(e->d - now < 0){
        e->periods++;
        e->r = now;
        if((e->flags & Sporadic) == 0){
            /*
```

```
             * Non sporadic processes stay true to their period;
             * calculate next release time.
             * Second test limits duration of while loop.
             */
            if((n = now - e->t) > 0){
                if(n < e->T)
                    e->t += e->T;
                else
                    e->t = now + e->T - (n % e->T);
            }
        }else{
            /* Sporadic processes may not be released earlier than
             * one period after this release
             */
            e->t = e->r + e->T;
        }
        e->d = e->r + e->D;
        e->S = e->C;
        DPRINT("%lud release %lud[%s], r=%lud, d=%lud, t=%lud, S=%lud\n",
            now, p->pid, statename[p->state], e->r, e->d, e->t, e->S);
        if(pt = proctrace){
            nowns = todget(nil);
            pt(p, SRelease, nowns);
            pt(p, SDeadline, nowns + 1000LL*e->D);
        }
    }else{
        DPRINT("%lud release %lud[%s], too late t=%lud, called from %#p\n",
            now, p->pid, statename[p->state], e->t, getcallerpc(&p));
    }
}

static void
releaseintr(Ureg*, Timer *t)
{
    Proc *p;
    Schedq *rq;

    if(panicking || active.exiting)
        return;

    p = t->ta;
    if((edflock(p)) == nil)
        return;
    DPRINT("%lud releaseintr %lud[%s]\n", now, p->pid, statename[p->state]);
    switch(p->state){
    default:
        edfunlock();
        return;
    case Ready:
        /* remove proc from current runq */
        rq = &runq[p->priority];
        if(dequeueproc(rq, p) != p){
            DPRINT("releaseintr: can't find proc or lock race\n");
            release(p); /* It'll start best effort */
            edfunlock();
            return;
        }
        p->state = Waitrelease;
        /* fall through */
    case Waitrelease:
```

```
        release(p);
        edfunlock();
        if(p->state == Wakeme){
            iprint("releaseintr: wakeme\n");
        }
        ready(p);
        if(up){
            up->delaysched++;
            delayedscheds++;
        }
        return;
    case Running:
        release(p);
        edfrun(p, 1);
        break;
    case Wakeme:
        release(p);
        edfunlock();
        if(p->trend)
            wakeup(p->trend);
        p->trend = nil;
        if(up){
            up->delaysched++;
            delayedscheds++;
        }
        return;
    }
    edfunlock();
}

void
edfrecord(Proc *p)
{
    long used;
    Edf *e;
    void (*pt)(Proc*, int, vlong);

    if((e = edflock(p)) == nil)
        return;
    used = now - e->s;
    if(e->d - now <= 0)
        e->edfused += used;
    else
        e->extraused += used;
    if(e->S > 0){
        if(e->S <= used){
            if(pt = proctrace)
                pt(p, SSlice, 0);
            DPRINT("%lud edfrecord slice used up\n", now);
            e->d = now;
            e->S = 0;
        }else
            e->S -= used;
    }
    e->s = now;
    edfunlock();
}

void
edfrun(Proc *p, int edfpri)
```

```
{
    Edf *e;
    void (*pt)(Proc*, int, vlong);
    long tns;

    e = p->edf;
    /* Called with edflock held */
    if(edfpri){
        tns = e->d - now;
        if(tns <= 0 || e->S == 0){
            /* Deadline reached or resources exhausted,
             * deschedule forthwith
             */
            p->delaysched++;
            delayedscheds++;
            e->s = now;
            return;
        }
        if(e->S < tns)
            tns = e->S;
        if(tns < 20)
            tns = 20;
        e->tns = 1000LL * tns;  /* µs to ns */
        if(e->tt == nil || e->tf != deadlineintr){
            DPRINT("%lud edfrun, deadline=%lud\n", now, tns);
        }else{
            DPRINT("v");
        }
        if(p->trace && (pt = proctrace))
            pt(p, SInte, todget(nil) + e->tns);
        e->tmode = Trelative;
        e->tf = deadlineintr;
        e->ta = p;
        timeradd(e);
    }else{
        DPRINT("<");
    }
    e->s = now;
}

char *
edfadmit(Proc *p)
{
    char *err;
    Edf *e;
    int i;
    Proc *r;
    void (*pt)(Proc*, int, vlong);
    long tns;

    e = p->edf;
    if (e->flags & Admitted)
        return "task state";    /* should never happen */

    /* simple sanity checks */
    if (e->T == 0)
        return "T not set";
    if (e->C == 0)
        return "C not set";
    if (e->D > e->T)
```

```
    return "D > T";
if (e->D == 0)  /* if D is not set, set it to T */
    e->D = e->T;
if (e->C > e->D)
    return "C > D";

qlock(&edfschedlock);
if (err = testschedulability(p)){
    qunlock(&edfschedlock);
    return err;
}
e->flags |= Admitted;

edflock(p);

if(p->trace && (pt = proctrace))
    pt(p, SAdmit, 0);

/* Look for another proc with the same period to synchronize to */
SET(r);
for(i=0; i<conf.nproc; i++) {
    r = proctab(i);
    if(r->state == Dead || r == p)
        continue;
    if (r->edf == nil || (r->edf->flags & Admitted) == 0)
        continue;
    if (r->edf->T == e->T)
            break;
}
if (i == conf.nproc){
    /* Can't synchronize to another proc, release now */
    e->t = now;
    e->d = 0;
    release(p);
    if (p == up){
        DPRINT("%lud edfadmit self %lud[%s], release now: r=%lud d=%lud t=%lud\n",
            now, p->pid, statename[p->state], e->r, e->d, e->t);
        /* We're already running */
        edfrun(p, 1);
    }else{
        /* We're releasing another proc */
        DPRINT("%lud edfadmit other %lud[%s], release now: r=%lud d=%lud t=%lud\n",
            now, p->pid, statename[p->state], e->r, e->d, e->t);
        p->ta = p;
        edfunlock();
        qunlock(&edfschedlock);
        releaseintr(nil, p);
        return nil;
    }
}else{
    /* Release in synch to something else */
    e->t = r->edf->t;
    if (p == up){
        DPRINT("%lud edfadmit self %lud[%s], release at %lud\n",
            now, p->pid, statename[p->state], e->t);
        edfunlock();
        qunlock(&edfschedlock);
        return nil;
    }else{
        DPRINT("%lud edfadmit other %lud[%s], release at %lud\n",
```

```
                now, p->pid, statename[p->state], e->t);
            if(e->tt == nil){
                e->tf = releaseintr;
                e->ta = p;
                tns = e->t - now;
                if(tns < 20)
                    tns = 20;
                e->tns = 1000LL * tns;
                e->tmode = Trelative;
                timeradd(e);
            }
        }
    }
    edfunlock();
    qunlock(&edfschedlock);
    return nil;
}

void
edfstop(Proc *p)
{
    Edf *e;
    void (*pt)(Proc*, int, vlong);

    if(e = edflock(p)){
        DPRINT("%lud edfstop %lud[%s]\n", now, p->pid, statename[p->state]);
        if(p->trace && (pt = proctrace))
            pt(p, SExpel, 0);
        e->flags &= ~Admitted;
        if(e->tt)
            timerdel(e);
        edfunlock();
    }
}

static int
yfn(void *)
{
    now = arch_us();
    return up->trend == nil || now - up->edf->r >= 0;
}

void
edfyield(void)
{
    /* sleep until next release */
    Edf *e;
    void (*pt)(Proc*, int, vlong);
    long n;

    if((e = edflock(up)) == nil)
        return;
    if(up->trace && (pt = proctrace))
        pt(up, SYield, 0);
    if((n = now - e->t) > 0){
        if(n < e->T)
            e->t += e->T;
        else
            e->t = now + e->T - (n % e->T);
    }
```

```
    e->r = e->t;
    e->flags |= Yield;
    e->d = now;
    if (up->tt == nil){
        n = e->t - now;
        if(n < 20)
            n = 20;
        up->tns = 1000LL * n;
        up->tf = releaseintr;
        up->tmode = Trelative;
        up->ta = up;
        up->trend = &up->sleepr;
        timeradd(up);
    }else if(up->tf != releaseintr)
        print("edfyield: surprise! %#p\n", up->tf);
    edfunlock();
    sleep(&up->sleepr, yfn, nil);
}

int
edfready(Proc *p)
{
    Edf *e;
    Schedq *rq;
    Proc *l, *pp;
    void (*pt)(Proc*, int, vlong);
    long n;

    if((e = edflock(p)) == nil)
        return 0;

    if(p->state == Wakeme && p->r){
        iprint("edfready: wakeme\n");
    }
    if(e->d - now <= 0){
        /* past deadline, arrange for next release */
        if((e->flags & Sporadic) == 0){
            /*
             * Non sporadic processes stay true to their period;
             * calculate next release time.
             */
            if((n = now - e->t) > 0){
                if(n < e->T)
                    e->t += e->T;
                else
                    e->t = now + e->T - (n % e->T);
            }
        }
        if(now - e->t < 0){
            /* Next release is in the future, schedule it */
            if(e->tt == nil || e->tf != releaseintr){
                n = e->t - now;
                if(n < 20)
                    n = 20;
                e->tns = 1000LL * n;
                e->tmode = Trelative;
                e->tf = releaseintr;
                e->ta = p;
                timeradd(e);
                DPRINT("%lud edfready %lud[%s], release=%lud\n",
```

```
                now, p->pid, statename[p->state], e->t);
            }
            if(p->state == Running && (e->flags & (Yield|Yieldonblock)) == 0 && (e->flags & Extratime)){
                /* If we were running, we've overrun our CPU allocation
                 * or missed the deadline, continue running best-effort at low priority
                 * Otherwise we were blocked.  If we don't yield on block, we continue
                 * best effort
                 */
                DPRINT(">");
                p->basepri = PriExtra;
                p->fixedpri = 1;
                edfunlock();
                return 0;   /* Stick on runq[PriExtra] */
            }
            DPRINT("%lud edfready %lud[%s] wait release at %lud\n",
                now, p->pid, statename[p->state], e->t);
            p->state = Waitrelease;
            edfunlock();
            return 1;   /* Make runnable later */
        }
        DPRINT("%lud edfready %lud %s release now\n", now, p->pid, statename[p->state]);
        /* release now */
        release(p);
    }
    edfunlock();
    DPRINT("^");
    rq = &runq[PriEdf];
    /* insert in queue in earliest deadline order */
    lock(runq);
    l = nil;
    for(pp = rq->head; pp; pp = pp->rnext){
        if(pp->edf->d > e->d)
            break;
        l = pp;
    }
    p->rnext = pp;
    if (l == nil)
        rq->head = p;
    else
        l->rnext = p;
    if(pp == nil)
        rq->tail = p;
    rq->n++;
    nrdy++;
    runvec |= 1 << PriEdf;
    p->priority = PriEdf;
    p->readytime = cpu->ticks;
    p->state = Ready;
    unlock(runq);
    if(p->trace && (pt = proctrace))
        pt(p, SReady, 0);
    return 1;
}


static void
testenq(Proc *p)
{
    Proc *xp, **xpp;
    Edf *e;
```

```
    e = p->edf;
    e->testnext = nil;
    if (qschedulability == nil) {
        qschedulability = p;
        return;
    }
    SET(xp);
    for (xpp = &qschedulability; *xpp; xpp = &xp->edf->testnext) {
        xp = *xpp;
        if (e->testtime - xp->edf->testtime < 0
        || (e->testtime == xp->edf->testtime && e->testtype < xp->edf->testtype)){
            e->testnext = xp;
            *xpp = p;
            return;
        }
    }
    assert(xp->edf->testnext == nil);
    xp->edf->testnext = p;
}

static char *
testschedulability(Proc *theproc)
{
    Proc *p;
    long H, G, Cb, ticks;
    int steps, i;

    /* initialize */
    DPRINT("schedulability test %lud\n", theproc->pid);
    qschedulability = nil;
    for(i=0; i<conf.nproc; i++) {
        p = proctab(i);
        if(p->state == Dead)
            continue;
        if ((p->edf == nil || (p->edf->flags & Admitted) == 0) && p != theproc)
            continue;
        p->edf->testtype = Rl;
        p->edf->testtime = 0;
        DPRINT("\tInit: edfenqueue %lud\n", p->pid);
        testenq(p);
    }
    H=0;
    G=0;
    for(steps = 0; steps < Maxsteps; steps++){
        p = qschedulability;
        qschedulability = p->edf->testnext;
        ticks = p->edf->testtime;
        switch (p->edf->testtype){
        case Dl:
            H += p->edf->C;
            Cb = 0;
            DPRINT("\tStep %3d, Ticks %lud, pid %lud, deadline, H += %lud → %lud, Cb = %lud\n",
                steps, ticks, p->pid, p->edf->C, H, Cb);
            if (H+Cb>ticks){
                DPRINT("not schedulable\n");
                return "not schedulable";
            }
            p->edf->testtime += p->edf->T - p->edf->D;
            p->edf->testtype = Rl;
```

```
            testenq(p);
            break;
        case Rl:
            DPRINT("\tStep %3d, Ticks %lud, pid %lud, release, G  %lud, C%lud\n",
                steps, ticks, p->pid, p->edf->C, G);
            if(ticks && G <= ticks){
                DPRINT("schedulable\n");
                return nil;
            }
            G += p->edf->C;
            p->edf->testtime += p->edf->D;
            p->edf->testtype = Dl;
            testenq(p);
            break;
        default:
            assert(0);
        }
    }
    DPRINT("probably not schedulable\n");
    return "probably not schedulable";
}
```

Tracing scheduling events

⟨enum Tevent *real-time scheduling events* 446a⟩≡ (376e)

```
SAdmit, /* Edf admit */
SRelease, /* Edf release, waiting to be scheduled */
SYield,  /* blocked waiting for release */
SSlice,  /* slice exhausted */
SDeadline, /* proc's deadline */
SExpel,  /* Edf expel */
SInts,  /* Interrupt start */
SInte,  /* Interrupt end */
```

22.11 Fileservers

22.11.1 Mnt

⟨*struct* Mnt 446b⟩≡ (569c)

```
struct Mnt
{
  Lock;
  /* references are counted using c->ref; channels on this mount point incref(c->mchan) == Mnt.c */
  Chan  *c;   /* Channel to file service */
  Proc  *rip;   /* Reader in progress */
  Mntrpc  *queue;   /* Queue of pending requests on this channel */
  ulong id;   /* Multiplexer id for channel check */
  Mnt *list;    /* Free list */
  int flags;    /* cache */
  int msize;    /* data + IOHDRSZ */
  char  *version; /* 9P version */
  Queue *q;   /* input queue */
};
```

⟨*struct* Mntalloc 446c⟩≡ (569c)

```
/*
 * References are managed as follows:
```

```
 * The channel to the server - a network connection or pipe - has one
 * reference for every Chan open on the server.  The server channel has
 * c->mux set to the Mnt used for muxing control to that server.  Mnts
 * have no reference count; they go away when c goes away.
 * Each channel derived from the mount point has mchan set to c,
 * and increfs/decrefs mchan to manage references on the server
 * connection.
 */
struct Mntalloc
{
    Mnt*    list;       /* Mount devices in use */
    Mnt*    mntfree;    /* Free list */
    Mntrpc* rpcfree;
    int nrpcfree;
    int nrpcused;
    ulong   id;
    ulong   tagmask[NMASK];

    // extra
    Lock;

};
```

⟨*global* mntalloc 447a⟩≡ (575c)

```
struct Mntalloc mntalloc;
```

⟨*constants tags* 447b⟩≡ (569c)

```
enum
{
    TAGSHIFT = 5,           /* ulong has to be 32 bits */
    TAGMASK = (1<<TAGSHIFT)-1,
    NMASK = (64*1024)>>TAGSHIFT,
};
```

⟨*function* freetag 447c⟩≡ (575c)

```
void
freetag(int t)
{
    mntalloc.tagmask[t>>TAGSHIFT] &= ~(1<<(t&TAGMASK));
}
```

⟨*function* mntfree 447d⟩≡ (575c)

```
void
mntfree(Mntrpc *r)
{
    if(r->b != nil)
        freeblist(r->b);
    lock(&mntalloc);
    if(mntalloc.nrpcfree >= 10){
        free(r->rpc);
        freetag(r->request.tag);
        free(r);
    }
    else{
        r->list = mntalloc.rpcfree;
        mntalloc.rpcfree = r;
        mntalloc.nrpcfree++;
    }
    mntalloc.nrpcused--;
    unlock(&mntalloc);
}
```

⟨*function* mntpntfree 448a⟩≡ (575c)

```
void
mntpntfree(Mnt *m)
{
    Mnt *f, **l;
    Queue *q;

    lock(&mntalloc);
    l = &mntalloc.list;
    for(f = *l; f; f = f->list) {
        if(f == m) {
            *l = m->list;
            break;
        }
        l = &f->list;
    }
    m->list = mntalloc.mntfree;
    mntalloc.mntfree = m;
    q = m->q;
    unlock(&mntalloc);

    qfree(q);
}
```

⟨*function* muxclose 448b⟩≡ (575c)

```
void
muxclose(Mnt *m)
{
    Mntrpc *q, *r;

    for(q = m->queue; q; q = r) {
        r = q->list;
        mntfree(q);
    }
    m->id = 0;
    free(m->version);
    m->version = nil;
    mntpntfree(m);
}
```

22.11.2 MntRpc

⟨*struct* Mntrpc 448c⟩≡ (569c)

```
struct Mntrpc
{
  Chan* c;    /* Channel for whom we are working */
  Mntrpc* list;   /* Free/pending list */
  Fcall request;  /* Outgoing file system protocol message */
  Fcall   reply;    /* Incoming reply */
  Mnt*  m;    /* Mount device during rpc */
  Rendez  r;    /* Place to hang out */
  byte*  rpc;    /* I/O Data buffer */
  uint  rpclen;   /* len of buffer */
  Block *b;   /* reply blocks */
  char  done;   /* Rpc completed */
  uvlong  stime;    /* start time for mnt statistics */
  ulong reqlen;   /* request length for mnt statistics */
  ulong replen;   /* reply length for mnt statistics */
  Mntrpc* flushed;  /* message this one flushes */
```

```
};
```

22.11.3 Fcall

22.11.4 mntversion()

⟨ChannelFlag *cases* 449a⟩+≡ (206b) ◁228h ??▷

```
CMSG  = 0x0002,   /* the message channel for a mount */
```

⟨*function* mntversion 449b⟩≡ (575c)

```
/*
 * Version is not multiplexed: message sent only once per connection.
 */
long
mntversion(Chan *c, char *version, int msize, int returnlen)
{
    Fcall f;
    uchar *msg;
    Mnt *m;
    char *v;
    long k, l;
    uvlong oo;
    char buf[128];

    qlock(&c->umqlock); /* make sure no one else does this until we've established ourselves */
    if(waserror()){
        qunlock(&c->umqlock);
        nexterror();
    }

    /* defaults */
    if(msize == 0)
        msize = MAXRPC;
    if(msize > c->iounit && c->iounit != 0)
        msize = c->iounit;
    v = version;
    if(v == nil || v[0] == '\0')
        v = VERSION9P;

    /* validity */
    if(msize < 0)
        error("bad iounit in version call");
    if(strncmp(v, VERSION9P, strlen(VERSION9P)) != 0)
        error("bad 9P version specification");

    m = c->mux;

    if(m != nil){
        qunlock(&c->umqlock);
        poperror();

        strecpy(buf, buf+sizeof buf, m->version);
        k = strlen(buf);
        if(strncmp(buf, v, k) != 0){
            snprint(buf, sizeof buf, "incompatible 9P versions %s %s", m->version, v);
            error(buf);
        }
        if(returnlen > 0){
            if(returnlen < k)
```

```
            error(Eshort);
        memmove(version, buf, k);
    }
    return k;
}

f.type = Tversion;
f.tag = NOTAG;
f.msize = msize;
f.version = v;
msg = malloc(8192+IOHDRSZ);
if(msg == nil)
    exhausted("version memory");
if(waserror()){
    free(msg);
    nexterror();
}
k = convS2M(&f, msg, 8192+IOHDRSZ);
if(k == 0)
    error("bad fversion conversion on send");

lock(c);
oo = c->offset;
c->offset += k;
unlock(c);

l = devtab[c->type]->write(c, msg, k, oo);

if(l < k){
    lock(c);
    c->offset -= k - l;
    unlock(c);
    error("short write in fversion");
}

/* message sent; receive and decode reply */
k = devtab[c->type]->read(c, msg, 8192+IOHDRSZ, c->offset);
if(k <= 0)
    error("EOF receiving fversion reply");

lock(c);
c->offset += k;
unlock(c);

l = convM2S(msg, k, &f);
if(l != k)
    error("bad fversion conversion on reply");
if(f.type != Rversion){
    if(f.type == Rerror)
        error(f.ename);
    error("unexpected reply type in fversion");
}
if(f.msize > msize)
    error("server tries to increase msize in fversion");
if(f.msize<256 || f.msize>1024*1024)
    error("nonsense value of msize in fversion");
k = strlen(f.version);
if(strncmp(f.version, v, k) != 0)
    error("bad 9P version returned from server");
```

```
    /* now build Mnt associated with this connection */
    lock(&mntalloc);
    m = mntalloc.mntfree;
    if(m != 0)
        mntalloc.mntfree = m->list;
    else {
        m = malloc(sizeof(Mnt));
        if(m == 0) {
            unlock(&mntalloc);
            exhausted("mount devices");
        }
    }
    m->list = mntalloc.list;
    mntalloc.list = m;
    m->version = nil;
    kstrdup(&m->version, f.version);
    m->id = mntalloc.id++;
    m->q = qopen(10*MAXRPC, 0, nil, nil);
    m->msize = f.msize;
    unlock(&mntalloc);

    if(returnlen > 0){
        if(returnlen < k)
            error(Eshort);
        memmove(version, f.version, k);
    }

    poperror(); /* msg */
    free(msg);

    lock(m);
    m->queue = 0;
    m->rip = 0;

    c->flag |= CMSG;
    c->mux = m;
    m->c = c;
    unlock(m);

    poperror(); /* c */
    qunlock(&c->umqlock);

    return k;
}
```

22.11.5 mountrpc()

⟨*function* mountrpc 451⟩≡ (? 0—1)

```
void
mountrpc(Mnt *m, Mntrpc *r)
{
    char *sn, *cn;
    int t;

    r->reply.tag = 0;
    r->reply.type = Tmax;   /* can't ever be a valid message type */

    mountio(m, r);
```

```
    t = r->reply.type;
    switch(t) {
    case Rerror:
        error(r->reply.ename);
    case Rflush:
        error(Eintr);
    default:
        if(t == r->request.type+1)
            break;
        sn = "?";
        if(m->c->path != nil)
            sn = m->c->path->s;
        cn = "?";
        if(r->c != nil && r->c->path != nil)
            cn = r->c->path->s;
        print("mnt: proc %s %lud: mismatch from %s %s rep %#p tag %d fid %d T%d R%d rp %d\n",
            up->text, up->pid, sn, cn,
            r, r->request.tag, r->request.fid, r->request.type,
            r->reply.type, r->reply.tag);
        error(Emountrpc);
    }
}
```

⟨*function* mountio 452⟩≡ (? 0—1)

```
void
mountio(Mnt *m, Mntrpc *r)
{
    int n;

    while(waserror()) {
        if(m->rip == up)
            mntgate(m);
        if(strcmp(up->errstr, Eintr) != 0){
            mntflushfree(m, r);
            nexterror();
        }
        r = mntflushalloc(r, m->msize);
    }

    lock(m);
    r->m = m;
    r->list = m->queue;
    m->queue = r;
    unlock(m);

    /* Transmit a file system rpc */
    if(m->msize == 0)
        panic("msize");
    n = convS2M(&r->request, r->rpc, m->msize);
    if(n < 0)
        panic("bad message type in mountio");
    if(devtab[m->c->type]->write(m->c, r->rpc, n, 0) != n)
        error(Emountrpc);
    r->stime = arch_fastticks(nil);
    r->reqlen = n;

    /* Gate readers onto the mount point one at a time */
    for(;;) {
        lock(m);
        if(m->rip == 0)
```

```
            break;
        unlock(m);
        sleep(&r->r, rpcattn, r);
        if(r->done){
            poperror();
            mntflushfree(m, r);
            return;
        }
    }
    m->rip = up;
    unlock(m);
    while(r->done == 0) {
        if(mntrpcread(m, r) < 0)
            error(Emountrpc);
        mountmux(m, r);
    }
    mntgate(m);
    poperror();
    mntflushfree(m, r);
}
```

22.11.6 devmnt

mntattach()

⟨*function* mntattach 453⟩≡ (? 0—1)

```
static Chan*
mntattach(char *muxattach)
{
    Mnt *m;
    Chan *c;
    Mntrpc *r;
    struct Bogus bogus;

    bogus = *((struct Bogus *)muxattach);
    c = bogus.chan;

    m = c->mux;

    if(m == nil){
        mntversion(c, nil, 0, 0);
        m = c->mux;
        if(m == nil)
            error(Enoversion);
    }

    c = mntchan();
    if(waserror()) {
        /* Close must not be called since it will
         * call mnt recursively
         */
        chanfree(c);
        nexterror();
    }

    r = mntralloc(0, m->msize);

    if(waserror()) {
        mntfree(r);
```

```
        nexterror();
    }

    r->request.type = Tattach;
    r->request.fid = c->fid;
    if(bogus.authchan == nil)
        r->request.afid = NOFID;
    else
        r->request.afid = bogus.authchan->fid;
    r->request.uname = up->user;
    r->request.aname = bogus.spec;

    mountrpc(m, r);

    c->qid = r->reply.qid;
    c->mchan = m->c;
    incref(m->c);
    c->mqid = c->qid;

    poperror(); /* r */
    mntfree(r);

    poperror(); /* c */

    if(bogus.flags&MCACHE)
        c->flag |= CCACHE;
    return c;
}
```

mntwalk()

⟨*function* mntwalk 454⟩≡ (? 0—1)

```
static Walkqid*
mntwalk(Chan *c, Chan *nc, char **name, int nname)
{
    int i, alloc;
    Mnt *m;
    Mntrpc *r;
    Walkqid *wq;

    if(nc != nil)
        print("mntwalk: nc != nil\n");
    if(nname > MAXWELEM)
        error("devmnt: too many name elements");
    alloc = 0;
    wq = smalloc(sizeof(Walkqid)+(nname-1)*sizeof(Qid));
    if(waserror()){
        if(alloc && wq->clone!=nil)
            cclose(wq->clone);
        free(wq);
        return nil;
    }

    alloc = 0;
    m = mntchk(c);
    r = mntralloc(c, m->msize);
    if(nc == nil){
        nc = devclone(c);
        /*
```

```
         * Until the other side accepts this fid, we can't mntclose it.
         * Therefore set type to 0 for now; rootclose is known to be safe.
         */
        nc->type = 0;
        alloc = 1;
    }
    wq->clone = nc;
    nc->flag |= c->flag&CCACHE;

    if(waserror()) {
        mntfree(r);
        nexterror();
    }
    r->request.type = Twalk;
    r->request.fid = c->fid;
    r->request.newfid = nc->fid;
    r->request.nwname = nname;
    memmove(r->request.wname, name, nname*sizeof(char*));

    mountrpc(m, r);

    if(r->reply.nwqid > nname)
        error("too many QIDs returned by walk");
    if(r->reply.nwqid < nname){
        if(alloc)
            cclose(nc);
        wq->clone = nil;
        if(r->reply.nwqid == 0){
            free(wq);
            wq = nil;
            goto Return;
        }
    }

    /* move new fid onto mnt device and update its qid */
    if(wq->clone != nil){
        if(wq->clone != c){
            wq->clone->type = c->type;
            wq->clone->mchan = c->mchan;
            incref(c->mchan);
        }
        if(r->reply.nwqid > 0)
            wq->clone->qid = r->reply.wqid[r->reply.nwqid-1];
    }
    wq->nqid = r->reply.nwqid;
    for(i=0; i<wq->nqid; i++)
        wq->qid[i] = r->reply.wqid[i];

    Return:
    poperror();
    mntfree(r);
    poperror();
    return wq;
}
```

22.12 Network

22.13 Security

⟨*global* eve 456a⟩≡ (534b)
```
char    *eve;
```

⟨*function* iseve 456b⟩≡ (534b)
```
bool iseve(void) {
  return strcmp(eve, up->user) == 0;
}
```

⟨systab *security syscalls* 456c⟩≡ (26c)
```
[FAUTH]     sysfauth,
[FVERSION]  sysfversion,
```

⟨*global* hostdomain 456d⟩≡ (605)
```
char    hostdomain[DOMLEN];
```

⟨*syscall fversion* 456e⟩≡ (605)
```
long
sysfversion(ulong *arg)
{
    char *vers;
    uint arglen, m, msize;
    Chan *c;

    msize = arg[1];
    vers = (char*)arg[2];
    arglen = arg[3];
    validaddr(arg[2], arglen, true);
    /* check there's a NUL in the version string */
    if(arglen==0 || memchr(vers, 0, arglen)==0)
        error(Ebadarg);
    c = fdtochan(arg[0], ORDWR, false, true);
    if(waserror()){
        cclose(c);
        nexterror();
    }

    m = mntversion(c, vers, msize, arglen);

    cclose(c);
    poperror();
    return m;
}
```

⟨*syscall fauth* 456f⟩≡ (605)
```
long
sysfauth(ulong *arg)
{
    Chan *c, *ac;
    char *aname;
    int fd;

    validaddr(arg[1], 1, false);
    aname = validnamedup((char*)arg[1], true);
    if(waserror()){
        free(aname);
```

```
        nexterror();
    }
    c = fdtochan(arg[0], ORDWR, false, true);
    if(waserror()){
        cclose(c);
        nexterror();
    }

    ac = mntauth(c, aname);
    /* at this point ac is responsible for keeping c alive */
    poperror(); /* c */
    cclose(c);
    poperror(); /* aname */
    free(aname);

    if(waserror()){
        cclose(ac);
        nexterror();
    }

    fd = newfd(ac);
    if(fd < 0)
        error(Enofd);
    poperror(); /* ac */

    /* always mark it close on exec */
    ac->flag |= CCEXEC;

    return fd;
}
```

⟨*function* `userwrite` 457a⟩≡ (605)

```
/*
 *  called by devcons() for user device
 *
 *  anyone can become none
 */
long
userwrite(char *a, int n)
{
    if(n!=4 || strncmp(a, "none", 4)!=0)
        error(Eperm);
    kstrdup(&up->user, "none");
    up->basepri = PriNormal;
    return n;
}
```

⟨*function* `hostdomainwrite` 457b⟩≡ (605)

```
long
hostdomainwrite(char *a, int n)
{
    char buf[DOMLEN];

    if(!iseve())
        error(Eperm);
    if(n >= DOMLEN)
        error(Ebadarg);
    memset(buf, 0, DOMLEN);
    strncpy(buf, a, n);
    if(buf[0] == 0)
```

```
        error(Ebadarg);
    memmove(hostdomain, buf, DOMLEN);
    return n;
}
```

22.14 Graphics

Chapter 23

Special Fileservers

23.1 Cmdtab

⟨*struct* Cmdtab 459a⟩≡ (591b)

```
struct Cmdtab
{
  int index;  /* used by client to switch on result */
  char  *cmd; /* command name */
  int narg; /* expected #args; 0 ==> variadic */
};
```

⟨*struct* Cmdbuf 459b⟩≡ (591b)

```
struct Cmdbuf
{
  char  *buf;
  char  **f;
  int nf;
};
```

⟨*function* parsecmd 459c⟩≡ (591c)

```
/*
 *  parse a command written to a device
 */
Cmdbuf*
parsecmd(char *p, int n)
{
    Cmdbuf *volatile cb;
    int nf;
    char *sp;

    nf = ncmdfield(p, n);

    /* allocate Cmdbuf plus string pointers plus copy of string including \0 */
    sp = smalloc(sizeof(*cb) + nf * sizeof(char*) + n + 1);
    cb = (Cmdbuf*)sp;
    cb->f = (char**)(&cb[1]);
    cb->buf = (char*)(&cb->f[nf]);

    if(up!=nil && waserror()){
        free(cb);
        nexterror();
    }
    memmove(cb->buf, p, n);
    if(up != nil)
        poperror();
```

```
    /* dump new line and null terminate */
    if(n > 0 && cb->buf[n-1] == '\n')
        n--;
    cb->buf[n] = '\0';

    cb->nf = tokenize(cb->buf, cb->f, nf-1);
    cb->f[cb->nf] = nil;

    return cb;
}
```

⟨*function* cmderror 460a⟩≡ (591c)

```
/*
 * Reconstruct original message, for error diagnostic
 */
void
cmderror(Cmdbuf *cb, char *s)
{
    int i;
    char *p, *e;

    p = up->genbuf;
    e = p+ERRMAX-10;
    p = seprint(p, e, "%s \"", s);
    for(i=0; i<cb->nf; i++){
        if(i > 0)
            p = seprint(p, e, " ");
        p = seprint(p, e, "%q", cb->f[i]);
    }
    strcpy(p, "\"");
    error(up->genbuf);
}
```

⟨*function* lookupcmd 460b⟩≡ (591c)

```
/*
 * Look up entry in table
 */
Cmdtab*
lookupcmd(Cmdbuf *cb, Cmdtab *ctab, int nctab)
{
    int i;
    Cmdtab *ct;

    if(cb->nf == 0)
        error("empty control message");

    for(ct = ctab, i=0; i<nctab; i++, ct++){
        if(strcmp(ct->cmd, "*") !=0)    /* wildcard always matches */
        if(strcmp(ct->cmd, cb->f[0]) != 0)
            continue;
        if(ct->narg != 0 && ct->narg != cb->nf)
            cmderror(cb, Ecmdargs);
        return ct;
    }

    cmderror(cb, "unknown control message");
    return nil;
}
```

⟨*ncmdfield* 461a⟩≡ (591c)

```
/*
 * Generous estimate of number of fields, including terminal nil pointer
 */
static int
ncmdfield(char *p, int n)
{
    int white, nwhite;
    char *ep;
    int nf;

    if(p == nil)
        return 1;

    nf = 0;
    ep = p+n;
    white = 1;  /* first text will start field */
    while(p < ep){
        nwhite = (strchr(" \t\r\n", *p++ & 0xFF) != 0); /* UTF is irrelevant */
        if(white && !nwhite)    /* beginning of field */
            nf++;
        white = nwhite;
    }
    return nf+1;    /* +1 for nil */
}
```

23.2 /env/

23.2.1 Egrp

⟨*struct* Evalue 461b⟩≡ (569c)

```
struct Evalue
{
    // string
    char  *name;
    // option<array<byte>, usually a string but can be something else
    byte  *value;
    // nelem(Evalue.value),
    int len;

    Qid qid;
};
```

⟨*struct* Egrp 461c⟩≡ (569c)

```
struct Egrp
{
    // array<ref_counted<Evalue>>
    Evalue  **ent;
    // used entries, <= ment
    int nent;
    // nelem(Egrp.ent), malloc'ed entries
    int ment;

    ulong path; /* qid.path of next Evalue to be allocated */
    ulong vers; /* of Egrp */

    // extra
    Ref;
```

```
    RWlock;
};
```

⟨Proc *other fields* 462a⟩+≡ (29c) ◁387g 464d▷

```
// ref_counted<Egrp>
Egrp  *egrp;    /* Environment group */
```

⟨*function* closeegrp 462b⟩≡ (575a)

```
void
closeegrp(Egrp *eg)
{
    int i;
    Evalue *e;

    if(decref(eg) == 0){
        for(i=0; i<eg->nent; i++){
            e = eg->ent[i];
            free(e->name);
            if(e->value)
                free(e->value);
            free(e);
        }
        free(eg->ent);
        free(eg);
    }
}
```

⟨*function* envcpy 462c⟩≡ (561)

```
void
envcpy(Egrp *to, Egrp *from)
{
    int i;
    Evalue *ne, *e;

    rlock(from);
    to->ment = (from->nent+31)&~31;
    to->ent = smalloc(to->ment*sizeof(to->ent[0]));
    for(i=0; i<from->nent; i++){
        e = from->ent[i];
        ne = smalloc(sizeof(Evalue));
        ne->name = smalloc(strlen(e->name)+1);
        strcpy(ne->name, e->name);
        if(e->value){
            ne->value = smalloc(e->len);
            memmove(ne->value, e->value, e->len);
            ne->len = e->len;
        }
        ne->qid.path = ++to->path;
        to->ent[i] = ne;
    }
    to->nent = from->nent;
    runlock(from);
}
```

23.2.2 /env/

⟨*function* ksetenv 462d⟩≡ (561)

```
/*
 *  to let the kernel set environment variables
```

```
 */
void
ksetenv(char *ename, char *eval, bool conf)
{
    Chan *c;
    char buf[2*KNAMELEN];

    snprint(buf, sizeof(buf), "#e%s/%s", conf?"c":"", ename);
    c = namec(buf, Acreate, OWRITE, 0600);
    devtab[c->type]->write(c, eval, strlen(eval), 0);
    cclose(c);
}
```

⟨*global* envdevtab 463a⟩≡ (561)

```
Dev envdevtab = {
    .dc       =    'e',
    .name     =    "env",

    .reset    =    devreset,
    .init     =    devinit,
    .shutdown =    devshutdown,
    .attach   =    envattach,
    .walk     =    envwalk,
    .stat     =    envstat,
    .open     =    envopen,
    .create   =    envcreate,
    .close    =    envclose,
    .read     =    envread,
    .bread    =    devbread,
    .write    =    envwrite,
    .bwrite   =    devbwrite,
    .remove   =    envremove,
    .wstat    =    devwstat,
};
```

23.3 /proc/

23.3.1 /proc/x/ctl

23.3.2 /proc/x/mem

23.3.3 /proc/x/status

23.3.4 /proc/x/args

23.3.5 /proc/x/wait

⟨Proc *hierarchy fields* 463b⟩+≡ (29c) ◁91c

```
QLock qwaitr;
```

⟨procread() *cases* 463c⟩+≡ (468c) ◁382d 470a▷

```
case Qwait:
    if(!canqlock(&p->qwaitr))
        error(Einuse);

    if(waserror()) {
        qunlock(&p->qwaitr);
```

```
        nexterror();
    }

    ⟨procread() when Qwait case, sanity check process has children 464a⟩
    pid = p->pid;
    while(p->waitq == nil) {
        unlock(&p->exl);
        sleep(&p->waitr, haswaitq, p);
        if(p->pid != pid)
            error(Eprocdied);
        lock(&p->exl);
    }
    wq = p->waitq;
    p->waitq = wq->next;
    p->nwait--;
    unlock(&p->exl);

    qunlock(&p->qwaitr);
    poperror();

    n = snprint(a, n, "%d %lud %lud %lud %q",
        wq->w.pid,
        ⟨procread() Qwait case, snprint time field arguments 381g⟩
        wq->w.msg);
    free(wq);
    return n;
```

⟨procread() *when Qwait case, sanity check process has children* 464a⟩≡ (463c)

```
// same in sysawait
lock(&p->exl);
if(up == p && p->nchild == 0 && p->waitq == 0) {
    unlock(&p->exl);
    error(Enochild);
}
```

⟨pwait() *lock qwaitr* 464b⟩≡ (91a)

```
if(!canqlock(&up->qwaitr))
    error(Einuse); // someone is reading /proc/pid/wait?
if(waserror()) {
    qunlock(&up->qwaitr);
    nexterror();
}
```

⟨pwait() *unlock qwaitr* 464c⟩≡ (91a)

```
qunlock(&up->qwaitr);
poperror();
```

23.3.6 Other /proc/x/xxx

23.3.7 XXX

⟨Proc *other fields* 464d⟩+≡ (29c) ◁462a 464f▷

```
ulong procmode; /* proc device default file mode */
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 464e⟩+≡ (78d) ◁387h 465b▷

```
p->procmode = up->procmode;
```

⟨Proc *other fields* 464f⟩+≡ (29c) ◁464d

```
bool privatemem; /* proc does not let anyone read mem */
```

⟨newproc() *set fields of* p 465a⟩+≡ (75b) ◁432c 479c▷

```
p->privatemem = false;
```

⟨sysrfork() *when* RFPROC *propagate fields from up to p* 465b⟩+≡ (78d) ◁464e

```
p->privatemem = up->privatemem;
```

⟨sysexec() *when hold debug lock, reset some fields* 465c⟩+≡ (83i) ◁335i

```
up->privatemem = false;
```

⟨procctlreq() *CMprivate case* 465d⟩≡ (482)

```
case CMprivate:
    p->privatemem = true;
    break;
```

⟨*devproc.c enum Qxxx* 465e⟩≡ (567)

```
enum
{
    Qdir,
    Qtrace,

    Qargs,
    Qctl,
    Qfd,
    Qfpregs,
    Qkregs,
    Qmem,
    Qnote,
    Qnoteid,
    Qnotepg,
    Qns,
    Qproc,
    Qregs,
    Qsegment,
    Qstatus,
    Qtext,
    Qwait,
    Qprofile,
    Qsyscall,
};
```

⟨*devproc enum CMxxx* 465f⟩≡ (567)

```
enum
{
    CMclose,
    CMclosefiles,
    CMhang,
    CMkill,
    CMnohang,
    CMnoswap,
    CMprivate,
    CMprofile,

    CMpri,
    CMfixedpri,
    CMwired,

    CMstart,
    CMstartstop,
    CMstartsyscall,
    CMstop,
```

```
    CMwaitstop,
    CMtrace,

    /* real time */
    CMperiod,
    CMdeadline,
    CMcost,
    CMsporadic,
    CMdeadlinenotes,
    CMadmit,
    CMextra,
    CMexpel,
    CMevent,
};
```

⟨*global* procdir 466a⟩≡ (567)

```
/*
 * Status, fd, and ns are left fully readable (0444) because of their use in debugging,
 * particularly on shared servers.
 * Arguably, ns and fd shouldn't be readable; if you'd prefer, change them to 0000
 */
Dirtab procdir[] =
{
    "args",     {Qargs},    0,          0660,
    "ctl",      {Qctl},     0,          0000,
    "fd",       {Qfd},      0,          0444,
    "fpregs",   {Qfpregs},  sizeof(Arch_FPsave),     0000,
    "kregs",    {Qkregs},   sizeof(Ureg),       0400,
    "mem",      {Qmem},     0,          0000,
    "note",     {Qnote},    0,          0000,
    "noteid",   {Qnoteid},  0,          0664,
    "notepg",   {Qnotepg},  0,          0000,
    "ns",       {Qns},      0,          0444,
    "proc",     {Qproc},    0,          0400,
    "regs",     {Qregs},    sizeof(Ureg),       0000,
    "segment",  {Qsegment}, 0,          0444,
    "status",   {Qstatus},  STATSIZE,       0444,
    "text",     {Qtext},    0,          0000,
    "wait",     {Qwait},    0,          0400,
    "profile",  {Qprofile}, 0,          0400,
    "syscall",  {Qsyscall}, 0,          0400,
};
```

⟨*global* proccmd 466b⟩≡ (567)

```
static
Cmdtab proccmd[] = {
    CMclose,        "close",        2,
    CMclosefiles,       "closefiles",       1,
    CMfixedpri,     "fixedpri",     2,
    CMhang,         "hang",         1,
    CMnohang,       "nohang",       1,
    CMnoswap,       "noswap",       1,
    CMkill,         "kill",         1,
    CMpri,          "pri",          2,
    CMprivate,      "private",      1,
    CMprofile,      "profile",      1,
    CMstart,        "start",        1,
    CMstartstop,        "startstop",        1,
    CMstartsyscall,     "startsyscall",     1,
    CMstop,         "stop",         1,
```

```
    CMwaitstop,     "waitstop",     1,
    CMwired,        "wired",        2,
    CMtrace,        "trace",        0,
    CMperiod,       "period",       2,
    CMdeadline,     "deadline",     2,
    CMcost,         "cost",         2,
    CMsporadic,     "sporadic",     1,
    CMdeadlinenotes,    "deadlinenotes",    1,
    CMadmit,        "admit",        1,
    CMextra,        "extra",        1,
    CMexpel,        "expel",        1,
    CMevent,        "event",        1,
};
```

⟨*devproc QXXX macros* 467a⟩≡ (567)

```
/*
 * Qids are, in path:
 *   5 bits of file type (qids above)
 *  26 bits of process slot number + 1
 *       in vers,
 *  32 bits of pid, for consistency checking
 * If notepg, c->pgrpid.path is pgrp slot, .vers is noteid.
 */
#define QSHIFT  5   /* location in qid of proc slot # */

#define QID(q)      ((((ulong)(q).path) & ((1<<QSHIFT)-1)) >> 0)
#define SLOT(q)     (((((ulong)(q).path) & ~(1UL<<31)) >> QSHIFT) - 1)
#define PID(q)      ((q).vers)
#define NOTEID(q)   ((q).vers)
```

⟨*function* proctab 467b⟩≡ (599c)

```
Proc*
proc_proctab(int i)
{
    return &procalloc.arena[i];
}
```

⟨Chan *union other fields* 467c⟩+≡ (31a) ◁343c 468a▷

```
Qid pgrpid;   /* for #p/notepg */
```

⟨*method procinit* 467d⟩≡ (567)

```
static void
proc_init(void)
{
    if(conf.nproc >= (1<<(31-QSHIFT))-1)
        print("warning: too many procs for devproc\n");

    ⟨proc_init() other init 381k⟩
}
```

⟨*global* procdevtab 467e⟩≡ (567)

```
Dev procdevtab = {
    .dc       =    'p',
    .name     =    "proc",

    .reset    =    devreset,
    .init     =    proc_init,
    .shutdown =    devshutdown,
    .attach   =    procattach,
    .walk     =    procwalk,
```

```
    .stat     =    procstat,
    .open     =    procopen,
    .create   =    devcreate,
    .close    =    procclose,
    .read     =    procread,
    .bread    =    devbread,
    .write    =    procwrite,
    .bwrite   =    devbwrite,
    .remove   =    devremove,
    .wstat    =    procwstat,
};
```

⟨Chan *union other fields* 468a⟩+≡ (31a) ◁467c

```
ulong mid;    /* for ns in devproc */
```

⟨procread() *locals* 468b⟩+≡ (468c) ◁254b

```
/* NSEG*32 was too small for worst cases */
char *a, *sps, *srv, statbuf[NSEG*64];
int i, j, m, navail, ne, pid, rsize;
long l;
byte *rptr;
ulong offset;
Confmem *cm;
Proc *p;
Segment *s;
Ureg kur;
Waitq *wq;
```

⟨*method procread* 468c⟩≡ (567)

```
static long
procread(Chan *c, void *va, long n, vlong off)
{
    ⟨procread() locals 69a⟩

    a = va;
    offset = off;

    if(c->qid.type & QTDIR)
        return devdirread(c, a, n, 0, 0, procgen);

    ⟨procread() Qtrace if 379b⟩

    p = proctab(SLOT(c->qid));
    if(p->pid != PID(c->qid))
        error(Eprocdied);

    switch(QID(c->qid)){

    ⟨procread() cases 69b⟩

    case Qargs:
        qlock(&p->debug);
        j = procargs(p, up->genbuf, sizeof up->genbuf);
        qunlock(&p->debug);
        if(offset >= j)
            return 0;
        if(offset+n > j)
            n = j-offset;
        memmove(a, &up->genbuf[offset], n);
        return n;
```

```
    case Qnote:
        qlock(&p->debug);
        if(waserror()){
            qunlock(&p->debug);
            nexterror();
        }
        if(p->pid != PID(c->qid))
            error(Eprocdied);
        if(n < 1)   /* must accept at least the '\0' */
            error(Etoosmall);
        if(p->nnote == 0)
            n = 0;
        else {
            m = strlen(p->note[0].msg) + 1;
            if(m > n)
                m = n;
            memmove(va, p->note[0].msg, m);
            ((char*)va)[m-1] = '\0';
            p->nnote--;
            memmove(p->note, p->note+1, p->nnote*sizeof(Note));
            n = m;
        }
        if(p->nnote == 0)
            p->notepending = false;
        poperror();
        qunlock(&p->debug);
        return n;

    case Qproc:
        if(offset >= sizeof(Proc))
            return 0;
        if(offset+n > sizeof(Proc))
            n = sizeof(Proc) - offset;
        memmove(a, ((char*)p)+offset, n);
        return n;


    case Qregs:
        rptr = (byte*)p->dbgreg;
        rsize = sizeof(Ureg);
        goto regread;


    ⟨procread() Qfpregs case 400b⟩

    regread:
        if(rptr == nil)
            error(Enoreg);
        if(offset >= rsize)
            return 0;
        if(offset+n > rsize)
            n = rsize - offset;
        memmove(a, rptr+offset, n);
        return n;


    // coupling with ps.c
/*
```

```
  * 0  text
  * 1  user
  * 2  state
  * 3  cputime[5]
  * 8  memory
  * 9 basepri
  * 10 pri
  */
    case Qstatus:
        if(offset >= STATSIZE)
            return 0;
        if(offset+n > STATSIZE)
            n = STATSIZE - offset;

        memset(statbuf, ' ', sizeof statbuf);
        readstr(0, statbuf+0*KNAMELEN, KNAMELEN-1, p->text);
        readstr(0, statbuf+1*KNAMELEN, KNAMELEN-1, p->user);
        sps = p->psstate;
        if(sps == nil)
            sps = statename[p->state];
        readstr(0, statbuf+2*KNAMELEN, 11, sps);

        j = 2*KNAMELEN + 12;
        ⟨procread() Qstatus case, time part 381a⟩

        /* ignore stack, which is mostly non-existent */
        l = 0;
        for(i=1; i<NSEG; i++){
            s = p->seg[i];
            if(s)
                l += s->top - s->base;
        }
        readnum(0, statbuf+j+NUMSIZE*5, NUMSIZE, l>>10, NUMSIZE);
        readnum(0, statbuf+j+NUMSIZE*6, NUMSIZE, p->basepri, NUMSIZE);
        readnum(0, statbuf+j+NUMSIZE*7, NUMSIZE, p->priority, NUMSIZE);
        memmove(a, statbuf+offset, n);
        return n;



    case Qnoteid:
        return readnum(offset, va, n, p->noteid, NUMSIZE);
    case Qfd:
        return procfds(p, va, n, offset);
    }
    error(Egreg);
    panic("procread: should not reach this point");
    return -1; // unreachable
}
```

⟨procread() *cases* 470a⟩+≡ (468c) ◁463c

```
  case Qkregs:
      memset(&kur, 0, sizeof(Ureg));
      arch_setkernur(&kur, p);
      rptr = (byte*)&kur;
      rsize = sizeof(Ureg);
      goto regread;
```

⟨*function* arch_setkernur*(arm)* 470b⟩≡ (641a)

```
  /* Give enough context in the ureg to produce a kernel stack for
```

```
 * a sleeping process
 */
void
arch_setkernur(Ureg* ureg, Proc* p)
{
    ureg->pc = p->sched.pc;
    ureg->sp = p->sched.sp+4;
    ureg->r14 = PTR2UINT(sched);
}
```

⟨*method procwrite* 471⟩≡ (567)

```
static long
procwrite(Chan *c, void *va, long n, vlong off)
{
    int id, m;
    Proc *p, *t, *et;
    char *a, *arg, buf[ERRMAX];
    ulong offset = off;

    a = va;
    if(c->qid.type & QTDIR)
        error(Eisdir);

    p = proctab(SLOT(c->qid));

    /* Use the remembered noteid in the channel rather
     * than the process pgrpid
     */
    if(QID(c->qid) == Qnotepg) {
        pgrpnote(NOTEID(c->pgrpid), va, n, NUser);
        return n;
    }

    qlock(&p->debug);
    if(waserror()){
        qunlock(&p->debug);
        nexterror();
    }
    if(p->pid != PID(c->qid))
        error(Eprocdied);

    switch(QID(c->qid)){

    ⟨procwrite() cases 367c⟩

    case Qctl:
        procctlreq(p, va, n);
        break;

    case Qargs:
        if(n == 0)
            error(Eshort);
        if(n >= ERRMAX)
            error(Etoobig);
        arg = malloc(n+1);
        if(arg == nil)
            error(Enomem);
        memmove(arg, va, n);
        m = n;
        if(arg[m-1] != 0)
```

```
            arg[m++] = 0;
        free(p->args);
        p->nargs = m;
        p->args = arg;
        p->setargs = true;
        break;


    case Qregs:
        if(offset >= sizeof(Ureg))
            n = 0;
        else if(offset+n > sizeof(Ureg))
            n = sizeof(Ureg) - offset;
        if(p->dbgreg == nil)
            error(Enoreg);
        arch_setregisters(p->dbgreg, (char*)(p->dbgreg)+offset, va, n);
        break;

    ⟨procwrite Qfpregs case 400c⟩

    case Qnote:
        if(p->kp)
            error(Eperm);
        if(n >= ERRMAX-1)
            error(Etoobig);
        memmove(buf, va, n);
        buf[n] = 0;
        if(!postnote(p, 0, buf, NUser))
            error("note not posted");
        break;
    case Qnoteid:
        id = atoi(a);
        if(id == p->pid) {
            p->noteid = id;
            break;
        }
        t = proctab(0);
        for(et = t+conf.nproc; t < et; t++) {
            if(t->state == Dead)
                continue;
            if(id == t->noteid) {
                if(strcmp(p->user, t->user) != 0)
                    error(Eperm);
                p->noteid = id;
                break;
            }
        }
        if(p->noteid != id)
            error(Ebadarg);
        break;

    default:
        pprint("unknown qid in procwrite\n");
        error(Egreg);
    }
    poperror();
    qunlock(&p->debug);
    return n;
}
```

⟨*function* pgrpnote 473a⟩≡ (597b)

```
void
pgrpnote(ulong noteid, char *a, long n, int flag)
{
    Proc *p, *ep;
    char buf[ERRMAX];

    if(n >= ERRMAX-1)
        error(Etoobig);

    memmove(buf, a, n);
    buf[n] = 0;
    p = proctab(0);
    ep = p+conf.nproc;
    for(; p < ep; p++) {
        if(p->state == Dead)
            continue;
        if(up != p && p->noteid == noteid && p->kp == 0) {
            qlock(&p->debug);
            if(p->pid == 0 || p->noteid != noteid){
                qunlock(&p->debug);
                continue;
            }
            if(!waserror()) {
                postnote(p, 0, buf, flag);
                poperror();
            }
            qunlock(&p->debug);
        }
    }
}
```

⟨*method procopen* 473b⟩≡ (567)

```
static Chan*
procopen(Chan *c, int omode)
{
    Proc *p;
    Pgrp *pg;
    Chan *tc;
    int pid;

    if(c->qid.type & QTDIR)
        return devopen(c, omode, 0, 0, procgen);

    ⟨procopen() Qtrace if 378e⟩

    p = proctab(SLOT(c->qid));
    qlock(&p->debug);
    if(waserror()){
        qunlock(&p->debug);
        nexterror();
    }
    pid = PID(c->qid);
    if(p->pid != pid)
        error(Eprocdied);

    omode = openmode(omode);

    switch(QID(c->qid)){
```

```
⟨procopen() cases 254c⟩
case Qtext:
    if(omode != OREAD)
        error(Eperm);
    tc = proctext(c, p);
    tc->offset = 0;
    qunlock(&p->debug);
    poperror();
    cclose(c);
    return tc;

case Qproc:
case Qkregs:
case Qsegment:
case Qprofile:
case Qfd:
    if(omode != OREAD)
        error(Eperm);
    break;

case Qnote:
    if(p->privatemem)
        error(Eperm);
    break;

case Qmem:
case Qctl:
    if(p->privatemem)
        error(Eperm);
    nonone(p);
    break;

case Qargs:
case Qnoteid:
case Qstatus:
case Qwait:
case Qregs:
case Qfpregs:
case Qsyscall:
    nonone(p);
    break;


case Qnotepg:
    nonone(p);
    pg = p->pgrp;
    if(pg == nil)
        error(Eprocdied);
    if(omode!=OWRITE || pg->pgrpid == 1)
        error(Eperm);
    c->pgrpid.path = pg->pgrpid+1;
    c->pgrpid.vers = p->noteid;
    break;

default:
    pprint("procopen %#lux\n", QID(c->qid));
    error(Egreg);
}

/* Affix pid to qid */
```

```
    if(p->state != Dead)
        c->qid.vers = p->pid;

    /* make sure the process slot didn't get reallocated while we were playing */
    arch_coherence();
    if(p->pid != pid)
        error(Eprocdied);

    tc = devopen(c, omode, 0, 0, procgen);
    qunlock(&p->debug);
    poperror();

    return tc;
}
```

⟨*method procwstat* 475⟩≡ (567)

```
static int
procwstat(Chan *c, byte *db, int n)
{
    Proc *p;
    DirEntry *d;

    if(c->qid.type&QTDIR)
        error(Eperm);

    if(QID(c->qid) == Qtrace)
        return devwstat(c, db, n);

    p = proctab(SLOT(c->qid));
    nonone(p);
    d = nil;
    if(waserror()){
        free(d);
        qunlock(&p->debug);
        nexterror();
    }
    qlock(&p->debug);

    if(p->pid != PID(c->qid))
        error(Eprocdied);

    if(strcmp(up->user, p->user) != 0 && strcmp(up->user, eve) != 0)
        error(Eperm);

    d = smalloc(sizeof(DirEntry)+n);
    n = convM2D(db, n, &d[0], (char*)&d[1]);
    if(n == 0)
        error(Eshortstat);
    if(!emptystr(d->uid) && strcmp(d->uid, p->user) != 0){
        if(strcmp(up->user, eve) != 0)
            error(Eperm);
        else
            kstrdup(&p->user, d->uid);
    }
    /* p->procmode determines default mode for files in /proc */
    if(d->mode != ~0UL)
        p->procmode = d->mode&0777;

    poperror();
    free(d);
```

```
    qunlock(&p->debug);
    return n;
}
```

⟨*function* procoffset 476a⟩≡ (567)

```
static long
procoffset(long offset, char *va, int *np)
{
    if(offset > 0) {
        offset -= *np;
        if(offset < 0) {
            memmove(va, va+*np+offset, -offset);
            *np = -offset;
        }
        else
            *np = 0;
    }
    return offset;
}
```

⟨*function* procqidwidth 476b⟩≡ (567)

```
static int
procqidwidth(Chan *c)
{
    char buf[32];

    return snprint(buf, sizeof buf, "%lud", c->qid.vers);
}
```

⟨*function* procfdprint 476c⟩≡ (567)

```
int
procfdprint(Chan *c, int fd, int w, char *s, int ns)
{
    int n;

    if(w == 0)
        w = procqidwidth(c);
    n = snprint(s, ns, "%3d %.2s %C %4ld (%.16llux %*lud %.2ux) %5ld %8lld %s\n",
        fd,
        &"r w rw"[(c->mode&3)<<1],
        devtab[c->type]->dc, c->dev,
        c->qid.path, w, c->qid.vers, c->qid.type,
        c->iounit, c->offset, c->path->s);
    return n;
}
```

⟨*function* procgen 476d⟩≡ (567)

```
static int
procgen(Chan *c, char *name, Dirtab *tab, int, int s, DirEntry *dp)
{
    Qid qid;
    Proc *p;
    char *ename;
    Segment *q;
    ulong pid, path, perm, len;

    if(s == DEVDOTDOT){
        mkqid(&qid, Qdir, 0, QTDIR);
        devdir(c, qid, "#p", 0, eve, 0555, dp);
        return 1;
```

```
}

if(c->qid.path == Qdir){
    if(s == 0){
        strcpy(up->genbuf, "trace");
        mkqid(&qid, Qtrace, -1, QTFILE);
        devdir(c, qid, up->genbuf, 0, eve, 0444, dp);
        return 1;
    }

    if(name != nil){
        /* ignore s and use name to find pid */
        pid = strtol(name, &ename, 10);
        if(pid==0 || ename[0]!='\0')
            return -1;
        s = procindex(pid);
        if(s < 0)
            return -1;
    }
    else if(--s >= conf.nproc)
        return -1;

    p = proctab(s);
    pid = p->pid;
    if(pid == 0)
        return 0;
    snprint(up->genbuf, sizeof up->genbuf, "%lud", pid);
    /*
     * String comparison is done in devwalk so name must match its formatted pid
    */
    if(name != nil && strcmp(name, up->genbuf) != 0)
        return -1;
    mkqid(&qid, (s+1)<<QSHIFT, pid, QTDIR);
    devdir(c, qid, up->genbuf, 0, p->user, DMDIR|0555, dp);
    return 1;
}
if(c->qid.path == Qtrace){
    strcpy(up->genbuf, "trace");
    mkqid(&qid, Qtrace, -1, QTFILE);
    devdir(c, qid, up->genbuf, 0, eve, 0444, dp);
    return 1;
}
if(s >= nelem(procdir))
    return -1;
if(tab)
    panic("procgen");

tab = &procdir[s];
path = c->qid.path&~(((1<<QSHIFT)-1));  /* slot component */

/* p->procmode determines default mode for files in /proc */
p = proctab(SLOT(c->qid));
perm = tab->perm;
if(perm == 0)
    perm = p->procmode;
else    /* just copy read bits */
    perm |= p->procmode & 0444;

len = tab->length;
switch(QID(c->qid)) {
```

```
    case Qwait:
        len = p->nwait; /* incorrect size, but >0 means there's something to read */
        break;

    ⟨procgen() Qprofile case 383a⟩
    }

    mkqid(&qid, path|tab->qid.path, c->qid.vers, QTFILE);
    devdir(c, qid, tab->name, len, p->user, perm, dp);
    return 1;
}
```

⟨*function* nonone 478a⟩≡ (567)

```
/*
 *  none can't read or write state on other
 *  processes.  This is to contain access of
 *  servers running as none should they be
 *  subverted by, for example, a stack attack.
 */
static void
nonone(Proc *p)
{
    if(p == up)
        return;
    if(strcmp(up->user, "none") != 0)
        return;
    if(iseve())
        return;
    error(Eperm);
}
```

⟨*function* procfds 478b⟩≡ (567)

```
static int
procfds(Proc *p, char *va, int count, long offset)
{
    Fgrp *f;
    Chan *c;
    char buf[256];
    int n, i, w, ww;
    char *a;

    /* print to buf to avoid holding fgrp lock while writing to user space */
    if(count > sizeof buf)
        count = sizeof buf;
    a = buf;

    qlock(&p->debug);
    f = p->fgrp;
    if(f == nil){
        qunlock(&p->debug);
        return 0;
    }
    lock(f);
    if(waserror()){
        unlock(f);
        qunlock(&p->debug);
        nexterror();
    }

    n = readstr(0, a, count, p->dot->path->s);
```

```
    n += snprint(a+n, count-n, "\n");
    offset = procoffset(offset, a, &n);
    /* compute width of qid.path */
    w = 0;
    for(i = 0; i <= f->maxfd; i++) {
        c = f->fd[i];
        if(c == nil)
            continue;
        ww = procqidwidth(c);
        if(ww > w)
            w = ww;
    }
    for(i = 0; i <= f->maxfd; i++) {
        c = f->fd[i];
        if(c == nil)
            continue;
        n += procfdprint(c, i, w, a+n, count-n);
        offset = procoffset(offset, a, &n);
    }
    unlock(f);
    qunlock(&p->debug);
    poperror();

    /* copy result to user space, now that locks are released */
    memmove(va, buf, n);

    return n;
}
```

⟨*method procclose* 479a⟩≡ (567)

```
static void
procclose(Chan* c)
{
    ⟨procclose() Qtrace if 379a⟩
    ⟨procclose() hooks 254d⟩
}
```

⟨Proc *debugger fields* 479b⟩+≡ (29c) ◁376b

```
bool setargs;
```

⟨newproc() *set fields of* p 479c⟩+≡ (75b) ◁465a 520c▷

```
p->setargs = false;
```

⟨*function* procargs 479d⟩≡ (567)

```
static int
procargs(Proc *p, char *buf, int nbuf)
{
    int j, k, m;
    char *a;
    int n;

    a = p->args;
    if(p->setargs){
        snprint(buf, nbuf, "%s [%s]", p->text, p->args);
        return strlen(buf);
    }
    n = p->nargs;
    for(j = 0; j < nbuf - 1; j += m){
        if(n <= 0)
            break;
```

```
        if(j != 0)
            buf[j++] = ' ';
        m = snprint(buf+j, nbuf-j, "%q",  a);
        k = strlen(a) + 1;
        a += k;
        n -= k;
    }
    return j;
}
```

⟨*function* proctext 480a⟩≡ (567)

```
Chan*
proctext(Chan *c, Proc *p)
{
    Chan *tc;
    KImage *i;
    Segment *s;

    s = p->seg[TSEG];
    if(s == 0)
        error(Enonexist);
    if(p->state==Dead)
        error(Eprocdied);

    lock(s);
    i = s->image;
    if(i == 0) {
        unlock(s);
        error(Eprocdied);
    }
    unlock(s);

    lock(i);
    if(waserror()) {
        unlock(i);
        nexterror();
    }

    tc = i->c;
    if(tc == 0)
        error(Eprocdied);

    if(incref(tc) == 1 || (tc->flag&COPEN) == 0 || tc->mode!=OREAD) {
        cclose(tc);
        error(Eprocdied);
    }

    if(p->pid != PID(c->qid)){
        cclose(tc);
        error(Eprocdied);
    }

    unlock(i);
    poperror();

    return tc;
}
```

⟨*function* procctlcloseone 480b⟩≡ (567)

```
static void
```

```
procctlcloseone(Proc *p, Fgrp *f, int fd)
{
    Chan *c;

    c = f->fd[fd];
    if(c == nil)
        return;
    f->fd[fd] = nil;
    unlock(f);
    qunlock(&p->debug);
    cclose(c);
    qlock(&p->debug);
    lock(f);
}
```

⟨*function* procctlclosefiles 481a⟩≡ (567)

```
void
procctlclosefiles(Proc *p, bool all, int fd)
{
    int i;
    Fgrp *f;

    f = p->fgrp;
    if(f == nil)
        error(Eprocdied);

    lock(f);
    f->ref++;
    if(all)
        for(i = 0; i < f->maxfd; i++)
            procctlcloseone(p, f, i);
    else
        procctlcloseone(p, f, fd);
    unlock(f);
    closefgrp(f);
}
```

⟨*function* parsetime 481b⟩≡ (567)

```
static char *
parsetime(vlong *rt, char *s)
{
    uvlong ticks;
    ulong l;
    char *e, *p;
    static int p10[] = {100000000, 10000000, 1000000, 100000, 10000, 1000, 100, 10, 1};

    if (s == nil)
        return "missing value";
    ticks=strtoul(s, &e, 10);
    if (*e == '.'){
        p = e+1;
        l = strtoul(p, &e, 10);
        if(e-p > nelem(p10))
            return "too many digits after decimal point";
        if(e-p == 0)
            return "ill-formed number";
        l *= p10[e-p-1];
    }else
        l = 0;
    if (*e == '\0' || strcmp(e, "s") == 0){
```

```
        ticks = 1000000000 * ticks + l;
    }else if (strcmp(e, "ms") == 0){
        ticks = 1000000 * ticks + l/1000;
    }else if (strcmp(e, "Âµs") == 0 || strcmp(e, "us") == 0){
        ticks = 1000 * ticks + l/1000000;
    }else if (strcmp(e, "ns") != 0)
        return "unrecognized unit";
    *rt = ticks;
    return nil;
}
```

⟨*function* procctlreq 482⟩≡ (567)

```
// assumes p->debug is held
void
procctlreq(Proc *p, char *va, int n)
{
    Segment *s;
    int npc, pri;
    Cmdbuf *cb;
    Cmdtab *ct;
    vlong time;
    char *e;
    void (*pt)(Proc*, int, vlong);

    if(p->kp)   /* no ctl requests to kprocs */
        error(Eperm);

    cb = parsecmd(va, n);
    if(waserror()){
        free(cb);
        nexterror();
    }

    ct = lookupcmd(cb, proccmd, nelem(proccmd));

    switch(ct->index){

    ⟨procctlreq() CMpri case 146b⟩
    ⟨procctlreq() CMfixedpri case 146c⟩
    ⟨procctlreq() CMwired case 160d⟩
    ⟨procctlreq() CMnoswap case 141e⟩

    ⟨procctlreq() CMprivate case 465d⟩

    ⟨procctlreq() CMhang case 366d⟩
    ⟨procctlreq() CMnohang case 366e⟩

    ⟨procctlreq() CMstop case 365a⟩
    ⟨procctlreq() CMstart case 365b⟩
    ⟨procctlreq() CMwaitstop case 366h⟩

    ⟨procctlreq() CMstartsyscall case 369b⟩
    ⟨procctlreq() CMstartstop case 379e⟩

    ⟨procctlreq() CMtrace case 376c⟩
    ⟨procctlreq() CMevent case 377j⟩

    ⟨procctlreq() CMprofile case 382c⟩
```

```
    ⟨procctlreq() CMkill case 98b⟩

    case CMclose:
        procctlclosefiles(p, false, atoi(cb->f[1]));
        break;
    case CMclosefiles:
        procctlclosefiles(p, true, 0);
        break;


    ⟨procctlreq() optional real-time commands 432j⟩

    }

    poperror();
    free(cb);
}
```

23.4 /fd/

23.4.1 sysdup()

⟨systab *special file syscalls* 483a⟩≡ (26c)

```
[DUP]       sysdup,
```

⟨*syscall dup* 483b⟩≡ (578)

```
// int dup(int oldfd, int newfd);
long
sysdup(ulong* arg)
{
    int fd;
    Chan *c, *oc;
    Fgrp *f = up->fgrp;

    /*
     * Close after dup'ing, so date > #d/1 works
     */
    c = fdtochan(arg[0], -1, false, true);
    fd = arg[1];
    if(fd != -1){
        lock(f);
        if(fd<0 || growfd(f, fd)<0) {
            unlockfgrp(f);
            cclose(c);
            error(Ebadfd);
        }
        if(fd > f->maxfd)
            f->maxfd = fd;

        oc = f->fd[fd];
        f->fd[fd] = c;
        unlockfgrp(f);
        if(oc)
            cclose(oc);
    }else{
        if(waserror()) {
            cclose(c);
            nexterror();
```

```
        }
        fd = newfd(c);
        if(fd < 0)
            error(Enofd);
        poperror();
    }

    return fd;
}
```

23.4.2 /fd/

23.5 /sys/

⟨*devsys.c enum Qxxx* 484a⟩≡ (560a)

```
enum{
    Qdir,

    ⟨devsys.c enum Qxxx cases 486a⟩
};
```

⟨*global* sysdir 484b⟩≡ (560a)

```
static Dirtab sysdir[]={
    ".",    {Qdir, 0, QTDIR},   0,      DMDIR|0555,

    ⟨sysdir fields 486b⟩
};
```

⟨*method sysinit* 484c⟩≡ (560a)

```
static void
sysinit(void)
{
}
```

⟨*method sysattach* 484d⟩≡ (560a)

```
static Chan*
sysattach(char *spec)
{
    return devattach('k', spec);
}
```

⟨*method sysread* 484e⟩≡ (560a)

```
static long
sysread(Chan *c, void *buf, long n, vlong off)
{
    vlong offset = off;
    char tmp[256];
    ⟨sysread() locals 485a⟩

    if(n <= 0)
        return n;

    switch((ulong)c->qid.path){
    case Qdir:
        return devdirread(c, buf, n, sysdir, nelem(sysdir), devgen);

    ⟨sysread() cases 486c⟩
```

```
    default:
        print("sysread %#llux\n", c->qid.path);
        error(Egreg);
    }
    return -1;      /* never reached */
}
```

⟨sysread() *locals* 485a⟩≡ (484e)

```
    char *b, *bp;
    int i, k;
    Cpu *mp;
    int id;
```

⟨*method syswrite* 485b⟩≡ (560a)

```
static long
syswrite(Chan *c, void *va, long n, vlong off)
{
    ulong offset;
    char *a;
    ⟨syswrite() locals 485c⟩

    offset = off;
    a = va;

    switch((ulong)c->qid.path){

    ⟨syswrite() cases 486k⟩

    default:
        print("syswrite: %#llux\n", c->qid.path);
        error(Egreg);
    }
    return n;
}
```

⟨syswrite() *locals* 485c⟩≡ (485b) 488b ▷

```
    char buf[256];
    Cpu *mp;
    int id;
```

⟨*global* sysdevtab 485d⟩≡ (560a)

```
Dev sysdevtab = {
    .dc       =    'k',
    .name     =    "sys",

    .reset    =    devreset,
    .init     =    sysinit,
    .shutdown =    devshutdown,
    .attach   =    sysattach,
    .walk     =    syswalk,
    .stat     =    sysstat,
    .open     =    sysopen,
    .create   =    devcreate,
    .close    =    sysclose,
    .read     =    sysread,
    .bread    =    devbread,
    .write    =    syswrite,
    .bwrite   =    devbwrite,
    .remove   =    devremove,
    .wstat    =    devwstat,
};
```

23.5.1 /sys/osversion

⟨*devsys.c enum Qxxx cases* 486a⟩≡ (484a) 486e▷
```
Qosversion,
```

⟨sysdir *fields* 486b⟩≡ (484b) 486f▷
```
"osversion",    {Qosversion},   0,      0444,
```

⟨sysread() *cases* 486c⟩≡ (484e) 486g▷
```
case Qosversion:
    snprint(tmp, sizeof tmp, "pad's version");
    n = readstr((ulong)offset, buf, n, tmp);
    return n;
```

23.5.2 /sys/config

⟨*devsys.c decls and globals* 486d⟩≡ (560a) 488a▷
```
extern byte configfile[]; // in $CONF.c
```

⟨*devsys.c enum Qxxx cases* 486e⟩+≡ (484a) ◁486a 486h▷
```
Qconfig,
```

⟨sysdir *fields* 486f⟩+≡ (484b) ◁486b 486i▷
```
"config",   {Qconfig},  0,      0444,
```

⟨sysread() *cases* 486g⟩+≡ (484e) ◁486c 486j▷
```
case Qconfig:
    return readstr((ulong)offset, buf, n, (char*) configfile);
```

23.5.3 /ys/hosthowner, /sys/hostdomain

⟨*devsys.c enum Qxxx cases* 486h⟩+≡ (484a) ◁486e 487a▷
```
Qhostowner,
Qhostdomain,
```

⟨sysdir *fields* 486i⟩+≡ (484b) ◁486f 487b▷
```
"hostowner",    {Qhostowner},   0,      0664,
"hostdomain",   {Qhostdomain},  DOMLEN,     0664,
```

⟨sysread() *cases* 486j⟩+≡ (484e) ◁486g 487c▷
```
case Qhostowner:
    return readstr((ulong)offset, buf, n, eve);

case Qhostdomain:
    return readstr((ulong)offset, buf, n, hostdomain);
```

⟨syswrite() *cases* 486k⟩≡ (485b) 487d▷
```
case Qhostowner:
    return hostownerwrite(a, n);

case Qhostdomain:
    return hostdomainwrite(a, n);
```

23.5.4 /sys/sysname

⟨*global* sysname 486l⟩≡ (560a)
```
char    *sysname;
```

⟨*devsys.c enum Qxxx cases* 487a⟩+≡ (484a) ◁486h 487e▷

```
Qsysname,
```

⟨sysdir *fields* 487b⟩+≡ (484b) ◁486i 487f▷

```
"sysname",  {Qsysname}, 0,      0664,
```

⟨sysread() *cases* 487c⟩+≡ (484e) ◁486j 487g▷

```
case Qsysname:
    if(sysname == nil)
        return 0;
    return readstr((ulong)offset, buf, n, sysname);
```

⟨syswrite() *cases* 487d⟩+≡ (485b) ◁486k 488c▷

```
case Qsysname:
    if(offset != 0)
        error(Ebadarg);
    if(n <= 0 || n >= sizeof buf)
        error(Ebadarg);
    strncpy(buf, a, n);
    buf[n] = 0;
    if(buf[n-1] == '\n')
        buf[n-1] = 0;
    kstrdup(&sysname, buf);
    break;
```

23.5.5 /sys/drivers

⟨*devsys.c enum Qxxx cases* 487e⟩+≡ (484a) ◁487a 487h▷

```
Qdrivers,
```

⟨sysdir *fields* 487f⟩+≡ (484b) ◁487b 487i▷

```
"drivers",  {Qdrivers}, 0,      0444,
```

⟨sysread() *cases* 487g⟩+≡ (484e) ◁487c 514f▷

```
case Qdrivers:
    b = malloc(READSTR);
    if(b == nil)
        error(Enomem);
    k = 0;
    for(i = 0; devtab[i] != nil; i++)
        k += snprint(b+k, READSTR-k, "#%C %s\n",
            devtab[i]->dc, devtab[i]->name);
    if(waserror()){
        free(b);
        nexterror();
    }
    n = readstr((ulong)offset, buf, n, b);
    free(b);
    poperror();
    return n;
```

23.5.6 /sys/reboot

⟨*devsys.c enum Qxxx cases* 487h⟩+≡ (484a) ◁487e 514d▷

```
Qreboot,
```

⟨sysdir *fields* 487i⟩+≡ (484b) ◁487f 514e▷

```
"reboot",   {Qreboot},  0,      0660,
```

⟨*devsys.c decls and globals* 488a⟩+≡ (560a) ◁486d

```
enum
{
    CMhalt,
    CMreboot,
    CMpanic,
};

Cmdtab rebootmsg[] =
{
    CMhalt,     "halt",     1,
    CMreboot,   "reboot",   0,
    CMpanic,    "panic",    0,
};
```

⟨`syswrite()` *locals* 488b⟩+≡ (485b) ◁485c

```
Cmdbuf *cb;
Cmdtab *ct;
```

⟨`syswrite()` *cases* 488c⟩+≡ (485b) ◁487d 515a▷

```
case Qreboot:
    if(!iseve())
        error(Eperm);
    cb = parsecmd(a, n);

    if(waserror()) {
        free(cb);
        nexterror();
    }
    ct = lookupcmd(cb, rebootmsg, nelem(rebootmsg));
    switch(ct->index) {
    case CMhalt:
        arch_reboot(nil, 0, 0);
        break;
    case CMreboot:
        rebootcmd(cb->nf-1, cb->f+1);
        break;
    case CMpanic:
        *(ulong*)0=0;
        panic("/dev/reboot");
    }
    poperror();
    free(cb);
    break;
```

⟨`kbdputsc()` *reboot if ctl-alt-del* 488d⟩≡ (275d)

```
if(kbscan->ctl)
    if(kbscan->alt && c == Del) // Ctl-Alt-Del
        arch_exit(0);
```

⟨`ConsKbd` *other fields* 488e⟩+≡ (278b) ◁290c

```
bool ctlpoff; // ^P will not reboot if true
```

⟨`conswrite()` *Qconsctl other ifs* 488f⟩≡ (291a)

```
else if(strncmp(a, "ctlpon", 6) == 0){
    kbd.ctlpoff = false;
} else if(strncmp(a, "ctlpoff", 7) == 0){
    kbd.ctlpoff = true;
}
```

⟨echo() *special key C-p* 489a⟩≡ (500)

```
case 0x10:  /* ^P */
    if(cpuserver && !kbd.ctlpoff){
        active.exiting = true;
        return;
    }
    break;
```

⟨Active *other fields* 489b⟩≡ (29a) 490b ▷

```
bool ispanic;    /* shutdown in response to a panic */
```

⟨*function* chandevshutdown 489c⟩≡ (571)

```
void
chandevshutdown(void)
{
    int i;

    /* shutdown in reverse order */
    for(i=0; devtab[i] != nil; i++)
        ;
    for(i--; i >= 0; i--)
        devtab[i]->shutdown();
}
```

⟨Dev *methods* 489d⟩+≡ (32d) ◁ 323a

```
void  (*shutdown)(void);
```

arch_exit()(ARM)

⟨*function* main_arch_exit*(arm)* 489e⟩≡ (628c)

```
/*
 *  exit kernel either on a panic or user request
 */
void
main_arch_exit(int code)
{
    void (*f)(ulong, ulong, ulong);

    shutdown(code);
    splfhi();
    if(cpu->cpuno == 0)
        archreboot();
    else{
        f = (void*)REBOOTADDR;
        intrcpushutdown();
        cacheuwbinv();
        l2cacheuwbinv();
        (*f)(0, 0, 0);
        for(;;){}
    }
}
```

⟨*constant* REBOOTADDR*(arm)* 489f⟩≡ (633b)

```
/* address at which to copy and execute rebootcode */
#define REBOOTADDR  (KZERO+0x1800)
```

⟨*function* shutdown*(*(init/arm/main.c)(arm)*)* 490a⟩≡ (628c)

```
static void
shutdown(int ispanic)
{
    int ms, once;

    lock(&active);
    if(ispanic)
        active.ispanic = ispanic;
    else if(cpu->cpuno == 0 && (active.cpus & (1 << cpu->cpuno)) == 0)
        active.ispanic = 0;
    once = active.cpus & (1 << cpu->cpuno);
    active.cpus &= ~(1 << cpu->cpuno);
    active.exiting = 1;
    unlock(&active);

    if(once)
        iprint("cpu%d: exiting\n", cpu->cpuno);
    arch_spllo();
    for(ms = 5*1000; ms > 0; ms -= TK2MS(2)){
        arch_delay(TK2MS(2));
        if(active.cpus == 0 && consactive() == 0)
            break;
    }
    arch_delay(100*cpu->cpuno);
}
```

⟨Active *other fields* 490b⟩+≡ (29a) ◁489b

```
bool rebooting;    /* just idle cpus > 0 */
```

⟨*function* arch_reboot*(arm)* 490c⟩≡ (628c)

```
/*
 * the new kernel is already loaded at address ‘code’
 * of size ‘size’ and entry point ‘entry’.
 */
void
arch_reboot(void *entry, void *code, ulong size)
{
    void (*f)(ulong, ulong, ulong);

    //writeconf();

    /*
     * the boot processor is cpu0.  execute this function on it
     * so that the new kernel has the same cpu0.
     */
    if (cpu->cpuno != 0) {
        procwired(up, 0);
        sched();
    }
    if (cpu->cpuno != 0)
        print("on cpu%d (not 0)!\n", cpu->cpuno);

    /* setup reboot trampoline function */
    f = (void*)REBOOTADDR;
    memmove(f, rebootcode, sizeof(rebootcode));
    cachedwbse(f, sizeof(rebootcode));

    shutdown(0);
```

```
    /*
     * should be the only processor running now
     */

    arch_delay(5000);
    print("active.machs = %x\n", active.cpus);
    print("reboot entry %#lux code %#lux size %ld\n",
        PADDR(entry), PADDR(code), size);
    arch_delay(100);

    /* turn off buffered serial console */
    serialoq = nil;
    kprintoq = nil;
    screenputs = nil;

    /* shutdown devices */
    if(!waserror()){
        chandevshutdown();
        poperror();
    }

    /* stop the clock (and watchdog if any) */
    clockshutdown();

    splfhi();
    intrshutdown();

    /* off we go - never to return */
    cacheuwbinv();
    l2cacheuwbinv();
    (*f)(PADDR(entry), PADDR(code), size);

    iprint("loaded kernel returned!\n");
    arch_delay(1000);
    archreboot();
}
```

⟨*function* intrshutdown*(arm)* 491a⟩≡ (636c)

```
/* called from cpu0 after other cpus are shutdown */
void
intrshutdown(void)
{
    intrsoff();
    intrcpushutdown();
}
```

⟨*function* clockshutdown*(arm)* 491b⟩≡ (639b)

```
void
clockshutdown(void)
{
    Armtimer *tm;

    tm = (Armtimer*)ARMTIMER;
    tm->ctl = 0;
    //wdogoff();
}
```

⟨*function* intrcpushutdown*(arm)* 491c⟩≡ (636c)

```
void
intrcpushutdown(void)
```

```
{
    u32int *enable;

    if(soc.armlocal == 0)
        return;
    enable = (u32int*)(LOCALREGS + Localtimerint) + cpu->cpuno;
    *enable = 0;
    if(cpu->cpuno){
        enable = (u32int*)(LOCALREGS + Localmboxint) + cpu->cpuno;
        *enable = 1;
    }
}
```

rebootcode.s (arm)

⟨*constant* PTEDRAM*(arm)* 492a⟩≡ (632)

```
#define PTEDRAM     (Dom0|L1AP(Krw)|Section)
```

⟨*instruction WFI(arm)* 492b⟩≡ (632)

```
#define WFI WORD    $0xe320f003 /* wait for interrupt */
```

⟨*instruction WFE(arm)* 492c⟩≡ (632)

```
#define WFE WORD    $0xe320f002 /* wait for event */
```

⟨*function main(rebootcode.s)(arm)* 492d⟩≡ (632)

```
/*
 * Turn off MMU, then copy the new kernel to its correct location
 * in physical memory.  Then jump to the start of the kernel.
 */

/* main(PADDR(entry), PADDR(code), size); */
TEXT    main(SB), 1, $-4
    MOVW    $setR12(SB), R12

    /* copy in arguments before stack gets unmapped */
    MOVW    R0, R8          /* entry point */
    MOVW    p2+4(FP), R9        /* source */
    MOVW    n+8(FP), R6     /* byte count */

    /* SVC mode, interrupts disabled */
    MOVW    $(PsrDirq|PsrDfiq|PsrMsvc), R1
    MOVW    R1, CPSR

    /* prepare to turn off mmu  */
    BL  cachesoff(SB)

    /* turn off mmu */
    MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    BIC $CpCmmu, R1
    MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl

    /* continue with reboot only on cpu0 */
    CPUID(R2)
    BEQ bootcpu

    /* other cpus wait for inter processor interrupt from cpu0 */
    /* turn icache back on */
    MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    ORR $(CpCicache), R1
```

```
    MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    BARRIERS
dowfi:
    WFI
    MOVW    $0x40000060, R1
    ADD     R2<<2, R1
    MOVW    0(R1), R0
    AND     $0x10, R0
    BEQ     dowfi
    MOVW    $0x8000, R1
    BL      (R1)
    B       dowfi

bootcpu:
    /* set up a tiny stack for local vars and memmove args */
    MOVW    R8, SP          /* stack top just before kernel dest */
    SUB $20, SP         /* allocate stack frame */

    /* copy the kernel to final destination */
    MOVW    R8, 16(SP)      /* save dest (entry point) */
    MOVW    R8, R0          /* first arg is dest */
    MOVW    R9, 8(SP)       /* push src */
    MOVW    R6, 12(SP)      /* push size */
    BL  memmove(SB)
    MOVW    16(SP), R8      /* restore entry point */

    /* jump to kernel physical entry point */
    ORR R8,R8
    B   (R8)
    B   0(PC)

/*
 * turn the caches off, double map PHYSDRAM & KZERO, invalidate TLBs, revert
 * to tiny addresses.  upon return, it will be safe to turn off the mmu.
 * clobbers R0-R2, and returns with SP invalid.
 */
TEXT cachesoff(SB), 1, $-4
    MOVM.DB.W [R14,R1-R10], (R13)       /* save regs on stack */

    /* turn caches off, invalidate icache */
    MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    BIC $(CpCdcache|CpCicache|CpCpredict), R1
    MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall

    /* invalidate stale TLBs before changing them */
    BARRIERS
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
    BARRIERS

    /* redo double map of first MiB PHYSDRAM = KZERO */
    MOVW    12(R(CPU)), R2      /* m->mmul1 (virtual addr) */
    MOVW    $PTEDRAM, R1            /* PTE bits */
    MOVW    R1, (R2)

    /* invalidate stale TLBs again */
    BARRIERS
    MOVW    $0, R0
```

```
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
    BARRIERS

    /* relocate SB and return address to PHYSDRAM addressing */
    MOVW    $KSEGM, R1      /* clear segment bits */
    BIC R1, R12         /* adjust SB */
    MOVM.IA.W (R13), [R14,R1-R10]       /* restore regs from stack */

    MOVW    $KSEGM, R1      /* clear segment bits */
    BIC R1, R14         /* adjust return address */

    RET
```

23.6 /dev/

⟨*devcons.c enum Qxxx cases* 494a⟩≡ (283b) 508a ▷

```
  Qbintime,
  Qcputime,
  Qnull,
  Qpgrpid,
  Qpid,
  Qppid,
  Qrandom,
  Qswap,
  Qtime,
  Quser,
  Qzero,
```

⟨consdir *fields* 494b⟩≡ (283c) 508b ▷

```
  "bintime",  {Qbintime}, 24,     0664,
  "cputime",  {Qcputime}, 6*NUMSIZE,  0444,
  "null",     {Qnull},    0,      0666,
  "pgrpid",   {Qpgrpid},  NUMSIZE,    0444,
  "pid",      {Qpid},     NUMSIZE,    0444,
  "ppid",     {Qppid},    NUMSIZE,    0444,
  "random",   {Qrandom},  0,      0444,
  "swap",     {Qswap},    0,      0664,
  "time",     {Qtime},    NUMSIZE+3*VLNUMSIZE,    0664,
  "user",     {Quser},    0,      0666,
  "zero",     {Qzero},    0,      0444,
```

⟨consinit() *initializing things* 494c⟩+≡ (288a) ◁ 279b

```
  todinit();
  randominit();
```

23.6.1 /dev/random

⟨*global* randn 494d⟩≡ (531)

```
  static  ulong   randn;
```

⟨*function* seedrand 494e⟩≡ (531)

```
  static void
  seedrand(void)
  {
      if(!waserror()){
          randomread((void*)&randn, sizeof(randn));
          poperror();
      }
  }
```

⟨*function* nrand 495a⟩≡ (531)

```
int
nrand(int n)
{
    if(randn == 0)
        seedrand();
    randn = randn*1103515245 + 12345 + CPUS(0)->ticks;
    return (randn>>16) % n;
}
```

⟨*struct* Rb 495b⟩≡ (592a)

```
struct Rb
{
    QLock;
    Rendez  producer;
    Rendez  consumer;
    ulong   randomcount;
    byte   buf[1024];
    byte   *ep;
    byte   *rp;
    byte   *wp;
    byte   next;
    byte   wakeme;
    ushort  bits;
    ulong   randn;
};
```

⟨*global* rb 495c⟩≡ (592a)

```
struct Rb rb;
```

⟨*struct* rbnotfull 495d⟩≡ (592a)

```
static int
rbnotfull(void*)
{
    int i;

    i = rb.rp - rb.wp;
    return i != 1 && i != (1 - sizeof(rb.buf));
}
```

⟨*struct* rbnotempty 495e⟩≡ (592a)

```
static int
rbnotempty(void*)
{
    return rb.wp != rb.rp;
}
```

⟨*function* genrandom 495f⟩≡ (592a)

```
static void
genrandom(void*)
{
    up->basepri = PriNormal;
    up->priority = up->basepri;

    for(;;){
        for(;;)
            if(++rb.randomcount > 100000)
                break;
        if(anyhigher())
            sched();
```

```
        if(!rbnotfull(0))
            sleep(&rb.producer, rbnotfull, 0);
    }
}
```

⟨*clock callback randomclock* 496a⟩≡ (592a)

```
/*
 *  produce random bits in a circular buffer
 */
static void
randomclock(void)
{
    if(rb.randomcount == 0 || !rbnotfull(0))
        return;

    rb.bits = (rb.bits<<2) ^ rb.randomcount;
    rb.randomcount = 0;

    rb.next++;
    if(rb.next != 8/2)
        return;
    rb.next = 0;

    *rb.wp ^= rb.bits;
    if(rb.wp+1 == rb.ep)
        rb.wp = rb.buf;
    else
        rb.wp = rb.wp+1;

    if(rb.wakeme)
        wakeup(&rb.consumer);
}
```

⟨*function* randominit 496b⟩≡ (592a)

```
void
randominit(void)
{
    /* Frequency close but not equal to HZ */
    addclock0link(randomclock, 13);
    rb.ep = rb.buf + sizeof(rb.buf);
    rb.rp = rb.wp = rb.buf;
    kproc("kgenrandom", genrandom, 0);
}
```

⟨*function* randomread 496c⟩≡ (592a)

```
/*
 *  consume random bytes from a circular buffer
 */
ulong
randomread(void *xp, ulong n)
{
    byte *e, *p;
    ulong x;

    p = xp;

    if(waserror()){
        qunlock(&rb);
        nexterror();
    }
```

```
    qlock(&rb);
    for(e = p + n; p < e; ){
        if(rb.wp == rb.rp){
            rb.wakeme = 1;
            wakeup(&rb.producer);
            sleep(&rb.consumer, rbnotempty, 0);
            rb.wakeme = 0;
            continue;
        }

        /*
         *  beating clocks will be predictable if
         *  they are synchronized.  Use a cheap pseudo-
         *  random number generator to obscure any cycles.
         */
        x = rb.randn*1103515245 ^ *rb.rp;
        *p++ = rb.randn = x;

        if(rb.rp+1 == rb.ep)
            rb.rp = rb.buf;
        else
            rb.rp = rb.rp+1;
    }
    qunlock(&rb);
    poperror();

    wakeup(&rb.producer);

    return n;
}
```

⟨consread() *cases* 497⟩+≡ (288d) ◁380f 508c▷

```
 case Qrandom:
     return randomread(buf, n);
```

Chapter 24

Conclusion

Appendix A

Debugging

A.1 sysnop()

A.2 Special keys and dumpers

⟨*global* kdebug 499a⟩≡ (536)
```
bool kdebug;
```

⟨kbdputsc() *special keyboard debug keys cases* 499b⟩≡ (275d)
```
case KF|11:
    print("kbd debug on, F12 turns it off\n");
    kdebug = true;
    kbdputc(c);
    break;
case KF|12:
    kdebug = false;
    kbdputc(c);
    break;
```

⟨kbdputsc() *debugging* 499c⟩≡ (275d)
```
if(kdebug)
    print("sc %x (ms %d)\n", k, mouseshifted);
```

⟨kbdputsc() *debugging up shift* 499d⟩≡ (275d)
```
if(kdebug)
    print("shiftclr\n");
```

⟨kbdputsc() *debugging down shift* 499e⟩≡ (275d)
```
if(kdebug)
    print("shift\n");
```

⟨kbdputc() *debugging* 499f⟩≡ (278d)
```
extern bool kdebug;

if(kdebug)
    print("kbdputc(0x%x)\n", ch);
```

⟨echo() *locals* 499g⟩≡ (279c)
```
static int ctrlt;
void* tmp;
int x;
char *e, *p;
```

⟨echo() *special keys handler* 500⟩≡ (279c)

```
    e = buf+n;
    for(p = buf; p < e; p++){
        switch(*p){

        ⟨echo() special key C-p 489a⟩

        case 0x14:  /* ^T */
            ctrlt++;
            if(ctrlt > 2)
                ctrlt = 2;
            continue;
        }
        if(ctrlt != 2)
            continue;

        /* ^T escapes */
        ctrlt = 0;
        switch(*p){
        case 's':
            arch_dumpstack();
            return;
        case 'S':
            x = arch_splhi();
            arch_dumpstack();
            procdump();
            arch_splx(x);
            return;
        case 'x':
            xsummary();
            tmp = xalloc(1000); // PAD: just to debug
            xalloc(1000);
            xfree(tmp);
            xsummary();
            return;
        case 'X':
            xsummary();
            ixsummary();
            mallocsummary();
            arch_memorysummary();
            pagersummary();
            return;
        case 'm':
            arch_memorysummary();
            return;
        case 'p':
            x = arch_spllo();
            procdump();
            arch_splx(x);
            return;
        case 'q':
            scheddump();
            return;
        case 'k':
            killbig("^t ^t k");
            return;
        case 'r':
            arch_exit(0);
            return;
        ⟨echo() C-t C-t special keys handler other cases 509g⟩
```

```
        }
    }
```

⟨*function* xsummary 501a⟩≡ (584b)

```
void
xsummary(void)
{
    int i;
    Hole *h;

    for(i = 0; i < Nhole && xlists.hole[i].top != 0; i++) {
        print("|i| = %d (0x%luX), addr 0x%luX, top = 0x%luX, size = %ld, link = 0x%luX\n",
              i, &xlists.hole[i],
              xlists.hole[i].addr, xlists.hole[i].top, xlists.hole[i].size,
              xlists.hole[i].next
              );
    }
    print("flists = 0x%luX, sorted_holes = 0x%luX\n", xlists.unused_slots, xlists.sorted_holes);
    i = 0;
    for(h = xlists.unused_slots; h; h = h->next)
        i++;

    print("%d holes free", i);
    i = 0;
    for(h = xlists.sorted_holes; h; h = h->next) {
        if (0) {
            print("addr %#.8lux top %#.8lux size %lud\n",
                h->addr, h->top, h->size);
            arch_delay(10);
        }
        i += h->size;
        if (h == h->next) {
            print("xsummary: infinite loop broken\n");
            break;
        }
    }
    print(" %d bytes free\n", i);
}
```

⟨*function* mallosummary 501b⟩≡ (585b)

```
void
mallocsummary(void)
{
    poolsummary(mainmem);
    poolsummary(imagmem);
}
```

⟨*function* poolsummary 501c⟩≡ (585b)

```
void
poolsummary(Pool *p)
{
    print("%s max %lud cur %lud free %lud alloc %lud\n", p->name,
        p->maxsize, p->cursize, p->curfree, p->curalloc);
}
```

⟨*function* arch_memorysummary*(arm)* 501d⟩≡ (628c)

```
// called from devcons.c
void
arch_memorysummary(void) {
}
```

⟨*function* pagersummary 502a⟩≡ (590)

```
void
pagersummary(void)
{
    print("%lud/%lud memory %lud/%lud swap %d iolist\n",
        palloc.user-palloc.freecount,
        palloc.user, conf.nswap-swapalloc.free, conf.nswap,
        ioptr);
}
```

⟨*function* ixsummary 502b⟩≡ (576b)

```
void
ixsummary(void)
{
    debugging ^= 1;
    iallocsummary();
    print("pad %lud, concat %lud, pullup %lud, copy %lud\n",
        padblockcnt, concatblockcnt, pullupblockcnt, copyblockcnt);
    print("consume %lud, produce %lud, qcopy %lud\n",
        consumecnt, producecnt, qcopycnt);
}
```

⟨*function* iallocsummary 502c⟩≡ (574)

```
void
iallocsummary(void)
{
    print("ialloc %lud/%lud\n", ialloc.bytes, conf.ialloc);
}
```

⟨*function* scheddump 502d⟩≡ (599c)

```
void
scheddump(void)
{
    Proc *p;
    Schedq *rq;

    for(rq = &runq[Nrq-1]; rq >= runq; rq--){
        if(rq->head == 0)
            continue;
        print("rq%ld:", rq-runq);
        for(p = rq->head; p; p = p->rnext)
            print(" %lud(%lud)", p->pid, cpu->ticks - p->readytime);
        print("\n");
        arch_delay(150);
    }
    print("nrdy %d\n", nrdy);
}
```

⟨*function* procdump 502e⟩≡ (599c)

```
void
procdump(void)
{
    int i;
    Proc *p;

    if(up)
        print("up %lud\n", up->pid);
    else
        print("no current process\n");
    for(i=0; i<conf.nproc; i++) {
```

```
        p = &procalloc.arena[i];
        if(p->state == Dead)
            continue;

        dumpaproc(p);
    }
}
```

⟨*function* dumpaproc 503a⟩≡ (599c)

```
void
proc_dumpaproc(Proc *p)
{
    ulong bss;
    char *s;

    if(p == nil)
        return;

    bss = 0;
    if(p->seg[BSEG])
        bss = p->seg[BSEG]->top;

    s = p->psstate;
    if(s == nil)
        s = statename[p->state];
    print("%3lud:%10s pc %8lux dbgpc %8lux  %8s (%s) ut %ld st %ld bss %lux qpc %lux nl %lud nd %lud lpc %lux p
        p->pid, p->text, p->pc, arch_dbgpc(p),  s, statename[p->state],
        p->time[TUser], p->time[TSys], bss, p->qpc, p->nlocks.ref, p->delaysched, p->lastlock ? p->lastlock->pc
}
```

⟨*function* arch_dbgpc*(arm)* 503b⟩≡ (641a)

```
/*
 *  pc output by dumpaproc
 */
uintptr
arch_dbgpc(Proc* p)
{
    Ureg *ureg;

    ureg = p->dbgreg;
    if(ureg == 0)
        return 0;

    return ureg->pc;
}
```

⟨*function* lockloop 503c⟩≡ (529b)

```
void
lockloop(Lock *l, ulong pc)
{
    Proc *p;

    p = l->p;
    print("lock %#p loop key %#lux pc %#lux held by pc %#lux proc %lud\n",
        l, l->key, pc, l->pc, p ? p->pid : 0);
    dumpaproc(up);
    if(p != nil)
        dumpaproc(p);
}
```

⟨*function* dumpregs*(arm)* 504a⟩≡ (636c)

```
void
dumpregs(Ureg* ureg)
{
    int s;

    if (ureg == nil) {
        iprint("trap: no user process\n");
        return;
    }
    s = arch_splhi();
    iprint("trap: %s", trapname(ureg->type));
    if(ureg != nil && (ureg->psr & PsrMask) != PsrMsvc)
        iprint(" in %s", trapname(ureg->psr));
    iprint("\n");
    iprint("psr %8.8lux type %2.2lux pc %8.8lux link %8.8lux\n",
        ureg->psr, ureg->type, ureg->pc, ureg->link);
    iprint("R14 %8.8lux R13 %8.8lux R12 %8.8lux R11 %8.8lux R10 %8.8lux\n",
        ureg->r14, ureg->r13, ureg->r12, ureg->r11, ureg->r10);
    iprint("R9  %8.8lux R8  %8.8lux R7  %8.8lux R6  %8.8lux R5  %8.8lux\n",
        ureg->r9, ureg->r8, ureg->r7, ureg->r6, ureg->r5);
    iprint("R4  %8.8lux R3  %8.8lux R2  %8.8lux R1  %8.8lux R0  %8.8lux\n",
        ureg->r4, ureg->r3, ureg->r2, ureg->r1, ureg->r0);
    iprint("stack is at %#p\n", ureg);
    iprint("pc %#lux link %#lux\n", ureg->pc, ureg->link);

    if(up)
        iprint("user stack: %#p-%#p\n", up->kstack, up->kstack+KSTACK-4);
    else
        iprint("kernel stack: %8.8lux-%8.8lux\n",
            (ulong)(cpu+1), (ulong)cpu+BY2PG-4);
    dumplongs("stack", (ulong *)(ureg + 1), 16);
    arch_delay(2000);
    arch_dumpstack();
    arch_splx(s);
}
```

⟨*function* dumplongs*(arm)* 504b⟩≡ (636c)

```
static void
dumplongs(char *msg, ulong *v, int n)
{
    int i, l;

    l = 0;
    iprint("%s at %.8p: ", msg, v);
    for(i=0; i<n; i++){
        if(l >= 4){
            iprint("\n    %.8p: ", v);
            l = 0;
        }
        if(isvalidaddr(v)){
            iprint(" %.8lux", *v++);
            l++;
        }else{
            iprint(" invalid");
            break;
        }
    }
    iprint("\n");
}
```

⟨*function* `isvalidaddr`*(arm)* 505a⟩≡ (636c)

```
int
isvalidaddr(void *v)
{
    return (uintptr)v >= KZERO;
}
```

⟨*function* `trapname`*(arm)* 505b⟩≡ (636c)

```
static char *
trapname(int psr)
{
    char *s;

    s = trapnames[psr & PsrMask];
    if(s == nil)
        s = "unknown trap number in psr";
    return s;
}
```

⟨`faultarm()` *if debug* 505c⟩≡ (116d)

```
if (Debug)
    ckfaultstuck(va);
```

⟨*function* `ckfaultstuck`*(arm)* 505d⟩≡ (636c)

```
/* this is quite helpful during mmu and cache debugging */
static void
ckfaultstuck(uintptr va)
{
    static int cnt, lastpid;
    static uintptr lastva;

    if (va == lastva && up->pid == lastpid) {
        ++cnt;
        if (cnt >= 2)
            /* fault() isn't fixing the underlying cause */
            panic("fault: %d consecutive faults for va %#p",
                cnt+1, va);
    } else {
        cnt = 0;
        lastva = va;
        lastpid = up->pid;
    }
}
```

A.3 /bin/ktrace (ARM)

⟨*function* `trap_arch_dumpstack`*(arm)* 505e⟩≡ (636c)

```
void
trap_arch_dumpstack(void)
{
    arch_callwithureg(dumpstackwithureg);
}
```

⟨*function* `arch_callwithureg`*(arm)* 505f⟩≡ (636c)

```
void
arch_callwithureg(void (*fn)(Ureg*))
{
    Ureg ureg;
```

```
    getpcsp((ulong*)&ureg.pc, (ulong*)&ureg.sp);
    ureg.r14 = getcallerpc(&fn);
    fn(&ureg);
}
```

⟨*function* getpcsp*(arm)* 506a⟩≡ (636c)

```
/*
 * Fill in enough of Ureg to get a stack trace, and call a function.
 * Used by debugging interface rdb.
 */
static void
getpcsp(ulong *pc, ulong *sp)
{
    *pc = getcallerpc(&pc);
    *sp = (ulong)&pc-4;
}
```

⟨*function* dumpstackwithureg*(arm)* 506b⟩≡ (636c)

```
static void
dumpstackwithureg(Ureg *ureg)
{
    uintptr l, i, v, estack;
    u32int *p;
    char *s;

    if((s = getconf("*nodumpstack")) != nil && strcmp(s, "0") != 0){
        iprint("dumpstack disabled\n");
        return;
    }
    iprint("ktrace /kernel/path %#.8lux %#.8lux %#.8lux # pc, sp, link\n",
        ureg->pc, ureg->sp, ureg->r14);
    arch_delay(2000);
    i = 0;
    if(up != nil && (uintptr)&l <= (uintptr)up->kstack+KSTACK)
        estack = (uintptr)up->kstack+KSTACK;
    else if((uintptr)&l >= (uintptr)cpu->stack
         && (uintptr)&l <= (uintptr)cpu + CPUSIZE)
        estack = (uintptr)cpu + CPUSIZE;
    else{
        if(up != nil)
            iprint("&up->kstack %#p &l %#p\n", up->kstack, &l);
        else
            iprint("&m %#p &l %#p\n", cpu, &l);
        return;
    }
    for(l = (uintptr)&l; l < estack; l += sizeof(uintptr)){
        v = *(uintptr*)l;
        if(KTZERO < v && v < (uintptr)etext && !(v & 3)){
            v -= sizeof(u32int);        /* back up an instr */
            p = (u32int*)v;
            if((*p & 0x0f000000) == 0x0b000000){    /* BL instr? */
                iprint("%#8.8lux=%#8.8lux ", l, v);
                i++;
            }
        }
        if(i == 4){
            i = 0;
            iprint("\n");
        }
```

```
    }
    if(i)
        iprint("\n");
}
```

A.4 /dev/kmesg

⟨*struct* KMesg 507a⟩≡ (531)

```
/*
 * Log console output so it can be retrieved via /dev/kmesg.
 * This is good for catching boot-time messages after the fact.
 */
struct KMesg {
    Lock lk;
    char buf[KMESGSIZE];
    uint n;
};
```

⟨*global* kmesg 507b⟩≡ (531)

```
struct KMesg kmesg;
```

⟨*constant* KMESGSIZE*(arm)* 507c⟩≡ (619a)

```
#define KMESGSIZE (16*1024) // for /dev/kmesg
```

⟨echo() *hooks* 507d⟩+≡ (279c) ◁280a 510c▷

```
kmesgputs(buf, n);
```

⟨putstrn0() *kmesg handling* 507e⟩≡ (285b)

```
/*
 *  how many different output devices do we need?
 */
kmesgputs(str, n);
```

⟨*function* kmesgputs 507f⟩≡ (531)

```
static void
kmesgputs(char *str, int n)
{
    uint nn, d;

    ilock(&kmesg.lk);
    /* take the tail of huge writes */
    if(n > sizeof kmesg.buf){
        d = n - sizeof kmesg.buf;
        str += d;
        n -= d;
    }

    /* slide the buffer down to make room */
    nn = kmesg.n;
    if(nn + n >= sizeof kmesg.buf){
        d = nn + n - sizeof kmesg.buf;
        if(d)
            memmove(kmesg.buf, kmesg.buf+d, sizeof kmesg.buf-d);
        nn -= d;
    }

    /* copy the data in */
    memmove(kmesg.buf+nn, str, n);
```

```
    nn += n;
    kmesg.n = nn;
    iunlock(&kmesg.lk);
}
```

⟨*devcons.c enum Qxxx cases* 508a⟩+≡ (283b) ◁494a 508d▷

```
Qkmesg,
```

⟨consdir *fields* 508b⟩+≡ (283c) ◁494b 508e▷

```
"kmesg",    {Qkmesg},   0,      0440,
```

⟨consread() *cases* 508c⟩+≡ (288d) ◁497 509b▷

```
case Qkmesg:
    /*
     * This is unlocked to avoid tying up a process
     * that's writing to the buffer.  kmesg.n never
     * gets smaller, so worst case the reader will
     * see a slurred buffer.
     */
    if(off >= kmesg.n)
        n = 0;
    else{
        if(off+n > kmesg.n)
            n = kmesg.n - off;
        memmove(buf, kmesg.buf+off, n);
    }
    return n;
```

A.5 /dev/kprint

⟨*devcons.c enum Qxxx cases* 508d⟩+≡ (283b) ◁508a

```
Qkprint,
```

⟨consdir *fields* 508e⟩+≡ (283c) ◁508b

```
"kprint",   {Qkprint, 0, QTEXCL},   0,  DMEXCL|0440,
```

⟨Dirmode *cases* 508f⟩+≡ (242c) ◁321a

```
DMEXCL = 0x20000000,  /* mode bit for exclusive use files */
```

⟨*global* kprintoq 508g⟩≡ (531)

```
Queue*  kprintoq;       /* console output, for /dev/kprint */
```

⟨*global* kprintinuse 508h⟩≡ (531)

```
ulong   kprintinuse;        /* test and set whether /dev/kprint is open */
```

⟨consopen() *cases* 508i⟩+≡ (288b) ◁290f

```
case Qkprint:
    if(arch_tas(&kprintinuse) != 0){
        c->flag &= ~COPEN;
        error(Einuse);
    }
    if(kprintoq == nil){
        kprintoq = qopen(8*1024, Qcoalesce, 0, 0);
        if(kprintoq == nil){
            c->flag &= ~COPEN;
            error(Enomem);
        }
        qnoblock(kprintoq, true);
    }else
        qreopen(kprintoq);
    c->iounit = qiomaxatomic;
    break;
```

⟨consclose() *cases* 509a⟩+≡ (288c) ◁290g

```
/* close of kprint allows other opens */
case Qkprint:
    if(c->flag & COPEN){
        kprintinuse = 0;
        qhangup(kprintoq, nil);
    }
    break;
```

⟨consread() *cases* 509b⟩+≡ (288d) ◁508c

```
case Qkprint:
    return qread(kprintoq, buf, n);
```

⟨putstrn0() *if kprint* 509c⟩≡ (285b)

```
/*
 *  if someone is reading /dev/kprint,
 *  put the message there.
 *  if not and there's an attached bit mapped display,
 *  put the message there.
 *
 *  if there's a serial line being used as a console,
 *  put the message there.
 */
if(kprintoq != nil && !qisclosed(kprintoq)){
    if(usewrite)
        qwrite(kprintoq, str, n);
    else
        qiwrite(kprintoq, str, n);
}
```

⟨panic() *reset kprintoq* 509d⟩≡ (518b)

```
kprintoq = nil; /* don't try to write to /dev/kprint */
```

A.6 Remote debugger

⟨*hook consdebug* 509e⟩≡ (531)

```
void    (*consdebug)(void) = nil; // for rdb
```

⟨panic() *run consdebug hook* 509f⟩≡ (518b)

```
if(consdebug)
    (*consdebug)();
```

⟨echo() *C-t C-t special keys handler other cases* 509g⟩≡ (500)

```
case 'd':
    if(consdebug == nil)
        consdebug = rdb;
    else
        consdebug = nil;
    print("consdebug now %#p\n", consdebug);
    return;
case 'D':
    if(consdebug == nil)
        consdebug = rdb;
    consdebug();
    return;
```

A.7 Serial line

A.7.1 serialoq

⟨*global* serialoq 510a⟩≡ (531)

```
// option<Queue>
Queue*  serialoq;       /* serial console output */
```

⟨*function* consactive 510b⟩≡ (531)

```
int
consactive(void)
{
    if(serialoq)
        return qlen(serialoq) > 0;
    return 0;
}
```

⟨echo() *hooks* 510c⟩+≡ (279c) ◁507d

```
if(serialoq)
    echoserialoq(buf, n);
```

⟨*function* echoserialoq 510d⟩≡ (531)

```
static void
echoserialoq(char *buf, int n)
{
    char *e, *p;
    char ebuf[128];
    int x;

    p = ebuf;
    e = ebuf + sizeof(ebuf) - 4;
    while(n-- > 0){
        if(p >= e){
            qiwrite(serialoq, ebuf, p - ebuf);
            p = ebuf;
        }
        x = *buf++;
        if(x == '\n'){
            *p++ = '\r';
            *p++ = '\n';
        } else if(x == 0x15){
            *p++ = '^';
            *p++ = 'U';
            *p++ = '\n';
        } else
            *p++ = x;
    }
    if(p != ebuf)
        qiwrite(serialoq, ebuf, p - ebuf);
}
```

⟨putstrn0() *serialoq handling* 510e⟩≡ (285b)

```
/*
 *   Convert \n to \r\n for serial
 *   line consoles.  Locking of the queues is left up to the screen
 *   or uart code.  Multi-line messages to serial consoles may get
 *   interspersed with other messages.
 */
```

⟨putstrn0() *if serialoq nil* 511a⟩

```
// else
```

```
while(n > 0) {
    t = memchr(str, '\n', n);
    if(t && !kbd.raw) {
        m = t-str;
        if(usewrite){
            qwrite(serialoq, str, m);
            qwrite(serialoq, "\r\n", 2);
        } else {
            qiwrite(serialoq, str, m);
            qiwrite(serialoq, "\r\n", 2);
        }
        n -= m+1;
        str = t+1;
    } else {
        if(usewrite)
            qwrite(serialoq, str, n);
        else
            qiwrite(serialoq, str, n);
        break;
    }
}
```

A.7.2 uartconsinit()

A.7.3 uartputs()

⟨putstrn0() *if serialoq nil* 511a⟩≡ (510e)

```
if(serialoq == nil){
    uartputs(str, n);
    return;
}
```

A.7.4 consuart

⟨*global* consuart 511b⟩≡ (? 0—1)

```
Uart* consuart;
```

A.7.5 Uart, PhysUart

Appendix B

Profiling

B.1 /sys/sysstat (ARM)

⟨Cpu *stat fields* 512a⟩≡ (28a) 512f▷

```
Perf  perf;     /* performance counters */
```

⟨*struct* Perf 512b⟩≡ (533)

```
/*
 *  performance timers, all units in perfticks
 */
struct Perf
{
    // intr-ts? interrupt time stamp?
    ulong intrts;   /* time of last interrupt */

    ulong inintr;   /* time since last clock tick in interrupt handlers */
    ulong avg_inintr; /* avg time per clock tick in interrupt handlers */

    ulong inidle;   /* time since last clock tick in idle loop */
    ulong avg_inidle; /* avg time per clock tick in idle loop */

    ulong last;   /* value of arch_perfticks() at last clock tick */
    ulong period;   /* arch_perfticks() per clock tick */
};
```

⟨runproc() *other locals* 512c⟩+≡ (150) ◁377d

```
ulong start, now;
```

⟨runproc() *start* 512d⟩≡ (150)

```
start = arch_perfticks();
```

⟨runproc() *after* arch_idlehands, *before restarting loop* 512e⟩≡ (150)

```
/* remember how much time we're here */
now = arch_perfticks();
cpu->perf.inidle += now-start;
start = now;
```

⟨Cpu *stat fields* 512f⟩+≡ (28a) ◁512a 514c▷

```
int load;
```

⟨hzclock() *accounttime* 512g⟩≡ (159e)

```
accounttime();
```

⟨*function* accounttime 513a⟩≡ (599c)

```
/*
 *  time accounting called by clock() splhi'd
 */
void
accounttime(void)
{
    Proc *p;
    ulong n, per;
    static ulong nrun;

    p = cpu->proc; // why not p = up?
    if(p) {
        nrun++;
        ⟨accountime() update time of current process 380e⟩
    }

    /* calculate decaying duty cycles */
    n = arch_perfticks();
    per = n - cpu->perf.last;
    cpu->perf.last = n;
    per = (cpu->perf.period*(Arch_HZ-1) + per)/Arch_HZ;
    if(per != 0)
        cpu->perf.period = per;

    cpu->perf.avg_inidle =
      (cpu->perf.avg_inidle*(Arch_HZ-1)+cpu->perf.inidle)/Arch_HZ;
    cpu->perf.inidle = 0;

    cpu->perf.avg_inintr =
      (cpu->perf.avg_inintr*(Arch_HZ-1)+cpu->perf.inintr)/Arch_HZ;
    cpu->perf.inintr = 0;

    /* only one processor gets to compute system load averages */
    if(cpu->cpuno != 0)
        return;

    /*
     * calculate decaying load average.
     * if we decay by (n-1)/n then it takes
     * n clock ticks to go from load L to .36 L once
     * things quiet down.  it takes about 5 n clock
     * ticks to go to zero.  so using HZ means this is
     * approximately the load over the last second,
     * with a tail lasting about 5 seconds.
     */
    n = nrun;
    nrun = 0;
    n = (nrdy+n)*1000;
    cpu->load = (cpu->load*(Arch_HZ-1)+n)/Arch_HZ;
}
```

⟨*function* intrtime*(arm)* 513b⟩≡ (636c)

```
static void
intrtime(void)
{
    ulong diff;
    ulong x;

    x = arch_perfticks();
```

```
    diff = x - cpu->perf.intrts;
    cpu->perf.intrts = x;

    cpu->perf.inintr += diff;
    if(up == nil && cpu->perf.inidle > diff)
        cpu->perf.inidle -= diff;
}
```

⟨*function* `arch_perfticks` *(arm)* 514a⟩≡ (639b)

```
ulong
arch_perfticks(void)
{
    Armtimer *tm;

    tm = (Armtimer*)ARMTIMER;
    return tm->count;
}
```

⟨*struct* `Armtimer` *(arm)* 514b⟩≡ (639b)

```
struct Armtimer {
    u32int  load;
    u32int  val;
    u32int  ctl;
    u32int  irqack;
    u32int  irq;
    u32int  maskedirq;
    u32int  reload;
    u32int  predivider;
    u32int  count;
};
```

⟨`Cpu` *stat fields* 514c⟩+≡ (28a) ◁512f

```
int cs; // context switch, sched() and sleep() call
int intr;
int syscall;
int pfault;
int tlbfault;
int tlbpurge;
ulong spuriousintr; // not really used
```

⟨*devsys.c enum Qxxx cases* 514d⟩+≡ (484a) ◁487h

```
Qsysstat,
```

⟨`sysdir` *fields* 514e⟩+≡ (484b) ◁487i

```
"sysstat",  {Qsysstat}, 0,      0666,
```

⟨`sysread()` *cases* 514f⟩+≡ (484e) ◁487g

```
case Qsysstat:
    b = smalloc(conf.ncpu*(NUMSIZE*11+1) + 1); /* +1 for NUL */
    bp = b;
    for(id = 0; id < MAXCPUS; id++) {
        if(active.cpus & (1<<id)) {
            mp = CPUS(id);
            readnum(0, bp, NUMSIZE, id, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->cs, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->intr, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->syscall, NUMSIZE);
```

```
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->pfault, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->tlbfault, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->tlbpurge, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE, mp->load, NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE,
                (mp->perf.avg_inidle*100)/mp->perf.period,
                NUMSIZE);
            bp += NUMSIZE;
            readnum(0, bp, NUMSIZE,
                (mp->perf.avg_inintr*100)/mp->perf.period,
                NUMSIZE);
            bp += NUMSIZE;
            *bp++ = '\n';
        }
    }
    if(waserror()){
        free(b);
        nexterror();
    }
    n = readstr((ulong)offset, buf, n, b);
    free(b);
    poperror();
    return n;
```

⟨syswrite() *cases* 515a⟩+≡ (485b) ◁488c

```
  case Qsysstat:
      for(id = 0; id < 32; id++) {
          if(active.cpus & (1<<id)) {
              mp = CPUS(id);
              mp->cs = 0;
              mp->intr = 0;
              mp->syscall = 0;
              mp->pfault = 0;
              mp->tlbfault = 0;
              mp->tlbpurge = 0;
          }
      }
      break;
```

B.2 /dev/kprof

⟨hzclock() *if kproftimer* 515b⟩≡ (159e)

```
  if(kproftimer != nil)
      kproftimer(ur->pc);
```

⟨*hook kproftimer* 515c⟩≡ (595a)

```
  void (*kproftimer)(ulong);
```

B.3 Statistics

B.3.1 Lock.lockcycles

⟨*lock ifdef LOCKCYCLES* 515d⟩≡ (37 35)

```
#ifdef LOCKCYCLES
        l->lockcycles = -arch_lcycles();
#endif
```

⟨Lock *debugging fields* 516a⟩+≡ (35d) ◁45c

```
//#ifdef LOCKCYCLES
long  lockcycles;
//#endif
```

⟨*unlock ifdef LOCKCYCLES* 516b⟩≡ (36f)

```
#ifdef LOCKCYCLES
    l->lockcycles += lcycles();
    cumlockcycles += l->lockcycles;
    if(l->lockcycles > maxlockcycles){
        maxlockcycles = l->lockcycles;
        maxlockpc = l->pc;
    }
#endif
```

⟨*iunlock ifdef LOCKCYCLES* 516c⟩≡ (38b)

```
#ifdef LOCKCYCLES
    l->lockcycles += lcycles();
    cumilockcycles += l->lockcycles;
    if(l->lockcycles > maxilockcycles){
        maxilockcycles = l->lockcycles;
        maxilockpc = l->pc;
    }
    if(l->lockcycles > 2400)
        ilockpcs[n++ & 0xff]  = l->pc;
#endif
```

⟨*globals lockcycles* 516d⟩≡ (529b)

```
#ifdef LOCKCYCLES
long maxlockcycles;
long maxilockcycles;
long cumlockcycles;
long cumilockcycles;
ulong maxlockpc;
ulong maxilockpc;

ulong ilockpcs[0x100] = { [0xff] = 1 };
static int n;
#endif
```

B.3.2 TaslockStats

⟨*struct* TaslockStats 516e⟩≡ (529b)

```
struct TaslockStats
{
    ulong   locks;
    ulong   glare;
    ulong   inglare;
};
```

⟨lock() *start of lock, stat* 516f⟩≡ (35e)

```
lockstats.locks++;
```

⟨lock() *when lock already held, stat* 516g⟩≡ (35e)

```
lockstats.glare++;
```

⟨`lock()` *in loop when lock already held, stat* 517a⟩≡ (35e)
```
lockstats.inglare++;
```

⟨*global* `lockstats` 517b⟩≡ (529b)
```
struct TaslockStats lockstats;
```

B.3.3 QlockStats

⟨*struct* `QlockStats` 517c⟩≡ (530a)
```
struct QlockStats {
    ulong rlock;
    ulong rlockq;
    ulong wlock;
    ulong wlockq;
    ulong qlock;
    ulong qlockq;
};
```

⟨*global* `rwstats` 517d⟩≡ (530a)
```
struct QlockStats rwstats;
```

⟨`qlock()` *start of qlock, stat* 517e⟩≡ (39h)
```
rwstats.qlock++;
```

⟨`qlock()` *when qlock already held, stat* 517f⟩≡ (39h)
```
rwstats.qlockq++;
```

B.3.4 Kimage statistics

⟨*struct* `Irstats` 517g⟩≡ (589)
```
struct Irstats {
    int calls;          /* times imagereclaim was called */
    int loops;          /* times the main loop was run */
    uvlong  ticks;          /* total time in the main loop */
    uvlong  maxt;           /* longest time in main loop */
};
```

⟨*segment.c global irstats* 517h⟩≡ (589)
```
static struct Irstats  irstats;
```

B.3.5 Scheduling

⟨*proc.c statistics* 517i⟩≡ (599c)
```
long preempts;
ulong delayedscheds;    /* statistics */
long skipscheds;
```

B.3.6 Clock

⟨*clock.c statistics* 517j⟩≡ (597a)
```
ulong intrcount[MAXCPUS];
ulong fcallcount[MAXCPUS];
```

⟨`timerintr()` *start, stat* 517k⟩≡ (171d)
```
intrcount[cpu->cpuno]++;
```

⟨`timerintr()` *in timers loop, stat* 517l⟩≡ (171d)
```
fcallcount[cpu->cpuno]++;
```

Appendix C

Error Management

C.1 panic()

⟨*global* panicking 518a⟩≡ (531)

```
bool panicking;
```

⟨*function* panic 518b⟩≡ (531)

```
void
devcons_panic(char *fmt, ...)
{
    int n, s;
    va_list arg;
    char buf[PRINTSIZE];

    ⟨panic() reset kprintoq 509d⟩

    if(panicking)
        for(;;);
    panicking = true;

    s = arch_splhi();
    strcpy(buf, "panic: ");
    va_start(arg, fmt);
    n = vseprint(buf+strlen(buf), buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);
    iprint("%s\n", buf);
    ⟨panic() run consdebug hook 509f⟩
    arch_splx(s);
    prflush();
    buf[n] = '\n';
    putstrn(buf, n+1);
    //TODO: put in comment for now because already got some panic with
    // lapic, but it seems to work still :)
    //arch_dumpstack();
    //exit(1);
}
```

⟨*function* prflush 518c⟩≡ (531)

```
void
prflush(void)
{
    ulong now;

    now = cpu->ticks;
    while(consactive())
```

```
        if(cpu->ticks - now >= Arch_HZ)
            break;
}
```

⟨*function* sysfatal 519a⟩≡ (531)

```
/* libmp at least contains a few calls to sysfatal; simulate with panic */
// note that this is not a system call, even though it's prefixed with sys
//@Scheck: no dead, override also sysfatal from libc/9sys/sysfatal.c
void sysfatal(char *fmt, ...)
{
    char err[256];
    va_list arg;

    va_start(arg, fmt);
    vseprint(err, err + sizeof err, fmt, arg);
    va_end(arg);
    panic("sysfatal: %s", err);
}
```

⟨*function* _assert 519b⟩≡ (531)

```
void
devcons__assert(char *fmt)
{
    panic("assert failed at %#p: %s", getcallerpc(&fmt), fmt);
}
```

C.2 error()

⟨*global Exxx errors* 519c⟩≡ (534a)

```
char Emount[] = "inconsistent mount";
char Eunmount[] = "not mounted";
char Eismtpt[] = "is a mount point";
char Eunion[] = "not in union";
char Emountrpc[] = "mount rpc error";
char Eshutdown[] = "device shut down";
char Enocreate[] = "mounted directory forbids creation";
char Enonexist[] = "file does not exist";
char Eexist[] = "file already exists";
char Ebadsharp[] = "unknown device in # filename";
char Enotdir[] = "not a directory";
char Eisdir[] = "file is a directory";
char Ebadchar[] = "bad character in file name";
char Efilename[] = "file name syntax";
char Eperm[] = "permission denied";
char Ebadusefd[] = "inappropriate use of fd";
char Ebadarg[] = "bad arg in system call";
char Einuse[] = "device or object already in use";
char Eio[] = "i/o error";
char Etoobig[] = "read or write too large";
char Etoosmall[] = "read or write too small";
char Ehungup[] = "i/o on hungup channel";
char Ebadctl[] = "bad process or channel control request";
char Enodev[] = "no free devices";
char Eprocdied[] = "process exited";
char Enochild[] = "no living children";
char Eioload[] = "i/o error in demand load";
char Enovmem[] = "virtual memory allocation failed";
char Ebadfd[] = "fd out of range or not open";
```

```
char Enofd[] = "no free file descriptors";
char Eisstream[] = "seek on a stream";
char Ebadexec[] = "exec header invalid";
char Etimedout[] = "connection timed out";
char Econrefused[] = "connection refused";
char Econinuse[] = "connection in use";
char Eintr[] = "interrupted";
char Enomem[] = "kernel allocate failed";
char Esoverlap[] = "segments overlap";
char Eshort[] = "i/o count too small";
char Egreg[] = "jmk added reentrancy for threads";
char Ebadspec[] = "bad attach specifier";
char Enoreg[] = "process has no saved registers";
char Enoattach[] = "mount/attach disallowed";
char Eshortstat[] = "stat buffer too small";
char Ebadstat[] = "malformed stat buffer";
char Enegoff[] = "negative i/o offset";
char Ecmdargs[] = "wrong #args in control message";
char Ebadip[] = "bad ip address syntax";
char Edirseek[] = "seek in directory";
char Echange[] = "media or partition has changed";
char Edetach[] = "device is detached";
char Enotconf[] = "endpoint not configured";
char Estalled[] = "endpoint stalled";
char Esbadstat[] = "invalid directory entry received from server";
char Enoversion[] = "version not established for mount channel";
```

⟨*global* Edoesnotexist 520a⟩≡ (571)

```
static char Edoesnotexist[] = "does not exist";
```

⟨Proc *error managment fields* 520b⟩≡ (29c) 521d ▷

```
// array<Label>, error labels, poor's man exceptions in C
Label errlab[NERR];
// length(errlab) used.
int nerrlab;

// ref<string> point to errbuf0 or to syserrstr (which points to errbuf1)
char  *errstr;  /* reason we're unwinding the error stack, errbuf1 or 0 */
char  errbuf0[ERRMAX];
char  errbuf1[ERRMAX];
```

⟨newproc() *set fields of* p 520c⟩+≡ (75b) ◁ 479c

```
p->nerrlab = 0;
p->errstr = p->errbuf0;
p->syserrstr = p->errbuf1;
p->errbuf0[0] = '\0';
p->errbuf1[0] = '\0';
```

⟨*constant* NERR 520d⟩≡ (592c)

```
NERR = 64,
```

⟨*function* error 520e⟩≡ (599c)

```
void
proc_error(char *err)
{
    arch_spllo();

    assert(up->nerrlab < NERR);
    kstrcpy(up->errstr, err, ERRMAX);
    arch_setlabel(&up->errlab[NERR-1]);
    nexterror();
}
```

⟨*function* exhausted 521a⟩≡ (599c)

```
void
exhausted(char *resource)
{
    char buf[ERRMAX];

    snprint(buf, sizeof buf, "no free %s", resource);
    iprint("%s\n", buf);
    error(buf);
}
```

C.3 waserror(), nexterror(), poperror()

⟨*macro waserror poperror* 521b⟩≡ (592c)

```
// poor's man exceptions in C
//  - waserror() =~ try
//     * if (!waserror()) { } else { } <=> try { } catch { }
//     * if (waserror()) { }  <=> finally { }
//  - poperror() = nothing
//  - error() =~ raise
//  - nexterror() =~ re raise from exn handler
// note, arch_setlabel() return false, so the branch is never taken first
// but nexterror() is using arch_gotolabel() which returns true, see l_switch.s
#define waserror()  (up->nerrlab++, arch_setlabel(&up->errlab[up->nerrlab-1]))
#define poperror()    up->nerrlab--
```

⟨*function* nexterror 521c⟩≡ (599c)

```
// raise an exception
void
proc_nexterror(void)
{
    arch_gotolabel(&up->errlab[--up->nerrlab]);
}
```

C.4 syserrstr()

⟨Proc *error managment fields* 521d⟩+≡ (29c) ◁520b

```
char  *syserrstr; /* last error from a system call, errbuf0 or 1 */
```

⟨*syscall errstr* 521e⟩≡ (603a)

```
// int errstr(char *err, uint nerr);
long
syserrstr(ulong* arg)
{
    return generrstr((char*)arg[0], arg[1]);
}
```

⟨*function* generrstr 521f⟩≡ (603a)

```
static long
generrstr(char *buf, uint nbuf)
{
    char tmp[ERRMAX];

    if(nbuf == 0)
        error(Ebadarg);
    validaddr((ulong)buf, nbuf, true);
```

```
    if(nbuf > sizeof tmp)
        nbuf = sizeof tmp;
    memmove(tmp, buf, nbuf);

    /* make sure it's NUL-terminated */
    tmp[nbuf-1] = '\0';
    memmove(buf, up->syserrstr, nbuf);
    buf[nbuf-1] = '\0';
    memmove(up->syserrstr, tmp, nbuf);
    return 0;
}
```

⟨*function* werrstr 522⟩≡ (603a)

```
//@Scheck: this is also defined in libc, so it's supposed to override it? TODO
void werrstr(char *fmt, ...)
{
    va_list va;

    if(up == nil)
        return;

    va_start(va, fmt);
    vseprint(up->syserrstr, up->syserrstr+ERRMAX, fmt, va);
    va_end(va);
}
```

Appendix D

Utilities

D.1 Memory areas operations

⟨*lib.h mem functions decl* 523a⟩≡ (580e)

```
extern  void* memccpy(void*, void*, int, ulong);
extern  void* memset(void*, int, ulong);
extern  int memcmp(void*, void*, ulong);
extern  void* memmove(void*, void*, ulong);
extern  void* memchr(void*, int, ulong);
```

⟨*function* memrchr 523b⟩≡ (571)

```
void*
memrchr(void *va, int c, long n)
{
    byte *a, *e;

    a = va;
    for(e=a+n-1; e>a; e--)
        if(*e == c)
            return e;
    return nil;
}
```

D.2 String functions

⟨*lib.h string functions decl* 523c⟩≡ (580e)

```
extern  char* strchr(char*, int);
extern  char* strrchr(char*, int);
extern  int strcmp(char*, char*);
extern  char* strcpy(char*, char*);
extern  char* strecpy(char*, char*, char*);
extern  char* strncpy(char*, char*, long);
extern  int strncmp(char*, char*, long);
extern  long  strlen(char*);
extern  char* strstr(char*, char*);
extern  int atoi(char*);
extern  int fullrune(char*, int);
```

⟨*function* kstrcpy 523d⟩≡ (585c)

```
/*
 * Rather than strncpy, which zeros the rest of the buffer, kstrcpy
 * truncates if necessary, always zero terminates, does not zero fill,
 * and puts ... at the end of the string if it's too long.  Usually used to
```

```
 * save a string in up->genbuf;
 */
void
kstrcpy(char *s, char *t, int ns)
{
    int nt;

    nt = strlen(t);
    if(nt+1 <= ns){
        memmove(s, t, nt+1);
        return;
    }
    /* too long */
    if(ns < 4){
        /* but very short! */
        strncpy(s, t, ns);
        return;
    }
    /* truncate with ... at character boundary (very rare case) */
    memmove(s, t, ns-4);
    ns -= 4;
    s[ns] = '\0';
    /* look for first byte of UTF-8 sequence by skipping continuation bytes */
    while(ns>0 && (s[--ns]&0xC0)==0x80)
        ;
    strcpy(s+ns, "...");
}
```

⟨*function* kstrdup 524a⟩≡ (585c)

```
/*
 * Atomically replace *p with copy of s
 */
void
kstrdup(char **p, char *s)
{
    int n;
    char *t, *prev;

    n = strlen(s)+1;
    /* if it's a user, we can wait for memory; if not, something's very wrong */
    if(up){
        t = smalloc(n);
        setmalloctag(t, getcallerpc(&p));
    }else{
        t = malloc(n);
        if(t == nil)
            panic("kstrdup: no memory");
    }
    memmove(t, s, n);
    prev = *p;
    *p = t;
    free(prev);
}
```

⟨*function* emptystr 524b⟩≡ (571)

```
bool
emptystr(char *s)
{
    if(s == nil)
        return true;
```

```
    if(s[0] == '\0')
        return true;
    return false;
}
```

D.3 Conversion functions

⟨*lib.h strto functions decl* 525a⟩≡ (580e)

```
extern  long  strtol(char*, char**, int);
extern  ulong strtoul(char*, char**, int);
extern  vlong strtoll(char*, char**, int);
extern  uvlong  strtoull(char*, char**, int);
```

⟨*function* `readnum` 525b⟩≡ (535a)

```
int
readnum(ulong off, char *buf, ulong n, ulong val, int size)
{
    char tmp[64];

    snprint(tmp, sizeof(tmp), "%*lud", size-1, val);
    tmp[size-1] = ' ';
    if(off >= size)
        return 0;
    if(off+n > size)
        n = size-off;
    memmove(buf, tmp+off, n);
    return n;
}
```

⟨*function* `readstr` 525c⟩≡ (535a)

```
int
readstr(ulong off, char *buf, ulong n, char *str)
{
    int size;

    size = strlen(str);
    if(off >= size)
        return 0;
    if(off+n > size)
        n = size-off;
    memmove(buf, str+off, n);
    return n;
}
```

D.4 Pool allocation

D.5 Printf and `Fmt`

⟨*lib.h printf functions decl* 525d⟩≡ (580e)

```
extern  char* seprint(char*, char*, char*, ...);
extern  char* vseprint(char*, char*, char*, va_list);
extern  int snprint(char*, int, char*, ...);
extern  int sprint(char*, char*, ...);
```

⟨*struct* Fmt 526a⟩≡ (580e)

```
struct Fmt{
  byte runes;      /* output buffer is runes or chars? */
  void  *start;     /* of buffer */
  void  *to;      /* current place in the buffer */
  void  *stop;      /* end of the buffer; overwritten if flush fails */
  int (*flush)(Fmt *);  /* called when to == stop */
  void  *farg;      /* to make flush a closure */
  int nfmt;     /* num chars formatted so far */
  va_list args;     /* args passed to dofmt */
  int r;      /* % format Rune */
  int width;
  int prec;
  ulong flags;
};
```

⟨*lib.h fmt functions decl* 526b⟩≡ (580e)

```
extern  int fmtstrinit(Fmt*);
extern  int fmtinstall(int, int (*)(Fmt*));
extern  void  quotefmtinstall(void);
extern  int fmtprint(Fmt*, char*, ...);
extern  int fmtstrcpy(Fmt*, char*);
extern  char* fmtstrflush(Fmt*);
```

⟨*lib.h pragmas* 526c⟩≡ (580e)

```
#pragma varargck  argpos  fmtprint  2
#pragma varargck  argpos  print   1
#pragma varargck  argpos  seprint   3
#pragma varargck  argpos  snprint   3
#pragma varargck  argpos  sprint    2

#pragma varargck  type  "lld" vlong
#pragma varargck  type  "llx" vlong
#pragma varargck  type  "lld" uvlong
#pragma varargck  type  "llx" uvlong
#pragma varargck  type  "ld"  long
#pragma varargck  type  "lx"  long
#pragma varargck  type  "ld"  ulong
#pragma varargck  type  "lx"  ulong
#pragma varargck  type  "d" int
#pragma varargck  type  "x" int
#pragma varargck  type  "c" int
#pragma varargck  type  "C" int
#pragma varargck  type  "d" uint
#pragma varargck  type  "x" uint
#pragma varargck  type  "c" uint
#pragma varargck  type  "C" uint
#pragma varargck  type  "s" char*
#pragma varargck  type  "q" char*
#pragma varargck  type  "S" Rune*
#pragma varargck  type  "%" void
#pragma varargck  type  "p" uintptr
#pragma varargck  type  "p" void*
#pragma varargck  flag  ','
```

D.6 Runes, a.k.a unicode

⟨*enum* utf 526d⟩≡ (580e)

```
enum
{
  UTFmax    = 4,    /* maximum bytes per rune */
  Runeself  = 0x80,   /* rune and UTF sequences are the same (<) */
};
```

⟨*lib.h rune functions decl* 527a⟩≡ (580e)

```
extern  int runetochar(char*, Rune*);
extern  int chartorune(Rune*, char*);
extern  char* utfrune(char*, long);
extern  int utfnlen(char*, long);
```

⟨Kbscan *other fields* 527b⟩+≡ (275a) ◁291b

```
bool collecting;
int nk;
Rune    kc[5];
```

⟨kbdputsc() *if collecting* 527c⟩≡ (275d)

```
if(kbscan->collecting){
    int i;
    // pad's additional overflow checking, just to make sure
    if(kbscan->nk >= nelem(kbscan->kc)) {
      kbscan->nk = 0;
      kbscan->collecting = false;
      print("collecting overflow, possible bug in latin1()\n");
    }
    kbscan->kc[kbscan->nk++] = c;
    c = latin1(kbscan->kc, kbscan->nk);
    if(c < -1)  /* need more keystrokes */
        return;
    if(c != -1) /* valid sequence */
        kbdputc(c);
    else    /* dump characters */
        for(i=0; i<kbscan->nk; i++)
            kbdputc(kbscan->kc[i]);
    kbscan->nk = 0;
    kbscan->collecting = false;
}
```

⟨kbdputsc() *start collecting* 527d⟩≡ (275d)

```
/*
 * VMware and Qemu use Ctl-Alt as the key combination
 * to make the VM give up keyboard and mouse focus.
 * This has the unfortunate side effect that when you
 * come back into focus, Plan 9 thinks you want to type
 * a compose sequence (you just typed alt).
 *
 * As a clumsy hack around this, we look for ctl-alt
 * and don't treat it as the start of a compose sequence.
 */
if(!kbscan->ctl){
    kbscan->collecting = true;
    kbscan->nk = 0;
}
```

Appendix E

Extra Code

E.1 arch/

E.2 buses/

E.3 concurrency/

E.3.1 concurrency/portdat_concurrency.h

⟨portdat_concurrency.h 528⟩≡

```
//*****************************************************************************
// Mutual exclusion
//*****************************************************************************

// arch_tas() < Lock|ILock < QLock|RWLock

// This used to be in <arch>/, but its fields are used from port/, so the
// code must be portable!
⟨struct Lock 35d⟩

⟨struct ILock 37d⟩

⟨struct QLock 39c⟩

⟨struct RWlock 41b⟩

//*****************************************************************************
// Atomicity
//*****************************************************************************

⟨struct Ref 44b⟩

⟨struct Counter 45b⟩

//*****************************************************************************
// Synchronization
//*****************************************************************************

// defined in this directory but no functions are operating on it in this dir
⟨struct Rendez 156e⟩

⟨struct Sema 355a⟩
```

```
// see also Waitq in portdat_processes.h?
```

E.3.2 concurrency/portfns_concurrency.h

⟨portfns_concurrency.h 529a⟩≡

```
// taslock.c
int  lock(Lock*);
void unlock(Lock*);
void ilock(Lock*);
void iunlock(Lock*);
int  canlock(Lock*);

// ref.c
long incref(Ref*);
long decref(Ref*);

// qlock.c
void qlock(QLock*);
void qunlock(QLock*);
int  canqlock(QLock*);

void rlock(RWlock*);
void runlock(RWlock*);
int  canrlock(RWlock*);
void wlock(RWlock*);
void wunlock(RWlock*);

// in <arch>/concurrency.s (but used in port)
//@Scheck: Assembly
int  arch_splhi(void);
//@Scheck: Assembly
int  arch_spllo(void);
//@Scheck: Assembly
void arch_splx(int);
//@Scheck: Assembly
bool arch_islo(void);

//test-and-set
//@Scheck: Assembly
int  arch_tas(void*);
//@Scheck: Assembly
void arch_xinc(long*);
//@Scheck: Assembly
long arch_xdec(long*);
// <arch>/??? (called from port but signature not portable across <arch>)
//int   arch_cmpswap(long*, long, long);

//void  arch_coherence(void); // now in core/ for backward deps
```

E.3.3 concurrency/taslock.c

⟨taslock.c 529b⟩≡

```
// TAS: Test And Set
⟨kernel basic includes 23m⟩

⟨struct TaslockStats 516e⟩
```

⟨*global* `lockstats` 517b⟩
⟨*globals lockcycles* 516d⟩

⟨*function* `inccnt` 46a⟩

⟨*function* `deccnt` 46b⟩

⟨*function* `lockloop` 503c⟩

⟨*function* `lock` 35e⟩

⟨*function* `ilock` 37g⟩

⟨*function* `canlock` 37b⟩

⟨*function* `unlock` 36f⟩

⟨*function* `iunlock` 38b⟩

E.3.4 concurrency/qlock.c

⟨`qlock.c` 530a⟩≡
⟨*kernel basic includes* 23m⟩

⟨*struct* `QlockStats` 517c⟩
⟨*global* `rwstats` 517d⟩

⟨*function* `qlock` 39h⟩

⟨*function* `canqlock` 41a⟩

⟨*function* `qunlock` 40c⟩

⟨*function* `rlock` 41e⟩

⟨*function* `runlock` 42b⟩

⟨*function* `wlock` 42d⟩

⟨*function* `wunlock` 43b⟩

⟨*function* `canrlock` 44a⟩

E.3.5 concurrency/ref.c

⟨`ref.c` 530b⟩≡
⟨*kernel basic includes* 23m⟩

⟨*function* `incref` 44c⟩

⟨*function* `decref` 45a⟩

E.4 console/

E.4.1 console/portdat_console.h

E.4.2 console/portfns_console.h

E.4.3 console/devcons.c

```
⟨devcons.c 531⟩≡
  ⟨kernel basic includes 23m⟩

  #include    <pool.h>

  // used also by edf.c
  ⟨global panicking 518a⟩

  ⟨hook screenputs 280b⟩
  ⟨hook consdebug 509e⟩

  ⟨global kbdq 283d⟩
  ⟨global lineq 284b⟩
  ⟨global serialoq 510a⟩
  ⟨global kprintoq 508g⟩

  ⟨global kprintinuse 508h⟩
  ⟨global iprintscreenputs 286d⟩

  ⟨struct ConsKbd 278b⟩

  ⟨global kbd 278c⟩

  ⟨global fasthz 182c⟩

  ⟨devcons.c forward decl ??⟩

  ⟨function kbdqinit 284a⟩

  ⟨function lineqinit 284c⟩

  ⟨function consactive 510b⟩

  ⟨function prflush 518c⟩

  ⟨struct KMesg 507a⟩

  ⟨global kmesg 507b⟩

  ⟨function kmesgputs 507f⟩

  ⟨function putstrn0 285b⟩

  ⟨function putstrn 286c⟩

  ⟨function print 286b⟩

  ⟨global iprintlock 287a⟩

  ⟨function iprintcanlock 287b⟩
```

⟨*function* iprint 286e⟩

⟨*function* panic 518b⟩

⟨*function* sysfatal 519a⟩

⟨*function* _assert 519b⟩

⟨*function* pprint 287c⟩

⟨*function* echoscreen 280c⟩

⟨*function* echoserialoq 510d⟩

⟨*function* echo 279c⟩

⟨*function* kbdcr2nl ??⟩

⟨*function* kbdputc 278d⟩

⟨*clock callback kbdputcclock* 279a⟩

⟨*devcons.c enum Qxxx* 283b⟩

```
enum
{
    ⟨constant VLNUMSIZE 178d⟩
};
```

⟨*global* consdir 283c⟩

⟨*method consinit* 288a⟩

```
static Chan*
consattach(char *spec)
{
    return devattach('c', spec);
}

static Walkqid*
conswalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name,nname, consdir, nelem(consdir), devgen);
}

static int
consstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, consdir, nelem(consdir), devgen);
}
```

⟨*method consopen* 288b⟩

⟨*method consclose* 288c⟩

⟨*method consread* 288d⟩

⟨*method conswrite* 289⟩

⟨*global* consdevtab 290b⟩

⟨*global* randn 494d⟩

⟨*function* seedrand 494e⟩

⟨*function* nrand 495a⟩

⟨*global* uvorder 184b⟩

⟨*function* le2vlong 184c⟩

⟨*function* vlong2le 185a⟩

⟨*global* order 185b⟩

⟨*function* le2long 185c⟩

⟨*devcons.c Exxx errors* 178c⟩

⟨*function* readtime 182e⟩

⟨*function* writetime 183a⟩

⟨*function* readbintime 183d⟩

⟨*function* writebintime 184a⟩

E.5 core/

E.5.1 core/portdat_forward.h

E.5.2 core/portdat_core.h

⟨portdat_core.h 533⟩≡

```
// All those structs used to be in <arch>/dat.h, but many of their fields
// were used from port/ so I've moved them here (and put the arch-specific
// fields in dat_core.h)

//*****************************************************************************
// Conf
//*****************************************************************************

⟨struct Confmem 27c⟩

⟨struct Conf 27a⟩

extern Conf conf;

⟨constant MAXCONF 27e⟩
// hash<string, string>
extern char *confname[];
extern char *confval[];
// Hashtbl.length(confname)
extern int nconf;

extern bool cpuserver; // defined in $CONF.c
```

```
char* getconf(char *name);

//*****************************************************************************
// Cpu
//*****************************************************************************
```

⟨*struct* Label 151d⟩

⟨*struct* Perf 512b⟩

⟨*struct* Cpu 28a⟩

```
// array<ref<Cpu>>, MAXCPUS is defined in <arch>/mem.h
extern Cpu* cpus[MAXCPUS];
```
⟨*macro* CPUS 28i⟩

```
// 'up' and 'cpu' can have different implement in different arch
// so they are not defined here.

//*****************************************************************************
// Other
//*****************************************************************************

extern char* eve;
int iseve(void);
// accessed by /dev/hostdomain, defined in auth.c
extern  char  hostdomain[];

// defined in ???
extern ulong    kerndate;
```

E.5.3 core/portfns_core.h

E.5.4 core/portdat.h

E.5.5 core/error.h

E.5.6 core/error.c

⟨error.c 534a⟩≡
⟨*global Exxx errors* 519c⟩

E.5.7 core/portdat.c

⟨portdat.c 534b⟩≡
⟨*kernel basic includes* 23m⟩

```
// was in init/main.c
```
⟨*global* conf 27b⟩
```
// bool cpuserver; // defined in $CONF.c
```

⟨*globals confname and confvar* 27d⟩
⟨*global* nconf 27f⟩

```
//TODO: to factorize code between pc and pi
```

```
//< <function [[getconf]]> >

// was in dat.h
⟨global cpus 28g⟩
// cpu is in init/main.c because it can differ in different archi

// was in security/auth.c
⟨global eve 456a⟩
// should be in portfns.c, but then backward deps
/*
 *  return true if current user is eve
 */
⟨function iseve 456b⟩
```

E.5.8 core/portfns.c

```
⟨portfns.c 535a⟩≡
  ⟨kernel basic includes 23m⟩

  ⟨portfns.c backward deps breaker ??⟩

  ⟨function returnfalse 55b⟩

  // was in devcons.c, could be in lib/misc.c
  ⟨function readnum 525b⟩

  ⟨function readstr 525c⟩
```

E.6 devices/

E.6.1 devices/portdat_devices.h

```
⟨struct Devport 535b⟩≡                                          (535d)
  struct Devport {
      ulong port;
      int size;
  };

⟨struct DevConf 535c⟩≡                                          (535d)
  struct DevConf
  {
      ulong intnum;     /* interrupt number */
      char  *type;      /* card type, malloced */
      int nports;     /* Number of ports */
      Devport *ports;     /* The ports themselves */
  };

⟨portdat_devices.h 535d⟩≡
  /*
   *  hardware info about a device
   */
  ⟨struct Devport 535b⟩

  ⟨struct DevConf 535c⟩

  // keyboard
```

⟨*enum* kbscan 275b⟩

⟨*struct* Kbscan 275a⟩

```
// keyboard/portkbd.c
extern Kbscan kbscans[KbNscans];

// mouse

/*
 *  mouse types
 */
enum MouseType
{
    Mouseother= 0,
    Mouseserial=    1,
    MousePS2=   2,
};

// mouse/devmouse.c (used in portmouse.c, portkbd.c, ...)
extern int mouseshifted;

// mouse/portmouse.c (used in <arch>/mouse.c)
extern int mousetype;
extern int packetsize;
extern bool mousehwaccel;
extern bool intellimouse;

/* devdraw.c */ // used in some <arch>/screen.c
extern QLock drawlock;
// for gscreen, gscreendata, see portscreen.h
```

E.6.2 devices/portfns_devices.h

E.6.3 devices/keyboard/portkbd.c

⟨portkbd.c 536⟩≡

```
/*
 * keyboard input // the portable part
 */
```

⟨*kernel basic includes* 23m⟩

```
// The main entry point in this file is kbdputsc() called
// from devkbin.c when a process (e.g., usbd) writes into /dev/kbin.

// The functions kbdputmap/kbdgetmap are also portable functions called
// from devkbmap.c
```

⟨*global* kdebug 499a⟩

⟨*enum* specialkey 273⟩

⟨*global* kbtab 274a⟩

⟨*global* kbtabshift 274b⟩

⟨*global* kbtabesc1 277b⟩

⟨*global* kbtabaltgr 274d⟩

```
// see kbtab.c for the ctrl one, put in another file because
// of issues with TeX with the special characters it contain
// can't LPize it.
```
⟨*global kbtabctrl decl* 274c⟩

⟨*global* kbscans 275c⟩

```
// called when write in /dev/kbin (see devkbin.c)
```
⟨*function* kbdputsc 275d⟩

```
// called when interact with /dev/kbmap (see devkbmap.c)
```
⟨*function* kbdputmap 281a⟩

⟨*function* kbdgetmap 280d⟩

E.6.4 devices/keyboard/latin1.c

⟨latin1.c 537⟩≡

⟨*kernel basic includes* 23m⟩

```
/*
 * The code makes two assumptions: strlen(ld) is 1 or 2; latintab[i].ld can be a
 * prefix of latintab[j].ld only when j<i.
 */
struct cvlist
{
    char    *ld;        /* must be seen before using this conversion */
    char    *si;        /* options for last input characters */

    Rune    *so;        /* the corresponding Rune for each si entry */
};

struct cvlist latintab[] = {
#include "../port/latin1.h"
    0,  0,      0
};

/*
 * Given n characters k[0]..k[n-1], find the rune or return -1 for failure.
 */
long
unicode(Rune *k, int n)
{
    long c;
    Rune *r;

    c = 0;
    for(r = &k[1]; r<&k[n]; r++){       /* +1 to skip [Xx] */
        c <<= 4;
        if('0'<=*r && *r<='9')
            c += *r-'0';
        else if('a'<=*r && *r<='f')
            c += 10 + *r-'a';
        else if('A'<=*r && *r<='F')
            c += 10 + *r-'A';
```

```
        else
            return -1;
    }
    return c;
}

/*
 * Given n characters k[0]..k[n-1], find the corresponding rune or return -1 for
 * failure, or something < -1 if n is too small.  In the latter case, the result
 * is minus the required n.
 */
long
latin1(Rune* k, int n)
{
    struct cvlist *l;
    char c;
    char* p;

//    print("latin1: %d, %d\n", n, k[n-1]);

    if(k[0] == 'X')
        if(n>=5)
            return unicode(k, 5);
        else
            return -5;
    if(k[0] == 'x')
        if(n>=UTFmax*2+1)
            return unicode(k, UTFmax*2+1);
        else
            return -(UTFmax+1);

    for(l=latintab; l->ld!=0; l++)
        if(k[0] == l->ld[0]){
//          print("found: %s, %s\n", l->ld, l->si);
            if(n == 1)
                return -2;
            if(l->ld[1] == 0)
                c = k[1];
            else if(l->ld[1] != k[1])
                continue;
            else if(n == 2)
                return -3;
            else
                c = k[2];
            for(p=l->si; *p!=0; p++)
                if(*p == c) {
                    print("found: 0x%x\n", l->so[p - l->si]);
                    return l->so[p - l->si];
                }
            return -1;
        }
    return -1;
}
```

E.6.5 devices/keyboard/latin1.h

E.6.6 TO LP SPLIT devices/keyboard/devkbin.c

⟨devkbin.c 538⟩≡

```
/*
 *  keyboard scan code input from outside the kernel.
 *  to avoid duplication of keyboard map processing for usb.
 */
```
⟨*kernel basic includes* 23m⟩

```
extern  void kbdputsc(byte, int);

enum {
    Qdir,
    Qkbd,
};

Dirtab kbintab[] = {
    ".",    {Qdir, 0, QTDIR},   0,  0555,
    "kbin", {Qkbd, 0},      0,  0200,
};

Lock    kbinlck;
int kbinbusy;

static Chan *
kbinattach(char *spec)
{
    return devattach(L'I', spec);
}

static Walkqid*
kbinwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, kbintab, nelem(kbintab), devgen);
}

static int
kbinstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, kbintab, nelem(kbintab), devgen);
}

static Chan*
kbinopen(Chan *c, int omode)
{
    if(!iseve())
        error(Eperm);
    if(c->qid.path == Qkbd){
        lock(&kbinlck);
        if(kbinbusy){
            unlock(&kbinlck);
            error(Einuse);
        }
        kbinbusy++;
        unlock(&kbinlck);
    }
    return devopen(c, omode, kbintab, nelem(kbintab), devgen);
}

static void
kbinclose(Chan *c)
{
    if(c->aux){
```

```
        free(c->aux);
        c->aux = nil;
    }
    if(c->qid.path == Qkbd)
        kbinbusy = 0;
}

static long
kbinread(Chan *c, void *a, long n, vlong )
{
    if(c->qid.type == QTDIR)
        return devdirread(c, a, n, kbintab, nelem(kbintab), devgen);
    return 0;
}

static long
kbinwrite(Chan *c, void *a, long n, vlong)
{
    int i;
    uchar *p = a;

    if(c->qid.type == QTDIR)
        error(Eisdir);
    switch((int)c->qid.path){
    case Qkbd:
        for(i = 0; i < n; i++)
            kbdputsc(*p++, 1);  /* external source */
        break;
    default:
        error(Egreg);
    }
    return n;
}

Dev kbindevtab = {
    .dc = L'I',
    .name = "kbin",

    .reset = devreset,
    .init = devinit,
    .shutdown = devshutdown,
    .attach = kbinattach,
    .walk = kbinwalk,
    .stat = kbinstat,
    .open = kbinopen,
    .create = devcreate,
    .close = kbinclose,
    .read = kbinread,
    .bread = devbread,
    .write = kbinwrite,
    .bwrite = devbwrite,
    .remove = devremove,
    .wstat = devwstat,
};
```

E.6.7 TO LPSPLIT devices/keyboard/devkbmap.c

⟨devkbmap.c 540⟩≡

```
/*
```

```
 *  keyboard map
 */
```
⟨*kernel basic includes* 23m⟩

```
enum{
    Qdir,
    Qdata,
};
Dirtab kbmaptab[]={
    ".",        {Qdir, 0, QTDIR},   0,  0555,
    "kbmap",    {Qdata, 0},     0,  0600,
};
#define NKBFILE sizeof(kbmaptab)/sizeof(kbmaptab[0])

#define KBLINELEN   (3*NUMSIZE+1)   /* t code val\n */

static Chan *
kbmapattach(char *spec)
{
    return devattach(L'K', spec);
}

static Walkqid*
kbmapwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, kbmaptab, NKBFILE, devgen);
}

static int
kbmapstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, kbmaptab, NKBFILE, devgen);
}

static Chan*
kbmapopen(Chan *c, int omode)
{
    if(!iseve())
        error(Eperm);
    return devopen(c, omode, kbmaptab, NKBFILE, devgen);
}

static void
kbmapclose(Chan *c)
{
    if(c->aux){
        free(c->aux);
        c->aux = nil;
    }
}

static long
kbmapread(Chan *c, void *a, long n, vlong offset)
{
    char *bp;
    char tmp[KBLINELEN+1];
    int t, sc;
    Rune r;

    if(c->qid.type == QTDIR)
```

```
        return devdirread(c, a, n, kbmaptab, NKBFILE, devgen);

    switch((int)(c->qid.path)){
    case Qdata:
        if(kbdgetmap(offset/KBLINELEN, &t, &sc, &r)) {
            bp = tmp;
            bp += readnum(0, bp, NUMSIZE, t, NUMSIZE);
            bp += readnum(0, bp, NUMSIZE, sc, NUMSIZE);
            bp += readnum(0, bp, NUMSIZE, r, NUMSIZE);
            *bp++ = '\n';
            *bp = 0;
            n = readstr(offset%KBLINELEN, a, n, tmp);
        } else
            n = 0;
        break;
    default:
        n=0;
        break;
    }
    return n;
}

static long
kbmapwrite(Chan *c, void *a, long n, vlong)
{
    char line[100], *lp, *b;
    int key, m, l;
    Rune r;

    if(c->qid.type == QTDIR)
        error(Eperm);

    switch((int)(c->qid.path)){
    case Qdata:
        b = a;
        l = n;
        lp = line;
        if(c->aux){
            strcpy(line, c->aux);
            lp = line+strlen(line);
            free(c->aux);
            c->aux = nil;
        }
        while(--l >= 0) {
            *lp++  = *b++;
            if(lp[-1] == '\n' || lp == &line[sizeof(line)-1]) {
                *lp = 0;
                if(*line == 0)
                    error(Ebadarg);
                if(*line == '\n' || *line == '#'){
                    lp = line;
                    continue;
                }
                lp = line;
                while(*lp == ' ' || *lp == '\t')
                    lp++;
                m = strtoul(line, &lp, 0);
                key = strtoul(lp, &lp, 0);
                while(*lp == ' ' || *lp == '\t')
                    lp++;
```

```
                r = 0;
                if(*lp == '\'' && lp[1])
                    chartorune(&r, lp+1);
                else if(*lp == '^' && lp[1]){
                    chartorune(&r, lp+1);
                    if(0x40 <= r && r < 0x60)
                        r -= 0x40;
                    else
                        error(Ebadarg);
                }else if(*lp == 'M' && ('1' <= lp[1] && lp[1] <= '5'))
                    r = 0xF900+lp[1]-'0';
                else if(*lp>='0' && *lp<='9') /* includes 0x... */
                    r = strtoul(lp, &lp, 0);
                else
                    error(Ebadarg);
                kbdputmap(m, key, r);
                lp = line;
            }
        }
        if(lp != line){
            l = lp-line;
            c->aux = lp = smalloc(l+1);
            memmove(lp, line, l);
            lp[l] = 0;
        }
        break;
    default:
        error(Ebadusefd);
    }
    return n;
}


Dev kbmapdevtab = {
    .dc = L'k',
    .name = "kbmap",

    .reset = devreset,
    .init = devinit,
    .shutdown = devshutdown,
    .attach = kbmapattach,
    .walk = kbmapwalk,
    .stat = kbmapstat,
    .open = kbmapopen,
    .create = devcreate,
    .close = kbmapclose,
    .read = kbmapread,
    .bread = devbread,
    .write = kbmapwrite,
    .bwrite = devbwrite,
    .remove = devremove,
    .wstat = devwstat,
};
```

E.6.8 TO LPSPLIT devices/mouse/portmouse.c

⟨portmouse.c 543⟩≡
  ⟨*kernel basic includes* 23m⟩

```
// enum<MouseType>
int mousetype;

int packetsize;
static int resolution;
static int accelerated;

bool mousehwaccel;
bool intellimouse;

static QLock mousectlqlock;

// for serial mouse
//static char mouseport[5];

enum
{
    CMaccelerated,
    CMhwaccel,
    CMintellimouse,
    CMlinear,
    CMps2,
    CMps2intellimouse,
    CMres,
    CMreset,
    CMserial,
};

static Cmdtab mousectlmsg[] =
{
    CMaccelerated     , "accelerated"     , 0 ,
    CMhwaccel         , "hwaccel"         , 2 ,
    CMintellimouse    , "intellimouse"    , 1 ,
    CMlinear          , "linear"          , 1 ,
    CMps2             , "ps2"             , 1 ,
    CMps2intellimouse , "ps2intellimouse" , 1 ,
    CMres             , "res"             , 0 ,
    CMreset           , "reset"           , 1 ,
    CMserial          , "serial"          , 0 ,
};


/*
 *  set up a ps2 mouse
 */
static void
ps2mouse(void)
{
    if(mousetype == MousePS2)
        return;
    arch_ps2mouse();

    mousetype = MousePS2;
    packetsize = 3;
    mousehwaccel = 1;
}

/*
 * The PS/2 Trackpoint multiplexor on the IBM Thinkpad T23 ignores
 * acceleration commands.  It is supposed to pass them on
```

```
 * to the attached device, but my Logitech mouse is simply
 * not behaving any differently.  For such devices, we allow
 * the user to use "hwaccel off" to tell us to back off to
 * software acceleration even if we're using the PS/2 port.
 * (Serial mice are always software accelerated.)
 * For more information on the Thinkpad multiplexor, see
 * http://wwwcssrv.almaden.ibm.com/trackpoint/
 */
static void
setaccelerated(int x)
{
    accelerated = x;
    arch_setaccelerated();
    // call in devmouse.c
    mouseaccelerate(x);
}

static void
setlinear(void)
{
    accelerated = 0;
    arch_setlinear();
    mouseaccelerate(0);
}

static void
setres(int n)
{
    resolution = n;
    arch_setres(n);
}

static void
setintellimouse(void)
{
    intellimouse = 1;
    packetsize = 4;
    arch_setintellimouse();
}

static void
resetmouse(void)
{
    packetsize = 3;
    arch_resetmouse();
}

void
kmousectl(Cmdbuf *cb)
{
    Cmdtab *ct;

    qlock(&mousectlqlock);
    if(waserror()){
        qunlock(&mousectlqlock);
        nexterror();
    }

    ct = lookupcmd(cb, mousectlmsg, nelem(mousectlmsg));
    switch(ct->index){
```

```
case CMaccelerated:
    setaccelerated(cb->nf == 1 ? 1: atoi(cb->f[1]));
    break;
case CMlinear:
    setlinear();
    break;
case CMps2:
    intellimouse = 0;
    ps2mouse();
    break;
case CMres:
    if(cb->nf >= 2)
        setres(atoi(cb->f[1]));
    else
        setres(1);
    break;
case CMreset:
    resetmouse();
    if(accelerated)
        setaccelerated(accelerated);
    if(resolution)
        setres(resolution);
    if(intellimouse)
        setintellimouse();
    break;
case CMhwaccel:
    if(strcmp(cb->f[1], "on")==0)
        mousehwaccel = 1;
    else if(strcmp(cb->f[1], "off")==0)
        mousehwaccel = 0;
    else
        cmderror(cb, "bad mouse control message");
    break;
case CMintellimouse:
    setintellimouse();
    break;
case CMps2intellimouse:
    ps2mouse();
    setintellimouse();
    break;
case CMserial:
    //if(mousetype == Mouseserial)
    //  error(Emouseset);
    //
    //if(cb->nf > 2){
    //  if(strcmp(cb->f[2], "M") == 0)
    //      i8250mouse(cb->f[1], m3mouseputc, 0);
    //  else if(strcmp(cb->f[2], "MI") == 0)
    //      i8250mouse(cb->f[1], m5mouseputc, 0);
    //  else
    //      i8250mouse(cb->f[1], mouseputc, cb->nf == 1);
    //} else
    //  i8250mouse(cb->f[1], mouseputc, cb->nf == 1);
    //
    //mousetype = Mouseserial;
    //strncpy(mouseport, cb->f[1], sizeof(mouseport)-1);
    //packetsize = 3;
    error("serial mice not supported anymore");
    break;
}
```

```
    qunlock(&mousectlqlock);
    poperror();
}
```

E.6.9 TO LPSPLIT devices/mouse/devmouse.c

⟨devmouse.c 547⟩≡
⟨*kernel basic includes* 23m⟩

```
#include    <draw.h>
#include    <draw_private.h> // for BPLONG
#include    <memdraw.h>
#include    <cursor.h>

#include    "../port/portscreen.h"

enum {
    ScrollUp = 0x08,
    ScrollDown = 0x10,
    ScrollLeft = 0x20,
    ScrollRight = 0x40,
};

typedef struct Mouseinfo    Mouseinfo;
typedef struct Mousestate   Mousestate;

struct Mousestate
{
    Point   xy;     /* mouse.xy */
    int buttons;    /* mouse.buttons */
    ulong   counter;    /* increments every update */
    ulong   msec;       /* time of last event */
};

struct Mouseinfo
{
    Lock;
    Mousestate;
    int dx;
    int dy;
    bool track;      /* dx & dy updated */
    bool redraw;     /* update cursor on screen */
    ulong   lastcounter;    /* value when /dev/mouse read */
    ulong   lastresize;
    ulong   resize;
    Rendez  r;
    Ref;
    QLock;
    bool open;
    int acceleration;
    int maxacc;
    Mousestate  queue[16];  /* circular buffer of click events */
    int ri;     /* read index into queue */
    int wi;     /* write index into queue */
    uchar   qfull;      /* queue is full */
};

Mouseinfo   mouse;
```

```
Cursorinfo  cursor;
Cursor      curs;


// set to mousefromkbd here in mouseinit(), but used outside devmouse.c
// to let know devmouse.c about some keyboard events related to the mouse
void    (*kbdmouse)(int);
int     kbdbuttons;

// not used in this file
bool     mouseshifted;


enum{
    Qdir,
    Qcursor,
    Qmouse,
    Qmousein,
    Qmousectl,
};

static Dirtab mousedir[]={
    ".",    {Qdir, 0, QTDIR},   0,          DMDIR|0555,
    "cursor",   {Qcursor},  0,          0666,
    "mouse",    {Qmouse},   0,          0666,
    "mousein",  {Qmousein}, 0,          0220,
    "mousectl", {Qmousectl},    0,          0220,
};

enum
{
    CMbuttonmap,
    CMscrollswap,
    CMswap,
    CMwildcard,
};

static Cmdtab mousectlmsg[] =
{
    CMbuttonmap,    "buttonmap",    0,
    CMscrollswap,   "scrollswap",   0,
    CMswap,     "swap",     1,
    CMwildcard, "*",        0,
};
// for CMwildcard, devmouse.c relies on arch defined kmousectl()



static uchar buttonmap[8] = {
    0, 1, 2, 3, 4, 5, 6, 7,
};
static bool mouseswap;
static bool scrollswap;
static ulong mousetime;



void Cursortocursor(Cursor*);
```

```
int mousechanged(void*);
static void mouseclock(void);




static void
mousereset(void)
{
    curs = arch_arrow;
    Cursortocursor(&arch_arrow);
    /* redraw cursor about 30 times per second */
    addclock0link(mouseclock, 33);
}

static void
mousefromkbd(int buttons)
{
    kbdbuttons = buttons;
    mousetrack(0, 0, 0, TK2MS(CPUS(0)->ticks));
}

static int
mousedevgen(Chan *c, char *name, Dirtab *tab, int ntab, int i, DirEntry *dp)
{
    int rc;

    rc = devgen(c, name, tab, ntab, i, dp);
    if(rc != -1)
        dp->atime = mousetime;
    return rc;
}

static void
mouseinit(void)
{
    curs = arch_arrow;
    Cursortocursor(&arch_arrow);
    arch_cursoron(true);
    kbdmouse = mousefromkbd;
    mousetime = seconds();
}

static Chan*
mouseattach(char *spec)
{
    return devattach('m', spec);
}

static Walkqid*
mousewalk(Chan *c, Chan *nc, char **name, int nname)
{
    Walkqid *wq;

    /*
     * We use devgen() and not mousedevgen() here
     * see "Ugly problem" in dev.c/devwalk()
     */
    wq = devwalk(c, nc, name, nname, mousedir, nelem(mousedir), devgen);
    if(wq != nil && wq->clone != c && wq->clone != nil && (wq->clone->qid.type&QTDIR)==0)
```

```
        incref(&mouse);
    return wq;
}

static int
mousestat(Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, mousedir, nelem(mousedir), mousedevgen);
}

static Chan*
mouseopen(Chan *c, int omode)
{
    switch((ulong)c->qid.path){
    case Qdir:
        if(omode != OREAD)
            error(Eperm);
        break;
    case Qmouse:
        lock(&mouse);
        if(mouse.open){
            unlock(&mouse);
            error(Einuse);
        }
        mouse.open = true;
        mouse.ref++;
        mouse.lastresize = mouse.resize;
        unlock(&mouse);
        break;
    case Qmousein:
        if(!iseve())
            error(Eperm);
        break;
    default:
        incref(&mouse);
    }
    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}

static void
mousecreate(Chan*, char*, int, ulong)
{
    error(Eperm);
}

static void
mouseclose(Chan *c)
{
    if((c->qid.type&QTDIR)==0 && (c->flag&COPEN)){
        if(c->qid.path == Qmousein)
            return;
        lock(&mouse);
        if(c->qid.path == Qmouse)
            mouse.open = false;
        if(--mouse.ref == 0){
            arch_cursoroff(true);
            curs = arch_arrow;
```

```
            Cursortocursor(&arch_arrow);
            arch_cursoron(true);
        }
        unlock(&mouse);
    }
}


static long
mouseread(Chan *c, void *va, long n, vlong off)
{
    char buf[1+4*12+1];
    uchar *p;
    static int map[8] = {0, 4, 2, 6, 1, 5, 3, 7 };
    ulong offset = off;
    Mousestate m;
    int b;

    p = va;
    switch((ulong)c->qid.path){
    case Qdir:
        return devdirread(c, va, n, mousedir, nelem(mousedir), mousedevgen);

    case Qcursor:
        if(offset != 0)
            return 0;
        if(n < 2*4+2*2*16)
            error(Eshort);
        n = 2*4+2*2*16;
        lock(&cursor);
        BPLONG(p+0, curs.offset.x);
        BPLONG(p+4, curs.offset.y);
        memmove(p+8, curs.clr, 2*16);
        memmove(p+40, curs.set, 2*16);
        unlock(&cursor);
        return n;

    case Qmouse:
        while(mousechanged(nil) == 0)
            sleep(&mouse.r, mousechanged, nil);

        mouse.qfull = 0;
        mousetime = seconds();

        /*
         * No lock of the indices is necessary here, because ri is only
         * updated by us, and there is only one mouse reader
         * at a time.  I suppose that more than one process
         * could try to read the fd at one time, but such behavior
         * is degenerate and already violates the calling
         * conventions for sleep above.
         */
        if(mouse.ri != mouse.wi) {
            m = mouse.queue[mouse.ri];
            if(++mouse.ri == nelem(mouse.queue))
                mouse.ri = 0;
        } else {
            while(!canlock(&cursor))
                tsleep(&up->sleepr, returnfalse, 0, TK2MS(1));
```

```
            m = mouse.Mousestate;
            unlock(&cursor);
        }

        b = buttonmap[m.buttons&7];
        /* put buttons 4 and 5 back in */
        b |= m.buttons & (3<<3);
        if (scrollswap)
            if (b == 8)
                b = 16;
            else if (b == 16)
                b = 8;
        snprint(buf, sizeof buf, "m%11d %11d %11d %11lud ",
            m.xy.x, m.xy.y,
            b,
            m.msec);
        mouse.lastcounter = m.counter;
        if(n > 1+4*12)
            n = 1+4*12;
        if(mouse.lastresize != mouse.resize){
            mouse.lastresize = mouse.resize;
            buf[0] = 'r';
        }
        memmove(va, buf, n);
        return n;
    }
    return 0;
}

static void
setbuttonmap(char* map)
{
    int i, x, one, two, three;

    one = two = three = 0;
    for(i = 0; i < 3; i++){
        if(map[i] == 0)
            error(Ebadarg);
        if(map[i] == '1'){
            if(one)
                error(Ebadarg);
            one = 1<<i;
        }
        else if(map[i] == '2'){
            if(two)
                error(Ebadarg);
            two = 1<<i;
        }
        else if(map[i] == '3'){
            if(three)
                error(Ebadarg);
            three = 1<<i;
        }
        else
            error(Ebadarg);
    }
    if(map[i])
        error(Ebadarg);

    memset(buttonmap, 0, 8);
```

```
    for(i = 0; i < 8; i++){
        x = 0;
        if(i & 1)
            x |= one;
        if(i & 2)
            x |= two;
        if(i & 4)
            x |= three;
        buttonmap[x] = i;
    }
}

static long
mousewrite(Chan *c, void *va, long n, vlong)
{
    char *p;
    Point pt;
    Cmdbuf *cb;
    Cmdtab *ct;
    char buf[64];
    int b, msec;

    p = va;
    switch((ulong)c->qid.path){
    case Qdir:
        error(Eisdir);

    case Qcursor:
        arch_cursoroff(true);
        if(n < 2*4+2*2*16){
            curs = arch_arrow;
            Cursortocursor(&arch_arrow);
        }else{
            n = 2*4+2*2*16;
            curs.offset.x = BGLONG(p+0);
            curs.offset.y = BGLONG(p+4);
            memmove(curs.clr, p+8, 2*16);
            memmove(curs.set, p+40, 2*16);
            Cursortocursor(&curs);
        }
        qlock(&mouse);
        mouse.redraw = true;
        mouseclock();
        qunlock(&mouse);
        arch_cursoron(true);
        return n;

    case Qmousectl:
        cb = parsecmd(va, n);
        if(waserror()){
            free(cb);
            nexterror();
        }

        ct = lookupcmd(cb, mousectlmsg, nelem(mousectlmsg));

        switch(ct->index){
        case CMswap:
            if(mouseswap)
                setbuttonmap("123");
```

```
        else
            setbuttonmap("321");
        mouseswap ^= true;
        break;

    case CMscrollswap:
        scrollswap ^= true;
        break;

    case CMbuttonmap:
        if(cb->nf == 1)
            setbuttonmap("123");
        else
            setbuttonmap(cb->f[1]);
        break;

    case CMwildcard:
        kmousectl(cb); // device specific hook
        break;
    }

    free(cb);
    poperror();
    return n;

case Qmousein:
    if(n > sizeof buf-1)
        n = sizeof buf -1;
    memmove(buf, va, n);
    buf[n] = 0;
    p = 0;
    pt.x = strtol(buf+1, &p, 0);
    if(p == 0)
        error(Eshort);
    pt.y = strtol(p, &p, 0);
    if(p == 0)
        error(Eshort);
    b = strtol(p, &p, 0);
    msec = strtol(p, &p, 0);
    if(msec == 0)
        msec = TK2MS(CPUS(0)->ticks);
    mousetrack(pt.x, pt.y, b, msec);
    return n;

case Qmouse:
    if(n > sizeof buf-1)
        n = sizeof buf -1;
    memmove(buf, va, n);
    buf[n] = 0;
    p = 0;
    pt.x = strtoul(buf+1, &p, 0);
    if(p == 0)
        error(Eshort);
    pt.y = strtoul(p, 0, 0);
    qlock(&mouse);
    if(ptinrect(pt, gscreen->r)){
        mouse.xy = pt;
        mouse.redraw = true;
        mouse.track = true;
        mouseclock();
```

```
        }
        qunlock(&mouse);
        return n;
    }

    error(Egreg);
    return -1;
}

Dev mousedevtab = {
    .dc       =    'm',
    .name     =    "mouse",

    .reset    =    mousereset,
    .init     =    mouseinit,
    .shutdown =    devshutdown,
    .attach   =    mouseattach,
    .walk     =    mousewalk,
    .stat     =    mousestat,
    .open     =    mouseopen,
    .create   =    mousecreate,
    .close    =    mouseclose,
    .read     =    mouseread,
    .bread    =    devbread,
    .write    =    mousewrite,
    .bwrite   =    devbwrite,
    .remove   =    devremove,
    .wstat    =    devwstat,
};

void
Cursortocursor(Cursor *c)
{
    lock(&cursor);
    memmove(&cursor.Cursor, c, sizeof(Cursor));
    arch_ksetcursor(c);
    unlock(&cursor);
}


/*
 *  called by the clock routine to redraw the cursor
 */
static void
mouseclock(void)
{
    if(mouse.track){
        mousetrack(mouse.dx, mouse.dy, mouse.buttons, TK2MS(CPUS(0)->ticks));
        mouse.track = false;
        mouse.dx = 0;
        mouse.dy = 0;
    }
    if(mouse.redraw && canlock(&cursor)){
        mouse.redraw = false;
        arch_cursoroff(false);
        mouse.redraw = arch_cursoron(false);
        unlock(&cursor);
    }
    drawactive(false);
}
```

```
static int
scale(int x)
{
    int sign = 1;

    if(x < 0){
        sign = -1;
        x = -x;
    }
    switch(x){
    case 0:
    case 1:
    case 2:
    case 3:
        break;
    case 4:
        x = 6 + (mouse.acceleration>>2);
        break;
    case 5:
        x = 9 + (mouse.acceleration>>1);
        break;
    default:
        x *= mouse.maxacc;
        break;
    }
    return sign*x;
}

/*
 *  called at interrupt level to update the structure and
 *  awaken any waiting procs.
 */
void
mousetrack(int dx, int dy, int b, int msec)
{
    int x, y, lastb;

    if(gscreen==nil)
        return;

    if(mouse.acceleration){
        dx = scale(dx);
        dy = scale(dy);
    }
    x = mouse.xy.x + dx;
    if(x < gscreen->clipr.min.x)
        x = gscreen->clipr.min.x;
    if(x >= gscreen->clipr.max.x)
        x = gscreen->clipr.max.x;
    y = mouse.xy.y + dy;
    if(y < gscreen->clipr.min.y)
        y = gscreen->clipr.min.y;
    if(y >= gscreen->clipr.max.y)
        y = gscreen->clipr.max.y;

    lastb = mouse.buttons;
    mouse.xy = Pt(x, y);
    mouse.buttons = b|kbdbuttons;
    mouse.redraw = true;
```

```
    mouse.counter++;
    mouse.msec = msec;

    /*
     * if the queue fills, we discard the entire queue and don't
     * queue any more events until a reader polls the mouse.
     */
    if(!mouse.qfull && lastb != b) {    /* add to ring */
        mouse.queue[mouse.wi] = mouse.Mousestate;
        if(++mouse.wi == nelem(mouse.queue))
            mouse.wi = 0;
        if(mouse.wi == mouse.ri)
            mouse.qfull = 1;
    }
    wakeup(&mouse.r);
    drawactive(true);
}

int
mousechanged(void*)
{
    return mouse.lastcounter != mouse.counter ||
        mouse.lastresize != mouse.resize;
}

// called from outside devmouse.c
Point
mousexy(void)
{
    return mouse.xy;
}

// called from outside devmouse.c to let know devmouse.c of changes
void
mouseaccelerate(int x)
{
    mouse.acceleration = x;
    if(mouse.acceleration < 3)
        mouse.maxacc = 2;
    else
        mouse.maxacc = mouse.acceleration;
}

// called from outside devmouse.c to let know devmouse.c of changes
/*
 * notify reader that screen has been resized
 */
void
mouseresize(void)
{
    mouse.resize++;
    wakeup(&mouse.r);
}
```

E.6.10 devices/sys/devroot.c

⟨*method rootclose* 557⟩≡ (558)

```
/*
```

```
 * sysremove() knows this is a nop
 */
static void
rootclose(Chan*)
{
}
```

⟨devroot.c 558⟩≡

```
⟨kernel basic includes 23m⟩

//*****************************************************************************
// Constants and types
//*****************************************************************************

⟨devroot enum Qxxx 313c⟩

⟨devroot.c forward decl ??⟩
⟨struct Dirlist 314a⟩

//*****************************************************************************
// Root
//*****************************************************************************

// note: directories have by convention 0 length

⟨globals rootdir, rootdata, rootlist 314b⟩

//*****************************************************************************
// Boot
//*****************************************************************************

⟨globals bootdir, bootdata, bootlist 316b⟩

//*****************************************************************************
// Functions
//*****************************************************************************

⟨function addlist 314c⟩

⟨function addbootfile 317a⟩

⟨function addrootdir 314d⟩

⟨method rootreset 315a⟩

static Chan*
rootattach(char *spec)
{
    return devattach('/', spec);
}

⟨function rootgen 315b⟩

static Walkqid*
rootwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c,  nc, name, nname, nil, 0, rootgen);
}

static int
```

```
rootstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, nil, 0, rootgen);
}

static Chan*
rootopen(Chan *c, int omode)
{
    return devopen(c, omode, nil, 0, devgen);
}
```

⟨*method rootclose* 557⟩

⟨*method rootread* 316a⟩

```
static long
rootwrite(Chan*, void*, long, vlong)
{
    error(Egreg);
    return 0;
}
```

⟨*global* rootdevtab 313b⟩

E.6.11 devices/sys/devsys.c

⟨*method syswalk* 559a⟩≡ (560a)

```
static Walkqid*
syswalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, sysdir, nelem(sysdir), devgen);
}
```

⟨*method sysstat* 559b⟩≡ (560a)

```
static int
sysstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, sysdir, nelem(sysdir), devgen);
}
```

⟨*method sysopen* 559c⟩≡ (560a)

```
static Chan*
sysopen(Chan *c, int omode)
{
    c->aux = nil;
    c = devopen(c, omode, sysdir, nelem(sysdir), devgen);
    switch((ulong)c->qid.path){
    }
    return c;
}
```

⟨*method sysclose* 559d⟩≡ (560a)

```
static void
sysclose(Chan *c)
{
    switch((ulong)c->qid.path){
    }
}
```

⟨devsys.c 560a⟩≡
⟨*kernel basic includes* 23m⟩

```
#include    <authsrv.h>
```

⟨*devsys.c enum Qxxx* 484a⟩

⟨*global* sysdir 484b⟩

⟨*devsys.c decls and globals* 486d⟩
⟨*global* sysname 486l⟩

⟨*method sysinit* 484c⟩

⟨*method sysattach* 484d⟩

⟨*method syswalk* 559a⟩

⟨*method sysstat* 559b⟩

⟨*method sysopen* 559c⟩

⟨*method sysclose* 559d⟩

⟨*method sysread* 484e⟩

⟨*method syswrite* 485b⟩

⟨*global* sysdevtab 485d⟩

E.6.12 TO LPSPLIT devices/sys/devenv.c

⟨*function* envgrp 560b⟩≡ (561)

```
static Egrp*
envgrp(Chan *c)
{
    if(c->aux == nil)
        return up->egrp;
    return c->aux;
}
```

⟨*function* envwriteable 560c⟩≡ (561)

```
static bool
envwriteable(Chan *c)
{
    return iseve() || c->aux == nil;
}
```

⟨*function* getconfenv 560d⟩≡ (561)

```
/*
 * Return a copy of configuration environment as a sequence of strings.
 * The strings alternate between name and value.  A zero length name string
 * indicates the end of the list
 */
char *
getconfenv(void)
{
    Egrp *eg = &confegrp;
```

```
    Evalue *e;
    char *p, *q;
    int i, n;

    rlock(eg);
    if(waserror()) {
        runlock(eg);
        nexterror();
    }

    /* determine size */
    n = 0;
    for(i=0; i<eg->nent; i++){
        e = eg->ent[i];
        n += strlen(e->name) + e->len + 2;
    }
    p = malloc(n + 1);
    if(p == nil)
        error(Enomem);
    q = p;
    for(i=0; i<eg->nent; i++){
        e = eg->ent[i];
        strcpy(q, e->name);
        q += strlen(q) + 1;
        memmove(q, e->value, e->len);
        q[e->len] = 0;
        /* move up to the first null */
        q += strlen(q) + 1;
    }
    *q = 0;

    poperror();
    runlock(eg);
    return p;
}
```

⟨devenv.c 561⟩≡

```
⟨kernel basic includes 23m⟩

enum
{
    Maxenvsize = 16300,
};

⟨devenv.c forward decl ??⟩

static Egrp confegrp;   /* global environment group containing the kernel configuration */

static Evalue*
envlookup(Egrp *eg, char *name, ulong qidpath)
{
    Evalue *e;
    int i;

    for(i=0; i<eg->nent; i++){
        e = eg->ent[i];
        if(e->qid.path == qidpath || (name && e->name[0]==name[0] && strcmp(e->name, name) == 0))
            return e;
    }
    return nil;
```

```
}

static int
envgen(Chan *c, char *name, Dirtab*, int, int s, DirEntry *dp)
{
    Egrp *eg;
    Evalue *e;

    if(s == DEVDOTDOT){
        devdir(c, c->qid, "#e", 0, eve, DMDIR|0775, dp);
        return 1;
    }

    eg = envgrp(c);
    rlock(eg);
    e = nil;
    if(name)
        e = envlookup(eg, name, -1);
    else if(s < eg->nent)
        e = eg->ent[s];

    if(e == nil) {
        runlock(eg);
        return -1;
    }

    /* make sure name string continues to exist after we release lock */
    kstrcpy(up->genbuf, e->name, sizeof up->genbuf);
    devdir(c, e->qid, up->genbuf, e->len, eve, 0666, dp);
    runlock(eg);
    return 1;
}

static Chan*
envattach(char *spec)
{
    Chan *c;
    Egrp *egrp = nil;

    if(spec && *spec) {
        if(strcmp(spec, "c") == 0)
            egrp = &confegrp;
        if(egrp == nil)
            error(Ebadarg);
    }

    c = devattach('e', spec);
    c->aux = egrp;
    return c;
}

static Walkqid*
envwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, 0, 0, envgen);
}

static int
envstat(Chan *c, uchar *db, int n)
{
```

```
    if(c->qid.type & QTDIR)
        c->qid.vers = envgrp(c)->vers;
    return devstat(c, db, n, 0, 0, envgen);
}

static Chan*
envopen(Chan *c, int omode)
{
    Egrp *eg;
    Evalue *e;
    bool trunc;

    eg = envgrp(c);
    if(c->qid.type & QTDIR) {
        if(omode != OREAD)
            error(Eperm);
    }
    else {
        trunc = omode & OTRUNC;
        if(omode != OREAD && !envwriteable(c))
            error(Eperm);
        if(trunc)
            wlock(eg);
        else
            rlock(eg);
        e = envlookup(eg, nil, c->qid.path);
        if(e == nil) {
            if(trunc)
                wunlock(eg);
            else
                runlock(eg);
            error(Enonexist);
        }
        if(trunc && e->value) {
            e->qid.vers++;
            free(e->value);
            e->value = nil;
            e->len = 0;
        }
        if(trunc)
            wunlock(eg);
        else
            runlock(eg);
    }
    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}

static void
envcreate(Chan *c, char *name, int omode, ulong)
{
    Egrp *eg;
    Evalue *e;
    Evalue **ent;

    if(c->qid.type != QTDIR)
        error(Eperm);
    if(strlen(name) >= sizeof up->genbuf)
```

```
        error("name too long");         /* protect envgen */

    omode = openmode(omode);
    eg = envgrp(c);

    wlock(eg);
    if(waserror()) {
        wunlock(eg);
        nexterror();
    }

    if(envlookup(eg, name, -1))
        error(Eexist);

    e = smalloc(sizeof(Evalue));
    e->name = smalloc(strlen(name)+1);
    strcpy(e->name, name);

    // realloc
    if(eg->nent == eg->ment){
        eg->ment += 32;
        ent = smalloc(sizeof(eg->ent[0])*eg->ment);
        if(eg->nent)
            memmove(ent, eg->ent, sizeof(eg->ent[0])*eg->nent);
        free(eg->ent);
        eg->ent = ent;
    }
    e->qid.path = ++eg->path;
    e->qid.vers = 0;
    eg->vers++;
    eg->ent[eg->nent++] = e;
    c->qid = e->qid;

    wunlock(eg);
    poperror();

    c->offset = 0;
    c->mode = omode;
    c->flag |= COPEN;
}

static void
envremove(Chan *c)
{
    int i;
    Egrp *eg;
    Evalue *e;

    if(c->qid.type & QTDIR)
        error(Eperm);

    eg = envgrp(c);
    wlock(eg);
    e = nil;
    for(i=0; i<eg->nent; i++){
        if(eg->ent[i]->qid.path == c->qid.path){
            e = eg->ent[i];
            eg->nent--;
            eg->ent[i] = eg->ent[eg->nent];
            eg->vers++;
```

```
            break;
        }
    }
    wunlock(eg);
    if(e == nil)
        error(Enonexist);
    free(e->name);
    if(e->value)
        free(e->value);
    free(e);
}

static void
envclose(Chan *c)
{
    /*
     * cclose can't fail, so errors from remove will be ignored.
     * since permissions aren't checked,
     * envremove can't not remove it if its there.
     */
    if(c->flag & CRCLOSE)
        envremove(c);
}

static long
envread(Chan *c, void *a, long n, vlong off)
{
    Egrp *eg;
    Evalue *e;
    ulong offset = off;

    if(c->qid.type & QTDIR)
        return devdirread(c, a, n, 0, 0, envgen);

    eg = envgrp(c);
    rlock(eg);
    e = envlookup(eg, nil, c->qid.path);
    if(e == nil) {
        runlock(eg);
        error(Enonexist);
    }

    if(offset > e->len) /* protects against overflow converting vlong to ulong */
        n = 0;
    else if(offset + n > e->len)
        n = e->len - offset;
    if(n <= 0)
        n = 0;
    else
        memmove(a, e->value+offset, n);
    runlock(eg);
    return n;
}

static long
envwrite(Chan *c, void *a, long n, vlong off)
{
    byte *s;
    ulong len;
    Egrp *eg;
```

```
    Evalue *e;
    ulong offset = off;

    if(n <= 0)
        return 0;
    if(offset > Maxenvsize || n > (Maxenvsize - offset))
        error(Etoobig);

    eg = envgrp(c);
    wlock(eg);
    e = envlookup(eg, nil, c->qid.path);
    if(e == nil) {
        wunlock(eg);
        error(Enonexist);
    }

    len = offset+n;
    if(len > e->len) {
        s = smalloc(len);
        if(e->value){
            memmove(s, e->value, e->len);
            free(e->value);
        }
        e->value = s;
        e->len = len;
    }
    memmove(e->value+offset, a, n);
    e->qid.vers++;
    eg->vers++;
    wunlock(eg);
    return n;
}
```

⟨*global* envdevtab 463a⟩

⟨*function* envcpy 462c⟩

⟨*function* envgrp 560b⟩

⟨*function* envwriteable 560c⟩

⟨*function* ksetenv 462d⟩

⟨*function* getconfenv 560d⟩

E.6.13 devices/sys/devpipe.c

⟨devpipe.c 566⟩≡

```
⟨kernel basic includes 23m⟩

// need that??
#include    "netif.h"

⟨devpipe.c forward decl ??⟩

⟨struct Pipe 342b⟩
```

⟨*struct* `Pipealloc` 342c⟩

⟨*global* `pipealloc` 342d⟩

⟨*devpipe.c enum Qxxx* 342e⟩

⟨*global* `pipedir` 342f⟩

```
#define NPIPEDIR 3
```

⟨*method pipeinit* 343b⟩

⟨*method pipeattach* 343d⟩

⟨*function* `pipegen` 343e⟩

⟨*method pipewalk* 344a⟩

⟨*method pipestat* 344b⟩

⟨*method pipewstat* 345a⟩

⟨*method pipeopen* 345b⟩

⟨*method pipeclose* 346⟩

⟨*method piperead* 347a⟩

⟨*method pipebread* 347b⟩

⟨*method pipewrite* 348a⟩

⟨*method pipebwrite* 348b⟩

⟨*global* `pipedevtab` 349a⟩

E.6.14 devices/sys/devproc.c

⟨`devproc.c` 567⟩≡
⟨*kernel basic includes* 23m⟩

```
#include    <trace.h>
#include    <tos.h>
#include    <ureg.h>
```

⟨*struct* `Mntwalk` 254a⟩

⟨*devproc.c enum Qxxx* 465e⟩

⟨*devproc enum CMxxx* 465f⟩

```
enum{
⟨constant Nevents 378c⟩
⟨constant Emask 378d⟩
};

#define STATSIZE    (2*KNAMELEN+12+9*12)
⟨global procdir 466a⟩
```

⟨*global* proccmd 466b⟩

```
/* Segment type from portdat.h */
```
⟨*global* sname 61b⟩

⟨*devproc QXXX macros* 467a⟩

⟨*devproc.c forward decl* ??⟩

⟨*global trace txxx* 377k⟩

⟨*clock callback profclock* 382a⟩

⟨*function* procgen 476d⟩

⟨*function* _proctrace 378b⟩

⟨*method procinit* 467d⟩

```
static Chan*
procattach(char *spec)
{
    return devattach('p', spec);
}

static Walkqid*
procwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, 0, 0, procgen);
}

static int
procstat(Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, 0, 0, procgen);
}
```

⟨*function* nonone 478a⟩

⟨*method procopen* 473b⟩

⟨*method procwstat* 475⟩

⟨*function* procoffset 476a⟩

⟨*function* procqidwidth 476b⟩

⟨*function* procfdprint 476c⟩

⟨*function* procfds 478b⟩

⟨*method procclose* 479a⟩

⟨*function* int2flag 255a⟩

⟨*function* procargs 479d⟩

⟨*function* eventsavailable 378a⟩

⟨*method procread* 468c⟩

⟨*function* mntscan 255b⟩

⟨*method procwrite* 471⟩

⟨*global* procdevtab 467e⟩

⟨*function* proctext 480a⟩

⟨*function* procstopwait 365f⟩

⟨*function* procctlcloseone 480b⟩

⟨*function* procctlclosefiles 481a⟩

⟨*function* parsetime 481b⟩

⟨*function* procctlreq 482⟩

⟨*function* procstopped 366a⟩

⟨*function* procctlmemio 367e⟩

⟨*function* text2data 368⟩

E.7 files/

E.7.1 files/portdat_files.h

⟨*struct* Extent 569a⟩≡ (569c)

```
struct Extent
{
  int bid;
  ulong start;
  int len;
  Page  *cache;
  Extent  *next;
};
```

⟨*struct* Mntcache 569b⟩≡ (569c)

```
struct Mntcache
{
  Qid qid;
  int dev;
  int type;
  QLock;
  Extent   *list;
  Mntcache *hash;
  Mntcache *prev;
  Mntcache *next;
};
```

⟨portdat_files.h 569c⟩≡

⟨*enum* blockflag 323d⟩

⟨*struct* Block 324a⟩

⟨*function* BLEN 324b⟩
⟨*function* BALLOC 324c⟩

⟨*enum* queuestate 293a⟩

```
// defined in qio.c
extern  uint  qiomaxatomic;

/*
 *  IO queues
 */
// was in qio.c
```
⟨*struct* Queue 293b⟩

```
// was in cache.c
```
⟨*struct* Extent 569a⟩

```
// was in cache.c
```
⟨*struct* Mntcache 569b⟩

⟨*struct* Mount 248b⟩

⟨*struct* Mhead 247c⟩

```
#include <fcall.h>

// was in devmnt.c
```
⟨*struct* Mntrpc 448c⟩

⟨*struct* Mnt 446b⟩

⟨*struct* Path 231a⟩

⟨*enum* accessnamec 215b⟩

⟨*enum* channelflag 206b⟩

⟨*struct* Chan 31a⟩

⟨*struct* Evalue 461b⟩

⟨*struct* Egrp 461c⟩

```
// internals
```

⟨*struct* Walkqid 234a⟩

⟨*struct* Dev 32d⟩

```
// array<Dev>, it looks like an allocated array<ref<dev>> but
// it is really a static array put here to avoid backward deps on conf_devtab,
// and it is not really a <ref<dev>> because it's pointers to static
// structures (e.g. mousedevtab, vgadevtab, etc).
extern Dev** devtab;

⟨struct Dirtab 313a⟩

// used by with devmnt.c and sysfile.c
struct Bogus {
    Chan    *chan;
    Chan    *authchan;
    char    *spec;
    int flags;
};


//*****************************************************************************
// Internal to files/
//*****************************************************************************

⟨constants tags 447b⟩

// actually internal to devmnt.c and mnt.c
⟨struct Mntalloc 446c⟩
extern struct Mntalloc mntalloc;
```

E.7.2 files/portfns_files.h

E.7.3 files/chan.c

```
⟨chan.c 571⟩≡
  ⟨kernel basic includes 23m⟩

  ⟨chan.c forward decl ??⟩

  ⟨chan.c debugging macro ??⟩

  enum
  {
  ⟨constant PATHSLOP 231c⟩
  ⟨constant PATHMSLOP 258b⟩
  };

  ⟨struct Chanalloc 206d⟩
  ⟨global chanalloc 207b⟩

  ⟨struct Elemlist 219b⟩

  ⟨function chanpath 242a⟩

  ⟨function isdotdot 233a⟩

  ⟨function emptystr 524b⟩

  ⟨function chandevreset 199f⟩

  ⟨function chandevinit 203a⟩
```

⟨*function* `chandevshutdown` 489c⟩

⟨*constructor* `newchan` 207c⟩

⟨*global* `npath` ??⟩

⟨*constructor* `newpath` 231b⟩

⟨*function* `copypath` 231d⟩

⟨*destructor* `pathclose` 232a⟩

⟨*function* `fixdotdotname` 233b⟩

⟨*function* `uniquepath` 232b⟩

⟨*function* `addelem` 232c⟩

⟨*destructor* `chanfree` 208a⟩

⟨*function* `cclose` 214d⟩

⟨*struct* `Clunkq` 227a⟩
⟨*global* `clunkq` 227b⟩

⟨*function* `ccloseq` 227c⟩

⟨*function* `clunkwork` 227d⟩

⟨*function* `closeproc` 227e⟩

⟨*function* `uniquechan` 208b⟩

⟨*function* `eqqid` 31e⟩

⟨*function* `eqchan` 32b⟩

⟨*function* `eqchantdqid` 32c⟩

⟨*constructor* `newmhead` 247d⟩

⟨*function* `cmount` 252e⟩

⟨*function* `cunmount` 266c⟩

⟨*function* `cclone` 237d⟩

⟨*function* `findmount` 256d⟩

⟨*function* `domount` 257⟩

⟨*function* `undomount` 259a⟩

⟨*function* `ewalk` 236a⟩

⟨*global* `Edoesnotexist` 520a⟩

⟨*function* `walk` 234h⟩

⟨*function* `createdir` 262c⟩

⟨*function* `growparse` 220a⟩

⟨*function* `parsename` 219e⟩

⟨*function* `memrchr` 523b⟩

⟨*function* `namelenerror` 221c⟩

⟨*function* `nameerror` 222a⟩

⟨*function* `namec` 215f⟩

⟨*function* `skipslash` 220b⟩

⟨*global* `isfrog` 219a⟩

⟨*function* `validname0` 218⟩

⟨*function* `validname` 217e⟩

⟨*function* `validnamedup` 217d⟩

⟨*function* `error_if_not_dir` 230b⟩

⟨*destructor* `putmhead` 248a⟩

E.7.4 files/dev.c

⟨`dev.c` 573⟩≡

```
⟨kernel basic includes 23m⟩

// was in $CONF.c
⟨global devtab 32e⟩

⟨function mkqid 32a⟩

⟨function devno 209a⟩

⟨function devdir 317c⟩

⟨function devgen 317d⟩

void
devreset(void)
{
}

void
devinit(void)
{
}

void
devshutdown(void)
{
}
```

⟨*function* devattach 208c⟩

⟨*function* devclone 318⟩

⟨*function* devwalk 319⟩

⟨*function* devstat 320⟩

⟨*function* devdirread 321c⟩

⟨*function* devpermcheck 322a⟩

⟨*function* devopen 322b⟩

```
void
devcreate(Chan*, char*, int, ulong)
{
    error(Eperm);
}
```

⟨*function* devbread 323b⟩

⟨*function* devbrwrite 323c⟩

```
void
devremove(Chan*)
{
    error(Eperm);
}

int
devwstat(Chan*, uchar*, int)
{
    error(Eperm);
    return 0;
}

//void
//devpower(int)
//{
//    error(Eperm);
//}
//
//int
//devconfig(int, char *, DevConf *)
//{
//    error(Eperm);
//    return 0;
//}
```

E.7.5 files/allocb.c

⟨allocb.c 574⟩≡

⟨*kernel basic includes* 23m⟩

```
enum
{
    Hdrspc      = 64,       /* leave room for high-level headers */
```

```
    Bdead       = 0x51494F42,   /* "QIOB" */
};
```

⟨*struct* Ialloc 325c⟩
⟨*global* ialloc 325d⟩

⟨*function* _allocb 324e⟩

⟨*function* allocb 325b⟩

⟨*function* iallocb 325e⟩

⟨*function* freeb 326⟩

⟨*function* checkb 327a⟩

⟨*function* iallocsummary 502c⟩

E.7.6 files/env.c

⟨env.c 575a⟩≡
⟨*kernel basic includes* 23m⟩

```
// this used to be in devenv.c, but to avoid backward deps I've splitted
// this file in 2
```
⟨*function* closeegrp 462b⟩

E.7.7 files/file.c

⟨file.c 575b⟩≡
⟨*kernel basic includes* 23m⟩

⟨*function* fdtochan 213b⟩

⟨*function* openmode 212e⟩

⟨*function* fdclose 214b⟩

⟨*function* validstat 245b⟩

E.7.8 files/mnt.c

⟨mnt.c 575c⟩≡
⟨*kernel basic includes* 23m⟩

```
// this used to be in devmnt.c, but to avoid backward deps I've splitted
// this file in 2 (which forced to put more stuff in portdat_files.h though).
```

⟨*global* mntalloc 447a⟩

```
#define MAXRPC (IOHDRSZ+8192)
```

⟨*function* freetag 447c⟩

⟨*function* mntfree 447d⟩

⟨*function* mntpntfree 448a⟩

⟨*function* muxclose 448b⟩

⟨*function* mntversion 449b⟩

E.7.9 files/qio.c

⟨*global* qiomaxatomic 576a⟩≡ (576b)

```
enum
{
    Maxatomic   = 64*1024,
};

uint    qiomaxatomic = Maxatomic;
```

⟨qio.c 576b⟩≡

⟨*kernel basic includes* 23m⟩

⟨*qio.c globals cnt* ??⟩

⟨*qio.c global debugging* ??⟩

⟨*global* qiomaxatomic 576a⟩

⟨*function* ixsummary 502b⟩

⟨*function* freeblist 327b⟩

⟨*function* padblock 327c⟩

⟨*function* blocklen 328a⟩

⟨*function* blockalloclen 328b⟩

⟨*function* concatblock 329a⟩

⟨*function* pullupblock 329b⟩

⟨*function* pullupqueue 293c⟩

⟨*function* trimblock 331a⟩

⟨*function* copyblock 331b⟩

⟨*function* adjustblock 332a⟩

⟨*function* pullblock 332b⟩

⟨*function* qget 294b⟩

⟨*function* qdiscard 295⟩

⟨*function* qconsume 296⟩

⟨*function* qpass 297⟩

⟨*function* qpassnolim 298⟩

⟨*function* packblock 330⟩

⟨*function* qproduce 299⟩

⟨*function* qcopy 300⟩

⟨*function* qopen 301a⟩

⟨*function* qbypass 301b⟩

⟨*function* notempty 301c⟩

⟨*function* qwait 302a⟩

⟨*function* qaddlist 302b⟩

⟨*function* qremove 302c⟩

⟨*function* bl2mem 333a⟩

⟨*function* qputback 303a⟩

⟨*function* qwakeup_iunlock 303b⟩

⟨*function* qbread 303c⟩

⟨*function* qread 304⟩

⟨*function* qnotfull 306a⟩

⟨*function* qbwrite 306b⟩

⟨*function* qwrite 294a⟩

⟨*function* qiwrite 308⟩

⟨*function* qfree 309a⟩

⟨*function* qclose 309b⟩

⟨*function* qhangup 310a⟩

⟨*function* qiclosed 310b⟩

⟨*function* qreopen 310c⟩

⟨*function* qlen 310d⟩

⟨*function* qwindow 310e⟩

⟨*function* qcanread 311a⟩

⟨*function* qsetlimit 311b⟩

⟨*function* qnoblock 311c⟩

⟨*function* qflush 311d⟩

⟨*function* qfull 312⟩

E.7.10 files/sysfile.c

⟨sysfile.c 578⟩≡

⟨*kernel basic includes* 23m⟩

```
/*
 * The sys*() routines needn't poperror() as they return directly to syscall().
 */
```

⟨*function* unlockfgrp 212b⟩

⟨*function* growfd 211b⟩

⟨*function* findfreefd 211a⟩

⟨*function* newfd 210c⟩

⟨*function* newfd2 341b⟩

⟨*syscall fd2path* 241c⟩

⟨*syscall pipe* 341a⟩

⟨*syscall dup* 483b⟩

⟨*syscall open* 212d⟩

⟨*syscall close* 213a⟩

⟨*function* unionread 261a⟩

⟨*function* unionrewind 262a⟩

⟨*function* dirfixed 272⟩

⟨*function* offsetof_name_direntry 243a⟩

⟨*function* direntry_setname 242d⟩

⟨*function* mountrock 269c⟩

⟨*function* mountrockread 270a⟩

⟨*function* mountrewind 270b⟩

⟨*function* mountfix 270c⟩

⟨*function* read 222d⟩

⟨*syscall pread* 222b⟩

⟨*function* write 224a⟩

⟨*syscall pwrite* 223a⟩

⟨*function* sseek 225⟩

⟨*syscall seek* 224b⟩

⟨*function* `pathlast` 244b⟩

⟨*syscall fstat* 244a⟩

⟨*syscall stat* 243c⟩

⟨*syscall chdir* 241a⟩

⟨*function* `bindmount` 251e⟩

⟨*syscall bind* 251c⟩

⟨*syscall mount* 264e⟩

⟨*syscall unmount* 265c⟩

⟨*syscall create* 238a⟩

⟨*syscall remove* 240d⟩

⟨*function* `wstat` 245c⟩

⟨*syscall wstat* 244d⟩

⟨*syscall fwstat* 245a⟩

E.8 filesystems/

E.9 init/

E.9.1 init/portfns_init.h

E.9.2 init/user/preboot/initcode.c

E.9.3 init/user/boot/aux.c

E.9.4 init/user/boot/boot.c

E.9.5 init/user/boot/boot.h

E.9.6 init/user/boot/local.c

E.10 lib/

E.10.1 lib/lib.h

⟨*constant* `ERRMAX` 579a⟩≡ (? 0—1)

```
ERRMAX = 128, /* max length of error string */
```

⟨*constant* `KNAMELEN` 579b⟩≡ (? 0—1)

```
KNAMELEN = 28,  /* max length of name held in kernel */
```

⟨*function* nelem 580a⟩≡ (580e)

```
#define nelem(x)  (sizeof(x)/sizeof((x)[0]))
```

⟨*function* offsetof 580b⟩≡ (580e)

```
//@Scheck: not dead, actually exanded in macros.h
#define offsetof(s, m)  (ulong)(&(((s*)0)->m))
```

⟨*function* assert 580c⟩≡ (580e)

```
#define assert(x) do { if(x) {}else _assert("x");}while(0)
```

⟨*lib.h print decl* 580d⟩≡ (580e)

```
// This used to be regular function, but to avoid backward deps in the kernel
// I made it into a pointer function (a bit ugly, and maybe unsafe)
extern  int (*print)(char*, ...);
```

⟨lib.h 580e⟩≡

```
// coupling: libc.h
// This file mostly exports code from lib_core/libc/ (linked with the kernel).
// Those functions are also exported in include/libc.h but they prefered to
// not include it and instead to reexport here a subset specific to the kernel.
// There are also the poolxxx() functions exported in include/pool.h
// (also part of libc) that are used in memory/alloc.c.
// Some functions in libc are also "overriden" (via linker abuse):
//  - print.c overrides _fmtlock(), _fmtunlock() that were in libc/fmt/
//  - devcons.s overrides the sysfatal() that was in libc/9sys/

// pad specific, could be in u.h
⟨pad basic types ??⟩

//#define nil (void*)0 in u.h
#define nilptr 0

⟨lib.h forward decl ??⟩

/*
 * functions (mostly) linked in from libc.
 */
⟨function nelem 580a⟩
⟨function offsetof 580b⟩
⟨function assert 580c⟩

/*
 * mem routines
 */
⟨lib.h mem functions decl 523a⟩

/*
 * string routines
 */
⟨lib.h string functions decl 523c⟩

//redefined in the kernel
extern  int cistrcmp(char*, char*);
extern  int cistrncmp(char*, char*, int);

⟨enum utf 526d⟩

/*
 * rune routines
 */
```

⟨*lib.h rune functions decl* 527a⟩

```
extern  int abs(int);

/*
 * print routines
 */
typedef int (*Fmts)(Fmt*);
```
⟨*struct* Fmt 526a⟩

⟨*lib.h print decl* 580d⟩

⟨*lib.h printf functions decl* 525d⟩

⟨*lib.h pragmas* 526c⟩

⟨*lib.h fmt functions decl* 526b⟩

```
/*
 * one-of-a-kind
 */
```
⟨*lib.h strto functions decl* 525a⟩

```
//todo: should return a (void*)
extern  ulong getcallerpc(void*);
extern  char* cleanname(char*);
extern  int getfields(char*, char**, int, int, char*);
extern  int tokenize(char*, char**, int);
extern  void  qsort(void*, long, long, int (*)(void*, void*));
```

⟨*lib.h exxx decl* 190b⟩

```
/*
 * Syscall data structures
 */
```
⟨*enum* mount 247b⟩

⟨*enum* open 212c⟩

⟨*enum* note 334d⟩

⟨*enum* miscsize **??**⟩

⟨*enum* qidtype 31d⟩

⟨*enum* dirmode 242c⟩

⟨*struct* Qid 31c⟩

⟨*struct* DirEntry 242b⟩

⟨*struct* Waitmsg 90c⟩

E.11 memory/

E.11.1 memory/portdat_memory.h

⟨portdat_memory.h 582⟩≡

```
// see also Arch_KMap in <arch>/ (but used in port)

//*****************************************************************************
// Page < Pagetable (can be filled by KImage) < Segment
//*****************************************************************************

// All the ref<Page> here are references to Pages in the array<Page> of
// Palloc.pages (pool allocator)
// All the ref<Kimage> here are references to KImage in the ?? of
// Imagealloc.free?

⟨enum modref 57e⟩

⟨enum Cachectl 423f⟩

⟨struct Page 57d⟩

⟨type PageOrSwap 135b⟩

⟨struct Pagetable 62b⟩

// used to be arch-specific, but many constants were used in port/ code
⟨type PTExxx 117b⟩

⟨struct KImage 124a⟩

⟨enum segtype 61a⟩

⟨constant PG_ONSWAP 135a⟩

⟨function onswap 135c⟩
⟨function pagedout 127c⟩
⟨function swapaddr 143b⟩

#define SEGMAXSIZE  (PAGEDIRSIZE*PAGETABMAPMEM)

⟨struct Physseg 424o⟩

enum
{
    ⟨constant LRESPROF 381j⟩
};

⟨struct Segment 61c⟩

//*****************************************************************************
// Internal to memory/
//*****************************************************************************

// See also RMap in 386/

// actually internal to xalloc.c, but important so here
⟨constants holes 48a⟩
```

⟨*struct* Hole 47b⟩

⟨*struct* Xhdr 48c⟩

⟨*struct* Xalloc 47c⟩
```
//IMPORTANT: static Xalloc xlists; // private to xalloc.c


// from pool.h
//struct Pool {
//  char* name;
//  ulong maxsize;
//
//  ulong cursize;
//  ulong curfree;
//  ulong curalloc;
//
//  ulong minarena; /* smallest size of new arena */
//  ulong quantum;  /* allocated blocks should be multiple of */
//  ulong minblock; /* smallest newly allocated block */
//
//  void* freeroot; /* actually Free* */
//  void* arenalist;  /* actually Arena* */
//
//  void* (*alloc)(ulong);
//  int (*merge)(void*, void*);
//  void  (*move)(void* from, void* to);
//
//  int flags;
//  int nfree;
//  int lastcompact;
//
//  void  (*lock)(Pool*);
//  void  (*unlock)(Pool*);
//  void  (*print)(Pool*, char*, ...);
//  void  (*panic)(Pool*, char*, ...);
//  void  (*logstack)(Pool*);
//
//  void* private;
//};

// exported by libc include/pool.h, used by malloc, defined in pool.c in this dir
// memory pools for malloc()/free() (using xalloc pools)
//IMPORTANT: extern Pool*  mainmem;
// memory pools for ??
//IMPORTANT: extern Pool*  imagmem;
```

⟨*struct* Pallocmem 58d⟩

⟨*function* pghash 122a⟩

⟨*struct* Palloc 58a⟩
```
extern  Palloc  palloc;
```

⟨*constant* NFREECHAN 199c⟩

⟨*function* ihash 124d⟩

⟨*struct* Imagealloc 124b⟩

```
//IMPORTANT: static struct Imagealloc imagealloc; (segment.c)
// so have conf.nimage + 1 Kimages
extern  KImage  swapimage;

⟨struct Swapalloc 133b⟩
extern struct Swapalloc swapalloc;
```

E.11.2 memory/portfns_memory.h

E.11.3 memory/portdat_memory.c

```
⟨portdat_memory.c 584a⟩≡
  ⟨kernel basic includes 23m⟩

  ⟨global palloc 58b⟩

  ⟨global swapalloc 134a⟩
  ⟨global swapimage 134c⟩
```

E.11.4 memory/xalloc.c

```
⟨xalloc.c 584b⟩≡
  ⟨kernel basic includes 23m⟩

  ⟨xalloc.c forward decl ??⟩

  //*****************************************************************************
  // The global
  //*****************************************************************************

  ⟨global xlists 48b⟩

  //*****************************************************************************
  // Initialization
  //*****************************************************************************

  ⟨function xinit 196b⟩

  //*****************************************************************************
  // Functions
  //*****************************************************************************

  ⟨function xspanalloc 51a⟩

  ⟨function xallocz 48e⟩

  ⟨function xalloc 48d⟩

  ⟨function xfree 49a⟩

  ⟨function xmerge 50a⟩

  ⟨function xhole 49c⟩

  //*****************************************************************************
  // Debugging
  //*****************************************************************************
```

⟨*function* xsummary 501a⟩

E.11.5 memory/pool.c

⟨*pool.c forward decl* 585a⟩≡ (585b)

```
typedef struct Private  Private;
```

⟨pool.c 585b⟩≡

```
⟨kernel basic includes 23m⟩

#include    <pool.h>

⟨pool.c forward decl 585a⟩

//*****************************************************************************
// Concurrency
//*****************************************************************************

// See Pool.private, for mutual exclusion on memory pools
⟨pool.c struct Private 52a⟩

//*****************************************************************************
// Pool methods
//*****************************************************************************

⟨function poolprint 53d⟩

⟨function ppanic 54a⟩

⟨function plock 52b⟩

⟨function punlock 52c⟩

⟨function poolsummary 501c⟩

//*****************************************************************************
// The globals
//*****************************************************************************

⟨global pmainmem 53a⟩

⟨global pimagmem 53b⟩

⟨global mainmem and imagmem 53c⟩

⟨function mallosummary 501b⟩
```

E.11.6 memory/alloc.c

⟨alloc.c 585c⟩≡

```
⟨kernel basic includes 23m⟩

#include    <pool.h>

/* everything from here down should be the same in libc, libdebugmalloc, and the kernel */
/* - except the code for malloc(), which alternately doesn't clear or does. */
```

```
/* - except the code for smalloc(), which lives only in the kernel. */

//*****************************************************************************
// Debugging support
//*****************************************************************************

/*
 * Npadlong is the number of 32-bit longs to leave at the beginning of
 * each allocated buffer for our own bookkeeping.  We return to the callers
 * a pointer that points immediately after our bookkeeping area.  Incoming pointers
 * must be decremented by that much, and outgoing pointers incremented.
 * The malloc tag is stored at MallocOffset from the beginning of the block,
 * and the realloc tag at ReallocOffset.  The offsets are from the true beginning
 * of the block, not the beginning the caller sees.
 *
 * The extra if(Npadlong != 0) in various places is a hint for the compiler to
 * compile out function calls that would otherwise be no-ops.
 */

/* tracing */
enum {
    Npadlong    = 2,
    MallocOffset = 0,
    ReallocOffset = 1
};
```

⟨*alloc.c debugging macro* ??⟩

⟨*function* setmalloctag 56c⟩

⟨*function* setrealloctag 56d⟩

```
//*****************************************************************************
// Pool wrappers
//*****************************************************************************
```

⟨*function* smalloc 55a⟩

⟨*function* malloc 54b⟩

⟨*function* mallocz 55c⟩

⟨*function* mallocalign 55d⟩

⟨*function* free 54c⟩

⟨*function* realloc 56a⟩

⟨*function* msize 325a⟩

```
//*****************************************************************************
// kstr functions
//*****************************************************************************

//pad: was in chan.c
```

⟨*function* kstrcpy 523d⟩

⟨*function* kstrdup 524a⟩

E.11.7 memory/fault.c

⟨*function* checkpages 587a⟩≡ (587b)

```
void
checkpages(void)
{
    int checked;
    ulong addr, off;
    Pagetable *p;
    Page *pg;
    Segment **sp, **ep, *s;

    if(up == nil)
        return;

    checked = 0;
    // foreach(up->seg)
    for(sp=up->seg, ep=&up->seg[NSEG]; sp<ep; sp++){
        s = *sp;
        if(s == nil)
            continue;
        qlock(&s->lk);
        for(addr=s->base; addr<s->top; addr+=BY2PG){
            off = addr - s->base;
            p = s->pagedir[off/PAGETABMAPMEM];
            if(p == nil)
                continue;
            pg = p->pagetab[(off&(PAGETABMAPMEM-1))/BY2PG];
            if(pg == nil || pagedout(pg))
                continue;
            arch_checkmmu(addr, pg->pa);
            checked++;
        }
        qunlock(&s->lk);
    }
    print("%ld %s: checked %d page table entries\n", up->pid, up->text, checked);
}
```

⟨fault.c 587b⟩≡

```
⟨kernel basic includes 23m⟩

⟨fault.c forward decl ??⟩

⟨function fault 117a⟩

⟨function faulterror 129a⟩

// for debugging SG_PHYSICAL
void    (*checkaddr)(ulong, Segment *, Page *);
ulong   addr2check;

⟨function fixfault 117c⟩

⟨function pio 127f⟩

⟨function okaddr 66b⟩

⟨function validaddr 67a⟩

⟨function vmemchr 67b⟩
```

⟨*function* seg 66a⟩

⟨*function* checkpages 587a⟩

E.11.8 memory/page.c

⟨page.c 588⟩≡

⟨*kernel basic includes* 23m⟩

⟨*page.c forward decl* ??⟩

```
//*****************************************************************************
// Initialization
//*****************************************************************************
```

⟨*function* pageinit 198a⟩

```
//*****************************************************************************
// Functions
//*****************************************************************************
```

⟨*function* pageunchain 59c⟩

⟨*function* pagechaintail 60b⟩

⟨*function* pagechainhead 60a⟩

⟨*constructor* newpage 58f⟩

⟨*function* hasfreepages 139c⟩

⟨*destructor* putpage 59d⟩

⟨*function* auxpage ??⟩

⟨*global* dupretries 129c⟩

⟨*function* duppage 129d⟩

⟨*function* copypage 121f⟩

⟨*function* uncachepage 123c⟩

⟨*function* cachepage 122d⟩

⟨*function* cachedel 123d⟩

⟨*function* lookpage 122e⟩

⟨*function* ptcpy 63a⟩

⟨*constructor* ptalloc 62e⟩

⟨*destructor* freept 62f⟩

⟨*function* pagenumber 428a⟩

⟨*function* checkpagerefs 426d⟩

⟨*function* portcountpagerefs 427c⟩

E.11.9 memory/segment.c

⟨segment.c 589⟩≡

```
⟨kernel basic includes 23m⟩
#include "io.h"

//*****************************************************************************
// Global
//*****************************************************************************

⟨global imagealloc 124e⟩

//*****************************************************************************
// Misc
//*****************************************************************************

⟨global physseg 425b⟩
⟨global physseglock 425c⟩

⟨hook _globalsegattach 350c⟩

⟨segment.c forward decl ??⟩

//*****************************************************************************
// Initialization
//*****************************************************************************

⟨function imageinit 199d⟩

//*****************************************************************************
// Functions
//*****************************************************************************

⟨constructor newseg 63b⟩

⟨destructor putseg 64a⟩

⟨function relocateseg 88b⟩

⟨function dupseg 67d⟩

⟨function segpage 65b⟩

⟨constructor attachimage 125c⟩

⟨struct Irstats 517g⟩
⟨segment.c global irstats 517h⟩

⟨function imagereclaim 133a⟩

⟨function imagechanreclaim 132b⟩

⟨destructor putimage 131d⟩

⟨function ibrk 69d⟩
```

⟨*function* mfreeseg 71d⟩

⟨*function* isoverlap 352a⟩

⟨*function* addphysseg 425d⟩

⟨*function* segattach 350d⟩

⟨*clock callback segclock* 382b⟩

```
// was in another file before
```
⟨*function* data2txt 127b⟩

E.11.10 memory/swap.c

⟨swap.c 590⟩≡
⟨*kernel basic includes* 23m⟩

⟨*swap.c forward decl* ??⟩

⟨*global* iolist 137b⟩
⟨*global* ioptr 137c⟩

⟨*global* genxxx 142c⟩

⟨*function* gentick 142b⟩

⟨*function* swapinit 199a⟩

⟨*function* newswap 134d⟩

⟨*function* putswap 135d⟩

⟨*function* dupswap 136a⟩

⟨*function* swapcount 136c⟩

⟨*function* kickpager 139d⟩

⟨*function* pager 139e⟩

⟨*function* pageout 137d⟩

⟨*function* canflush 141f⟩

⟨*function* pagepte 136d⟩

⟨*function* pagersummary 502a⟩

⟨*function* pageiocomp 143a⟩

⟨*function* executeio 142e⟩

⟨*function* needpages 141a⟩

⟨*function* setswapchan 144⟩

E.11.11 memory/sysmemory.c

⟨sysmemory.c 591a⟩≡
⟨*kernel basic includes* 23m⟩

```
// see also sysbrk in sysproc.c

// those functions used to be in segment.c

void
ptflush(Pagetable *pt, int s, int e)
{
 int i;
 Page *p;

 for(i = s; i < e; i++) {
  p = pt->pagetab[i];
  if(pagedout(p) == 0)
   memset(p->cachectl, PG_TXTFLUSH, sizeof(p->cachectl));
 }
}
```

⟨*syscall segflush* 354c⟩

E.12 misc/

E.12.1 misc/portdat_misc.h

⟨portdat_misc.h 591b⟩≡

```
enum misc_constants_portable {
    /* READSTR was 1000, which is way too small for usb's ctl file */
    READSTR = 4000,   /* temporary buffer size for device reads */

    KB = 1024,
    MB = (1024*1024),

    NUMSIZE = 12,   /* size of formatted number */

    ⟨constant PRINTSIZE 286a⟩
};
```

⟨*struct* Cmdbuf 459b⟩

⟨*struct* Cmdtab 459a⟩

E.12.2 misc/portfns_misc.h

E.12.3 misc/parse.c

⟨parse.c 591c⟩≡
⟨*kernel basic includes* 23m⟩

⟨*ncmdfield* 461a⟩

⟨*function* parsecmd 459c⟩

⟨*function* cmderror 460a⟩

⟨*function* lookupcmd 460b⟩

E.12.4 misc/random.c

⟨random.c 592a⟩≡
⟨*kernel basic includes* 23m⟩

⟨*struct* Rb 495b⟩

⟨*global* rb 495c⟩

⟨*struct* rbnotfull 495d⟩

⟨*struct* rbnotempty 495e⟩

⟨*function* genrandom 495f⟩

⟨*clock callback randomclock* 496a⟩

⟨*function* randominit 496b⟩

⟨*function* randomread 496c⟩

E.13 interrupts/

E.13.1 interrupts/portfns_interrupts.h

⟨portfns_interrupts.h 592b⟩≡

```
// <arch>/trap.c (used in arch specific)
void  arch_intrenable(int, void (*)(Ureg*, void*), void*, int, char*);

// <arch>/trap.c (called from main)
void  arch__trapinit(void);
```

E.14 processes/

E.14.1 processes/portdat_processes.h

⟨portdat_processes.h 592c⟩≡

```
// in lib.h: Waitmsg, ERRMAX

//*****************************************************************************
// Proc components
//*****************************************************************************
// All the ref<Proc> here are references to Proc in the array<Proc> of
// Procalloc.arena (pool allocator)

//-----------------------------------------------------------------------
```

```
// State
//-----------------------------------------------------------------

// TODO: state transition diagram
⟨enum procstate 30c⟩

// hash<enum<procstate>, string>, to debug
extern  char* statename[];

//-----------------------------------------------------------------
// Memory
//-----------------------------------------------------------------
⟨enum procseg 65d⟩

//-----------------------------------------------------------------
// Files
//-----------------------------------------------------------------

enum
{
   ⟨constant DELTAFD 210a⟩
};

⟨struct Fgrp 209b⟩


⟨function MOUNTH 249e⟩

⟨struct Pgrp 249c⟩

//-----------------------------------------------------------------
// System call
//-----------------------------------------------------------------

⟨constant MAXSYSARG 100d⟩

⟨struct Sargs 101a⟩

//-----------------------------------------------------------------
// Notes
//-----------------------------------------------------------------

enum {
    ⟨constant NNOTE 335j⟩
};

⟨enum notekind 334c⟩

⟨struct Note 335a⟩
extern Counter  noteidalloc;

//-----------------------------------------------------------------
// Process children waiting
//-----------------------------------------------------------------

⟨struct Waitq 90d⟩

//-----------------------------------------------------------------
// Synchronization (Rendez vous)
//-----------------------------------------------------------------
```

⟨*function* REND 360a⟩

⟨*struct* Rgrp 360b⟩

```
//-----------------------------------------------------------------
// Alarms, timers
//-----------------------------------------------------------------
// now in portdat_time.h

//-----------------------------------------------------------------
// Scheduling
//-----------------------------------------------------------------

enum {
    ⟨constant Npriq 145a⟩
    ⟨constant Nrq 146f⟩
};
```

⟨*enum* priority 145b⟩

⟨*enum* edfflags 434a⟩

⟨*struct* Edf 434b⟩

```
//-----------------------------------------------------------------
// Error management
//-----------------------------------------------------------------
enum {
    ⟨constant NERR 520d⟩
};

//-----------------------------------------------------------------
// Stats, profiling
//-----------------------------------------------------------------
```

⟨*enum* proctimer 380a⟩

```
//-----------------------------------------------------------------
// Debugger
//-----------------------------------------------------------------
```

⟨*enum* procctl 364c⟩

```
//-----------------------------------------------------------------
// Misc
//-----------------------------------------------------------------
```

⟨*enum* fpsavestatus 387a⟩

```
//*****************************************************************************
// Proc, the big one
//*****************************************************************************
```

⟨*struct* Proc 29c⟩

⟨*macro waserror poperror* 521b⟩

```
//*****************************************************************************
// Internal to process/
```

```
//*****************************************************************************

// Proc allocator (singleton), was actually in proc.c, but important so here
⟨struct Procalloc 73d⟩
//IMPORTANT: static struct Procalloc procalloc; (in proc.c)

⟨struct Schedq 146h⟩
// hash<enum<priority>, Schedq>, Nrq is the number of priority level (20+2)
//IMPORTANT: Schedq  runq[Nrq];  (in proc.c)

// was in alarm.c, but important so here
⟨struct Alarms 175d⟩
//IMPORTANT: static Alarms alarms; (in alarm.c)
//IMPORTANT: static Rendez alarmr; (in alarm.c)

⟨struct Active 29a⟩
extern struct Active active;
```

E.14.2 processes/portfns_processes.h

E.14.3 processes/portdat_processes.c

```
⟨portdat_processes.c 595a⟩≡
  ⟨kernel basic includes 23m⟩

  ⟨hook proctrace 376d⟩
  ⟨hook kproftimer 515c⟩

  ⟨global active 29b⟩
```

E.14.4 time/portfns_time.h

```
⟨portfns_time.h 595b⟩≡

  // portclock.c
  ulong   tk2ms(ulong);
  ⟨macro TK2MS 165e⟩
  ⟨macro MS2NS 165c⟩
  #define TK2SEC(t) ((t)/Arch_HZ) /* ticks to seconds */ // not used in port
  //#define MS2HZ  (1000/Arch_HZ)  /* millisec per clock tick */
  ulong   ms2tk(ulong);
  void    timerdel(Timer*);
  void    timeradd(Timer*);
  Timer*  addclock0link(void (*)(void), int);
  void    timerintr(Ureg*, Tval);
  void    timersinit(void);

  // alarm.c
  ulong   procalarm(ulong);
  void    alarmkproc(void*);
  void    checkalarms(void);

  // tod.c
  // initialize the hidden global tod.c#tod
  void    todinit(void);
  void    todsetfreq(vlong);
  void    todset(vlong, vlong, int);
```

```
vlong   todget(vlong*);
uvlong  fastticks2us(uvlong);
uvlong  ns2fastticks(uvlong);
long    seconds(void);

// <arch>/clock.c (called from port)
void   arch_timerset(Tval x); // called from portclock.c (e.g., addclock0link)
ulong  arch_us(void);         // called from edf.c
long   arch_lcycles(void); // called from edf.c, sysproc.c, taslock.c
ulong  arch_perfticks(void);
// now in core/ for backward deps
//uvlong arch_fastticks(uvlong *hz); // called from portclock.c
//void   arch_microdelay(int);
//void   arch_delay(int);
// <arch>/clock.c (called from port but signature not portable across <arch>)
//void arch_cycles(uvlong*);
```

E.14.5 time/portdat_time.h

⟨portdat_time.h 596a⟩≡

```
⟨enum timermode 168a⟩

⟨type Txxx 165b⟩

⟨struct Timer 167d⟩

// was in clock.c
⟨struct Timers 168b⟩

// used only in arm/ for now
enum {
 Mhz = 1000 * 1000,
};

// <arch>/dat_time.h should defined Arch_HZ

#pragma varargck  type  "t"   long
#pragma varargck  type  "U"   uvlong
```

E.14.6 time/alarm.c

⟨alarm.c 596b⟩≡

```
⟨kernel basic includes 23m⟩

⟨global alarms 175f⟩
⟨global alarmr 177b⟩

⟨kernel process alarmkproc 177c⟩

⟨function checkalarms 177e⟩

⟨function procalarm 176b⟩
```

E.14.7 time/portclock.c

⟨portclock.c 597a⟩≡

```
⟨kernel basic includes 23m⟩

#include "io.h"

#include <ureg.h>

enum {
    ⟨constant Maxtimerloops 172d⟩
};

⟨global timers 168e⟩
⟨global timersinited 195b⟩

⟨clock.c statistics 517j⟩

⟨function tadd 168f⟩

⟨function tdel 169d⟩

⟨function timeradd 170a⟩

⟨function timerdel 170d⟩

⟨clock callback hzclock 159e⟩

⟨interrupt callback timerintr 171d⟩

⟨function timersinit 195c⟩

⟨function addclock0link 172f⟩

⟨function tk2ms 165f⟩

⟨function ms2tk 166a⟩
```

E.14.8 processes/pgrp.c

⟨pgrp.c 597b⟩≡

```
// Process group, and Namespace.

⟨kernel basic includes 23m⟩

enum {
    ⟨constant Whinesecs 77a⟩
};

⟨global pgrpid 250c⟩
⟨global mountid 267b⟩

⟨function pgrpnote 473a⟩

⟨constructor newpgrp 250d⟩

⟨constructor newrgrp 360d⟩

⟨destructor closergrp 360e⟩
```

⟨*destructor* `closepgrp` 250e⟩

⟨*function* `pgrpinsert` 267e⟩

⟨*function* `pgrpcpy` 268a⟩

⟨*function* `dupfgrp` 209d⟩

⟨*function* `closefgrp` 210b⟩

⟨*function* `forceclosefgrp` 226c⟩

⟨*constructor* `newmount` 248c⟩

⟨*destructor* `mountfree` 249b⟩

⟨*function* `resrcwait` 76c⟩

E.14.9 processes/proc.c

⟨`Procctl` *cases* 598a⟩+≡ (364c) ◁379c

```
Proc_exitbig,
```

⟨`procctl()` `Proc_exitbig` *case* 598b⟩≡ (364e)

```
case Proc_exitbig:
    arch_spllo();
    pexit("Killed: Insufficient physical memory", true);
    // Fallthrough
```

⟨*function* `killbig` 598c⟩≡ (599c)

```
void
killbig(char *why)
{
    int i;
    Segment *s;
    ulong l, max;
    Proc *p, *ep, *kp;

    max = 0;
    kp = 0;
    ep = procalloc.arena+conf.nproc;
    for(p = procalloc.arena; p < ep; p++) {
        if(p->state == Dead || p->kp)
            continue;
        l = 0;
        for(i=1; i<NSEG; i++) {
            s = p->seg[i];
            if(s != 0)
                l += s->top - s->base;
        }
        if(l > max && ((p->procmode&0222) || strcmp(eve, p->user)!=0)) {
            kp = p;
            max = l;
        }
    }

    print("%lud: %s killed: %s\n", kp->pid, kp->text, why);
    for(p = procalloc.arena; p < ep; p++) {
```

```
        if(p->state == Dead || p->kp)
            continue;
        if(p != kp && p->seg[BSEG] && p->seg[BSEG] == kp->seg[BSEG])
            p->procctl = Proc_exitbig;
    }
    kp->procctl = Proc_exitbig;
    for(i = 0; i < NSEG; i++) {
        s = kp->seg[i];
        if(s != 0 && canqlock(&s->lk)) {
            mfreeseg(s, s->base, (s->top - s->base)/BY2PG);
            qunlock(&s->lk);
        }
    }
}
```

⟨Proc *scheduling fields* 599a⟩+≡ (29c) ◁433d

```
bool preempted;  /* true if this process hasn't finished the interrupt
       *  that last preempted it
       */
```

⟨*function* preempt 599b⟩≡ (599c)

```
/*
 *  here at the end of non-clock interrupts to see if we should preempt the
 *  current process.
 */
void
preempt(void)
{
    if(up && up->state == Running)
      if(up->preempted == false)
        if(anyhigher())
          if(!active.exiting){
              cpu->readied = nil;   /* avoid cooperative scheduling */
              up->preempted = true;
              sched();
              arch_splhi(); // still in interrupt context
              up->preempted = false;
          }
    return;
}
```

⟨proc.c 599c⟩≡

```
⟨kernel basic includes 23m⟩
#include    <trace.h>

//*****************************************************************************
// Globals
//*****************************************************************************

⟨global runq 146g⟩
⟨global runveq 147b⟩

⟨global procalloc 73e⟩

⟨global nrdy 147d⟩
⟨global noteidalloc 334e⟩

⟨proc.c statistics 517i⟩

⟨global pidalloc 73c⟩
```

```
enum
{
    ⟨constant Schedagain 161h⟩
    ⟨constant Scaling 161g⟩
};

⟨global statename 30d⟩

⟨proc.c forward decl ??⟩

//*****************************************************************************
// Error management
//*****************************************************************************

// see also waserror() and poperror() macro in portdat_processes.h

⟨function error 520e⟩

⟨function nexterror 521c⟩

⟨function exhausted 521a⟩

//*****************************************************************************
// Functions
//*****************************************************************************

⟨function schedinit 149g⟩

⟨function sched 148c⟩

⟨function anyready 147c⟩

⟨function anyhigher 159c⟩

⟨function hzsched 159b⟩

⟨function preempt 599b⟩

⟨function updatecpu 161j⟩

⟨function reprioritize 162⟩

⟨function queueproc 147e⟩

⟨function dequeueproc 147f⟩

⟨function ready 155c⟩

⟨function yield 155a⟩

⟨global balancetime 164a⟩

⟨function rebalance 163b⟩

⟨function runproc 150⟩

⟨function canpage 142a⟩

⟨function noprocpanic 76b⟩
```

⟨*constructor* newproc 75b⟩

⟨*function* procwired 160e⟩

⟨*function* procpriority 145e⟩

⟨*function* procinit 197a⟩

⟨*function* sleep 156i⟩

⟨*function* tfn 175b⟩

⟨*function* twakeup 175c⟩

⟨*function* tsleep 174c⟩

⟨*function* wakeup 158c⟩

⟨*function* postnote 339⟩

⟨*struct* Broken 362e⟩
⟨*global* broken 363a⟩

⟨*function* addbroken 363c⟩

⟨*function* unbreak 363d⟩

⟨*function* freebroken 363e⟩

⟨*function* pexit 92d⟩

⟨*function* haswaitq 91e⟩

⟨*function* pwait 91a⟩

⟨*function* proctab 467b⟩

⟨*function* dumpaproc 503a⟩

⟨*function* procdump 502e⟩

⟨*function* procflushseg 353d⟩

⟨*function* scheddump 502d⟩

⟨*function* kproc 96d⟩

⟨*function* procctl 364e⟩

⟨*function* killbig 598c⟩

⟨*function* renameuser 77e⟩

⟨*function* accounttime 513a⟩

⟨*function* pidhash 74c⟩

⟨*function* pidunhash 74d⟩

⟨*function* procindex 74e⟩

E.14.10 time/tod.c

⟨tod.c 602⟩≡

```
// TOD: Time Of Day.

⟨kernel basic includes 23m⟩

⟨tod.c forward decl ??⟩

/*
 * Compute nanosecond epoch time from the fastest ticking clock
 * on the system.  Converting the time to nanoseconds requires
 * the following formula
 *
 *  t = (((1000000000<<31)/f)*ticks)>>31
 *
 *  where
 *
 *  'f'     is the clock frequency
 *  'ticks'     are clock ticks
 *
 *  to avoid too much calculation in todget(), we calculate
 *
 *  mult = (1000000000<<32)/f
 *
 *  each time f is set.  f is normally set by a user level
 *  program writing to /dev/fastclock.  arch_mul64fract will then
 *  take that fractional multiplier and a 64 bit integer and
 *  return the resulting integer product.
 *
 *  We assume that the cpu's of a multiprocessor are synchronized.
 *  This assumption needs to be questioned with each new architecture.
 */

/* frequency of the tod clock */
#define TODFREQ     1000000000ULL
#define MicroFREQ   1000000ULL

⟨struct TOD 178a⟩
⟨global tod 178b⟩

⟨function todinit 178e⟩

⟨function todsetfreq 179a⟩

⟨function todset 179b⟩

⟨function todget 179c⟩

⟨function todfix 180⟩

⟨function seconds 181a⟩

⟨function fastticks2us 181c⟩

⟨function ns2fastticks 181d⟩
```

⟨*function* `mk64fract` 182a⟩

E.14.11 processes/sysproc.c

⟨`sysproc.c` 603a⟩≡
⟨*kernel basic includes* 23m⟩

```
#include    <tos.h>
#include    <a.out.h>
```

⟨*enum* `rfork` 78a⟩

⟨*sysproc.c forward decl* **??**⟩

⟨*syscall nop* 26e⟩

⟨*syscall rfork* 78b⟩

⟨*function* `l2be` 85b⟩

⟨*syscall exec* 83i⟩

⟨*function* `shargs` 386a⟩

⟨*syscall sleep* 174a⟩

⟨*syscall alarm* 175g⟩

⟨*syscall exits* 92a⟩

⟨*syscall await* 90e⟩

⟨*function* `werrstr` 522⟩

⟨*function* `generrstr` 521f⟩

⟨*syscall errstr* 521e⟩

⟨*syscall notify* 336b⟩

⟨*syscall noted* 338a⟩

⟨*syscall segbrk* 350a⟩

⟨*syscall segattach* 350b⟩

⟨*syscall segdetach* 352b⟩

⟨*syscall segfree* 353a⟩

⟨*syscall brk* 69c⟩

⟨*syscall rendezvous* 361b⟩

E.14.12 processes/syssema.c

⟨`syssema.c` 603b⟩≡

⟨*kernel basic includes* 23m⟩

```
/*
 * The implementation of semaphores is complicated by needing
 * to avoid rescheduling in syssemrelease, so that it is safe
 * to call from real-time processes.  This means syssemrelease
 * cannot acquire any qlocks, only spin locks.
 *
 * Semacquire and semrelease must both manipulate the semaphore
 * wait list.  Lock-free linked lists only exist in theory, not
 * in practice, so the wait list is protected by a spin lock.
 *
 * The semaphore value *addr is stored in user memory, so it
 * cannot be read or written while holding spin locks.
 *
 * Thus, we can access the list only when holding the lock, and
 * we can access the semaphore only when not holding the lock.
 * This makes things interesting.  Note that sleep's condition function
 * is called while holding two locks - r and up->rlock - so it cannot
 * access the semaphore value either.
 *
 * An acquirer announces its intention to try for the semaphore
 * by putting a Sema structure onto the wait list and then
 * setting Sema.waiting.  After one last check of semaphore,
 * the acquirer sleeps until Sema.waiting==0.  A releaser of n
 * must wake up n acquirers who have Sema.waiting set.  It does
 * this by clearing Sema.waiting and then calling wakeup.
 *
 * There are three interesting races here.

 * The first is that in this particular sleep/wakeup usage, a single
 * wakeup can rouse a process from two consecutive sleeps!
 * The ordering is:
 *
 *  (a) set Sema.waiting = 1
 *  (a) call sleep
 *  (b) set Sema.waiting = 0
 *  (a) check Sema.waiting inside sleep, return w/o sleeping
 *  (a) try for semaphore, fail
 *  (a) set Sema.waiting = 1
 *  (a) call sleep
 *  (b) call wakeup(a)
 *  (a) wake up again
 *
 * This is okay - semacquire will just go around the loop
 * again.  It does mean that at the top of the for(;;) loop in
 * semacquire, phore.waiting might already be set to 1.
 *
 * The second is that a releaser might wake an acquirer who is
 * interrupted before he can acquire the lock.  Since
 * release(n) issues only n wakeup calls -- only n can be used
 * anyway -- if the interrupted process is not going to use his
 * wakeup call he must pass it on to another acquirer.
 *
 * The third race is similar to the second but more subtle.  An
 * acquirer sets waiting=1 and then does a final canacquire()
 * before going to sleep.  The opposite order would result in
 * missing wakeups that happen between canacquire and
 * waiting=1.  (In fact, the whole point of Sema.waiting is to
 * avoid missing wakeups between canacquire() and sleep().) But
```

```
 * there can be spurious wakeups between a successful
 * canacquire() and the following semdequeue().  This wakeup is
 * not useful to the acquirer, since he has already acquired
 * the semaphore.  Like in the previous case, though, the
 * acquirer must pass the wakeup call along.
 *
 * This is all rather subtle.  The code below has been verified
 * with the spin model /sys/src/9/port/semaphore.p.  The
 * original code anticipated the second race but not the first
 * or third, which were caught only with spin.  The first race
 * is mentioned in /sys/doc/sleep.ps, but I'd forgotten about it.
 * It was lucky that my abstract model of sleep/wakeup still managed
 * to preserve that behavior.
 *
 * I remain slightly concerned about memory coherence
 * outside of locks.  The spin model does not take
 * queued processor writes into account so we have to
 * think hard.  The only variables accessed outside locks
 * are the semaphore value itself and the boolean flag
 * Sema.waiting.  The value is only accessed with cmpswap,
 * whose job description includes doing the right thing as
 * far as memory coherence across processors.  That leaves
 * Sema.waiting.  To handle it, we call coherence() before each
 * read and after each write.       - rsc
 */
```

⟨*function* semqueue 356c⟩

⟨*function* semdequeue 356d⟩

⟨*function* semwakeup 356e⟩

⟨*function* semrelease 357a⟩

⟨*function* canacquire 357b⟩

⟨*function* semawoke 357c⟩

⟨*function* semacquire 357d⟩

⟨*function* tsemacquire 358⟩

⟨*syscall semacquire* 359a⟩

⟨*syscall tsemacquire* 359b⟩

⟨*syscall semrelease* 359c⟩

E.15 security/

E.15.1 security/portfns_security.h

E.15.2 security/auth.c

⟨auth.c 605⟩≡
  ⟨*kernel basic includes* 23m⟩

```
#include    <authsrv.h>

//now in core/
//char  *eve;
//int iseve(void) { }
```

⟨*global* hostdomain 456d⟩

⟨*syscall fversion* 456e⟩

⟨*syscall fauth* 456f⟩

⟨*function* userwrite 457a⟩

⟨*function* hostownerwrite 77d⟩

⟨*function* hostdomainwrite 457b⟩

E.16 syscalls/

E.16.1 syscalls/systab.h

E.16.2 syscalls/syscallfmt.c

⟨syscallfmt.c 606a⟩≡

```
/*
 * Print functions for system call tracing.
 */
```

⟨*kernel basic includes* 23m⟩

```
#include "../port/systab.h"
```

⟨*function* fmtrwdata 375⟩

⟨*function* fmtuserstring 376a⟩

⟨*function* syscallfmt 370f⟩

⟨*function* sysretfmt 374⟩

E.16.3 syscalls/systab.c

⟨systab.c 606b⟩≡

⟨*kernel basic includes* 23m⟩

```
#include "../port/systab.h"
```

⟨*systab.c forward decl* ??⟩

```
//coupling: debuggers/acid/conf/syscall
//coupling: debuggers/acid/conf/truss
```

⟨*global* systab 26c⟩
⟨*global* nsyscall 26d⟩

⟨*global* sysstab ??⟩

E.17 ../include/arch/arm/

E.17.1 ../include/arch/arm/ureg.h

⟨arm/ureg.h 607a⟩≡
```
typedef struct Ureg Ureg;
⟨struct Ureg(arm) 20c⟩
```

E.18 arch/arm/

E.18.1 arch/arm/dat_arch.h

⟨arch/arm/dat_arch.h 607b⟩≡
```

⟨struct Soc(arm) 33a⟩

extern Soc soc;
```

E.18.2 arch/arm/arm.h

⟨arch/arm/arm.h 607c⟩≡
```
/*
 * ARM-specific definitions for armv6 (arm11), armv7 (cortex-a7 and -a8)
 * these are used in C and assembler
 */

/*
 * Program Status Registers
 */
⟨type PsrMode(arm) 21a⟩

⟨type PsrDisable(arm) 21b⟩

/*
 * Coprocessors
 */
⟨type Coprocessors(arm) 21d⟩

/*
 * Primary (CRn) CpSC registers.
 */
⟨type CpSC primary registers(arm) 21e⟩

// CpSC secondary registers

⟨type CpID secondary registers(arm) 405i⟩
⟨type CpID opcode2(arm) 405j⟩
⟨type CpID CpIDid op1(arm) 417f⟩

⟨type CpCONTROL opcode2(arm) 21f⟩
⟨type CpCONTROL CpMainctl(arm) 21g⟩
⟨type CpCONTROL CpAuxctl(arm) 409e⟩

⟨type CpTTB(arm) 406d⟩

⟨type CpFSR(arm) 108a⟩
```

⟨*type CpCACHE secondary registers(arm)* 401j⟩
⟨*type CpCACHE opcode2(arm)* 402a⟩

⟨*type CpTLB secondary registers(arm)* 115b⟩
⟨*type CpTLB opcode2(arm)* 115c⟩

⟨*type CpCLD secondary registers(arm)* 407g⟩
⟨*type CpCLD opcode2(arm)* 407h⟩

⟨*type* `CpTIMER`*(arm)* 407e⟩

⟨*type CpSPM secondary registers(arm)* 383f⟩
⟨*type CpSPM opcode2(arm)* 383g⟩

```
// For MCRR
```

⟨*type CpCACHERANGE opcode2(arm)* 401a⟩

```
// Misc
```

⟨*type CpTTB cache control bits(arm)* 406e⟩

```
/*
 * MMU page table entries.
 */
```
⟨*constant* `Mbz`*(arm)* 113e⟩

⟨*constant* `Fault`*(arm)* 113i⟩

⟨*type* `PageDirGranularity`*(arm)* 113d⟩
⟨*type* `PageTableGranularity`*(arm)* 113f⟩

⟨*type* `PageTableEntryAttribute1`*(arm)* 401c⟩

⟨*constant* `Dom0`*(arm)* 113j⟩

⟨*type* `PageTableEntryAttribute2`*(arm)* 406a⟩

⟨*type* `PageTableEntryAttribute3`*(arm)* 113h⟩

⟨*type* `DAC`*(arm)* 113g⟩

⟨*macro* `F`*(arm)* 114b⟩

⟨*macro* `AP`*(arm)* 114a⟩
⟨*macro* `L1AP`*(arm)* 113k⟩
```
/* L2AP differs between armv6 and armv7 -- see l2ap in arch*.c */
```
⟨*macro* `DAC`*(arm)* 114c⟩

⟨*constant* `HVECTORS`*(arm)* 21c⟩

E.18.3 arch/arm/arminstr.ha

⟨*arch/arm/arminstr.ha* 608⟩≡
```
/*
 * armv6/v7 machine assist, definitions
 *
 * loader uses R11 as scratch.
```

```
 */

⟨macro PADDR bis(arm) 25j⟩
⟨macro L1X bis(arm) 113c⟩

/*
 * new instructions
 */
⟨instruction ISB(arm) 401h⟩
⟨instruction DSB(arm) 401i⟩
⟨instruction BARRIERS(arm) 405f⟩

⟨instruction MCRR(arm) 401b⟩
⟨instruction MRRC(arm) 640b⟩

⟨instruction LDREX(arm) 415d⟩
⟨instruction STREX(arm) 415e⟩
⟨instruction CLREX(arm) 415f⟩

⟨instruction CPSID(arm) 417e⟩

⟨instruction CPUID(arm) 405k⟩
```

E.18.4 arch/arm/io.h

```
⟨arch/arm/io.h 609a⟩≡
  // could be in dat_interrupts.h?
  ⟨enum IRQ(arm) 110b⟩
  ⟨macro IRQDMA(arm) 713a⟩

  ⟨enum Dma(arm) 713b⟩

  ⟨enum Power(arm) 430c⟩

  // could be in dat_time.h?
  ⟨enum Clock(arm) 614b⟩
```

E.18.5 arch/arm/coproc.c

```
⟨enum _anon_(arm) 609b⟩≡                                        (611b)
  enum {
      /* alternates:
       *  0xe12fff1e  BX (R14); last e is R14
       *  0xe28ef000  B 0(R14); second e is R14 (ken)
       */
      RETinst = 0xe1a0f00e,       /* MOV R14, R15 */

      Opmask  = MASK(3),
      Regmask = MASK(4),
  };

⟨function setupcpop(arm) 609c⟩≡                                 (611b)
  // setup coprocessor operation
  static void
  setupcpop(ulong instr[2], ulong opcode, int cp, int op1, int crn, int crm,
      int op2)
  {
      ulong instrsz[2];
```

```
    op1 &= Opmask;
    op2 &= Opmask;
    crn &= Regmask;
    crm &= Regmask;
    cp  &= Regmask;
    instr[0] = opcode | op1 << 21 | crn << 16 | cp << 8 | op2 << 5 | crm;
    instr[1] = RETinst;

    cachedwbse(instr, sizeof instrsz);
    cacheiinv();
}
```

⟨*function* cprd*(arm)* 610a⟩≡ (611b)

```
ulong
cprd(int cp, int op1, int crn, int crm, int op2)
{
    int s, r;
    volatile ulong instr[2];
    Pufv fp;

    s = arch_splhi();
    /*
     * MRC.  return value will be in R0, which is convenient.
     * Rt will be R0.
     */
    // MRC 0x e(Mxx) e(Always) 0 (MRC) .... 10 (part of Mxxx opcode)
    setupcpop(instr, 0xee100010, cp, op1, crn, crm, op2);
    fp = (Pufv)instr;
    r = fp();
    arch_splx(s);
    return r;
}
```

⟨*function* cpwr*(arm)* 610b⟩≡ (611b)

```
void
cpwr(int cp, int op1, int crn, int crm, int op2, ulong val)
{
    int s;
    volatile ulong instr[2];
    Pvfu fp;

    s = arch_splhi();
    // MCR 0x e(Mxx) e(Always) 0 (MCR) .... 10 (part of Mxxx opcode)
    setupcpop(instr, 0xee000010, cp, op1, crn, crm, op2); /* MCR, Rt is R0 */
    fp = (Pvfu)instr;
    fp(val);
    arch_coherence();
    arch_splx(s);
}
```

⟨*function* cprdsc*(arm)* 610c⟩≡ (611b)

```
ulong
cprdsc(int op1, int crn, int crm, int op2)
{
    return cprd(CpSC, op1, crn, crm, op2);
}
```

⟨*function* cpwrsc*(arm)* 611a⟩≡ (611b)

```
void
cpwrsc(int op1, int crn, int crm, int op2, ulong val)
{
    cpwr(CpSC, op1, crn, crm, op2, val);
}
```

⟨arch/arm/coproc.c 611b⟩≡

```
/*
 * ARM co-processors
 * mainly to cope with ARM hard-wiring register numbers into instructions.
 *
 * CP15 (system control) is the one that gets used the most in practice.
 * these routines must be callable from KZERO space or the 0 segment.
 */
⟨kernel basic includes 23m⟩

#include "io.h"
#include "arm.h"

⟨enum _anon_(arm) 609b⟩

// pointer to ulong function void
typedef ulong (*Pufv)(void);
// pointer to void function ulong
typedef void  (*Pvfu)(ulong);

⟨function setupcpop(arm) 609c⟩

⟨function cprd(arm) 610a⟩

⟨function cpwr(arm) 610b⟩

⟨function cprdsc(arm) 610c⟩

⟨function cpwrsc(arm) 611a⟩

/* floating point */

⟨function setupfpctlop(arm) 391f⟩

⟨function fprd(arm) 391g⟩

⟨function fpwr(arm) 392a⟩

⟨function setupfpop(arm) 392b⟩

⟨function fpsavereg(arm) 393a⟩

⟨function fprestreg(arm) 392c⟩
```

E.18.6 arch/arm/vcore.c

⟨*enum* _anon_*(*(arch/arm/vcore.c)(arm)*)* 611c⟩≡ (615d)

```
enum {
    Read        = 0x00>>2,
    Write       = 0x00>>2,
    Peek        = 0x10>>2,
    Sender      = 0x14>>2,
```

```
    Status      = 0x18>>2,
        Full        = 1<<31,
        Empty       = 1<<30,
    Config      = 0x1C>>2,

    NRegs       = 0x20>>2,
};
```

⟨*enum* _anon_*(*(arch/arm/vcore.c)2(arm)*)* 612a⟩≡ (615d)

```
enum {
    ChanFb      = 1,
    ChanProps   = 8,
    ChanMask    = 0xF,
};
```

⟨*enum* _anon_*(*(arch/arm/vcore.c)3(arm)*)* 612b⟩≡ (615d)

```
enum {
    Req         = 0x0,
    RspOk       = 0x80000000,
    TagResp     = 1<<31,
};
```

⟨*enum* _anon_*(*(arch/arm/vcore.c)4(arm)*)* 612c⟩≡ (615d)

```
enum {
    TagGetfwrev = 0x00000001,
    TagGetrev   = 0x00010002,

    TagGetmac   = 0x00010003,
    TagGetram   = 0x00010005,

    TagGetpower = 0x00020001,
    TagSetpower = 0x00028001,
        Powerwait   = 1<<1,

    TagGetclkspd= 0x00030002,
    TagGetclkmax= 0x00030004,
    TagSetclkspd= 0x00038002,

    TagGettemp  = 0x00030006,

    TagFballoc  = 0x00040001,
    TagFbfree   = 0x00048001,
    TagFbblank  = 0x00040002,

    TagGetres   = 0x00040003,
    TagSetres   = 0x00048003,
    TagGetvres  = 0x00040004,
    TagSetvres  = 0x00048004,
    TagGetdepth = 0x00040005,
    TagSetdepth = 0x00048005,
    TagGetrgb   = 0x00044006,
    TagSetrgb   = 0x00048006,
};
```

⟨*struct* Prophdr*(arm)* 612d⟩≡ (615d)

```
struct Prophdr {
    u32int  len;
    u32int  req;
    u32int  tag;
    u32int  tagbuflen;
```

```
    u32int  taglen;
    u32int  data[1];
};
```

⟨*function* vcwrite*(arm)* 613a⟩≡ (615d)

```
static void
vcwrite(uint chan, int val)
{
    u32int *r;

    r = (u32int*)MAILBOX + NRegs;
    val &= ~ChanMask;
    while(r[Status] & Full)
        ;
    arch_coherence();
    r[Write] = val | chan;
}
```

⟨*function* vcread*(arm)* 613b⟩≡ (615d)

```
static int
vcread(uint chan)
{
    u32int *r;
    int x;

    r = (u32int*)MAILBOX;
    do{
        while(r[Status]&Empty)
            ;
        arch_coherence();
        x = r[Read];
    }while((x&ChanMask) != chan);
    return x & ~ChanMask;
}
```

⟨*function* vcreq*(arm)* 613c⟩≡ (615d)

```
/*
 * Property interface
 */

static int
vcreq(int tag, void *buf, int vallen, int rsplen)
{
    uintptr r;
    int n;
    Prophdr *prop = (Prophdr*)(VCBUFFER);
    uintptr aprop;
    static bool busaddr = true;

    if(rsplen < vallen)
        rsplen = vallen;
    rsplen = (rsplen+3) & ~3;

    n = sizeof(Prophdr) + rsplen + 8;
    memset(prop, 0, n);
    prop->len = n;
    prop->req = Req;
    prop->tag = tag;
    prop->tagbuflen = rsplen;
    prop->taglen = vallen;
```

```
    if(vallen > 0)
        memmove(prop->data, buf, vallen);
    cachedwbinvse(prop, prop->len);
    for(;;){
        aprop = busaddr? dmaaddr(prop) : PTR2UINT(prop);
        vcwrite(ChanProps, aprop);
        r = vcread(ChanProps);
        if(r == aprop)
            break;
        if(!busaddr)
            return -1;
        busaddr = false;
    }
    if(prop->req == RspOk &&
       prop->tag == tag &&
       (prop->taglen&TagResp)) {
        if((n = prop->taglen & ~TagResp) < rsplen)
            rsplen = n;
        memmove(buf, prop->data, rsplen);
    }else
        rsplen = -1;

    return rsplen;
}
```

⟨*function* getethermac*(arm)* 614a⟩≡ (615d)

```
/*
 * Get ethernet address (as hex string)
 *   [not reentrant]
 */
char *
getethermac(void)
{
    uchar ea[8];
    char *p;
    int i;
    static char buf[16];

    memset(ea, 0, sizeof ea);
    vcreq(TagGetmac, ea, 0, sizeof ea);
    p = buf;
    for(i = 0; i < 6; i++)
        p += sprint(p, "%.2x", ea[i]);
    return buf;
}
```

⟨*enum* Clock*(arm)* 614b⟩≡ (609a)

```
enum {
    ClkEmmc     = 1,
    ClkUart,
    ClkArm,
    ClkCore,
    ClkV3d,
    ClkH264,
    ClkIsp,
    ClkSdram,
    ClkPixel,
    ClkPwm,
};
```

⟨*function* getclkrate*(arm)* 615a⟩≡ (615d)

```
/*
 * Get clock rate
 */
ulong
getclkrate(int clkid)
{
    u32int buf[2];

    buf[0] = clkid;
    if(vcreq(TagGetclkspd, buf, sizeof(buf[0]), sizeof(buf)) != sizeof buf)
        return 0;
    return buf[1];
}
```

⟨*function* setclkrate*(arm)* 615b⟩≡ (615d)

```
/*
 * Set clock rate to hz (or max speed if hz == 0)
 */
void
setclkrate(int clkid, ulong hz)
{
    u32int buf[2];

    buf[0] = clkid;
    if(hz != 0)
        buf[1] = hz;
    else if(vcreq(TagGetclkmax, buf, sizeof(buf[0]), sizeof(buf)) != sizeof buf)
        return;
    vcreq(TagSetclkspd, buf, sizeof(buf), sizeof(buf));
}
```

⟨*function* getcputemp*(arm)* 615c⟩≡ (615d)

```
/*
 * Get cpu temperature
 */
uint
getcputemp(void)
{
    u32int buf[2];

    buf[0] = 0;
    if(vcreq(TagGettemp, buf, sizeof(buf[0]), sizeof buf) != sizeof buf)
        return 0;
    return buf[1];
}
```

⟨arch/arm/vcore.c 615d⟩≡

```
⟨kernel basic includes 23m⟩

/*
 * Mailbox interface with videocore gpu
 */
⟨constant MAILBOX(arm) 22k⟩

typedef struct Fbinfo Fbinfo;
typedef struct Prophdr Prophdr;

⟨enum _anon_((arch/arm/vcore.c)(arm)) 611c⟩
⟨enum _anon_((arch/arm/vcore.c)2(arm)) 612a⟩
```

⟨*enum* _anon_*(*(arch/arm/vcore.c)3(arm)*)* 612b⟩

⟨*enum* _anon_*(*(arch/arm/vcore.c)4(arm)*)* 612c⟩

⟨*struct* Fbinfo*(arm)* 22l⟩

⟨*struct* Prophdr*(arm)* 612d⟩

⟨*function* vcwrite*(arm)* 613a⟩

⟨*function* vcread*(arm)* 613b⟩

⟨*function* vcreq*(arm)* 613c⟩

⟨*function* fbdefault*(arm)* 282c⟩

⟨*function* fbinit*(arm)* 282a⟩

⟨*function* fbblank*(arm)* 283a⟩

⟨*function* setpower*(arm)* 430d⟩

⟨*function* getpower*(arm)* 431a⟩

⟨*function* getethermac*(arm)* 614a⟩

⟨*function* getboardrev*(arm)* 200f⟩

⟨*function* getfirmware*(arm)* 200g⟩

⟨*function* getramsize*(arm)* 191d⟩

⟨*function* getclkrate*(arm)* 615a⟩

⟨*function* setclkrate*(arm)* 615b⟩

⟨*function* getcputemp*(arm)* 615c⟩

E.19 concurrency/arm/

E.19.1 concurrency/arm/spl.s

⟨concurrency/arm/spl.s 616⟩≡

```
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"
```

⟨*function* arch_splhi*(arm)* 34a⟩

⟨*function* arch_spllo*(arm)* 34b⟩

⟨*function* arch_spllx*(arm)* 34c⟩

```
TEXT spldone(SB), 1, $0             /* end marker for devkprof.c */
    RET
```

⟨*function* `arch_islo`*(arm)* 34d⟩

⟨*function* `arm_arch_coherence`*(arm)* 417c⟩

```
// FIQ
```
⟨*function* `splfhi`*(arm)* 430b⟩

E.19.2 concurrency/arm/tas_raspi1.s

⟨`concurrency/arm/tas_raspi1.s` 617a⟩≡
```
TEXT    _tas(SB), $-4
```
⟨*function* `arch_tas`*(raspberry pi1)(arm)* 35a⟩

E.19.3 concurrency/arm/tas_raspi2.s

⟨`concurrency/arm/tas_raspi2.s` 617b⟩≡
```
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"
```

⟨*instruction DMB(arm)* 415g⟩

```
/* tas/cas strex debugging limits; started at 10000 */
#define MAXSC 100000

TEXT    _tas(SB), $-4           /* _tas(ulong *) */
```
⟨*function* `arch_tas`*(raspberry pi2)(arm)* 416a⟩

E.19.4 concurrency/arm/concurrency_raspi1.c

⟨*function* `ainc`*(arm)* 617c⟩≡ (618a)
```
// overwrite libc!
int
ainc(int *p)
{
    int s, v;

    s = arch_splhi();
    v = ++*p;
    arch_splx(s);
    return v;
}
```

⟨*function* `adec`*(arm)* 617d⟩≡ (618a)
```
int
adec(int *p)
{
    int s, v;

    s = arch_splhi();
    v = --*p;
    arch_splx(s);
    return v;
}
```

⟨concurrency/arm/concurrency_raspi1.c 618a⟩≡

```
/*
 * atomic ops
 * make sure that we don't drag in the C library versions
 */
```

⟨*kernel basic includes* 23m⟩

⟨*function* arch_xdec*(raspberry pi1)(arm)* 35c⟩

⟨*function* arch_xinc*(raspberry pi1)(arm)* 35b⟩

⟨*function* ainc*(arm)* 617c⟩

⟨*function* adec*(arm)* 617d⟩

⟨*function* cas32*(arm)* 356b⟩

⟨*function* arch_cmpswap*(raspberry pi1)(arm)* 356a⟩

E.19.5 concurrency/arm/concurrency_raspi2.c

⟨concurrency/arm/concurrency_raspi2.c 618b⟩≡

⟨*kernel basic includes* 23m⟩

```
// in libc/arm/atom.s (use LDREX, STREX special instructions)
extern long _xdec(long *p);
extern long _xinc(long *p);

long
arch_xdec(long *p)
{
  return _xdec(p);
}

void
arch_xinc(long *p)
{
  _xinc(p);
}

int
arch_cmpswap(long *addr, long old, long new)
{
    return cas((ulong*)addr, old, new);
}
```

E.20 core/arm/

E.20.1 core/arm/dat.h

⟨*constant* STAGESIZE*(arm)* 618c⟩≡ (619a)

```
// was a default in portdat.h
#define STAGESIZE 64 // for struct Uart
```

⟨core/arm/dat.h 619a⟩≡

```
// pad's memory pointers (used in Lock so must be early)
⟨pad memory pointer types(arm) 24c⟩
//TODO: need to update port/ code to use the new pointer types
typedef void* virt_addr3;
typedef void* kern_addr3;

#include "dat_forward.h"
#include "../port/portdat_forward.h"
⟨constant STAGESIZE(arm) 618c⟩

⟨dat.h includes(arm) 24a⟩

// DO NOT switch those declarations. 5c allocates R10
// for the first 'extern register' declaration seen in a file
// and R9 for the second one. See Compiler.nw.
// Then mem.h relies on this to define the UP and CPU macros used
// inside assembly code.
⟨global cpu(arm) 28f⟩
⟨global up(arm) 30e⟩

// in main.c (used in mmu.c)
extern ulong memsize;

⟨constant KMESGSIZE(arm) 507c⟩

// TODO: put that in portdat.h or in netif.h?
#pragma varargck  type  "I" uchar*
#pragma varargck  type  "V" uchar*
#pragma varargck  type  "E" uchar*
#pragma varargck  type  "M" uchar*
```

E.20.2 core/arm/dat_core.h

⟨core/arm/dat_core.h 619b⟩≡

```
⟨type PTE(arm) 114g⟩

⟨struct Arch_Cpu(arm) 28e⟩
```

E.20.3 core/arm/dat_forward.h

⟨core/arm/dat_forward.h 619c⟩≡

```
// could be in portdat_forward.h
typedef struct Arch_FPsave  Arch_FPsave;
typedef struct Ureg Ureg;
#pragma incomplete Ureg

typedef struct Soc Soc;
```

E.20.4 core/arm/fns.h

⟨core/arm/fns.h 619d⟩≡

```
⟨fns.h includes(arm) 24b⟩

// not classified yet
```

```
extern void arch_cpuidprint(void);

// different signatures in archs or macro so cant factorize in portdat_xxx.h
extern void  arch_touser(uintptr);
extern int   arch_cmpswap(long*, long, long);
⟨macro getpgcolor(arm) 423d⟩
⟨macro arch_cycles(arm) 383e⟩
⟨macro arch_countpagerefs(arm) 427a⟩
⟨macro arch_intrenable(arm) 111c⟩

extern int arch__notify(Ureg*);
extern void arch__kexit(Ureg*);

extern void dumpregs(Ureg*);


extern void mmuinit1(void*);
extern void mmuinvalidate(void);
extern void mmuinvalidateaddr(u32int);


extern void irqenable(int, void (*)(Ureg*, void*), void*);

extern void intrcpushutdown(void);
extern void intrshutdown(void);
extern void intrsoff(void);


extern void archreboot(void);
extern void archreset(void);

extern void clockinit(void);
extern void clockshutdown(void);


extern int cas32(void*, u32int, u32int);
extern int cas(ulong*, ulong, ulong);

extern char *cputype2name(char *buf, int size);

extern void armtimerset(int);

extern void cachedwb(void);
extern void cachedwbinv(void);
extern void cachedinvse(void*, int);
extern void cachedwbse(void*, int);
extern void cachedwbinvse(void*, int);
extern void cacheiinv(void);
extern void cacheuwbinv(void);


// coproc.c
extern ulong cprd(int cp, int op1, int crn, int crm, int op2);
extern ulong cprdsc(int op1, int crn, int crm, int op2);
extern void cpwr(int cp, int op1, int crn, int crm, int op2, ulong val);
extern void cpwrsc(int op1, int crn, int crm, int op2, ulong val);

// dma.c
extern uintptr dmaaddr(void *va);
extern void dmastart(int, int, int, void*, void*, int);
```

```
extern int dmawait(int);

// vcore.c
extern int fbblank(int);
extern void* fbinit(bool, int*, int*, int*);


extern void fpon(void);
extern ulong fprd(int fpreg);
extern void fprestreg(int fpreg, uvlong val);
extern void fpsave(Arch_FPsave *);
extern ulong fpsavereg(int fpreg, uvlong *fpp);
extern void fpwr(int fpreg, ulong val);

extern u32int fsrget(void);
extern u32int farget(void);
extern u32int ifsrget(void);

// vcore.c
extern uint getboardrev(void);
extern ulong getclkrate(int);
extern uint getcputemp(void);
extern char *getethermac(void);
extern uint getfirmware(void);
extern void getramsize(Confmem*);

extern int getncpus(void);
extern int getpower(int);


extern void gpiosel(uint, int);
extern void gpiopullup(uint);
extern void gpiopulloff(uint);
extern void gpiopulldown(uint);

extern void i2cread(uint, void*, int);
extern void i2cwrite(uint, void*, int);


extern void okay(int);

extern int l2ap(int);
extern void l2cacheuwbinv(void);
```

⟨*macro* sdfree*(arm)* ??⟩
⟨*macro* sdmalloc*(arm)* ??⟩

```
extern void setclkrate(int, ulong);
extern void setpower(int, int);
extern void setr13(int, u32int*);

extern void spirw(uint, void*, int);
extern int splfhi(void);
extern int splflo(void);

extern int startcpus(uint);
extern void stopcpu(uint);


extern void uartconsinit(void);
```

```
extern void vectors(void);
extern void vtable(void);

extern void wdogoff(void);

/*
 * floating point emulation
 */
extern int fpiarm(Ureg*);
extern int fpudevprocio(Proc*, void*, long, uintptr, int);
extern void fpuinit(void);
extern void fpunoted(void);
extern void fpunotify(Ureg*);
extern void fpuprocrestore(Proc*);
extern void fpuprocsave(Proc*);
extern void fpusysprocsetup(Proc*);
extern void fpusysrfork(Ureg*);
extern void fpusysrforkchild(Proc*, Ureg*, Proc*);
extern int fpuemu(Ureg*);

/*
 * Miscellaneous machine dependent stuff.
 */
extern char* getenv(char*, char*, int);

uintptr mmukmap(uintptr, uintptr, usize);
uintptr mmukunmap(uintptr, uintptr, usize);
```

⟨*macro* PTR2UINT*(arm)* 25n⟩
⟨*macro* UINT2PTR*(arm)* 25o⟩

⟨*macro* KADDR*(arm)* 25m⟩
⟨*macro* PADDR*(arm)* 25i⟩

E.21 buses/arm/

E.22 devices/

E.22.1 devices/keyboard/arm/kbd.c

⟨devices/keyboard/arm/kbd.c 622a⟩≡
⟨*kernel basic includes* 23m⟩

```
// In the Raspberry Pi, the keyboard is handled by usbd
// with help from /dev/kbin (which then calls functions from portkbd
// which calls arch_setleds below).
```

⟨*function* arch_setleds*(arm)* 277a⟩

E.22.2 devices/mouse/arm/mouse.c

⟨devices/mouse/arm/mouse.c 622b⟩≡
⟨*kernel basic includes* 23m⟩

```
// In the Raspberry Pi, the mouse is handled by usbd
// with help from /dev/mousein.

// empty callbacks for portmouse.c

void arch_ps2mouse(void) { }
void arch_setaccelerated(void) { }
void arch_setlinear(void) { }
void arch_setres(int n) { USED(n); }
void arch_setintellimouse(void) { }
void arch_resetmouse(void) { }
```

E.22.3 devices/sys/arm/devarch.c

⟨*enum* _anon_*(*(devices/sys/arm/devarch.c)(arm)*)* 623a⟩≡ (626c)

```
enum {
    Qdir = 0,
    Qbase,

    Qmax = 16,
};
```

⟨*global* readfn*(arm)* 623b⟩≡ (626c)

```
static Rdwrfn *readfn[Qmax];
```

⟨*global* writefn*(arm)* 623c⟩≡ (626c)

```
static Rdwrfn *writefn[Qmax];
```

⟨*global* archdir*(arm)* 623d⟩≡ (626c)

```
static Dirtab archdir[Qmax] = {
    ".",        { Qdir, 0, QTDIR }, 0,  0555,
};
```

⟨*global* archwlock*(arm)* 623e⟩≡ (626c)

```
Lock archwlock; /* the lock is only for changing archdir */
```

⟨*global* narchdir*(arm)* 623f⟩≡ (626c)

```
int narchdir = Qbase;
```

⟨*function* addarchfile*(arm)* 623g⟩≡ (626c)

```
/*
 * Add a file to the #P listing.  Once added, you can't delete it.
 * You can't add a file with the same name as one already there,
 * and you get a pointer to the Dirtab entry so you can do things
 * like change the Qid version.  Changing the Qid path is disallowed.
 */
Dirtab*
addarchfile(char *name, int perm, Rdwrfn *rdfn, Rdwrfn *wrfn)
{
    int i;
    Dirtab d;
    Dirtab *dp;

    memset(&d, 0, sizeof d);
    strcpy(d.name, name);
    d.perm = perm;

    lock(&archwlock);
    if(narchdir >= Qmax){
```

```
        unlock(&archwlock);
        return nil;
    }

    for(i=0; i<narchdir; i++)
        if(strcmp(archdir[i].name, name) == 0){
            unlock(&archwlock);
            return nil;
        }

    d.qid.path = narchdir;
    archdir[narchdir] = d;
    readfn[narchdir] = rdfn;
    writefn[narchdir] = wrfn;
    dp = &archdir[narchdir++];
    unlock(&archwlock);

    return dp;
}
```

⟨*function* `archattach`*(arm)* 624a⟩≡ (626c)

```
static Chan*
archattach(char* spec)
{
    return devattach('P', spec);
}
```

⟨*function* `archwalk`*(arm)* 624b⟩≡ (626c)

```
Walkqid*
archwalk(Chan* c, Chan *nc, char** name, int nname)
{
    return devwalk(c, nc, name, nname, archdir, narchdir, devgen);
}
```

⟨*function* `archstat`*(arm)* 624c⟩≡ (626c)

```
static int
archstat(Chan* c, uchar* dp, int n)
{
    return devstat(c, dp, n, archdir, narchdir, devgen);
}
```

⟨*function* `archopen`*(arm)* 624d⟩≡ (626c)

```
static Chan*
archopen(Chan* c, int omode)
{
    return devopen(c, omode, archdir, narchdir, devgen);
}
```

⟨*function* `archclose`*(arm)* 624e⟩≡ (626c)

```
static void
archclose(Chan*)
{
}
```

⟨*function* `archread`*(arm)* 624f⟩≡ (626c)

```
static long
archread(Chan *c, void *a, long n, vlong offset)
{
    Rdwrfn *fn;
```

```
    switch((ulong)c->qid.path){
    case Qdir:
        return devdirread(c, a, n, archdir, narchdir, devgen);

    default:
        if(c->qid.path < narchdir && (fn = readfn[c->qid.path]))
            return fn(c, a, n, offset);
        error(Eperm);
        break;
    }

    return 0;
}
```

⟨*function* `archwrite`*(arm)* 625a⟩≡ (626c)

```
static long
archwrite(Chan *c, void *a, long n, vlong offset)
{
    Rdwrfn *fn;

    if(c->qid.path < narchdir && (fn = writefn[c->qid.path]))
        return fn(c, a, n, offset);
    error(Eperm);

    return 0;
}
```

⟨*global* `archdevtab`*(arm)* 625b⟩≡ (626c)

```
Dev archdevtab = {
    .dc = 'P',
    .name = "arch",

    .reset = devreset,
    .init = archinit,
    .shutdown = devshutdown,
    .attach = archattach,
    .walk = archwalk,
    .stat = archstat,
    .open = archopen,
    .create = devcreate,
    .close = archclose,
    .read = archread,
    .bread = devbread,
    .write = archwrite,
    .bwrite = devbwrite,
    .remove = devremove,
    .wstat = devwstat,
};
```

⟨*function* `cputyperead`*(arm)* 625c⟩≡ (626c)

```
static long
cputyperead(Chan*, void *a, long n, vlong offset)
{
    char name[64], str[128];

    cputype2name(name, sizeof name);
    snprint(str, sizeof str, "ARM %s %d\n", name, cpu->cpumhz);
    return readstr(offset, a, n, str);
}
```

⟨*function* cputempread*(arm)* 626a⟩≡ (626c)

```
static long
cputempread(Chan*, void *a, long n, vlong offset)
{
    char str[16];

    snprint(str, sizeof str, "%ud\n", (getcputemp()+500)/1000);
    return readstr(offset, a, n, str);
}
```

⟨*function* archinit*(arm)* 626b⟩≡ (626c)

```
void
archinit(void)
{
    addarchfile("cputype", 0444, cputyperead, nil);
    addarchfile("cputemp", 0444, cputempread, nil);
}
```

⟨devices/sys/arm/devarch.c 626c⟩≡

⟨*kernel basic includes* 23m⟩

```
#include "io.h"
```

⟨*enum* _anon_*(*(devices/sys/arm/devarch.c)(arm)*)* 623a⟩

```
typedef long Rdwrfn(Chan*, void*, long, vlong);
void archinit(void);
```

⟨*global* readfn*(arm)* 623b⟩
⟨*global* writefn*(arm)* 623c⟩

⟨*global* archdir*(arm)* 623d⟩

⟨*global* archwlock*(arm)* 623e⟩
⟨*global* narchdir*(arm)* 623f⟩

⟨*function* addarchfile*(arm)* 623g⟩

⟨*function* archattach*(arm)* 624a⟩

⟨*function* archwalk*(arm)* 624b⟩

⟨*function* archstat*(arm)* 624c⟩

⟨*function* archopen*(arm)* 624d⟩

⟨*function* archclose*(arm)* 624e⟩

⟨*function* archread*(arm)* 624f⟩

⟨*function* archwrite*(arm)* 625a⟩

⟨*global* archdevtab*(arm)* 625b⟩

⟨*function* cputyperead*(arm)* 625c⟩

⟨*function* cputempread*(arm)* 626a⟩

⟨*function* archinit*(arm)* 626b⟩

E.23 init/arm/

E.23.1 init/arm/start.s

⟨init/arm/start.s 627a⟩≡

```
/*
 * Common startup for armv6 and armv7
 * The rest of l.s has been moved to armv[67].s
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨function _start(arm) 186a⟩
```

E.23.2 init/arm/startv6.s

⟨init/arm/startv6.s 627b⟩≡

```
/*
 * Broadcom bcm2835 SoC, as used in Raspberry Pi
 * arm1176jzf-s processor (armv6)
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨function armstart(raspberry pi1)(arm) 186b⟩

⟨function _startpg(raspberry pi1)(arm) 188d⟩



⟨function arch_idlehands(raspberry pi1)(arm) 164c⟩
```

E.23.3 init/arm/startv7.s

⟨init/arm/startv7.s 627c⟩≡

```
/*
 * Broadcom bcm2836 SoC, as used in Raspberry Pi 2
 * 4 x Cortex-A7 processor (armv7)
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨instruction WFI_EQ(arm) 415a⟩

⟨function armstart(raspberry pi2)(arm) 407i⟩

⟨function _startpg(raspberry pi2)(arm) 409f⟩

⟨function cpureset(raspberry pi2)(arm) 411d⟩

⟨function _startpg2(raspberry pi2)(arm) 413a⟩

⟨function cpidget(raspberry pi2)(arm) 414d⟩
```

⟨*function* `arch_idlehands`*(raspberry pi2)(arm)* 414e⟩

E.23.4 init/arm/main.c

⟨*function* `getconf`*(arm)* 628a⟩≡ (628c)

```
char*
getconf(char *name)
{
    USED(name);
    //int i;
    //
    //i = findconf(name);
    //if(i >= 0)
    //  return confval[i];
    return nil;
}
```

⟨*function* `main_arch_isaconfig`*(arm)* 628b⟩≡ (628c)

```
/*
 * stub for ../omap/devether.c
 */
int
main_arch_isaconfig(char *class, int ctlrno, ISAConf *isa)
{
    char cc[32], *p;
    int i;

    if(strcmp(class, "ether") != 0)
        return 0;
    snprint(cc, sizeof cc, "%s%d", class, ctlrno);
    p = getconf(cc);
    if(p == nil)
        return (ctlrno == 0);
    isa->type = "";
    isa->nopt = tokenize(p, isa->opt, NISAOPT);
    for(i = 0; i < isa->nopt; i++){
        p = isa->opt[i];
        if(cistrncmp(p, "type=", 5) == 0)
            isa->type = p + 5;
    }
    return 1;
}
```

⟨`init/arm/main.c` 628c⟩≡

⟨*kernel basic includes* 23m⟩

```
#include "io.h"
// for mainmem and imagmem
#include <pool.h>

// initcode binary
#include "init.h"

// rebootcode binary
#include "reboot.h"

// in portscreen.h
extern void swcursor_init(void);
```

⟨*constant* Minfirmrev*(arm)* 200d⟩
⟨*constant* Minfirmdate*(arm)* 200e⟩

```
// part of a trick to remove some backward dependencies
int devcons_print(char*, ...);
int devcons_iprint(char*, ...);
int devcons_pprint(char*, ...);
void devcons_panic(char*, ...);
void devcons__assert(char*);
void proc_dumpaproc(Proc *p);
void proc_error(char*);
void proc_nexterror(void);
void proc_sched(void);
void proc_ready(Proc*);
void proc_sleep(Rendez*, int(*)(void*), void*);
void proc_tsleep(Rendez *r, int (*fn)(void*), void *arg, ulong ms);
Proc* proc_wakeup(Rendez*);
void proc_pexit(char *exitstr, bool freemem);
Proc* proc_proctab(int i);
int proc_postnote(Proc *p, int dolock, char *n, int flag);
void chan_cclose(Chan *c);

uvlong clock_arch_fastticks(uvlong *hz);
void clock_arch_delay(int millisecs);
void clock_arch_microdelay(int microsecs);
void trap_arch_dumpstack(void);
void arm_arch_coherence(void);
void main_arch_exit(int ispanic);
int  main_arch_isaconfig(char *class, int ctlrno, ISAConf *isa);

//*****************************************************************************
// Configuration
//*****************************************************************************
// See globals in portdat_globals.h

// <conf>.c
extern  Dev*  conf_devtab[];
extern  char*   conffile;
```

⟨*function* getconf*(arm)* 628a⟩

```
//*****************************************************************************
// Boot parameters, see bootconf.c (not used by pad)
//*****************************************************************************

//*****************************************************************************
// Cpu init
//*****************************************************************************
```

⟨*function* arch__cpuinit*(arm)* 190d⟩

⟨*function* arch__cpu0init*(arm)* 190c⟩

⟨*function* machon*(arm)* 191a⟩

```
//*****************************************************************************
// Conf init
//*****************************************************************************
```

```
⟨global memsize(arm) 192b⟩

⟨function arch__confinit(arm) 192c⟩

//*****************************************************************************
// First process init
//*****************************************************************************

⟨global sp(arm) 201a⟩

// kernel space instructions executed by first process
⟨function init0(arm) 202⟩

// user space instructions executed by first process
// see initcode in init.h (comes from ../port/initcode.c and init9.s

extern uintptr bootargs(uintptr base);
⟨function arch__userinit(arm) 201c⟩


//*****************************************************************************
// Shutdown/reboot
//*****************************************************************************

⟨function shutdown((init/arm/main.c)(arm)) 490a⟩

⟨function main_arch_exit(arm) 489e⟩

⟨function arch_reboot(arm) 490c⟩

//*****************************************************************************
// Misc
//*****************************************************************************

⟨function main_arch_isaconfig(arm) 628b⟩

⟨function arch_memorysummary(arm) 501d⟩

//*****************************************************************************
// Main entry point!
//*****************************************************************************

⟨function launchinit(arm) 410a⟩

⟨function main(arm) 189⟩
```

E.23.5 init/arm/raspi1.c

```
⟨function archether(raspberry pi1)(arm) 630⟩≡                                (631a)
  int
  archether(unsigned ctlrno, Ether *ether)
  {
      ether->type = "usb";
      ether->ctlrno = ctlrno;
      ether->irq = -1;
      ether->nopt = 0;
      return 1;
  }
```

⟨init/arm/raspi1.c 631a⟩≡

```
/*
 * bcm2835 (e.g. original raspberry pi) architecture-specific stuff
 */
⟨kernel basic includes 23m⟩

#include "io.h"
#include "arm.h"

#include "../port/netif.h"
#include "../port/etherif.h"

⟨global soc(raspberry pi1)(arm) 33b⟩


⟨function cputype2name(raspberry pi1)(arm) 191c⟩

⟨function arch_cpuidprint(raspberry pi1)(arm) 191b⟩

⟨function getncpus(raspberry pi1)(arm) 410e⟩

⟨function startcpus(raspberry pi1)(arm) 410d⟩


⟨function archether(raspberry pi1)(arm) 630⟩

⟨function l2ap(raspberry pi1)(arm) 114e⟩
```

E.23.6 init/arm/raspi2.c

⟨init/arm/raspi2.c 631b⟩≡

```
/*
 * bcm2836 (e.g.raspberry pi 2) architecture-specific stuff
 */
⟨kernel basic includes 23m⟩

#include "io.h"
#include "arm.h"

#include "../port/netif.h"
#include "../port/etherif.h"

typedef struct Mbox Mbox;
typedef struct Mboxes Mboxes;

⟨constant ARMLOCAL(arm) 406h⟩

⟨global soc(raspberry pi2)(arm) 405g⟩


⟨struct Mbox(arm) 406f⟩
⟨struct Mboxes(arm) 406g⟩

enum {
    Mboxregs    = 0x80
};

static Lock startlock[MAXCPUS + 1];
```

```
// in startv7.s
extern u32int cpidget(void);
```

⟨*function* cputype2name*(raspberry pi2)(arm)* 414c⟩

⟨*function* arch_cpuidprint*(raspberry pi2)(arm)* 414b⟩

⟨*function* getncpus*(raspberry pi2)(arm)* 410f⟩

⟨*function* startcpu*(raspberry pi2)(arm)* 411c⟩

⟨*function* startcpus*(raspberry pi2)(arm)* 411b⟩

```
int
archether(unsigned ctlrno, Ether *ether)
{
    ether->type = "usb";
    ether->ctlrno = ctlrno;
    ether->irq = -1;
    ether->nopt = 0;
    return 1;
}
```

⟨*function* l2ap*(raspberry pi2)(arm)* 406c⟩

```
// in main.c
extern void arch__cpuinit(void);
extern void machon(uint);
```

⟨*function* cpustart*(raspberry pi2)(arm)* 413b⟩

E.23.7 init/user/preboot/arm/init9.s

E.23.8 init/arm/rebootcode.s

⟨init/arm/rebootcode.s 632⟩≡

```
/*
 * armv6/armv7 reboot code
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"
```

⟨*constant* PTEDRAM*(arm)* 492a⟩

⟨*instruction WFI(arm)* 492b⟩
⟨*instruction WFE(arm)* 492c⟩

⟨*function main(rebootcode.s)(arm)* 492d⟩

E.24 memory/arm/

E.24.1 memory/arm/dat_memory.h

⟨memory/arm/dat_memory.h 633a⟩≡

```
/*
 *  MMU stuff in proc
 */
⟨constant NCOLOR(arm) 423e⟩

⟨struct Arch_Proc(arm) 30a⟩

/*
 * Fake kmap.
 */
typedef void        Arch_KMap;
⟨macro VA(arm) 114l⟩
```

E.24.2 memory/arm/mem.h

⟨memory/arm/mem.h 633b⟩≡

```
/*
 * Memory and machine-specific definitions.  Used in C and assembler.
 */

⟨constant KiB(arm) 23j⟩
⟨constant MiB(arm) 23k⟩
⟨constant GiB(arm) 23l⟩

/*
 * Sizes
 */
⟨constant BY2PG(arm) 22a⟩
⟨constant PGSHIFT(arm) 22b⟩

⟨constant MAXCPUS(arm) 28h⟩
⟨constant CPUSIZE(arm) 28d⟩
⟨constant L1SIZE(arm) 410b⟩

⟨macro STACKALIGN(arm) 201b⟩

/*
 * Magic registers
 */
⟨constant UP(arm) 100b⟩
⟨constant CPU(arm) 100a⟩

/*
 * Address spaces.
 * KTZERO is used by kprof and dumpstack (if any).
 *
 * KZERO is mapped to physical 0 (start of ram).
 *
 * vectors are at 0, plan9.ini is at KZERO+256 and is limited to 16K by
 * devenv.
 */
⟨constant KSEG0(arm) 25a⟩
```

⟨*constant* KSEGM*(arm)* 25l⟩

⟨*constant* KZERO*(arm)* 25b⟩

⟨*constant* CONFADDR*(arm)* 643e⟩

```
// L1 must be 16KiB aligned so this dictates the organization below
```
⟨*constant* CPUADDR*(arm)* 25d⟩
⟨*constant* L2*(arm)* 188b⟩
⟨*constant* VCBUFFER*(arm)* 281b⟩
⟨*constant* FIQSTKTOP*(arm)* 428e⟩
⟨*constant* L1*(arm)* 188a⟩

⟨*constant* KTZERO*(arm)* 25c⟩

⟨*constant* VIRTIO*(arm)* 22c⟩
⟨*constant* FRAMEBUFFER*(arm)* 23a⟩

⟨*constant* UZERO*(arm)* 25e⟩
⟨*constant* UTZERO*(arm)* 25f⟩
⟨*macro* UTROUND*(arm)* 85c⟩
⟨*constant* USTKTOP*(arm)* 25g⟩
⟨*constant* USTKSIZE*(arm)* 25h⟩
⟨*constant* TSTKTOP*(arm)* 87a⟩
⟨*constant* TSTKSIZ*(arm)* 87b⟩

⟨*constant* REBOOTADDR*(arm)* 489f⟩

```
/*
 * Legacy...
 */
//old: was 64 in bcm-latest, but then get panic in _allocb
```
⟨*constant* BLOCKALIGN*(arm)* 324d⟩
⟨*constant* KSTACK*(arm)* 151g⟩

```
/*
 * Sizes
 */
```
⟨*constant* BY2WD*(arm)* 20a⟩
⟨*constant* BY2V*(arm)* 20b⟩

⟨*constant* PAGETABMAPMEM*(arm)* 62c⟩
⟨*constant* PAGETABSIZE*(arm)* 62d⟩
⟨*constant* PAGEDIRSIZE*(arm)* 62a⟩
⟨*constant* SMALLPAGEDIRSIZE*(arm)* 64d⟩
⟨*macro* PPN*(arm)* 114k⟩

```
/*
 * Physical machine information from here on.
 *  PHYS addresses as seen from the arm cpu.
 *  BUS  addresses as seen from the videocore gpu.
 */
```
⟨*constant* PHYSDRAM*(arm)* 25k⟩
⟨*constant* IOSIZE*(arm)* 22d⟩

E.24.3 memory/arm/mmu.c

⟨*enum* _anon_*(*(memory/arm/mmu.c)(arm)*)* 634⟩≡ (635a)

```
enum {
```

⟨*constant* L1lo*(arm)* 114h⟩
⟨*constant* L1hi*(arm)* 114i⟩
};

⟨memory/arm/mmu.c 635a⟩≡
⟨*kernel basic includes* 23m⟩
#include "arm.h"

⟨*macro* L1X*(arm)* 113a⟩
⟨*macro* L2X*(arm)* 113b⟩
⟨*macro* L2AP*(arm)* 114d⟩

⟨*constant* L1ptedramattrs*(arm)* 401f⟩
⟨*constant* L2ptedramattrs*(arm)* 401g⟩

⟨*enum* _anon_*(*(memory/arm/mmu.c)(arm)*)* 634⟩

⟨*function* mmuinit*(arm)* 188c⟩

⟨*function* mmuinit1*(arm)* 196a⟩

⟨*function* mmul2empty*(arm)* 115d⟩

⟨*function* mmul1empty*(arm)* 116a⟩

⟨*function* arch_mmuswitch*(arm)* 154a⟩

⟨*function* arch_flushmmu*(arm)* 116c⟩

⟨*function* arch_mmurelease*(arm)* 114m⟩

⟨*function* arch_putmmu*(arm)* 119⟩

⟨*function* arch_cankaddr*(arm)* 25p⟩

⟨*function* mmukmap*(arm)* 282b⟩

⟨*function* arch_checkmmu*(arm)* 120e⟩

⟨*function* arch_kmap*(arm)* 121a⟩

⟨*function* arch_kunmap*(arm)* 121b⟩

E.24.4 memory/arm/mmuinvalidate.s

⟨memory/arm/mmuinvalidate.s 635b⟩≡
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨*function* mmuinvalidate*(arm)* 115a⟩

⟨*function* mmuinvalidateaddr*(arm)* 120b⟩

E.24.5 memory/arm/cache_raspi1.s

⟨memory/arm/cache_raspi1.s 635c⟩≡

```
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨constant CACHELINESZ(raspberry pi1)(ram) 402b⟩

⟨function cachexxx(raspberry pi1)(ram) 403e⟩
```

E.24.6 memory/arm/cache_raspi2.s

```
⟨memory/arm/cache_raspi2.s 636a⟩≡
  #include "mem.h"
  #include "arm.h"
  #include "arminstr.ha"

  ⟨constant CACHELINESZ(raspberry pi2)(ram) 417d⟩

  ⟨function cachexxx(raspberry pi2)(ram) 417g⟩
```

E.25 interrupts/arm/

E.25.1 interrupts/arm/trap.c

```
⟨enum _anon_((interrupts/arm/trap.c)(arm)) 636b⟩≡                    (636c)
  enum {
      Debug = 0,

      ⟨constant Nvec(arm) 193e⟩
      ⟨constant Fiqenable(arm) 429e⟩

      ⟨constant Localtimerint(arm) 422e⟩
      ⟨constant Localmboxint(arm) 422g⟩
      ⟨constant Localintpending(arm) 422f⟩
  };

⟨interrupts/arm/trap.c 636c⟩≡
  /*
   * traps, exceptions, interrupts, system calls.
   */
  ⟨kernel basic includes 23m⟩
  #include "io.h"
  #include "ureg.h"
  #include "arm.h"

  ⟨constant INTREGS(arm) 22g⟩
  ⟨constant LOCALREGS(arm) 422h⟩

  typedef struct Intregs Intregs;
  typedef struct Vctl Vctl;
  typedef struct Vpage0 Vpage0;

  ⟨enum _anon_((interrupts/arm/trap.c)(arm)) 636b⟩

  ⟨struct Vpage0(arm) 193d⟩
```

⟨*struct* `Intregs`*(arm)* 111b⟩

⟨*struct* `Vctl`*(arm)* 110c⟩

⟨*global* `vctllock`*(arm)* 111a⟩

⟨*global* `vctl`*(arm)* 110d⟩

⟨*global* `vfiq`*(arm)* 429d⟩

⟨*global* `trapnames`*(arm)* 99b⟩

⟨*function* `arch__trapinit`*(arm)* 194a⟩

⟨*function* `intrcpushutdown`*(arm)* 491c⟩

⟨*function* `intrsoff`*(arm)* 194b⟩

⟨*function* `intrshutdown`*(arm)* 491a⟩

⟨*function* `intrtime`*(arm)* 513b⟩

⟨*function* `irq`*(arm)* 112e⟩

⟨*function* `fiq`*(arm)* 430a⟩

⟨*function* `irqenable`*(arm)* 111d⟩

⟨*function* `trapname`*(arm)* 505b⟩

⟨*function* `ckfaultstuck`*(arm)* 505d⟩

⟨*function* `faultarm`*(arm)* 116d⟩

⟨*function* `writetomem`*(arm)* 108g⟩

⟨*function* `arch__trap`*(arm)* 105⟩

⟨*function* `isvalidaddr`*(arm)* 505a⟩

⟨*function* `dumplongs`*(arm)* 504b⟩

⟨*function* `dumpstackwithureg`*(arm)* 506b⟩

⟨*function* `getpcsp`*(arm)* 506a⟩

⟨*function* `arch_callwithureg`*(arm)* 505f⟩

⟨*function* `trap_arch_dumpstack`*(arm)* 505e⟩

⟨*function* `dumpregs`*(arm)* 504a⟩

E.25.2 interrupts/arm/fault_helpers.s

⟨`interrupts/arm/fault_helpers.s` 637⟩≡

```
#include "mem.h"
#include "arm.h"
```

⟨*function* `fstget`*(arm)* 108e⟩

⟨*function* `ifsrget`*(arm)* 107f⟩

⟨*function* `farget`*(arm)* 108d⟩

E.25.3 interrupts/arm/lexception.s

⟨`interrupts/arm/lexception.s` 638a⟩≡

```
/*
 * arm exception handlers
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"
```

⟨*global* `vectors`*(arm)* 99c⟩

⟨*global* `vtable`*(arm)* 99d⟩

⟨*function* `_vsvc`*(arm)* 100c⟩

⟨*function* `_vund`*(arm)* 103d⟩

⟨*function* `_vpabt`*(arm)* 103e⟩

⟨*function* `_vdabt`*(arm)* 103f⟩

⟨*function* `_virq`*(arm)* 103g⟩

⟨*label* `_vswitch`*(arm)* 104a⟩

⟨*function* `_vfiq`*(arm)* 429a⟩

⟨*function* `setr13`*(arm)* 194e⟩

E.26 time/arm/

E.26.1 time/arm/dat_time.h

⟨`time/arm/dat_time.h` 638b⟩≡

```
/*
 * Time.
 *
 * HZ should divide 1000 evenly, ideally.
 * 100, 125, 200, 250 and 333 are okay.
 */
```

⟨*constant* `Arch_HZ`*(arm)* 158e⟩

```
// was vlong in x86
//typedef uvlong        Tval;
```

E.26.2 time/arm/clock.c

⟨*enum* `_anon_`*(*`(time/arm/clock.c)(arm)`*)* 638c⟩≡ (639b)

```
enum {
    ⟨constant Localctl(arm) 416b⟩
    ⟨constant Prescaler(arm) 416c⟩

    ⟨constant SystimerFreq(arm) 166c⟩

    ⟨constant MaxPeriod(arm) 171b⟩
    ⟨constant MinPeriod(arm) 171c⟩
};
```

⟨*enum* _anon_*(*(time/arm/clock.c)2(arm)*)* 639a⟩≡ (639b)

```
enum {
    CntPrescaleShift= 16,   /* freq is sys_clk/(prescale+1) */
    CntPrescaleMask = 0xFF,
    CntEnable   = 1<<9,
    TmrDbgHalt  = 1<<8,
    TmrEnable   = 1<<7,
    TmrIntEnable    = 1<<5,
    TmrPrescale1    = 0x00<<2,
    TmrPrescale16   = 0x01<<2,
    TmrPrescale256  = 0x02<<2,
    CntWidth16  = 0<<1,
    CntWidth32  = 1<<1,

    /* generic timer (cortex-a7) */
    Enable  = 1<<0,
    Imask   = 1<<1,
    Istatus = 1<<2,
};
```

⟨time/arm/clock.c 639b⟩≡

```
/*
 * bcm283[56] timers
 *  System timers run at 1MHz (timers 1 and 2 are used by GPU)
 *  ARM timer usually runs at 250MHz (may be slower in low power modes)
 *  Cycle counter runs at 700MHz (unless overclocked)
 *    All are free-running up-counters
 *  Cortex-a7 has local generic timers per cpu (which we run at 1MHz)
 *
 * Use system timer 3 (64 bits) for hzclock interrupts and fastticks
 *   For smp on bcm2836, use local generic timer for interrupts on cpu1-3
 * Use ARM timer (32 bits) for perfticks
 * Use ARM timer to force immediate interrupt
 * Use cycle counter for cycles()
 */
⟨kernel basic includes 23m⟩
#include "io.h"
#include "ureg.h"
#include "arm.h"

⟨constant SYSTIMERS(arm) 22e⟩
⟨constant ARMTIMER(arm) 22f⟩
//TODO factorize in mem.h
⟨constant ARMLOCAL(arm) 406h⟩

⟨enum _anon_*(*(time/arm/clock.c)(arm)*)* 638c⟩

typedef struct Systimers Systimers;
typedef struct Armtimer Armtimer;
```

⟨*struct* Systimers*(arm)* 167b⟩

⟨*struct* Armtimer*(arm)* 514b⟩

⟨*enum* _anon_*(*(time/arm/clock.c)2(arm)*)* 639a⟩

⟨*function* clockintr*(arm)* 167c⟩

⟨*function* localclockintr*(arm)* 416f⟩

⟨*function* clockshutdown*(arm)* 491b⟩

⟨*function* clockinit*(arm)* 195a⟩

⟨*function* arch_timerset*(arm)* 170e⟩

⟨*function* clock_arch_fastticks*(arm)* 166f⟩

⟨*function* arch_perfticks*(arm)* 514a⟩

⟨*function* armtimerset*(arm)* 705a⟩

⟨*function* arch_us*(arm)* 166e⟩

⟨*function* clock_arch_microdelay*(arm)* 173d⟩

⟨*function* clock_arch_delay*(arm)* 173e⟩

E.26.3 time/arm/time_raspi1.s

⟨time/arm/time_raspi1.s 640a⟩≡

```
#include "mem.h"
#include "arm.h"
```

⟨*function* arch_lcycles*(raspberry pi1)(arm)* 384a⟩

E.26.4 time/arm/time_raspi2.s

⟨*instruction MRRC(arm)* 640b⟩≡ (608)

```
#define MRRC(coproc, op, rd, rn, crm) \
    WORD $(0xec500000|(rn)<<16|(rd)<<12|(coproc)<<8|(op)<<4|(crm))
```

⟨time/arm/time_raspi2.s 640c⟩≡

```
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"
```

⟨*function* arch_lcycles*(raspberry pi2)(arm)* 417a⟩

E.27 processes/arm/

E.27.1 processes/arm/dat_processes.h

⟨processes/arm/dat_processes.h 640d⟩≡

⟨*constant* AOUT_MAGIC*(arm)* 83c⟩

⟨*enum* _anon_*(*(arch/arm/dat_processes.h)(arm)*)* 388d⟩

⟨*struct* Arch_FPsave*(arm)* 388b⟩

E.27.2 processes/arm/arch.c

⟨processes/arm/arch.c 641a⟩≡

```
⟨kernel basic includes 23m⟩

#include <tos.h>
#include "ureg.h"

#include "arm.h"

/*
 * A lot of this stuff doesn't belong here
 * but this is a convenient dumping ground for
 * later sorting into the appropriate buckets.
 */

⟨function arch_userpc(arm) 103a⟩

⟨function arch_dbgpc(arm) 503b⟩

⟨function arch_userureg(arm) 103c⟩


⟨function arch_setregisters(arm) 362c⟩

⟨function arch_setkernur(arm) 470b⟩

⟨function arch_validalign(arm) 431b⟩

⟨function arch__kexit(arm) 106a⟩


⟨function linkproc(arm) 97c⟩

⟨function arch_kprocchild(arm) 97b⟩
```

E.27.3 processes/arm/vfp3.c

⟨processes/arm/vfp3.c 641b⟩≡

```
/*
 * VFPv2 or VFPv3 floating point unit
 */
⟨kernel basic includes 23m⟩
#include "ureg.h"
#include "arm.h"

⟨enum _anon_*(*(processes/arm/vfp3.c)(arm)*)* 391b⟩

⟨enum _anon_*(*(processes/arm/vfp3.c)2(arm)*)* 393b⟩
⟨enum _anon_*(*(processes/arm/vfp3.c)3(arm)*)* 393c⟩
⟨enum _anon_*(*(processes/arm/vfp3.c)4(arm)*)* 394a⟩
```

⟨*function* subarch*(arm)* 394b⟩

⟨*function* implement*(arm)* 394c⟩

⟨*function* havefp*(arm)* 394e⟩

⟨*function* fpoff*(arm)* 395a⟩

⟨*function* fponly*(arm)* 395b⟩

⟨*function* fpcfg*(arm)* 395c⟩

⟨*function* fpinit*(arm)* 396b⟩

⟨*function* fpon*(arm)* 396c⟩

⟨*function* fpclear*(arm)* 396d⟩

⟨*function* fpunotify*(arm)* 390c⟩

⟨*function* fpunoted*(arm)* 390d⟩

⟨*function* fpusysrfork*(arm)* 390a⟩

⟨*function* fpusysrforkchild*(arm)* 388f⟩

⟨*function* fpsave*(arm)* 390b⟩

⟨*function* fprestore*(arm)* 400a⟩

⟨*function* fpuprocsave*(arm)* 389b⟩

⟨*function* fpuprocrestore*(arm)* 389d⟩

⟨*function* fpusysprocsetup*(arm)* 388h⟩

⟨*function* arch_procsetup*(arm)* 85e⟩

⟨*function* arch_procsave*(arm)* 154b⟩

⟨*function* arch_procrestore*(arm)* 154c⟩

⟨*function* mathnote*(arm)* 399a⟩

⟨*function* mathemu*(arm)* 399b⟩

⟨*function* fpstuck*(arm)* 397a⟩

⟨*enum* _anon_*(*(processes/arm/vfp3.c)5(arm)*)* 397c⟩

⟨*function* condok*(arm)* 397d⟩

```
/* instruction decoding */ // was in arm.h
```
⟨*macro* ISCPOP*(arm)* 398a⟩
⟨*macro* ISVFPOP*(arm)* 398b⟩

⟨*function* fpuemu*(arm)* 398c⟩

E.27.4 processes/arm/lproc.s

⟨processes/arm/lproc.s 643a⟩≡
```
#include "mem.h"
#include "arm.h"

⟨function arch_touser(arm) 203c⟩

⟨function arch__forkret(arm) 80c⟩
```

E.27.5 processes/arm/switch.s

⟨processes/arm/switch.s 643b⟩≡
```
⟨function arch_setlabel(arm) 151i⟩

⟨function arch_gotolabel(arm) 151h⟩

⟨function getcallerpc(arm) 20d⟩
```

E.28 syscalls/arm/

E.28.1 syscalls/arm/syscall.c

⟨syscalls/arm/syscall.c 643c⟩≡
```
⟨kernel basic includes 23m⟩
#include "../port/systab.h"
#include <tos.h>
#include "ureg.h"
#include "arm.h"

typedef struct NFrame NFrame;
⟨struct NFrame(arm) 336d⟩

⟨function arch__noted(arm) 338c⟩

⟨function arch__notify(arm) 336e⟩

⟨function arch__syscall(arm) 101d⟩

⟨function arch_execregs(arm) 89a⟩

⟨function arch_forkchild(arm) 80a⟩
```

E.29 Extra files

E.29.1 init/arm/bootconf.c

⟨enum _anon_((init/arm/bootconf.c)(arm)) 643d⟩≡ (647)
```
enum {
    /* space for syscall args, return PC, top-of-stack struct */
    Ustkheadroom    = sizeof(Sargs) + sizeof(uintptr) + sizeof(Tos),
};
```

⟨constant CONFADDR(arm) 643e⟩≡ (633b)
```
#define CONFADDR    (KZERO+0x100)       /* unparsed plan9.ini */
```

⟨*constant* BOOTARGS*(arm)* 644a⟩≡ (647)

```
/*
 * Where configuration info is left for the loaded programme.
 */
#define BOOTARGS    ((char*)CONFADDR)
```

⟨*constant* BOOTARGSLEN*(arm)* 644b⟩≡ (647)

```
#define BOOTARGSLEN (CPUADDR-CONFADDR)
```

⟨*global* oargc*(arm)* 644c⟩≡ (647)

```
/*
 * Option arguments from the command line.
 * oargv[0] is the boot file.
 */
static int oargc;
```

⟨*global* oargv*(arm)* 644d⟩≡ (647)

```
static char* oargv[20];
```

⟨*global* oargb*(arm)* 644e⟩≡ (647)

```
static char oargb[128];
```

⟨*global* oargblen*(arm)* 644f⟩≡ (647)

```
static int oargblen;
```

⟨*struct* Atag*(arm)* 644g⟩≡ (647)

```
struct Atag {
    u32int  size;   /* size of atag in words, including this header */
    u32int  tag;    /* atag type */
    union {
        u32int  data[1];    /* actually [size-2] */
        /* AtagMem */
        struct {
            u32int  size;
            u32int  base;
        } mem;
        /* AtagCmdLine */
        char    cmdline[1]; /* actually [4*(size-2)] */
    };
};
```

⟨*enum* _anon_*((init/arm/bootconf.c)2(arm))* 644h⟩≡ (647)

```
enum {
    AtagNone    = 0x00000000,
    AtagCore    = 0x54410001,
    AtagMem     = 0x54410002,
    AtagCmdline = 0x54410009,
};
```

⟨*function* findconf*(arm)* 644i⟩≡ (647)

```
static int
findconf(char *name)
{
    int i;

    for(i = 0; i < nconf; i++)
        if(cistrcmp(confname[i], name) == 0)
            return i;
    return -1;
}
```

⟨*function* addconf *(arm)* 645a⟩≡ (647)

```
void
addconf(char *name, char *val)
{
    int i;

    i = findconf(name);
    if(i < 0){
        if(val == nil || nconf >= MAXCONF)
            return;
        i = nconf++;
        strecpy(confname[i], confname[i]+sizeof(confname[i]), name);
    }
    strecpy(confval[i], confval[i]+sizeof(confval[i]), val);
}
```

⟨*function* writeconf *(arm)* 645b⟩≡ (647)

```
static void
writeconf(void)
{
    char *p, *q;
    int n;

    p = getconfenv();

    if(waserror()) {
        free(p);
        nexterror();
    }

    /* convert to name=value\n format */
    for(q=p; *q; q++) {
        q += strlen(q);
        *q = '=';
        q += strlen(q);
        *q = '\n';
    }
    n = q - p + 1;
    if(n >= BOOTARGSLEN)
        error("kernel configuration too large");
    memmove(BOOTARGS, p, n);
    memset(BOOTARGS + n, '\n', BOOTARGSLEN - n);
    poperror();
    free(p);
}
```

⟨*function* plan9iniinit *(arm)* 645c⟩≡ (647)

```
static void
plan9iniinit(char *s, int cmdline)
{
    char *toks[MAXCONF];
    int i, c, n;
    char *v;

    if((c = *s) < ' ' || c >= 0x80)
        return;
    if(cmdline)
        n = tokenize(s, toks, MAXCONF);
    else
        n = getfields(s, toks, MAXCONF, 1, "\n");
```

```
    for(i = 0; i < n; i++){
        if(toks[i][0] == '#')
            continue;
        v = strchr(toks[i], '=');
        if(v == nil)
            continue;
        *v++ = '\0';
        addconf(toks[i], v);
    }
}
```

⟨*function* ataginit*(arm)* 646a⟩≡ (647)

```
static void
ataginit(Atag *a)
{
    int n;

    if(a->tag != AtagCore){
        plan9iniinit((char*)a, 0);
        return;
    }
    while(a->tag != AtagNone){
        switch(a->tag){
        case AtagMem:
            /* use only first bank */
            if(conf.mem[0].limit == 0 && a->mem.size != 0){
                memsize = a->mem.size;
                conf.mem[0].base = a->mem.base;
                conf.mem[0].limit = a->mem.base + memsize;
            }
            break;
        case AtagCmdline:
            n = (a->size * sizeof(u32int)) - offsetof(Atag, cmdline[0]);
            if(a->cmdline + n < BOOTARGS + BOOTARGSLEN)
                a->cmdline[n] = 0;
            else
                BOOTARGS[BOOTARGSLEN-1] = 0;
            plan9iniinit(a->cmdline, 1);
            break;
        }
        a = (Atag*)((u32int*)a + a->size);
    }
}
```

⟨*function* optionsinit*(arm)* 646b⟩≡ (647)

```
static void
optionsinit(char* s)
{
    strecpy(oargb, oargb+sizeof(oargb), s);

    oargblen = strlen(oargb);
    oargc = tokenize(oargb, oargv, nelem(oargv)-1);
    oargv[oargc] = nil;
}
```

⟨*function* bootargs*(arm)* 646c⟩≡ (647)

```
uintptr
bootargs(uintptr base)
{
    int i;
```

```
    ulong ssize;
    char **av, *p;

    /*
     * Push the boot args onto the stack.
     * The initial value of the user stack must be such
     * that the total used is larger than the maximum size
     * of the argument list checked in syscall.
     */
    i = oargblen+1;
    p = UINT2PTR(STACKALIGN(base + BY2PG - Ustkheadroom - i));
    memmove(p, oargb, i);

    /*
     * Now push the argv pointers.
     * The code jumped to by touser in lproc.s expects arguments
     *  main(char* argv0, ...)
     * and calls
     *  startboot("/boot/boot", &argv0)
     * not the usual (int argc, char* argv[])
     */
    av = (char**)(p - (oargc+1)*sizeof(char*));
    ssize = base + BY2PG - PTR2UINT(av);
    for(i = 0; i < oargc; i++)
        *av++ = (oargv[i] - oargb) + (p - base) + (USTKTOP - BY2PG);
    *av = nil;
    //sp = USTKTOP - ssize;
    return USTKTOP - ssize;
}
```

⟨init/arm/bootconf.c 647⟩≡

```
⟨kernel basic includes 23m⟩

#include "tos.h"

⟨enum _anon_((init/arm/bootconf.c)(arm)) 643d⟩

⟨constant BOOTARGS(arm) 644a⟩
⟨constant BOOTARGSLEN(arm) 644b⟩
//#define MAXCONF       64
//#define MAXCONFLINE   160

⟨global oargc(arm) 644c⟩
⟨global oargv(arm) 644d⟩
⟨global oargb(arm) 644e⟩
⟨global oargblen(arm) 644f⟩

/* store plan9.ini contents here at least until we stash them in #ec */
//static char confname[MAXCONF][KNAMELEN];
//static char confval[MAXCONF][MAXCONFLINE];
//static int nconf;

typedef struct Atag Atag;
⟨struct Atag(arm) 644g⟩

⟨enum _anon_((init/arm/bootconf.c)2(arm)) 644h⟩

⟨function findconf(arm) 644i⟩
```

⟨*function* addconf*(arm)* 645a⟩

⟨*function* writeconf*(arm)* 645b⟩

⟨*function* plan9iniinit*(arm)* 645c⟩

⟨*function* ataginit*(arm)* 646a⟩

⟨*function* optionsinit*(arm)* 646b⟩

⟨*function* bootargs*(arm)* 646c⟩

E.29.2 devices/storage/arm/emmc.c

⟨*enum* _anon_*(*(devices/storage/arm/emmc.c)(arm)*)* 648a⟩≡ (655b)

```
enum {
    Extfreq     = 100*Mhz,  /* guess external clock frequency if */
                    /* not available from vcore */
    Initfreq    = 400000,   /* initialisation frequency for MMC */
    SDfreq      = 25*Mhz,   /* standard SD frequency */
    DTO     = 14,       /* data timeout exponent (guesswork) */

    MMCSelect   = 7,        /* mmc/sd card select command */
    Setbuswidth = 6,        /* mmc/sd set bus width command */
};
```

⟨*enum* _anon_*(*(devices/storage/arm/emmc.c)2(arm)*)* 648b⟩≡ (655b)

```
enum {
    /* Controller registers */
    Arg2            = 0x00>>2,
    Blksizecnt      = 0x04>>2,
    Arg1            = 0x08>>2,
    Cmdtm           = 0x0c>>2,
    Resp0           = 0x10>>2,
    Resp1           = 0x14>>2,
    Resp2           = 0x18>>2,
    Resp3           = 0x1c>>2,
    Data            = 0x20>>2,
    Status          = 0x24>>2,
    Control0        = 0x28>>2,
    Control1        = 0x2c>>2,
    Interrupt       = 0x30>>2,
    Irptmask        = 0x34>>2,
    Irpten          = 0x38>>2,
    Control2        = 0x3c>>2,
    Forceirpt       = 0x50>>2,
    Boottimeout     = 0x70>>2,
    Dbgsel          = 0x74>>2,
    Exrdfifocfg     = 0x80>>2,
    Exrdfifoen      = 0x84>>2,
    Tunestep        = 0x88>>2,
    Tunestepsstd        = 0x8c>>2,
    Tunestepsddr        = 0x90>>2,
    Spiintspt       = 0xf0>>2,
    Slotisrver      = 0xfc>>2,

    /* Control0 */
    Dwidth4         = 1<<1,
    Dwidth1         = 0<<1,
```

```
/* Control1 */
Srstdata        = 1<<26,    /* reset data circuit */
Srstcmd         = 1<<25,    /* reset command circuit */
Srsthc          = 1<<24,    /* reset complete host controller */
Datatoshift     = 16,       /* data timeout unit exponent */
Datatomask      = 0xF0000,
Clkfreq8shift       = 8,        /* SD clock base divider LSBs */
Clkfreq8mask        = 0xFF00,
Clkfreqms2shift     = 6,        /* SD clock base divider MSBs */
Clkfreqms2mask      = 0xC0,
Clkgendiv       = 0<<5,     /* SD clock divided */
Clkgenprog      = 1<<5,     /* SD clock programmable */
Clken           = 1<<2,     /* SD clock enable */
Clkstable       = 1<<1,
Clkintlen       = 1<<0,     /* enable internal EMMC clocks */

/* Cmdtm */
Indexshift      = 24,
Suspend         = 1<<22,
Resume          = 2<<22,
Abort           = 3<<22,
Isdata          = 1<<21,
Ixchken         = 1<<20,
Crcchken        = 1<<19,
Respmask        = 3<<16,
Respnone        = 0<<16,
Resp136         = 1<<16,
Resp48          = 2<<16,
Resp48busy      = 3<<16,
Multiblock      = 1<<5,
Host2card       = 0<<4,
Card2host       = 1<<4,
Autocmd12       = 1<<2,
Autocmd23       = 2<<2,
Blkcnten        = 1<<1,

/* Interrupt */
Acmderr     = 1<<24,
Denderr     = 1<<22,
Dcrcerr     = 1<<21,
Dtoerr      = 1<<20,
Cbaderr     = 1<<19,
Cenderr     = 1<<18,
Ccrcerr     = 1<<17,
Ctoerr      = 1<<16,
Err     = 1<<15,
Cardintr    = 1<<8,     /* not in Broadcom datasheet */
Cardinsert  = 1<<6,     /* not in Broadcom datasheet */
Readrdy     = 1<<5,
Writerdy    = 1<<4,
Datadone    = 1<<1,
Cmddone     = 1<<0,

/* Status */
Bufread     = 1<<11,    /* not in Broadcom datasheet */
Bufwrite    = 1<<10,    /* not in Broadcom datasheet */
Readtrans   = 1<<9,
Writetrans  = 1<<8,
Datactive   = 1<<2,
```

```
    Datinhibit  = 1<<1,
    Cmdinhibit  = 1<<0,
};
```

⟨*global* cmdinfo*(arm)* 650a⟩≡ (655b)

```
int cmdinfo[64] = {
[0]  Ixchken,
[2]  Resp136,
[3]  Resp48 | Ixchken | Crcchken,
[6]  Resp48 | Ixchken | Crcchken,
[7]  Resp48busy | Ixchken | Crcchken,
[8]  Resp48 | Ixchken | Crcchken,
[9]  Resp136,
[12] Resp48busy | Ixchken | Crcchken,
[13] Resp48 | Ixchken | Crcchken,
[16] Resp48,
[17] Resp48 | Isdata | Card2host | Ixchken | Crcchken,
[18] Resp48 | Isdata | Card2host | Multiblock | Blkcnten | Ixchken | Crcchken,
[24] Resp48 | Isdata | Host2card | Ixchken | Crcchken,
[25] Resp48 | Isdata | Host2card | Multiblock | Blkcnten | Ixchken | Crcchken,
[41] Resp48,
[55] Resp48 | Ixchken | Crcchken,
};
```

⟨*struct* Ctlr*((devices/storage/arm/emmc.c)(arm))* 650b⟩≡ (655b)

```
struct Ctlr {
    Rendez  r;
    int datadone;
    int fastclock;
    ulong   extclk;
};
```

⟨*global* emmc*(arm)* 650c⟩≡ (655b)

```
static Ctlr emmc;
```

⟨*function* WR*(arm)* 650d⟩≡ (655b)

```
static void
WR(int reg, u32int val)
{
    u32int *r = (u32int*)EMMCREGS;

    //if(0)print("WR %2.2ux %ux\n", reg<<2, val);
    arch_microdelay(emmc.fastclock? 2 : 20);
    r[reg] = val;
}
```

⟨*function* clkdiv*(arm)* 650e⟩≡ (655b)

```
static uint
clkdiv(uint d)
{
    uint v;

    assert(d < 1<<10);
    v = (d << Clkfreq8shift) & Clkfreq8mask;
    v |= ((d >> 8) << Clkfreqms2shift) & Clkfreqms2mask;
    return v;
}
```

⟨*function* datadone*(arm)* 651a⟩≡ (655b)

```
static int
datadone(void*)
{
    int i;

    u32int *r = (u32int*)EMMCREGS;
    i = r[Interrupt];
    return i & (Datadone|Err);
}
```

⟨*function* emmcinit*(arm)* 651b⟩≡ (655b)

```
static int
emmcinit(void)
{
    u32int *r;
    ulong clk;
    char *s;

    clk = getclkrate(ClkEmmc);
    s = "";
    if(clk == 0){
        s = "Assuming ";
        clk = Extfreq;
    }
    emmc.extclk = clk;
    print("%seMMC external clock %lud Mhz\n", s, clk / Mhz);
    r = (u32int*)EMMCREGS;
    //if(0)print("emmc control %8.8ux %8.8ux %8.8ux\n", r[Control0], r[Control1], r[Control2]);
    WR(Control1, Srsthc);
    arch_delay(10);
    while(r[Control1] & Srsthc)
        ;
    return 0;
}
```

⟨*function* emmcinquiry*(arm)* 651c⟩≡ (655b)

```
static int
emmcinquiry(char *inquiry, int inqlen)
{
    u32int *r;
    uint ver;

    r = (u32int*)EMMCREGS;
    ver = r[Slotisrver] >> 16;
    return snprint(inquiry, inqlen,
        "Arasan eMMC SD Host Controller %2.2x Version %2.2x",
        ver&0xFF, ver>>8);
}
```

⟨*function* mmcinterrupt*(arm)* 651d⟩≡ (655b)

```
static void
mmcinterrupt(Ureg*, void*)
{
    u32int *r;
    r = (u32int*)EMMCREGS;
    if(r[Interrupt]&(Datadone|Err)){
        WR(Irpten, 0);
        wakeup(&emmc.r);
    }
}
```

⟨*function* emmcenable*(arm)* 652a⟩≡ (655b)

```
static void
emmcenable(void)
{
    u32int *r;
    int i;

    r = (u32int*)EMMCREGS;
    WR(Control1, clkdiv(emmc.extclk/Initfreq - 1) |
        DTO<<Datatoshift | Clkgendiv | Clken | Clkintlen);
    for(i = 0; i < 1000; i++){
        arch_delay(1);
        if(r[Control1] & Clkstable)
            break;
    }
    if(i == 1000)
        print("SD clock won't initialise!\n");
    WR(Irptmask, ~(Dtoerr|Cardintr));
    arch_intrenable(IRQmmc, mmcinterrupt, nil, 0, "mmc");
}
```

⟨*function* emmccmd*(arm)* 652b⟩≡ (655b)

```
static int
emmccmd(u32int cmd, u32int arg, u32int *resp)
{
    u32int *r;
    u32int c;
    int i;
    ulong now;

    r = (u32int*)EMMCREGS;
    assert(cmd < nelem(cmdinfo) && cmdinfo[cmd] != 0);
    c = (cmd << Indexshift) | cmdinfo[cmd];
    if(r[Status] & Cmdinhibit){
        print("emmccmd: need to reset Cmdinhibit intr %ux stat %ux\n",
            r[Interrupt], r[Status]);
        WR(Control1, r[Control1] | Srstcmd);
        while(r[Control1] & Srstcmd)
            ;
        while(r[Status] & Cmdinhibit)
            ;
    }
    if((r[Status] & Datinhibit) &&
       ((c & Isdata) || (c & Respmask) == Resp48busy)){
        print("emmccmd: need to reset Datinhibit intr %ux stat %ux\n",
            r[Interrupt], r[Status]);
        WR(Control1, r[Control1] | Srstdata);
        while(r[Control1] & Srstdata)
            ;
        while(r[Status] & Datinhibit)
            ;
    }
    WR(Arg1, arg);
    if((i = r[Interrupt]) != 0){
        if(i != Cardinsert)
            print("emmc: before command, intr was %ux\n", i);
        WR(Interrupt, i);
    }
    WR(Cmdtm, c);
    now = cpu->ticks;
```

```
while(((i=r[Interrupt])&(Cmddone|Err)) == 0)
    if(cpu->ticks - now > Arch_HZ)
        break;
if((i&(Cmddone|Err)) != Cmddone){
    if((i&~Err) != Ctoerr)
        print("emmc: cmd %ux error intr %ux stat %ux\n", c, i, r[Status]);
    WR(Interrupt, i);
    if(r[Status]&Cmdinhibit){
        WR(Control1, r[Control1]|Srstcmd);
        while(r[Control1]&Srstcmd)
            ;
    }
    error(Eio);
}
WR(Interrupt, i & ~(Datadone|Readrdy|Writerdy));
switch(c & Respmask){
case Resp136:
    resp[0] = r[Resp0]<<8;
    resp[1] = r[Resp0]>>24 | r[Resp1]<<8;
    resp[2] = r[Resp1]>>24 | r[Resp2]<<8;
    resp[3] = r[Resp2]>>24 | r[Resp3]<<8;
    break;
case Resp48:
case Resp48busy:
    resp[0] = r[Resp0];
    break;
case Respnone:
    resp[0] = 0;
    break;
}
if((c & Respmask) == Resp48busy){
    WR(Irpten, Datadone|Err);
    tsleep(&emmc.r, datadone, 0, 3000);
    WR(Irpten, 0);
    emmc.datadone = 0;
    i = r[Interrupt];
    if((i & Datadone) == 0)
        print("emmcio: no Datadone after CMD%d\n", cmd);
    if(i & Err)
        print("emmcio: CMD%d error interrupt %ux\n",
            cmd, r[Interrupt]);
    WR(Interrupt, i);
}
/*
 * Once card is selected, use faster clock
 */
if(cmd == MMCSelect){
    arch_delay(10);
    WR(Control1, clkdiv(emmc.extclk/SDfreq - 1) |
        DTO<<Datatoshift | Clkgendiv | Clken | Clkintlen);
    for(i = 0; i < 1000; i++){
        arch_delay(1);
        if(r[Control1] & Clkstable)
            break;
    }
    arch_delay(10);
    emmc.fastclock = 1;
}
/*
 * If card bus width changes, change host bus width
```

```
     */
    if(cmd == Setbuswidth){
        switch(arg){
        case 0:
            WR(Control0, r[Control0] & ~Dwidth4);
            break;
        case 2:
            WR(Control0, r[Control0] | Dwidth4);
            break;
        }
    }
    return 0;
}
```

⟨*function* emmciosetup*(arm)* 654a⟩≡ (655b)

```
void
emmciosetup(int write, void *buf, int bsize, int bcount)
{
    USED(write);
    USED(buf);
    WR(Blksizecnt, bcount<<16 | bsize);
}
```

⟨*function* emmcio*(arm)* 654b⟩≡ (655b)

```
static void
emmcio(int write, uchar *buf, int len)
{
    u32int *r;
    int i;

    r = (u32int*)EMMCREGS;
    assert((len&3) == 0);
    okay(1);
    if(waserror()){
        okay(0);
        nexterror();
    }
    if(write)
        dmastart(DmaChanEmmc, DmaDevEmmc, DmaM2D,
            buf, &r[Data], len);
    else
        dmastart(DmaChanEmmc, DmaDevEmmc, DmaD2M,
            &r[Data], buf, len);
    if(dmawait(DmaChanEmmc) < 0)
        error(Eio);
    if(!write)
        cachedinvse(buf, len);
    WR(Irpten, Datadone|Err);
    tsleep(&emmc.r, datadone, 0, 3000);
    WR(Irpten, 0);
    emmc.datadone = 0;
    i = r[Interrupt];
    if((i & Datadone) == 0){
        print("emmcio: %d timeout intr %ux stat %ux\n",
            write, i, r[Status]);
        WR(Interrupt, i);
        error(Eio);
    }
    if(i & Err){
        print("emmcio: %d error intr %ux stat %ux\n",
```

```
            write, r[Interrupt], r[Status]);
        WR(Interrupt, i);
        error(Eio);
    }
    if(i)
        WR(Interrupt, i);
    poperror();
    okay(0);
}
```

⟨*global* sdio*(arm)* 655a⟩≡ (655b)

```
SDio sdio = {
    "emmc",
    emmcinit,
    emmcenable,
    emmcinquiry,
    emmccmd,
    emmciosetup,
    emmcio,
};
```

⟨devices/storage/arm/emmc.c 655b⟩≡

```
/*
 * bcm2835 external mass media controller (mmc / sd host interface)
 *
 * Copyright © 2012 Richard Miller <r.miller@acm.org>
 */
⟨kernel basic includes 23m⟩
#include "io.h"
#include "../port/sd.h"

⟨constant EMMCREGS(arm) 23c⟩

⟨enum _anon_((devices/storage/arm/emmc.c)(arm)) 648a⟩

⟨enum _anon_((devices/storage/arm/emmc.c)2(arm)) 648b⟩

⟨global cmdinfo(arm) 650a⟩

typedef struct Ctlr Ctlr;
⟨struct Ctlr((devices/storage/arm/emmc.c)(arm)) 650b⟩

⟨global emmc(arm) 650c⟩

⟨function WR(arm) 650d⟩

⟨function clkdiv(arm) 650e⟩

⟨function datadone(arm) 651a⟩

⟨function emmcinit(arm) 651b⟩

⟨function emmcinquiry(arm) 651c⟩

⟨function mmcinterrupt(arm) 651d⟩

⟨function emmcenable(arm) 652a⟩

⟨function emmccmd(arm) 652b⟩
```

⟨*function* emmciosetup*(arm)* 654a⟩

⟨*function* emmcio*(arm)* 654b⟩

⟨*global* sdio*(arm)* 655a⟩

E.29.3 init/arm/watchdog.c

⟨*enum* _anon_*(*(init/arm/watchdog.c)(arm)*)* 656a⟩≡ (657c)

```
enum {
    Wdogfreq    = 65536,
    Wdogtime    = 5,    /* seconds, <= 15 */
};
```

⟨*enum* _anon_*(*(init/arm/watchdog.c)2(arm)*)* 656b⟩≡ (657c)

```
/*
 * Power management / watchdog registers
 */
enum {
    Rstc        = 0x1c>>2,
        Password    = 0x5A<<24,
        CfgMask     = 0x03<<4,
        CfgReset    = 0x02<<4,
    Rsts        = 0x20>>2,
    Wdog        = 0x24>>2,
};
```

⟨*function* archreset*(arm)* 656c⟩≡ (657c)

```
void
archreset(void)
{
    fpon();
}
```

⟨*function* archreboot*(arm)* 656d⟩≡ (657c)

```
void
archreboot(void)
{
    u32int *r;

    r = (u32int*)POWERREGS;
    r[Wdog] = Password | 1;
    r[Rstc] = Password | (r[Rstc] & ~CfgMask) | CfgReset;
    arch_coherence();
    for(;;)
        ;
}
```

⟨*function* wdogfeed*(arm)* 656e⟩≡ (657c)

```
static void
wdogfeed(void)
{
    u32int *r;

    r = (u32int*)POWERREGS;
    r[Wdog] = Password | (Wdogtime * Wdogfreq);
    r[Rstc] = Password | (r[Rstc] & ~CfgMask) | CfgReset;
}
```

⟨*function* wdogoff*(arm)* 657a⟩≡ (657c)

```
void
wdogoff(void)
{
    u32int *r;

    r = (u32int*)POWERREGS;
    r[Rstc] = Password | (r[Rstc] & ~CfgMask);
}
```

⟨*function* watchdoglink*(arm)* 657b⟩≡ (657c)

```
void
watchdoglink(void)
{
    addclock0link(wdogfeed, Arch_HZ);
}
```

⟨init/arm/watchdog.c 657c⟩≡

⟨*kernel basic includes* 23m⟩

⟨*constant* POWERREGS*(arm)* 22i⟩

⟨*enum* _anon_*(*(init/arm/watchdog.c)(arm)*)* 656a⟩

⟨*enum* _anon_*(*(init/arm/watchdog.c)2(arm)*)* 656b⟩

⟨*function* archreset*(arm)* 656c⟩

⟨*function* archreboot*(arm)* 656d⟩

⟨*function* wdogfeed*(arm)* 656e⟩

⟨*function* wdogoff*(arm)* 657a⟩

⟨*function* watchdoglink*(arm)* 657b⟩

E.29.4 buses/arm/dwcotg.h

⟨*enum* _anon_*(*(buses/arm/dwcotg.h)(arm)*)* 657d⟩≡ (658)

```
enum {
    Maxchans    = 16,   /* actual number of channels in ghwcfg2 */
};
```

⟨*struct* Dwcregs*(arm)* 657e⟩≡ (658)

```
struct Dwcregs {
    /* Core global registers 0x000-0x140 */
    Reg gotgctl;    /* OTG Control and Status */
    Reg gotgint;    /* OTG Interrupt */
    Reg gahbcfg;    /* Core AHB Configuration */
    Reg gusbcfg;    /* Core USB Configuration */
    Reg grstctl;    /* Core Reset */
    Reg gintsts;    /* Core Interrupt */
    Reg gintmsk;    /* Core Interrupt Mask */
    Reg grxstsr;    /* Receive Status Queue Read (RO) */
    Reg grxstsp;    /* Receive Status Queue Read & POP (RO) */
    Reg grxfsiz;    /* Receive FIFO Size */
    Reg gnptxfsiz;  /* Non Periodic Transmit FIFO Size */
    Reg gnptxsts;   /* Non Periodic Transmit FIFO/Queue Status (RO) */
    Reg gi2cctl;    /* I2C Access */
```

```
    Reg gpvndctl;   /* PHY Vendor Control */
    Reg ggpio;      /* General Purpose Input/Output */
    Reg guid;       /* User ID */
    Reg gsnpsid;    /* Synopsys ID (RO) */
    Reg ghwcfg1;    /* User HW Config1 (RO) (DEVICE) */
    Reg ghwcfg2;    /* User HW Config2 (RO) */
    Reg ghwcfg3;    /* User HW Config3 (RO) */
    Reg ghwcfg4;    /* User HW Config4 (RO)*/
    Reg glpmcfg;    /* Core LPM Configuration */
    Reg gpwrdn;     /* Global PowerDn */
    Reg gdfifocfg;  /* Global DFIFO SW Config (DEVICE?) */
    Reg adpctl;     /* ADP Control */
    Reg reserved0[39];
    Reg hptxfsiz;   /* Host Periodic Transmit FIFO Size */
    Reg dtxfsiz[15];    /* Device Periodic Transmit FIFOs (DEVICE) */
    char    pad0[0x400-0x140];

    /* Host global registers 0x400-0x420 */
    Reg hcfg;       /* Configuration */
    Reg hfir;       /* Frame Interval */
    Reg hfnum;      /* Frame Number / Frame Remaining (RO) */
    Reg reserved1;
    Reg hptxsts;    /* Periodic Transmit FIFO / Queue Status */
    Reg haint;      /* All Channels Interrupt */
    Reg haintmsk;   /* All Channels Interrupt Mask */
    Reg hflbaddr;   /* Frame List Base Address */
    char    pad1[0x440-0x420];

    /* Host port register 0x440 */
    Reg hport0;     /* Host Port 0 Control and Status */
    char    pad2[0x500-0x444];

    /* Host channel specific registers 0x500-0x700 */
    struct  Hostchan {
        Reg hcchar; /* Characteristic */
        Reg hcsplt; /* Split Control */
        Reg hcint;  /* Interrupt */
        Reg hcintmsk; /* Interrupt Mask */
        Reg hctsiz; /* Transfer Size */
        Reg hcdma;  /* DMA Address */
        Reg reserved;
        Reg hcdmab; /* DMA Buffer Address */
    } hchan[Maxchans];
    char    pad3[0xE00-0x700];

    /* Power & clock gating control register 0xE00 */
    Reg pcgcctl;
};
```

⟨buses/arm/dwcotg.h 658⟩≡

```
/*
 * USB host driver for BCM2835
 *  Synopsis DesignWare Core USB 2.0 OTG controller
 *
 * Device register definitions
 */

typedef unsigned int Reg;
typedef struct Dwcregs Dwcregs;
typedef struct Hostchan Hostchan;
```

⟨*enum* _anon_*(*(buses/arm/dwcotg.h)(arm)*)* 657d⟩

⟨*struct* Dwcregs*(arm)* 657e⟩

```
enum {
    /* gotgctl */
    Sesreqscs   = 1<<0,
    Sesreq      = 1<<1,
    Vbvalidoven = 1<<2,
    Vbvalidovval    = 1<<3,
    Avalidoven  = 1<<4,
    Avalidovval = 1<<5,
    Bvalidoven  = 1<<6,
    Bvalidovval = 1<<7,
    Hstnegscs   = 1<<8,
    Hnpreq      = 1<<9,
    Hstsethnpen = 1<<10,
    Devhnpen    = 1<<11,
    Conidsts    = 1<<16,
    Dbnctime    = 1<<17,
    Asesvld     = 1<<18,
    Bsesvld     = 1<<19,
    Otgver      = 1<<20,
    Multvalidbc = 0x1F<<22,
    Chirpen     = 1<<27,

    /* gotgint */
    Sesenddet   = 1<<2,
    Sesreqsucstschng= 1<<8,
    Hstnegsucstschng= 1<<9,
    Hstnegdet   = 1<<17,
    Adevtoutchng    = 1<<18,
    Debdone     = 1<<19,
    Mvic        = 1<<20,

    /* gahbcfg */
    Glblintrmsk = 1<<0,
    /* bits 1:4 redefined for BCM2835 */
    Axiburstlen = 0x3<<1,
        BURST1      = 3<<1,
        BURST2      = 2<<1,
        BURST3      = 1<<1,
        BURST4      = 0<<1,
    Axiwaitwrites   = 1<<4,
    Dmaenable   = 1<<5,
    Nptxfemplvl = 1<<7,
        NPTX_HALFEMPTY  = 0<<7,
        NPTX_EMPTY  = 1<<7,
    Ptxfemplvl  = 1<<8,
        PTX_HALFEMPTY   = 0<<8,
        PTX_EMPTY   = 1<<8,
    Remmemsupp  = 1<<21,
    Notialldmawrit  = 1<<22,
    Ahbsingle   = 1<<23,

    /* gusbcfg */
    Toutcal     = 0x7<<0,
    Phyif       = 1<<3,
    Ulpi_utmi_sel   = 1<<4,
```

```
Fsintf      = 1<<5,
    FsUnidir    = 0<<5,
    FsBidir     = 1<<5,
Physel      = 1<<6,
    PhyHighspeed    = 0<<6,
    PhyFullspeed    = 1<<6,
Ddrsel      = 1<<7,
Srpcap      = 1<<8,
Hnpcap      = 1<<9,
Usbtrdtim   = 0xf<<10,
    OUsbtrdtim      = 10,
Phylpwrclksel   = 1<<15,
Otgutmifssel    = 1<<16,
Ulpi_fsls   = 1<<17,
Ulpi_auto_res   = 1<<18,
Ulpi_clk_sus_m  = 1<<19,
Ulpi_ext_vbus_drv= 1<<20,
Ulpi_int_vbus_indicator= 1<<21,
Term_sel_dl_pulse= 1<<22,
Indicator_complement= 1<<23,
Indicator_pass_through= 1<<24,
Ulpi_int_prot_dis= 1<<25,
Ic_usb_cap  = 1<<26,
Ic_traffic_pull_remove= 1<<27,
Tx_end_delay    = 1<<28,
Force_host_mode = 1<<29,
Force_dev_mode  = 1<<30,

/* grstctl */
Csftrst     = 1<<0,
Hsftrst     = 1<<1,
Hstfrm      = 1<<2,
Intknqflsh  = 1<<3,
Rxfflsh     = 1<<4,
Txfflsh     = 1<<5,
Txfnum      = 0x1f<<6,
    TXF_ALL     = 0x10<<6,
Dmareq      = 1<<30,
Ahbidle     = 1<<31,

/* gintsts, gintmsk */
Curmode     = 1<<0,
    HOSTMODE    = 1<<0,
    DEVMODE     = 0<<0,
Modemismatch    = 1<<1,
Otgintr     = 1<<2,
Sofintr     = 1<<3,
Rxstsqlvl   = 1<<4,
Nptxfempty  = 1<<5,
Ginnakeff   = 1<<6,
Goutnakeff  = 1<<7,
Ulpickint   = 1<<8,
I2cintr     = 1<<9,
Erlysuspend = 1<<10,
Usbsuspend  = 1<<11,
Usbreset    = 1<<12,
Enumdone    = 1<<13,
Isooutdrop  = 1<<14,
Eopframe    = 1<<15,
Restoredone = 1<<16,
```

```
Epmismatch  = 1<<17,
Inepintr    = 1<<18,
Outepintr   = 1<<19,
Incomplisoin    = 1<<20,
Incomplisoout   = 1<<21,
Fetsusp     = 1<<22,
Resetdet    = 1<<23,
Portintr    = 1<<24,
Hcintr      = 1<<25,
Ptxfempty   = 1<<26,
Lpmtranrcvd = 1<<27,
Conidstschng    = 1<<28,
Disconnect  = 1<<29,
Sessreqintr = 1<<30,
Wkupintr    = 1<<31,

/* grxsts[rp] */
Chnum       = 0xf<<0,
Bcnt        = 0x7ff<<4,
Dpid        = 0x3<<15,
Pktsts      = 0xf<<17,
    PKTSTS_IN       = 2<<17,
    PKTSTS_IN_XFER_COMP = 3<<17,
    PKTSTS_DATA_TOGGLE_ERR  = 5<<17,
    PKTSTS_CH_HALTED    = 7<<17,

/* hptxfsiz, gnptxfsiz */
Startaddr   = 0xffff<<0,
Depth       = 0xffff<<16,
    ODepth      = 16,

/* gnptxsts */
Nptxfspcavail   = 0xffff<<0,
Nptxqspcavail   = 0xff<<16,
Nptxqtop_terminate= 1<<24,
Nptxqtop_token  = 0x3<<25,
Nptxqtop_chnep  = 0xf<<27,

/* gpvndctl */
Regdata     = 0xff<<0,
Vctrl       = 0xff<<8,
Regaddr16_21    = 0x3f<<16,
Regwr       = 1<<22,
Newregreq   = 1<<25,
Vstsbsy     = 1<<26,
Vstsdone    = 1<<27,
Disulpidrvr = 1<<31,

/* ggpio */
Gpi     = 0xffff<<0,
Gpo     = 0xffff<<16,

/* ghwcfg2 */
Op_mode     = 0x7<<0,
    HNP_SRP_CAPABLE_OTG = 0<<0,
    SRP_ONLY_CAPABLE_OTG    = 1<<0,
    NO_HNP_SRP_CAPABLE  = 2<<0,
    SRP_CAPABLE_DEVICE  = 3<<0,
    NO_SRP_CAPABLE_DEVICE   = 4<<0,
    SRP_CAPABLE_HOST    = 5<<0,
```

```
    NO_SRP_CAPABLE_HOST = 6<<0,
Architecture    = 0x3<<3,
    SLAVE_ONLY      = 0<<3,
    EXT_DMA         = 1<<3,
    INT_DMA         = 2<<3,
Point2point = 1<<5,
Hs_phy_type = 0x3<<6,
    PHY_NOT_SUPPORTED   = 0<<6,
    PHY_UTMI        = 1<<6,
    PHY_ULPI        = 2<<6,
    PHY_UTMI_ULPI       = 3<<6,
Fs_phy_type = 0x3<<8,
Num_dev_ep  = 0xf<<10,
Num_host_chan   = 0xf<<14,
    ONum_host_chan      = 14,
Perio_ep_supported= 1<<18,
Dynamic_fifo    = 1<<19,
Nonperio_tx_q_depth= 0x3<<22,
Host_perio_tx_q_depth= 0x3<<24,
Dev_token_q_depth= 0x1f<<26,
Otg_enable_ic_usb= 1<<31,

/* ghwcfg3 */
Xfer_size_cntr_width    = 0xf<<0,
Packet_size_cntr_width  = 0x7<<4,
Otg_func        = 1<<7,
I2c         = 1<<8,
Vendor_ctrl_if      = 1<<9,
Optional_features   = 1<<10,
Synch_reset_type    = 1<<11,
Adp_supp        = 1<<12,
Otg_enable_hsic     = 1<<13,
Bc_support      = 1<<14,
Otg_lpm_en      = 1<<15,
Dfifo_depth     = 0xffff<<16,
    ODfifo_depth        = 16,

/* ghwcfg4 */
Num_dev_perio_in_ep = 0xf<<0,
Power_optimiz       = 1<<4,
Min_ahb_freq        = 1<<5,
Hiber           = 1<<6,
Xhiber          = 1<<7,
Utmi_phy_data_width = 0x3<<14,
Num_dev_mode_ctrl_ep    = 0xf<<16,
Iddig_filt_en       = 1<<20,
Vbus_valid_filt_en  = 1<<21,
A_valid_filt_en     = 1<<22,
B_valid_filt_en     = 1<<23,
Session_end_filt_en = 1<<24,
Ded_fifo_en     = 1<<25,
Num_in_eps      = 0xf<<26,
Desc_dma        = 1<<30,
Desc_dma_dyn        = 1<<31,

/* glpmcfg */
Lpm_cap_en  = 1<<0,
Appl_resp   = 1<<1,
Hird        = 0xf<<2,
Rem_wkup_en = 1<<6,
```

```
En_utmi_sleep   = 1<<7,
Hird_thres  = 0x1f<<8,
Lpm_resp    = 0x3<<13,
Prt_sleep_sts   = 1<<15,
Sleep_state_resumeok= 1<<16,
Lpm_chan_index  = 0xf<<17,
Retry_count = 0x7<<21,
Send_lpm    = 1<<24,
Retry_count_sts = 0x7<<25,
Hsic_connect    = 1<<30,
Inv_sel_hsic    = 1<<31,

/* gpwrdn */
Pmuintsel   = 1<<0,
Pmuactv     = 1<<1,
Restore     = 1<<2,
Pwrdnclmp   = 1<<3,
Pwrdnrstn   = 1<<4,
Pwrdnswtch  = 1<<5,
Dis_vbus    = 1<<6,
Lnstschng   = 1<<7,
Lnstchng_msk    = 1<<8,
Rst_det     = 1<<9,
Rst_det_msk = 1<<10,
Disconn_det = 1<<11,
Disconn_det_msk = 1<<12,
Connect_det = 1<<13,
Connect_det_msk = 1<<14,
Srp_det     = 1<<15,
Srp_det_msk = 1<<16,
Sts_chngint = 1<<17,
Sts_chngint_msk = 1<<18,
Linestate   = 0x3<<19,
Idsts       = 1<<21,
Bsessvld    = 1<<22,
Adp_int     = 1<<23,
Mult_val_id_bc  = 0x1f<<24,

/* gdfifocfg */
Gdfifocfg   = 0xffff<<0,
Epinfobase  = 0xffff<<16,

/* adpctl */
Prb_dschg   = 0x3<<0,
Prb_delta   = 0x3<<2,
Prb_per     = 0x3<<4,
Rtim        = 0x7ff<<6,
Enaprb      = 1<<17,
Enasns      = 1<<18,
Adpres      = 1<<19,
Adpen       = 1<<20,
Adp_prb_int = 1<<21,
Adp_sns_int = 1<<22,
Adp_tmout_int   = 1<<23,
Adp_prb_int_msk = 1<<24,
Adp_sns_int_msk = 1<<25,
Adp_tmout_int_msk= 1<<26,
Ar      = 0x3<<27,

/* hcfg */
```

```
    Fslspclksel = 0x3<<0,
        HCFG_30_60_MHZ  = 0<<0,
        HCFG_48_MHZ = 1<<0,
        HCFG_6_MHZ  = 2<<0,
    Fslssupp    = 1<<2,
    Ena32khzs   = 1<<7,
    Resvalid    = 0xff<<8,
    Descdma     = 1<<23,
    Frlisten    = 0x3<<24,
    Modechtimen = 1<<31,

    /* hfir */
    Frint       = 0xffff<<0,
    Hfirrldctrl = 1<<16,

    /* hfnum */
    Frnum       = 0xffff<<0,
        MAX_FRNUM   = 0x3FFF<<0,
    Frrem       = 0xffff<<16,

    /* hptxsts */
    Ptxfspcavail    = 0xffff<<0,
    Ptxqspcavail    = 0xff<<16,
    Ptxqtop_terminate= 1<<24,
    Ptxqtop_token   = 0x3<<25,
    Ptxqtop_chnum   = 0xf<<27,
    Ptxqtop_odd = 1<<31,

    /* haint, haintmsk */
#define CHANINT(n)  (1<<(n))

    /* hport0 */
    Prtconnsts  = 1<<0,     /* connect status (RO) */
    Prtconndet  = 1<<1,     /* connect detected R/W1C) */
    Prtena      = 1<<2,     /* enable (R/W1C) */
    Prtenchng   = 1<<3,     /* enable/disable change (R/W1C) */
    Prtovrcurract   = 1<<4,     /* overcurrent active (RO) */
    Prtovrcurrchng  = 1<<5,     /* overcurrent change (R/W1C) */
    Prtres      = 1<<6,     /* resume */
    Prtsusp     = 1<<7,     /* suspend */
    Prtrst      = 1<<8,     /* reset */
    Prtlnsts    = 0x3<<10,  /* line state {D+,D-} (RO) */
    Prtpwr      = 1<<12,    /* power on */
    Prttstctl   = 0xf<<13,  /* test */
    Prtspd      = 0x3<<17,  /* speed (RO) */
        HIGHSPEED   = 0<<17,
        FULLSPEED   = 1<<17,
        LOWSPEED    = 2<<17,

    /* hcchar */
    Mps     = 0x7ff<<0, /* endpoint maximum packet size */
    Epnum       = 0xf<<11,  /* endpoint number */
        OEpnum      = 11,
    Epdir       = 1<<15,    /* endpoint direction */
        Epout       = 0<<15,
        Epin        = 1<<15,
    Lspddev     = 1<<17,    /* device is lowspeed */
    Eptype      = 0x3<<18,  /* endpoint type */
        Epctl       = 0<<18,
        Episo       = 1<<18,
```

```
    Epbulk      = 2<<18,
    Epintr      = 3<<18,
Multicnt    = 0x3<<20,  /* transactions per μframe */
                /* or retries per periodic split */
    OMulticnt   = 20,
Devaddr     = 0x7f<<22, /* device address */
    ODevaddr    = 22,
Oddfrm      = 1<<29,    /* xfer in odd frame (iso/interrupt) */
Chdis       = 1<<30,    /* channel disable (write 1 only) */
Chen        = 1<<31,    /* channel enable (write 1 only) */

/* hcsplt */
Prtaddr     = 0x7f<<0,  /* port address of recipient */
                /* transaction translator */
Hubaddr     = 0x7f<<7,  /* dev address of transaction */
                /* translator's hub */
    OHubaddr    = 7,
Xactpos     = 0x3<<14,  /* payload's position within transaction */
    POS_MID     = 0<<14,
    POS_END     = 1<<14,
    POS_BEGIN   = 2<<14,
    POS_ALL     = 3<<14, /* all of data (<= 188 bytes) */
Compsplt    = 1<<16,    /* do complete split */
Spltena     = 1<<31,    /* channel enabled to do splits */

/* hcint, hcintmsk */
Xfercomp    = 1<<0,     /* transfer completed without error */
Chhltd      = 1<<1,     /* channel halted */
Ahberr      = 1<<2,     /* AHB dma error */
Stall       = 1<<3,
Nak     = 1<<4,
Ack     = 1<<5,
Nyet        = 1<<6,
Xacterr     = 1<<7, /* transaction error (crc, t/o, bit stuff, eop) */
Bblerr      = 1<<8,
Frmovrun    = 1<<9,
Datatglerr  = 1<<10,
Bna     = 1<<11,
Xcs_xact    = 1<<12,
Frm_list_roll   = 1<<13,

/* hctsiz */
Xfersize    = 0x7ffff<<0,   /* expected total bytes */
Pktcnt      = 0x3ff<<19,    /* expected number of packets */
    OPktcnt     = 19,
Pid     = 0x3<<29,  /* packet id for initial transaction */
    DATA0       = 0<<29,
    DATA1       = 2<<29,    /* sic */
    DATA2       = 1<<29,    /* sic */
    MDATA       = 3<<29,    /* (non-ctl ep) */
    SETUP       = 3<<29,    /* (ctl ep) */
Dopng       = 1<<31,    /* do PING protocol */

/* pcgcctl */
Stoppclk        = 1<<0,
Gatehclk        = 1<<1,
Pwrclmp         = 1<<2,
Rstpdwnmodule       = 1<<3,
Enbl_sleep_gating   = 1<<5,
Phy_in_sleep        = 1<<6,
```

```
    Deep_sleep      = 1<<7,
    Resetaftsusp        = 1<<8,
    Restoremode     = 1<<9,
    Enbl_extnd_hiber    = 1<<10,
    Extnd_hiber_pwrclmp = 1<<11,
    Extnd_hiber_switch  = 1<<12,
    Ess_reg_restored    = 1<<13,
    Prt_clk_sel     = 0x3<<14,
    Port_power      = 1<<16,
    Max_xcvrselect      = 0x3<<17,
    Max_termsel     = 1<<19,
    Mac_dev_addr        = 0x7f<<20,
    P2hd_dev_enum_spd   = 0x3<<27,
    P2hd_prt_spd        = 0x3<<29,
    If_dev_mode     = 1<<31,
};
```

E.29.5 buses/arm/devusb.c

⟨*enum* _anon_(buses/arm/devusb.c)(arm) 666⟩≡ (691b)

```
enum
{
    /* Qid numbers */
    Qdir = 0,       /* #u */
    Qusbdir,            /* #u/usb */
    Qctl,           /* #u/usb/ctl - control requests */

    Qep0dir,            /* #u/usb/ep0.0 - endpoint 0 dir */
    Qep0io,         /* #u/usb/ep0.0/data - endpoint 0 I/O */
    Qep0ctl,        /* #u/usb/ep0.0/ctl - endpoint 0 ctl. */
    Qep0dummy,      /* give 4 qids to each endpoint */

    Qepdir = 0,     /* (qid-qep0dir)&3 is one of these */
    Qepio,          /* to identify which file for the endpoint */
    Qepctl,

    /* ... */

    /* Usb ctls. */
    CMdebug = 0,        /* debug on|off */
    CMdump,         /* dump (data structures for debug) */

    /* Ep. ctls */
    CMnew = 0,      /* new nb ctl|bulk|intr|iso r|w|rw (endpoint) */
    CMnewdev,       /* newdev full|low|high portnb (allocate new devices) */
    CMhub,          /* hub (set the device as a hub) */
    CMspeed,        /* speed full|low|high|no */
    CMmaxpkt,       /* maxpkt size */
    CMntds,         /* ntds nb (max nb. of tds per µframe) */
    CMclrhalt,      /* clrhalt (halt was cleared on endpoint) */
    CMpollival,     /* pollival interval (interrupt/iso) */
    CMhz,           /* hz n (samples/sec; iso) */
    CMsamplesz,     /* samplesz n (sample size; iso) */
    CMinfo,         /* info infostr (ke.ep info for humans) */
    CMdetach,       /* detach (abort I/O forever on this ep). */
    CMaddress,      /* address (address is assigned) */
    CMdebugep,      /* debug n (set/clear debug for this ep) */
    CMname,         /* name str (show up as #u/name as well) */
    CMtmout,        /* timeout n (activate timeouts for ep) */
```

```
    CMpreset,       /* reset the port */

    /* Hub feature selectors */
    Rportenable = 1,
    Rportreset  = 4,

};
```

⟨*struct* Hcitype*(arm)* 667a⟩≡ (691b)

```
struct Hcitype
{
    char*   type;
    int (*reset)(Hci*);
};
```

⟨*macro* QID*(arm)* 667b⟩≡ (691b)

```
#define QID(q)  ((int)(q).path)
```

⟨*global* usbctls*(arm)* 667c⟩≡ (691b)

```
static Cmdtab usbctls[] =
{
    {CMdebug,   "debug",    2},
    {CMdump,    "dump",     1},
};
```

⟨*global* epctls*(arm)* 667d⟩≡ (691b)

```
static Cmdtab epctls[] =
{
    {CMnew,     "new",      4},
    {CMnewdev,  "newdev",   3},
    {CMhub,     "hub",      1},
    {CMspeed,   "speed",    2},
    {CMmaxpkt,  "maxpkt",   2},
    {CMntds,    "ntds",     2},
    {CMpollival,    "pollival", 2},
    {CMsamplesz,    "samplesz", 2},
    {CMhz,      "hz",       2},
    {CMinfo,    "info",     0},
    {CMdetach,  "detach",   1},
    {CMaddress, "address",  1},
    {CMdebugep, "debug",    2},
    {CMclrhalt, "clrhalt",  1},
    {CMname,    "name",     2},
    {CMtmout,   "timeout",  2},
    {CMpreset,  "reset",    1},
};
```

⟨*global* usbdir*(arm)* 667e⟩≡ (691b)

```
static Dirtab usbdir[] =
{
    "ctl",      {Qctl},     0,  0666,
};
```

⟨*global* usbmodename*(arm)* 667f⟩≡ (691b)

```
char *usbmodename[] =
{
    [OREAD] "r",
    [OWRITE]    "w",
    [ORDWR] "rw",
};
```

⟨*global* ttname*(arm)* 668a⟩≡ (691b)

```
static char *ttname[] =
{
    [Tnone] "none",
    [Tctl]  "control",
    [Tiso]  "iso",
    [Tintr] "interrupt",
    [Tbulk] "bulk",
};
```

⟨*global* spname*(arm)* 668b⟩≡ (691b)

```
static char *spname[] =
{
    [Fullspeed] "full",
    [Lowspeed]  "low",
    [Highspeed] "high",
    [Nospeed]   "no",
};
```

⟨*global* debug*(arm)* 668c⟩≡ (691b)

```
static int  debug;
```

⟨*global* hcitypes*(arm)* 668d⟩≡ (691b)

```
static Hcitype  hcitypes[Nhcis];
```

⟨*global* hcis*(arm)* 668e⟩≡ (691b)

```
static Hci* hcis[Nhcis];
```

⟨*global* epslck*(arm)* 668f⟩≡ (691b)

```
static QLock    epslck;     /* add, del, lookup endpoints */
```

⟨*global* eps*(arm)* 668g⟩≡ (691b)

```
static Ep*  eps[Neps];  /* all endpoints known */
```

⟨*global* epmax*(arm)* 668h⟩≡ (691b)

```
static int  epmax;      /* 1 + last endpoint index used  */
```

⟨*global* usbidgen*(arm)* 668i⟩≡ (691b)

```
static int  usbidgen;   /* device address generator */
```

⟨*function* seprintdata*(arm)* 668j⟩≡ (691b)

```
/*
 * Is there something like this in a library? should it be?
 */
char*
seprintdata(char *s, char *se, uchar *d, int n)
{
    int i, l;

    s = seprint(s, se, " %#p[%d]: ", d, n);
    l = n;
    if(l > 10)
        l = 10;
    for(i=0; i<l; i++)
        s = seprint(s, se, " %2.2ux", d[i]);
    if(l < n)
        s = seprint(s, se, "...");
    return s;
}
```

⟨*function* name2speed*(arm)* 669a⟩≡ (691b)

```
static int
name2speed(char *name)
{
    int i;

    for(i = 0; i < nelem(spname); i++)
        if(strcmp(name, spname[i]) == 0)
            return i;
    return Nospeed;
}
```

⟨*function* name2ttype*(arm)* 669b⟩≡ (691b)

```
static int
name2ttype(char *name)
{
    int i;

    for(i = 0; i < nelem(ttname); i++)
        if(strcmp(name, ttname[i]) == 0)
            return i;
    /* may be a std. USB ep. type */
    i = strtol(name, nil, 0);
    switch(i+1){
    case Tctl:
    case Tiso:
    case Tbulk:
    case Tintr:
        return i+1;
    default:
        return Tnone;
    }
}
```

⟨*function* name2mode*(arm)* 669c⟩≡ (691b)

```
static int
name2mode(char *mode)
{
    int i;

    for(i = 0; i < nelem(usbmodename); i++)
        if(strcmp(mode, usbmodename[i]) == 0)
            return i;
    return -1;
}
```

⟨*function* qid2epidx*(arm)* 669d⟩≡ (691b)

```
static int
qid2epidx(int q)
{
    q = (q-Qep0dir)/4;
    if(q < 0 || q >= epmax || eps[q] == nil)
        return -1;
    return q;
}
```

⟨*function* isqtype*(arm)* 669e⟩≡ (691b)

```
static int
isqtype(int q, int type)
{
```

```
    if(q < Qep0dir)
        return 0;
    q -= Qep0dir;
    return (q & 3) == type;
}
```

⟨*function* addhcitype*(arm)* 670a⟩≡ (691b)

```
void
addhcitype(char* t, int (*r)(Hci*))
{
    static int ntype;

    if(ntype == Nhcis)
        panic("too many USB host interface types");
    hcitypes[ntype].type = t;
    hcitypes[ntype].reset = r;
    ntype++;
}
```

⟨*function* seprintep*(arm)* 670b⟩≡ (691b)

```
static char*
seprintep(char *s, char *se, Ep *ep, int all)
{
    static char* dsnames[] = { "config", "enabled", "detached", "reset" };
    Udev *d;
    int i;
    int di;

    d = ep->dev;

    qlock(ep);
    if(waserror()){
        qunlock(ep);
        nexterror();
    }
    di = ep->dev->nb;
    if(all)
        s = seprint(s, se, "dev %d ep %d ", di, ep->nb);
    s = seprint(s, se, "%s", dsnames[ep->dev->state]);
    s = seprint(s, se, " %s", ttname[ep->ttype]);
    assert(ep->mode == OREAD || ep->mode == OWRITE || ep->mode == ORDWR);
    s = seprint(s, se, " %s", usbmodename[ep->mode]);
    s = seprint(s, se, " speed %s", spname[d->speed]);
    s = seprint(s, se, " maxpkt %ld", ep->maxpkt);
    s = seprint(s, se, " pollival %ld", ep->pollival);
    s = seprint(s, se, " samplesz %ld", ep->samplesz);
    s = seprint(s, se, " hz %ld", ep->hz);
    s = seprint(s, se, " hub %d", ep->dev->hub);
    s = seprint(s, se, " port %d", ep->dev->port);
    if(ep->inuse)
        s = seprint(s, se, " busy");
    else
        s = seprint(s, se, " idle");
    if(all){
        s = seprint(s, se, " load %uld", ep->load);
        s = seprint(s, se, " ref %ld addr %#p", ep->ref, ep);
        s = seprint(s, se, " idx %d", ep->idx);
        if(ep->name != nil)
            s = seprint(s, se, " name '%s'", ep->name);
        if(ep->tmout != 0)
```

```
            s = seprint(s, se, " tmout");
        if(ep == ep->ep0){
            s = seprint(s, se, " ctlrno %#x", ep->hp->ctlrno);
            s = seprint(s, se, " eps:");
            for(i = 0; i < nelem(d->eps); i++)
                if(d->eps[i] != nil)
                    s = seprint(s, se, " ep%d.%d", di, i);
        }
    }
    if(ep->info != nil)
        s = seprint(s, se, "\n%s %s\n", ep->info, ep->hp->type);
    else
        s = seprint(s, se, "\n");
    qunlock(ep);
    poperror();
    return s;
}
```

⟨*function* epalloc*(arm)* 671a⟩≡ (691b)

```
static Ep*
epalloc(Hci *hp)
{
    Ep *ep;
    int i;

    ep = smalloc(sizeof(Ep));
    ep->ref = 1;
    qlock(&epslck);
    for(i = 0; i < Neps; i++)
        if(eps[i] == nil)
            break;
    if(i == Neps){
        qunlock(&epslck);
        free(ep);
        print("usb: bug: too few endpoints.\n");
        return nil;
    }
    ep->idx = i;
    if(epmax <= i)
        epmax = i+1;
    eps[i] = ep;
    ep->hp = hp;
    ep->maxpkt = 8;
    ep->ntds = 1;
    ep->samplesz = ep->pollival = ep->hz = 0; /* make them void */
    qunlock(&epslck);
    return ep;
}
```

⟨*function* getep*(arm)* 671b⟩≡ (691b)

```
static Ep*
getep(int i)
{
    Ep *ep;

    if(i < 0 || i >= epmax || eps[i] == nil)
        return nil;
    qlock(&epslck);
    ep = eps[i];
    if(ep != nil)
```

```
        incref(ep);
    qunlock(&epslck);
    return ep;
}
```

⟨*function* putep*(arm)* 672a⟩≡ (691b)

```
static void
putep(Ep *ep)
{
    Udev *d;

    if(ep != nil && decref(ep) == 0){
        d = ep->dev;
        deprint("usb: ep%d.%d %#p released\n", d->nb, ep->nb, ep);
        qlock(&epslck);
        eps[ep->idx] = nil;
        if(ep->idx == epmax-1)
            epmax--;
        if(ep == ep->ep0 && ep->dev != nil && ep->dev->nb == usbidgen)
            usbidgen--;
        qunlock(&epslck);
        if(d != nil){
            qlock(ep->ep0);
            d->eps[ep->nb] = nil;
            qunlock(ep->ep0);
        }
        if(ep->ep0 != ep){
            putep(ep->ep0);
            ep->ep0 = nil;
        }
        free(ep->info);
        free(ep->name);
        free(ep);
    }
}
```

⟨*function* dumpeps*(arm)* 672b⟩≡ (691b)

```
static void
dumpeps(void)
{
    int i;
    static char buf[512];
    char *s;
    char *e;
    Ep *ep;

    print("usb dump eps: epmax %d Neps %d (ref=1+ for dump):\n", epmax, Neps);
    for(i = 0; i < epmax; i++){
        s = buf;
        e = buf+sizeof(buf);
        ep = getep(i);
        if(ep != nil){
            if(waserror()){
                putep(ep);
                nexterror();
            }
            s = seprint(s, e, "ep%d.%d ", ep->dev->nb, ep->nb);
            seprintep(s, e, ep, 1);
            print("%s", buf);
            ep->hp->seprintep(buf, e, ep);
```

```
            print("%s", buf);
            poperror();
            putep(ep);
        }
    }
    print("usb dump hcis:\n");
    for(i = 0; i < Nhcis; i++)
        if(hcis[i] != nil)
            hcis[i]->dump(hcis[i]);
}
```

⟨*function* newusbid*(arm)* 673a⟩≡ (691b)

```
static int
newusbid(Hci *)
{
    int id;

    qlock(&epslck);
    id = ++usbidgen;
    if(id >= 0x7F)
        print("#u: too many device addresses; reuse them more\n");
    qunlock(&epslck);
    return id;
}
```

⟨*function* newdev*(arm)* 673b⟩≡ (691b)

```
/*
 * Create endpoint 0 for a new device
 */
static Ep*
newdev(Hci *hp, int ishub, int isroot)
{
    Ep *ep;
    Udev *d;

    ep = epalloc(hp);
    d = ep->dev = smalloc(sizeof(Udev));
    d->nb = newusbid(hp);
    d->eps[0] = ep;
    ep->nb = 0;
    ep->toggle[0] = ep->toggle[1] = 0;
    d->ishub = ishub;
    d->isroot = isroot;
    if(hp->highspeed != 0)
        d->speed = Highspeed;
    else
        d->speed = Fullspeed;
    d->state = Dconfig;     /* address not yet set */
    ep->dev = d;
    ep->ep0 = ep;           /* no ref counted here */
    ep->ttype = Tctl;
    ep->tmout = Xfertmout;
    ep->mode = ORDWR;
    dprint("newdev %#p ep%d.%d %#p\n", d, d->nb, ep->nb, ep);
    return ep;
}
```

⟨*function* newdevep*(arm)* 673c⟩≡ (691b)

```
/*
 * Create a new endpoint for the device
```

```
 * accessed via the given endpoint 0.
 */
static Ep*
newdevep(Ep *ep, int i, int tt, int mode)
{
    Ep *nep;
    Udev *d;

    d = ep->dev;
    if(d->eps[i] != nil)
        error("endpoint already in use");
    nep = epalloc(ep->hp);
    incref(ep);
    d->eps[i] = nep;
    nep->nb = i;
    nep->toggle[0] = nep->toggle[1] = 0;
    nep->ep0 = ep;
    nep->dev = ep->dev;
    nep->mode = mode;
    nep->ttype = tt;
    nep->debug = ep->debug;
    /* set defaults */
    switch(tt){
    case Tctl:
        nep->tmout = Xfertmout;
        break;
    case Tintr:
        nep->pollival = 10;
        break;
    case Tiso:
        nep->tmout = Xfertmout;
        nep->pollival = 10;
        nep->samplesz = 4;
        nep->hz = 44100;
        break;
    }
    deprint("newdevep ep%d.%d %#p\n", d->nb, nep->nb, nep);
    return ep;
}
```

⟨*function* epdataperm*(arm)* 674a⟩≡ (691b)

```
static int
epdataperm(int mode)
{

    switch(mode){
    case OREAD:
        return 0440|DMEXCL;
        break;
    case OWRITE:
        return 0220|DMEXCL;
        break;
    default:
        return 0660|DMEXCL;
    }
}
```

⟨*function* usbgen*(arm)* 674b⟩≡ (691b)

```
static int
usbgen(Chan *c, char *, Dirtab*, int, int s, Dir *dp)
```

```
{
    Qid q;
    Dirtab *dir;
    int perm;
    char *se;
    Ep *ep;
    int nb;
    int mode;

    if(0)ddprint("usbgen q %#x s %d...", QID(c->qid), s);
    if(s == DEVDOTDOT){
        if(QID(c->qid) <= Qusbdir){
            mkqid(&q, Qdir, 0, QTDIR);
            devdir(c, q, "#u", 0, eve, 0555, dp);
        }else{
            mkqid(&q, Qusbdir, 0, QTDIR);
            devdir(c, q, "usb", 0, eve, 0555, dp);
        }
        if(0)ddprint("ok\n");
        return 1;
    }

    switch(QID(c->qid)){
    case Qdir:              /* list #u */
        if(s == 0){
            mkqid(&q, Qusbdir, 0, QTDIR);
            devdir(c, q, "usb", 0, eve, 0555, dp);
            if(0)ddprint("ok\n");
            return 1;
        }
        s--;
        if(s < 0 || s >= epmax)
            goto Fail;
        ep = getep(s);
        if(ep == nil || ep->name == nil){
            if(ep != nil)
                putep(ep);
            if(0)ddprint("skip\n");
            return 0;
        }
        if(waserror()){
            putep(ep);
            nexterror();
        }
        mkqid(&q, Qep0io+s*4, 0, QTFILE);
        devdir(c, q, ep->name, 0, eve, epdataperm(ep->mode), dp);
        putep(ep);
        poperror();
        if(0)ddprint("ok\n");
        return 1;

    case Qusbdir:               /* list #u/usb */
    Usbdir:
        if(s < nelem(usbdir)){
            dir = &usbdir[s];
            mkqid(&q, dir->qid.path, 0, QTFILE);
            devdir(c, q, dir->name, dir->length, eve, dir->perm, dp);
            if(0)ddprint("ok\n");
            return 1;
        }
```

```
        s -= nelem(usbdir);
        if(s < 0 || s >= epmax)
            goto Fail;
        ep = getep(s);
        if(ep == nil){
            if(0)ddprint("skip\n");
            return 0;
        }
        if(waserror()){
            putep(ep);
            nexterror();
        }
        se = up->genbuf+sizeof(up->genbuf);
        seprint(up->genbuf, se, "ep%d.%d", ep->dev->nb, ep->nb);
        mkqid(&q, Qep0dir+4*s, 0, QTDIR);
        putep(ep);
        poperror();
        devdir(c, q, up->genbuf, 0, eve, 0755, dp);
        if(0)ddprint("ok\n");
        return 1;

    case Qctl:
        s = 0;
        goto Usbdir;

    default:                /* list #u/usb/epN.M */
        nb = qid2epidx(QID(c->qid));
        ep = getep(nb);
        if(ep == nil)
            goto Fail;
        mode = ep->mode;
        putep(ep);
        if(isqtype(QID(c->qid), Qepdir)){
        Epdir:
            switch(s){
            case 0:
                mkqid(&q, Qep0io+nb*4, 0, QTFILE);
                perm = epdataperm(mode);
                devdir(c, q, "data", 0, eve, perm, dp);
                break;
            case 1:
                mkqid(&q, Qep0ctl+nb*4, 0, QTFILE);
                devdir(c, q, "ctl", 0, eve, 0664, dp);
                break;
            default:
                goto Fail;
            }
        }else if(isqtype(QID(c->qid), Qepctl)){
            s = 1;
            goto Epdir;
        }else{
            s = 0;
            goto Epdir;
        }
        if(0)ddprint("ok\n");
        return 1;
    }
Fail:
    if(0)ddprint("fail\n");
    return -1;
```

```
}
```

⟨*function* hciprobe*(arm)* 677a⟩≡ (691b)

```
static Hci*
hciprobe(int cardno, int ctlrno)
{
    Hci *hp;
    char *type;
    char name[64];
    static int epnb = 1;    /* guess the endpoint nb. for the controller */

    ddprint("hciprobe %d %d\n", cardno, ctlrno);
    hp = smalloc(sizeof(Hci));
    hp->ctlrno = ctlrno;

    if(cardno < 0)
        for(cardno = 0; cardno < Nhcis; cardno++){
            if(hcitypes[cardno].type == nil)
                break;
            type = hp->type;
            if(type==nil || *type==0)
                type = "uhci";
            if(cistrcmp(hcitypes[cardno].type, type) == 0)
                break;
        }

    if(cardno >= Nhcis || hcitypes[cardno].type == nil){
        free(hp);
        return nil;
    }
    dprint("%s...", hcitypes[cardno].type);
    if(hcitypes[cardno].reset(hp) < 0){
        free(hp);
        return nil;
    }

    snprint(name, sizeof(name), "usb%s", hcitypes[cardno].type);
    arch_intrenable(hp->irq, hp->interrupt, hp, UNKNOWN, name);

    print("#u/usb/ep%d.0: %s: port %#luX irq %d\n",
        epnb, hcitypes[cardno].type, hp->port, hp->irq);
    epnb++;

    return hp;
}
```

⟨*function* usbreset*(arm)* 677b⟩≡ (691b)

```
static void
usbreset(void)
{
    int cardno, ctlrno;
    Hci *hp;

    dprint("usbreset\n");

    for(ctlrno = 0; ctlrno < Nhcis; ctlrno++)
        if((hp = hciprobe(-1, ctlrno)) != nil)
            hcis[ctlrno] = hp;
    cardno = ctlrno = 0;
    while(cardno < Nhcis && ctlrno < Nhcis && hcitypes[cardno].type != nil)
```

```
        if(hcis[ctlrno] != nil)
            ctlrno++;
        else{
            hp = hciprobe(cardno, ctlrno);
            if(hp == nil)
                cardno++;
            hcis[ctlrno++] = hp;
        }
    if(hcis[Nhcis-1] != nil)
        print("usbreset: bug: Nhcis too small\n");
}
```

⟨*function* usbinit*(arm)* 678a⟩≡ (691b)

```
static void
usbinit(void)
{
    Hci *hp;
    int ctlrno;
    Ep *d;
    char info[40];

    dprint("usbinit\n");
    for(ctlrno = 0; ctlrno < Nhcis; ctlrno++){
        hp = hcis[ctlrno];
        if(hp != nil){
            if(hp->init != nil)
                hp->init(hp);
            d = newdev(hp, 1, 1);       /* new root hub */
            d->dev->state = Denabled;   /* although addr == 0 */
            d->maxpkt = 64;
            snprint(info, sizeof(info), "ports %d", hp->nports);
            kstrdup(&d->info, info);
        }
    }
}
```

⟨*function* usbattach*(arm)* 678b⟩≡ (691b)

```
static Chan*
usbattach(char *spec)
{
    return devattach(L'u', spec);
}
```

⟨*function* usbwalk*(arm)* 678c⟩≡ (691b)

```
static Walkqid*
usbwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, nil, 0, usbgen);
}
```

⟨*function* usbstat*(arm)* 678d⟩≡ (691b)

```
static int
usbstat(Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, nil, 0, usbgen);
}
```

⟨*function* usbload*(arm)* 679a⟩≡ (691b)

```
/*
 * µs for the given transfer, for bandwidth allocation.
 * This is a very rough worst case for what 5.11.3
 * of the usb 2.0 spec says.
 * Also, we are using maxpkt and not actual transfer sizes.
 * Only when we are sure we
 * are not exceeding b/w might we consider adjusting it.
 */
static ulong
usbload(int speed, int maxpkt)
{
    enum{ Hostns = 1000, Hubns = 333 };
    ulong l;
    ulong bs;

    l = 0;
    bs = 10UL * maxpkt;
    switch(speed){
    case Highspeed:
        l = 55*8*2 + 2 * (3 + bs) + Hostns;
        break;
    case Fullspeed:
        l = 9107 + 84 * (4 + bs) + Hostns;
        break;
    case Lowspeed:
        l = 64107 + 2 * Hubns + 667 * (3 + bs) + Hostns;
        break;
    default:
        print("usbload: bad speed %d\n", speed);
        /* let it run */
    }
    return l / 1000UL;  /* in µs */
}
```

⟨*function* usbopen*(arm)* 679b⟩≡ (691b)

```
static Chan*
usbopen(Chan *c, int omode)
{
    int q;
    Ep *ep;
    int mode;

    mode = openmode(omode);
    q = QID(c->qid);

    if(q >= Qep0dir && qid2epidx(q) < 0)
        error(Eio);
    if(q < Qep0dir || isqtype(q, Qepctl) || isqtype(q, Qepdir))
        return devopen(c, omode, nil, 0, usbgen);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    deprint("usbopen q %#x fid %d omode %d\n", q, c->fid, mode);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    qlock(ep);
```

```
    if(ep->inuse){
        qunlock(ep);
        error(Einuse);
    }
    ep->inuse = 1;
    qunlock(ep);
    if(waserror()){
        ep->inuse = 0;
        nexterror();
    }
    if(mode != OREAD && ep->mode == OREAD)
        error(Eperm);
    if(mode != OWRITE && ep->mode == OWRITE)
        error(Eperm);
    if(ep->ttype == Tnone)
        error(Enotconf);
    ep->clrhalt = 0;
    ep->rhrepl = -1;
    if(ep->load == 0)
        ep->load = usbload(ep->dev->speed, ep->maxpkt);
    ep->hp->epopen(ep);

    poperror(); /* ep->inuse */
    poperror(); /* don't putep(): ref kept for fid using the ep. */

    c->mode = mode;
    c->flag |= COPEN;
    c->offset = 0;
    c->aux = nil;   /* paranoia */
    return c;
}
```

⟨*function* epclose*(arm)* 680a⟩≡ (691b)

```
static void
epclose(Ep *ep)
{
    qlock(ep);
    if(waserror()){
        qunlock(ep);
        nexterror();
    }
    if(ep->inuse){
        ep->hp->epclose(ep);
        ep->inuse = 0;
    }
    qunlock(ep);
    poperror();
}
```

⟨*function* usbclose*(arm)* 680b⟩≡ (691b)

```
static void
usbclose(Chan *c)
{
    int q;
    Ep *ep;

    q = QID(c->qid);
    if(q < Qep0dir || isqtype(q, Qepctl) || isqtype(q, Qepdir))
        return;
```

```
    ep = getep(qid2epidx(q));
    if(ep == nil)
        return;
    deprint("usbclose q %#x fid %d ref %ld\n", q, c->fid, ep->ref);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(c->flag & COPEN){
        free(c->aux);
        c->aux = nil;
        epclose(ep);
        putep(ep);  /* release ref kept since usbopen */
        c->flag &= ~COPEN;
    }
    poperror();
    putep(ep);
}
```

⟨*function* ctlread*(arm)* 681⟩≡ (691b)

```
static long
ctlread(Chan *c, void *a, long n, vlong offset)
{
    int q;
    char *s;
    char *us;
    char *se;
    Ep *ep;
    int i;

    q = QID(c->qid);
    us = s = smalloc(READSTR);
    se = s + READSTR;
    if(waserror()){
        free(us);
        nexterror();
    }
    if(q == Qctl)
        for(i = 0; i < epmax; i++){
            ep = getep(i);
            if(ep != nil){
                if(waserror()){
                    putep(ep);
                    nexterror();
                }
                s = seprint(s, se, "ep%d.%d ", ep->dev->nb, ep->nb);
                s = seprintep(s, se, ep, 0);
                poperror();
            }
            putep(ep);
        }
    else{
        ep = getep(qid2epidx(q));
        if(ep == nil)
            error(Eio);
        if(waserror()){
            putep(ep);
            nexterror();
        }
        if(c->aux != nil){
```

```
            /* After a new endpoint request we read
             * the new endpoint name back.
             */
            strecpy(s, se, c->aux);
            free(c->aux);
            c->aux = nil;
        }else
            seprintep(s, se, ep, 0);
        poperror();
        putep(ep);
    }
    n = readstr(offset, a, n, us);
    poperror();
    free(us);
    return n;
}
```

⟨*function* rhubread*(arm)* 682a⟩≡ (691b)

```
/*
 * Fake root hub emulation.
 */
static long
rhubread(Ep *ep, void *a, long n)
{
    char *b;

    if(ep->dev->isroot == 0 || ep->nb != 0 || n < 2)
        return -1;
    if(ep->rhrepl < 0)
        return -1;

    b = a;
    memset(b, 0, n);
    PUT2(b, ep->rhrepl);
    ep->rhrepl = -1;
    return n;
}
```

⟨*function* rhubwrite*(arm)* 682b⟩≡ (691b)

```
static long
rhubwrite(Ep *ep, void *a, long n)
{
    uchar *s;
    int cmd;
    int feature;
    int port;
    Hci *hp;

    if(ep->dev == nil || ep->dev->isroot == 0 || ep->nb != 0)
        return -1;
    if(n != Rsetuplen)
        error("root hub is a toy hub");
    ep->rhrepl = -1;
    s = a;
    if(s[Rtype] != (Rh2d|Rclass|Rother) && s[Rtype] != (Rd2h|Rclass|Rother))
        error("root hub is a toy hub");
    hp = ep->hp;
    cmd = s[Rreq];
    feature = GET2(s+Rvalue);
    port = GET2(s+Rindex);
```

```
    if(port < 1 || port > hp->nports)
        error("bad hub port number");
    switch(feature){
    case Rportenable:
        ep->rhrepl = hp->portenable(hp, port, cmd == Rsetfeature);
        break;
    case Rportreset:
        ep->rhrepl = hp->portreset(hp, port, cmd == Rsetfeature);
        break;
    case Rgetstatus:
        ep->rhrepl = hp->portstatus(hp, port);
        break;
    default:
        ep->rhrepl = 0;
    }
    return n;
}
```

⟨*function* usbread*(arm)* 683⟩≡ (691b)

```
static long
usbread(Chan *c, void *a, long n, vlong offset)
{
    int q;
    Ep *ep;
    int nr;

    q = QID(c->qid);

    if(c->qid.type == QTDIR)
        return devdirread(c, a, n, nil, 0, usbgen);

    if(q == Qctl || isqtype(q, Qepctl))
        return ctlread(c, a, n, offset);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(ep->dev->state == Ddetach)
        error(Edetach);
    if(ep->mode == OWRITE || ep->inuse == 0)
        error(Ebadusefd);
    switch(ep->ttype){
    case Tnone:
        error("endpoint not configured");
    case Tctl:
        nr = rhubread(ep, a, n);
        if(nr >= 0){
            n = nr;
            break;
        }
        /* else fall */
    default:
        ddeprint("\nusbread q %#x fid %d cnt %ld off %lld\n",q,c->fid,n,offset);
        n = ep->hp->epread(ep, a, n);
        break;
    }
```

```
    poperror();
    putep(ep);
    return n;
}
```

⟨*function* pow2*(arm)* 684a⟩≡ (691b)

```
static long
pow2(int n)
{
    return 1 << n;
}
```

⟨*function* setmaxpkt*(arm)* 684b⟩≡ (691b)

```
static void
setmaxpkt(Ep *ep, char* s)
{
    long spp;   /* samples per packet */

    if(ep->dev->speed == Highspeed)
        spp = (ep->hz * ep->pollival * ep->ntds + 7999) / 8000;
    else
        spp = (ep->hz * ep->pollival + 999) / 1000;
    ep->maxpkt = spp * ep->samplesz;
    deprint("usb: %s: setmaxpkt: hz %ld poll %ld"
        " ntds %d %s speed -> spp %ld maxpkt %ld\n", s,
        ep->hz, ep->pollival, ep->ntds, spname[ep->dev->speed],
        spp, ep->maxpkt);
    if(ep->maxpkt > 1024){
        print("usb: %s: maxpkt %ld > 1024. truncating\n", s, ep->maxpkt);
        ep->maxpkt = 1024;
    }
}
```

⟨*function* epctl*(arm)* 684c⟩≡ (691b)

```
/*
 * Many endpoint ctls. simply update the portable representation
 * of the endpoint. The actual controller driver will look
 * at them to setup the endpoints as dictated.
 */
static long
epctl(Ep *ep, Chan *c, void *a, long n)
{
    int i, l, mode, nb, tt;
    char *b, *s;
    Cmdbuf *cb;
    Cmdtab *ct;
    Ep *nep;
    Udev *d;
    static char *Info = "info ";

    d = ep->dev;

    cb = parsecmd(a, n);
    if(waserror()){
        free(cb);
        nexterror();
    }
    ct = lookupcmd(cb, epctls, nelem(epctls));
    if(ct == nil)
        error(Ebadctl);
```

```
i = ct->index;
if(i == CMnew || i == CMspeed || i == CMhub || i == CMpreset)
    if(ep != ep->ep0)
        error("allowed only on a setup endpoint");
if(i != CMclrhalt && i != CMdetach && i != CMdebugep && i != CMname)
    if(ep != ep->ep0 && ep->inuse != 0)
        error("must configure before using");
switch(i){
case CMnew:
    deprint("usb epctl %s\n", cb->f[0]);
    nb = strtol(cb->f[1], nil, 0);
    if(nb < 0 || nb >= Ndeveps)
        error("bad endpoint number");
    tt = name2ttype(cb->f[2]);
    if(tt == Tnone)
        error("unknown endpoint type");
    mode = name2mode(cb->f[3]);
    if(mode < 0)
        error("unknown i/o mode");
    newdevep(ep, nb, tt, mode);
    break;
case CMnewdev:
    deprint("usb epctl %s\n", cb->f[0]);
    if(ep != ep->ep0 || d->ishub == 0)
        error("not a hub setup endpoint");
    l = name2speed(cb->f[1]);
    if(l == Nospeed)
        error("speed must be full|low|high");
    nep = newdev(ep->hp, 0, 0);
    nep->dev->speed = l;
    if(nep->dev->speed  != Lowspeed)
        nep->maxpkt = 64;   /* assume full speed */
    nep->dev->hub = d->nb;
    nep->dev->port = atoi(cb->f[2]);
    /* next read request will read
     * the name for the new endpoint
     */
    l = sizeof(up->genbuf);
    snprint(up->genbuf, l, "ep%d.%d", nep->dev->nb, nep->nb);
    kstrdup(&c->aux, up->genbuf);
    break;
case CMhub:
    deprint("usb epctl %s\n", cb->f[0]);
    d->ishub = 1;
    break;
case CMspeed:
    l = name2speed(cb->f[1]);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l == Nospeed)
        error("speed must be full|low|high");
    qlock(ep->ep0);
    d->speed = l;
    qunlock(ep->ep0);
    break;
case CMmaxpkt:
    l = strtoul(cb->f[1], nil, 0);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l < 1 || l > 1024)
        error("maxpkt not in [1:1024]");
    qlock(ep);
```

```
        ep->maxpkt = l;
        qunlock(ep);
        break;
    case CMntds:
        l = strtoul(cb->f[1], nil, 0);
        deprint("usb epctl %s %d\n", cb->f[0], l);
        if(l < 1 || l > 3)
            error("ntds not in [1:3]");
        qlock(ep);
        ep->ntds = l;
        qunlock(ep);
        break;
    case CMpollival:
        if(ep->ttype != Tintr && ep->ttype != Tiso)
            error("not an intr or iso endpoint");
        l = strtoul(cb->f[1], nil, 0);
        deprint("usb epctl %s %d\n", cb->f[0], l);
        if(ep->ttype == Tiso ||
           (ep->ttype == Tintr && ep->dev->speed == Highspeed)){
            if(l < 1 || l > 16)
                error("pollival power not in [1:16]");
            l = pow2(l-1);
        }else
            if(l < 1 || l > 255)
                error("pollival not in [1:255]");
        qlock(ep);
        ep->pollival = l;
        if(ep->ttype == Tiso)
            setmaxpkt(ep, "pollival");
        qunlock(ep);
        break;
    case CMsamplesz:
        if(ep->ttype != Tiso)
            error("not an iso endpoint");
        l = strtoul(cb->f[1], nil, 0);
        deprint("usb epctl %s %d\n", cb->f[0], l);
        if(l <= 0 || l > 8)
            error("samplesz not in [1:8]");
        qlock(ep);
        ep->samplesz = l;
        setmaxpkt(ep, "samplesz");
        qunlock(ep);
        break;
    case CMhz:
        if(ep->ttype != Tiso)
            error("not an iso endpoint");
        l = strtoul(cb->f[1], nil, 0);
        deprint("usb epctl %s %d\n", cb->f[0], l);
        if(l <= 0 || l > 100000)
            error("hz not in [1:100000]");
        qlock(ep);
        ep->hz = l;
        setmaxpkt(ep, "hz");
        qunlock(ep);
        break;
    case CMclrhalt:
        qlock(ep);
        deprint("usb epctl %s\n", cb->f[0]);
        ep->clrhalt = 1;
        qunlock(ep);
```

```
        break;
    case CMinfo:
        deprint("usb epctl %s\n", cb->f[0]);
        l = strlen(Info);
        s = a;
        if(n < l+2 || strncmp(Info, s, l) != 0)
            error(Ebadctl);
        if(n > 1024)
            n = 1024;
        b = smalloc(n);
        memmove(b, s+l, n-l);
        b[n-l] = 0;
        if(b[n-l-1] == '\n')
            b[n-l-1] = 0;
        qlock(ep);
        free(ep->info);
        ep->info = b;
        qunlock(ep);
        break;
    case CMaddress:
        deprint("usb epctl %s\n", cb->f[0]);
        ep->dev->state = Denabled;
        break;
    case CMdetach:
        if(ep->dev->isroot != 0)
            error("can't detach a root hub");
        deprint("usb epctl %s ep%d.%d\n",
            cb->f[0], ep->dev->nb, ep->nb);
        ep->dev->state = Ddetach;
        /* Release file system ref. for its endpoints */
        for(i = 0; i < nelem(ep->dev->eps); i++)
            putep(ep->dev->eps[i]);
        break;
    case CMdebugep:
        if(strcmp(cb->f[1], "on") == 0)
            ep->debug = 1;
        else if(strcmp(cb->f[1], "off") == 0)
            ep->debug = 0;
        else
            ep->debug = strtoul(cb->f[1], nil, 0);
        print("usb: ep%d.%d debug %d\n",
            ep->dev->nb, ep->nb, ep->debug);
        break;
    case CMname:
        deprint("usb epctl %s %s\n", cb->f[0], cb->f[1]);
        validname(cb->f[1], 0);
        kstrdup(&ep->name, cb->f[1]);
        break;
    case CMtmout:
        deprint("usb epctl %s\n", cb->f[0]);
        if(ep->ttype == Tiso || ep->ttype == Tctl)
            error("ctl ignored for this endpoint type");
        ep->tmout = strtoul(cb->f[1], nil, 0);
        if(ep->tmout != 0 && ep->tmout < Xfertmout)
            ep->tmout = Xfertmout;
        break;
    case CMpreset:
        deprint("usb epctl %s\n", cb->f[0]);
        if(ep->ttype != Tctl)
            error("not a control endpoint");
```

```
        if(ep->dev->state != Denabled)
            error("forbidden on devices not enabled");
        ep->dev->state = Dreset;
        break;
    default:
        panic("usb: unknown epctl %d", ct->index);
    }
    free(cb);
    poperror();
    return n;
}
```

⟨*function* usbctl*(arm)* 688a⟩≡ (691b)

```
static long
usbctl(void *a, long n)
{
    Cmdtab *ct;
    Cmdbuf *cb;
    Ep *ep;
    int i;

    cb = parsecmd(a, n);
    if(waserror()){
        free(cb);
        nexterror();
    }
    ct = lookupcmd(cb, usbctls, nelem(usbctls));
    dprint("usb ctl %s\n", cb->f[0]);
    switch(ct->index){
    case CMdebug:
        if(strcmp(cb->f[1], "on") == 0)
            debug = 1;
        else if(strcmp(cb->f[1], "off") == 0)
            debug = 0;
        else
            debug = strtol(cb->f[1], nil, 0);
        print("usb: debug %d\n", debug);
        for(i = 0; i < epmax; i++)
            if((ep = getep(i)) != nil){
                ep->hp->debug(ep->hp, debug);
                putep(ep);
            }
        break;
    case CMdump:
        dumpeps();
        break;
    }
    free(cb);
    poperror();
    return n;
}
```

⟨*function* ctlwrite*(arm)* 688b⟩≡ (691b)

```
static long
ctlwrite(Chan *c, void *a, long n)
{
    int q;
    Ep *ep;

    q = QID(c->qid);
```

```
    if(q == Qctl)
        return usbctl(a, n);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(ep->dev->state == Ddetach)
        error(Edetach);
    if(isqtype(q, Qepctl) && c->aux != nil){
        /* Be sure we don't keep a cloned ep name */
        free(c->aux);
        c->aux = nil;
        error("read, not write, expected");
    }
    n = epctl(ep, c, a, n);
    putep(ep);
    poperror();
    return n;
}
```

⟨*function* usbwrite*(arm)* 689⟩≡ (691b)

```
static long
usbwrite(Chan *c, void *a, long n, vlong off)
{
    int nr, q;
    Ep *ep;

    if(c->qid.type == QTDIR)
        error(Eisdir);

    q = QID(c->qid);

    if(q == Qctl || isqtype(q, Qepctl))
        return ctlwrite(c, a, n);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(ep->dev->state == Ddetach)
        error(Edetach);
    if(ep->mode == OREAD || ep->inuse == 0)
        error(Ebadusefd);

    switch(ep->ttype){
    case Tnone:
        error("endpoint not configured");
    case Tctl:
        nr = rhubwrite(ep, a, n);
        if(nr >= 0){
            n = nr;
            break;
        }
```

```
        /* else fall */
    default:
        ddeprint("\nusbwrite q %#x fid %d cnt %ld off %lld\n",q, c->fid, n, off);
        ep->hp->epwrite(ep, a, n);
    }
    putep(ep);
    poperror();
    return n;
}
```

⟨*function* usbbread*(arm)* 690a⟩≡ (691b)

```
Block*
usbbread(Chan *c, long n, ulong offset)
{
    Block *bp;

    bp = allocb(n);
    if(bp == 0)
        error(Enomem);
    if(waserror()) {
        freeb(bp);
        nexterror();
    }
    bp->wp += usbread(c, bp->wp, n, offset);
    poperror();
    return bp;
}
```

⟨*function* usbbwrite*(arm)* 690b⟩≡ (691b)

```
long
usbbwrite(Chan *c, Block *bp, ulong offset)
{
    long n;

    if(waserror()) {
        freeb(bp);
        nexterror();
    }
    n = usbwrite(c, bp->rp, BLEN(bp), offset);
    poperror();
    freeb(bp);

    return n;
}
```

⟨*function* usbshutdown*(arm)* 690c⟩≡ (691b)

```
void
usbshutdown(void)
{
    Hci *hp;
    int i;

    for(i = 0; i < Nhcis; i++){
        hp = hcis[i];
        if(hp == nil)
            continue;
        if(hp->shutdown == nil)
            print("#u: no shutdown function for %s\n", hp->type);
        else
            hp->shutdown(hp);
```

```
    }
}
```

⟨*global* usbdevtab*(arm)* 691a⟩≡ (691b)

```
Dev usbdevtab = {
    .dc = L'u',
    .name = "usb",

    .reset = usbreset,
    .init = usbinit,
    .shutdown = usbshutdown,
    .attach = usbattach,
    .walk = usbwalk,
    .stat = usbstat,
    .open = usbopen,
    .create = devcreate,
    .close = usbclose,
    .read = usbread,
    .bread = usbbread,
    .write = usbwrite,
    .bwrite = usbbwrite,
    .remove = devremove,
    .wstat = devwstat,
};
```

⟨buses/arm/devusb.c 691b⟩≡

```
/*
 * USB device driver framework.
 *
 * This is in charge of providing access to actual HCIs
 * and providing I/O to the various endpoints of devices.
 * A separate user program (usbd) is in charge of
 * enumerating the bus, setting up endpoints and
 * starting devices (also user programs).
 *
 * The interface provided is a violation of the standard:
 * you're welcome.
 *
 * The interface consists of a root directory with several files
 * plus a directory (epN.M) with two files per endpoint.
 * A device is represented by its first endpoint, which
 * is a control endpoint automatically allocated for each device.
 * Device control endpoints may be used to create new endpoints.
 * Devices corresponding to hubs may also allocate new devices,
 * perhaps also hubs. Initially, a hub device is allocated for
 * each controller present, to represent its root hub. Those can
 * never be removed.
 *
 * All endpoints refer to the first endpoint (epN.0) of the device,
 * which keeps per-device information, and also to the HCI used
 * to reach them. Although all endpoints cache that information.
 *
 * epN.M/data files permit I/O and are considered DMEXCL.
 * epN.M/ctl files provide status info and accept control requests.
 *
 * Endpoints may be given file names to be listed also at #u,
 * for those drivers that have nothing to do after configuring the
 * device and its endpoints.
 *
 * Drivers for different controllers are kept at usb[oue]hci.c
```

```
 * It's likely we could factor out much from controllers into
 * a generic controller driver, the problem is that details
 * regarding how to handle toggles, tokens, Tds, etc. will
 * get in the way. Thus, code is probably easier the way it is.
 */
```
⟨*kernel basic includes* 23m⟩

```
#include    "io.h"
#include    "../port/usb.h"

typedef struct Hcitype Hcitype;
```

⟨*enum* `_anon_`*(*`(buses/arm/devusb.c)(arm)`*)* 666⟩

⟨*struct* `Hcitype`*(arm)* 667a⟩

⟨*macro* `QID`*(arm)* 667b⟩

⟨*global* `usbctls`*(arm)* 667c⟩

⟨*global* `epctls`*(arm)* 667d⟩

⟨*global* `usbdir`*(arm)* 667e⟩

⟨*global* `usbmodename`*(arm)* 667f⟩

⟨*global* `ttname`*(arm)* 668a⟩

⟨*global* `spname`*(arm)* 668b⟩

⟨*global* `debug`*(arm)* 668c⟩
⟨*global* `hcitypes`*(arm)* 668d⟩
⟨*global* `hcis`*(arm)* 668e⟩
⟨*global* `epslck`*(arm)* 668f⟩
⟨*global* `eps`*(arm)* 668g⟩
⟨*global* `epmax`*(arm)* 668h⟩
⟨*global* `usbidgen`*(arm)* 668i⟩

⟨*function* `seprintdata`*(arm)* 668j⟩

⟨*function* `name2speed`*(arm)* 669a⟩

⟨*function* `name2ttype`*(arm)* 669b⟩

⟨*function* `name2mode`*(arm)* 669c⟩

⟨*function* `qid2epidx`*(arm)* 669d⟩

⟨*function* `isqtype`*(arm)* 669e⟩

⟨*function* `addhcitype`*(arm)* 670a⟩

⟨*function* `seprintep`*(arm)* 670b⟩

⟨*function* `epalloc`*(arm)* 671a⟩

⟨*function* `getep`*(arm)* 671b⟩

⟨*function* `putep`*(arm)* 672a⟩

⟨*function* dumpeps*(arm)* 672b⟩

⟨*function* newusbid*(arm)* 673a⟩

⟨*function* newdev*(arm)* 673b⟩

⟨*function* newdevep*(arm)* 673c⟩

⟨*function* epdataperm*(arm)* 674a⟩

⟨*function* usbgen*(arm)* 674b⟩

⟨*function* hciprobe*(arm)* 677a⟩

⟨*function* usbreset*(arm)* 677b⟩

⟨*function* usbinit*(arm)* 678a⟩

⟨*function* usbattach*(arm)* 678b⟩

⟨*function* usbwalk*(arm)* 678c⟩

⟨*function* usbstat*(arm)* 678d⟩

⟨*function* usbload*(arm)* 679a⟩

⟨*function* usbopen*(arm)* 679b⟩

⟨*function* epclose*(arm)* 680a⟩

⟨*function* usbclose*(arm)* 680b⟩

⟨*function* ctlread*(arm)* 681⟩

⟨*function* rhubread*(arm)* 682a⟩

⟨*function* rhubwrite*(arm)* 682b⟩

⟨*function* usbread*(arm)* 683⟩

⟨*function* pow2*(arm)* 684a⟩

⟨*function* setmaxpkt*(arm)* 684b⟩

⟨*function* epctl*(arm)* 684c⟩

⟨*function* usbctl*(arm)* 688a⟩

⟨*function* ctlwrite*(arm)* 688b⟩

⟨*function* usbwrite*(arm)* 689⟩

⟨*function* usbbread*(arm)* 690a⟩

⟨*function* usbbwrite*(arm)* 690b⟩

⟨*function* usbshutdown*(arm)* 690c⟩

⟨*global* usbdevtab*(arm)* 691a⟩

E.29.6 buses/arm/usbdwc.c

⟨*enum* _anon_*(*(buses/arm/usbdwc.c)(arm)*)* 694a⟩≡ (711b)

```
enum
{
    Enabledelay = 50,
    Resetdelay  = 10,
    ResetdelayHS    = 50,

    Read        = 0,
    Write       = 1,
};
```

⟨*struct* Ctlr*(*(buses/arm/usbdwc.c)(arm)*)* 694b⟩≡ (711b)

```
struct Ctlr {
    Lock;
    Dwcregs *regs;      /* controller registers */
    int nchan;      /* number of host channels */
    ulong   chanbusy;   /* bitmap of in-use channels */
    QLock   chanlock;   /* serialise access to chanbusy */
    QLock   split;      /* serialise split transactions */
    int splitretry; /* count retries of Nyet */
    int sofchan;    /* bitmap of channels waiting for sof */
    int wakechan;   /* bitmap of channels to wakeup after fiq */
    int debugchan;  /* bitmap of channels for interrupt debug */
    Rendez  *chanintr;  /* sleep till interrupt on channel N */
};
```

⟨*struct* Epio*(arm)* 694c⟩≡ (711b)

```
struct Epio {
    QLock;
    Block   *cb;
    ulong   lastpoll;
};
```

⟨*global* dwc*(arm)* 694d⟩≡ (711b)

```
static Ctlr dwc;
```

⟨*global* debug*(*(buses/arm/usbdwc.c)(arm)*)* 694e⟩≡ (711b)

```
static int debug;
```

⟨*global* Ebadlen*(arm)* 694f⟩≡ (711b)

```
static char Ebadlen[] = "bad usb request length";
```

⟨*function* filock*(arm)* 694g⟩≡ (711b)

```
static void
filock(Lock *l)
{
    int x;

    x = splfhi();
    ilock(l);
    l->sr = x;
}
```

⟨*function* fiunlock*(arm)* 694h⟩≡ (711b)

```
static void
fiunlock(Lock *l)
{
    iunlock(l);
}
```

⟨*function* chanalloc*(arm)* 695a⟩≡ (711b)

```
static Hostchan*
chanalloc(Ep *ep)
{
    Ctlr *ctlr;
    int bitmap, i;

    ctlr = ep->hp->aux;
    qlock(&ctlr->chanlock);
    bitmap = ctlr->chanbusy;
    for(i = 0; i < ctlr->nchan; i++)
        if((bitmap & (1<<i)) == 0){
            ctlr->chanbusy = bitmap | 1<<i;
            qunlock(&ctlr->chanlock);
            return &ctlr->regs->hchan[i];
        }
    qunlock(&ctlr->chanlock);
    panic("miller is a lazy git");
    return nil;
}
```

⟨*function* chanrelease*(arm)* 695b⟩≡ (711b)

```
static void
chanrelease(Ep *ep, Hostchan *chan)
{
    Ctlr *ctlr;
    int i;

    ctlr = ep->hp->aux;
    i = chan - ctlr->regs->hchan;
    qlock(&ctlr->chanlock);
    ctlr->chanbusy &= ~(1<<i);
    qunlock(&ctlr->chanlock);
}
```

⟨*function* chansetup*(arm)* 695c⟩≡ (711b)

```
static void
chansetup(Hostchan *hc, Ep *ep)
{
    int hcc;
    Ctlr *ctlr = ep->hp->aux;

    if(ep->debug)
        ctlr->debugchan |= 1 << (hc - ctlr->regs->hchan);
    else
        ctlr->debugchan &= ~(1 << (hc - ctlr->regs->hchan));
    switch(ep->dev->state){
    case Dconfig:
    case Dreset:
        hcc = 0;
        break;
    default:
        hcc = ep->dev->nb<<ODevaddr;
        break;
    }
    hcc |= ep->maxpkt | 1<<OMulticnt | ep->nb<<OEpnum;
    switch(ep->ttype){
    case Tctl:
        hcc |= Epctl;
        break;
```

```
    case Tiso:
        hcc |= Episo;
        break;
    case Tbulk:
        hcc |= Epbulk;
        break;
    case Tintr:
        hcc |= Epintr;
        break;
    }
    switch(ep->dev->speed){
    case Lowspeed:
        hcc |= Lspddev;
        /* fall through */
    case Fullspeed:
        if(ep->dev->hub > 1){
            hc->hcsplt = Spltena | POS_ALL | ep->dev->hub<<OHubaddr |
                ep->dev->port;
            break;
        }
        /* fall through */
    default:
        hc->hcsplt = 0;
        break;
    }
    hc->hcchar = hcc;
    hc->hcint = ~0;
}
```

⟨*function* sofdone*(arm)* 696a⟩≡ (711b)

```
static int
sofdone(void *a)
{
    Dwcregs *r;

    r = a;
    return (r->gintmsk & Sofintr) == 0;
}
```

⟨*function* sofwait*(arm)* 696b⟩≡ (711b)

```
static void
sofwait(Ctlr *ctlr, int n)
{
    Dwcregs *r;

    r = ctlr->regs;
    do{
        filock(ctlr);
        r->gintsts = Sofintr;
        ctlr->sofchan |= 1<<n;
        r->gintmsk |= Sofintr;
        fiunlock(ctlr);
        sleep(&ctlr->chanintr[n], sofdone, r);
    }while((r->hfnum & 7) == 6);
}
```

⟨*function* chandone*(arm)* 696c⟩≡ (711b)

```
static int
chandone(void *a)
{
```

```
    Hostchan *hc;

    hc = a;
    if(hc->hcint == (Chhltd|Ack))
        return 0;
    return (hc->hcint & hc->hcintmsk) != 0;
}
```

⟨*function* chanwait*(arm)* 697⟩≡ (711b)

```
static int
chanwait(Ep *ep, Ctlr *ctlr, Hostchan *hc, int mask)
{
    int intr, n, ointr;
    ulong start, now;
    Dwcregs *r;

    r = ctlr->regs;
    n = hc - r->hchan;
    for(;;){
restart:
        filock(ctlr);
        r->haintmsk |= 1<<n;
        hc->hcintmsk = mask;
        fiunlock(ctlr);
        sleep(&ctlr->chanintr[n], chandone, hc);
        hc->hcintmsk = 0;
        intr = hc->hcint;
        if(intr & Chhltd)
            return intr;
        start = arch_fastticks(0);
        ointr = intr;
        now = start;
        do{
            intr = hc->hcint;
            if(intr & Chhltd){
                if((ointr != Ack && ointr != (Ack|Xfercomp)) ||
                   intr != (Ack|Chhltd|Xfercomp) ||
                   (now - start) > 60)
                    dprint("await %x after %ld %x -> %x\n",
                        mask, now - start, ointr, intr);
                return intr;
            }
            if((intr & mask) == 0){
                dprint("ep%d.%d await %x intr %x -> %x\n",
                    ep->dev->nb, ep->nb, mask, ointr, intr);
                goto restart;
            }
            now = arch_fastticks(0);
        }while(now - start < 100);
        dprint("ep%d.%d halting channel %8.8ux hcchar %8.8ux "
            "grxstsr %8.8ux gnptxsts %8.8ux hptxsts %8.8ux\n",
            ep->dev->nb, ep->nb, intr, hc->hcchar, r->grxstsr,
            r->gnptxsts, r->hptxsts);
        mask = Chhltd;
        hc->hcchar |= Chdis;
        start = cpu->ticks;
        while(hc->hcchar & Chen){
            if(cpu->ticks - start >= 100){
                print("ep%d.%d channel won't halt hcchar %8.8ux\n",
                    ep->dev->nb, ep->nb, hc->hcchar);
```

```
                break;
            }
        }
        logdump(ep);
    }
}
```

⟨*function* chanintr*(arm)* 698a⟩≡ (711b)

```
static int
chanintr(Ctlr *ctlr, int n)
{
    Hostchan *hc;
    int i;

    hc = &ctlr->regs->hchan[n];
    if(ctlr->debugchan & (1<<n))
        clog(nil, hc);
    if((hc->hcsplt & Spltena) == 0)
        return 0;
    i = hc->hcint;
    if(i == (Chhltd|Ack)){
        hc->hcsplt |= Compsplt;
        ctlr->splitretry = 0;
    }else if(i == (Chhltd|Nyet)){
        if(++ctlr->splitretry >= 3)
            return 0;
    }else
        return 0;
    if(hc->hcchar & Chen){
        iprint("hcchar %8.8ux hcint %8.8ux", hc->hcchar, hc->hcint);
        hc->hcchar |= Chen | Chdis;
        while(hc->hcchar&Chen)
            ;
        iprint(" %8.8ux\n", hc->hcint);
    }
    hc->hcint = i;
    if(ctlr->regs->hfnum & 1)
        hc->hcchar &= ~Oddfrm;
    else
        hc->hcchar |= Oddfrm;
    hc->hcchar = (hc->hcchar &~ Chdis) | Chen;
    return 1;
}
```

⟨*global* chanlog*(arm)* 698b⟩≡ (711b)

```
static Reg chanlog[32][5];
```

⟨*global* nchanlog*(arm)* 698c⟩≡ (711b)

```
static int nchanlog;
```

⟨*function* logstart*(arm)* 698d⟩≡ (711b)

```
static void
logstart(Ep *ep)
{
    if(ep->debug)
        nchanlog = 0;
}
```

⟨*function* clog*(arm)* 699a⟩≡ (711b)

```
static void
clog(Ep *ep, Hostchan *hc)
{
    Reg *p;

    if(ep != nil && !ep->debug)
        return;
    if(nchanlog == 32)
        nchanlog--;
    p = chanlog[nchanlog];
    p[0] = dwc.regs->hfnum;
    p[1] = hc->hcchar;
    p[2] = hc->hcint;
    p[3] = hc->hctsiz;
    p[4] = hc->hcdma;
    nchanlog++;
}
```

⟨*function* logdump*(arm)* 699b⟩≡ (711b)

```
static void
logdump(Ep *ep)
{
    Reg *p;
    int i;

    if(!ep->debug)
        return;
    p = chanlog[0];
    for(i = 0; i < nchanlog; i++){
        print("%5.5d.%5.5d %8.8ux %8.8ux %8.8ux %8.8ux\n",
            p[0]&0xFFFF, p[0]>>16, p[1], p[2], p[3], p[4]);
        p += 5;
    }
    nchanlog = 0;
}
```

⟨*function* chanio*(arm)* 699c⟩≡ (711b)

```
static int
chanio(Ep *ep, Hostchan *hc, int dir, int pid, void *a, int len)
{
    Ctlr *ctlr;
    int nleft, n, nt, i, maxpkt, npkt;
    uint hcdma, hctsiz;

    ctlr = ep->hp->aux;
    maxpkt = ep->maxpkt;
    npkt = HOWMANY(len, ep->maxpkt);
    if(npkt == 0)
        npkt = 1;

    hc->hcchar = (hc->hcchar & ~Epdir) | dir;
    if(dir == Epin)
        n = ROUND(len, ep->maxpkt);
    else
        n = len;
    hc->hctsiz = n | npkt<<OPktcnt | pid;
    hc->hcdma  = dmaaddr(a);

    nleft = len;
```

```
logstart(ep);
for(;;){
    hcdma = hc->hcdma;
    hctsiz = hc->hctsiz;
    hc->hctsiz = hctsiz & ~Dopng;
    if(hc->hcchar&Chen){
        dprint("ep%d.%d before chanio hcchar=%8.8ux\n",
            ep->dev->nb, ep->nb, hc->hcchar);
        hc->hcchar |= Chen | Chdis;
        while(hc->hcchar&Chen)
            ;
        hc->hcint = Chhltd;
    }
    if((i = hc->hcint) != 0){
        dprint("ep%d.%d before chanio hcint=%8.8ux\n",
            ep->dev->nb, ep->nb, i);
        hc->hcint = i;
    }
    if(hc->hcsplt & Spltena){
        qlock(&ctlr->split);
        sofwait(ctlr, hc - ctlr->regs->hchan);
        if((dwc.regs->hfnum & 1) == 0)
            hc->hcchar &= ~Oddfrm;
        else
            hc->hcchar |= Oddfrm;
    }
    hc->hcchar = (hc->hcchar &~ Chdis) | Chen;
    clog(ep, hc);
    if(ep->ttype == Tbulk && dir == Epin)
        i = chanwait(ep, ctlr, hc, /* Ack| */ Chhltd);
    else if(ep->ttype == Tintr && (hc->hcsplt & Spltena))
        i = chanwait(ep, ctlr, hc, Chhltd);
    else
        i = chanwait(ep, ctlr, hc, Chhltd|Nak);
    clog(ep, hc);
    hc->hcint = i;

    if(hc->hcsplt & Spltena){
        hc->hcsplt &= ~Compsplt;
        qunlock(&ctlr->split);
    }

    if((i & Xfercomp) == 0 && i != (Chhltd|Ack) && i != Chhltd){
        if(i & Stall)
            error(Estalled);
        if(i & (Nyet|Frmovrun))
            continue;
        if(i & Nak){
            if(ep->ttype == Tintr)
                tsleep(&up->sleepr, returnfalse, 0, ep->pollival);
            else
                tsleep(&up->sleepr, returnfalse, 0, 1);
            continue;
        }
        logdump(ep);
        print("usbotg: ep%d.%d error intr %8.8ux\n",
            ep->dev->nb, ep->nb, i);
        if(i & ~(Chhltd|Ack))
            error(Eio);
        if(hc->hcdma != hcdma)
```

```
                print("usbotg: weird hcdma %x->%x intr %x->%x\n",
                    hcdma, hc->hcdma, i, hc->hcint);
        }
        n = hc->hcdma - hcdma;
        if(n == 0){
            if((hc->hctsiz & Pktcnt) != (hctsiz & Pktcnt))
                break;
            else
                continue;
        }
        if(dir == Epin && ep->ttype == Tbulk && n == nleft){
            nt = (hctsiz & Xfersize) - (hc->hctsiz & Xfersize);
            if(nt != n){
                if(n == ROUND(nt, 4))
                    n = nt;
                else
                    print("usbotg: intr %8.8ux "
                        "dma %8.8ux-%8.8ux "
                        "hctsiz %8.8ux-%8.ux\n",
                        i, hcdma, hc->hcdma, hctsiz,
                        hc->hctsiz);
            }
        }
        if(n > nleft){
            if(n != ROUND(nleft, 4))
                dprint("too much: wanted %d got %d\n",
                    len, len - nleft + n);
            n = nleft;
        }
        nleft -= n;
        if(nleft == 0 || (n % maxpkt) != 0)
            break;
        if((i & Xfercomp) && ep->ttype != Tctl)
            break;
        if(dir == Epout)
            dprint("too little: nleft %d hcdma %x->%x hctsiz %x->%x intr %x\n",
                nleft, hcdma, hc->hcdma, hctsiz, hc->hctsiz, i);
    }
    logdump(ep);
    return len - nleft;
}
```

⟨*function* multitrans*(arm)* 701⟩≡ (711b)

```
static long
multitrans(Ep *ep, Hostchan *hc, int rw, void *a, long n)
{
    long sofar, m;

    sofar = 0;
    do{
        m = n - sofar;
        if(m > ep->maxpkt)
            m = ep->maxpkt;
        m = chanio(ep, hc, rw == Read? Epin : Epout, ep->toggle[rw],
            (char*)a + sofar, m);
        ep->toggle[rw] = hc->hctsiz & Pid;
        sofar += m;
    }while(sofar < n && m == ep->maxpkt);
    return sofar;
}
```

⟨*function* eptrans*(arm)* 702a⟩≡ (711b)

```
static long
eptrans(Ep *ep, int rw, void *a, long n)
{
    Hostchan *hc;

    if(ep->clrhalt){
        ep->clrhalt = 0;
        if(ep->mode != OREAD)
            ep->toggle[Write] = DATA0;
        if(ep->mode != OWRITE)
            ep->toggle[Read] = DATA0;
    }
    hc = chanalloc(ep);
    if(waserror()){
        ep->toggle[rw] = hc->hctsiz & Pid;
        chanrelease(ep, hc);
        if(strcmp(up->errstr, Estalled) == 0)
            return 0;
        nexterror();
    }
    chansetup(hc, ep);
    if(rw == Read && ep->ttype == Tbulk)
        n = multitrans(ep, hc, rw, a, n);
    else{
        n = chanio(ep, hc, rw == Read? Epin : Epout, ep->toggle[rw],
            a, n);
        ep->toggle[rw] = hc->hctsiz & Pid;
    }
    chanrelease(ep, hc);
    poperror();
    return n;
}
```

⟨*function* ctltrans*(arm)* 702b⟩≡ (711b)

```
static long
ctltrans(Ep *ep, uchar *req, long n)
{
    Hostchan *hc;
    Epio *epio;
    Block *b;
    uchar *data;
    int datalen;

    epio = ep->aux;
    if(epio->cb != nil){
        freeb(epio->cb);
        epio->cb = nil;
    }
    if(n < Rsetuplen)
        error(Ebadlen);
    if(req[Rtype] & Rd2h){
        datalen = GET2(req+Rcount);
        if(datalen <= 0 || datalen > Maxctllen)
            error(Ebadlen);
        /* XXX cache madness */
        epio->cb = b = allocb(ROUND(datalen, ep->maxpkt));
        assert(((uintptr)b->wp & (BLOCKALIGN-1)) == 0);
        memset(b->wp, 0x55, b->lim - b->wp);
        cachedwbinvse(b->wp, b->lim - b->wp);
```

```
        data = b->wp;
    }else{
        b = nil;
        datalen = n - Rsetuplen;
        data = req + Rsetuplen;
    }
    hc = chanalloc(ep);
    if(waserror()){
        chanrelease(ep, hc);
        if(strcmp(up->errstr, Estalled) == 0)
            return 0;
        nexterror();
    }
    chansetup(hc, ep);
    chanio(ep, hc, Epout, SETUP, req, Rsetuplen);
    if(req[Rtype] & Rd2h){
        if(ep->dev->hub <= 1){
            ep->toggle[Read] = DATA1;
            b->wp += multitrans(ep, hc, Read, data, datalen);
        }else
            b->wp += chanio(ep, hc, Epin, DATA1, data, datalen);
        chanio(ep, hc, Epout, DATA1, nil, 0);
        n = Rsetuplen;
    }else{
        if(datalen > 0)
            chanio(ep, hc, Epout, DATA1, data, datalen);
        chanio(ep, hc, Epin, DATA1, nil, 0);
        n = Rsetuplen + datalen;
    }
    chanrelease(ep, hc);
    poperror();
    return n;
}
```

⟨*function* ctldata*(arm)* 703a⟩≡ (711b)

```
static long
ctldata(Ep *ep, void *a, long n)
{
    Epio *epio;
    Block *b;

    epio = ep->aux;
    b = epio->cb;
    if(b == nil)
        return 0;
    if(n > BLEN(b))
        n = BLEN(b);
    memmove(a, b->rp, n);
    b->rp += n;
    if(BLEN(b) == 0){
        freeb(b);
        epio->cb = nil;
    }
    return n;
}
```

⟨*function* greset*(arm)* 703b⟩≡ (711b)

```
static void
greset(Dwcregs *r, int bits)
{
```

```
    r->grstctl |= bits;
    while(r->grstctl & bits)
        ;
    arch_microdelay(10);
}
```

⟨*function* `init`*(arm)* 704a⟩≡ (711b)

```
static void
init(Hci *hp)
{
    Ctlr *ctlr;
    Dwcregs *r;
    uint n, rx, tx, ptx;

    ctlr = hp->aux;
    r = ctlr->regs;

    ctlr->nchan = 1 + ((r->ghwcfg2 & Num_host_chan) >> ONum_host_chan);
    ctlr->chanintr = malloc(ctlr->nchan * sizeof(Rendez));

    r->gahbcfg = 0;
    setpower(PowerUsb, 1);

    while((r->grstctl&Ahbidle) == 0)
        ;
    greset(r, Csftrst);

    r->gusbcfg |= Force_host_mode;
    tsleep(&up->sleepr, returnfalse, 0, 25);
    r->gahbcfg |= Dmaenable;

    n = (r->ghwcfg3 & Dfifo_depth) >> ODfifo_depth;
    rx = 0x306;
    tx = 0x100;
    ptx = 0x200;
    r->grxfsiz = rx;
    r->gnptxfsiz = rx | tx<<ODepth;
    tsleep(&up->sleepr, returnfalse, 0, 1);
    r->hptxfsiz = (rx + tx) | ptx << ODepth;
    greset(r, Rxfflsh);
    r->grstctl = TXF_ALL;
    greset(r, Txfflsh);
    dprint("usbotg: FIFO depth %d sizes rx/nptx/ptx %8.8ux %8.8ux %8.8ux\n",
        n, r->grxfsiz, r->gnptxfsiz, r->hptxfsiz);

    r->hport0 = Prtpwr|Prtconndet|Prtenchng|Prtovrcurrchng;
    r->gintsts = ~0;
    r->gintmsk = Hcintr;
    r->gahbcfg |= Glblintrmsk;
}
```

⟨*function* `dump`*(*`(buses/arm/usbdwc.c)(arm)`*)* 704b⟩≡ (711b)

```
static void
dump(Hci*)
{
}
```

⟨*function* `fiqintr`*(arm)* 704c⟩≡ (711b)

```
static void
fiqintr(Ureg*, void *a)
```

```
{
    Hci *hp;
    Ctlr *ctlr;
    Dwcregs *r;
    uint intr, haint, wakechan;
    int i;

    hp = a;
    ctlr = hp->aux;
    r = ctlr->regs;
    wakechan = 0;
    filock(ctlr);
    intr = r->gintsts;
    if(intr & Hcintr){
        haint = r->haint & r->haintmsk;
        for(i = 0; haint; i++){
            if(haint & 1){
                if(chanintr(ctlr, i) == 0){
                    r->haintmsk &= ~(1<<i);
                    wakechan |= 1<<i;
                }
            }
            haint >>= 1;
        }
    }
    if(intr & Sofintr){
        r->gintsts = Sofintr;
        if((r->hfnum&7) != 6){
            r->gintmsk &= ~Sofintr;
            wakechan |= ctlr->sofchan;
            ctlr->sofchan = 0;
        }
    }
    if(wakechan){
        ctlr->wakechan |= wakechan;
        armtimerset(1);
    }
    fiunlock(ctlr);
}
```

⟨*function* armtimerset*(arm)* 705a⟩≡ (639b)

```
void
armtimerset(int n)
{
    Armtimer *tm;

    tm = (Armtimer*)ARMTIMER;
    if(n > 0){
        tm->ctl |= TmrEnable|TmrIntEnable;
        tm->load = n;
    }else{
        tm->load = 0;
        tm->ctl &= ~(TmrEnable|TmrIntEnable);
        tm->irq = 1;
    }
}
```

⟨*function* irqintr*(arm)* 705b⟩≡ (711b)

```
static void
irqintr(Ureg*, void *a)
```

```
{
    Ctlr *ctlr;
    uint wakechan;
    int i;

    ctlr = a;
    filock(ctlr);
    armtimerset(0);
    wakechan = ctlr->wakechan;
    ctlr->wakechan = 0;
    fiunlock(ctlr);
    for(i = 0; wakechan; i++){
        if(wakechan & 1)
            wakeup(&ctlr->chanintr[i]);
        wakechan >>= 1;
    }
}
```

⟨*function* epopen*(arm)* 706a⟩≡ (711b)

```
static void
epopen(Ep *ep)
{
    ddprint("usbotg: epopen ep%d.%d ttype %d\n",
        ep->dev->nb, ep->nb, ep->ttype);
    switch(ep->ttype){
    case Tnone:
        error(Enotconf);
    case Tintr:
        assert(ep->pollival > 0);
        /* fall through */
    case Tbulk:
        if(ep->toggle[Read] == 0)
            ep->toggle[Read] = DATA0;
        if(ep->toggle[Write] == 0)
            ep->toggle[Write] = DATA0;
        break;
    }
    ep->aux = malloc(sizeof(Epio));
    if(ep->aux == nil)
        error(Enomem);
}
```

⟨*function* epclose*(*(buses/arm/usbdwc.c)(arm)*)* 706b⟩≡ (711b)

```
static void
epclose(Ep *ep)
{
    ddprint("usbotg: epclose ep%d.%d ttype %d\n",
        ep->dev->nb, ep->nb, ep->ttype);
    switch(ep->ttype){
    case Tctl:
        freeb(((Epio*)ep->aux)->cb);
        /* fall through */
    default:
        free(ep->aux);
        break;
    }
}
```

⟨*function* epread*(arm)* 706c⟩≡ (711b)

```
static long
```

```
epread(Ep *ep, void *a, long n)
{
    Epio *epio;
    Block *b;
    uchar *p;
    ulong elapsed;
    long nr;

    ddprint("epread ep%d.%d %ld\n", ep->dev->nb, ep->nb, n);
    epio = ep->aux;
    b = nil;
    qlock(epio);
    if(waserror()){
        qunlock(epio);
        if(b)
            freeb(b);
        nexterror();
    }
    switch(ep->ttype){
    default:
        error(Egreg);
    case Tctl:
        nr = ctldata(ep, a, n);
        qunlock(epio);
        poperror();
        return nr;
    case Tintr:
        elapsed = TK2MS(cpu->ticks) - epio->lastpoll;
        if(elapsed < ep->pollival)
            tsleep(&up->sleepr, returnfalse, 0, ep->pollival - elapsed);
        /* fall through */
    case Tbulk:
        /* XXX cache madness */
        b = allocb(ROUND(n, ep->maxpkt));
        p = b->rp;
        assert(((uintptr)p & (BLOCKALIGN-1)) == 0);
        cachedinvse(p, n);
        nr = eptrans(ep, Read, p, n);
        cachedinvse(p, nr);
        epio->lastpoll = TK2MS(cpu->ticks);
        memmove(a, p, nr);
        qunlock(epio);
        freeb(b);
        poperror();
        return nr;
    }
}
```

⟨*function* epwrite*(arm)* 707⟩≡ (711b)

```
static long
epwrite(Ep *ep, void *a, long n)
{
    Epio *epio;
    Block *b;
    uchar *p;
    ulong elapsed;

    ddprint("epwrite ep%d.%d %ld\n", ep->dev->nb, ep->nb, n);
    epio = ep->aux;
    b = nil;
```

```
    qlock(epio);
    if(waserror()){
        qunlock(epio);
        if(b)
            freeb(b);
        nexterror();
    }
    switch(ep->ttype){
    default:
        error(Egreg);
    case Tintr:
        elapsed = TK2MS(cpu->ticks) - epio->lastpoll;
        if(elapsed < ep->pollival)
            tsleep(&up->sleepr, returnfalse, 0, ep->pollival - elapsed);
        /* fall through */
    case Tctl:
    case Tbulk:
        /* XXX cache madness */
        b = allocb(n);
        p = b->wp;
        assert(((uintptr)p & (BLOCKALIGN-1)) == 0);
        memmove(p, a, n);
        cachedwbse(p, n);
        if(ep->ttype == Tctl)
            n = ctltrans(ep, p, n);
        else{
            n = eptrans(ep, Write, p, n);
            epio->lastpoll = TK2MS(cpu->ticks);
        }
        qunlock(epio);
        freeb(b);
        poperror();
        return n;
    }
}
```

⟨*function* seprintep*(*(buses/arm/usbdwc.c)(arm)*)* 708a⟩≡ (711b)

```
static char*
seprintep(char *s, char*, Ep*)
{
    return s;
}
```

⟨*function* portenable*(arm)* 708b⟩≡ (711b)

```
static int
portenable(Hci *hp, int port, int on)
{
    Ctlr *ctlr;
    Dwcregs *r;

    assert(port == 1);
    ctlr = hp->aux;
    r = ctlr->regs;
    dprint("usbotg enable=%d; sts %#x\n", on, r->hport0);
    if(!on)
        r->hport0 = Prtpwr | Prtena;
    tsleep(&up->sleepr, returnfalse, 0, Enabledelay);
    dprint("usbotg enable=%d; sts %#x\n", on, r->hport0);
    return 0;
}
```

⟨*function* portreset*(arm)* 709a⟩≡ (711b)

```
static int
portreset(Hci *hp, int port, int on)
{
    Ctlr *ctlr;
    Dwcregs *r;
    int b, s;

    assert(port == 1);
    ctlr = hp->aux;
    r = ctlr->regs;
    dprint("usbotg reset=%d; sts %#x\n", on, r->hport0);
    if(!on)
        return 0;
    r->hport0 = Prtpwr | Prtrst;
    tsleep(&up->sleepr, returnfalse, 0, ResetdelayHS);
    r->hport0 = Prtpwr;
    tsleep(&up->sleepr, returnfalse, 0, Enabledelay);
    s = r->hport0;
    b = s & (Prtconndet|Prtenchng|Prtovrcurrchng);
    if(b != 0)
        r->hport0 = Prtpwr | b;
    dprint("usbotg reset=%d; sts %#x\n", on, s);
    if((s & Prtena) == 0)
        print("usbotg: host port not enabled after reset");
    return 0;
}
```

⟨*function* portstatus*(arm)* 709b⟩≡ (711b)

```
static int
portstatus(Hci *hp, int port)
{
    Ctlr *ctlr;
    Dwcregs *r;
    int b, s;

    assert(port == 1);
    ctlr = hp->aux;
    r = ctlr->regs;
    s = r->hport0;
    b = s & (Prtconndet|Prtenchng|Prtovrcurrchng);
    if(b != 0)
        r->hport0 = Prtpwr | b;
    b = 0;
    if(s & Prtconnsts)
        b |= HPpresent;
    if(s & Prtconndet)
        b |= HPstatuschg;
    if(s & Prtena)
        b |= HPenable;
    if(s & Prtenchng)
        b |= HPchange;
    if(s & Prtovrcurract)
         b |= HPovercurrent;
    if(s & Prtsusp)
        b |= HPsuspend;
    if(s & Prtrst)
        b |= HPreset;
    if(s & Prtpwr)
        b |= HPpower;
```

```
    switch(s & Prtspd){
    case HIGHSPEED:
        b |= HPhigh;
        break;
    case LOWSPEED:
        b |= HPslow;
        break;
    }
    return b;
}
```

⟨*function* shutdown*(arm)* 710a⟩≡ (711b)

```
static void
shutdown(Hci*)
{
}
```

⟨*function* setdebug*(arm)* 710b⟩≡ (711b)

```
static void
setdebug(Hci*, int d)
{
    debug = d;
}
```

⟨*function* reset*(arm)* 710c⟩≡ (711b)

```
static int
reset(Hci *hp)
{
    Ctlr *ctlr;
    uint id;

    ctlr = &dwc;
    if(ctlr->regs != nil)
        return -1;
    ctlr->regs = (Dwcregs*)USBREGS;
    id = ctlr->regs->gsnpsid;
    if((id>>16) != ('O'<<8 | 'T'))
        return -1;
    dprint("usbotg: rev %d.%3.3x\n", (id>>12)&0xF, id&0xFFF);

    arch_intrenable(IRQtimerArm, irqintr, ctlr, 0, "dwc");

    hp->aux = ctlr;
    hp->port = 0;
    hp->irq = IRQusb;
    hp->tbdf = 0;
    hp->nports = 1;
    hp->highspeed = 1;

    hp->init = init;
    hp->dump = dump;
    hp->interrupt = fiqintr;
    hp->epopen = epopen;
    hp->epclose = epclose;
    hp->epread = epread;
    hp->epwrite = epwrite;
    hp->seprintep = seprintep;
    hp->portenable = portenable;
    hp->portreset = portreset;
    hp->portstatus = portstatus;
```

```
    hp->shutdown = shutdown;
    hp->debug = setdebug;
    hp->type = "dwcotg";
    return 0;
}
```

⟨*function* usbdwclink*(arm)* 711a⟩≡ (711b)

```
void
usbdwclink(void)
{
    addhcitype("dwcotg", reset);
}
```

⟨buses/arm/usbdwc.c 711b⟩≡

```
/*
 * USB host driver for BCM2835
 *  Synopsis DesignWare Core USB 2.0 OTG controller
 *
 * Copyright © 2012 Richard Miller <r.miller@acm.org>
 *
 * This is work in progress:
 * - no isochronous pipes
 * - no bandwidth budgeting
 * - frame scheduling is crude
 * - error handling is overly optimistic
 * It should be just about adequate for a Plan 9 terminal with
 * keyboard, mouse, ethernet adapter, and an external flash drive.
 */
```

⟨*kernel basic includes* 23m⟩

```
#include    "io.h"
#include    "../port/usb.h"

#include "dwcotg.h"
```

⟨*constant* USBREGS*(arm)* 23b⟩
⟨*enum* _anon_*(*(buses/arm/usbdwc.c)(arm)*)* 694a⟩

```
typedef struct Ctlr Ctlr;
typedef struct Epio Epio;
```

⟨*struct* Ctlr*(*(buses/arm/usbdwc.c)(arm)*)* 694b⟩

⟨*struct* Epio*(arm)* 694c⟩

⟨*global* dwc*(arm)* 694d⟩
⟨*global* debug*(*(buses/arm/usbdwc.c)(arm)*)* 694e⟩

⟨*global* Ebadlen*(arm)* 694f⟩

```
static void clog(Ep *ep, Hostchan *hc);
static void logdump(Ep *ep);
```

⟨*function* filock*(arm)* 694g⟩

⟨*function* fiunlock*(arm)* 694h⟩

⟨*function* chanalloc*(arm)* 695a⟩

⟨*function* chanrelease*(arm)* 695b⟩

⟨*function* `chansetup`*(arm)* 695c⟩

⟨*function* `sofdone`*(arm)* 696a⟩

⟨*function* `sofwait`*(arm)* 696b⟩

⟨*function* `chandone`*(arm)* 696c⟩

⟨*function* `chanwait`*(arm)* 697⟩

⟨*function* `chanintr`*(arm)* 698a⟩

⟨*global* `chanlog`*(arm)* 698b⟩
⟨*global* `nchanlog`*(arm)* 698c⟩

⟨*function* `logstart`*(arm)* 698d⟩

⟨*function* `clog`*(arm)* 699a⟩

⟨*function* `logdump`*(arm)* 699b⟩

⟨*function* `chanio`*(arm)* 699c⟩

⟨*function* `multitrans`*(arm)* 701⟩

⟨*function* `eptrans`*(arm)* 702a⟩

⟨*function* `ctltrans`*(arm)* 702b⟩

⟨*function* `ctldata`*(arm)* 703a⟩

⟨*function* `greset`*(arm)* 703b⟩

⟨*function* `init`*(arm)* 704a⟩

⟨*function* `dump`*(*`(buses/arm/usbdwc.c)(arm)`*)* 704b⟩

⟨*function* `fiqintr`*(arm)* 704c⟩

⟨*function* `irqintr`*(arm)* 705b⟩

⟨*function* `epopen`*(arm)* 706a⟩

⟨*function* `epclose`*(*`(buses/arm/usbdwc.c)(arm)`*)* 706b⟩

⟨*function* `epread`*(arm)* 706c⟩

⟨*function* `epwrite`*(arm)* 707⟩

⟨*function* `seprintep`*(*`(buses/arm/usbdwc.c)(arm)`*)* 708a⟩

⟨*function* `portenable`*(arm)* 708b⟩

⟨*function* `portreset`*(arm)* 709a⟩

⟨*function* `portstatus`*(arm)* 709b⟩

⟨*function* `shutdown`*(arm)* 710a⟩

⟨*function* setdebug*(arm)* 710b⟩

⟨*function* reset*(arm)* 710c⟩

⟨*function* usbdwclink*(arm)* 711a⟩

E.29.7 buses/arm/dma.c

⟨*macro* IRQDMA*(arm)* 713a⟩≡ (609a)

```
#define IRQDMA(chan)    (IRQdma0+(chan))
```

⟨*enum* Dma*(arm)* 713b⟩≡ (609a)

```
enum DmaFlags {
    DmaD2M      = 0,        /* device to memory */
    DmaM2D      = 1,        /* memory to device */
    DmaM2M      = 2,        /* memory to memory */

    DmaChanEmmc = 4,        /* can only use 2-5, maybe 0 */
    DmaChanSpiTx= 2,
    DmaChanSpiRx= 0,

    DmaDevSpiTx = 6,
    DmaDevSpiRx = 7,
    DmaDevEmmc  = 11,
};
```

⟨*enum* _anon_*(*(buses/arm/dma.c)(arm)*)* 713c⟩≡ (717b)

```
enum {
    Nchan       = 7,        /* number of dma channels */
    Regsize     = 0x100,    /* size of regs for each chan */
    Cbalign     = 32,       /* control block byte alignment */
    Dbg     = 0,

    /* registers for each dma controller */
    Cs      = 0x00>>2,
    Conblkad    = 0x04>>2,
    Ti      = 0x08>>2,
    Sourcead    = 0x0c>>2,
    Destad      = 0x10>>2,
    Txfrlen     = 0x14>>2,
    Stride      = 0x18>>2,
    Nextconbk   = 0x1c>>2,
    Debug       = 0x20>>2,

    /* collective registers */
    Intstatus   = 0xfe0>>2,
    Enable      = 0xff0>>2,

    /* Cs */
    Reset       = 1<<31,
    Abort       = 1<<30,
    Error       = 1<<8,
    Waitwrite   = 1<<6,
    Waitdreq    = 1<<5,
    Paused      = 1<<4,
    Dreq        = 1<<3,
    Int     = 1<<2,
    End     = 1<<1,
    Active      = 1<<0,
```

```
    /* Ti */
    Permapshift= 16,
    Srcignore   = 1<<11,
    Srcdreq     = 1<<10,
    Srcwidth128 = 1<<9,
    Srcinc      = 1<<8,
    Destignore  = 1<<7,
    Destdreq    = 1<<6,
    Destwidth128    = 1<<5,
    Destinc     = 1<<4,
    Waitresp    = 1<<3,
    Tdmode      = 1<<1,
    Inten       = 1<<0,

    /* Debug */
    Lite        = 1<<28,
    Clrerrors   = 7<<0,
};
```

⟨*struct* Ctlr*(arm)* 714a⟩≡ (717b)

```
struct Ctlr {
    u32int  *regs;
    Cb  *cb;
    Rendez  r;
    int dmadone;
};
```

⟨*struct* Cb*(arm)* 714b⟩≡ (717b)

```
struct Cb {
    u32int  ti;
    u32int  sourcead;
    u32int  destad;
    u32int  txfrlen;
    u32int  stride;
    u32int  nextconbk;
    u32int  reserved[2];
};
```

⟨*global* dma*(arm)* 714c⟩≡ (717b)

```
static Ctlr dma[Nchan];
```

⟨*global* dmaregs*(arm)* 714d⟩≡ (717b)

```
static u32int *dmaregs = (u32int*)DMAREGS;
```

⟨Soc *bus fields* 714e⟩≡ (33a) 715a ▷

```
uintptr busdram;
```

⟨soc(raspberry pi1) *other fields(arm)* 714f⟩+≡ (33b) ◁ 407a 715b ▷

```
.busdram    = 0x40000000,
```

⟨soc(raspberry pi2) *other fields(arm)* 714g⟩+≡ (405g) ◁ 407b 715c ▷

```
.busdram    = 0xC0000000,
```

⟨*function* dmaaddr*(arm)* 714h⟩≡ (717b)

```
uintptr
dmaaddr(void *va)
{
    return soc.busdram | (PTR2UINT(va) & ~KSEGM); // PADDR?
}
```

⟨Soc *bus fields* 715a⟩+≡ (33a) ◁714e

```
uintptr busio;
```

⟨soc(raspberry pi1) *other fields(arm)* 715b⟩+≡ (33b) ◁714f

```
.busio      = 0x7E000000,
```

⟨soc(raspberry pi2) *other fields(arm)* 715c⟩+≡ (405g) ◁714g

```
.busio      = 0x7E000000,
```

⟨*function* dmaioaddr*(arm)* 715d⟩≡ (717b)

```
static uintptr
dmaioaddr(void *va)
{
    return soc.busio | (PTR2UINT(va) & ~VIRTIO);
}
```

⟨*function* dump*(arm)* 715e⟩≡ (717b)

```
static void
dump(char *msg, uchar *p, int n)
{
    print("%s", msg);
    while(n-- > 0)
        print(" %2.2x", *p++);
    print("\n");
}
```

⟨*function* dumpdregs*(arm)* 715f⟩≡ (717b)

```
static void
dumpdregs(char *msg, u32int *r)
{
    int i;

    print("%s: %#p =", msg, r);
    for(i = 0; i < 9; i++)
        print(" %8.8uX", r[i]);
    print("\n");
}
```

⟨*function* dmadone*(arm)* 715g⟩≡ (717b)

```
static int
dmadone(void *a)
{
    return ((Ctlr*)a)->dmadone;
}
```

⟨*function* dmainterrupt*(arm)* 715h⟩≡ (717b)

```
static void
dmainterrupt(Ureg*, void *a)
{
    Ctlr *ctlr;

    ctlr = a;
    ctlr->regs[Cs] = Int;
    ctlr->dmadone = 1;
    wakeup(&ctlr->r);
}
```

⟨*function* dmastart*(arm)* 716⟩≡ (717b)

```
void
dmastart(int chan, int dev, int dir, void *src, void *dst, int len)
{
    Ctlr *ctlr;
    Cb *cb;
    int ti;

    ctlr = &dma[chan];
    if(ctlr->regs == nil){
        ctlr->regs = (u32int*)(DMAREGS + chan*Regsize);
        ctlr->cb = xspanalloc(sizeof(Cb), Cbalign, 0);
        assert(ctlr->cb != nil);
        dmaregs[Enable] |= 1<<chan;
        ctlr->regs[Cs] = Reset;
        while(ctlr->regs[Cs] & Reset)
            ;
        arch_intrenable(IRQDMA(chan), dmainterrupt, ctlr, 0, "dma");
    }
    cb = ctlr->cb;
    ti = 0;
    switch(dir){
    case DmaD2M:
        cachedinvse(dst, len);
        ti = Srcdreq | Destinc;
        cb->sourcead = dmaioaddr(src);
        cb->destad = dmaaddr(dst);
        break;
    case DmaM2D:
        cachedwbse(src, len);
        ti = Destdreq | Srcinc;
        cb->sourcead = dmaaddr(src);
        cb->destad = dmaioaddr(dst);
        break;
    case DmaM2M:
        cachedwbse(src, len);
        cachedinvse(dst, len);
        ti = Srcinc | Destinc;
        cb->sourcead = dmaaddr(src);
        cb->destad = dmaaddr(dst);
        break;
    }
    cb->ti = ti | dev<<Permapshift | Inten;
    cb->txfrlen = len;
    cb->stride = 0;
    cb->nextconbk = 0;
    cachedwbse(cb, sizeof(Cb));
    ctlr->regs[Cs] = 0;
    arch_microdelay(1);
    ctlr->regs[Conblkad] = dmaaddr(cb);
    DBG print("dma start: %ux %ux %ux %ux %ux %ux\n",
        cb->ti, cb->sourcead, cb->destad, cb->txfrlen,
        cb->stride, cb->nextconbk);
    DBG print("intstatus %ux\n", dmaregs[Intstatus]);
    dmaregs[Intstatus] = 0;
    ctlr->regs[Cs] = Int;
    arch_microdelay(1);
    arch_coherence();
    DBG dumpdregs("before Active", ctlr->regs);
    ctlr->regs[Cs] = Active;
```

```
    DBG dumpdregs("after Active", ctlr->regs);
}
```

⟨*function* dmawait*(arm)* 717a⟩≡ (717b)

```
int
dmawait(int chan)
{
    Ctlr *ctlr;
    u32int *r;
    int s;

    ctlr = &dma[chan];
    tsleep(&ctlr->r, dmadone, ctlr, 3000);
    ctlr->dmadone = 0;
    r = ctlr->regs;
    DBG dumpdregs("after sleep", r);
    s = r[Cs];
    if((s & (Active|End|Error)) != End){
        print("dma chan %d %s Cs %ux Debug %ux\n", chan,
            (s&End)? "error" : "timeout", s, r[Debug]);
        r[Cs] = Reset;
        r[Debug] = Clrerrors;
        return -1;
    }
    r[Cs] = Int|End;
    return 0;
}
```

⟨buses/arm/dma.c 717b⟩≡

```
/*
 * bcm2835 dma controller
 *
 * simplest to use only channels 0-6
 *  channels 7-14 have reduced functionality
 *  channel 15 is at a weird address
 *  channels 0 and 15 have an "external 128 bit 8 word read FIFO"
 *    for memory to memory transfers
 *
 * Experiments show that only channels 2-5,11-12 work with mmc
 */
```

⟨*kernel basic includes* 23m⟩

```
#include "io.h"
```

⟨*constant* DMAREGS*(arm)* 22h⟩

```
#define DBG if(Dbg)
```

⟨*enum* _anon_*(*(buses/arm/dma.c)(arm)*)* 713c⟩

```
typedef struct Ctlr Ctlr;
typedef struct Cb Cb;
```

⟨*struct* Ctlr*(arm)* 714a⟩

⟨*struct* Cb*(arm)* 714b⟩

⟨*global* dma*(arm)* 714c⟩
⟨*global* dmaregs*(arm)* 714d⟩

⟨*function* dmaaddr*(arm)* 714h⟩

⟨*function* dmaioaddr*(arm)* 715d⟩

⟨*function* dump*(arm)* 715e⟩

⟨*function* dumpdregs*(arm)* 715f⟩

⟨*function* dmadone*(arm)* 715g⟩

⟨*function* dmainterrupt*(arm)* 715h⟩

⟨*function* dmastart*(arm)* 716⟩

⟨*function* dmawait*(arm)* 717a⟩

E.29.8 buses/arm/uartmini.c

⟨*constant* OkLed*(arm)* 718a⟩≡ (726)

```
#define OkLed       16
```

⟨*constant* TxPin*(arm)* 718b⟩≡ (726)

```
#define TxPin       14
```

⟨*constant* RxPin*(arm)* 718c⟩≡ (726)

```
#define RxPin       15
```

⟨*enum* _anon_*(*(buses/arm/uartmini.c)(arm)*)* 718d⟩≡ (726)

```
/* GPIO regs */
enum {
    Fsel0   = 0x00>>2,
        FuncMask= 0x7,
        Input   = 0x0,
        Output  = 0x1,
        Alt0    = 0x4,
        Alt1    = 0x5,
        Alt2    = 0x6,
        Alt3    = 0x7,
        Alt4    = 0x3,
        Alt5    = 0x2,
    Set0    = 0x1c>>2,
    Clr0    = 0x28>>2,
    Lev0    = 0x34>>2,
    PUD = 0x94>>2,
        Off = 0x0,
        Pulldown= 0x1,
        Pullup  = 0x2,
    PUDclk0 = 0x98>>2,
    PUDclk1 = 0x9c>>2,
};
```

⟨*enum* _anon_*(*(buses/arm/uartmini.c)2(arm)*)* 718e⟩≡ (726)

```
/* AUX regs */
enum {
    Irq = 0x00>>2,
        UartIrq = 1<<0,
    Enables = 0x04>>2,
        UartEn  = 1<<0,
    MuIo    = 0x40>>2,
```

```
    MuIer   = 0x44>>2,
        RxIen   = 1<<0,
        TxIen   = 1<<1,
    MuIir   = 0x48>>2,
    MuLcr   = 0x4c>>2,
        Bitsmask= 3<<0,
        Bits7   = 2<<0,
        Bits8   = 3<<0,
    MuMcr   = 0x50>>2,
        RtsN    = 1<<1,
    MuLsr   = 0x54>>2,
        TxDone  = 1<<6,
        TxRdy   = 1<<5,
        RxRdy   = 1<<0,
    MuCntl  = 0x60>>2,
        CtsFlow = 1<<3,
        TxEn    = 1<<1,
        RxEn    = 1<<0,
    MuBaud  = 0x68>>2,
};
```

⟨*global* `miniuart` *(arm)* 719a⟩≡ (726)

```
static Uart miniuart = {
    .regs   = (u32int*)AUXREGS,
    .name   = "uart0",
    .freq   = 250000000,
    .phys   = &miniphysuart,
};
```

⟨*function* `gpiosel` *(arm)* 719b⟩≡ (726)

```
void
gpiosel(uint pin, int func)
{
    u32int *gp, *fsel;
    int off;

    gp = (u32int*)GPIOREGS;
    fsel = &gp[Fsel0 + pin/10];
    off = (pin % 10) * 3;
    *fsel = (*fsel & ~(FuncMask<<off)) | func<<off;
}
```

⟨*function* `gpiopull` *(arm)* 719c⟩≡ (726)

```
static void
gpiopull(uint pin, int func)
{
    u32int *gp, *reg;
    u32int mask;

    gp = (u32int*)GPIOREGS;
    reg = &gp[PUDclk0 + pin/32];
    mask = 1 << (pin % 32);
    gp[PUD] = func;
    arch_microdelay(1);
    *reg = mask;
    arch_microdelay(1);
    *reg = 0;
}
```

⟨*function* gpiopulloff*(arm)* 720a⟩≡ (726)

```
void
gpiopulloff(uint pin)
{
    gpiopull(pin, Off);
}
```

⟨*function* gpiopullup*(arm)* 720b⟩≡ (726)

```
void
gpiopullup(uint pin)
{
    gpiopull(pin, Pullup);
}
```

⟨*function* gpiopulldown*(arm)* 720c⟩≡ (726)

```
void
gpiopulldown(uint pin)
{
    gpiopull(pin, Pulldown);
}
```

⟨*function* gpioout*(arm)* 720d⟩≡ (726)

```
void
gpioout(uint pin, int set)
{
    u32int *gp;
    int v;

    gp = (u32int*)GPIOREGS;
    v = set? Set0 : Clr0;
    gp[v + pin/32] = 1 << (pin % 32);
}
```

⟨*function* gpioin*(arm)* 720e⟩≡ (726)

```
int
gpioin(uint pin)
{
    u32int *gp;

    gp = (u32int*)GPIOREGS;
    return (gp[Lev0 + pin/32] & (1 << (pin % 32))) != 0;
}
```

⟨*function* interrupt*(arm)* 720f⟩≡ (726)

```
static void
interrupt(Ureg*, void *arg)
{
    Uart *uart;
    u32int *ap;

    uart = arg;
    ap = (u32int*)uart->regs;

    arch_coherence();
    if(0 && (ap[Irq] & UartIrq) == 0)
        return;
    if(ap[MuLsr] & TxRdy)
        uartkick(uart);
    if(ap[MuLsr] & RxRdy){
        if(uart->console){
```

```
            if(uart->opens == 1)
                uart->putc = kbdcr2nl;
            else
                uart->putc = nil;
        }
        do{
            uartrecv(uart, ap[MuIo] & 0xFF);
        }while(ap[MuLsr] & RxRdy);
    }
    arch_coherence();
}
```

⟨*function* pnp*(arm)* 721a⟩≡ (726)

```
static Uart*
pnp(void)
{
    Uart *uart;

    uart = &miniuart;
    if(uart->console == 0)
        kbdq = qopen(8*1024, 0, nil, nil);
    return uart;
}
```

⟨*function* enable*(arm)* 721b⟩≡ (726)

```
static void
enable(Uart *uart, int ie)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    arch_delay(10);
    gpiosel(TxPin, Alt5);
    gpiosel(RxPin, Alt5);
    gpiopulloff(TxPin);
    gpiopulloff(RxPin);
    ap[Enables] |= UartEn;
    ap[MuIir] = 6;
    ap[MuLcr] = Bits8;
    ap[MuCntl] = TxEn|RxEn;
    ap[MuBaud] = uart->freq/(115200*8) - 1;
    if(ie){
        arch_intrenable(IRQaux, interrupt, uart, 0, "uart");
        ap[MuIer] = RxIen|TxIen;
    }else
        ap[MuIer] = 0;
}
```

⟨*function* disable*(arm)* 721c⟩≡ (726)

```
static void
disable(Uart *uart)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    ap[MuCntl] = 0;
    ap[MuIer] = 0;
}
```

⟨*function* kick*(arm)* 722a⟩≡ (726)

```
static void
kick(Uart *uart)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    if(uart->blocked)
        return;
    arch_coherence();
    while(ap[MuLsr] & TxRdy){
        if(uart->op >= uart->oe && uartstageoutput(uart) == 0)
            break;
        ap[MuIo] = *(uart->op++);
    }
    if(ap[MuLsr] & TxDone)
        ap[MuIer] &= ~TxIen;
    else
        ap[MuIer] |= TxIen;
    arch_coherence();
}
```

⟨*function* dobreak*(arm)* 722b⟩≡ (726)

```
/* TODO */
static void
dobreak(Uart *uart, int ms)
{
    USED(uart, ms);
}
```

⟨*function* baud*(arm)* 722c⟩≡ (726)

```
static int
baud(Uart *uart, int n)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    if(uart->freq == 0 || n <= 0)
        return -1;
    ap[MuBaud] = (uart->freq + 4*n - 1) / (8 * n) - 1;
    uart->baud = n;
    return 0;
}
```

⟨*function* bits*(arm)* 722d⟩≡ (726)

```
static int
bits(Uart *uart, int n)
{
    u32int *ap;
    int set;

    ap = (u32int*)uart->regs;
    switch(n){
    case 7:
        set = Bits7;
        break;
    case 8:
        set = Bits8;
        break;
    default:
```

```
        return -1;
    }
    ap[MuLcr] = (ap[MuLcr] & ~Bitsmask) | set;
    uart->bits = n;
    return 0;
}
```

⟨*function* stop*(arm)* 723a⟩≡ (726)

```
static int
stop(Uart *uart, int n)
{
    if(n != 1)
        return -1;
    uart->stop = n;
    return 0;
}
```

⟨*function* parity*(arm)* 723b⟩≡ (726)

```
static int
parity(Uart *uart, int n)
{
    if(n != 'n')
        return -1;
    uart->parity = n;
    return 0;
}
```

⟨*function* modemctl*(arm)* 723c⟩≡ (726)

```
/*
 * cts/rts flow control
 *   need to bring signals to gpio pins before enabling this
 */

static void
modemctl(Uart *uart, int on)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    if(on)
        ap[MuCntl] |= CtsFlow;
    else
        ap[MuCntl] &= ~CtsFlow;
    uart->modem = on;
}
```

⟨*function* rts*(arm)* 723d⟩≡ (726)

```
static void
rts(Uart *uart, int on)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    if(on)
        ap[MuMcr] &= ~RtsN;
    else
        ap[MuMcr] |= RtsN;
}
```

⟨*function* status*(arm)* 724a⟩≡ (726)

```
static long
status(Uart *uart, void *buf, long n, long offset)
{
    char *p;

    p = malloc(READSTR);
    if(p == nil)
        error(Enomem);
    snprint(p, READSTR,
        "b%d\n"
        "dev(%d) type(%d) framing(%d) overruns(%d) "
        "berr(%d) serr(%d)\n",

        uart->baud,
        uart->dev,
        uart->type,
        uart->ferr,
        uart->oerr,
        uart->berr,
        uart->serr
    );
    n = readstr(offset, buf, n, p);
    free(p);

    return n;
}
```

⟨*function* donothing*(arm)* 724b⟩≡ (726)

```
static void
donothing(Uart*, int)
{
}
```

⟨*function* putc*(arm)* 724c⟩≡ (726)

```
void
putc(Uart*, int c)
{
    u32int *ap;

    ap = (u32int*)AUXREGS;
    while((ap[MuLsr] & TxRdy) == 0)
        ;
    ap[MuIo] = c;
    while((ap[MuLsr] & TxRdy) == 0)
        ;
}
```

⟨*function* getc*(arm)* 724d⟩≡ (726)

```
int
getc(Uart*)
{
    u32int *ap;

    ap = (u32int*)AUXREGS;
    while((ap[MuLsr] & RxRdy) == 0)
        ;
    return ap[MuIo] & 0xFF;
}
```

⟨*function* uartconsinit*(arm)* 725a⟩≡ (726)

```
void
uartconsinit(void)
{
    Uart *uart;
    int n;
    char *p, *cmd;

    if((p = getconf("console")) == nil)
        return;
    n = strtoul(p, &cmd, 0);
    if(p == cmd)
        return;
    switch(n){
    default:
        return;
    case 0:
        uart = &miniuart;
        break;
    }

    if(!uart->enabled)
        (*uart->phys->enable)(uart, 0);
    uartctl(uart, "b9600 l8 pn s1");
    if(*cmd != '\0')
        uartctl(uart, cmd);

    consuart = uart;
    uart->console = 1;
}
```

⟨*global* miniphysuart*(arm)* 725b⟩≡ (726)

```
PhysUart miniphysuart = {
    .name       = "miniuart",
    .pnp        = pnp,
    .enable     = enable,
    .disable    = disable,
    .kick       = kick,
    .dobreak    = dobreak,
    .baud       = baud,
    .bits       = bits,
    .stop       = stop,
    .parity     = parity,
    .modemctl   = donothing,
    .rts        = rts,
    .dtr        = donothing,
    .status     = status,
    .fifo       = donothing,
    .getc       = getc,
    .putc       = putc,
};
```

⟨*function* okay*(arm)* 725c⟩≡ (726)

```
void
okay(int on)
{
    static int first;
    static int okled, polarity;
    char *p;
```

```
    if(!first++){
        p = getconf("bcm2709.disk_led_gpio");
        if(p == nil)
            p = getconf("bcm2708.disk_led_gpio");
        if(p != nil)
            okled = strtol(p, 0, 0);
        else
            okled = OkLed;
        p = getconf("bcm2709.disk_led_active_low");
        if(p == nil)
            p = getconf("bcm2708.disk_led_active_low");
        polarity = (p == nil || *p == '1');
        gpiosel(okled, Output);
    }
    gpioout(okled, on^polarity);
}
```

⟨buses/arm/uartmini.c 726⟩≡

```
/*
 * bcm2835 mini uart (UART1)
 */
⟨kernel basic includes 23m⟩

#include "io.h"

⟨constant GPIOREGS(arm) 23d⟩
⟨constant AUXREGS(arm) 22j⟩
⟨constant OkLed(arm) 718a⟩
⟨constant TxPin(arm) 718b⟩
⟨constant RxPin(arm) 718c⟩

⟨enum _anon_((buses/arm/uartmini.c)(arm)) 718d⟩

⟨enum _anon_((buses/arm/uartmini.c)2(arm)) 718e⟩

extern PhysUart miniphysuart;

⟨global miniuart(arm) 719a⟩

⟨function gpiosel(arm) 719b⟩

⟨function gpiopull(arm) 719c⟩

⟨function gpiopulloff(arm) 720a⟩

⟨function gpiopullup(arm) 720b⟩

⟨function gpiopulldown(arm) 720c⟩

⟨function gpioout(arm) 720d⟩

⟨function gpioin(arm) 720e⟩

⟨function interrupt(arm) 720f⟩

⟨function pnp(arm) 721a⟩

⟨function enable(arm) 721b⟩

⟨function disable(arm) 721c⟩
```

⟨*function* `kick`*(arm)* 722a⟩

⟨*function* `dobreak`*(arm)* 722b⟩

⟨*function* `baud`*(arm)* 722c⟩

⟨*function* `bits`*(arm)* 722d⟩

⟨*function* `stop`*(arm)* 723a⟩

⟨*function* `parity`*(arm)* 723b⟩

⟨*function* `modemctl`*(arm)* 723c⟩

⟨*function* `rts`*(arm)* 723d⟩

⟨*function* `status`*(arm)* 724a⟩

⟨*function* `donothing`*(arm)* 724b⟩

⟨*function* `putc`*(arm)* 724c⟩

⟨*function* `getc`*(arm)* 724d⟩

⟨*function* `uartconsinit`*(arm)* 725a⟩

⟨*global* `miniphysuart`*(arm)* 725b⟩

⟨*function* `okay`*(arm)* 725c⟩

Glossary

```
UP   = User Process
PC   = Program Counter
SPL  = Set Priority Level
PD   = Page Directory
PT   = Page Table
PDB  = Page Directory Base
PDE  = Page Directory Entry
PTE  = Page Table Entry
PPN  = Physical Page Number?
EDF  = Earliest Deadline First
RPC  = Remote Procedure Call
EGRP = Environment GRouP
FGRP = File descriptor GRouP (channels)
PGRP = Process GRouP (namespace)
QID  = uniQue IDentifier (uid is taken for user id)
TOD  = Time Of Day
UART = Universal Asynchronous Receive/Transmitter

(ARM)
ARM = Acorn Risc Machines
CPSR = Current Program Status Register
SPSR = Saved Program Status Register
MCR = Move to Coprocessor from ARM Register
MRC = Move to ARM Register from Coprocessor
TTB = Translation Table Base
TLB = Translation Lookup Table
AP  = Access Permission
DAC = Domain Access Control
RFE = Return From Exception
IRQ  = Interrupt Request

(x86)
IRQ  = Interrupt Request
INTR = INTeRrupt (not INT, probably to avoid ambiguity with integer)
VNO  = Vector Number (interrupt)
ISR  = Interrupt Service Routine
NMI  = Non Maskable Interrupts (hardware error)
AP   = Application Processor
RMAP = RAM map
UMB  = Upper Memory Block
```

```
GDT  = Global Descriptor Table
VPT  = Virtual Page Table
APIC = Advanded Programmable Interrupt Controller (a next gen 8259)
ACPI = Advanced Configuration & Power Interface
TSC  = Time Stamp Counter
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. For an introduction see http://en.wikipedia.org/wiki/Literate_Program. cited page(s) 17

[KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 17

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 17